

blslambda

Black-Scholes elasticity

Syntax

```
[CallEl,PutEl] = blslambda(Price,Strike,Rate,Time,Volatility)
[CallEl,PutEl] = blslambda( ___,Yield)
```

Description

[CallEl,PutEl] = blslambda(Price,Strike,Rate,Time,Volatility) returns the elasticity of an option. CallEl is the call option elasticity or leverage factor, and PutEl is the put option elasticity or leverage factor. Elasticity (the leverage of an option position) measures the percent change in an option price per 1 percent change in the underlying asset price. blslambda uses normcdf, the normal cumulative distribution function in the Statistics and Machine Learning Toolbox.

In addition, you can use the Financial Instruments Toolbox object framework with the BlackScholes pricer object to obtain price and lambda values for a Vanilla, Barrier, Touch, DoubleTouch, or Binary instrument using a BlackScholes model.

Note blslambda can handle other types of underlies like Futures and Currencies. When pricing Futures (Black model), enter the input argument Yield as:

Yield = Rate

When pricing currencies (Garman-Kohlhagen model), enter the input argument Yield as:

Yield = ForeignRate

where ForeignRate is the continuously compounded, annualized risk-free interest rate in the foreign country.

[CallEl,PutEl] = blslambda(___,Yield) adds an optional argument for Yield.

Examples

Find the Black-Scholes Elasticity (Lambda) for an Option

This example shows how to find the Black-Scholes elasticity, or leverage, of an option position.

```
[CallEl, PutEl] = blslambda(50, 50, 0.12, 0.25, 0.3)
```

```
CallEl =  
8.1274
```

```
PutEl =  
-8.6466
```

Input Arguments

Price — Current price of underlying asset

numeric

Current price of the underlying asset, specified as a numeric value.

Data Types: double

Strike — Exercise price of option

numeric

Exercise price of the option, specified as a numeric value.

Data Types: double

Rate — Annualized, continuously compounded risk-free rate of return over life of option

positive decimal

Annualized, continuously compounded risk-free rate of return over the life of the option, specified as a positive decimal value.

Data Types: double

Time — Time (in years) to expiration of the option

numeric

Time (in years) to expiration of the option, specified as a numeric value.

Data Types: double

Volatility — Annualized asset price volatility

positive decimal

Annualized asset price volatility (annualized standard deviation of the continuously compounded asset return), specified as a positive decimal value.

Data Types: double

Yield — Annualized, continuously compounded yield of the underlying asset over life of option

0 (default) | decimal

(Optional) Annualized, continuously compounded yield of the underlying asset over the life of the option, specified as a decimal value. For example, for options written on stock indices, Yield could represent the dividend yield. For currency options, Yield could be the foreign risk-free interest rate.

Data Types: double

Output Arguments

CallEl — Call option elasticity

numeric

Call option elasticity or leverage factor, returned as a numeric value.

PutEL — Put option elasticity

numeric

Put option elasticity or leverage factor, returned as a numeric value.

More About**Lambda**

A lambda sensitivity measures the percentage change in an option's price for a 1% change in the price of the underlying asset.

Lambda is a measure of leverage, indicating how much more sensitive an option is to price movements in the underlying asset compared to owning the asset outright. For example, if an option has a lambda of 3, it means that if the underlying asset's price moves by 1%, the option's price is expected to move by 3%.

Version History

Introduced in R2006a

References

[1] Daigler, R. *Advanced Options Trading*. McGraw-Hill, 1993.

See Also

blsgamma | blsdelta | blsprice | blsrho | blstheta | blsvega

Topics

"Pricing and Analyzing Equity Derivatives" on page 2-39

"Greek-Neutral Portfolios of European Stock Options" on page 10-14

"Plotting Sensitivities of an Option" on page 10-25

"Plotting Sensitivities of a Portfolio of Options" on page 10-27

blsprice

Black-Scholes put and call option pricing

Syntax

```
[Call,Put] = blsprice(Price,Strike,Rate,Time,Volatility)
[Call,Put] = blsprice(____,Yield)
```

Description

[Call,Put] = blsprice(Price,Strike,Rate,Time,Volatility) computes European put and call option prices using a Black-Scholes model.

Note Any input argument can be a scalar, vector, or matrix. If a scalar, then that value is used to price all options. If more than one input is a vector or matrix, then the dimensions of those non-scalar inputs must be the same.

Ensure that Rate, Time, Volatility, and Yield are expressed in consistent units of time.

In addition, you can use the Financial Instruments Toolbox object framework with the BlackScholes pricer object to obtain price values for a Vanilla, Barrier, Touch, DoubleTouch, or Binary instrument using a BlackScholes model.

[Call,Put] = blsprice(____,Yield) adds an optional argument for Yield.

Examples

Compute European Put and Call Option Prices Using a Black-Scholes Model

This example shows how to price European stock options that expire in three months with an exercise price of \$95. Assume that the underlying stock pays no dividend, trades at \$100, and has a volatility of 50% per annum. The risk-free rate is 10% per annum.

```
[Call, Put] = blsprice(100, 95, 0.1, 0.25, 0.5)
```

```
Call =  
13.6953
```

```
Put =  
6.3497
```

Compute European Put and Call Option Prices on a Stock Index Using a Black-Scholes Model

The S&P 100 index is at 910 and has a volatility of 25% per annum. The risk-free rate of interest is 2% per annum and the index provides a dividend yield of 2.5% per annum. Calculate the value of a three-month European call and put with a strike price of 980.

```
[Call,Put] = blsprice(910,980,.02,.25,.25,.025)

Call =
19.6863

Put =
90.4683
```

Price a European Call Option with the Garman-Kohlhagen Model

Price an FX option on buying GBP with USD.

```
S = 1.6; % spot exchange rate
X = 1.6; % strike
T = .3333;
r_d = .08; % USD interest rate
r_f = .11; % GBP interest rate
sigma = .2;

Price = blsprice(S,X,r_d,T,sigma,r_f)

Price =
0.0639
```

Input Arguments

Price — Current price of underlying asset

numeric

Current price of the underlying asset, specified as a numeric value.

Data Types: double

Strike — Exercise price of the option

numeric

Exercise price of the option, specified as a numeric value.

Data Types: double

Rate — Annualized continuously compounded risk-free rate of return over life of the option

positive decimal

Annualized continuously compounded risk-free rate of return over the life of the option, specified as a positive decimal number.

Data Types: double

Time — Time to expiration of option

numeric

Time to expiration of the option, specified as the number of years.

Data Types: double

Volatility — Annualized asset price volatility

positive decimal

Annualized asset price volatility (that is, annualized standard deviation of the continuously compounded asset return), specified as a positive decimal number.

Data Types: double

Yield — Annualized continuously compounded yield of underlying asset over life of the option

0 (default) | decimal

(Optional) Annualized continuously compounded yield of the underlying asset over the life of the option, specified as a decimal number. If `Yield` is empty or missing, the default value is 0.

For example, `Yield` could represent the dividend yield (annual dividend rate expressed as a percentage of the price of the security) or foreign risk-free interest rate for options written on stock indices and currencies.

Note `blsprice` can handle other types of underlies like Futures and Currencies. When pricing Futures (Black model), enter the input argument `Yield` as:

$$\text{Yield} = \text{Rate}$$

When pricing currencies (Garman-Kohlhagen model), enter the input argument `Yield` as:

$$\text{Yield} = \text{ForeignRate}$$

where `ForeignRate` is the continuously compounded, annualized risk-free interest rate in the foreign country.

Data Types: double

Output Arguments**Call — Price of a European call option**

matrix

Price of a European call option, returned as a matrix.

Put — Price of a European put option

matrix

Price of a European put option, returned as a matrix.

More About**Black-Scholes Model**

The Black-Scholes model is a mathematical model used for pricing European-style options, which can only be exercised at expiration.

Developed by economists Fischer Black, Myron Scholes, and Robert Merton in the early 1970s, the model provides a theoretical framework for determining the fair value of options. The primary

purpose of the Black-Scholes model is to calculate the theoretical price of options, allowing traders and investors to assess whether an option is fairly priced, underpriced, or overpriced in the market.

Version History

Introduced in R2006a

References

[1] Hull, John C. *Options, Futures, and Other Derivatives*. 5th edition, Prentice Hall, 2003.

[2] Luenberger, David G. *Investment Science*. Oxford University Press, 1998.

See Also

[blsimpv](#) | [blkprice](#) | [blsgamma](#) | [blsdelta](#) | [blslambda](#) | [blsrho](#) | [blstheta](#) | [blsvega](#)

Topics

“Pricing and Analyzing Equity Derivatives” on page 2-39

“Greek-Neutral Portfolios of European Stock Options” on page 10-14

“Plotting Sensitivities of an Option” on page 10-25

“Plotting Sensitivities of a Portfolio of Options” on page 10-27

blsrho

Black-Scholes sensitivity to interest-rate change

Syntax

```
[CallRho,PutRho] = blsrho(Price,Strike,Rate,Time,Volatility)
[CallRho,PutRho] = blsrho( ____,Yield)
```

Description

[CallRho,PutRho] = blsrho(Price,Strike,Rate,Time,Volatility) returns the call option rho CallRho, and the put option rho PutRho. Rho is the rate of change in value of derivative securities with respect to interest rates. blsrho uses normcdf, the normal cumulative distribution function in the Statistics and Machine Learning Toolbox.

In addition, you can use the Financial Instruments Toolbox object framework with the BlackScholes pricer object to obtain price and rho values for a Vanilla, Barrier, Touch, DoubleTouch, or Binary instrument using a BlackScholes model.

Note blsrho can also handle an underlying asset such as currencies. When pricing currencies (Garman-Kohlhagen model), enter the input argument Yield as:

Yield = ForeignRate

where ForeignRate is the continuously compounded, annualized risk-free interest rate in the foreign country.

[CallRho,PutRho] = blsrho(____,Yield) adds an optional argument for Yield.

Examples

Find the Black-Scholes Sensitivity (Rho) to Interest-Rate Change

This example shows how to find the Black-Scholes sensitivity, rho, to interest-rate change.

```
[CallRho, PutRho] = blsrho(50, 50, 0.12, 0.25, 0.3, 0)
```

```
CallRho =  
6.6686
```

```
PutRho =  
-5.4619
```

Input Arguments

Price — Current price of underlying asset
numeric

Current price of the underlying asset, specified as a numeric value.

Data Types: double

Strike — Exercise price of option

numeric

Exercise price of the option, specified as a numeric value.

Data Types: double

Rate — Annualized, continuously compounded risk-free rate of return over life of option

positive decimal

Annualized, continuously compounded risk-free rate of return over the life of the option, specified as a positive decimal value.

Data Types: double

Time — Time (in years) to expiration of the option

numeric

Time (in years) to expiration of the option, specified as a numeric value.

Data Types: double

Volatility — Annualized asset price volatility

positive decimal

Annualized asset price volatility (annualized standard deviation of the continuously compounded asset return), specified as a positive decimal value.

Data Types: double

Yield — Annualized, continuously compounded yield of the underlying asset over life of option

0 (default) | decimal

(Optional) Annualized, continuously compounded yield of the underlying asset over the life of the option, specified as a decimal value. For example, for options written on stock indices, Yield could represent the dividend yield. For currency options, Yield could be the foreign risk-free interest rate.

Data Types: double

Output Arguments

CallRho — Call option rho

numeric

Call option rho, returned as a numeric value.

PutRho — Put option rho

numeric

Put option rho, returned as a numeric value.

More About

Rho

A rho sensitivity measures the rate at which the price of an option is expected to change in response to a change in the risk-free interest rate.

Rho is expressed as the amount of money an option's price would gain or lose for a one percentage point (1%) change in the risk-free interest rate. For instance, if a call option has a rho of 0.05 , it means that for every one percentage point increase in interest rates, the price of the call option is expected to increase by \$0.05.

Version History

Introduced in R2006a

References

[1] Hull, John C. *Options, Futures, and Other Derivatives*. 5th edition, Prentice Hall, 2003.

See Also

`blsgamma` | `blsdelta` | `blslambda` | `blsprice` | `blstheta` | `blsvega`

Topics

“Pricing and Analyzing Equity Derivatives” on page 2-39

“Greek-Neutral Portfolios of European Stock Options” on page 10-14

“Plotting Sensitivities of an Option” on page 10-25

“Plotting Sensitivities of a Portfolio of Options” on page 10-27

blsttheta

Black-Scholes sensitivity to time-until-maturity change

Syntax

```
[CallTheta,PutTheta] = blsttheta(Price,Strike,Rate,Time,Volatility)
[CallTheta,PutTheta] = blsttheta( ___,Yield)
```

Description

[CallTheta,PutTheta] = blsttheta(Price,Strike,Rate,Time,Volatility) returns the call option theta CallTheta, and the put option theta PutTheta.

Theta is the sensitivity in option value with respect to time and is measured in years. CallTheta or PutTheta can be divided by 365 to get Theta per calendar day or by 252 to get Theta by trading day.

blsttheta uses normcdf, the normal cumulative distribution function, and normpdf, the normal probability density function, in the Statistics and Machine Learning Toolbox.

In addition, you can use the Financial Instruments Toolbox object framework with the BlackScholes pricer object to obtain price and theta values for a Vanilla, Barrier, Touch, DoubleTouch, or Binary instrument using a BlackScholes model.

Note blsttheta can handle other types of underlies like Futures and Currencies. When pricing Futures (Black model), enter the input argument Yield as:

Yield = Rate

When pricing currencies (Garman-Kohlhagen model), enter the input argument Yield as:

Yield = ForeignRate

where ForeignRate is the continuously compounded, annualized risk-free interest rate in the foreign country.

[CallTheta,PutTheta] = blsttheta(___,Yield) adds an optional argument for Yield.

Examples

Compute the Black-Scholes Sensitivity to Time-Until-Maturity Change (Theta)

This example shows how to compute theta, the sensitivity in option value with respect to time.

```
[CallTheta, PutTheta] = blsttheta(50, 50, 0.12, 0.25, 0.3, 0)
```

```
CallTheta =  
-8.9630
```

```
PutTheta =  
-3.1404
```

Input Arguments

Price — Current price of underlying asset

numeric

Current price of the underlying asset, specified as a numeric value.

Data Types: double

Strike — Exercise price of option

numeric

Exercise price of the option, specified as a numeric value.

Data Types: double

Rate — Annualized, continuously compounded risk-free rate of return over life of option

positive decimal

Annualized, continuously compounded risk-free rate of return over the life of the option, specified as a positive decimal value.

Data Types: double

Time — Time (in years) to expiration of the option

numeric

Time (in years) to expiration of the option, specified as a numeric value.

Data Types: double

Volatility — Annualized asset price volatility

positive decimal

Annualized asset price volatility (annualized standard deviation of the continuously compounded asset return), specified as a positive decimal value.

Data Types: double

Yield — Annualized, continuously compounded yield of the underlying asset over life of option

0 (default) | decimal

(Optional) Annualized, continuously compounded yield of the underlying asset over the life of the option, specified as a decimal value. For example, for options written on stock indices, Yield could represent the dividend yield. For currency options, Yield could be the foreign risk-free interest rate.

Data Types: double

Output Arguments

CallTheta — Call option theta

numeric

Call option theta, returned as a numeric value.

PutTheta — Put option theta

numeric

Put option theta, returned as a numeric value.

More About**Theta**

A theta sensitivity measures the rate at which the price of an option decreases as time passes, all else being equal.

Theta is essentially a quantification of time decay, which is a key concept in options pricing. Theta provides an estimate of the dollar amount that an option's price would decrease each day, assuming no movement in the price of the underlying asset and no change in volatility. For example, if a call option has a theta of -0.05 , it means that the option's price is expected to decrease by \$0.05 per day, holding all other factors constant.

Version History

Introduced in R2006a

References

[1] Hull, John C. *Options, Futures, and Other Derivatives*. 5th edition, Prentice Hall, 2003.

See Also

[blslambda](#) | [blsgamma](#) | [blsdelta](#) | [blsprice](#) | [blsrho](#) | [blsvega](#)

Topics

"Pricing and Analyzing Equity Derivatives" on page 2-39

"Greek-Neutral Portfolios of European Stock Options" on page 10-14

"Plotting Sensitivities of an Option" on page 10-25

"Plotting Sensitivities of a Portfolio of Options" on page 10-27

blsvega

Black-Scholes sensitivity to underlying price volatility

Syntax

```
Vega = blsvega(Price,Strike,Rate,Time,Volatility)
Vega = blsvega( ___,Yield)
```

Description

`Vega = blsvega(Price,Strike,Rate,Time,Volatility)` returns the rate of change of the option value with respect to the volatility of the underlying asset. `blsvega` uses `normpdf`, the normal probability density function in the Statistics and Machine Learning Toolbox.

In addition, you can use the Financial Instruments Toolbox object framework with the `BlackScholes` pricer object to obtain price and vega values for a `Vanilla`, `Barrier`, `Touch`, `DoubleTouch`, or `Binary` instrument using a `BlackScholes` model.

Note `blsvega` can handle other types of underlies like Futures and Currencies. When pricing Futures (Black model), enter the input argument `Yield` as:

```
Yield = Rate
```

When pricing currencies (Garman-Kohlhagen model), enter the input argument `Yield` as:

```
Yield = ForeignRate
```

where `ForeignRate` is the continuously compounded, annualized risk-free interest rate in the foreign country.

`Vega = blsvega( ___,Yield)` adds an optional argument for `Yield`.

Examples

Compute Black-Scholes Sensitivity to Underlying Price Volatility (Vega)

This example shows how to compute vega, the rate of change of the option value with respect to the volatility of the underlying asset.

```
Vega = blsvega(50, 50, 0.12, 0.25, 0.3, 0)
```

```
Vega =
9.6035
```

Input Arguments

Price — Current price of underlying asset

numeric

Current price of the underlying asset, specified as a numeric value.

Data Types: double

Strike — Exercise price of option

numeric

Exercise price of the option, specified as a numeric value.

Data Types: double

Rate — Annualized, continuously compounded risk-free rate of return over life of option

positive decimal

Annualized, continuously compounded risk-free rate of return over the life of the option, specified as a positive decimal value.

Data Types: double

Time — Time (in years) to expiration of the option

numeric

Time (in years) to expiration of the option, specified as a numeric value.

Data Types: double

Volatility — Annualized asset price volatility

positive decimal

Annualized asset price volatility (annualized standard deviation of the continuously compounded asset return), specified as a positive decimal value.

Data Types: double

Yield — Annualized, continuously compounded yield of the underlying asset over life of option

0 (default) | decimal

(Optional) Annualized, continuously compounded yield of the underlying asset over the life of the option, specified as a decimal value. For example, for options written on stock indices, Yield could represent the dividend yield. For currency options, Yield could be the foreign risk-free interest rate.

Data Types: double

Output Arguments

Vega — Rate of change of option value with respect to volatility of underlying asset

numeric

Rate of change of the option value with respect to the volatility of the underlying asset, returned as a numeric value.

More About

Vega

A vega sensitivity measures the sensitivity of an option's price to changes in the volatility of the underlying asset.

Vega represents the amount by which the price of an option would be expected to change for a 1% change in the implied volatility of the underlying asset. Vega is expressed as the amount of money per underlying share that the option's value will gain or lose as volatility rises or falls. For example, if an option has a vega of 0.10, it means that the option's price is expected to change by \$0.10 for every 1% change in the implied volatility of the underlying asset.

Version History

Introduced in R2006a

References

[1] Hull, John C. *Options, Futures, and Other Derivatives*. 5th edition, Prentice Hall, 2003.

See Also

`blslambda` | `blsgamma` | `blsdelta` | `blsprice` | `blsrho` | `blstheta`

Topics

"Pricing and Analyzing Equity Derivatives" on page 2-39

"Greek-Neutral Portfolios of European Stock Options" on page 10-14

"Plotting Sensitivities of an Option" on page 10-25

"Plotting Sensitivities of a Portfolio of Options" on page 10-27

bondDefaultBootstrap

Bootstrap default probability curve from bond prices

Syntax

```
[ProbabilityData,HazardData] = bondDefaultBootstrap(ZeroData,MarketData,Settle)
[ProbabilityData,HazardData] = bondDefaultBootstrap( ___,Name,Value)
```

Description

[ProbabilityData,HazardData] = bondDefaultBootstrap(ZeroData,MarketData,Settle) bootstraps the default probability curve from bond prices.

Using bondDefaultBootstrap, you can:

- Extract discrete default probabilities for a certain period from market bond data.
- Interpolate these default probabilities to get the default probability curve for pricing and risk management purposes.

[ProbabilityData,HazardData] = bondDefaultBootstrap(___,Name,Value) adds optional name-value pair arguments.

Examples

Determine the Default Probability and Hazard Rate Values for Treasury Bonds

Use the following bond data.

```
Settle = datenum('08-Jul-2016');
MarketDate = datenum({'06/15/2018', '01/08/2019', '02/01/2021', '03/18/2021', '08/04/2025'}','mm/dd/yyyy');
CouponRate = [2.240 2.943 5.750 3.336 4.134]/100;
MarketPrice = [101.300 103.020 115.423 104.683 108.642]';
MarketData = [MarketDate,MarketPrice,CouponRate];
```

Calculate the ProbabilityData and HazardData.

```
TreasuryParYield = [0.26 0.28 0.36 0.48 0.61 0.71 0.95 1.19 1.37 1.69 2.11]'/100;
TreasuryDates = datemnth(Settle, [[1 3 6], 12 * [1 2 3 5 7 10 20 30]]');
[ZeroRates, CurveDates] = pyld2zero(TreasuryParYield, TreasuryDates, Settle);
ZeroData = [CurveDates, ZeroRates];
format longg
[ProbabilityData,HazardData]=bondDefaultBootstrap(ZeroData,MarketData,Settle)
```

ProbabilityData = 5×2

737226	0.0299675399937611
737433	0.0418832295824674
738188	0.090518332884262
738233	0.101248065083713

```
739833      0.233002708031915
```

```
HazardData = 5×2
```

```
737226      0.0157077745460244
737433      0.0217939816590403
738188      0.025184912824721
738233      0.0962608718640789
739833      0.0361632398787917
```

In `bondDefaultBootstrap`, the first column of the `ProbabilityData` output and the first column of the `HazardData` output contain the respective ending dates for the corresponding default probabilities and hazard rates. However, the starting dates used for the computation of the time ranges for default probabilities can be different from those of hazard rates. For default probabilities, the time ranges are all computed from the `Settle` date to the respective end dates shown in the first column of `ProbabilityData`. In contrast, the time ranges for the hazard rates are computed using the `Settle` date and the first column of `HazardData`, so that the first hazard rate applies from the `Settle` date to the first market date, the second hazard rate from the first to the second market date, and so on, and the last hazard rate applies from the second-to-last market date onwards.

```
datestr(Settle)
```

```
ans =
'08-Jul-2016'
```

```
datestr(ProbabilityData(:,1))
```

```
ans = 5×11 char array
'15-Jun-2018'
'08-Jan-2019'
'01-Feb-2021'
'18-Mar-2021'
'04-Aug-2025'
```

```
datestr(HazardData(:,1))
```

```
ans = 5×11 char array
'15-Jun-2018'
'08-Jan-2019'
'01-Feb-2021'
'18-Mar-2021'
'04-Aug-2025'
```

The time ranges for the default probabilities all start on '08-Jul-2016' and they end on '15-Jun-2018', '08-Jan-2019', '01-Feb-2021', '18-Mar-2021', and '04-Aug-2025', respectively. As for the hazard rates, the first hazard rate starts on '08-Jul-2016' and ends on '15-Jun-2018', the second hazard rate starts on '15-Jun-2018' and ends on '08-Jan-2019', the third hazard rate starts on '08-Jan-2019' and ends on '01-Feb-2021', and so forth.

Reprice a Bond Listed on the Default Probability Curve

Reprice one of the bonds from bonds list based on the default probability curve. The expected result of this repricing is a perfect match with the market quote.

Use the following Treasury data from US Department of the Treasury.

```
Settle = datetime('08-Jul-2016','Locale','en_US');
TreasuryParYield = [0.26 0.28 0.36 0.48 0.61 0.71 0.95 1.19 1.37 1.69 2.11]'/100;
TreasuryDates = datemnth(Settle, [[1 3 6], 12 * [1 2 3 5 7 10 20 30]]');
```

Preview the bond data using semiannual coupon bonds with market quotes, coupon rates, and a settle date of July-08-2016.

```
MarketDate = datenum({'06/01/2017','06/01/2019','06/01/2020','06/01/2022'}, 'mm/dd/yyyy');
CouponRate = [7 8 9 10]'/100;
MarketPrice = [101.300 109.020 114.42 118.62]';
MarketData = [MarketDate, MarketPrice, CouponRate];
```

```
BondList = array2table(MarketData, 'VariableNames', {'Maturity', 'Price', 'Coupon'});
BondList.Maturity = datetime(BondList.Maturity, 'Locale', 'en_US', 'ConvertFrom', 'datenum');
BondList.Maturity.Format = 'MMM-dd-yyyy'
```

```
BondList=4x3 table
      Maturity      Price      Coupon
      _____      _____      _____
Jun-01-2017      101.3      0.07
Jun-01-2019      109.02      0.08
Jun-01-2020      114.42      0.09
Jun-01-2022      118.62      0.1
```

Choose the second coupon bond as the one to be priced.

```
number = 2;
TestCase = BondList(number, :);
```

Preview the risk-free rate data provided here that is based on a continuous compound rate.

```
[ZeroRates, CurveDates] = pyld2zero(TreasuryParYield, TreasuryDates, Settle);
ZeroData = [datenum(CurveDates), ZeroRates];
RiskFreeRate = array2table(ZeroData, 'VariableNames', {'Date', 'Rate'});
RiskFreeRate.Date = datetime(RiskFreeRate.Date, 'Locale', 'en_US', 'ConvertFrom', 'datenum');
RiskFreeRate.Date.Format = 'MMM-dd-yyyy'
```

```
RiskFreeRate=11x2 table
      Date      Rate
      _____      _____
Aug-08-2016      0.0026057
Oct-08-2016      0.0027914
Jan-08-2017      0.0035706
Jul-08-2017      0.0048014
Jul-08-2018      0.0061053
Jul-08-2019      0.0071115
Jul-08-2021      0.0095416
Jul-08-2023      0.012014
```

```

Jul-08-2026    0.013883
Jul-08-2036    0.017359
Jul-08-2046    0.022704

```

Bootstrap the probability of default (PD) curve from the bonds.

```

format longg
[defaultProb1, hazard1] = bondDefaultBootstrap(ZeroData, MarketData, Settle)

```

```
defaultProb1 = 4×2
```

```

736847    0.0704863142317494
737577    0.162569420050034
737943    0.217308133826188
738673    0.38956773145021

```

```
hazard1 = 4×2
```

```

736847    0.0813390794774647
737577    0.0521615800986281
737943    0.0674145844133183
738673    0.12428587278862

```

```
format
```

Reformat the default probability and hazard rate for a better representation.

```

DefProbHazard = [defaultProb1, hazard1(:,2)];
DefProbHazardTable = array2table(DefProbHazard, 'VariableNames', {'Date', 'DefaultProbability',
DefProbHazardTable.Date = datetime(DefProbHazardTable.Date, 'Locale', 'en_US', 'ConvertFrom', 'datetime
DefProbHazardTable.Date.Format = 'MMM-dd-yyyy'

```

```
DefProbHazardTable=4×3 table
```

Date	DefaultProbability	HazardRate
Jun-01-2017	0.070486	0.081339
Jun-01-2019	0.16257	0.052162
Jun-01-2020	0.21731	0.067415
Jun-01-2022	0.38957	0.12429

Preview the selected bond to reprice based on the PD curve.

```
TestCase
```

```
TestCase=1×3 table
```

Maturity	Price	Coupon
Jun-01-2019	109.02	0.08

To reprice the bond, first generate cash flows and payment dates.

```

[Payments, PaymentDates] = cfamounts(TestCase.Coupon, Settle, TestCase.Maturity);
AccInt=-Payments(1);

```

```

% Truncate the payments as well as payment dates for calculation
% PaymentDates(1) is the settle date, no need for following calculations
PaymentDates = PaymentDates(2:end)

```

```

PaymentDates = 1×6 datetime
    01-Dec-2016    01-Jun-2017    01-Dec-2017    01-Jun-2018    01-Dec-2018    01-Jun-2019

```

```

Payments = Payments(2:end)

```

```

Payments = 1×6
         4         4         4         4         4        104

```

Calculate the discount factors on the payment dates.

```

DF = zero2disc(interp1(RiskFreeRate.Date, RiskFreeRate.Rate, PaymentDates, 'linear', 'extrap'), 1)

```

```

DF = 1×6
    0.9987    0.9959    0.9926    0.9887    0.9845    0.9799

```

Assume that the recovery amount is a fixed proportion of bond's face value. The bond's face value is 100, and the recovery ratio is set to 40% as assumed in bondDefaultBootstrap.

```

Num = length(Payments);
RecoveryAmount = repmat(100*0.4, 1, Num)

```

```

RecoveryAmount = 1×6
         40         40         40         40         40         40

```

Calculate the probability of default based on the default curve.

```

DefaultProb1 = bondDefaultBootstrap(ZeroData, MarketData, Settle, 'ZeroCompounding', -1, 'Probability');
SurvivalProb = 1 - DefaultProb1(:,2)

```

```

SurvivalProb = 6×1

```

```

    0.9680
    0.9295
    0.9055
    0.8823
    0.8595
    0.8375

```

Calculate the model-based clean bond price.

```

DirtyPrice = DF * (SurvivalProb.*Payments') + (RecoveryAmount.*DF) * (-diff([1;SurvivalProb]));
ModelPrice = DirtyPrice - AccInt

```

```

ModelPrice =
    109.0200

```

Compare the repriced bond to the market quote.

```
ResultTable = TestCase;
ResultTable.ModelPrice = ModelPrice;
ResultTable.Difference = ModelPrice - TestCase.Price
```

```
ResultTable=1x5 table
    Maturity    Price    Coupon    ModelPrice    Difference
    _____    _____    _____    _____    _____
    Jun-01-2019    109.02    0.08    109.02    1.4211e-14
```

Input Arguments

ZeroData — Zero rate data

matrix | IRDataCurve object

Zero rate data, specified as an M-by-2 matrix of dates and zero rates or an IRDataCurve object of zero rates. For array input, the dates must be entered as serial date numbers, and discount rate must be in decimal form.

When ZeroData is an IRDataCurve object, ZeroCompounding and ZeroBasis are implicit in ZeroData and are redundant inside this function. In this case, specify these optional parameters when constructing the IRDataCurve object before using this bondDefaultBootstrap function.

For more information on an IRDataCurve object, see “Creating an IRDataCurve Object” (Financial Instruments Toolbox).

Data Types: double

MarketData — Bond market data

matrix

Bond market data, specified as an N-by-3 matrix of maturity dates, market prices, and coupon rates for bonds. The dates must be entered as serial date numbers, market prices must be numeric values, and coupon rate must be in decimal form.

Note A warning is displayed when MarketData is not sorted in ascending order by time.

Data Types: double

Settle — Settlement date

datetime scalar | string scalar | date character vector

Settlement date, specified as a scalar datetime, string, or date character vector. Settle must be earlier than or equal to the maturity dates in MarketData.

To support existing code, bondDefaultBootstrap also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | datetime | string

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Note Any optional input of size N-by-1 is also acceptable as an array of size 1-by-N, or as a single value applicable to all contracts.

Example: `[ProbabilityData,HazardData] = bondDefaultBootstrap(ZeroData,MarketData,Settle,'RecoveryRate',Recovery,'ZeroCompounding',-1)`

RecoveryRate — Recovery rate

0.4 (default) | decimal

Recovery rate, specified as the comma-separated pair consisting of `'RecoveryRate'` and a N-by-1 vector of recovery rates, expressed as a decimal from 0 through 1.

Data Types: `double`

ProbabilityDates — Dates for output of default probability data

column of dates in `MarketData` (default) | datetime array | string array | date character vector

Dates for the output of default probability data, specified as the comma-separated pair consisting of `'ProbabilityDates'` and a P-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `bondDefaultBootstrap` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `datetime` | `string`

ZeroCompounding — Compounding frequency of the zero curve

2 (semiannual) (default) | integer with value of 1,2,3,4,6,12, or -1

Compounding frequency of the zero curve, specified as the comma-separated pair consisting of `'ZeroCompounding'` and a N-by-1 vector. Values are:

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- -1 — Continuous compounding

Data Types: `double`

ZeroBasis — Basis of the zero curve

0 (actual/actual) (default) | integer with value of 0 to 13

Basis of the zero curve, specified as the comma-separated pair consisting of 'ZeroBasis' and the same values listed for Basis.

Data Types: double

RecoveryMethod — Recovery method

'facevalue' (default) | character vector with value of 'presentvalue' or 'facevalue' | string object with value of 'presentvalue' or 'facevalue'

Recovery method, specified as the comma-separated pair consisting of 'RecoveryMethod' and a character vector or a string with a value of 'presentvalue' or 'facevalue'.

- 'presentvalue' assumes that upon default, a bond is valued at a given fraction to the hypothetical present value of its remaining cash flows, discounted at risk-free rate.
- 'facevalue' assumes that a bond recovers a given fraction of its face value upon recovery.

Data Types: char | string

Face — Face or par value

100 (default) | numeric

Face or par value, specified as the comma-separated pair consisting of 'Face' and a NINST-by-1 vector of bonds.

Data Types: double

Period — Payment frequency

2 (default) | numeric with values 0, 1, 2, 3, 4, 6 or 12

Payment frequency, specified as the comma-separated pair consisting of 'Period' and a N-by-1 vector with values of 0, 1, 2, 3, 4, 6, or 12.

Data Types: double

Basis — Day-count basis of the instrument

0 (actual/actual) (default) | integers of the set [0...13] | vector of integers of the set [0...13]

Day-count basis of the instrument, specified as the comma-separated pair consisting of 'Basis' and a positive integer using a NINST-by-1 vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)

- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as the comma-separated pair consisting of 'EndMonthRule' and a nonnegative integer, 0 or 1, using a NINST-by-1 vector. This rule applies only when Maturity is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: double

IssueDate — Bond issue date

if IssueDate not specified, cash flow payment dates determined from other inputs (default) | datetime array | string array | date character vector

Bond issue date, specified as the comma-separated pair consisting of 'IssueDate' and a N-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, bondDefaultBootstrap also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | datetime | string

FirstCouponDate — First actual coupon date

if you do not specify a FirstCouponDate, cash flow payment dates are determined from other inputs (default) | datetime scalar | string scalar | date character vector

First actual coupon date, specified as the comma-separated pair consisting of 'FirstCouponDate' and a scalar datetime, string, or date character vector. FirstCouponDate is used when a bond has an irregular first coupon period. When FirstCouponDate and LastCouponDate are both specified, FirstCouponDate takes precedence in determining the coupon payment structure.

To support existing code, bondDefaultBootstrap also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | datetime | string

LastCouponDate — Last actual coupon date

if you do not specify a LastCouponDate, cash flow payment dates are determined from other inputs (default) | datetime scalar | string scalar | date character vector

Last actual coupon date, specified as the comma-separated pair consisting of 'LastCouponDate' and a scalar datetime, string, or date character vector. LastCouponDate is used when a bond has an irregular last coupon period. In the absence of a specified FirstCouponDate, a specified LastCouponDate determines the coupon structure of the bond. The coupon structure of a bond is truncated at the LastCouponDate, regardless of where it falls, and is followed only by the bond's maturity cash flow date.

To support existing code, bondDefaultBootstrap also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | datetime | string

StartDate — Forward starting date of payments

if you do not specify StartDate, effective start date is Settle date (default) | datetime scalar | string scalar | date character vector

Forward starting date of payments, specified as the comma-separated pair consisting of 'StartDate' and a scalar datetime, string, or date character vector. StartDate is when a bond actually starts (the date from which a bond cash flow is considered). To make an instrument forward-starting, specify this date as a future date.

To support existing code, bondDefaultBootstrap also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | datetime | string

BusinessDayConvention — Business day conventions

'actual' (default) | character vector or string object with values 'actual', 'follow', 'modifiedfollow', 'previous' or 'modifiedprevious'

Business day conventions, specified as the comma-separated pair consisting of 'BusinessDayConvention' and a character vector or a string object. The selection for business day convention determines how nonbusiness days are treated. Nonbusiness days are defined as weekends plus any other date that businesses are not open (for example, statutory holidays). Values are:

- 'actual' — Nonbusiness days are effectively ignored. Cash flows that fall on non-business days are assumed to be distributed on the actual date.
- 'follow' — Cash flows that fall on a nonbusiness day are assumed to be distributed on the following business day.
- 'modifiedfollow' — Cash flows that fall on a non-business day are assumed to be distributed on the following business day. However if the following business day is in a different month, the previous business day is adopted instead.
- 'previous' — Cash flows that fall on a nonbusiness day are assumed to be distributed on the previous business day.
- 'modifiedprevious' — Cash flows that fall on a nonbusiness day are assumed to be distributed on the previous business day. However if the previous business day is in a different month, the following business day is adopted instead.

Data Types: char | cell | string

Output Arguments

ProbabilityData — Default probability values

matrix

Default probability values, returned as a P-by-2 matrix with dates and corresponding cumulative default probability values. The dates match those in `MarketData`, unless the optional input parameter `ProbabilityDates` is provided.

HazardData — Hazard rate values

matrix

Hazard rate values, returned as an N-by-2 matrix with dates and corresponding hazard rate values for the survival probability model. The dates match those in `MarketData`.

Note A warning is displayed when nonmonotone default probabilities (that is, negative hazard rates) are found.

More About

Bootstrap Default Probability

A default probability curve can be bootstrapped from a collection of bond market quotes.

Extracting discrete default probabilities for a certain period from market bond data is represented by the formula

$$Price = Disc(t_N) \times FV \times Q(t_N) + \frac{C}{f} \times \sum_{i=1}^N Disc(t_i) \times Q(t_i) + \sum_{(i=1)}^N Disc(t_i) \times R(t_i) \times (Q(t_i - 1) - Q(t_i))$$

where:

FV — Face value

Q — Survival probability

C — Coupon

R — Recovery amount

f — Payment frequency (for example, 2 for semiannual coupon bonds)

The default probability is:

`DefaultProbability = 1 - SurvivalProbability`

Version History

Introduced in R2017a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `bondDefaultBootstrap` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] Jarrow, Robert A., and Stuart Turnbull. "Pricing Derivatives on Financial Securities Subject to Credit Risk." *Journal of Finance*. 50.1, 1995, pp. 53-85.
- [2] Berd, A., Mashal, R. and Peili Wang. "Defining, Estimating and Using Credit Term Structures." Research report, Lehman Brothers, 2004.

See Also

`IRDataCurve` | `cdsbootstrap`

Topics

"Creating an `IRDataCurve` Object" (Financial Instruments Toolbox)

bndconvp

Bond convexity given price

Syntax

```
[YearConvexity,PerConvexity] = bndconvp(Price,CouponRate,Settle,Maturity)
[YearConvexity,PerConvexity] = bndconvp( ___,Name,Value)
```

Description

`[YearConvexity,PerConvexity] = bndconvp(Price,CouponRate,Settle,Maturity)` computes the convexity of `NUMBONDS` fixed income securities given a clean price for each bond. The clean price of a bond excludes any interest that has accrued since issue or the most recent coupon payment.

`bndconvp` determines the convexity for a bond whether the first or last coupon periods in the coupon structure are short or long (that is, whether the coupon structure is synchronized to maturity). `bndconvp` also determines the convexity of a zero coupon bond. Convexity is a measure of the rate of change in duration; measured in time. The greater the rate of change, the more the duration changes as yield changes.

`[YearConvexity,PerConvexity] = bndconvp( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Find Bond Convexity Given Price

This example shows how to compute the convexity of three bonds given their prices.

```
Price = [106; 100; 98];
CouponRate = 0.055;
Settle = datetime(1999,8,2);
Maturity = datetime(2004,6,15);
Period = 2;
Basis = 0;

[YearConvexity, PerConvexity] = bndconvp(Price,...
    CouponRate,Settle, Maturity, Period, Basis)
```

```
YearConvexity = 3×1
```

```
    21.4447
    21.0363
    20.8951
```

```
PerConvexity = 3×1
```

```
    85.7788
```

```
84.1454  
83.5803
```

Input Arguments

Price — Clean price (excludes accrued interest)

numeric

Clean price (excludes accrued interest), specified as numeric value using a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector.

Data Types: double

CouponRate — Annual percentage rate used to determine coupons payable on a bond

decimal

Annual percentage rate used to determine the coupons payable on a bond, specified as decimal value using a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector.

Data Types: double

Settle — Settlement date for certificate of deposit

datetime array | string array | date character vector

Settlement date for the certificate of deposit, specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a datetime array, string array, or date character vectors. The `Settle` date must be before the `Maturity` date.

To support existing code, `bndconvp` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date for certificate of deposit

datetime array | string array | date character vector

Maturity date for the certificate of deposit, specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a datetime array, string array, or date character vectors.

To support existing code, `bndconvp` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[YearConvexity,PerConvexity] = bndconvp(Price,CouponRate,Settle,Maturity,'Period',4,'Basis',7)`

Period — Number of coupon payments per year

2 (default) | numeric with values 0, 1, 2, 3, 4, 6 or 12

Number of coupon payments per year, specified as the comma-separated pair consisting of 'Period' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using the values: 0, 1, 2, 3, 4, 6, or 12.

Data Types: double

Basis — Day-count basis of instrument

0 (default) | numeric values: 0,1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day-count of the instrument, specified as the comma-separated pair consisting of 'Basis' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a supported value:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as the comma-separated pair consisting of 'EndMonthRule' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. This rule applies only when Maturity is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: logical

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond Issue date, specified as the comma-separated pair consisting of 'IssueDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify an IssueDate, the cash flow payment dates are determined from other inputs.

To support existing code, bndconvp also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

FirstCouponDate — Irregular or normal first coupon date

datetime array | string array | date character vector

Irregular or normal first coupon date, specified as the comma-separated pair consisting of 'FirstCouponDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify a FirstCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, bndconvp also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

LastCouponDate — Irregular or normal last coupon date

datetime array | string array | date character vector

Irregular or normal last coupon date, specified as the comma-separated pair consisting of 'LastCouponDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify a LastCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, bndconvp also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

StartDate — Forward starting date of payments

datetime array | string array | date character vector

Forward starting date of payments, specified as the comma-separated pair consisting of 'StartDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors. The StartDate is when a bond actually starts (the date from which a bond cash flow is considered). To make an instrument forward-starting, specify this date as a future date.

If you do not specify a StartDate, the effective start date is the Settle date.

To support existing code, bndconvp also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Face — Face value of bond

100 (default) | numeric

Face value of the bond, specified as the comma-separated pair consisting of 'Face' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

Data Types: double

CompoundingFrequency — Compounding frequency for yield calculation

SIA bases uses 2, ICMA bases uses 1 (default) | integer with value of 1, 2, 3, 4, 6, or 12

Compounding frequency for yield calculation, specified as the comma-separated pair consisting of 'CompoundingFrequency' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note By default, SIA bases (0-7) and BUS/252 use a semiannual compounding convention and ICMA bases (8-12) use an annual compounding convention.

Data Types: double

DiscountBasis — Basis used to compute the discount factors for computing the yield

SIA uses 0 (default) | integers of the set [0...13] | vector of integers of the set [0...13]

Basis used to compute the discount factors for computing the yield, specified as the comma-separated pair consisting of 'DiscountBasis' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)

- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If a SIA day-count basis is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the default behavior is for SIA bases to use the actual/actual day count to compute discount factors.

If an ICMA day-count basis or BUS/252 is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the specified bases from the `Basis` input argument are used.

Data Types: double

LastCouponInterest — Compounding convention for computing yield of a bond in last coupon period

compound (default) | values are simple or compound

Compounding convention for computing the yield of a bond in the last coupon period, specified as the comma-separated pair consisting of 'LastCouponInterest' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. LastCouponInterest is based on only the last coupon and the face value to be repaid. Acceptable values are:

- simple
- compound

Data Types: char | cell

Output Arguments

YearConvexity — Yearly (annualized) convexity

numeric

Yearly (annualized) convexity, returned as a NUMBONDS-by-1 vector.

PerConvexity — Periodic convexity reported on semiannual bond basis

numeric

Periodic convexity reported on a semiannual bond basis (in accordance with SIA convention), returned as a NUMBONDS-by-1 vector.

More About

Bond Convexity

Bond convexity is a measure of the curvature in the relationship between bond prices and bond yields.

Bond convexity quantifies how much the price of a bond changes in response to changes in interest rates, taking into account the nonlinear relationship between price and yield. Convexity is an important concept in fixed-income investing because it helps investors understand the sensitivity of a bond's price to interest rate movements beyond what is captured by duration alone.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although bndconvp supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

R2017b: Support for optional name-value input arguments

Behavior changed in R2017b

The specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the optional name-value pair inputs: `Period`, `Basis`, `EndMonthRule`, `IssueDate`, `FirstCouponDate`, `LastCouponDate`, `StartDate`, `Face`, `CompoundingFrequency`, `DiscountBasis`, and `LastCouponInterest`.

References

- [1] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.
- [2] Mayle, J. "Standard Securities Calculations Methods: Fixed Income Securities Formulas for Analytic Measures." SIA, Vol 2, Jan 1994.
- [3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

`cfconv` | `bndconvy` | `bnddurp` | `bnddury` | `cfdur` | `datetime`

Topics

"Bond Portfolio for Hedging Duration and Convexity" on page 10-6
 "Yield Conventions" on page 2-21

bndconvy

Bond convexity given yield

Syntax

```
[YearConvexity,PerConvexity] = bndconvy(Yield,CouponRate,Settle,Maturity)
[YearConvexity,PerConvexity] = bndconvy( ___,Name,Value)
```

Description

[YearConvexity,PerConvexity] = bndconvy(Yield,CouponRate,Settle,Maturity) computes the convexity of NUMBONDS fixed income securities given a clean price for each bond.

bndconvy determines the convexity for a bond whether the first or last coupon periods in the coupon structure are short or long (that is, whether the coupon structure is synchronized to maturity). bndconvy also determines the convexity of a zero coupon bond.

[YearConvexity,PerConvexity] = bndconvy(___,Name,Value) adds optional name-value pair arguments.

Examples

Find Bond Convexity Given Yield

This example shows how to compute the convexity of a bond at three different yield values.

```
Yield = [0.04; 0.055; 0.06];
CouponRate = 0.055;
Settle = datetime(1999,8,2);
Maturity = datetime(2004,6,15);
Period = 2;
Basis = 0;
```

```
[YearConvexity, PerConvexity]=bndconvy(Yield, CouponRate,...
Settle, Maturity, Period, Basis)
```

```
YearConvexity = 3×1
```

```
21.4825
21.0358
20.8885
```

```
PerConvexity = 3×1
```

```
85.9298
84.1434
83.5541
```

Input Arguments

Yield — Yield to maturity on semiannual basis

numeric

Yield to maturity on a semiannual basis, specified as numeric value using a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

Data Types: double

CouponRate — Annual percentage rate used to determine coupons payable on a bond

decimal

Annual percentage rate used to determine the coupons payable on a bond, specified as decimal value using a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

Data Types: double

Settle — Settlement date for certificate of deposit

datetime array | string array | date character vector

Settlement date for the certificate of deposit, specified as a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors. The Settle date must be before the Maturity date.

To support existing code, bndconvy also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date for certificate of deposit

datetime array | string array | date character vector

Maturity date for the certificate of deposit, specified as a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

To support existing code, bndconvy also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[YearConvexity,PerConvexity] = bndconvy(Yield,CouponRate,Settle,Maturity,'Period',4,'Basis',7)`

Period — Number of coupon payments per year

2 (default) | numeric with values 0, 1, 2, 3, 4, 6 or 12

Number of coupon payments per year, specified as the comma-separated pair consisting of 'Period' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using the values: 0, 1, 2, 3, 4, 6, or 12.

Data Types: double

Basis — Day-count basis of instrument

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day-count of the instrument, specified as the comma-separated pair consisting of 'Basis' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a supported value:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as the comma-separated pair consisting of 'EndMonthRule' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. This rule applies only when Maturity is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: logical

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond Issue date, specified as the comma-separated pair consisting of 'IssueDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify an IssueDate, the cash flow payment dates are determined from other inputs.

To support existing code, `bndconvy` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

FirstCouponDate — Irregular or normal first coupon date

`datetime` array | `string` array | `date` character vector

Irregular or normal first coupon date, specified as the comma-separated pair consisting of 'FirstCouponDate' and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime` array, `string` array, or `date` character vectors.

If you do not specify a `FirstCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `bndconvy` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

LastCouponDate — Irregular or normal last coupon date

`datetime` array | `string` array | `date` character vector

Irregular or normal last coupon date, specified as the comma-separated pair consisting of 'LastCouponDate' and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime` array, `string` array, or `date` character vectors.

If you do not specify a `LastCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `bndconvy` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

StartDate — Forward starting date of payments

`datetime` array | `string` array | `date` character vector

Forward starting date of payments, specified as the comma-separated pair consisting of 'StartDate' and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime` array, `string` array, or `date` character vectors. The `StartDate` is when a bond actually starts (the date from which a bond cash flow is considered). To make an instrument forward-starting, specify this date as a future date.

If you do not specify a `StartDate`, the effective start date is the `Settle` date.

To support existing code, `bndconvy` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Face — Face value of bond

100 (default) | numeric

Face value of the bond, specified as the comma-separated pair consisting of 'Face' and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector.

Data Types: double

CompoundingFrequency — Compounding frequency for yield calculation

SIA bases uses 2, ICMA bases uses 1 (default) | integer with value of 1, 2, 3, 4, 6, or 12

Compounding frequency for yield calculation, specified as the comma-separated pair consisting of 'CompoundingFrequency' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note By default, SIA bases (0-7) and BUS/252 use a semiannual compounding convention and ICMA bases (8-12) use an annual compounding convention.

Data Types: double

DiscountBasis — Basis used to compute the discount factors for computing the yield

SIA uses 0 (default) | integers of the set [0...13] | vector of integers of the set [0...13]

Basis used to compute the discount factors for computing the yield, specified as the comma-separated pair consisting of 'DiscountBasis' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If a SIA day-count basis is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the default behavior is for SIA bases to use the actual/actual day count to compute discount factors.

If an ICMA day-count basis or BUS/252 is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the specified bases from the `Basis` input argument are used.

Data Types: `double`

LastCouponInterest — Compounding convention for computing yield of a bond in last coupon period

`compound` (default) | values are `simple` or `compound`

Compounding convention for computing the yield of a bond in the last coupon period, specified as the comma-separated pair consisting of 'LastCouponInterest' and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector. LastCouponInterest is based on only the last coupon and the face value to be repaid. Acceptable values are:

- `simple`
- `compound`

Data Types: `char` | `cell`

Output Arguments

YearConvexity — Yearly (annualized) convexity

`numeric`

Yearly (annualized) convexity, returned as a `NUMBONDS-by-1` vector.

PerConvexity — Periodic convexity reported on semiannual bond basis

`numeric`

Periodic convexity reported on a semiannual bond basis (in accordance with SIA convention), returned as a `NUMBONDS-by-1` vector.

More About

Bond Convexity

Bond convexity is a measure of the curvature in the relationship between bond prices and bond yields.

Bond convexity quantifies how much the price of a bond changes in response to changes in interest rates, taking into account the nonlinear relationship between price and yield. Convexity is an important concept in fixed-income investing because it helps investors understand the sensitivity of a bond's price to interest rate movements beyond what is captured by duration alone.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `bndconvy` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)  
  
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

R2017b: Support for optional name-value input arguments

Behavior changed in R2017b

The specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the optional name-value pair inputs: `Period`, `Basis`, `EndMonthRule`, `IssueDate`, `FirstCouponDate`, `LastCouponDate`, `StartDate`, `Face`, `CompoundingFrequency`, `DiscountBasis`, and `LastCouponInterest`.

References

- [1] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.
- [2] Mayle, J. "Standard Securities Calculations Methods: Fixed Income Securities Formulas for Analytic Measures." SIA, Vol 2, Jan 1994.
- [3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

`cfconv` | `bndconvp` | `bnddurp` | `bnddury` | `cfdur` | `datetime`

Topics

"Bond Portfolio for Hedging Duration and Convexity" on page 10-6
"Yield Conventions" on page 2-21

bnddurp

Bond duration given price

Syntax

```
[ModDuration, YearDuration, PerDuration] = bnddurp(Price, CouponRate, Settle,
Maturity)
[ModDuration, YearDuration, PerDuration] = bnddurp( ___, Name, Value)
```

Description

[ModDuration, YearDuration, PerDuration] = bnddurp(Price, CouponRate, Settle, Maturity) computes the Macaulay and modified duration of NUMBONDS fixed-income securities given a clean price for each bond.

bnddurp determines the Macaulay and modified duration for a bond whether the first or last coupon periods in the coupon structure are short or long (that is, whether the coupon structure is synchronized to maturity). bnddurp also determines the Macaulay and modified duration for a zero coupon bond.

[ModDuration, YearDuration, PerDuration] = bnddurp(___, Name, Value) adds optional name-value pair arguments.

Examples

Find Bond Duration Given Price

This example shows how to compute the duration of three bonds given their prices.

```
Price = [106; 100; 98];
CouponRate = 0.055;
Settle = datetime(1999,8,2);
Maturity = datetime(2004,6,15);
Period = 2;
Basis = 0;

[ModDuration, YearDuration, PerDuration] = bnddurp(Price,...
CouponRate, Settle, Maturity, Period, Basis)
```

```
ModDuration = 3×1
```

```
4.2400
4.1925
4.1759
```

```
YearDuration = 3×1
```

```
4.3275
4.3077
```

```
4.3007
```

```
PerDuration = 3×1
```

```
8.6549  
8.6154  
8.6014
```

Input Arguments

Price — Clean price (excludes accrued interest)

numeric

Clean price (excludes accrued interest), specified as numeric value using a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector.

Data Types: `double`

CouponRate — Annual percentage rate used to determine coupons payable on a bond

decimal

Annual percentage rate used to determine the coupons payable on a bond, specified as decimal value using a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector.

Data Types: `double`

Settle — Settlement date for certificate of deposit

`datetime array` | `string array` | `date character vector`

Settlement date for the certificate of deposit, specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime array`, `string array`, or `date character vectors`. The `Settle` date must be before the `Maturity` date.

To support existing code, `bnddurp` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date for certificate of deposit

`datetime array` | `string array` | `date character vector`

Maturity date for the certificate of deposit, specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime array`, `string array`, or `date character vectors`.

To support existing code, `bnddurp` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: [ModDuration,YearDuration,PerDuration] =
bnddurp(Price,CouponRate,Settle,Maturity,'Period',4,'Basis',7)

Period — Number of coupon payments per year

2 (default) | numeric with values 0, 1, 2, 3, 4, 6 or 12

Number of coupon payments per year, specified as the comma-separated pair consisting of 'Period' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using the values: 0, 1, 2, 3, 4, 6, or 12.

Data Types: double

Basis — Day-count basis of instrument

0 (default) | numeric values: 0,1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day-count of the instrument, specified as the comma-separated pair consisting of 'Basis' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a supported value:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as the comma-separated pair consisting of 'EndMonthRule' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. This rule applies only when Maturity is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: logical

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond Issue date, specified as the comma-separated pair consisting of 'IssueDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify an IssueDate, the cash flow payment dates are determined from other inputs.

To support existing code, `bnddurp` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

FirstCouponDate — Irregular or normal first coupon date

datetime array | string array | date character vector

Irregular or normal first coupon date, specified as the comma-separated pair consisting of 'FirstCouponDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify a FirstCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, `bnddurp` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

LastCouponDate — Irregular or normal last coupon date

datetime array | string array | date character vector

Irregular or normal last coupon date, specified as the comma-separated pair consisting of 'LastCouponDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify a LastCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, `bnddurp` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

StartDate — Forward starting date of payments

datetime array | string array | date character vector

Forward starting date of payments, specified as the comma-separated pair consisting of 'StartDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors. The StartDate is when a bond actually starts (the date from which a bond cash flow is considered). To make an instrument forward-starting, specify this date as a future date.

If you do not specify a StartDate, the effective start date is the `Settle` date.

To support existing code, `bnddurp` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Face — Face value of bond

100 (default) | numeric

Face value of the bond, specified as the comma-separated pair consisting of 'Face' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

Data Types: double

CompoundingFrequency — Compounding frequency for yield calculation

SIA bases uses 2, ICMA bases uses 1 (default) | integer with value of 1, 2, 3, 4, 6, or 12

Compounding frequency for yield calculation, specified as the comma-separated pair consisting of 'CompoundingFrequency' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note By default, SIA bases (0-7) and BUS/252 use a semiannual compounding convention and ICMA bases (8-12) use an annual compounding convention.

Data Types: double

DiscountBasis — Basis used to compute the discount factors for computing the yield

SIA uses 0 (default) | integers of the set [0...13] | vector of integers of the set [0...13]

Basis used to compute the discount factors for computing the yield, specified as the comma-separated pair consisting of 'DiscountBasis' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)

- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If a SIA day-count basis is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the default behavior is for SIA bases to use the actual/actual day count to compute discount factors.

If an ICMA day-count basis or BUS/252 is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the specified bases from the `Basis` input argument are used.

Data Types: double

LastCouponInterest — Compounding convention for computing yield of a bond in last coupon period

compound (default) | values are simple or compound

Compounding convention for computing the yield of a bond in the last coupon period, specified as the comma-separated pair consisting of 'LastCouponInterest' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. LastCouponInterest is based on only the last coupon and the face value to be repaid. Acceptable values are:

- simple
- compound

Data Types: char | cell

Output Arguments

ModDuration — Modified duration in years

numeric

Modified duration in years reported on a semiannual bond basis (in accordance with SIA convention), returned as a NUMBONDS-by-1 vector.

YearDuration — Macaulay duration in years

numeric

Macaulay duration in years, returned as a NUMBONDS-by-1 vector.

PerDuration — Periodic Macaulay duration

numeric

Periodic Macaulay duration reported on a semiannual bond basis (in accordance with SIA convention), returned as a NUMBONDS-by-1 vector.

More About

Bond Duration

Bond duration is a measure of the sensitivity of a bond's price to changes in interest rates.

Bond duration represents the weighted average time until a bond's cash flows (interest and principal payments) are received. Duration is a critical concept in fixed-income investing because it helps investors assess the interest-rate risk associated with a bond or a bond portfolio.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although bnddurp supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

R2017b: Support for optional name-value input arguments

Behavior changed in R2017b

The specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the optional name-value pair inputs: `Period`, `Basis`, `EndMonthRule`, `IssueDate`, `FirstCouponDate`, `LastCouponDate`, `StartDate`, `Face`, `CompoundingFrequency`, `DiscountBasis`, and `LastCouponInterest`.

References

- [1] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.
- [2] Mayle, J. "Standard Securities Calculations Methods: Fixed Income Securities Formulas for Analytic Measures." SIA, Vol 2, Jan 1994.
- [3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

bndconvy | bndconvp | bndkrdur | bnddury | datetime

Topics

"Bond Portfolio for Hedging Duration and Convexity" on page 10-6
 "Yield Conventions" on page 2-21

bnddury

Bond duration given yield

Syntax

```
[ModDuration,YearDuration,PerDuration] = bnddury(Yield,CouponRate,Settle,
Maturity)
[ModDuration,YearDuration,PerDuration] = bnddury( ___,Name,Value)
```

Description

[ModDuration,YearDuration,PerDuration] = bnddury(Yield,CouponRate,Settle,Maturity) computes the Macaulay and modified duration of NUMBONDS fixed income securities given yield to maturity for each bond.

bnddury determines the Macaulay and modified duration for a bond whether the first or last coupon periods in the coupon structure are short or long (that is, whether the coupon structure is synchronized to maturity). bnddury also determines the Macaulay and modified duration for a zero coupon bond.

[ModDuration,YearDuration,PerDuration] = bnddury(___,Name,Value) adds optional name-value pair arguments.

Examples

Find Bond Duration Given Yield

This example shows how to compute the duration of a bond at three different yield values.

```
Yield = [0.04; 0.055; 0.06];
CouponRate = 0.055;
Settle = datetime(1999,8,2);
Maturity = datetime(2004,6,15);
Period = 2;
Basis = 0;
```

```
[ModDuration,YearDuration,PerDuration]=bnddury(Yield,...
CouponRate, Settle, Maturity, Period, Basis)
```

```
ModDuration = 3×1
```

```
4.2444
4.1924
4.1751
```

```
YearDuration = 3×1
```

```
4.3292
4.3077
```

```
4.3004
```

```
PerDuration = 3×1
```

```
8.6585
8.6154
8.6007
```

Input Arguments

Yield — Yield to maturity on a semiannual basis

decimal

Yield to maturity on a semiannual basis, specified as a decimal value using a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

Data Types: double

CouponRate — Annual percentage rate used to determine coupons payable on a bond

decimal

Annual percentage rate used to determine the coupons payable on a bond, specified as decimal value using a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

Data Types: double

Settle — Settlement date for certificate of deposit

datetime array | string array | date character vector

Settlement date for the certificate of deposit, specified as a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors. The **Settle** date must be before the **Maturity** date.

To support existing code, bnddury also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date for certificate of deposit

datetime array | string array | date character vector

Maturity date for the certificate of deposit, specified as a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

To support existing code, bnddury also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Name-Value Pair Arguments

Specify optional pairs of arguments as **Name1=Value1, ..., NameN=ValueN**, where **Name** is the argument name and **Value** is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[ModDuration,YearDuration,PerDuration] =
bnddury(Yield,CouponRate,Settle,Maturity,'Period',4,'Basis',7)`

Period — Number of coupon payments per year

2 (default) | numeric with values 0, 1, 2, 3, 4, 6 or 12

Number of coupon payments per year, specified as the comma-separated pair consisting of 'Period' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using the values: 0, 1, 2, 3, 4, 6, or 12.

Data Types: double

Basis — Day-count basis of instrument

0 (default) | numeric values: 0,1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day-count of the instrument, specified as the comma-separated pair consisting of 'Basis' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a supported value:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as the comma-separated pair consisting of 'EndMonthRule' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. This rule applies only when Maturity is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: logical

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond Issue date, specified as the comma-separated pair consisting of 'IssueDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify an IssueDate, the cash flow payment dates are determined from other inputs.

To support existing code, bnddury also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

FirstCouponDate — Irregular or normal first coupon date

datetime array | string array | date character vector

Irregular or normal first coupon date, specified as the comma-separated pair consisting of 'FirstCouponDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify a FirstCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, bnddury also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

LastCouponDate — Irregular or normal last coupon date

datetime array | string array | date character vector

Irregular or normal last coupon date, specified as the comma-separated pair consisting of 'LastCouponDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify a LastCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, bnddury also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

StartDate — Forward starting date of payments

datetime array | string array | date character vector

Forward starting date of payments, specified as the comma-separated pair consisting of 'StartDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors. The StartDate is when a bond actually starts (the date from which a bond cash flow is considered). To make an instrument forward-starting, specify this date as a future date.

If you do not specify a StartDate, the effective start date is the Settle date.

To support existing code, bnddury also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Face — Face value of bond

100 (default) | numeric

Face value of the bond, specified as the comma-separated pair consisting of 'Face' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

Data Types: double

CompoundingFrequency — Compounding frequency for yield calculation

SIA bases uses 2, ICMA bases uses 1 (default) | integer with value of 1, 2, 3, 4, 6, or 12

Compounding frequency for yield calculation, specified as the comma-separated pair consisting of 'CompoundingFrequency' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note By default, SIA bases (0-7) and BUS/252 use a semiannual compounding convention and ICMA bases (8-12) use an annual compounding convention.

Data Types: double

DiscountBasis — Basis used to compute the discount factors for computing the yield

SIA uses 0 (default) | integers of the set [0...13] | vector of integers of the set [0...13]

Basis used to compute the discount factors for computing the yield, specified as the comma-separated pair consisting of 'DiscountBasis' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)

- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If a SIA day-count basis is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the default behavior is for SIA bases to use the actual/actual day count to compute discount factors.

If an ICMA day-count basis or BUS/252 is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the specified bases from the `Basis` input argument are used.

Data Types: double

LastCouponInterest — Compounding convention for computing yield of a bond in last coupon period

compound (default) | values are simple or compound

Compounding convention for computing the yield of a bond in the last coupon period, specified as the comma-separated pair consisting of 'LastCouponInterest' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. LastCouponInterest is based on only the last coupon and the face value to be repaid. Acceptable values are:

- simple
- compound

Data Types: char | cell

Output Arguments

ModDuration — Modified duration in years

numeric

Modified duration in years reported on a semiannual bond basis (in accordance with SIA convention), returned as a NUMBONDS-by-1 vector.

YearDuration — Macaulay duration in years

numeric

Macaulay duration in years, returned as a NUMBONDS-by-1 vector.

PerDuration — Periodic Macaulay duration

numeric

Periodic Macaulay duration reported on a semiannual bond basis (in accordance with SIA convention), returned as a NUMBONDS-by-1 vector.

More About

Bond Duration

Bond duration is a measure of the sensitivity of a bond's price to changes in interest rates.

Bond duration represents the weighted average time until a bond's cash flows (interest and principal payments) are received. Duration is a critical concept in fixed-income investing because it helps investors assess the interest-rate risk associated with a bond or a bond portfolio.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `bnddury` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

R2017b: Support for optional name-value input arguments

Behavior changed in R2017b

The specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the optional name-value pair inputs: `Period`, `Basis`, `EndMonthRule`, `IssueDate`, `FirstCouponDate`, `LastCouponDate`, `StartDate`, `Face`, `CompoundingFrequency`, `DiscountBasis`, and `LastCouponInterest`.

References

- [1] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.
- [2] Mayle, J. "Standard Securities Calculations Methods: Fixed Income Securities Formulas for Analytic Measures." SIA, Vol 2, Jan 1994.
- [3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

`bndconvy` | `bndconvp` | `bndkrdur` | `bnddurp` | `datetime`

Topics

"Bond Portfolio for Hedging Duration and Convexity" on page 10-6
"Yield Conventions" on page 2-21

bndkrdur

Bond key rate duration given zero curve

Syntax

```
KeyRateDuration = bndkrdur(ZeroData,CouponRate,Settle,Maturity)
KeyRateDuration = bndkrdur( ___,Name,Value)
```

Description

KeyRateDuration = bndkrdur(ZeroData,CouponRate,Settle,Maturity) computes the key rate durations for one or more bonds given a zero curve and a set of key rates.

KeyRateDuration = bndkrdur(___,Name,Value) adds optional name-value pair arguments.

Examples

Find the Bond Key Rate Duration Given the Zero Curve

This example shows how to compute the key rate duration of a bond for key rate times of 2, 5, 10, and 30 years.

```
ZeroRates = [0.0476 .0466 .0465 .0468 .0473 .0478 ...
             .0493 .0539 .0572 .0553 .0530]';

ZeroDates = daysadd('31-Dec-1998',[30 360 360*2 360*3 360*5 ...
                                   360*7 360*10 360*15 360*20 360*25 360*30],1);

ZeroData = [ZeroDates ZeroRates];

krdur = bndkrdur(ZeroData,.0525,'12/31/1998',...
                '11/15/2028','KeyRates',[2 5 10 30])

krdur = 1×4

    0.2986    0.8791    4.1353    9.5814
```

Find the Bond Key Rate Duration Given the Zero Curve Using datetime Inputs

This example shows how to use datetime inputs for Settle and Maturity and also use a table for ZeroData to compute the key rate duration of a bond for key rate times of 2, 5, 10, and 30 years.

```
ZeroRates = [0.0476 .0466 .0465 .0468 .0473 .0478 ...
             .0493 .0539 .0572 .0553 .0530]';

ZeroDates = daysadd('31-Dec-1998',[30 360 360*2 360*3 360*5 ...
                                   360*7 360*10 360*15 360*20 360*25 360*30],1);
```

```
ZeroData = table(datetime(ZeroDates,'ConvertFrom','datenum','Locale','en_US'), ZeroRates);  
  
krdur = bndkrdur(ZeroData,.0525,datetime('12/31/1998','Locale','en_US'),...  
datetime('11/15/2028','Locale','en_US'),'KeyRates',[2 5 10 30])  
  
krdur = 1×4  
  
    0.2986    0.8791    4.1353    9.5814
```

Input Arguments

ZeroData — Zero curve

matrix | table

Zero Curve, specified as a numRates-by-2 matrix or a numRates-by-2 table.

If **ZeroData** is represented as a numRates-by-2 matrix, the first column is a MATLAB serial date number and the second column is the accompanying zero rates.

If **ZeroData** is a table, the first column can be a datetime array, string array, or date character vectors. The second column must be numeric data corresponding to the zero rates.

Data Types: double | datetime | char | string | table

CouponRate — Annual percentage rate used to determine coupons payable on a bond

decimal

Annual percentage rate used to determine the coupons payable on a bond, specified as decimal value using a scalar or a NUMBONDS-by-1 vector.

Data Types: double

Settle — Settlement date for all bonds and zero curve

datetime scalar | string scalar | date character vector

Settlement date for all bonds and zero curve, specified as a scalar datetime, string, or date character vector. **Settle** must be the same settlement date for all the bonds and the zero curve.

To support existing code, **bndkrdur** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date for bonds

datetime array | string array | date character vector

Maturity date for bonds, specified as a scalar or a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, **bndkrdur** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `KeyRateDuration =`
`bndkrdur(ZeroData,.0525,'12/31/1998','11/15/2028','KeyRates',[2 5 10 30])`

InterpMethod — Interpolation method used to obtain points from zero curve

'linear' (default) | 'cubic' | 'pchip'

Interpolation method used to obtain points from the zero curve, specified as the comma-separated pair consisting of 'InterpMethod' and a character vector using one of the following values:

- 'linear' (default)
- 'cubic'
- 'pchip'

Data Types: char

ShiftValue — Value that zero curve is shifted up and down to compute duration

.01 (100 basis points) (default) | numeric

Value that zero curve is shifted up and down to compute duration, specified as the comma-separated pair consisting of 'ShiftValue' and a scalar numeric value.

Data Types: double

KeyRates — Rates to perform the duration calculation

set to each of the zero dates (default) | numeric

Rates to perform the duration calculation, specified as the comma-separated pair consisting of 'KeyRates' and a time to maturity using a scalar or a NUMBONDS-by-1 vector.

Data Types: double

CurveCompounding — Compounding frequency of curve

2 (default) | integer with value of 1, 2, 3, 4, 6, or 12

Compounding frequency of the curve, specified as the comma-separated pair consisting of 'CurveCompounding' and a scalar using one of the following values:

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

.

Data Types: double

CurveBasis — Basis of the curve

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Basis of the curve, specified as the comma-separated pair consisting of 'CurveBasis' and a scalar using one of the following values:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

Period — Number of coupon payments per year

2 (default) | numeric with values 0, 1, 2, 3, 4, 6 or 12

Number of coupon payments per year, specified as the comma-separated pair consisting of 'Period' and a scalar or a NUMBONDS-by-1 vector using the values: 0, 1, 2, 3, 4, 6, or 12.

Data Types: double

Basis — Day-count basis of instrument

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day-count of the instrument, specified as the comma-separated pair consisting of 'Basis' and a scalar or a NUMBONDS-by-1 vector using a supported value:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)

- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as the comma-separated pair consisting of 'EndMonthRule' and a scalar or a NUMBONDS-by-1 vector. This rule applies only when Maturity is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: logical

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond Issue date, specified as the comma-separated pair consisting of 'IssueDate' and a scalar or a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors.

If you do not specify an IssueDate, the cash flow payment dates are determined from other inputs.

To support existing code, bndkrdur also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

FirstCouponDate — Irregular or normal first coupon date

datetime array | string array | date character vector

Irregular or normal first coupon date, specified as the comma-separated pair consisting of 'FirstCouponDate' and a scalar or a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors.

If you do not specify a FirstCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, bndkrdur also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

LastCouponDate — Irregular or normal last coupon date

datetime array | string array | date character vector

Irregular or normal last coupon date, specified as the comma-separated pair consisting of 'LastCouponDate' and a scalar or a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors.

If you do not specify a LastCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, bndkrdur also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

StartDate — Forward starting date of payments

datetime array | string array | date character vector

Forward starting date of payments, specified as the comma-separated pair consisting of 'StartDate' and a scalar or a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors. The StartDate is when a bond actually starts (the date from which a bond cash flow is considered). To make an instrument forward-starting, specify this date as a future date.

If you do not specify a StartDate, the effective start date is the Settle date.

To support existing code, bndkrdur also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Face — Face value of bond

100 (default) | numeric

Face value of the bond, specified as the comma-separated pair consisting of 'Face' and a scalar or a NUMBONDS-by-1 vector.

Data Types: double

Output Arguments**KeyRateDuration — Key rate durations**

matrix

Key rate durations, returned as a numBonds-by-numRates matrix.

More About**Key Rate Duration**

Key rate duration is a measure of interest-rate risk that assesses the sensitivity of a bond's price to changes in interest rates at specific maturities along the yield curve.

Unlike traditional duration measures, which provide a single estimate of price sensitivity to parallel shifts in interest rates, key rate duration allows investors to understand how changes in interest rates at different points on the yield curve affect the bond's price.

Algorithms

bndkrdur computes the key rate durations for one or more bonds given a zero curve and a set of key rates. By default, the key rates are each of the zero curve rates. For each key rate, the duration is computed by shifting the zero curve up and down by a specified amount (`ShiftValue`) at that particular key rate, computing the present value of the bond in each case with the new zero curves, and then evaluating the following:

$$krdur_i = \frac{(PV_{down} - PV_{up})}{(PV \times ShiftValue \times 2)}$$

Note The shift to the curve is computed by shifting the particular key rate by the `ShiftValue` and then interpolating the values of the curve in the interval between the previous and next key rates. For the first key rate, any curve values before the date are equal to the `ShiftValue`; likewise, for the last key rate, any curve values after the date are equal to the `ShiftValue`.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although bndkrdur supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] Golub, B., Tilman, L. *Risk Management: Approaches for Fixed Income Markets*. Wiley, 2000.
- [2] Tuckman, B. *Fixed Income Securities: Tools for Today's Markets*. Wiley, 2002.

See Also

bndconvy | bndconvp | bnddury | bnddurp | datetime

Topics

“Bond Portfolio for Hedging Duration and Convexity” on page 10-6
 “Yield Conventions” on page 2-21

bndprice

Price fixed-income security from yield to maturity

Syntax

```
[Price,AccruedInt] = bndprice(Yield,CouponRate,Settle,Maturity)
[Price,AccruedInt] = bndprice( ___,Name,Value)
```

Description

[Price,AccruedInt] = bndprice(Yield,CouponRate,Settle,Maturity) given bonds with SIA date parameters and yields to maturity, returns the clean prices and accrued interest due.

In addition, you can use the Financial Instruments Toolbox object framework with a FixedBond instrument to price a fixed bond.

[Price,AccruedInt] = bndprice(___,Name,Value) adds optional name-value pair arguments.

Examples

Price a Treasury Bond from Yield to Maturity

This example shows how to price a treasury bond at three different yield values.

```
Yield = [0.04; 0.05; 0.06];
CouponRate = 0.05;
Settle = '20-Jan-1997';
Maturity = '15-Jun-2002';
Period = 2;
Basis = 0;
```

```
[Price, AccruedInt] = bndprice(Yield, CouponRate, Settle,...
Maturity, Period, Basis)
```

```
Price = 3×1
```

```
104.8106
99.9951
95.4384
```

```
AccruedInt = 3×1
```

```
0.4945
0.4945
0.4945
```


Price a Treasury Bond from Yield to Maturity Using datetime Inputs

This example shows how to use `datetime` inputs to price a treasury bond at three different yield values.

```
Yield = [0.04; 0.05; 0.06];
CouponRate = 0.05;
Settle = datetime('20-Jan-1997','Locale','en_US');
Maturity = datetime('15-Jun-2002','Locale','en_US');
Period = 2;
Basis = 0;
[Price, AccruedInt] = bndprice(Yield, CouponRate, Settle,...
Maturity, Period, Basis)
```

```
Price = 3×1
```

```
104.8106
99.9951
95.4384
```

```
AccruedInt = 3×1
```

```
0.4945
0.4945
0.4945
```

Price a Treasury Bond with Different Yield Values

This example shows how to price a Treasury bond at two different yield values that include parameter/value pairs for `CompoundingFrequency`, `DiscountBasis`, and `LastCouponPeriodInterest`.

```
bndprice(.04,0.08,datetime(2004,5,25),datetime(2005,4,21),'Period',1,'Basis',8, ...
'LastCouponInterest','simple')
```

```
ans =
103.4743
```

Input Arguments

Yield — Bond yield to maturity

numeric

Bond yield to maturity is specified as a scalar or a `NUMBONDS`-by-1 or 1-by-`NUMBONDS` vector. Yield is on a semiannual basis for `Basis` values 0 through 7 and 13 and an annual basis for `Basis` values 8 through 12.

Data Types: double

CouponRate — Annual percentage rate used to determine coupons payable on a bond

decimal

Annual percentage rate used to determine the coupons payable on a bond, specified as decimal using a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector.

Data Types: `double`

Settle — Settlement date of bond

`datetime array` | `string array` | `date character vector`

Settlement date of the bond, specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime array`, `string array`, or `date character vectors`. The `Settle` date must be before the `Maturity` date.

To support existing code, `bdnprice` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date of bond

`datetime array` | `string array` | `date character vector`

Maturity date of the bond, specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime array`, `string array`, or `date character vectors`.

To support existing code, `bdnprice` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[Price,AccruedInt] = bdnprice(Yield,CouponRate,Settle,Maturity, 'Period',4, 'Basis',9)`

Period — Number of coupon payments per year

2 (default) | numeric with values 0, 1, 2, 3, 4, 6 or 12

Number of coupon payments per year, specified as the comma-separated pair consisting of `'Period'` and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using the values: 0, 1, 2, 3, 4, 6, or 12.

Data Types: `double`

Basis — Day-count basis of instrument

0 (default) | numeric values: 0,1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day-count of the instrument, specified as the comma-separated pair consisting of `'Basis'` and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a supported value:

- 0 = actual/actual
- 1 = 30/360 (SIA)

- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as the comma-separated pair consisting of 'EndMonthRule' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. This rule applies only when Maturity is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: `logical`

IssueDate — Bond issue date

`datetime array` | `string array` | `date character vector`

Bond Issue date, specified as the comma-separated pair consisting of 'IssueDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify an IssueDate, the cash flow payment dates are determined from other inputs.

To support existing code, bndprice also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

FirstCouponDate — Irregular or normal first coupon date

`datetime array` | `string array` | `date character vector`

Irregular or normal first coupon date, specified as the comma-separated pair consisting of 'FirstCouponDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify a `FirstCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `bndprice` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

LastCouponDate — Irregular or normal last coupon date

`datetime array` | `string array` | `date character vector`

Irregular or normal last coupon date, specified as the comma-separated pair consisting of `'LastCouponDate'` and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime array`, `string array`, or `date character vectors`.

If you do not specify a `LastCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `bndprice` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

StartDate — Forward starting date of payments

`datetime array` | `string array` | `date character vector`

Forward starting date of payments, specified as the comma-separated pair consisting of `'StartDate'` and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime array`, `string array`, or `date character vectors`.

If you do not specify a `StartDate`, the effective start date is the `Settle` date.

To support existing code, `bndprice` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Face — Face value of bond

`100 (default)` | `numeric`

Face value of the bond, specified as the comma-separated pair consisting of `'Face'` and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector.

Data Types: `double`

CompoundingFrequency — Compounding frequency for yield calculation

SIA bases uses 2, ICMA bases uses 1 (default) | integer with value of 1, 2, 3, 4, 6, or 12

Compounding frequency for yield calculation, specified as the comma-separated pair consisting of `'CompoundingFrequency'` and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector.

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding

- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note By default, SIA bases (0-7) and BUS/252 use a semiannual compounding convention and ICMA bases (8-12) use an annual compounding convention.

Data Types: double

DiscountBasis — Basis used to compute the discount factors for computing the yield

SIA bases uses 0 (default) | integers of the set [0...13] | vector of integers of the set [0...13]

Basis used to compute the discount factors for computing the yield, specified as the comma-separated pair consisting of 'DiscountBasis' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If a SIA day-count basis is defined in the Basis input argument and there is no value assigned for DiscountBasis, the default behavior is for SIA bases to use the actual/actual day count to compute discount factors.

If an ICMA day-count basis or BUS/252 is defined in the Basis input argument and there is no value assigned for DiscountBasis, the specified bases from the Basis input argument are used.

Data Types: double

LastCouponInterest — Compounding convention for computing yield of a bond in last coupon period

compound (default) | values are simple or compound

Compounding convention for computing the yield of a bond in the last coupon period, specified as the comma-separated pair consisting of 'LastCouponInterest' and a scalar or a NUMBONDS-by-1 or 1-

by-**NUMBONDS** vector. **LastCouponInterest** is based on only the last coupon and the face value to be repaid. Acceptable values are:

- **simple**
- **compound**

Data Types: **char** | **cell**

Output Arguments

Price — Clean price of bond

numeric

Clean price of bond, returned as a **NUMBONDS**-by-1 vector. The dirty price of the bond is the clean price plus the accrued interest. It equals the present value of the bond cash flows of the yield to maturity with semiannual compounding.

AccruedInt — Accrued interest payable at settlement

numeric

accrued interest payable at settlement, returned as a **NUMBONDS**-by-1 vector.

More About

Price and Yield Conventions

The **Price** and **Yield** are related to different formulae for **SIA** and **ICMA** conventions.

For **SIA** conventions, **Price** and **Yield** are related by the formula:

$$\text{Price} + \text{Accrued Interest} = \text{sum}(\text{Cash_Flow} * (1 + \text{Yield}/2)^{(-\text{Time})})$$

where the sum is over the bond's cash flows and corresponding times in units of semiannual coupon periods.

For **ICMA** conventions, the **Price** and **Yield** are related by the formula:

$$\text{Price} + \text{Accrued Interest} = \text{sum}(\text{Cash_Flow} * (1 + \text{Yield})^{(-\text{Time})})$$

Algorithms

For **SIA** conventions, the following formula defines bond price and yield:

$$PV = \sum_{i=1}^n \left(\frac{CF}{(1 + \frac{z}{f})^{TF}} \right),$$

where:

PV =	Present value of a cash flow.
CF =	Cash flow amount.

$z =$	Risk-adjusted annualized rate or yield corresponding to a given cash flow. The yield is quoted on a semiannual basis.
$f =$	Frequency of quotes for the yield. Default is 2 for Basis values 0 to 7 and 13 and 1 for Basis values 8 to 12. The default can be overridden by specifying the CompoundingFrequency name-value pair.
$TF =$	Time factor for a given cash flow. The time factor is computed using the compounding frequency and the discount basis. If these values are not specified, then the defaults are as follows: CompoundingFrequency default is 2 for Basis values 0 to 7 and 13 and 1 for Basis values 8 to 12. DiscountBasis is 0 for Basis values 0 to 7 and 13 and the input Basis for Basis values 8 to 12.

Note The Basis is always used to compute accrued interest.

For ICMA conventions, the frequency of annual coupon payments determines bond price and yield.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although bndprice supports serial date numbers, datetime values are recommended instead. The datetime data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to datetime values, use the datetime function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

R2017b: Support for optional name-value input arguments

Behavior changed in R2017b

The specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the optional name-value pair inputs: Period, Basis, EndMonthRule, IssueDate, FirstCouponDate, LastCouponDate, StartDate, Face, CompoundingFrequency, DiscountBasis, and LastCouponInterest.

References

- [1] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.
- [2] Mayle, J. "Standard Securities Calculations Methods: Fixed Income Securities Formulas for Analytic Measures." SIA, Vol 2, Jan 1994.

[3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

`bndyield` | `cfamounts` | `datetime`

Topics

“Bond Portfolio for Hedging Duration and Convexity” on page 10-6

“Pricing Functions” on page 2-21

“Sensitivity of Bond Prices to Interest Rates” on page 10-2

“Bond Portfolio Optimization Using Portfolio Object” on page 10-30

“Yield Conventions” on page 2-21

bndspread

Static spread over spot curve

Note In R2017b, the specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the optional name-value pair inputs: `Period`, `Basis`, `EndMonthRule`, `IssueDate`, `FirstCouponDate`, `LastCouponDate`, `StartDate`, `Face`, `CompoundingFrequency`, `DiscountBasis`, and `LastCouponInterest`.

Syntax

```
Spread = bndspread(SpotInfo,Price,Coupon,Settle,Maturity)
Spread = bndspread( ___,Name,Value)
```

Description

`Spread = bndspread(SpotInfo,Price,Coupon,Settle,Maturity)` computes the static spread (Z-spread) to benchmark in basis points.

`Spread = bndspread( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Compute the Static Spread Over a Spot Curve Using datetime Inputs

This example shows how to compute a Federal National Mortgage Association (FNMA) 4 3/8 spread over a Treasury spot curve using `datetime` inputs for `Settle` and `Maturity` and a table for `SpotInfo` and plot the results.

```
RefMaturity = [datetime('02/27/2003');
               datetime('05/29/2003');
               datetime('10/31/2004');
               datetime('11/15/2007');
               datetime('11/15/2012');
               datetime('02/15/2031')];
```

```
RefCpn = [0;
          0;
          2.125;
          3;
          4;
          5.375] / 100;
```

```
RefPrices = [99.6964;
             99.3572;
             100.3662;
             99.4511;
             99.4299;
             106.5756];
```

```
RefBonds = [RefPrices, RefMaturity, RefCpn];
Settle = datetime('26-Nov-2002','Locale','en_US');
[ZeroRates, CurveDates] = zbtprice(RefBonds(:, 2:end), ...
RefPrices, Settle)

ZeroRates = 6×1

    0.0121
    0.0127
    0.0194
    0.0317
    0.0423
    0.0550

CurveDates = 6×1 datetime
    27-Feb-2003
    29-May-2003
    31-Oct-2004
    15-Nov-2007
    15-Nov-2012
    15-Feb-2031

% FNMA 4 3/8 maturing 10/06 at 4.30 pm Tuesday
Price = 105.484;
Coupon = 0.04375;
Maturity = datetime('15-Oct-2006','Locale','en_US');

% All optional inputs are accounted by default,
% except the accrued interest under 30/360 (SIA), so:
Period = 2;
Basis = 1;

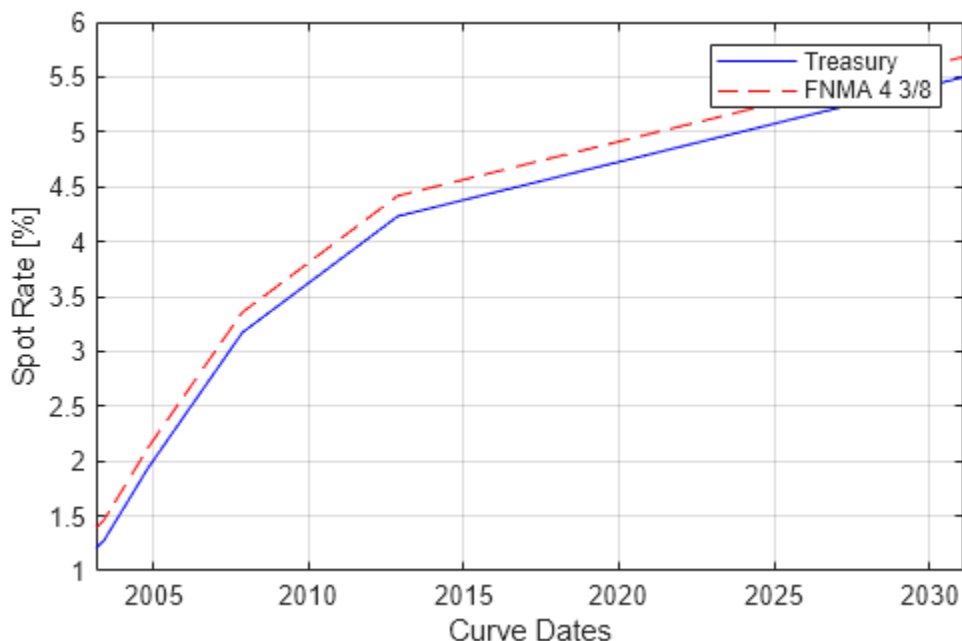
SpotInfo = table(CurveDates, ZeroRates);

% Compute static spread over treasury curve, taking into account
% the shape of curve as derived by bootstrapping method embedded
% within bndspread.

SpreadInBP = bndspread(SpotInfo, Price, Coupon, Settle, ...
Maturity, Period, Basis)

SpreadInBP =
    18.5669

plot(CurveDates, ZeroRates*100, 'b', CurveDates, ...
ZeroRates*100+SpreadInBP/100, 'r--')
legend({'Treasury'; 'FNMA 4 3/8'})
xlabel('Curve Dates')
ylabel('Spot Rate [%]')
grid;
```



Input Arguments

SpotInfo — Spot-rates information

matrix | table | term structure

Spot-rates information, specified as matrix of two columns, an annualized term structure created by `intenvset`, or a table.

- Matrix of two columns— The first column is the `SpotDate`, and the second column, `ZeroRates`, is the zero-rate corresponding to maturities on the `SpotDate`. It is recommended that the spot-rates are spaced as evenly apart as possible, perhaps one that is built from 3-months deposit rates. For example, using the 3-month deposit rates:

```
SpotInfo = ...
[datetime('2-Jan-2004') , 0.03840;
 datetime('2-Jan-2005') , 0.04512;
 datetime('2-Jan-2006') , 0.05086];
```

- Annualized term structure — Refer to `intenvset` to create an annualized term structure. For example:

```
Settle = datetime(2004,1,1);
Rates = [0.03840; 0.04512; 0.05086];
EndDates = [datetime(2004,1,2) datetime('2-Jan-2004'); datetime(2005,1,2); ...
            datetime(2006,1,2)];
SpotInfo = intenvset('StartDates' , Settle , ...
                    'Rates'       , Rates , ...
                    'EndDates'    , EndDates, ...
                    'Compounding' , 2 , ...
                    'Basis'       , 0);
```

- Table — If `SpotInfo` is a table, the first column can be either a datetime array, string array, or date character vector. The second column is numerical data representing zero rates. For example:

```
ZeroRates = [0.012067955808764;0.012730933424479;0.019360902068703;0.031704525214251;0.042306085224510;0.054987415342936];
CurveDates = [731639;731730;732251;733361;735188;741854];
Settle = datetime(2002,11,26);
Price = 105.484;
Coupon = 0.04375;
Maturity = datetime(2006,10,15);
Period = 2;
Basis = 1;
SpotInfo = table(datestr(CurveDates), ZeroRates);
```

Data Types: double | string | char | datetime | table | struct

Price — Clean price for every \$100 notional amount of bonds whose spreads are computed
numeric

Clean price for every \$100 notional amount of bonds whose spreads are computed, specified as numeric value using a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

The clean price is the price of a bond not including accrued interest payments. The clean price is typically the quoted price on financial news sites. This price does not include any interest accrued between the scheduled coupon payments for the bond.

Data Types: double

Coupon — Annual coupon rate of bonds whose spreads are computed
decimal

Annual coupon rate of bonds whose spreads are computed, specified as decimal value using a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

Data Types: double

Settle — Settlement date of bond
datetime array | string array | date character vector

Settlement date of the bond, specified as a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors. The Settle date must be before the Maturity date.

To support existing code, `bndspread` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date of bond
datetime array | string array | date character vector

Maturity date of the bond, specified as a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

To support existing code, `bndspread` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: Spread =

```
bndspread(SpotInfo,Price,Coupon,Settle,Maturity,'Period',4,'Basis',7)
```

Period — Number of coupon payments per year

2 (default) | numeric with values 0, 1, 2, 3, 4, 6 or 12

Number of coupon payments per year, specified as the comma-separated pair consisting of 'Period' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using the values: 0, 1, 2, 3, 4, 6, or 12.

Data Types: double

Basis — Day-count basis of instrument

0 (default) | numeric values: 0,1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day-count of the instrument, specified as the comma-separated pair consisting of 'Basis' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a supported value:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as the comma-separated pair consisting of 'EndMonthRule' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. This rule applies only when Maturity is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: logical

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond Issue date, specified as the comma-separated pair consisting of 'IssueDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify an IssueDate, the cash flow payment dates are determined from other inputs.

To support existing code, `bndspread` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

FirstCouponDate — Irregular or normal first coupon date

datetime array | string array | date character vector

Irregular or normal first coupon date, specified as the comma-separated pair consisting of 'FirstCouponDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify a FirstCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, `bndspread` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

LastCouponDate — Irregular or normal last coupon date

datetime array | string array | date character vector

Irregular or normal last coupon date, specified as the comma-separated pair consisting of 'LastCouponDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify a LastCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, `bndspread` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

StartDate — Forward starting date of payments

datetime array | string array | date character vector

Forward starting date of payments, specified as the comma-separated pair consisting of 'StartDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors. The StartDate is when a bond actually starts (the date from which a bond cash flow is considered). To make an instrument forward-starting, specify this date as a future date.

If you do not specify a StartDate, the effective start date is the Settle date.

To support existing code, `bndspread` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Face — Face value of bond

100 (default) | numeric

Face value of the bond, specified as the comma-separated pair consisting of 'Face' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

Data Types: double

CompoundingFrequency — Compounding frequency for yield calculation

SIA bases uses 2, ICMA bases uses 1 (default) | integer with value of 1, 2, 3, 4, 6, or 12

Compounding frequency for yield calculation, specified as the comma-separated pair consisting of 'CompoundingFrequency' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note By default, SIA bases (0-7) and BUS/252 use a semiannual compounding convention and ICMA bases (8-12) use an annual compounding convention.

Data Types: double

DiscountBasis — Basis used to compute the discount factors for computing the yield

SIA uses 0 (default) | integers of the set [0...13] | vector of integers of the set [0...13]

Basis used to compute the discount factors for computing the yield, specified as the comma-separated pair consisting of 'DiscountBasis' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)

- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If a SIA day-count basis is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the default behavior is for SIA bases to use the actual/actual day count to compute discount factors.

If an ICMA day-count basis or BUS/252 is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the specified bases from the `Basis` input argument are used.

Data Types: `double`

LastCouponInterest — Compounding convention for computing yield of a bond in last coupon period

`compound` (default) | values are `simple` or `compound`

Compounding convention for computing the yield of a bond in the last coupon period, specified as the comma-separated pair consisting of 'LastCouponInterest' and a scalar or a `NUMBONDS`-by-1 or 1-by-`NUMBONDS` vector. `LastCouponInterest` is based on only the last coupon and the face value to be repaid. Acceptable values are:

- `simple`
- `compound`

Data Types: `char` | `cell`

Output Arguments

Spread — Static spread to benchmark in basis points

`numeric`

Static spread to benchmark, returned in basis points as a scalar or a `NUMBONDS`-by-1 vector.

More About

Z-Spread

Z-spread, or static spread, is a measure used in fixed-income analysis to assess the yield spread of a bond over the benchmark yield curve, specifically the risk-free rate, when the bond is assumed to be held to maturity.

The Z-spread represents the constant spread that an investor would add to the yield of a benchmark yield curve (typically the Treasury yield curve) to discount the bond's future cash flows to its current market price.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended*Not recommended starting in R2022b*

Although `bndspread` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

R2017b: Support for optional name-value input arguments*Behavior changed in R2017b*

The specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the optional name-value pair inputs: `Period`, `Basis`, `EndMonthRule`, `IssueDate`, `FirstCouponDate`, `LastCouponDate`, `StartDate`, `Face`, `CompoundingFrequency`, `DiscountBasis`, and `LastCouponInterest`.

References

- [1] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.
- [2] Mayle, J. "Standard Securities Calculations Methods: Fixed Income Securities Formulas for Analytic Measures." SIA, Vol 2, Jan 1994.
- [3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

`bndyield` | `bndprice` | `datetime`

Topics

- "Bond Portfolio for Hedging Duration and Convexity" on page 10-6
- "Pricing Functions" on page 2-21
- "Sensitivity of Bond Prices to Interest Rates" on page 10-2
- "Yield Conventions" on page 2-21

bndtotalreturn

Total return of fixed-coupon bond

Syntax

```
[BondEquiv,EffectiveRate] = bndtotalreturn(Price,CouponRate,Settle,Maturity,ReinvestRate)
[BondEquiv,EffectiveRate] = bndtotalreturn( ___,Name,Value)
```

Description

[BondEquiv,EffectiveRate] = bndtotalreturn(Price,CouponRate,Settle,Maturity,ReinvestRate) calculates the total return for fixed-coupon bonds to maturity or to a specific investment horizon.

[BondEquiv,EffectiveRate] = bndtotalreturn(___,Name,Value) adds optional name-value pair arguments.

Examples

Compute the Total Return of a Fixed-Coupon Bond

Use `bndtotalreturn` to compute the total return for a fixed-coupon bond, given an investment horizon date.

Define fixed-coupon bond.

```
Price = 101;
CouponRate = 0.05;
Settle = '15-Nov-2011';
Maturity = '15-Nov-2031';
ReinvestRate = 0.04;
```

Calculate the total return to maturity.

```
[BondEquiv, EffectiveRate] = bndtotalreturn(Price, CouponRate, ...
Settle, Maturity, ReinvestRate)
```

```
BondEquiv =
0.0460
```

```
EffectiveRate =
0.0466
```

Specify an investment horizon.

```
HorizonDate = '15-Nov-2021';
[BondEquiv, EffectiveRate] = bndtotalreturn(Price, CouponRate, ...
Settle, Maturity, ReinvestRate, 'HorizonDate', HorizonDate)
```

```
BondEquiv =
0.0521
```

```
EffectiveRate =
0.0528
```

Perform scenario analysis on the reinvestment rate.

```
ReinvestRate = [0.03; 0.035; 0.04; 0.045; 0.05];
[BondEquiv, EffectiveRate] = bndtotalreturn(Price, CouponRate, ...
Settle, Maturity, ReinvestRate, 'HorizonDate', HorizonDate)
```

```
BondEquiv = 5×1
```

```
0.0557
0.0538
0.0521
0.0505
0.0490
```

```
EffectiveRate = 5×1
```

```
0.0565
0.0546
0.0528
0.0511
0.0496
```

Compute the Total Return of a Fixed-Coupon Bond Using datetime Inputs

Use `bndtotalreturn` with `datetime` inputs to compute the total return for a fixed-coupon bond, given an investment horizon date.

```
Price = 101;
CouponRate = 0.05;
Settle = datetime('15-Nov-2011','Locale','en_US');
Maturity = datetime('15-Nov-2031','Locale','en_US');
HorizonDate = datetime('15-Nov-2021','Locale','en_US');
ReinvestRate = 0.04;
[BondEquiv, EffectiveRate] = bndtotalreturn(Price, CouponRate, ...
Settle, Maturity, ReinvestRate, 'HorizonDate', HorizonDate)
```

```
BondEquiv =
0.0521
```

```
EffectiveRate =
0.0528
```

Input Arguments

Price — Clean price at settlement date

matrix

Clean price at the settlement date, specified as a scalar or a NINST-by-1 vector.

Data Types: double

CouponRate — Coupon rate

decimal

Coupon rate, specified as a scalar or a NINST-by-1 vector of decimal values.

Data Types: double

Settle — Settlement date of fixed-coupon bond

datetime array | string array | date character vector

Settlement date of the fixed-coupon bond, specified as scalar or a NINST-by-1 vector using a datetime array, string array, or date character vectors. If supplied as a NINST-by-1 vector of dates, settlement dates can be different, as long as they are before the Maturity date and HorizonDate.

To support existing code, `bndtotalreturn` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date of fixed-coupon bond

datetime array | string array | date character vector

Maturity date of the fixed-coupon bond, specified as scalar or a NINST-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `bndtotalreturn` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

ReinvestRate — Reinvestment rate

decimal

Reinvestment rate (the rate earned by reinvesting the coupons), specified as scalar or a NINST-by-2 vector of decimal values.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

```
Example: [BondEquiv,EffectiveRate] =  
bndtotalreturn(Price,CouponRate,Settle,Maturity,ReinvestRate,'HorizonDate','1  
5-Nov-2021')
```

HorizonDate — Investment horizon date

Maturity date (default) | datetime array | string array | date character vector

Investment horizon date, specified as the comma-separated pair consisting of 'HorizonDate' and a scalar or a NINST-by-1 vector using a datetime array, string array, or date character vectors.

If `HorizonDate` is unspecified, the total return is calculated to `Maturity`.

To support existing code, `bndtotalreturn` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

HorizonPrice — Price at investment horizon date

calculated based on `ReinvestRate` (default) | `numeric`

Price at investment horizon date, specified as the comma-separated pair consisting of `'HorizonPrice'` and a scalar or a NINST-by-1 vector.

If `HorizonPrice` is unspecified, the price at the `HorizonDate` is calculated based on the `ReinvestRate`. If the `HorizonDate` equals the `Maturity` date, the `HorizonPrice` is ignored and the total return to maturity is calculated based on the `Face` value.

Data Types: `double`

Period — Number of coupon payments per year

2 (default) | `numeric` with values 0, 1, 2, 3, 4, 6 or 12

Number of coupon payments per year, specified as the comma-separated pair consisting of `'Period'` and a scalar or a NINST-by-1 vector using the values: 0, 1, 2, 3, 4, 6, or 12.

Data Types: `double`

Basis — Day-count basis

0 (actual/actual) (default) | integers of the set [0...13] | vector of integers of the set [0...13]

Day-count basis, specified as the comma-separated pair consisting of `'Basis'` and a scalar or a NINST-by-1 vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as the comma-separated pair consisting of 'EndMonthRule' and a scalar or a NINST-by-1 vector. This rule applies only when Maturity is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: logical

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond Issue date, specified as the comma-separated pair consisting of 'IssueDate' and a scalar or a NINST-by-1 vector using a datetime array, string array, or date character vectors.

If you do not specify an IssueDate, the cash flow payment dates are determined from other inputs.

To support existing code, `bndtotalreturn` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

FirstCouponDate — Irregular or normal first coupon date

datetime array | string array | date character vector

Irregular or normal first coupon date, specified as the comma-separated pair consisting of 'FirstCouponDate' and a scalar or a NINST-by-1 vector using a datetime array, string array, or date character vectors.

If you do not specify a FirstCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, `bndtotalreturn` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

LastCouponDate — Irregular or normal last coupon date

datetime array | string array | date character vector

Irregular or normal last coupon date, specified as the comma-separated pair consisting of 'LastCouponDate' and a scalar or a NINST-by-1 vector using a datetime array, string array, or date character vectors.

If you do not specify a LastCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, `bndtotalreturn` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

StartDate — Forward starting date of payments

datetime array | string array | date character vector

Forward starting date of payments, specified as the comma-separated pair consisting of 'StartDate' and a scalar or a NINST-by-1 vector using a datetime array, string array, or date character vectors.

If you do not specify a StartDate, the effective start date is the Settle date.

To support existing code, bndtotalreturn also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Face — Face value of bond

100 (default) | numeric

Face value of the bond, specified as the comma-separated pair consisting of 'Face' and a scalar or a NINST-by-1 vector.

Data Types: double

CompoundingFrequency — Compounding frequency for yield calculation

integer with value of 1, 2, 3, 4, 6, or 12

Compounding frequency for yield calculation, specified as the comma-separated pair consisting of 'CompoundingFrequency' and a scalar or a NINST-by-1 vector.

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note By default, SIA bases (0-7) and BUS/252 use a semiannual compounding convention and ICMA bases (8-12) use an annual compounding convention.

Data Types: double

DiscountBasis — Basis used to compute the discount factors for computing the yield

integers of the set [0...13] | vector of integers of the set [0...13]

Basis used to compute the discount factors for computing the yield, specified as the comma-separated pair consisting of 'DiscountBasis' and a scalar or a NINST-by-1 vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)

- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note The default behavior is for SIA bases (0-7) to use the `actual/actual` day count to compute discount factors, and for ICMA day counts (8 - 12) and BUS/252 to use the specified `DiscountBasis`.

Data Types: `double`

Output Arguments

BondEquiv — Total return in bond equivalent basis

`numeric`

Total return in bond equivalent basis, returned as a `NUMBONDS`-by-1 vector.

EffectiveRate — Total return in effective rate basis

`numeric`

Total return in effective rate basis, returned as a `NUMBONDS`-by-1 vector.

Version History

Introduced in R2012b

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `bndtotalreturn` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)  
  
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] Fabozzi, Frank J., Mann, Steven V. *Introduction to Fixed Income Analytics: Relative Value Analysis, Risk Measures and Valuation*. John Wiley and Sons, New York, 2010.

See Also

`adjustedClosingPrices` | `bndyield` | `bndprice` | `cfamounts` | `datetime`

Topics

"Bond Portfolio for Hedging Duration and Convexity" on page 10-6

"Pricing Functions" on page 2-21

"Sensitivity of Bond Prices to Interest Rates" on page 10-2

"Yield Conventions" on page 2-21

bndyield

Yield to maturity for fixed-income security

Syntax

```
Yield = bndyield(Price,CouponRate,Settle,Maturity)
Yield = bndyield( ____,Name,Value)
```

Description

Yield = bndyield(Price,CouponRate,Settle,Maturity) given NUMBONDS bonds with SIA date parameters and clean prices (excludes accrued interest), returns the bond equivalent yields to maturity.

Yield = bndyield(____,Name,Value) adds optional name-value arguments.

Examples

Compute Yield to Maturity for a Treasury Bond

This example shows how to compute the yield of a Treasury bond at three different price values.

```
Price = [95; 100; 105];
CouponRate = 0.05;
Settle = '20-Jan-1997';
Maturity = '15-Jun-2002';
Period = 2;
Basis = 0;

Yield = bndyield(Price, CouponRate, Settle,...
Maturity, Period, Basis)

Yield = 3×1

    0.0610
    0.0500
    0.0396
```

Compute Yield to Maturity for a Treasury Bond Using datetime Inputs

This example shows how to use datetime inputs to compute the yield of a Treasury bond at three different price values.

```
Price = [95; 100; 105];
CouponRate = 0.05;
Settle = datetime('20-Jan-1997','Locale','en_US');
Maturity = datetime('15-Jun-2002','Locale','en_US');
```

```
Period = 2;  
Basis = 0;  
Yield = bndyield(Price, CouponRate, Settle,...  
Maturity, Period, Basis)
```

```
Yield = 3×1  
  
    0.0610  
    0.0500  
    0.0396
```

Compute the Yield of a Treasury Bond Using the Same Basis for Discounting and Generating the Cash Flows

Compute the yield of a Treasury bond.

```
Price = [95; 100; 105];  
CouponRate = 0.0345;  
Settle = datetime(2016,5,15);  
Maturity = datetime(2026,2,2);  
Period = 2;  
Basis = 1;  
format long
```

```
Yield = bndyield(Price,CouponRate,Settle,Maturity,Period,Basis)
```

```
Yield = 3×1  
  
    0.040764403932618  
    0.034482347625316  
    0.028554719853118
```

Using the same data, compute the yield of a Treasury bond using the same basis for discounting and generating the cash flows.

```
DiscountBasis = 1;
```

```
Yield = bndyield(Price,CouponRate,Settle,Maturity,'Period',Period,'Basis',Basis, ...  
'DiscountBasis',DiscountBasis)
```

```
Yield = 3×1  
  
    0.040780176658036  
    0.034495592361619  
    0.028565614029497
```

Input Arguments

Price — Clean price of the bond

numeric

Clean price of the bond (current price without accrued interest), specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector.

Data Types: `double`

CouponRate — Annual percentage rate used to determine coupons payable on a bond
`decimal`

Annual percentage rate used to determine the coupons payable on a bond, specified as decimal using a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector.

Data Types: `double`

Settle — Settlement date of bond

`datetime array` | `string array` | `date character vector`

Settlement date of the bond, specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime array`, `string array`, or `date character vectors`. The `Settle` date must be before the `Maturity` date.

To support existing code, `bndyield` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date of bond

`datetime array` | `string array` | `date character vector`

Maturity date of the bond, specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime array`, `string array`, or `date character vectors`.

To support existing code, `bndyield` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `Yield =`

```
bndyield(Price,CouponRate,Settle,Maturity,'Period',4,'Basis',9)
```

Period — Number of coupon payments per year

2 (default) | numeric with values 0, 1, 2, 3, 4, 6 or 12

Number of coupon payments per year, specified as the comma-separated pair consisting of `'Period'` and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using the values: 0, 1, 2, 3, 4, 6, or 12.

Data Types: `double`

Basis — Day-count basis of instrument

0 (default) | numeric values: 0,1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day-count of the instrument, specified as the comma-separated pair consisting of 'Basis' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a supported value:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as the comma-separated pair consisting of 'EndMonthRule' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. This rule applies only when Maturity is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: logical

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond Issue date, specified as the comma-separated pair consisting of 'IssueDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify an IssueDate, the cash flow payment dates are determined from other inputs.

To support existing code, bndyield also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

FirstCouponDate — Irregular or normal first coupon date

datetime array | string array | date character vector

Irregular or normal first coupon date, specified as the comma-separated pair consisting of 'FirstCouponDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify a FirstCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, bndyield also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

LastCouponDate — Irregular or normal last coupon date

datetime array | string array | date character vector

Irregular or normal last coupon date, specified as the comma-separated pair consisting of 'LastCouponDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify a LastCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, bndyield also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

StartDate — Forward starting date of payments

datetime array | string array | date character vector

Forward starting date of payments, specified as the comma-separated pair consisting of 'StartDate' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify a StartDate, the effective start date is the Settle date.

To support existing code, bndyield also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Face — Face value of bond

100 (default) | numeric

Face value of the bond, specified as the comma-separated pair consisting of 'Face' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

Data Types: double

CompoundingFrequency — Compounding frequency for yield calculation

SIA uses 2, ICMA uses 1 (default) | integer with value of 1, 2, 3, 4, 6, or 12

Compounding frequency for yield calculation, specified as the comma-separated pair consisting of 'CompoundingFrequency' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. Values are:

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note By default, SIA bases (0-7) and BUS/252 use a semiannual compounding convention and ICMA bases (8-12) use an annual compounding convention.

Data Types: double

DiscountBasis — Basis used to compute the discount factors for computing the yield

SIA uses 0 (default) | integers of the set [0 . . . 13] | vector of integers of the set [0 . . . 13]

Basis used to compute the discount factors for computing the yield, specified as the comma-separated pair consisting of 'DiscountBasis' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If a SIA day-count basis is defined in the **Basis** input argument and there is no value assigned for **DiscountBasis**, the default behavior is for SIA bases to use the actual/actual day count to compute discount factors.

If an ICMA day-count basis or BUS/252 is defined in the **Basis** input argument and there is no value assigned for **DiscountBasis**, the specified bases from the **Basis** input argument are used.

Data Types: double

LastCouponInterest — Compounding convention for computing yield of a bond in last coupon period

compound (default) | values are simple or compound

Compounding convention for computing the yield of a bond in the last coupon period, specified as the comma-separated pair consisting of 'LastCouponInterest' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector. This is based on only the last coupon and the face value to be repaid. Acceptable values are:

- simple
- compound

Data Types: char | cell

Output Arguments**Yield — Yield to maturity**

numeric

Yield to maturity, returned as a NUMBONDS-by-1 vector.

Note By default, when you specify **Basis**, the SIA bases (0-7) and BUS/252 (base 13) use a semiannual compounding convention and the ISMA bases (8-12) use an annual compounding convention.

More About**Price and Yield Conventions**

The Price and Yield are related to different formulae for SIA and ICMA conventions.

For SIA conventions, Price and Yield are related by the formula:

$$\text{Price} + \text{Accrued Interest} = \text{sum}(\text{Cash_Flow} * (1 + \text{Yield}/2)^{(-\text{Time})})$$

where the sum is over the bond's cash flows and corresponding times in units of semiannual coupon periods.

For ICMA conventions, the Price and Yield are related by the formula:

$$\text{Price} + \text{Accrued Interest} = \text{sum}(\text{Cash_Flow} * (1 + \text{Yield})^{(-\text{Time})})$$

Algorithms

For SIA conventions, the following formula defines bond price and yield:

$$PV = \frac{CF}{(1 + \frac{z}{f})^{TF}},$$

where:

$PV =$	Present value of a cash flow.
$CF =$	The cash flow amount.
$z =$	The risk-adjusted annualized rate or yield corresponding to a given cash flow. The yield is quoted on a semiannual basis.
$f =$	The frequency of quotes for the yield.
$TF =$	Time factor for a given cash flow. Time is measured in semiannual periods from the settlement date to the cash flow date. In computing time factors, use SIA actual/actual day count conventions for all time factor calculations.

For ICMA conventions, the frequency of annual coupon payments determines bond price and yield.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `bndyield` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

R2017b: Support for optional name-value input arguments

Behavior changed in R2017b

The specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the optional name-value pair inputs: `Period`, `Basis`, `EndMonthRule`, `IssueDate`, `FirstCouponDate`, `LastCouponDate`, `StartDate`, `Face`, `CompoundingFrequency`, `DiscountBasis`, and `LastCouponInterest`.

References

- [1] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.
- [2] Mayle, J. "Standard Securities Calculations Methods: Fixed Income Securities Formulas for Analytic Measures." SIA, Vol 2, Jan 1994.
- [3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

`bndprice` | `cfamounts` | `datetime`

Topics

"Bond Portfolio for Hedging Duration and Convexity" on page 10-6

"Pricing Functions" on page 2-21

"Sensitivity of Bond Prices to Interest Rates" on page 10-2

"Yield Conventions" on page 2-21

bollinger

Time series Bollinger band

Syntax

```
[middle,upper,lower] = bollinger(Data)
[middle,upper,lower] = bollinger( ___,Name,Value)
```

Description

`[middle,upper,lower] = bollinger(Data)` calculates the middle, upper, and lower bands that make up the Bollinger bands from a series of data. A Bollinger band chart plots actual asset data along with three other bands of data: the upper band that is two standard deviations above a user-specified moving average; the lower band that is two standard deviations below that moving average; and the middle band that is the moving average itself.

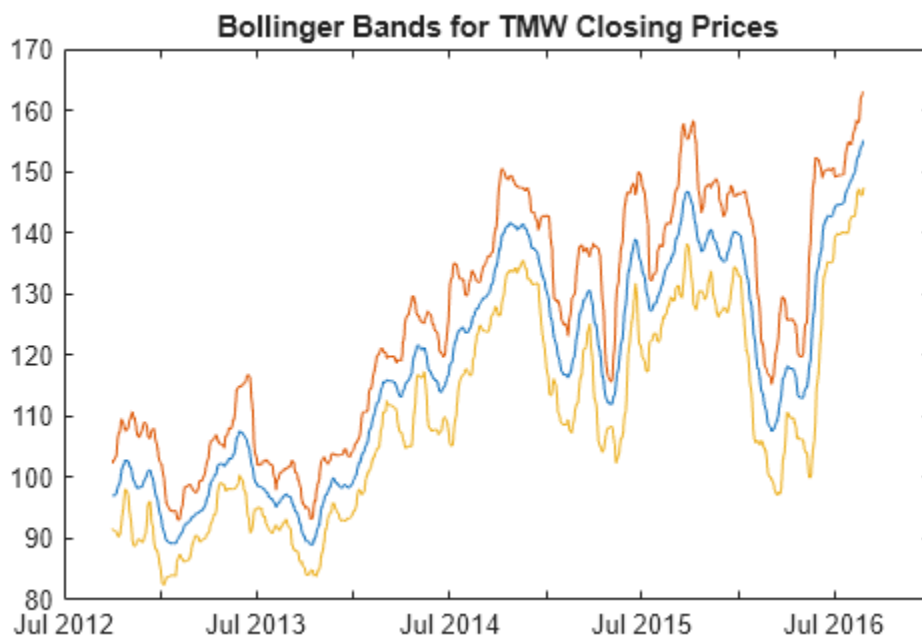
`[middle,upper,lower] = bollinger( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Calculate the Bollinger Bands for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat
[middle,upper,lower]= bollinger(TMW);
CloseBolling = [middle.Close, upper.Close,...
lower.Close];
plot(middle.Time,CloseBolling)
title('Bollinger Bands for TMW Closing Prices')
```



Input Arguments

Data — Data for market prices

matrix | table | timetable

Data for market prices, specified as a matrix, table, or timetable. For matrix input, Data must be column-oriented.

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where Name is the argument name and Value is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[middle,upper,lower] = bollinger(TMW_CLOSE,'WindowSize',10,'Type',1)`

WindowSize — Number of observations of input series to include in moving average in periods

10 (default) | positive integer

Number of observations of the input series to include in the moving average in periods, specified as the comma-separated pair consisting of 'WindowSize' and a scalar positive integer.

Data Types: double

Type — Type of moving average to compute

0 (simple) (default) | integer with value 0 or 1

Type of moving average to compute, specified as the comma-separated pair consisting of 'Type' and a scalar integer with a value of 0 (simple) or 1 (linear).

Data Types: double

NumStd — Number of standard deviations for the upper and lower bounds

2 (default) | positive integer

Number of standard deviations for the upper and lower bounds, specified as the comma-separated pair consisting of 'NumStd' and a scalar positive integer.

Data Types: double

Output Arguments

middle — Moving average series representing the middle band

matrix | table | timetable

Moving average series representing the middle band, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input Data.

upper — Moving average series representing the upper band

matrix | table | timetable

Moving average series representing the upper band, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input Data.

lower — Moving average series representing the lower band

matrix | table | timetable

Moving average series representing the lower band, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input Data.

More About

Bollinger Band

The Bollinger band is a technical analysis indicator used to measure market volatility and identify potential overbought or oversold conditions in a time series of price data.

The Bollinger bands consist of a middle band and two outer bands that expand and contract based on market volatility. Widening bands indicate increased volatility and potential price swings and narrowing bands suggest decreased volatility and potential price consolidation.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the Data input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 72–74.

See Also

`timetable` | `table` | `movavg`

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

boxcox

Box-Cox transformation

Syntax

```
[transdat,lambda] = boxcox(data)
transdat = boxcox(lambda,data)
```

Description

`[transdat,lambda] = boxcox(data)` transforms the data vector `data` using the Box-Cox transformation method into `transdat`. It also estimates the transformation parameter λ . For more information, see “Box Cox Transformation” on page 15-599.

`transdat = boxcox(lambda,data)` transform the `data` using a certain specified λ for the Box-Cox transformation. This syntax does not find the optimum λ that maximizes the LLF. For more information, see “Box Cox Transformation” on page 15-599.

Examples

Transform a Data Series Contained in Vector of Data

Use `boxcox` to transform the data series contained in a vector of data into another set of data series with relatively normal distributions.

Load the `SimulatedStock.mat` data file.

```
load SimulatedStock.mat
```

Transform the nonnormally distributed filled data series `TMW_CLOSE` into a normally distributed one using Box-Cox transformation.

```
[Xbc, lambdabc] = boxcox(TMW_CLOSE)
```

```
Xbc = 1000×1
```

```
7.8756
7.8805
7.9173
7.8557
7.8245
7.7844
7.7811
7.8029
7.8015
7.7229
7.6870
7.6825
7.7447
7.6958
7.6596
```

```

:
lambdabc =
0.2151

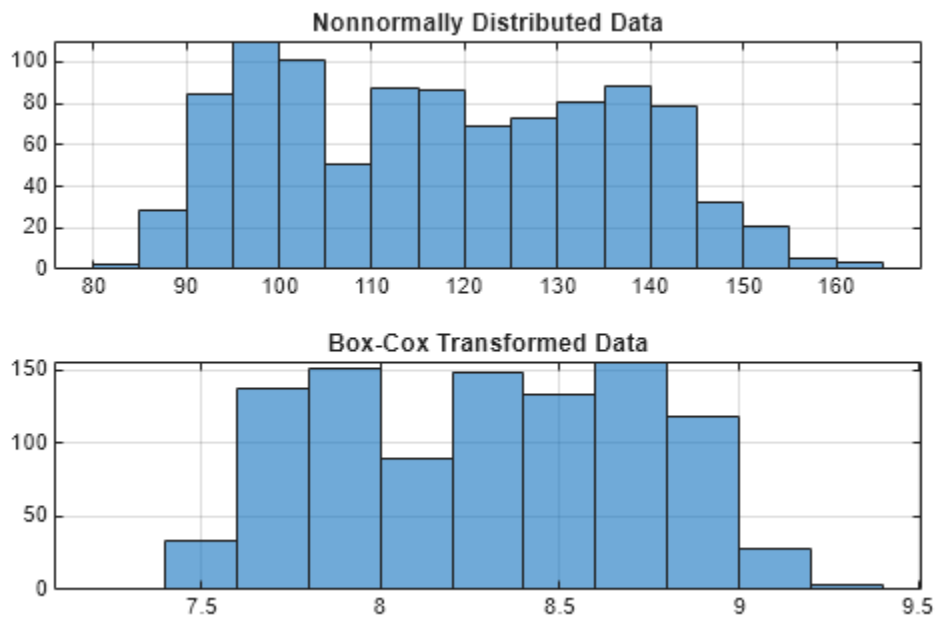
```

Compare the result of the TMW_CLOSE data series with a normal (Gaussian) probability distribution function and the nonnormally distributed TMW_CLOSE.

```

subplot(2, 1, 1);
histogram(TMW_CLOSE);
grid; title('Nonnormally Distributed Data');
subplot(2, 1, 2);
histogram(Xbc);
grid; title('Box-Cox Transformed Data');

```



The bar chart on the top represents the probability distribution function of the data series, TMW_CLOSE, which is the original data series. The distribution is skewed toward the left (not normally distributed). The bar chart on the bottom is less skewed to the left. If you plot a Gaussian probability distribution function (PDF) with similar mean and standard deviation, the distribution of the transformed data is close to normal (Gaussian). When you examine the contents of the resulting object Xbc, you find an identical object to the original object TMW_CLOSE but the contents are the transformed data series.

Input Arguments

data — Data

positive column vector

Data, specified as a positive column vector.

Data Types: double

lambda — Lambda

numeric | structure

Lambda, specified as a scalar numeric or structure.

If the input `data` is a vector, `lambda` is a scalar. If the input is a financial time series object (`tsobj`), `lambda` is a structure with fields similar to the components of the object. For example, if `tsobj` contains series names `Open` and `Close`, `lambda` has fields `lambda.Open` and `lambda.Close`.

Data Types: `double` | `struct`

Output Arguments**transdat — Data Box-Cox transformation**

vector

Data Box-Cox transformation, returned as a vector.

lambda — Lambda transformation parameter

numeric

Lambda transformation parameter, returned as a numeric.

More About**Box Cox Transformation**

`boxcox` transforms nonnormally distributed data to a set of data that has approximately normal distribution. The Box-Cox transformation is a family of power transformations.

If λ is not $= 0$, then

$$data(\lambda) = \frac{data^\lambda - 1}{\lambda}$$

If λ is $= 0$, then

$$data(\lambda) = \log(data)$$

The logarithm is the natural logarithm (log base e). The algorithm calls for finding the λ value that maximizes the Log-Likelihood Function (LLF). The search is conducted using `fminsearch`.

Version History**Introduced before R2006a****R2023a: `fints` support for `tsobj` input argument and `transfts` output argument are removed***Behavior changed in R2023a*

The `tsobj` input argument that supports a `fints` object is and the `transfts` output argument are removed from `boxcox`.

See Also

fminsearch

busdate

Next or previous business day

Syntax

```
Busday = busdate(Date)
Busday = busdate( ___, DirFlag, Holiday, Weekend)
```

Description

`Busday = busdate(Date)` returns the scalar, vector, or matrix of the next or previous business days, depending on the definition for `Holiday`.

`Busday = busdate( ___, DirFlag, Holiday, Weekend)` returns the scalar, vector, or matrix of the next or previous business days, depending on the optional input arguments, including `Holiday`.

If both `Date` and `Holiday` are either strings or date character vectors, `Busday` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

However, if either `Date` or `Holiday` are datetime arrays, `Busday` is returned as a datetime array.

Examples

Determine Business Days

Determine the next business day when `Date` is a character vector.

```
Busday = busdate('3-Jul-2001', 1)

Busday =
731037

datestr(Busday)

ans =
'05-Jul-2001'
```

Indicate that Saturday is a business day by appropriately setting the `Weekend` argument. July 4, 2003 falls on a Friday. Use `busdate` to verify that Saturday, July 5, is actually a business day.

```
Weekend = [1 0 0 0 0 0 0];
Date = datestr(busdate('3-Jul-2003', 1, [], Weekend))

Date =
'05-Jul-2003'
```

If either `Date` or `Holiday` are datetime arrays, `Busday` is returned as a datetime array.

```
Busday = busdate(datetime('3-Jul-2001', 'Locale', 'en_US'), 1)
```

```
Busday = datetime
05-Jul-2001
```

Also, you can bypass a holiday.

```
busdate(datetime(2022,11,23),1,NaT)
```

```
ans = datetime
24-Nov-2022
```

Input Arguments

Date — Reference business date

`datetime array | string array | date character vector`

Reference business date, specified as a scalar, vector, or matrix using a `datetime` array, string array, or date character vectors.

To support existing code, `busdate` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

DirFlag — Business day convention

`follow` (default) | character vector with values of `follow`, `modifiedfollow`, `previous`, or `modifiedprevious` | cell array of character vectors with values of `follow`, `modifiedfollow`, `previous`, or `modifiedprevious`

Business day convention, specified date character vector or cell array of date character vectors with values of `follow`, `modifiedfollow`, `previous`, or `modifiedprevious`.

Also, `DirFlag` can be a scalar, vector, or matrix of search directions, where Next is `DIREC = 1` (default) or Previous is `DIREC = -1`.

Data Types: `char` | `string`

Holiday — Holidays and nontrading-day dates

`non-trading day vector is determined by the routine holidays` (default) | `datetime array | string array | date character vector`

Holidays and nontrading-day dates, specified as vector using a `datetime` array, string array, or date character vectors.

All dates in `Holiday` must be the same format: either datetimes, strings, or date character vectors.

Note To bypass holidays and nontrading dates, use `NaT`. For example, `busdate(datetime(2022,11,23),1,NaT)`.

To support existing code, `busdate` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Weekend — Weekend days

[1 0 0 0 0 0 1] (Saturday and Sunday form the weekend) (default) | vector of length 7, containing 0 and 1, where 1 indicates weekend days

Weekend days, specified as a vector of length 7, containing 0 and 1, where 1 indicates weekend days and the first element of this vector corresponds to Sunday.

Data Types: double

Output Arguments**Busday — Next or previous business day**

scalar | vector | matrix

Next or previous business day, returned as a scalar, vector, or matrix depending on the definition for Holiday.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although busdate supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datetime");
y = year(t)

y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

holidays | datetime

Topics

“Handle and Convert Dates” on page 2-2

“Trading Calendars User Interface” on page 12-2

busdays

Business days for given period

Syntax

```
bdates = busdays(sdate,edate)
bdates = busdays( ___,bdmode,Holiday)
```

Description

`bdates = busdays(sdate,edate)` generates a vector of business days between the last business date of the period that contains the start date (`sdate`), and the last business date of period that contains the end date (`edate`).

`bdates = busdays( ___,bdmode,Holiday)` generates a vector of business days between the last business date of the period that contains the start date (`sdate`), and the last business date of period that contains the end date (`edate`) using optional input arguments. If `Holiday` is not supplied, the dates are generated based on United States holidays. If you do not supply `bdmode`, `bdates` generates a daily vector.

Examples

Determine Business Days for a Given Period

Determine the business days for a weekly period.

```
bdates = datestr(busdays('1/2/01','1/9/01','weekly'))
bdates = 2×11 char array
    '05-Jan-2001'
    '12-Jan-2001'
```

The end of the week is considered to be a Friday. Between 1/2/01 (Monday) and 1/9/01 (Tuesday), there is only one end-of-week day, 1/5/01 (Friday). Because 1/9/01 is part of the following week, the following Friday (1/12/01) is also reported.

Determine the business days for a weekly period using a datetime input for `sdate`.

```
bdates = busdays(datetime('2-Jan-2001','Locale','en_US'),'9-Jan-2001','weekly')
bdates = 2×1 datetime
    05-Jan-2001
    12-Jan-2001
```

Determine the business days for a monthly period.

```
vec = datestr(busdays('1/8/16','3/1/16','monthly'))
vec = 3×11 char array
    '29-Jan-2016'
```

```
'29-Feb-2016'
'31-Mar-2016'
```

The start date (1/8/16) is in the month of January, 2016. The last business day for the month of January is 1/29/16 (Friday). The end date (3/1/16) is in the month of March, 2016. The last business day for the month of March is 3/31/16 (Thursday). The month of February, 2016 lies between the start date and the end date. The last business day for the month of February is 2/29/16 (Monday).

Input Arguments

sdate — Start date

datetime scalar | string scalar | date character vector

Start date, specified as a scalar datetime, string, or date character vector.

To support existing code, `busdays` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

edate — End date

datetime scalar | string scalar | date character vector

End date, specified as a scalar datetime, string, or date character vector.

To support existing code, `busdays` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

bdmode — Frequency of business days

DAILY (1) (default) | nonnegative numeric with values 1 through 5 | date character vector with values DAILY, WEEKLY, MONTHLY, QUARTERLY, SEMIANNUAL or ANNUAL

Frequency of business days, specified as a nonnegative numeric with values 1 through 5 or date character vector with values of DAILY, WEEKLY, MONTHLY, QUARTERLY, SEMIANNUAL, or ANNUAL

Valid periodicities include:

- DAILY, Daily, daily, D, d, 1 (default)
- WEEKLY, Weekly, weekly, W, w, 2
- MONTHLY, Monthly, monthly, M, m, 3
- QUARTERLY, Quarterly, quarterly, Q, q, 4
- SEMIANNUAL, Semiannual, semiannual, S, s, 5
- ANNUAL, Annual, annual, A, a, 6

Character vectors must be enclosed in single quotation marks.

For example, if `bdmode` is set to `monthly`, `busdays` returns end-of-month business dates for all full or partial months between the start date and end date inclusive.

Data Types: `double` | `char`

Holiday — Holiday and nontrading-day dates

if `Holiday` is [] holiday dates are based on United States holidays specified by `holidays` (default)
| datetime array | string array | date character vector

Holiday and nontrading-day dates, specified as a vector using a datetime array, string array, or date character vectors. If you specify `Holiday`, you must also supply the frequency `bdmode`. Using a `Holiday` value of `NaN` uses a holiday list that has no dates.

To support existing code, `busdays` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Output Arguments**bdates — Business days**

column vector

Business days, returned as a column vector of business dates, in datetime format (if `sdate`, `edate`, or `Holiday` are in datetime format). Business dates can exist before and/or after the specified `sdate` and `edate`.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `busdays` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`isbusday` | `holidays` | `datetime`

Topics

“Handle and Convert Dates” on page 2-2

“Trading Calendars User Interface” on page 12-2

candle

Candlestick chart

Syntax

```
candle(Data)
candle(Data,Color)
h = candle(ax____)
```

Description

`candle(Data)` plots a candlestick chart from a series of opening, high, low, and closing prices of a security. If the closing price is greater than the opening price, the body (the region between the open and close price) is unfilled; otherwise the body is filled.

`candle(Data,Color)` adds an optional argument for `Color`.

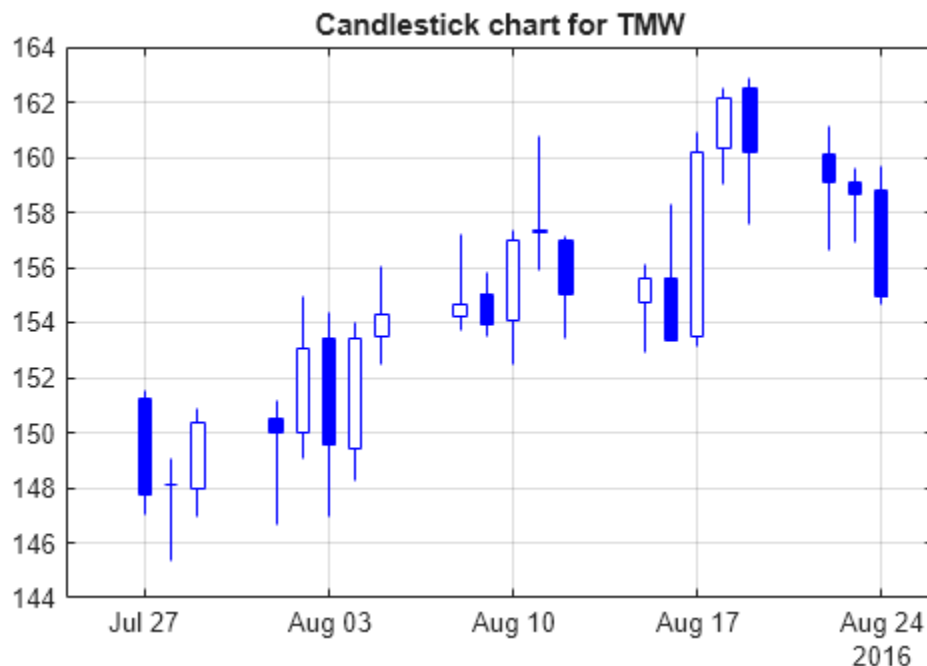
`h = candle(ax____)` adds an optional argument for `ax`.

Examples

Generate a Candlestick Chart for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock. This is a candlestick chart with blue candles, for the most recent 21 days in `SimulatedStock.mat`.

```
load SimulatedStock.mat;
candle(TMW(end-20:end,:), 'b');
title('Candlestick chart for TMW')
```



Input Arguments

Data — Data for opening, high, low, and closing prices

matrix | table | timetable

Data for opening, high, low, and closing prices, specified as a matrix, table, or timetable. For matrix input, **Data** is an M-by-4 matrix of opening, high, low, and closing prices stored in the corresponding columns. Timetables and tables with M rows must contain variables named 'Open', 'High', 'Low', and 'Close' (case insensitive).

Data Types: double | table | timetable

Color — (Optional) Three element color vector

background color of figure window (default) | color vector [R G B] | string

Three element color vector, specified as a [R G B] color vector or a string specifying the color name. The default color differs depending on the background color of the figure window.

RGB triplets and hexadecimal color codes are useful for specifying custom colors.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range [0, 1]; for example, [0.4 0.6 0.7].
- A hexadecimal color code is a character vector or a string scalar that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Thus, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and hexadecimal color codes.

Color Name	Short Name	RGB Triplet	Hexadecimal Color Code	Appearance
"red"	"r"	[1 0 0]	"#FF0000"	
"green"	"g"	[0 1 0]	"#00FF00"	
"blue"	"b"	[0 0 1]	"#0000FF"	
"cyan"	"c"	[0 1 1]	"#00FFFF"	
"magenta"	"m"	[1 0 1]	"#FF00FF"	
"yellow"	"y"	[1 1 0]	"#FFFF00"	
"black"	"k"	[0 0 0]	"#000000"	
"white"	"w"	[1 1 1]	"#FFFFFF"	

This table lists the default color palettes for plots in the light and dark themes.

Palette	Palette Colors
"gem" — Light theme default Most plots use these colors by default.	
"glow" — Dark theme default	

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Data Types: `double` | `string`

ax — Valid axis object

current axes (`ax = gca`) (default) | axes object

(Optional) Valid axis object, specified as an axes object. The candle plot is created in the axes specified by `ax` instead of in the current axes (`ax = gca`). The option `ax` can precede any of the input argument combinations.

Data Types: `object`

Output Arguments

h — Graphic handle of the figure

handle object

Graphic handle of the figure, returned as a handle object.

More About

Candlestick Chart

A candlestick chart is a financial chart used to represent the price movements of an asset over a specific time period.

The candlestick chart provides more information than traditional line charts by displaying the open, high, low, and close prices (OHLC) for each time period in a single "candlestick."

Version History

Introduced before R2006a

R2022b: Support for negative price data

Behavior changed in R2022b

The `Data` input accepts negative prices.

R2018a: Support for matrix, time, or timetable input for data

Behavior changed in R2018a

`candle` accepts `Data` input as a matrix, timetable, or table.

See Also

`timetable` | `table` | `highlow` | `movavg` | `pointfig` | `kagi` | `linebreak` | `priceandvol` | `renko` | `volarea`

Topics

"Use Timetables in Finance" on page 11-7

"Convert Financial Time Series Objects (fints) to Timetables" on page 11-2

cdai

Accrued interest on certificate of deposit

Syntax

```
AccrInt = cdai(CouponRate,Settle,Maturity,IssueDate)
AccrInt = cdai( ____,Basis)
```

Description

`AccrInt = cdai(CouponRate,Settle,Maturity,IssueDate)` computes the accrued interest on a certificate of deposit.

`cdai` assumes that the certificates of deposit pay interest at maturity. Because of the simple interest treatment of these securities, this function is best used for short-term maturities (less than 1 year). The default simple interest calculation uses the `Basis` for the actual/360 convention (2).

`AccrInt = cdai( ____,Basis)` adds an optional argument for `Basis`.

Examples

Find the Accrued Interest on a Certificate of Deposit

This example shows how to compute the accrued interest due, given a certificate of deposit with the following characteristics.

```
CouponRate    = 0.05;
Settle         = '02-Jan-02';
Maturity       = '31-Mar-02';
IssueDate      = '1-Oct-01';
```

```
AccrInt = cdai(CouponRate, Settle, Maturity, IssueDate)
```

```
AccrInt =
1.2917
```

Find the Accrued Interest on a Certificate of Deposit Using datetime Inputs

This example shows how to use `datetime` inputs to compute the accrued interest due, given a certificate of deposit with the following characteristics.

```
CouponRate = 0.05;
Settle = datetime('02-Jan-02','Locale','en_US');
Maturity = datetime('31-Mar-02','Locale','en_US');
IssueDate = datetime('1-Oct-01','Locale','en_US');
AccrInt = cdai(CouponRate, Settle, Maturity, IssueDate)
```

```
AccrInt =
1.2917
```

Input Arguments

CouponRate — Annual interest rate

decimal

Annual interest rate, specified as decimal using a scalar or a NCDS-by-1 or 1-by-NCDS vector.

Data Types: `double`

Settle — Settlement date for certificate of deposit

datetime array | string array | date character vector

Settlement date for the certificate of deposit, specified as a scalar or a NCDS-by-1 or 1-by-NCDS vector using a datetime array, string array, or date character vectors. The `Settle` date must be before the `Maturity` date.

To support existing code, `cdai` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date for certificate of deposit

datetime array | string array | date character vector

Maturity date for the certificate of deposit, specified as a scalar or a NCDS-by-1 or 1-by-NCDS vector using a datetime array, string array, or date character vectors.

To support existing code, `cdai` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

IssueDate — Issue date for certificate of deposit

datetime array | string array | date character vector

Issue date for the certificate of deposit, specified as a scalar or a NCDS-by-1 or 1-by-NCDS vector using a datetime array, string array, or date character vectors.

To support existing code, `cdai` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Basis — Day-count basis for certificate of deposit

2 (actual/360) (default) | integers of the set `[0...13]` | vector of integers of the set `[0...13]`

(Optional) Day-count basis for the certificate of deposit, specified as a scalar or a NINST-by-1 vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365

- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

Output Arguments

AccrInt — Accrued interest per \$100 of face value

`numeric`

Accrued interest per \$100 of face value, returned as a NCDS-by-1 or 1-by-NCDS vector.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cdai` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`accrfrac` | `bdnyield` | `stepcpnyield` | `tbillyield` | `cdprice` | `cdyield` | `zeroyield` | `datetime`

Topics

“Coupon Date Calculations” on page 2-20

“Yield Conventions” on page 2-21

cdprice

Price of certificate of deposit

Syntax

```
[Price, AccrInt] = cdprice(Yield, CouponRate, Settle, Maturity, IssueDate)
[PriceAccrInt] = cdprice( ___, Basis)
```

Description

[Price, AccrInt] = cdprice(Yield, CouponRate, Settle, Maturity, IssueDate) computes the price of a certificate of deposit given its yield.

cdprice assumes that the certificates of deposit pay interest at maturity. Because of the simple interest treatment of these securities, this function is best used for short-term maturities (less than 1 year). The default simple interest calculation uses the Basis for the actual/360 convention (2).

[PriceAccrInt] = cdprice(___, Basis) adds an optional argument for Basis.

Examples

Compute the Price and Accrued Interest for a Certificate of Deposit

This example shows how to compute the price and the accrued interest due on the settlement date, given a certificate of deposit with the following characteristics.

```
Yield          = 0.0525;
CouponRate     = 0.05;
Settle         = '02-Jan-02';
Maturity       = '31-Mar-02';
IssueDate      = '1-Oct-01';
```

```
[Price, AccruedInt] = cdprice(Yield, CouponRate, Settle, ...
Maturity, IssueDate)
```

```
Price =
99.9233
```

```
AccruedInt =
1.2917
```

Compute the Price and Accrued Interest for a Certificate of Deposit Using datetime Inputs

This example shows how to use datetime inputs to compute the price and the accrued interest due on the settlement date, given a certificate of deposit with the following characteristics.

```
Yield = 0.0525;
CouponRate = 0.05;
```

```
Settle = datetime('02-Jan-02','Locale','en_US');
Maturity = datetime('31-Mar-02','Locale','en_US');
IssueDate = datetime('1-Oct-01','Locale','en_US');

[Price, AccruedInt] = cdprice(Yield, CouponRate, Settle, ...
Maturity, IssueDate)

Price =
99.9233

AccruedInt =
1.2917
```

Input Arguments

Yield — Simple yield to maturity over basis denominator

numeric

Simple yield to maturity over the basis denominator, specified as a numeric value using a scalar or a NCDS-by-1 or 1-by-NCDS vector.

Data Types: double

CouponRate — Coupon annual interest rate

decimal

Coupon annual interest rate, specified as decimal using a scalar or a NCDS-by-1 or 1-by-NCDS vector.

Data Types: double

Settle — Settlement date for certificate of deposit

datetime array | string array | date character vector

Settlement date for the certificate of deposit, specified as a scalar or a NCDS-by-1 or 1-by-NCDS vector using a datetime array, string array, or date character vectors. The **Settle** date must be before the **Maturity** date.

To support existing code, **cdprice** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date for certificate of deposit

datetime array | string array | date character vector

Maturity date for the certificate of deposit, specified as a scalar or a NCDS-by-1 or 1-by-NCDS vector using a datetime array, string array, or date character vectors.

To support existing code, **cdprice** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

IssueDate — Issue date for certificate of deposit

datetime array | string array | date character vector

Issue date for the certificate of deposit, specified as a scalar or a NCDS-by-1 or 1-by-NCDS vector using a datetime array, string array, or date character vectors.

To support existing code, `cdprice` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Basis — Day-count basis for certificate of deposit

2 (actual/360) (default) | integers of the set [0...13] | vector of integers of the set [0...13]

(Optional) Day-count basis for the certificate of deposit, specified as a scalar or a NINST-by-1 vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

Output Arguments

Price — Clean price of certificate of deposit per \$100

`numeric`

Clean price of the certificate of deposit per \$100, returned as a NCDS-by-1 or 1-by-NCDS vector.

AccrInt — Accrued interest payable at settlement per unit of face value

`numeric`

Accrued interest payable at settlement per unit of face value, returned as a NCDS-by-1 or 1-by-NCDS vector.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cdprice` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`bndprice` | `cdai` | `cdyield` | `stepcpnprice` | `tbillprice` | `datetime`

Topics

"Coupon Date Calculations" on page 2-20

"Yield Conventions" on page 2-21

cdsbootstrap

Bootstrap default probability curve from credit default swap market quotes

Syntax

```
[ProbData,HazData] = cdsbootstrap(ZeroData,MarketData,Settle)
[ProbData,HazData] = cdsbootstrap( ___,Name,Value)
```

Description

[ProbData,HazData] = cdsbootstrap(ZeroData,MarketData,Settle) bootstraps the default probability curve using credit default swap (CDS) market quotes. The market quotes can be expressed as a list of maturity dates and corresponding CDS market spreads, or as a list of maturities and corresponding upfronts and standard spreads for standard CDS contracts. The estimation uses the standard model of the survival probability.

Note Alternatively, you can use the Financial Instruments Toolbox `defprobstrip` function to bootstrap a `defprobcurve` object from market CDS instruments. For more information, see “Get Started with Workflows Using Object-Based Framework for Pricing Financial Instruments” (Financial Instruments Toolbox).

[ProbData,HazData] = cdsbootstrap(___,Name,Value) adds optional name-value pair arguments.

Examples

Bootstrap Default Probability Curve from Credit Default Swap Market Quotes

This example shows how to use `cdsbootstrap` with market quotes for CDS contracts to generate `ProbData` and `HazData` values.

```
Settle = datetime(2009,7,17); % valuation date for the CDS
Spread_Time = [1 2 3 5 7]';
Spread = [140 175 210 265 310]';
Market_Dates = daysadd(datenum(Settle),360*Spread_Time,1);
MarketData = [Market_Dates Spread];
Zero_Time = [.5 1 2 3 4 5]';
Zero_Rate = [1.35 1.43 1.9 2.47 2.936 3.311]'/100;
Zero_Dates = daysadd(datenum(Settle),360*Zero_Time,1);
ZeroData = [Zero_Dates Zero_Rate];

format long
[ProbData,HazData] = cdsbootstrap(ZeroData,MarketData,Settle)

ProbData = 5×2
105 ×

    7.343360000000000    0.000000233427859
```

```
7.3470100000000000 0.000000575839968
7.3506700000000000 0.000001021397017
7.3579700000000000 0.000002064539982
7.3652800000000000 0.000003234110940
```

```
HazData = 5×2
105 ×
```

```
7.3433600000000000 0.000000232959886
7.3470100000000000 0.000000352000512
7.3506700000000000 0.000000476383354
7.3579700000000000 0.000000609055766
7.3652800000000000 0.000000785241515
```

Input Arguments

ZeroData — Zero rate data

vector | IRDataCurve object

Zero rate data, specified as a M-by-2 vector of dates, using a serial date number format, and zero rates or an IRDataCurve object of zero rates.

When ZeroData is an IRDataCurve object, ZeroCompounding and ZeroBasis are implicit in ZeroData and are redundant inside this function. In this case, specify these optional parameters when constructing the IRDataCurve object before using the `cdsbootstrap` function.

For more information on an IRDataCurve object, see “Creating an IRDataCurve Object” (Financial Instruments Toolbox).

Data Types: double | object

MarketData — Bond market data

matrix

Bond market data, specified as a N-by-2 matrix of dates and corresponding market spreads or N-by-3 matrix of dates, upfronts, and standard spreads of CDS contracts. The dates must be entered as serial date numbers, upfronts must be numeric values between 0 and 1, and spreads must be in basis points.

Data Types: double

Settle — Settlement date

datetime scalar | string scalar | date character vector

Settlement date, specified as a scalar datetime, string, or date character vector. The Settle date must be earlier than or equal to the dates in MarketData

To support existing code, `cdsbootstrap` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Note Any optional input of size N-by-1 is also acceptable as an array of size 1-by-N, or as a single value applicable to all contracts. Single values are internally expanded to an array of size N-by-1.

Example: `[ProbData,HazData] = cdsbootstrap(ZeroData,MarketData,Settle,'RecoveryRate',Recovery,'ZeroCompounding',-1)`

RecoveryRate — Recovery rate

0.4 (default) | decimal

Recovery rate, specified as the comma-separated pair consisting of `'RecoveryRate'` and a N-by-1 vector of recovery rates, specified as a decimal from 0 to 1.

Data Types: double

Period — Premium payment frequency

4 (default) | numeric with values 1, 2, 3, 4, 6 or 12

Premium payment frequency, specified as the comma-separated pair consisting of `'Period'` and a N-by-1 vector with values of 1, 2, 3, 4, 6, or 12.

Data Types: double

Basis — Day-count basis of contract

2 (actual/360) (default) | integers of the set [0...13] | vector of integers of the set [0...13]

Day-count basis of the contract, specified as the comma-separated pair consisting of `'Basis'` and a positive integer using a NINST-by-1 vector.

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)

- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

BusinessDayConvention — Business day conventions

`'actual'` (default) | character vector

Business day conventions, specified as the comma-separated pair consisting of `'BusinessDayConvention'` and a character vector. The selection for business day convention determines how non-business days are treated. Non-business days are defined as weekends plus any other date that businesses are not open (for example, statutory holidays). Values are:

- `'actual'` — Non-business days are effectively ignored. Cash flows that fall on non-business days are assumed to be distributed on the actual date.
- `'follow'` — Cash flows that fall on a non-business day are assumed to be distributed on the following business day.
- `'modifiedfollow'` — Cash flows that fall on a non-business day are assumed to be distributed on the following business day. However if the following business day is in a different month, the previous business day is adopted instead.
- `'previous'` — Cash flows that fall on a non-business day are assumed to be distributed on the previous business day.
- `'modifiedprevious'` — Cash flows that fall on a non-business day are assumed to be distributed on the previous business day. However if the previous business day is in a different month, the following business day is adopted instead.

Data Types: `char`

PayAccruedPremium — Flag for accrued premiums paid upon default

`true` (default) | integer with value 1 or 0

Flag for accrued premiums paid upon default, specified as the comma-separated pair consisting of `'PayAccruedPremium'` and a N-by-1 vector of Boolean flags that is `true` (default) if accrued premiums are paid upon default, `false` otherwise.

Data Types: `logical`

TimeStep — Number of days as time step for numerical integration

10 (days) (default) | nonnegative integer

Number of days to take as time step for the numerical integration, specified as the comma-separated pair consisting of `'TimeStep'` and a nonnegative integer.

Data Types: `double`

ZeroCompounding — Compounding frequency of the zero curve

2 (semiannual) (default) | integer with value of 1,2,3,4,6,12, or -1

Compounding frequency of the zero curve, specified as the comma-separated pair consisting of `'ZeroCompounding'` and an integer with values:

- 1 — Annual compounding

- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- -1 — Continuous compounding

Data Types: double

ZeroBasis — Basis of the zero curve

0 (actual/actual) (default) | integer with value of 0 to 13

Basis of the zero curve, specified as the comma-separated pair consisting of 'ZeroBasis' and an integer with values that are identical to Basis.

Data Types: double

ProbDates — Dates for probability data

column of dates in MarketData (default) | datetime array | string array | date character vector

Dates for probability data, specified as the comma-separated pair consisting of 'ProbDates' and a P-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, cdsbootstrap also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Output Arguments

ProbData — Default probability values

matrix

Default probability values, returned as a P-by-2 matrix with dates and corresponding cumulative default probability values. The dates match those in MarketData, unless the optional input parameter ProbDates is provided.

HazData — Hazard rate values

matrix

Hazard rate values, returned as a N-by-2 matrix with dates and corresponding hazard rate values for the survival probability model. The dates match those in MarketData.

Note A warning is displayed when non-monotone default probabilities (that is, negative hazard rates) are found.

More About

Credit Default Swap

A Credit Default Swap (CDS) is a financial derivative contract that allows an investor to "swap" or transfer the credit risk of a borrower (usually a corporation or government) to another party.

The parties involved are:

- **Protection Buyer** — The party that purchases the CDS to protect against the risk of default. They pay periodic premiums (known as the CDS spread) to the protection seller.
- **Protection Seller** — The party that provides the credit protection and receives the premiums from the protection buyer. In the event of a default, the protection seller is obligated to compensate the protection buyer.

Algorithms

If the time to default is denoted by τ , the default probability curve, or function, $PD(t)$, and its complement, the survival function $Q(t)$, are given by:

$$PD(t) = P[\tau \leq t] = 1 - P[\tau > t] = 1 - Q(t)$$

In the standard model, the survival probability is defined in terms of a piecewise constant hazard rate $h(t)$. For example, if $h(t) =$

$$\lambda_1, \text{ for } 0 \leq t \leq t_1$$

$$\lambda_2, \text{ for } t_1 < t \leq t_2$$

$$\lambda_3, \text{ for } t_2 < t$$

then the survival function is given by $Q(t) =$

$$e^{-\lambda_1 t}, \text{ for } 0 \leq t \leq t_1$$

$$e^{-\lambda_1 t_1 - \lambda_2(t - t_1)}, \text{ for } t_1 < t \leq t_2$$

$$e^{-\lambda_1 t_1 - \lambda_2(t_2 - t_1) - \lambda_3(t - t_2)}, \text{ for } t_2 < t$$

Given n market dates $t_1, \dots, t_n$ and corresponding market CDS spreads $S_1, \dots, S_n$, `cdsbootstrap` calibrates the parameters $\lambda_1, \dots, \lambda_n$ and evaluates $PD(t)$ on the market dates, or an optional user-defined set of dates.

Version History

Introduced in R2010b

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cdsbootstrap` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

$y =$

2021

There are no plans to remove support for serial date number inputs.

References

- [1] Beumee, J., D. Brigo, D. Schiemert, and G. Stoye. *"Charting a Course Through the CDS Big Bang."* Fitch Solutions, Quantitative Research, Global Special Report. April 7, 2009.
- [2] Hull, J., and A. White. "Valuing Credit Default Swaps I: No Counterparty Default Risk." *Journal of Derivatives*. Vol. 8, pp. 29-40.
- [3] O'Kane, D. and S. Turnbull. *"Valuation of Credit Default Swaps."* Lehman Brothers, Fixed Income Quantitative Credit Research, April 2003.

See Also

`cdsspread` | `cdsprice` | `cdsrpv01` | `IRDataCurve` | `defprobstrip`

Topics

- "Bootstrapping a Default Probability Curve" on page 8-101
- "Bootstrapping from Inverted Market Curves" on page 8-111
- "Credit Default Swap (CDS)" on page 8-100

cdsprice

Determine price for credit default swap

Syntax

```
[Price,AccPrem,PaymentDates,PaymentTimes,PaymentCF] = cdsprice(ZeroData,  
ProbData,Settle,Maturity,ContractSpread)  
[Price,AccPrem,PaymentDates,PaymentTimes,PaymentCF] = cdsprice( __ ,  
Name,Value)
```

Description

[Price,AccPrem,PaymentDates,PaymentTimes,PaymentCF] = cdsprice(ZeroData, ProbData,Settle,Maturity,ContractSpread) computes the price, or the mark-to-market value for CDS instruments.

Note Alternatively, you can use the Financial Instruments Toolbox CDS object to price credit default swaps. For more information, see “Get Started with Workflows Using Object-Based Framework for Pricing Financial Instruments” (Financial Instruments Toolbox).

[Price,AccPrem,PaymentDates,PaymentTimes,PaymentCF] = cdsprice(__ , Name,Value) adds optional name-value pair arguments.

Examples

Determine the Price for a Credit Default Swap

This example shows how to use `cdsprice` to compute the clean price for a CDS contract using the following data.

```
Settle = '17-Jul-2009'; % valuation date for the CDS  
Zero_Time = [.5 1 2 3 4 5]';  
Zero_Rate = [1.35 1.43 1.9 2.47 2.936 3.311]'/100;  
Zero_Dates = daysadd(Settle,360*Zero_Time,1);  
ZeroData = [Zero_Dates Zero_Rate];  
  
ProbData = [daysadd(datenum(Settle),360,1), 0.0247];  
Maturity = datetime(2010,9,20);  
ContractSpread = 135;  
  
[Price,AccPrem] = cdsprice(ZeroData,ProbData,Settle,Maturity,ContractSpread)  
  
Price =  
1.5461e+04  
  
AccPrem =  
10500
```

Input Arguments

ZeroData — Zero rate data

vector | IRDataCurve object

Zero rate data, specified as a M-by-2 vector of dates, using a serial date number format, and zero rates or an IRDataCurve object of zero rates.

When ZeroData is an IRDataCurve object, ZeroCompounding and ZeroBasis are implicit in ZeroData and are redundant inside this function. In this case, specify these optional parameters when constructing the IRDataCurve object before using the cdsprice function.

For more information on an IRDataCurve object, see “Creating an IRDataCurve Object” (Financial Instruments Toolbox).

Data Types: double | object

ProbData — Default probability values

matrix

Default probability values, specified as a P-by-2 matrix with dates, using a serial date number format, and the corresponding cumulative default probability values.

Data Types: double

Settle — Settlement date

datetime scalar | string scalar | date character vector

Settlement date, specified as a scalar datetime, string, or date character vector. The Settle date must be earlier than or equal to the dates in Maturity.

To support existing code, cdsprice also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date

datetime array | string array | date character vector

Maturity date, specified as a N-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, cdsprice also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

ContractSpread — Contract spreads

numeric

Contract spreads, specified as a N-by-1 vector of spreads, expressed in basis points.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Note Any optional input of size N-by-1 is also acceptable as an array of size 1-by-N, or as a single value applicable to all contracts. Single values are internally expanded to an array of size N-by-1.

Example: `[Price, AccPrem] = cdsprice(ZeroData, ProbData, Settle, Maturity, ContractSpread, 'Basis', 7, 'BusinessDayConvention', 'previous')`

RecoveryRate — Recovery rate

0.4 (default) | decimal

Recovery rate, specified as the comma-separated pair consisting of 'RecoveryRate' and a N-by-1 vector of recovery rates, specified as a decimal from 0 to 1.

Data Types: double

Period — Premium payment frequency

4 (default) | numeric with values 1, 2, 3, 4, 6 or 12

Premium payment frequency, specified as the comma-separated pair consisting of 'Period' and a N-by-1 vector with values of 1, 2, 3, 4, 6, or 12.

Data Types: double

Basis — Day-count basis of contract

2 (actual/360) (default) | positive integers of the set [1...13] | vector of positive integers of the set [1...13]

Day-count basis of the contract, specified as the comma-separated pair consisting of 'Basis' and a positive integer using a NINST-by-1 vector.

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)

- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

BusinessDayConvention — Business day conventions

`actual` (default) | character vector

Business day conventions, specified as the comma-separated pair consisting of 'BusinessDayConvention' and a character vector. The selection for business day convention determines how non-business days are treated. Non-business days are defined as weekends plus any other date that businesses are not open (for example, statutory holidays). Values are:

- `actual` — Non-business days are effectively ignored. Cash flows that fall on non-business days are assumed to be distributed on the actual date.
- `follow` — Cash flows that fall on a non-business day are assumed to be distributed on the following business day.
- `modifiedfollow` — Cash flows that fall on a non-business day are assumed to be distributed on the following business day. However if the following business day is in a different month, the previous business day is adopted instead.
- `previous` — Cash flows that fall on a non-business day are assumed to be distributed on the previous business day.
- `modifiedprevious` — Cash flows that fall on a non-business day are assumed to be distributed on the previous business day. However if the previous business day is in a different month, the following business day is adopted instead.

Data Types: `char`

PayAccruedPremium — Flag for accrued premiums paid upon default

`true` (default) | integer with value 1 or 0

Flag for accrued premiums paid upon default, specified as the comma-separated pair consisting of 'PayAccruedPremium' and a N-by-1 vector of Boolean flags that is `true` (default) if accrued premiums are paid upon default, `false` otherwise.

Data Types: `logical`

Notional — Contract notional values

`10MM` (default) | positive or negative integer

Contract notional values, specified as the comma-separated pair consisting of 'Notional' and a N-by-1 vector of integers. Use positive integer values for long positions and negative integer values for short positions.

Data Types: `double`

TimeStep — Number of days as time step for numerical integration

`10` (days) (default) | nonnegative integer

Number of days to take as time step for the numerical integration, specified as the comma-separated pair consisting of 'TimeStep' and a nonnegative integer.

Data Types: double

ZeroCompounding — Compounding frequency of the zero curve

2 (semiannual) (default) | integer with value of 1,2,3,4,6,12, or -1

Compounding frequency of the zero curve, specified as the comma-separated pair consisting of 'ZeroCompounding' and an integer with values:

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- -1 — Continuous compounding

Data Types: double

ZeroBasis — Basis of the zero curve

0 (actual/actual) (default) | integer with value of 0 to 13

Basis of the zero curve, specified as the comma-separated pair consisting of 'ZeroBasis' and an integer with values that are identical to Basis.

Data Types: double

Output Arguments

Price — CDS clean prices

vector

CDS clean prices, returned as a N-by-1 vector.

AccPrem — Accrued premiums

vector

Accrued premiums, returned as a N-by-1 vector.

PaymentDates — Payment dates

matrix

Payment dates, returned as a N-by-numCF matrix.

PaymentTimes — Payment times

matrix

Payment times, returned as a N-by-numCF matrix of accrual fractions.

PaymentCF — Payments

matrix

Payments, returned as a N-by-numCF matrix.

More About

Credit Default Swap

A Credit Default Swap (CDS) is a financial derivative contract that allows an investor to "swap" or transfer the credit risk of a borrower (usually a corporation or government) to another party.

The parties involved are:

- **Protection Buyer** — The party that purchases the CDS to protect against the risk of default. They pay periodic premiums (known as the CDS spread) to the protection seller.
- **Protection Seller** — The party that provides the credit protection and receives the premiums from the protection buyer. In the event of a default, the protection seller is obligated to compensate the protection buyer.

CDS Price

The price or mark-to-market (MtM) value of an existing CDS contract.

The CDS price is computed using the following formula:

$$\text{CDS price} = \text{Notional} * (\text{Current Spread} - \text{Contract Spread}) * \text{RPV01}$$

Current Spread is the current breakeven spread for a similar contract, according to current market conditions. **RPV01** is the 'risky present value of a basis point,' the present value of the premium payments, considering the default probability. This formula assumes a long position, and the right side is multiplied by -1 for short positions.

Algorithms

The premium leg is computed as the product of a spread S and the risky present value of a basis point (RPV01). The RPV01 is given by:

$$\text{RPV01} = \sum_{j=1}^N Z(t_j) \Delta(t_{j-1}, t_j, B) Q(t_j)$$

when no accrued premiums are paid upon default, and it can be approximated by

$$\text{RPV01} \approx \frac{1}{2} \sum_{j=1}^N Z(t_j) \Delta(t_{j-1}, t_j, B) (Q(t_{j-1}) + Q(t_j))$$

when accrued premiums are paid upon default. Here, $t_0 = 0$ is the valuation date, and $t_1, \dots, t_n = T$ are the premium payment dates over the life of the contract, T is the maturity of the contract, $Z(t)$ is the discount factor for a payment received at time t , and $\Delta(t_{j-1}, t_j, B)$ is a day count between dates t_{j-1} and t_j corresponding to a basis B .

The protection leg of a CDS contract is given by the following formula:

$$\begin{aligned} \text{ProtectionLeg} &= \int_0^T Z(\tau) (1 - R) dPD(\tau) \\ &\approx (1 - R) \sum_{i=1}^M Z(\tau_i) (PD(\tau_i) - PD(\tau_{i-1})) \end{aligned}$$

$$= (1 - R) \sum_{i=1}^M Z(\tau_i)(Q(\tau_i - 1) - Q(\tau_i))$$

where the integral is approximated with a finite sum over the discretization $\tau_0 = 0, \tau_1, \dots, \tau_M = T$.

If the spread of an existing CDS contract is S_C , and the current breakeven spread for a comparable contract is S_0 , the current price, or mark-to-market value of the contract is given by:

$$\text{MtM} = \text{Notional} (S_0 - S_C) \text{RPV01}$$

This assumes a long position from the protection standpoint (protection was bought). For short positions, the sign is reversed.

Version History

Introduced in R2010b

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cdsprice` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] Beumee, J., D. Brigo, D. Schiemert, and G. Stoye. "Charting a Course Through the CDS Big Bang." Fitch Solutions, Quantitative Research, Global Special Report. April 7, 2009.
- [2] Hull, J., and A. White. "Valuing Credit Default Swaps I: No Counterparty Default Risk." *Journal of Derivatives*. Vol. 8, pp. 29-40.
- [3] O'Kane, D. and S. Turnbull. "Valuation of Credit Default Swaps." Lehman Brothers, Fixed Income Quantitative Credit Research, April 2003.

See Also

`cdsbootstrap` | `cdsspread` | `cdsoptprice` | `IRDataCurve`

Topics

- "Finding Breakeven Spread for New CDS Contract" on page 8-104
- "Valuing an Existing CDS Contract" on page 8-107
- "Converting from Running to Upfront" on page 8-109

"Bootstrapping a Default Probability Curve from Credit Default Swaps" (Financial Instruments Toolbox)

"Credit Default Swap (CDS)" on page 8-100

cdsspread

Determine spread of credit default swap

Syntax

```
[Spread,PaymentDates,PaymentTimes,] = cdsspread(ZeroData,ProbData,Settle,
Maturity,)  
[Spread,PaymentDates,PaymentTimes,] = cdsspread( ___,Name,Value)
```

Description

[Spread,PaymentDates,PaymentTimes,] = cdsspread(ZeroData,ProbData,Settle, Maturity,) computes the spread of the CDS.

[Spread,PaymentDates,PaymentTimes,] = cdsspread(___,Name,Value) adds optional name-value pair arguments.

Examples

Determine the Spread of a Credit Default Swap

This example shows how to use `cdsspread` to compute the spread (in basis points) for a CDS contract with the following data.

```
Settle = '17-Jul-2009'; % valuation date for the CDS  
Zero_Time = [.5 1 2 3 4 5]';  
Zero_Rate = [1.35 1.43 1.9 2.47 2.936 3.311]'/100;  
Zero_Dates = daysadd(Settle,360*Zero_Time,1);  
ZeroData = [Zero_Dates Zero_Rate];  
ProbData = [daysadd(datenum(Settle),360,1), 0.0247];  
Maturity = datetime(2010,9,20);
```

```
Spread = cdsspread(ZeroData,ProbData,Settle,Maturity)
```

```
Spread =  
148.2705
```

Input Arguments

ZeroData — Zero rate data

vector | IRDataCurve object

Zero rate data, specified as a M-by-2 vector of dates, using a serial date number format, and zero rates or an IRDataCurve object of zero rates.

When ZeroData is an IRDataCurve object, ZeroCompounding and ZeroBasis are implicit in ZeroData and are redundant inside this function. In this case, specify these optional parameters when constructing the IRDataCurve object before using the `cdsspread` function.

For more information on an `IRDataCurve` object, see “Creating an `IRDataCurve` Object” (Financial Instruments Toolbox).

Data Types: `double` | `object`

ProbData — Default probability values

`matrix`

Default probability values, specified as a P-by-2 matrix with dates, using a serial date number format, and corresponding cumulative default probability values.

Data Types: `double`

Settle — Settlement date

`datetime scalar` | `string scalar` | `date character vector`

Settlement date, specified as a scalar `datetime`, `string`, or `date character vector`. The `Settle` date must be earlier than or equal to the dates in `Maturity`.

To support existing code, `cdsspread` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date

`datetime array` | `string array` | `date character vector`

Maturity date, specified as a N-by-1 vector using a `datetime` array, `string array`, or `date character vectors`.

To support existing code, `cdsspread` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Note Any optional input of size N-by-1 is also acceptable as an array of size 1-by-N, or as a single value applicable to all contracts. Single values are internally expanded to an array of size N-by-1.

Example: `Spread =`

```
cdsspread(ZeroData,ProbData,Settle,Maturity,'Basis',7,'BusinessDayConvention',
'previous')
```

RecoveryRate — Recovery rate

`0.4 (default)` | `decimal`

Recovery rate, specified as the comma-separated pair consisting of `'RecoveryRate'` and a N-by-1 vector of recovery rates, specified as a decimal from 0 to 1.

Data Types: double

Period — Premium payment frequency

4 (default) | numeric with values 1, 2, 3, 4, 6 or 12

Premium payment frequency, specified as the comma-separated pair consisting of 'Period' and a N-by-1 vector with values of 1, 2, 3, 4, 6, or 12.

Data Types: double

Basis — Day-count basis of contract

2 (actual/360) (default) | positive integers of the set [1...13] | vector of positive integers of the set [1...13]

Day-count basis of the contract, specified as the comma-separated pair consisting of 'Basis' and a positive integer using a NINST-by-1 vector.

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

BusinessDayConvention — Business day conventions

actual (default) | character vector

Business day conventions, specified as the comma-separated pair consisting of 'BusinessDayConvention' and a character vector. The selection for business day convention determines how non-business days are treated. Non-business days are defined as weekends plus any other date that businesses are not open (for example, statutory holidays). Values are:

- **actual** — Non-business days are effectively ignored. Cash flows that fall on non-business days are assumed to be distributed on the actual date.
- **follow** — Cash flows that fall on a non-business day are assumed to be distributed on the following business day.

- **modifiedfollow** — Cash flows that fall on a non-business day are assumed to be distributed on the following business day. However if the following business day is in a different month, the previous business day is adopted instead.
- **previous** — Cash flows that fall on a non-business day are assumed to be distributed on the previous business day.
- **modifiedprevious** — Cash flows that fall on a non-business day are assumed to be distributed on the previous business day. However if the previous business day is in a different month, the following business day is adopted instead.

Data Types: `char`

PayAccruedPremium — Flag for accrued premiums paid upon default

`true` (default) | integer with value 1 or 0

Flag for accrued premiums paid upon default, specified as the comma-separated pair consisting of 'PayAccruedPremium' and a N-by-1 vector of Boolean flags that is `true` (default) if accrued premiums are paid upon default, `false` otherwise.

Data Types: `logical`

TimeStep — Number of days as time step for numerical integration

10 (days) (default) | nonnegative integer

Number of days to take as time step for the numerical integration, specified as the comma-separated pair consisting of 'TimeStep' and a nonnegative integer.

Data Types: `double`

ZeroCompounding — Compounding frequency of the zero curve

2 (semiannual) (default) | integer with value of 1,2,3,4,6,12, or -1

Compounding frequency of the zero curve, specified as the comma-separated pair consisting of 'ZeroCompounding' and an integer with values:

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- -1 — Continuous compounding

Data Types: `double`

ZeroBasis — Basis of the zero curve

0 (actual/actual) (default) | integer with value of 0 to 13

Basis of the zero curve, specified as the comma-separated pair consisting of 'ZeroBasis' and a positive integer with values that are identical to `Basis`.

Data Types: `double`

Output Arguments

Spread — Spreads (in basis points)

vector

Spreads (in basis points), returned as a N-by-1 vector.

PaymentDates — Payment dates

matrix

Payment dates, returned as a N-by-numCF matrix.

PaymentTimes — Payment times

matrix

Payment times, returned as a N-by-numCF matrix of accrual fractions.

More About

Credit Default Swap

A Credit Default Swap (CDS) is a financial derivative contract that allows an investor to "swap" or transfer the credit risk of a borrower (usually a corporation or government) to another party.

The parties involved are:

- **Protection Buyer** — The party that purchases the CDS to protect against the risk of default. They pay periodic premiums (known as the CDS spread) to the protection seller.
- **Protection Seller** — The party that provides the credit protection and receives the premiums from the protection buyer. In the event of a default, the protection seller is obligated to compensate the protection buyer.

CDS Spread

The market, or breakeven, spread value of a CDS.

The CDS spread can be computed by equating the value of the protection leg with the value of the premium leg:

$$\text{Market Spread} * \text{RPV01} = \text{Value of Protection Leg}$$

The left side corresponds to the value of the premium leg, and this has been decomposed as the product of the market or breakeven spread times the RPV01 or 'risky present value of a basis point' of the contract. The latter is the present value of the premium payments, considering the default probability. The Market Spread can be computed as the ratio of the value of the protection leg, to the RPV01 of the contract. `cdsspread` returns the resulting spread in basis points.

Algorithms

The premium leg is computed as the product of a spread S and the risky present value of a basis point (RPV01). The RPV01 is given by:

$$\text{RPV01} = \sum_{j=1}^N Z(t_j) \Delta(t_{j-1}, t_j, B) Q(t_j)$$

when no accrued premiums are paid upon default, and it can be approximated by

$$RPV01 \approx \frac{1}{2} \sum_{j=1}^N Z(t_j) \Delta(t_{j-1}, t_j, B) (Q(t_{j-1}) + Q(t_j))$$

when accrued premiums are paid upon default. Here, $t_0 = 0$ is the valuation date, and $t_1, \dots, t_n = T$ are the premium payment dates over the life of the contract, T is the maturity of the contract, $Z(t)$ is the discount factor for a payment received at time t , and $\Delta(t_{j-1}, t_j, B)$ is a day count between dates t_{j-1} and t_j corresponding to a basis B .

The protection leg of a CDS contract is given by the following formula:

$$\begin{aligned} \text{ProtectionLeg} &= \int_0^T Z(\tau) (1 - R) dPD(\tau) \\ &\approx (1 - R) \sum_{i=1}^M Z(\tau_i) (PD(\tau_i) - PD(\tau_{i-1})) \\ &= (1 - R) \sum_{i=1}^M Z(\tau_i) (Q(\tau_{i-1}) - Q(\tau_i)) \end{aligned}$$

where the integral is approximated with a finite sum over the discretization $\tau_0 = 0, \tau_1, \dots, \tau_M = T$.

A breakeven spread S_0 makes the value of the premium and protection legs equal. It follows that:

$$S_0 = \frac{\text{ProtectionLeg}}{RPV01}$$

Version History

Introduced in R2010b

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cdsspread` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] Beumee, J., D. Brigo, D. Schiemert, and G. Stoye. "Charting a Course Through the CDS Big Bang." Fitch Solutions, Quantitative Research, Global Special Report. April 7, 2009.

- [2] Hull, J., and A. White. "Valuing Credit Default Swaps I: No Counterparty Default Risk." *Journal of Derivatives*. Vol. 8, pp. 29-40.
- [3] O'Kane, D. and S. Turnbull. "*Valuation of Credit Default Swaps*." Lehman Brothers, Fixed Income Quantitative Credit Research, April 2003.

See Also

`cdsbootstrap` | `cdsprice` | `IRDataCurve`

Topics

- "Finding Breakeven Spread for New CDS Contract" on page 8-104
- "Valuing an Existing CDS Contract" on page 8-107
- "Converting from Running to Upfront" on page 8-109
- "First-to-Default Swaps" (Financial Instruments Toolbox)
- "Pricing a CDS Index Option" (Financial Instruments Toolbox)
- "Credit Default Swap (CDS)" on page 8-100

cdsrpv01

Compute risky present value of a basis point for credit default swap

Syntax

```
RPV01 = cdsrpv01(ZeroData,ProbData,Settle,Maturity)
RPV01 = cdsrpv01( ___,Name,Value)

[RPV01,PaymentDates,PaymentTimes] = cdsrpv01(ZeroData,ProbData,Settle,
Maturity)
[RPV01,PaymentDates,PaymentTimes] = cdsrpv01( ___,Name,Value)
```

Description

`RPV01 = cdsrpv01(ZeroData,ProbData,Settle,Maturity)` computes the risky present value of a basis point (RPV01) for a credit default swap (CDS).

`RPV01 = cdsrpv01( ___,Name,Value)` adds optional name-value arguments.

`[RPV01,PaymentDates,PaymentTimes] = cdsrpv01(ZeroData,ProbData,Settle,Maturity)` computes the risky present value of a basis point (RPV01), `PaymentDates`, and `PaymentTimes` for a credit default swap (CDS).

`[RPV01,PaymentDates,PaymentTimes] = cdsrpv01( ___,Name,Value)` computes the risky present value of a basis point (RPV01), `PaymentDates`, and `PaymentTimes` for a credit default swap (CDS) using optional name-value pair arguments.

Examples

Calculate the RPV01 Value for a CDS

Calculate the RPV01 value, given the following specification for a CDS.

```
Settle = '17-Jul-2009'; % valuation date for the CDS
Zero_Time = [.5 1 2 3 4 5]';
Zero_Rate = [1.35 1.43 1.9 2.47 2.936 3.311]'/100;
Zero_Dates = daysadd(Settle,360*Zero_Time,1);
ZeroData = [Zero_Dates Zero_Rate];
ProbData = [daysadd(datetime(Settle),360,1), 0.0247];
Maturity = datetime(2010,9,20);
```

```
RPV01 = cdsrpv01(ZeroData,ProbData,Settle,Maturity)
```

```
RPV01 =
1.1651
```

Input Arguments

ZeroData — Dates and zero rates

object from `IRDataCurve` or vector of dates and zero rates

Dates and zero rates, specified by an M-by-2 vector of dates, using a serial date number format, and zero rates or the `IRDataCurve` object for zero rates. For more information on an `IRDataCurve` object, see “Creating an `IRDataCurve` Object” (Financial Instruments Toolbox).

Data Types: `object` | `double`

ProbData — Dates and default probabilities

vector of dates and default probabilities

Dates and default probabilities, specified by a P-by-2 array, where dates use a serial date number format.

Data Types: `double`

Settle — Settlement date

`datetime` scalar | `string` scalar | `date` character vector

Settlement date, specified by a scalar `datetime`, `string`, or `date` character vector. This must be earlier than or equal to the dates in `Maturity`.

To support existing code, `cdsrpv01` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — CDS maturity date

`datetime` array | `string` array | `date` character vector | `serial date number`

CDS maturity date, specified by an N-by-1 vector using a `datetime` array, `string` array, or `date` character vectors. The CDS premium payment dates occur at regular intervals, and the last payment occurs on these maturity dates.

To support existing code, `cdsrpv01` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `RPV01 =`

```
cdsrpv01(ZeroData,ProbData,Settle,Maturity,'Period',1,'StartDate','20-  
Sep-2010','Basis',1,  
'BusinessDayConvention','actual','CleanRPV01',true,'PayAccruedPremium',true,'Ze  
roCompounding',1,'ZeroBasis',1)
```

Period — Number of premium payments per year

4 (default) | positive integer from the set [1, 2, 3, 4, 6, 12] | vector of positive integers from the set [1, 2, 3, 4, 6, 12]

Number of premium payments per year, specified as the comma-separated pair consisting of 'Period' and an N-by-1 vector. Values are 1, 2, 3, 4, 6, and 12.

Data Types: double

StartDate — Dates the CDS premium leg starts

Settle date (default) | datetime array | string array | date character vector

Dates when the CDS premium leg actually starts, specified as the comma-separated pair consisting of 'StartDate' and an N-by-1 vector using a datetime array, string array, or date character vectors. Must be on or between the Settle and Maturity dates. For a forward-starting CDS, specify this date as a future date after Settle.

To support existing code, cdsrpv01 also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Basis — Day-count basis of contract

2 (actual/360) (default) | positive integers of the set [1...13] | vector of positive integers of the set [1...13]

Day-count basis of the contract, specified as the comma-separated pair consisting of 'Basis' and a positive integer using a NINST-by-1 vector.

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

BusinessDayConvention — Business day conventions

actual (default) | character vector | cell array of character vectors

Business day conventions, specified as the comma-separated pair consisting of 'BusinessDayConvention' and a character vector or N-by-1 cell array of character vectors of business day conventions. The selection for business day convention determines how non-business days are treated. Non-business days are defined as weekends plus any other date that businesses are not open (e.g. statutory holidays). Values are:

- **actual** — Non-business days are effectively ignored. Cash flows that fall on non-business days are assumed to be distributed on the actual date.
- **follow** — Cash flows that fall on a non-business day are assumed to be distributed on the following business day.
- **modifiedfollow** — Cash flows that fall on a non-business day are assumed to be distributed on the following business day. However if the following business day is in a different month, the previous business day is adopted instead.
- **previous** — Cash flows that fall on a non-business day are assumed to be distributed on the previous business day.
- **modifiedprevious** — Cash flows that fall on a non-business day are assumed to be distributed on the previous business day. However if the previous business day is in a different month, the following business day is adopted instead.

Data Types: `char` | `cell`

CleanRPV01 — Flag for premium accrual

`true` (default) | boolean flag with value `true` or `false`

Flag for premium accrual, specified as the comma-separated pair consisting of 'CleanRPV01' and a N-by-1 vector of Boolean flags, which is `true` if the premium accrued at `StartDate` is excluded in the RPV01, and `false` otherwise.

Data Types: `logical`

PayAccruedPremium — Flag for accrued premium payment

`true` (default) | boolean flag with value `true` or `false`

Flag for accrued premium payment, specified as the comma-separated pair consisting of 'PayAccruedPremium' and a N-by-1 vector of Boolean flags, `true` if accrued premiums are paid upon default, `false` otherwise.

Data Types: `logical`

ZeroCompounding — Compounding frequency of zero curve

2 semiannual compounding (default) | integer with acceptable value `[1, 2, 3, 4, 6, 12, -1]`

Compounding frequency of the zero curve, specified as the comma-separated pair consisting of 'ZeroCompounding' and an integer with values:

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

- -1 — Continuous compounding

Note When `ZeroData` is an `IRDataCurve` object, the arguments `ZeroCompounding` and `ZeroBasis` are implicit in `ZeroData` and are redundant inside this function. In that case, specify these optional arguments when constructing the `IRDataCurve` object before calling this function.

Data Types: double

ZeroBasis — Basis of zero curve

0 (actual/actual) (default) | positive integers of the set [1...13] | vector of positive integers of the set [1...13]

Basis of the zero curve, specified as the comma-separated pair consisting of 'ZeroBasis' and a positive integer using a NINST-by-1 vector.

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

Output Arguments

RPV01 — RPV01 value

scalar | vector

RPV01 value, returned as an N-by-1 vector.

PaymentDates — Payment dates

scalar | vector

Payment dates, returned as an N-by-numCF matrix of dates.

PaymentTimes — Payment times

scalar | vector

Payment times, returned as an N-by-numCF matrix of accrual fractions.

More About

Credit Default Swap

A Credit Default Swap (CDS) is a financial derivative contract that allows an investor to "swap" or transfer the credit risk of a borrower (usually a corporation or government) to another party.

The parties involved are:

- **Protection Buyer** — The party that purchases the CDS to protect against the risk of default. They pay periodic premiums (known as the CDS spread) to the protection seller.
- **Protection Seller** — The party that provides the credit protection and receives the premiums from the protection buyer. In the event of a default, the protection seller is obligated to compensate the protection buyer.

RPV01

RPV01, associated with a CDS, is the value of a stream of 1-basis-point premiums according to the payment structure of the CDS contract, and considering the default probability over time.

For more information, see [3] and [4] for details.

Version History

Introduced in R2013b

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cdsrpv01` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)  
  
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] Beumee, J., D. Brigo, D. Schiemert, and G. Stoye. "Charting a Course Through the CDS Big Bang." *Fitch Solutions, Quantitative Research*. Global Special Report. April 7, 2009.
- [2] Hull, J., and A. White. "Valuing Credit Default Swaps I: No Counterparty Default Risk." *Journal of Derivatives*. Vol. 8, pp. 29-40.

[3] O'Kane, D. and S. Turnbull. "Valuation of Credit Default Swaps." *Lehman Brothers, Fixed Income Quantitative Credit Research*. April, 2003.

[4] O'Kane, D. *Modelling Single-name and Multi-name Credit Derivatives*. Wiley Finance, 2008.

See Also

[cdsbootstrap](#) | [cdsspread](#) | [cdsprice](#) | [cdsoptprice](#) | [IRDataCurve](#)

Topics

["Pricing a CDS Index Option" \(Financial Instruments Toolbox\)](#)

["Credit Default Swap Option" \(Financial Instruments Toolbox\)](#)

creditexposures

Compute credit exposures from contract values

Syntax

```
[exposures,exposurecpty] = creditexposures(values,counterparties)
[exposures,exposurecpty] = creditexposures( ___,Name,Value)

[exposures,exposurecpty,collateral] = creditexposures( ___,Name,Value)
```

Description

`[exposures,exposurecpty] = creditexposures(values,counterparties)` computes the counterparty credit exposures from an array of mark-to-market OTC contract values. These exposures are used when calculating the CVA (credit value adjustment) for a portfolio.

`[exposures,exposurecpty] = creditexposures( ___,Name,Value)` adds optional name-value arguments.

`[exposures,exposurecpty,collateral] = creditexposures( ___,Name,Value)` computes the counterparty credit exposures from an array of mark-to-market OTC contract values using optional name-value pair arguments for `CollateralTable` and `Dates`, the `collateral` output is returned for the simulated collateral amounts available to counterparties at each simulation date and over each scenario.

Examples

View Contract Values and Exposures Over Time for a Particular Counterparty

After computing the mark-to-market contract values for a portfolio of swaps over many scenarios, compute the credit exposure for a particular counterparty. View the contract values and credit exposure over time.

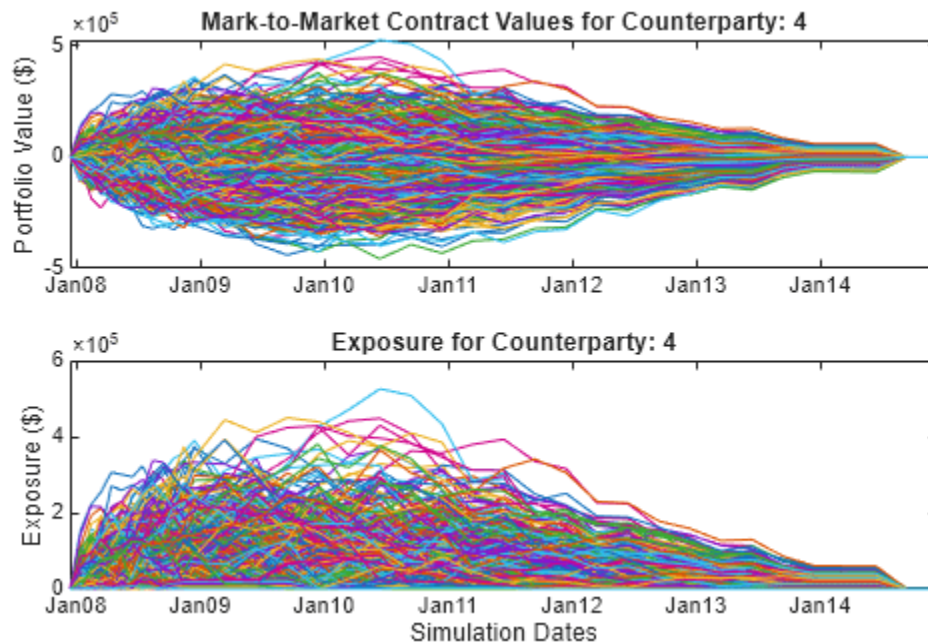
First, load data (`ccr.mat`) containing the mark-to-market contract values for a portfolio of swaps over many scenarios.

```
load ccr.mat
% Look at one counterparty.
cpID = 4;
cpValues = squeeze(sum(values(:,swaps.Counterparty == cpID,:),2));
subplot(2,1,1)
simulationDates = datetime(simulationDates,"ConvertFrom","datenum");
plot(simulationDates,cpValues);
title(sprintf('Mark-to-Market Contract Values for Counterparty: %d',cpID));
xtickformat('MMMyy');
ylabel('Portfolio Value ($)')
% Compute the exposure by counterparty.
[exposures, expcpty] = creditexposures(values,swaps.Counterparty, ...
'NettingID',swaps.NettingID);
% View the credit exposure over time for the counterparty.
```

```

subplot(2,1,2)
cpIdx = find(expcpty == cpID);
plot(simulationDates,squeeze(exposures(:,cpIdx,:)));
title(sprintf('Exposure for Counterparty: %d',cpIdx));
xtickformat('MMMyy');
ylabel('Exposure ($)')
xlabel('Simulation Dates')

```



Compute the Credit Exposure and Determine the Incremental Exposure for a New Trade

Load the data (ccr.mat) containing the mark-to-market contract values for a portfolio of swaps over many scenarios.

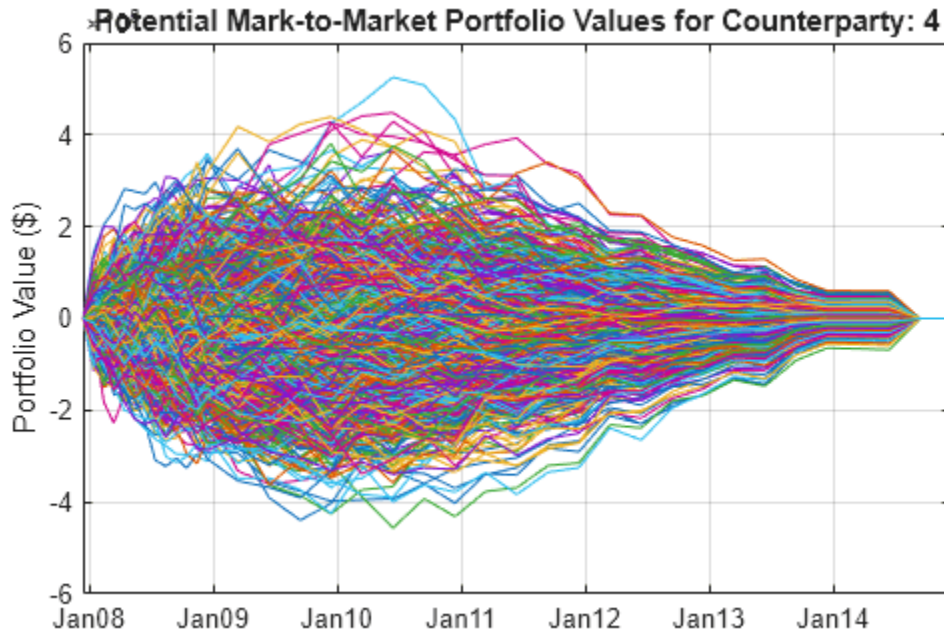
```
load ccr.mat
```

Look at one counterparty.

```

cpID = 4;
cpIdx = swaps.Counterparty == cpID;
cpValues = values(:,cpIdx,:);
simulationDates = datetime(simulationDates,"ConvertFrom","datenum");
plot(simulationDates,squeeze(sum(cpValues,2)));
grid on;
title(sprintf('Potential Mark-to-Market Portfolio Values for Counterparty: %d',cpID));
xtickformat('MMMyy');
ylabel('Portfolio Value ($)')

```

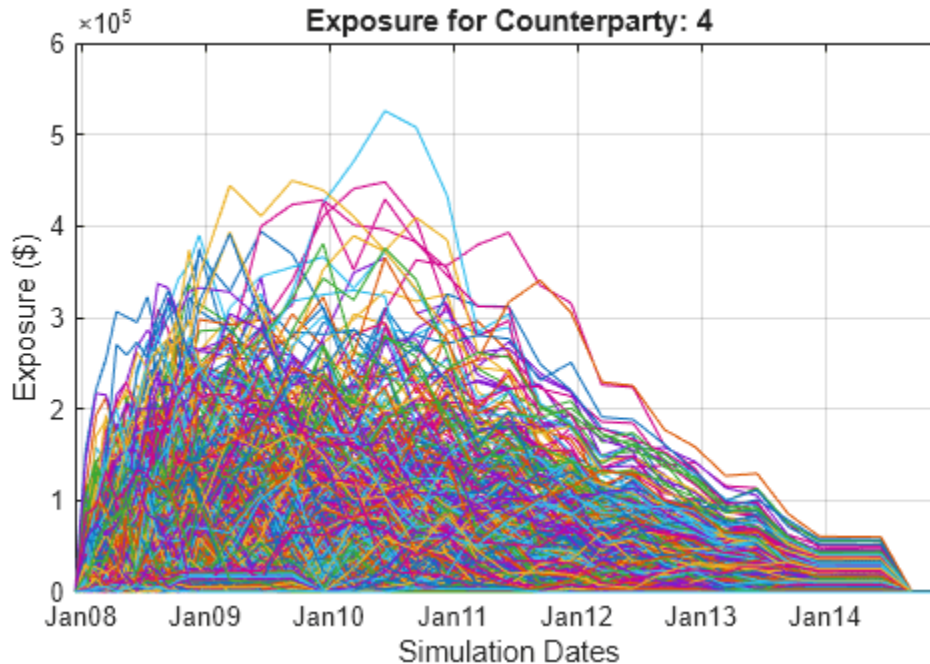


Compute the exposures.

```
netting = swaps.NettingID(cpIdx);
exposures = creditexposures(cpValues,cpID,'NettingID',netting);
```

View the credit exposure over time for the counterparty.

```
figure;
plot(simulationDates,squeeze(exposures));
grid on
title(sprintf('Exposure for Counterparty: %d',cpID));
xtickformat('MMMy');
ylabel('Exposure ($)')
xlabel('Simulation Dates')
```



Compute the credit exposure profiles.

```
simulationDates = convertTo(simulationDates,"datetime");
profilesBefore = exposureprofiles(simulationDates,exposures)
```

```
profilesBefore = struct with fields:
    Dates: [37x1 double]
    EE: [37x1 double]
    PFE: [37x1 double]
    MPFE: 2.1580e+05
    EffEE: [37x1 double]
    EPE: 2.8602e+04
    EffEPE: 4.9579e+04
```

Consider a new trade with a counterparty. For this example, take another trade from the original swap portfolio and "copy" it for a new counterparty. This example is only for illustrative purposes.

```
newTradeIdx = 3;
newTradeValues = values(:,newTradeIdx,:);

% Append a new trade to your existing portfolio.
cpValues = [cpValues newTradeValues];
netting = [netting; cpID];
exposures = creditexposures(cpValues,cpID,'NettingID',netting);
```

Compute the new credit exposure profiles.

```
profilesAfter = exposureprofiles(simulationDates,exposures)

profilesAfter = struct with fields:
    Dates: [37x1 double]
    EE: [37x1 double]
    PFE: [37x1 double]
```

```

MPFE: 2.4689e+05
EffEE: [37×1 double]
EPE: 3.1609e+04
EffEPE: 5.6178e+04

```

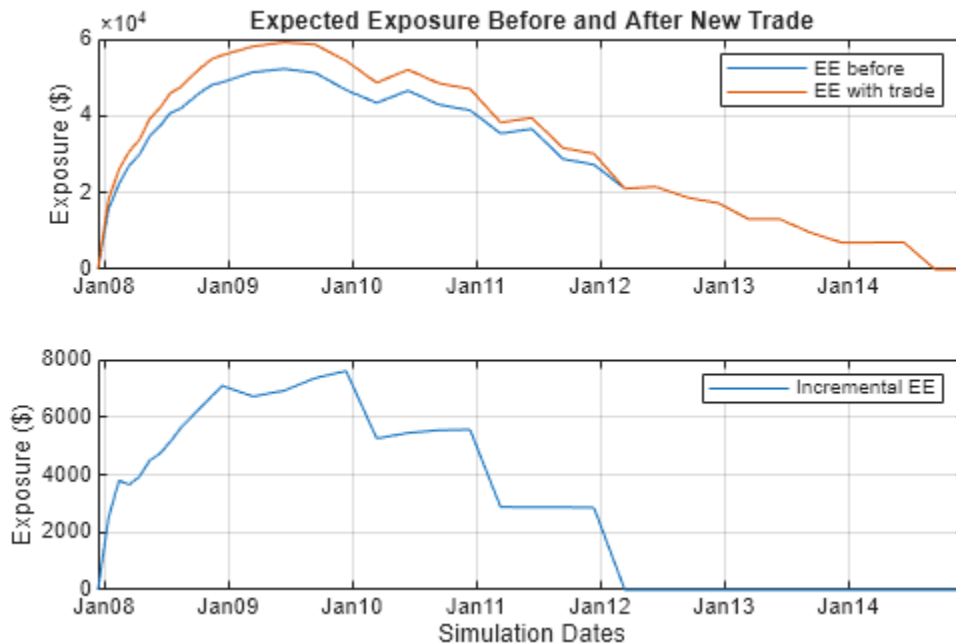
Visualize the expected exposures and the new trade's incremental exposure. Use the incremental exposure to compute the incremental credit value adjustment (CVA) charge.

```

figure;
subplot(2,1,1)
simulationDates = datetime(simulationDates,"ConvertFrom","datenum");
plot(simulationDates,profilesBefore.EE,...
      simulationDates,profilesAfter.EE);
grid on;
legend({'EE before','EE with trade'})
xtickformat('MMMyy');
title('Expected Exposure Before and After New Trade');
ylabel('Exposure ($)')

subplot(2,1,2)
incrementalEE = profilesAfter.EE - profilesBefore.EE;
plot(simulationDates,incrementalEE);
grid on;
legend('Incremental EE')
xtickformat('MMMyy');
ylabel('Exposure ($)')
xlabel('Simulation Dates')

```



Compute Exposures for Counterparties Under Collateral Agreement

Load the data (`ccr.mat`) containing the mark-to-market contract values for a portfolio of swaps over many scenarios.

```
load ccr.mat
```

Only look at a single counterparty for this example.

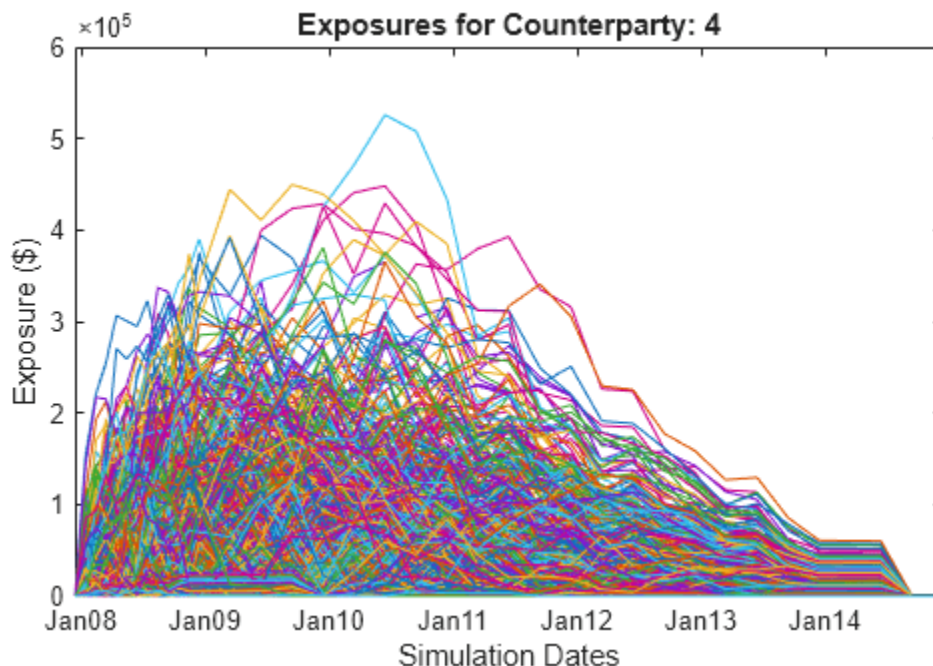
```
cpID = 4;
cpIdx = swaps.Counterparty == cpID;
cpValues = values(:,cpIdx,:);
```

Compute the uncollateralized exposures.

```
exposures = creditexposures(cpValues,swaps.Counterparty(cpIdx),...
    'NettingID',swaps.NettingID(cpIdx));
```

View the credit exposure over time for the counterparty.

```
simulationDates = datetime(simulationDates,"ConvertFrom","datenum");
plot(simulationDates,squeeze(exposures));
expYLim = get(gca,'YLim');
title(sprintf('Exposures for Counterparty: %d',cpID));
xtickformat('MMMy');
ylabel('Exposure ($)')
xlabel('Simulation Dates')
```



Add a collateral agreement for the counterparty. The `'CollateralTable'` parameter is a MATLAB® table. You can create tables from spreadsheets or other data sources, in addition to building them inline as seen here. For more information, see [table](#).

```
collateralVariables = {'Counterparty';'PeriodOfRisk';'Threshold';'MinimumTransfer'};
periodOfRisk = 14;
```

```
threshold = 100000;
minTransfer = 10000;
collateralTable = table(cpID,periodOfRisk,threshold,minTransfer,...
    'VariableNames',collateralVariables)
```

```
collateralTable=1×4 table
    Counterparty    PeriodOfRisk    Threshold    MinimumTransfer
    _____    _____    _____    _____
                4                14                1e+05                10000
```

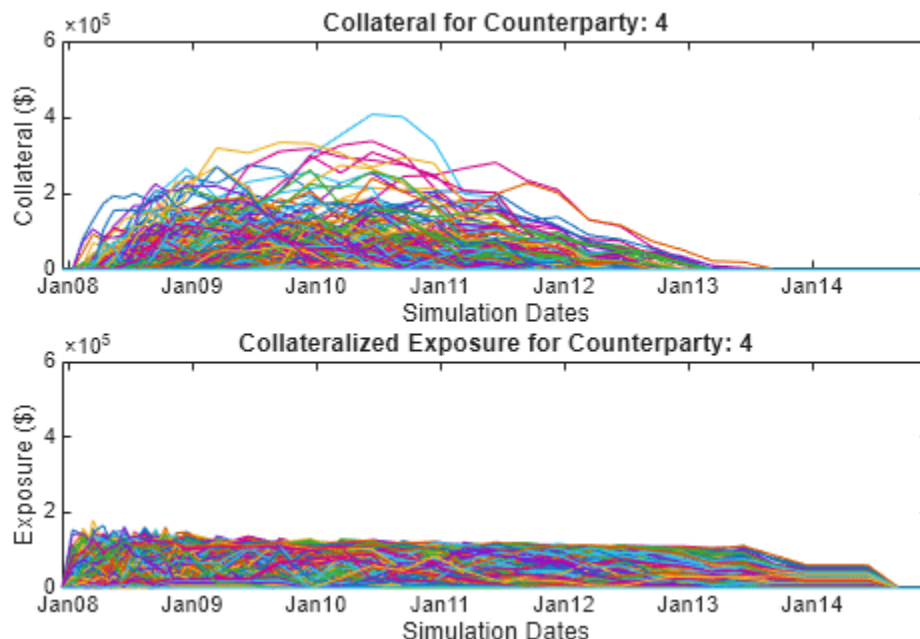
Compute the collateralized exposures.

```
[collatExp, collatcpty, collateral] = creditexposures(cpValues,...
    swaps.Counterparty(cpIdx),'NettingID',swaps.NettingID(cpIdx),...
    'CollateralTable',collateralTable,'Dates',simulationDates);
```

Plot the collateral levels and collateralized exposures.

```
figure;
subplot(2,1,1)
plot(simulationDates,squeeze(collateral));
set(gca,'YLim',expYLim);
title(sprintf('Collateral for Counterparty: %d',cpID));
xtickformat('MMMyy');
ylabel('Collateral ($)')
xlabel('Simulation Dates')

subplot(2,1,2)
plot(simulationDates,squeeze(collatExp));
set(gca,'YLim',expYLim);
title(sprintf('Collateralized Exposure for Counterparty: %d',cpID));
xtickformat('MMMyy');
ylabel('Exposure ($)')
xlabel('Simulation Dates');
```

Input Arguments

values — 3-D array of simulated mark-to-market values of portfolio of contracts

array

3-D array of simulated mark-to-market values of a portfolio of contracts simulated over a series of simulation dates and across many scenarios, specified as a NumDates-by-NumContracts-by-NumScenarios “cube” of contract values. Each row represents a different simulation date, each column a different contract, and each “page” is a different scenario from a Monte-Carlo simulation.

Data Types: double

counterparties — Counterparties corresponding to each contract

vector | cell array

Counterparties corresponding to each contract in values, specified as a NumContracts-element vector of counterparties. Counterparties can be a vector of numeric IDs or a cell array of counterparty names. By default, each counterparty is assumed to have one netting set that covers all of its contracts. If counterparties are covered by multiple netting sets, then use the NettingID parameter. A value of NaN (or ' ' in a cell array) indicates that a contract is not included in any netting set unless otherwise specified by NettingID. counterparties is case insensitive and leading or trailing white spaces are removed.

Data Types: double | cell

Name-Value Pair Arguments

Specify optional pairs of arguments as Name1=Value1, ..., NameN=ValueN, where Name is the argument name and Value is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[exposures,exposureecpty] =
creditexposures(values,counterparties,'NettingID','10','ExposureType','Additive')`

NettingID — Netting set IDs indicate which netting set each contract belongs

vector | cell array

Netting set IDs to indicate to which netting set each contract in `values` belongs, specified by a `NumContracts`-element vector of netting set IDs. `NettingID` can be a vector of numeric IDs or else a cell array of character vector identifiers. The `creditexposures` function uses `counterparties` and `NettingID` to define each unique netting set (all contracts in a netting set must be with the same counterparty). By default, each counterparty has a single netting set which covers all of their contracts. A value of `NaN` (or `''` in a cell array) indicates that a contract is not included in any netting set. `NettingID` is case insensitive and leading or trailing white spaces are removed.

Data Types: double | cell

ExposureType — Calculation method for exposures

'Counterparty' (default) | character vector with value of 'Counterparty' or 'Additive'

Calculation method for exposures, specified with values:

- 'Counterparty' — Compute exposures per counterparty.
- 'Additive' — Compute additive exposures at the contract level. Exposures are computed per contract and sum to the total counterparty exposure.

Data Types: char

CollateralTable — Table containing information on collateral agreements of counterparties

MATLAB table

Table containing information on collateral agreements of counterparties, specified as a MATLAB table. The table consists of one entry (row) per collateralized counterparty and must have the following variables (columns):

- 'Counterparty' — Counterparty name or ID. The Counterparty name or ID should match the parameter 'Counterparty' for the `ExposureType` argument.
- 'PeriodOfRisk' — Margin period of risk in days. The number of days from a margin call until the posted collateral is available from the counterparty.
- 'Threshold' — Collateral threshold. When counterparty exposures exceed this amount, the counterparty must post collateral.
- 'MinimumTransfer' — Minimum transfer amount. The minimum amount over/under the threshold required to trigger transfer of collateral.

Note When computing collateralized exposures, both the `CollateralTable` parameter and the `Dates` parameter must be specified.

Data Types: table

Dates — Simulation dates corresponding to each row of the values array

vector of date numbers | cell array of character vectors

Simulation dates corresponding to each row of the `values` array, specified as a `NUMDATES`-by-1 vector of simulation dates. `Dates` is either a vector of MATLAB date numbers or else a cell array of character vectors in a known date format.

Note When computing collateralized exposures, both the `CollateralTable` parameter and the `Dates` parameter must be specified.

Data Types: double | cell

Output Arguments**exposures — 3-D array of credit exposures**

array

3-D array of credit exposures representing the potential losses from each counterparty or contract at each date and over all scenarios. The size of `exposures` depends on the `ExposureType` input argument:

- When `ExposureType` is 'Counterparty', `exposures` returns a `NumDates`-by-`NumCounterparties`-by-`NumScenarios` “cube” of credit exposures representing potential losses that could be incurred over all dates, counterparties, and scenarios, if a counterparty defaulted (ignoring any post-default recovery).
- When `ExposureType` is 'Additive', `exposures` returns a `NumDates`-by-`NumContracts`-by-`NumScenarios` “cube,” where each element is the additive exposure of each contract (over all dates and scenarios). Additive exposures sum to the counterparty-level exposure.

exposureecpty — Counterparties that correspond to columns of exposures array

vector

Counterparties that correspond to columns of the `exposures` array, returned as `NumCounterparties` or `NumContracts` elements depending on the `ExposureType`.

collateral — Simulated collateral amounts available to counterparties at each simulation date and over each scenario

3D array

Simulated collateral amounts available to counterparties at each simulation date and over each scenario, returned as a `NumDates`-by-`NumCounterparties`-by-`NumScenarios` 3D array. Collateral amounts are calculated using a Brownian bridge to estimate contract values between simulation dates. For more information, see “Brownian Bridge” on page 15-658. If the `CollateralTable` was not specified, this output is empty.

More About**Counterparty Credit Exposure**

Counterparty credit exposure refers to the risk that a counterparty in a financial transaction may default on its obligations, leading to potential financial losses for the other party involved in the transaction.

Brownian Bridge

A Brownian bridge is used to simulate portfolio values at intermediate dates to compute collateral available at the subsequent simulation dates.

For example, to estimate collateral available at a particular simulation date, t_i , you need to know the state of the portfolio at time $t_i - dt$, where dt is the margin period of risk. Portfolio values are simulated at these intermediate dates by drawing from a distribution defined by the Brownian bridge between t_i and the previous simulation date, t_{i-1} .

If the contract values at time t_{i-1} and t_i are known and you want to estimate the contract value at time t_c (where t_c is $t_i - dt$), then a sample from a normal distribution is used with variance:

$$\frac{(t_i - t_c)(t_c - t_{i-1})}{(t_i - t_{i-1})}$$

and with mean that is simply the linear interpolation of the contract values between the two simulation dates at time t_c . For more details, see References.

Version History

Introduced in R2014a

References

- [1] Lomibao, D., and S. Zhu. "A Conditional Valuation Approach for Path-Dependent Instruments." August 2005.
- [2] Pykhtin M. "Modeling credit exposure for collateralized counterparties." December 2009.
- [3] Pykhtin M., and S. Zhu. "A Guide to Modeling Counterparty Credit Risk." GARP, July/August 2007, issue 37.
- [4] Pykhtin, Michael., and Dan Rosen. "Pricing Counterparty Risk at the Trade Level and CVA Allocations." FEDS Working Paper No. 10., February 1, 2010.

See Also

[exposureprofiles](#) | [table](#)

Topics

"Counterparty Credit Risk and CVA" (Financial Instruments Toolbox)
"Wrong Way Risk with Copulas" (Financial Instruments Toolbox)

exposureprofiles

Compute exposure profiles from credit exposures

Syntax

```
profilestructs = exposureprofiles(dates,exposures)
profilestructs = exposureprofiles( ___,Name,Value)
```

Description

`profilestructs = exposureprofiles(dates,exposures)` computes common counterparty credit exposures profiles from an array of exposures.

`profilestructs = exposureprofiles( ___,Name,Value)` adds optional name-value arguments.

Examples

View Exposure Profiles of a Particular Counterparty

After computing the mark-to-market contract values for a portfolio of swaps over many scenarios, view the exposure profiles of a particular counterparty.

Load the data (`ccr.mat`) that contains the mark-to-market contract values for a portfolio of swaps over many scenarios.

```
load ccr.mat
```

Compute the exposure by counterparty.

```
[exposures, expcpty] = creditexposures(values,swaps.Counterparty,...
'NettingID',swaps.NettingID);
```

Compute the credit exposure profiles for all counterparties.

```
cpProfiles = exposureprofiles(simulationDates,exposures)
```

`cpProfiles=5×1 struct array with fields:`

```
Dates
EE
PFE
MPFE
EffEE
EPE
EffEPE
```

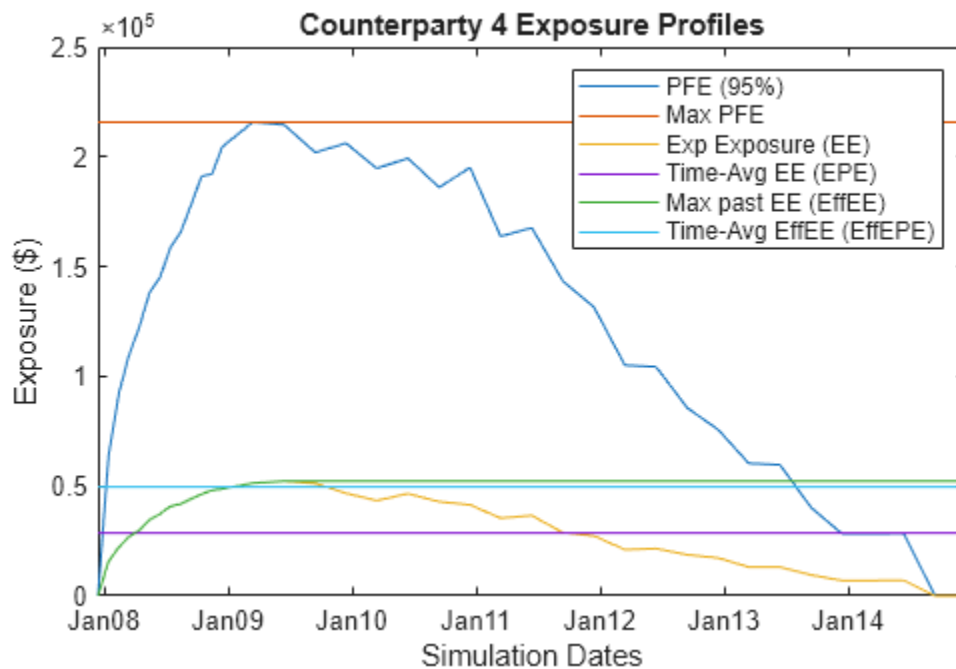
Visualize the exposure profiles for a particular counterparty.

```
cpIdx = find(expcpty == 4);
numDates = numel(simulationDates);
```

```

simulationDates = datetime(simulationDates,"ConvertFrom","datetime");
plot(simulationDates,cpProfiles(cpIdx).PFE, ...
     simulationDates,cpProfiles(cpIdx).MPFE * ones(numDates,1), ...
     simulationDates,cpProfiles(cpIdx).EE,...
     simulationDates,cpProfiles(cpIdx).EPE * ones(numDates,1), ...
     simulationDates,cpProfiles(cpIdx).EffEE,...
     simulationDates,cpProfiles(cpIdx).EffEPE * ones(numDates,1));
legend({'PFE (95%)','Max PFE','Exp Exposure (EE)', ...
       'Time-Avg EE (EPE)','Max past EE (EffEE)', ...
       'Time-Avg EffEE (EffEPE)'});
xtickformat('MMMy');
title(sprintf('Counterparty %d Exposure Profiles',cpIdx));
ylabel('Exposure ($)')
xlabel('Simulation Dates')

```



Input Arguments

dates — Simulation dates

vector of date numbers | cell array of character vectors

Simulation dates, specified as vector of date numbers or a cell array of character vectors in a known date format. For more information for known date formats, see the function `datetime`.

Data Types: `double` | `char` | `cell`

exposures — 3-D array of potential losses due to counterparty default

array

3-D array of potential losses due to counterparty default on a set of instruments simulated over a series of simulation dates and across many scenarios, specified as a `NumDates`-by-`NumCounterParties`-by-`NumScenarios` “cube” of credit exposures. Each row represents a different

simulation date, each column a different counterparty, and each “page” is a different scenario from a Monte-Carlo simulation.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

```
Example: profilestructs =  
exposureprofiles(dates, exposures, 'ProfileSpec', 'PFE', 'PFEProbabilityLevel', .9  
)
```

ProfileSpec — Exposure profiles

All (generate all profiles) (default) | character vector with possible values EE, PFE, MPE, EffEE, EPE, EffEPE, All | cell array of character vectors with possible values EE, PFE, MPE, EffEE, EPE, EffEPE

Exposure profiles, specified as a character vector or cell array of character vectors with the following possible values:

- **EE** — Expected Exposure. The mean of the distribution of exposures at each date. A [NumDates-by-1] vector.
- **PFE** — Potential Future Exposure. A high percentile (default 95%) of the distribution of possible exposures at each date. This is sometimes referred to as “Peak Exposure.” A [NumDates-by-1] vector.
- **MPFE** — Maximum Potential Future Exposure. The maximum potential future exposure (PFE) over all dates
- **EffEE** — Effective Expected Exposure. The maximum expected exposure (at a specific date) that occurs at that date or any prior date. This is the expected exposure, but constrained to be nondecreasing over time. A [NumDates-by-1] vector.
- **EPE** — Expected Positive Exposure. The weighted average over time of expected exposures. A scalar.
- **EffEPE** — Effective Expected Positive Exposure. The weighted average over time of the effective expected exposure (EffEE). A scalar.
- **All** — Generate all the previous profiles.

Note Exposure profiles are computed on a per-counterparty basis.

Data Types: char | cell

PFEProbabilityLevel — Level for potential future exposure (PFE) and maximum potential future exposure (MPFE)

.95 (the 95th percentile) (default) | scalar with value [0..1]

Level for potential future exposure (PFE) and maximum potential future exposure (MPFE), specified as a scalar with value [0..1].

Data Types: `double`

Output Arguments

profilestructs — Structure of credit exposure profiles

array of structs holding credit exposure profiles for each counterparty

Structure of credit exposure profiles, returned as an array of structs holding credit exposure profiles for each counterparty, returned as a struct, with the fields of the struct as the (abbreviated) names of every exposure profile. Profiles listed in the `ProfileSpec` (and their related profiles) are populated, while those not requested contain empty (`[]`). `profilestructs` contains the `dates` information as a vector of MATLAB date numbers requested in the `ProfileSpec` argument.

Version History

Introduced in R2014a

References

[1] *Basel II: International Convergence of Capital Measurement and Capital Standards: A Revised Framework - Comprehensive Version*. at <https://www.bis.org/publ/bcbs128.htm>, 2006.

See Also

`creditexposures` | `datetime`

Topics

“Counterparty Credit Risk and CVA” (Financial Instruments Toolbox)

“Wrong Way Risk with Copulas” (Financial Instruments Toolbox)

cdyield

Yield on certificate of deposit (CD)

Syntax

```
Yield = cdyield(Price,CouponRate,Settle,Maturity,IssueDate)
Yield = cdyield( ____,Basis)
```

Description

`Yield = cdyield(Price,CouponRate,Settle,Maturity,IssueDate)` computes the yield to maturity of a certificate of deposit given its clean price.

`cdyield` assumes that the certificates of deposit pay interest at maturity. Because of the simple interest treatment of these securities, this function is best used for short-term maturities (less than 1 year). The default simple interest calculation uses the **Basis** for the actual/360 convention (2).

`Yield = cdyield( ____,Basis)` adds an optional argument for **Basis**.

Examples

Compute the Yield to Maturity of a Certificate of Deposit

This example shows how to compute the yield on the certificate of deposit (CD), given a CD with the following characteristics.

```
Price      = 101.125;
CouponRate = 0.05;
Settle     = '02-Jan-02';
Maturity   = '31-Mar-02';
IssueDate  = '1-Oct-01';
```

```
Yield = cdyield(Price, CouponRate, Settle, Maturity, IssueDate)
```

```
Yield =
0.0039
```

Compute the Yield to Maturity of a Certificate of Deposit Using datetime Inputs

This example shows how to use `datetime` inputs to compute the yield on the certificate of deposit (CD), given a CD with the following characteristics.

```
Price      = 101.125;
CouponRate = 0.05;
Settle     = datetime('02-Jan-02','Locale','en_US');
Maturity   = datetime('31-Mar-02','Locale','en_US');
IssueDate  = datetime('1-Oct-01','Locale','en_US');
```

```
Yield = cdyield(Price, CouponRate, Settle, Maturity, IssueDate)
```

```
Yield =  
0.0039
```

Input Arguments

Price — Clean price of certificate of deposit per \$100 face

numeric

Clean price of the certificate of deposit per \$100 face, specified as a numeric value using a scalar or a NCDS-by-1 or 1-by-NCDS vector.

Data Types: double

CouponRate — Coupon annual interest rate

decimal

Coupon annual interest rate, specified as decimal using a scalar or a NCDS-by-1 or 1-by-NCDS vector.

Data Types: double

Settle — Settlement date of certificate of deposit

datetime array | string array | date character vector

Settlement date of the certificate of deposit, specified as a scalar or a NCDS-by-1 or 1-by-NCDS vector using a datetime array, string array, or date character vectors. The `Settle` date must be before the `Maturity` date.

To support existing code, `cdyield` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date of certificate of deposit

datetime array | string array | date character vector

Maturity date of the certificate of deposit, specified as a scalar or a NCDS-by-1 or 1-by-NCDS vector using a datetime array, string array, or date character vectors.

To support existing code, `cdyield` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

IssueDate — Issue date for certificate of deposit

datetime array | string array | date character vector

Issue date for the certificate of deposit, specified as a scalar or a NCDS-by-1 or 1-by-NCDS vector using a datetime array, string array, or date character vectors.

To support existing code, `cdyield` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: string | char | datetime

Basis — Day-count basis for certificate of deposit

2 (actual/360) (default) | integers of the set [0 . . . 13] | vector of integers of the set [0 . . . 13]

(Optional) Day-count basis for the certificate of deposit, specified as a scalar or a NINST-by-1 vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

Output Arguments**Yield — Simple yield to maturity of certificate of deposit**

numeric

Simple yield to maturity of the certificate of deposit, returned as a NCDS-by-1 or 1-by-NCDS vector.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cdyield` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

y =

2021

There are no plans to remove support for serial date number inputs.

See Also

bndprice | cdai | cdprice | stepcpnprice | tbillprice | datetime

Topics

“Coupon Date Calculations” on page 2-20

“Yield Conventions” on page 2-21

cfamounts

Cash flow and time mapping for bond portfolio

Syntax

```
[CFlowAmounts,CFlowDates,TFactors,CFlowFlags,CFPrincipal] = cfamounts(
CouponRate,Settle,Maturity)
[CFlowAmounts,CFlowDates,TFactors,CFlowFlags,CFPrincipal] = cfamounts( __ ,
Name,Value)
```

Description

[CFlowAmounts,CFlowDates,TFactors,CFlowFlags,CFPrincipal] = cfamounts(CouponRate,Settle,Maturity) returns matrices of cash flow amounts, cash flow dates, time factors, and cash flow flags for a portfolio of NUMBONDS fixed-income securities.

The elements contained in the cfamounts outputs for the cash flow matrix, time factor matrix, and cash flow flag matrix correspond to the cash flow dates for each security. The first element of each row in the cash flow matrix is the accrued interest payable on each bond. This accrued interest is zero in the case of all zero coupon bonds. cfamounts determines all cash flows and time mappings for a bond whether or not the coupon structure contains odd first or last periods. All output matrices are padded with NaNs as necessary to ensure that all rows have the same number of elements.

[CFlowAmounts,CFlowDates,TFactors,CFlowFlags,CFPrincipal] = cfamounts(__ , Name,Value) adds optional name-value arguments.

Examples

Compute the Cash Flow Structure and Time Factors for a Bond Portfolio

This example shows how to compute the cash flow structure and time factors for a bond portfolio that contains a corporate bond paying interest quarterly and a Treasury bond paying interest semiannually.

```
Settle = '01-Nov-1993';
Maturity = ['15-Dec-1994';'15-Jun-1995'];
CouponRate= [0.06; 0.05];
Period = [4; 2];
Basis = [1; 0];
[CFlowAmounts,CFlowDates,TFactors,CFlowFlags] = ...
cfamounts(CouponRate,Settle,Maturity,Period,Basis)
```

```
CFlowAmounts = 2×6
```

```
    -0.7667    1.5000    1.5000    1.5000    1.5000   101.5000
    -1.8989    2.5000    2.5000    2.5000   102.5000         NaN
```

```
CFlowDates = 2×6
```

728234	728278	728368	728460	728552	728643
728234	728278	728460	728643	728825	NaN

TFactors = 2×6

0	0.2404	0.7403	1.2404	1.7403	2.2404
0	0.2404	1.2404	2.2404	3.2404	NaN

CFlowFlags = 2×6

0	3	3	3	3	4
0	3	3	3	4	NaN

Compute the Cash Flow Structure and Time Factors for a Bond Portfolio and Return a datetime array for CFlowDates

This example shows how to compute the cash flow structure and time factors for a bond portfolio that contains a corporate bond paying interest quarterly and a Treasury bond paying interest semiannually and CFlowDates is returned as a datetime array.

```
Settle = datetime(1993,11,1);
Maturity = [datetime(1994,12,15) ; datetime(1995,6,15)];
CouponRate= [0.06; 0.05];
Period = [4; 2];
Basis = [1; 0];
[CFlowAmounts, CFlowDates, TFactors, CFlowFlags] = cfamounts(CouponRate,...
Settle, Maturity, Period, Basis)
```

CFlowAmounts = 2×6

-0.7667	1.5000	1.5000	1.5000	1.5000	101.5000
-1.8989	2.5000	2.5000	2.5000	102.5000	NaN

CFlowDates = 2×6 *datetime*

01-Nov-1993	15-Dec-1993	15-Mar-1994	15-Jun-1994	15-Sep-1994	15-Dec-1994
01-Nov-1993	15-Dec-1993	15-Jun-1994	15-Dec-1994	15-Jun-1995	NaN

TFactors = 2×6

0	0.2404	0.7403	1.2404	1.7403	2.2404
0	0.2404	1.2404	2.2404	3.2404	NaN

CFlowFlags = 2×6

0	3	3	3	3	4
0	3	3	3	4	NaN

Compute the Cash Flow Structure and Time Factors for a Bond Portfolio Using Optional Name-Value Pairs

This example shows how to compute the cash flow structure and time factors for a bond portfolio that contains a corporate bond paying interest quarterly and a Treasury bond paying interest semiannually. This example uses the following Name-Value pairs for Period, Basis, BusinessDayConvention, and AdjustCashFlowsBasis.

```
Settle = '01-Jun-2010';
Maturity = ['15-Dec-2011'; '15-Jun-2012'];
CouponRate = [0.06; 0.05];
Period = [4; 2];
Basis = [1; 0];

[CFlowAmounts, CFlowDates, TFactors, CFlowFlags] = ...
cfamounts(CouponRate, Settle, Maturity, 'Period', Period, ...
'Basis', Basis, 'AdjustCashFlowsBasis', true, ...
'BusinessDayConvention', 'modifiedfollow')
```

CFlowAmounts = 2×8

-1.2667	1.5000	1.5000	1.5000	1.5000	1.5000	1.5000	101.5000
-2.3077	2.4932	2.5068	2.4932	2.5000	102.5000	NaN	NaN

CFlowDates = 2×8

734290	734304	734396	734487	734577	734669	734761	734852
734290	734304	734487	734669	734852	735035	NaN	NaN

TFactors = 2×8

0	0.0778	0.5778	1.0778	1.5778	2.0778	2.5778	3.0778
0	0.0769	1.0769	2.0769	3.0769	4.0769	NaN	NaN

CFlowFlags = 2×8

0	3	3	3	3	3	3	4
0	3	3	3	3	4	NaN	NaN

Use cfamounts With a CouponRate Schedule

This example shows how to use cfamounts with a CouponRate schedule. For CouponRate and Face that change over the life of the bond, schedules for CouponRate and Face can be specified with an NINST-by-1 cell array, where each element is a NumDates-by-2 matrix where the first column is dates and the second column is associated rates.

```
CouponSchedule = {[datenum('15-Mar-2012') .04; datenum('15-Mar-2013') .05; ...
datenum('15-Mar-2015') .06]}
```

```
CouponSchedule = 1×1 cell array
{3×2 double}
```

```
cfamounts(CouponSchedule, '01-Mar-2011', '15-Mar-2015' )
```

```
ans = 1×10
```

```
-1.8453    2.0000    2.0000    2.0000    2.5000    2.5000    3.0000    3.0000    3.0000    103.
```

Use cfamounts With a Face Schedule

This example shows how to use `cfamounts` with a Face schedule. For `CouponRate` and `Face` that change over the life of the bond, schedules for `CouponRate` and `Face` can be specified with an NINST-by-1 cell array, where each element is a NumDates-by-2 matrix where the first column is dates and the second column is associated rates.

```
FaceSchedule = {[datenum('15-Mar-2012') 100; datenum('15-Mar-2013') 90; ...  
datenum('15-Mar-2015') 80]}
```

```
FaceSchedule = 1×1 cell array  
               {3×2 double}
```

```
cfamounts(.05, '01-Mar-2011', '15-Mar-2015', 'Face', FaceSchedule)
```

```
ans = 1×10
```

```
-2.3066    2.5000    2.5000    12.5000    2.2500    12.2500    2.0000    2.0000    2.0000    82.
```

Use cfamounts to Generate the Cash Flows for a Sinking Bond

This example shows how to use `cfamounts` to generate the cash flows for a sinking bond.

```
[CFflowAmounts, CFDates, TFactors, CFFlags, CFPrincipal] = cfamounts(.05, '04-Nov-2010', ...  
{ '15-Jul-2014'; '15-Jul-2015' }, 'Face', {[datenum('15-Jul-2013') 100; datenum('15-Jul-2014') ...  
90; datenum('15-Jul-2015') 80]} )
```

```
CFflowAmounts = 2×11
```

```
-1.5217    2.5000    2.5000    2.5000    2.5000    2.5000    12.5000    2.2500    92.2500  
-1.5217    2.5000    2.5000    2.5000    2.5000    2.5000    12.5000    2.2500    12.2500    2.
```

```
CFDates = 2×11
```

```
734446    734518    734699    734883    735065    735249    735430    735614  
734446    734518    734699    734883    735065    735249    735430    735614
```

```
TFactors = 2×11
```

```
0    0.3913    1.3913    2.3913    3.3913    4.3913    5.3913    6.3913    7.3913  
0    0.3913    1.3913    2.3913    3.3913    4.3913    5.3913    6.3913    7.3913    8.
```



```
CFFlags = 2×11
```

```
    0    3    3    3    3    3   13    3    4   NaN   NaN
    0    3    3    3    3    3   13    3   13    3    4
```

```
CFPrincipal = 2×11
```

```
    0    0    0    0    0    0   10    0   90   NaN   NaN
    0    0    0    0    0    0   10    0   10    0   80
```

Use cfamounts with User-Defined Holiday Calendar

This example shows how you can optionally specify a different holiday calendar in `cfamounts` to calculate the cash flows for a specific bond.

```
CouponRate = 0.0325;
FutSettle = datetime(2024,4,12);
Maturity = datetime(2055,5,25);
Period = 1;
Basis = 8; % Act/Act ICMA
BusinessDayConvention = 'modifiedfollow';

Hols = datetime(2000,1,1) + calyears(0:100);

[CFlowAmounts, CFlowDates] = cfamounts(CouponRate, FutSettle, Maturity, ...
    'Period',Period, 'Basis',Basis, 'BusinessDayConvention',BusinessDayConvention, ...
    'Holidays',Hols);
table(CFlowDates',CFlowAmounts')
```

```
ans=33×2 table
```

Var1	Var2
12-Apr-2024	-2.8682
27-May-2024	3.25
26-May-2025	3.25
25-May-2026	3.25
25-May-2027	3.25
25-May-2028	3.25
25-May-2029	3.25
27-May-2030	3.25
26-May-2031	3.25
25-May-2032	3.25
25-May-2033	3.25
25-May-2034	3.25
25-May-2035	3.25
26-May-2036	3.25
25-May-2037	3.25
25-May-2038	3.25
⋮	

In this example, you use `cfamounts` with the name-value argument `'Holidays'` to create a simple holiday calendar (`Hols`) that only includes January 1 of each year.

Input Arguments

CouponRate — Annual percentage rate used to determine coupons payable on a bond
decimal

Annual percentage rate used to determine the coupons payable on a bond, specified as decimal using a scalar or a NBONDS-by-1 vector.

CouponRate is 0 for zero coupon bonds.

Note CouponRate and Face can change over the life of the bond. Schedules for CouponRate and Face can be specified with an NBONDS-by-1 cell array, where each element is a NumDates-by-2 matrix or cell array, where the first column is dates (serial date numbers or character vectors) and the second column is associated rates. The date indicates the last day that the coupon rate or face value is valid. This means that the corresponding CouponRate and Face value applies "on or before" the specified ending date.

Data Types: double | cell | char

Settle — Settlement date of bond
datetime array | string array | date character vector

Settlement date of the bond, specified as a scalar or a NBONDS-by-1 vector using a datetime array, string array, or date character vectors. The Settle date must be before the Maturity date.

To support existing code, cfamounts also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date of bond
datetime array | string array | date character vector

Maturity date of the bond, specified as a scalar or a NBONDS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, cfamounts also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Name-Value Pair Arguments

Specify optional pairs of arguments as Name1=Value1,...,NameN=ValueN, where Name is the argument name and Value is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: [CFlowAmounts, CFlowDates, TFactors, CFlowFlags] = ...
cfamounts(CouponRate,Settle,

Maturity, 'Period', 4, 'Basis', 3, 'AdjustCashFlowsBasis', true, 'BusinessDayConvention', 'modifiedfollow')

Period — Number of coupon payments per year for bond

2 (default) | numeric with values 0, 1, 2, 3, 4, 6 or 12

Number of coupon payments per year for the bond, specified as the comma-separated pair consisting of 'Period' and a scalar or a NBONDS-by-1 vector using the values: 0, 1, 2, 3, 4, 6, or 12.

Data Types: double

Basis — Day-count basis of bond

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day-count basis of the bond, specified as the comma-separated pair consisting of 'Basis' and a scalar or a NBONDS-by-1 vector using a supported value:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as the comma-separated pair consisting of 'EndMonthRule' and a scalar or a NBONDS-by-1 vector. This rule applies only when Maturity is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: logical

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond issue date (the date the bond begins to accrue interest), specified as the comma-separated pair consisting of 'IssueDate' and a scalar or a NBONDS-by-1 vector using a datetime array, string array, or date character vectors. The IssueDate cannot be after the Settle date.

If you do not specify an IssueDate, the cash flow payment dates are determined from other inputs.

To support existing code, cfamounts also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

FirstCouponDate — Irregular or normal first coupon date

datetime array | string array | date character vector

Irregular or normal first coupon date, specified as the comma-separated pair consisting of 'FirstCouponDate' and a scalar or a NBONDS-by-1 vector using a datetime array, string array, or date character vectors.

If you do not specify a FirstCouponDate, the cash flow payment dates are determined from other inputs.

Note When FirstCouponDate and LastCouponDate are both specified, the FirstCouponDate takes precedence in determining the coupon payment structure. If FirstCouponDate is not specified, then LastCouponDate determines the coupon structure of the bond.

To support existing code, cfamounts also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

LastCouponDate — Irregular or normal last coupon date

datetime array | string array | date character vector

Irregular or normal last coupon date, specified as the comma-separated pair consisting of 'LastCouponDate' and a scalar or a NBONDS-by-1 vector using a datetime array, string array, or date character vectors.

Note When FirstCouponDate and LastCouponDate are both specified, the FirstCouponDate takes precedence in determining the coupon payment structure. If FirstCouponDate is not specified, then LastCouponDate determines the coupon structure of the bond.

To support existing code, cfamounts also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

StartDate — Forward starting date of coupon payments

datetime array | string array | date character vector

Forward starting date of coupon payments after the `Settle` date, specified as the comma-separated pair consisting of `'StartDate'` and a scalar or a NBONDS-by-1 vector using a datetime array, string array, or date character vectors.

Note To make an instrument forward starting, specify `StartDate` as a future date.

If you do not specify a `StartDate`, the effective start date is the `Settle` date.

To support existing code, `cfamounts` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Face — Face value of bond

100 (default) | numeric

Face value of the bond, specified as the comma-separated pair consisting of `'Face'` and a scalar or a NBONDS-by-1 vector.

Note `CouponRate` and `Face` can change over the life of the bond. Schedules for `CouponRate` and `Face` can be specified with an NBONDS-by-1 cell array where each element is a NumDates-by-2 matrix or cell array, where the first column is dates (serial date numbers or character vectors) and the second column is associated rates. The date indicates the last day that the coupon rate or face value is valid. This means that the corresponding `CouponRate` and `Face` value applies "on or before" the specified ending date.

When the corresponding `Face` value is used to compute the coupon cashflow on the specified ending date. Three things happen on the specified ending date:

- 1 The last coupon corresponding to the current `Face` value is paid.
 - 2 The principal differential (between the current and the next `Face` value) is paid.
 - 3 The date marks the beginning of the period with the next `Face` value, for which the cashflow does not occur until later.
-

Data Types: `double` | `cell` | `char`

AdjustCashFlowsBasis — Adjusts cash flows according to accrual amount based on actual period day count

false (default) | logical with a value of true or false

Adjusts cash flows according to the accrual amount based on the actual period day count, specified as the comma-separated pair consisting of `'AdjustCashFlowsBasis'` and a scalar or a NBONDS-by-1 vector.

Data Types: `logical`

BusinessDayConvention — Business day conventions

'actual' (default) | character vector with values 'actual', 'follow', 'modifiedfollow', 'previous' or 'modifiedprevious'

Business day conventions, specified as the comma-separated pair consisting of `'BusinessDayConvention'` and a scalar or NBONDS-by-1 cell array of character vectors of business

day conventions to be used in computing payment dates. The selection for business day convention determines how nonbusiness days are treated. Nonbusiness days are defined as weekends plus any other date that businesses are not open (for example, statutory holidays). Values are:

- 'actual' — Nonbusiness days are effectively ignored. Cash flows that fall on non-business days are assumed to be distributed on the actual date.
- 'follow' — Cash flows that fall on a nonbusiness day are assumed to be distributed on the following business day.
- 'modifiedfollow' — Cash flows that fall on a non-business day are assumed to be distributed on the following business day. However if the following business day is in a different month, the previous business day is adopted instead.
- 'previous' — Cash flows that fall on a nonbusiness day are assumed to be distributed on the previous business day.
- 'modifiedprevious' — Cash flows that fall on a nonbusiness day are assumed to be distributed on the previous business day. However if the previous business day is in a different month, the following business day is adopted instead.

Data Types: char | cell

CompoundingFrequency — Compounding frequency for yield calculation

SIA uses 2, ICMA uses 1 (default) | integer with value of 1, 2, 3, 4, 6, or 12

Compounding frequency for yield calculation, specified as the comma-separated pair consisting of 'CompoundingFrequency' and a scalar or a NBONDS-by-1 vector. Values are:

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note By default, SIA bases (0-7) and BUS/252 use a semiannual compounding convention and ICMA bases (8-12) use an annual compounding convention.

Data Types: double

DiscountBasis — Basis used to compute the discount factors for computing the yield

SIA uses 0 (default) | integers of the set [0...13] | vector of integers of the set [0...13]

Basis used to compute the discount factors for computing the yield, specified as the comma-separated pair consisting of 'DiscountBasis' and a scalar or a NBONDS-by-1 vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)

- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If a SIA day-count basis is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the default behavior is for SIA bases to use the `actual/actual` day count to compute discount factors.

If an ICMA day-count basis or BUS/252 is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the specified bases from the `Basis` input argument are used.

Data Types: `double`

Holidays — Dates for holidays

`holidays.m` used (default) | `datetime` array | `string` array | `date` character vector

Dates for holidays, specified as the comma-separated pair consisting of 'Holidays' and a `NHOLIDAYS-by-1` vector using a `datetime` array, `string` array, or `date` character vectors. Holidays are used in computing business days. For an example, see “Use `cfamounts` with User-Defined Holiday Calendar” on page 15-671.

To support existing code, `cfamounts` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

PrincipalType — Type of principal when a Face schedule is specified

`sinking` (default) | `character` vector with values 'sinking' or 'bullet'

Type of principal when a Face schedule, specified as the comma-separated pair consisting of 'PrincipalType' and a value of 'sinking' or 'bullet' using a scalar or a `NBONDS-by-1` vector.

If 'sinking', principal cash flows are returned throughout the life of the bond.

If 'bullet', principal cash flow is only returned at maturity.

Data Types: `char` | `cell`

Output Arguments

CFlowAmounts — Cash flow amounts

`matrix`

Cash flow amounts, returned as a NBONDS-by-NCFS (number of cash flows) matrix. The first entry in each row vector is the accrued interest due at settlement. This amount could be zero, positive or negative. If no accrued interest is due, the first column is zero. If the bond is trading ex-coupon then the accrued interest is negative.

CFlowDates — Cash flow dates for a portfolio of bonds

matrix

Cash flow dates for a portfolio of bonds, returned as a NBONDS-by-NCFS matrix. Each row represents a single bond in the portfolio. Each element in a column represents a cash flow date of that bond.

If all the above inputs (*Settle*, *Maturity*, *IssueDate*, *FirstCouponDate*, *LastCouponDate*, and *StartDate*) are either strings *s* or date character vectors, then *CFlowDates* is returned as a serial date number. Use the function *datestr* to convert serial date numbers to formatted date character vectors. If any of these inputs are datetime arrays, then *CFlowDates* is returned as a datetime array.

TFactors — Matrix of time factors for a portfolio of bonds

matrix

Matrix of time factors for a portfolio of bonds, returned as a NBONDS-by-NCFS matrix. Each row corresponds to the vector of time factors for each bond. Each element in a column corresponds to the specific time factor associated with each cash flow of a bond.

Time factors are for price/yield conversion and time factors are in units of whole semiannual coupon periods plus any fractional period using an actual day count. For more information on time factors, see “Time Factors” on page 15-679.

CFlowFlags — Cash flow flags for a portfolio of bonds

matrix

Cash flow flags for a portfolio of bonds, returned as a NBONDS-by-NCFS matrix. Each row corresponds to the vector of cash flow flags for each bond. Each element in a column corresponds to the specific flag associated with each cash flow of a bond. Flags identify the type of each cash flow (for example, nominal coupon cash flow, front, or end partial, or “stub” coupon, maturity cash flow).

Flag	Cash Flow Type
0	Accrued interest due on a bond at settlement.
1	Initial cash flow amount smaller than normal due to a “stub” coupon period. A stub period is created when the time from issue date to first coupon date is shorter than normal.
2	Larger than normal initial cash flow amount because the first coupon period is longer than normal.
3	Nominal coupon cash flow amount.
4	Normal maturity cash flow amount (face value plus the nominal coupon amount).
5	End “stub” coupon amount (last coupon period is abnormally short and actual maturity cash flow is smaller than normal).
6	Larger than normal maturity cash flow because the last coupon period longer than normal.
7	Maturity cash flow on a coupon bond when the bond has less than one coupon period to maturity.

Flag	Cash Flow Type
8	Smaller than normal maturity cash flow when the bond has less than one coupon period to maturity.
9	Larger than normal maturity cash flow when the bond has less than one coupon period to maturity.
10	Maturity cash flow on a zero coupon bond.
11	Sinking principal and initial cash flow amount smaller than normal due to a "stub" coupon period. A stub period is created when the time from issue date to first coupon date is shorter than normal.
12	Sinking principal and larger than normal initial cash flow amount because the first coupon period is longer than normal.
13	Sinking principal and nominal coupon cash flow amount.

CFPrincipal — Principal cash flows

matrix

Principal cash flows, returned as a NBONDS-by-NCFS matrix.

If `PrincipalType` is 'sinking', `CFPrincipal` output indicates when the principal is returned.

If `PrincipalType` is 'bullet', `CFPrincipal` is all zeros and, at `Maturity`, the appropriate `Face` value.

More About

Time Factors

Time factors help determine the present value of a stream of cash flows.

The term time factors refer to the exponent TF in the discounting equation

$$PV = \sum_{i=1}^n \left(\frac{CF}{\left(1 + \frac{z}{f}\right)^{TF}} \right),$$

where:

$PV =$	Present value of a cash flow.
$CF =$	Cash flow amount.
$z =$	Risk-adjusted annualized rate or yield corresponding to a given cash flow. The yield is quoted on a semiannual basis.
$f =$	Frequency of quotes for the yield. Default is 2 for <code>Basis</code> values 0 to 7 and 13 and 1 for <code>Basis</code> values 8 to 12. The default can be overridden by specifying the <code>CompoundingFrequency</code> name-value pair.

$TF =$	Time factor for a given cash flow. The time factor is computed using the compounding frequency and the discount basis. If these values are not specified, then the defaults are as follows: <code>CompoundingFrequency</code> default is 2 for <code>Basis</code> values 0 to 7 and 13 and 1 for <code>Basis</code> values 8 to 12. <code>DiscountBasis</code> is 0 for <code>Basis</code> values 0 to 7 and 13 and the value of the input <code>Basis</code> for <code>Basis</code> values 8 to 12.
--------	--

Note The `Basis` is always used to compute accrued interest.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cfamounts` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datetime");
y = year(t)

y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

R2017b: Support for optional name-value input arguments

Behavior changed in R2017b

The specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the optional name-value pair inputs: `Period`, `Basis`, `EndMonthRule`, `IssueDate`, `FirstCouponDate`, `LastCouponDate`, `StartDate`, `Face`, `AdjustCashFlowsBasis`, `BusinessDayConvention`, `CompoundingFrequency`, `DiscountBasis`, `Holidays`, and `PrincipalType`.

References

- [1] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.
- [2] Mayle, J. "Standard Securities Calculations Methods: Fixed Income Securities Formulas for Analytic Measures." SIA, Vol 2, Jan 1994.
- [3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

`accrfrac` | `cfdates` | `cpncount` | `cftimes` | `cpndaten` | `cpndatenq` | `cpndatep` | `cpndaysn` | `cpndaysp` | `datetime` | `cpndatepq`

Topics

"Analyzing and Computing Cash Flows" on page 2-11

External Websites

Asset Liability Management Using MATLAB (3 min 58 sec)

cfconv

Cash flow convexity

Syntax

```
CFlowConvexity = cfconv(CashFlow,Yield)
```

Description

`CFlowConvexity = cfconv(CashFlow,Yield)` returns the convexity of a cash flow in periods.

Examples

Compute the Convexity of a Cash Flow

This example shows how to return the convexity of a cash flow, given a cash flow of nine payments of \$2.50 and a final payment \$102.50, with a periodic yield of 2.5%.

```
CashFlow = [2.5 2.5 2.5 2.5 2.5 2.5 2.5 2.5 2.5 102.5];
```

```
Convex = cfconv(CashFlow, 0.025)
```

```
Convex =  
90.4493
```

Input Arguments

CashFlow — Cash flow

vector of real numbers

Cash flow, specified as a vector of real numbers.

Data Types: double

Yield — Periodic yield

scalar decimal

Periodic yield, specified as a scalar decimal.

Data Types: double

Output Arguments

CFlowConvexity — Convexity

scalar

Convexity returned as a scalar.

More About

Cash Flow Convexity

Cash flow convexity is a financial metric that measures the sensitivity of the cash flows from an investment to changes in interest rates, specifically focusing on how the present value of those cash flows changes as interest rates fluctuate.

Understanding cash flow convexity helps investors assess the risk associated with interest-rate movements. Higher convexity indicates that the cash flows will be less sensitive to interest-rate changes, which can be beneficial in volatile interest-rate environments. Cash flow convexity provides a more comprehensive measure of interest-rate risk than duration alone, as duration only captures the linear relationship between price and yield.

Version History

Introduced before R2006a

See Also

bndconvp | bndconvy | bnddurp | bnddury | cfdur

Topics

“Analyzing and Computing Cash Flows” on page 2-11

cfdates

Cash flow dates for fixed-income security

Syntax

```
CFlowDates = cfdates(Settle,Maturity)
CFlowDates = cfdates( ___,Period,Basis,EndMonthRule,IssueDate,FirstCouponDate,
LastCouponDate)
```

Description

`CFlowDates = cfdates(Settle,Maturity)` generates a matrix of actual cash flow payment dates for NUMBONDS fixed income securities. All cash flow dates are determined regardless of whether the first and last coupon periods are normal, long or short.

`CFlowDates = cfdates( ___,Period,Basis,EndMonthRule,IssueDate,FirstCouponDate,LastCouponDate)` specifies options using one or more optional arguments in addition to the input arguments in the previous syntax.

Examples

Obtain Cash Flow Dates for Fixed-Income Security

Compute the cash flow dates given the `Settle` and `Maturity` dates.

```
CFlowDates = cfdates('14 Mar 1997', '30 Nov 1998', 2, 0, 1)
```

```
CFlowDates = 1×4
```

```
729541    729724    729906    730089
```

```
datestr(CFlowDates)
```

```
ans = 4×11 char array
```

```
'31-May-1997'
'30-Nov-1997'
'31-May-1998'
'30-Nov-1998'
```

If any of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, or `LastCouponDate` are datetime arrays, then `CFlowDates` is returned as a datetime array. For example:

```
CFlowDates = cfdates('14-Mar-1997', datetime('30-Nov-1998','Locale','en_US'), 2, 0, 1)
```

```
CFlowDates = 1×4 datetime
```

```
31-May-1997    30-Nov-1997    31-May-1998    30-Nov-1998
```

Given three securities with different maturity dates and the same default arguments:

```
Maturity = ['30-Sep-1997'; '31-Oct-1998'; '30-Nov-1998'];
CFlowDates = cfdates('14-Mar-1997', Maturity)
```

```
CFlowDates = 3×4
```

```
    729480    729663         NaN         NaN
    729510    729694    729875    730059
    729541    729724    729906    730089
```

To look at the cash-flow dates for the last security:

```
datestr(CFlowDates(3,:))
```

```
ans = 4×11 char array
```

```
'31-May-1997'
'30-Nov-1997'
'31-May-1998'
'30-Nov-1998'
```

Input Arguments

Settle — Settlement date

datetime array | string array | date character vector

Settlement date, specified as an NINST-by-1 vector using a datetime array, string array, or date character vectors. **Settle** must be earlier than **Maturity**.

To support existing code, **cfdates** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date

datetime array | string array | date character vector

Maturity date, specified as an NINST-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, **cfdates** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Period — Coupons per year of the bond

2 (semiannual) (default) | vector of positive integers from the set [1,2,3,4,6,12]

(Optional) Coupons per year of the bond, specified as a vector of positive integers from the set [1,2,3,4,6,12].

Data Types: double

Basis — Day-count basis

0 (actual/actual) (default) | positive integers of the set [1...13] | vector of positive integers of the set [1...13]

(Optional) Day-count basis, specified as positive integers using a **NINST-by-1** vector.

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

(Optional) End-of-month rule flag, specified as a scalar or a **NUMBONDS-by-1** or **1-by-NUMBONDS** vector. This rule applies only when **Maturity** is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: `logical`

IssueDate — Bond issue date

cash flow payment dates are determined from other inputs (default) | `datetime` array | `string` array | `date` character vector

(Optional) Bond Issue date, specified as a scalar or a **NUMBONDS-by-1** or **1-by-NUMBONDS** vector using a `datetime` array, `string` array, or `date` character vectors.

If you do not specify an **IssueDate**, the cash flow payment dates are determined from other inputs.

To support existing code, `cfdates` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

FirstCouponDate — Irregular or normal first coupon date

cash flow payment dates are determined from other inputs (default) | `datetime` array | `string` array | `date` character vector

Irregular or normal first coupon date, specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a datetime array, string array, or date character vectors.

If you do not specify a `FirstCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cfdates` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

LastCouponDate — Irregular or normal last coupon date

cash flow payment dates are determined from other inputs (default) | datetime array | string array | date character vector

Irregular or normal last coupon date, specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a datetime array, string array, or date character vectors.

If you do not specify a `LastCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cfdates` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Output Arguments

CFlowDates — Actual cash flow payment dates

matrix

Actual cash flow payment dates, returned as a `N`-row matrix of dates in datetime format (if inputs are in datetime format). `CFlowDates` has `NUMBONDS` rows and the number of columns is determined by the maximum number of cash flow payment dates required to hold the bond portfolio. NaNs are padded for bonds which have less than the maximum number of cash flow payment dates.

If all of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, or `LastCouponDate` are either strings or date character vectors, then `CFlowDates` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

If any of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, or `LastCouponDate` are datetime arrays, then `CFlowDates` is returned as a datetime array.

Note The cash flow flags for a portfolio of bonds were formerly available as the `cfdates` second output argument, `CFlowFlags`. You can now use `cfamounts` to get these flags. If you specify a `CFlowFlags` argument, `cfdates` displays a message directing you to use `cfamounts`.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cfdates` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`accrfrac` | `cfamounts` | `cftimes` | `cpncount` | `cpndaten` | `cpndatenq` | `cpndatep` | `cpndaysn` | `cpndaysp` | `cpnpersz` | `datetime` | `cpndatepq`

Topics

"Analyzing and Computing Cash Flows" on page 2-11

cfdatesq

Quasi-coupon dates for fixed-income security

Syntax

```
QuasiCouponDates = cfdatesq(Settle,Maturity)
QuasiCouponDates = cfdatesq( ___,Period,Basis,EndMonthRule,IssueDate,
FirstCouponDate,LastCouponDate,PeriodsBeforeSettle,PeriodsAfterMaturity)
```

Description

`QuasiCouponDates = cfdatesq(Settle,Maturity)` returns a matrix of quasi-coupon dates expressed in datetime format (if any inputs are in datetime format).

Successive quasi-coupon dates determine the length of the standard coupon period for the fixed-income security of interest, and do not necessarily coincide with actual coupon payment dates. Quasi-coupon dates are determined regardless of whether the first or last coupon periods are normal, long, or short.

`QuasiCouponDates` has `NUMBONDS` rows and the number of columns is determined by the maximum number of quasi-coupon dates required to hold the bond portfolio. NaNs are padded for bonds which have less than the maximum number quasi-coupon dates. By default, quasi-coupon dates after settlement and on or preceding maturity are returned. If settlement occurs on maturity, and maturity is a quasi-coupon date, then the maturity date is returned.

`QuasiCouponDates = cfdatesq( ___,Period,Basis,EndMonthRule,IssueDate,FirstCouponDate,LastCouponDate,PeriodsBeforeSettle,PeriodsAfterMaturity)` specifies options using one or more optional arguments in addition to the input arguments in the previous syntax.

Examples

Obtain Quasi-Coupon Dates for Fixed-Income Security

Compute the quasi-coupon dates given the `Settle` and `Maturity` dates.

```
QuasiCouponDates = cfdatesq('14-Mar-1997', '30-Nov-1998', 2, 0, 1)
```

```
QuasiCouponDates = 1×4
```

```
    729541    729724    729906    730089
```

If any of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, or `LastCouponDate` are datetime arrays, then `QuasiCouponDates` is returned as a datetime array. For example:

```
QuasiCouponDates = cfdatesq('14-Mar-1997', datetime('30-Nov-1998','Locale','en_US'), 2, 0, 1)
```

```
QuasiCouponDates = 1×4 datetime
    31-May-1997    30-Nov-1997    31-May-1998    30-Nov-1998
```

Input Arguments

Settle — Settlement date

datetime array | string array | date character vector

Settlement date, specified as an NINST-by-1 vector using a datetime array, string array, or date character vectors. **Settle** must be earlier than **Maturity**.

To support existing code, `cfdatesq` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date

datetime array | string array | date character vector

Maturity date, specified as an NINST-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `cfdatesq` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Period — Coupons per year of the bond

2 (semiannual) (default) | vector of positive integers from the set [1,2,3,4,6,12]

(Optional) Coupons per year of the bond, specified as a vector of positive integers from the set [1,2,3,4,6,12].

Data Types: double

Basis — Day-count basis

0 (actual/actual) (default) | positive integers of the set [1...13] | vector of positive integers of the set [1...13]

(Optional) Day-count basis, specified as positive integers using a NINST-by-1 vector.

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)

- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

(Optional) End-of-month rule flag, specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector. This rule applies only when `Maturity` is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: `logical`

IssueDate — Bond issue date

cash flow payment dates are determined from other inputs (default) | `datetime` array | `string` array | `date` character vector

(Optional) Bond Issue date, specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime` array, `string` array, or `date` character vectors.

If you do not specify an `IssueDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cfdatesq` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

FirstCouponDate — Irregular or normal first coupon date

cash flow payment dates are determined from other inputs (default) | `datetime` array | `string` array | `date` character vector

Irregular or normal first coupon date, specified as a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime` array, `string` array, or `date` character vectors.

If you do not specify a `FirstCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cfdatesq` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

LastCouponDate — Irregular or normal last coupon date

cash flow payment dates are determined from other inputs (default) | `datetime` array | `string` array | `date` character vector

Irregular or normal last coupon date, specified as a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector using a datetime array, string array, or date character vectors.

If you do not specify a `LastCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cfdatesq` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

PeriodsBeforeSettle — Number of quasi-coupon dates on or before settlement to include

0 (default) | nonnegative integer

(Optional) Number of quasi-coupon dates on or before settlement to include, specified as a nonnegative integer.

Data Types: `double`

PeriodsAfterMaturity — Number of quasi-coupon dates after maturity to include

0 (default) | nonnegative integer

(Optional) Number of quasi-coupon dates after maturity to include, specified as a nonnegative integer.

Data Types: `double`

Output Arguments

QuasiCouponDates — Quasi-coupon dates

matrix

Quasi-coupon dates, returned as a N-row matrix of dates in datetime format (if any inputs are in datetime format). `QuasiCouponDates` has NUMBONDS rows and the number of columns is determined by the maximum number of quasi-coupon dates required to hold the bond portfolio. NaNs are padded for bonds which have less than the maximum number quasi-coupon dates. By default, quasi-coupon dates after settlement and on or preceding maturity are returned. If settlement occurs on maturity, and maturity is a quasi-coupon date, then the maturity date is returned.

If all of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, or `LastCouponDate` are either strings or date character vectors, then `QuasiCouponDates` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

If any of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, or `LastCouponDate` are datetime arrays, then `QuasiCouponDates` is returned as a datetime array.

More About

Quasi Coupon

A quasi coupon is a term used in the context of certain fixed-income securities, particularly bonds, that do not pay traditional periodic interest payments (coupons) but instead provide returns in a different manner.

A quasi coupon can include returns that are contingent on certain conditions or linked to the performance of an underlying asset.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cfdatesq` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

See Also

`accrfrac` | `cfamounts` | `cftimes` | `cpncount` | `cpndaten` | `cpndatenq` | `cpndatep` | `cpndaysn` | `cpndaysp` | `cpnpersz` | `datetime` | `cpndatepq`

Topics

“Analyzing and Computing Cash Flows” on page 2-11

cfdur

Cash-flow duration and modified duration

Syntax

```
[Duration,ModDuration] = cfdur(CashFlow,Yield)
```

Description

`[Duration,ModDuration] = cfdur(CashFlow,Yield)` calculates the duration and modified duration of a cash flow in periods.

Examples

Compute the Duration and Modified Duration of a Cash Flow

This example shows how to calculate the duration and modified duration of a cash flow, given a cash flow of nine payments of \$2.50 and a final payment \$102.50, with a periodic yield of 2.5%.

```
CashFlow=[2.5 2.5 2.5 2.5 2.5 2.5 2.5 2.5 2.5 102.5];
```

```
[Duration, ModDuration] = cfdur(CashFlow, 0.025)
```

```
Duration =  
8.9709
```

```
ModDuration =  
8.7521
```

Input Arguments

CashFlow — Cash flow

vector of real numbers | matrix of real numbers

Cash flow, specified as a vector or matrix of real numbers. When using a matrix, each column of the matrix is a separate CashFlow.

Data Types: double

Yield — Periodic yield

scalar decimal | vector of decimals

Periodic yield, specified as a scalar decimal or a vector of decimals.

Data Types: double

Output Arguments

Duration — Duration

vector

Duration returned as a scalar or vector.

ModDuration — Modified duration

vector

Modified duration, returned as a scalar or vector.

More About

Cash Flow Duration

Cash flow duration is a measure used in finance to assess the sensitivity of the present value of an investment's cash flows to changes in interest rates.

A higher cash flow duration indicates that the cash flows are received further in the future, making the investment more sensitive to changes in interest rates. Conversely, a lower cash flow duration suggests that cash flows are received sooner, resulting in less sensitivity to interest-rate changes.

Version History

Introduced before R2006a

See Also

bndconvp | bndconvy | bnddurp | bnddury | cfconv

Topics

“Analyzing and Computing Cash Flows” on page 2-11

cfplot

Visualize cash flows of financial instruments

Syntax

```
cfplot(CFlowDates,CFlowAmounts)
cfplot(ax,CFlowDates,CFlowAmounts)
cfplot( ___,Name,Value)

h = cfplot( ___,Name,Value)
[h,axes_handle] = cfplot( ___,Name,Value)
```

Description

`cfplot(CFlowDates,CFlowAmounts)` plots a cash flow diagram for the specified cash flow amounts (`CFlowAmounts`) and dates (`CFlowDates`). The length and orientation of each arrow correspond to the cash flow amount.

`cfplot(ax,CFlowDates,CFlowAmounts)` plots a cash flow diagram for the specified cash flow amounts using an optional `ax` argument.

`cfplot( ___,Name,Value)` plots a cash flow diagram for the specified cash flow amounts (`CFlowAmounts`), dates (`CFlowDates`), and optional name-value pair arguments.

`h = cfplot( ___,Name,Value)` returns the handle to the line objects used in the cash flow diagram.

`[h,axes_handle] = cfplot( ___,Name,Value)` returns the handles to the line objects and the axes using optional name-value pair arguments.

Examples

Plot Cash Flows

Define `CFlowAmounts` and `CFlowDates` using the `cfamounts` function.

```
CouponRate = [0.06; 0.05; 0.03];
Settle = '03-Jun-1999';
Maturity = ['15-Aug-2000'; '15-Dec-2000'; '15-Jun-2000'];
Period = [1; 2; 2]; Basis = [1; 0; 0];
[CFlowAmounts, CFlowDates] = cfamounts(...
    CouponRate, Settle, Maturity, Period, Basis)
```

```
CFlowAmounts = 3×5
```

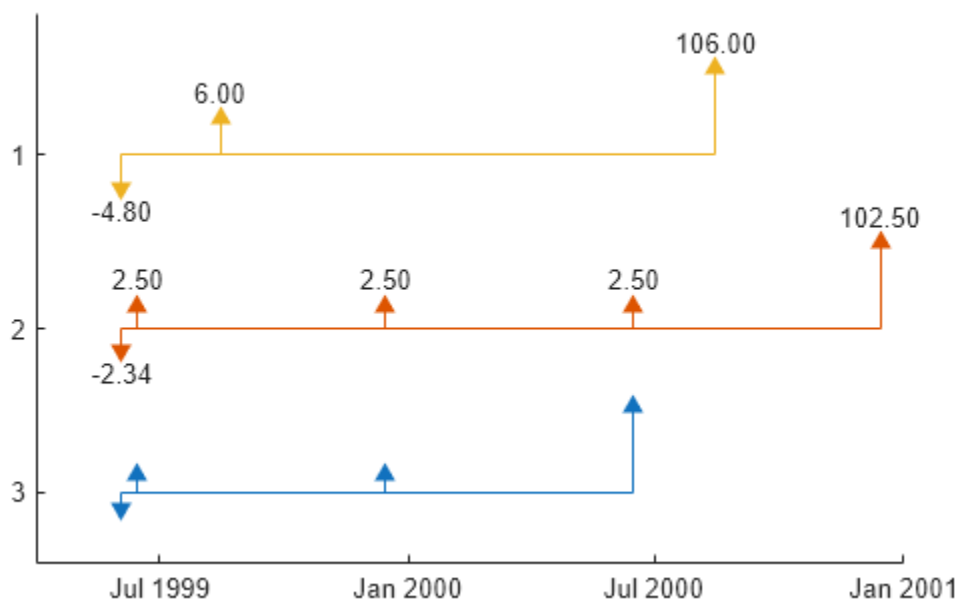
```
-4.8000    6.0000   106.0000         NaN         NaN
-2.3352    2.5000    2.5000    2.5000   102.5000
-1.4011    1.5000    1.5000   101.5000         NaN
```

```
CFlowDates = 3x5
```

730274	730347	730713	NaN	NaN
730274	730286	730469	730652	730835
730274	730286	730469	730652	NaN

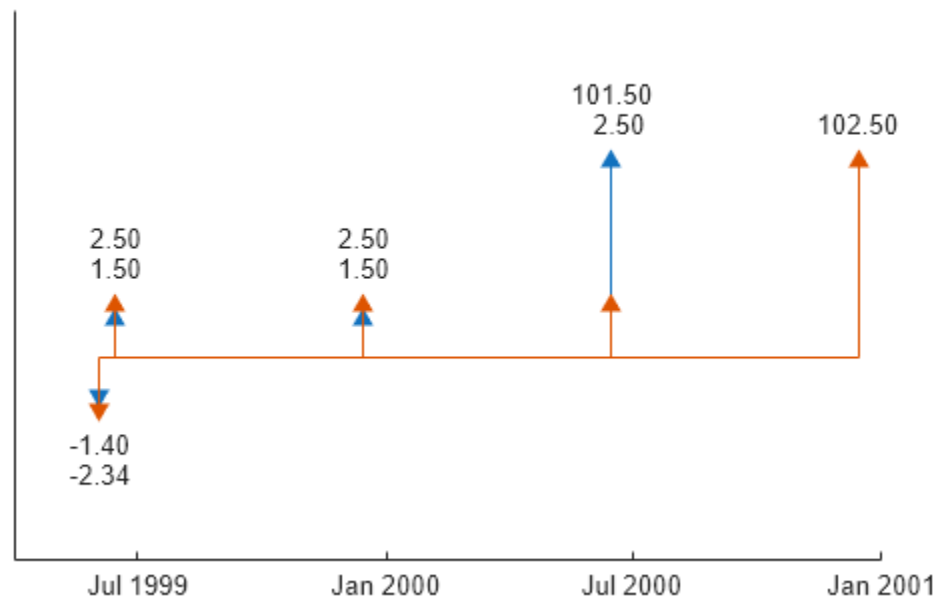
Plot all cash flows on the same axes, and label the first two.

```
cfplot(CFlowDates, CFlowAmounts, 'ShowAmnt', [1 2])
```



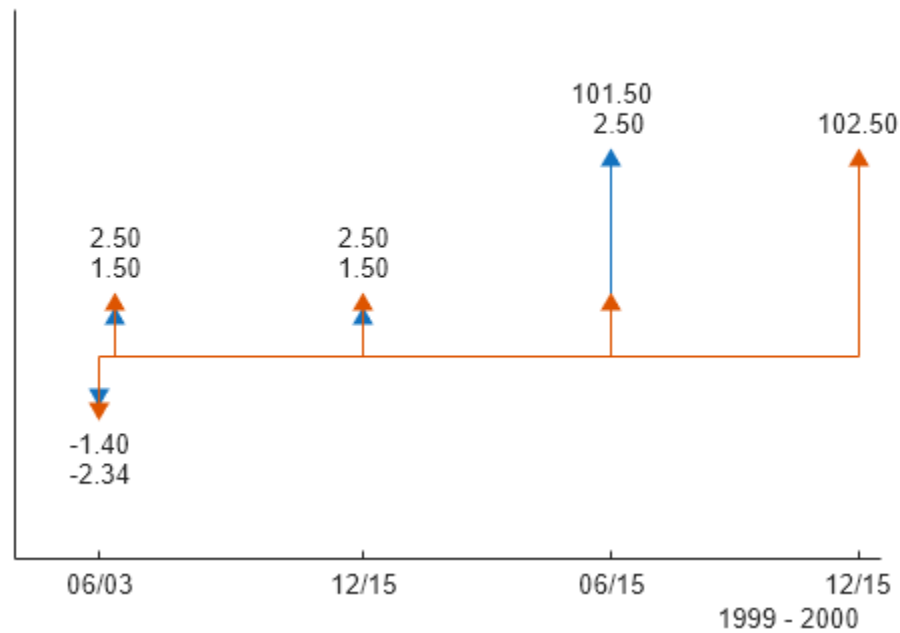
Group the second and third cash flows.

```
figure;  
cfplot(CFlowDates, CFlowAmounts, 'Groups', {[2 3]}, 'ShowAmnt', 1);
```



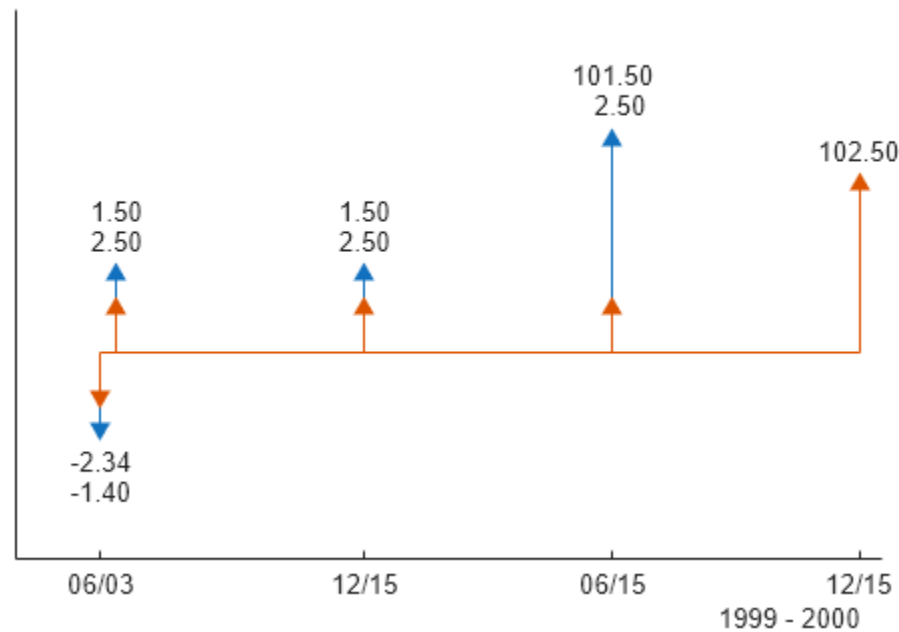
Format the date axis and place ticks on actual cash flow dates.

```
figure;
cfplot(CFlowDates, CFlowAmounts, 'Groups', {[2 3]}, 'ShowAmnt', 1, ...
'DateFormat', 6, 'DateSpacing', 100);
```



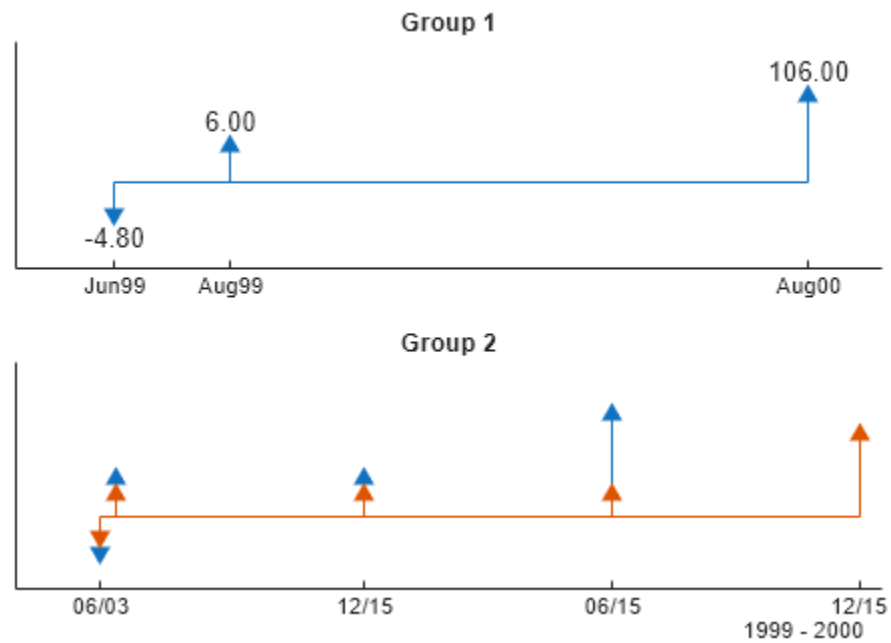
Stack the cash flow arrows occurring on the same dates.

```
figure;
cfplot(CFlowDates, CFlowAmounts, 'Groups', {[2 3]}, 'ShowAmnt', 1, ...
'DateFormat', 6, 'DateSpacing', 100, 'Stacked', 1);
```



Form subplots of multiple groups and add titles using axes handles.

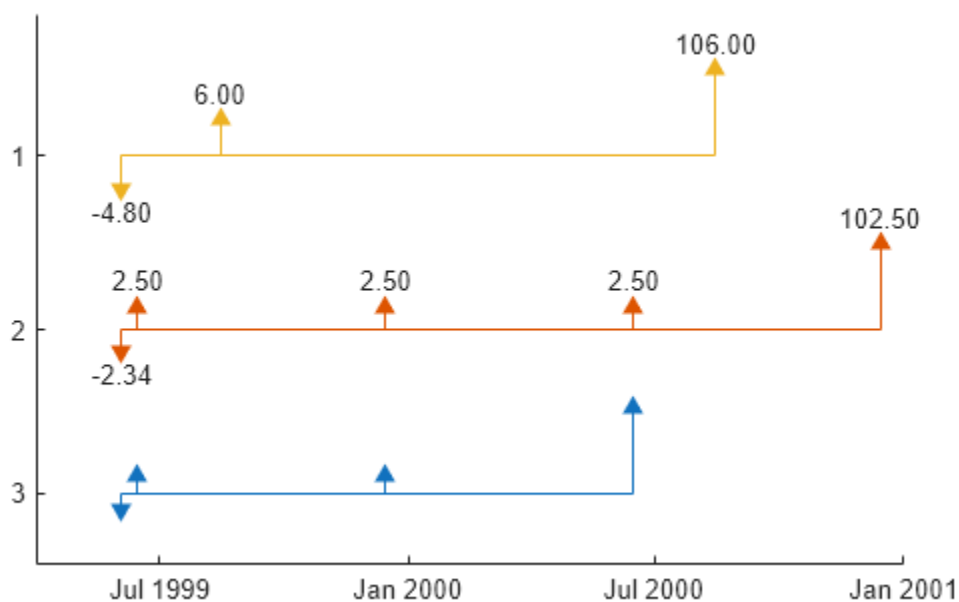
```
figure;
[h, axes_handle] = cfplot(CFlowDates, CFlowAmounts, ...
'Groups', {[1] [2 3]}, 'ShowAmnt', 1, 'Stacked', 2, ...
'DateSpacing', [1 60 2 100], 'DateFormat', [1 12 2 6]);
title(axes_handle(1), 'Group 1', 'FontWeight', 'bold');
title(axes_handle(2), 'Group 2', 'FontWeight', 'bold');
```



Plot Cash Flows Using datetime Input for CFlowDates

Define CFlowDates using datetime input and plot the cash flow.

```
CouponRate = [0.06; 0.05; 0.03];
Settle = '03-Jun-1999';
Maturity = ['15-Aug-2000'; '15-Dec-2000'; '15-Jun-2000'];
Period = [1; 2; 2]; Basis = [1; 0; 0];
[CFlowAmounts, CFlowDates] = cfamounts(...
CouponRate, Settle, Maturity, Period, Basis);
cfplot(datetime(CFlowDates, 'ConvertFrom', 'datenum', 'Locale', 'en_US'), CFlowAmounts, 'ShowAmnt',
```



Plot Cash Flows for Swap

Define the swap using the `swapbyzero` (Financial Instruments Toolbox) function.

```
Settle = datetime(2010,6,8);
RateSpec = intenvset('Rates', [.005 .0075 .01 .014 .02 .025 .03]',...
'StartDates',Settle, 'EndDates',[datetime(2010,12,8),datetime(2011,6,8),datetime(2012,6,8),datet...
Maturity = datetime(2020,9,15);
LegRate = [.025 50];
LegType = [1 0]; % Fixed/Floating
LatestFloatingRate = .005;
[Price, SwapRate, AI, RecCF, RecCFDates, PayCF, PayCFDates] = ...
swapbyzero(RateSpec, LegRate, Settle, Maturity, 'LegType', LegType, ...
'LatestFloatingRate', LatestFloatingRate)
```

```
Price =
-6.7258
```

```
SwapRate =
NaN
```

```
AI =
1.4575
```

```
RecCF = 1x12
```

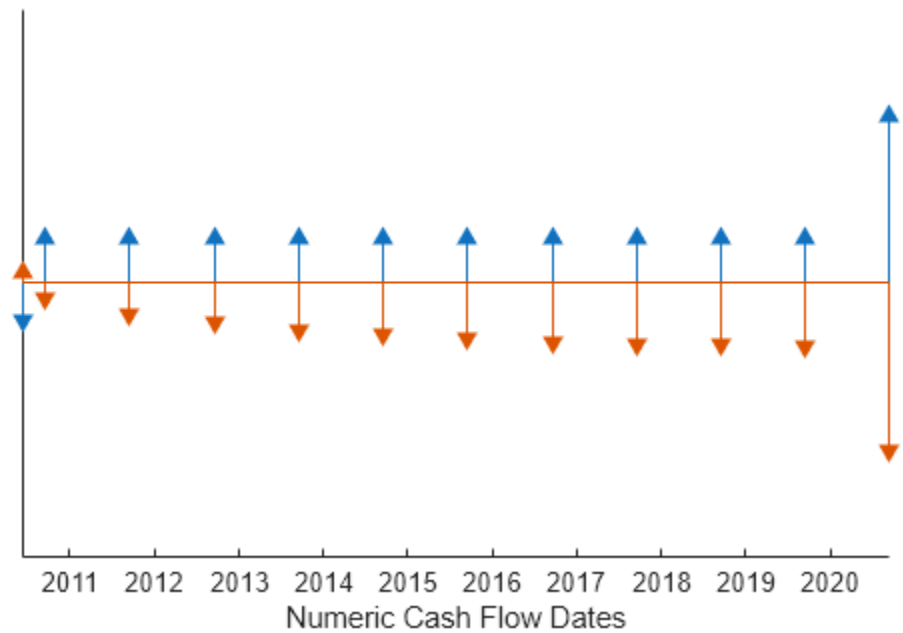
```
-1.8219    2.5000    2.5000    2.5000    2.5000    2.5000    2.5000    2.5000    2.5000    2.5000    2.5000    2.5000
```

```
RecCFDates = 1x12
```

	734297	734396	734761	735127	735492	735857	736222	736588	
PayCF = 1×12									
	-0.3644	0.5000	1.4048	1.9823	2.8436	3.2842	3.8218	4.1733	4.5164
PayCFDates = 1×12									
	734297	734396	734761	735127	735492	735857	736222	736588	

Define CFlowDates and CFlowAmounts for the swap and generate a cash flow plot using cfplot.

```
CFlowDates = [PayCFDates;RecCFDates];
CFlowAmounts = [-PayCF;RecCF];
cfplot(CFlowDates, CFlowAmounts, 'Groups', {[1 2]});
xlabel('Numeric Cash Flow Dates');
```



Input Arguments

CFlowDates — Matrix of dates for cash flows
vector

Matrix of datetime arrays for cash flows, specified as a NINST-by-(Number of cash flows) matrix of cash flow dates using datetime format, with empty entries padded with NaNs.

Each row of the CFlowDates matrix represents an instrument so that CFlowDates(k,:) is the vector of cash flow dates for the kth instrument. Rows are padded with trailing NaNs if the number of cash flows is not the same for all instruments.

`cfamounts` can be used to generate `CFlowDates`.

Data Types: `datetime`

CFlowAmounts — Matrix of cash flow amounts

vector

Matrix of cash flow amounts, specified as a NINST-by-(Number of cash flows) matrix of cash flow amounts, with empty entries padded with NaNs. The `CFlowAmounts` matrix must be the same size as `CFlowDates`.

`cfamounts` can be used to generate `CFlowAmounts`.

Data Types: `double`

ax — Valid axis object

object

(Optional) Valid axis object, specified as an `ax` object that is created using `axes`. The plot will be created in the axes specified by the optional `ax` argument instead of in the current axes (`gca`). The optional argument `ax` can precede any of the input argument combinations.

Data Types: `object`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `cfplot(CFlowDates,CFlowAmounts,'Groups',{[2 3]},'ShowAmnt',1,'DateFormat',6,'DateSpacing',100)`

Groups — Group cash flows

'off' (default) | character vector with value 'off' or 'individual' | cell array of character vectors

Group cash flows, specified as the comma-separated pair consisting of 'Groups' and the following values:

- 'off' — Show all instruments in one set of axes, arranged from top to bottom.
- 'individual' — Generate subplots and plot each instrument in its own axis.
- GRP — Cell array of instrument groups, {Group1, Group2, ... }. This generates subplots and plots each group in each axis. When specifying {Group1, Group2, ... }, each Group must be mutually exclusive vectors of `INSTIndex`. Unspecified instruments are not shown in the grouped plot.

Data Types: `char` | `cell`

Stacked — Stack arrows if cash flows are in same direction on same day

ignored when 'Groups' is 'off', otherwise 'off' (default) | character vector with values 'off', 'all', or 'GRPIndex'

Stack arrows if the cash flows are in the same direction on the same day, specified as the comma-separated pair consisting of **'Stacked'** and the following values:

- **'off'** — For all groups, all arrows originate from the horizontal line.
- **'all'** — For all groups, arrows are stacked if the cash flows are in the same direction on the same day.
- **'GRPIndex'** — For specified groups, arrows are stacked if the cash flows are in the same direction on the same day.

Data Types: char

ShowAmnt — Show amount on arrows

'off' (default) | character vector with values **'off'** or **'individual'** | cell array of character vectors

Show amount on the arrows, specified as the comma-separated pair consisting of **'ShowAmnt'** and the following values:

- **'off'** — Hide cash flow amounts on arrows.
- **'all'** — Show cash flow amounts on arrows.
- **[INSTIndex or GRPIndex]** — Show cash flow amounts for the specified vector of instruments (when **'Groups'** is **'off'**) or groups.

Data Types: char | cell

DateSpacing — Control for date axis tick spacing

'off' (default) | character vector with values **'off'** or **TickDateSpace** | numeric value for **TickDateSpace**

Control for data spacing, specified as the comma-separated pair consisting of **'DateSpacing'** and the following values:

- **'off'** — The date axis ticks are spaced regularly.
- **TickDateSpace** — The date axis ticks are placed on actual cash flow dates. The ticks skip some cash flows if they are less than **TickDateSpace** apart.

Data Types: char | double

DateFormat — Date format

'off' display in **datetime** format (default) | character vector with values **'off'** or **DateFormNum** | numeric value for **DateFormNum**

Date format, specified as the comma-separated pair consisting of **'DateFormat'** and the following values:

- **'off'** — The date axis tick labels (**Xlabel**) are in **datetime** format.
- **DateFormNum** — The date format number (2 = **'mm/dd/yy'**, 6 = **'mm/dd'**, and 10 = **'yyyy'**). Additional values for **DateFormNum** are as follows:

DateFormNum	Example
2	03/01/00
3	Mar

DateFormNum	Example
5	03
6	03/01
7	01
8	Wed
9	W
10	2000
11	00
12	Mar00
17	Q1-00
18	Q1
19	01/03
20	01/03/00
27	Q1-2000
28	Mar2000
29	2000-03-01

Data Types: char | double

Output Arguments

h — Handles to line objects

vector

Handles to line objects, returned as a NINST-by-3 matrix of handles to line objects, containing [hLines, hUArrowHead, hDArrowHead] where:

- hLines — Horizontal and vertical lines used in the cash flow diagram
- hUArrowHead — "Up" arrowheads
- hDArrowHead — "Down" arrowheads

axes_handle — Handles to axes for plot or subplots

vector

Handles to axes for the plot or subplots, returned as a (Number of axes)-by-1 vector of handles to axes.

Version History

Introduced in R2013a

R2025a: Default Display Change for DateFormat

The default display for the DateFormat name-value argument changes from `datenum` to `datetime`.

See Also

`cfamounts` | `cfdates` | `swapbyzero` | `datetime`

Topics

“Analyzing and Computing Cash Flows” on page 2-11

cfport

Portfolio form of cash flow amounts

Syntax

```
[CFBondDate,AllDates,AllTF,IndByBond] = cfport(CFlowAmounts,CFlowDates)
[CFBondDate,AllDates,AllTF,IndByBond] = cfport( ___,TFactors)
```

Description

[CFBondDate,AllDates,AllTF,IndByBond] = cfport(CFlowAmounts,CFlowDates) computes a vector of all cash flow dates of a bond portfolio, and a matrix mapping the cash flows of each bond to those dates. Use the matrix for pricing the bonds against a curve of discount factors.

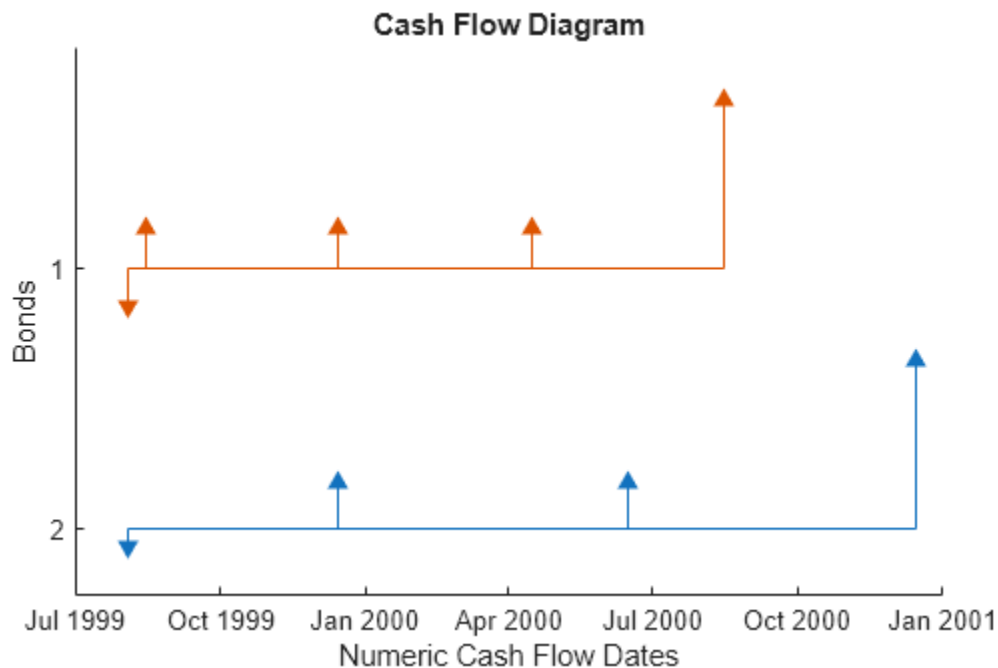
[CFBondDate,AllDates,AllTF,IndByBond] = cfport(___,TFactors) specifies options using one or more optional arguments in addition to the input arguments in the previous syntax.

Examples

Calculate the Cash Flow Amounts, Cash Flow Dates, and Time Factors for Each of Two Bonds

Use the function `cfamounts` to calculate the cash flow amounts, cash flow dates, and time factors for each of two bonds. Then use the function `cfplot` to plot the cash flow diagram.

```
Settle = '03-Aug-1999';
Maturity = ['15-Aug-2000';'15-Dec-2000'];
CouponRate= [0.06; 0.05];
Period = [3;2];
Basis = [1;0];
[CFlowAmounts, CFlowDates, TFactors] = cfamounts(CouponRate,...
Settle, Maturity, Period, Basis);
cfplot(CFlowDates,CFlowAmounts)
xlabel('Numeric Cash Flow Dates')
ylabel('Bonds')
title('Cash Flow Diagram')
```



Call the function `cfport` to map the cash flow amounts to the cash flow dates. Each row in the resultant `CFBondDate` matrix represents a bond. Each column represents a date on which one or more of the bonds has a cash flow. A 0 means the bond did not have a cash flow on that date. The dates associated with the columns are listed in `AllDates`. For example, the first bond had a cash flow of 2.000 on 730347. The second bond had no cash flow on this date. For each bond, `IndByBond` indicates the columns of `CFBondDate`, or dates in `AllDates`, for which a bond has a cash flow.

```
[CFBondDate, AllDates, AllTF, IndByBond] = ...
cfport(CFlowAmounts, CFlowDates, TFactors)
```

```
CFBondDate = 2×7
```

```
-1.8000    2.0000    2.0000    2.0000         0 102.0000         0
-0.6694         0    2.5000         0    2.5000         0 102.5000
```

```
AllDates = 7×1
```

```
730335
730347
730469
730591
730652
730713
730835
```

```
AllTF = 7×1
```

```
0
0.0663
0.7322
1.3989
```

```
1.7322
2.0663
2.7322
```

```
IndByBond = 2×5
```

```
1    2    3    4    6
1    3    5    7   NaN
```

Calculate the Cash Flow Amounts, Cash Flow Dates Using a datetime Array, and Time Factors for Each of Two Bonds

Use the function `cfamounts` to calculate the cash flow amounts, cash flow dates, and time factors for each of two bonds.

```
Settle = datetime(1999,8,3);
Maturity = [datetime(2000,8,15) ; datetime(2000,12,15)];
CouponRate= [0.06; 0.05];
Period = [3;2];
Basis = [1;0];
[CFlowAmounts, CFlowDates, TFactors] = cfamounts(CouponRate,...
Settle, Maturity, Period, Basis);
```

Call the function `cfport` to map the cash flow amounts to the cash flow dates. Each row in the resultant `CFBondDate` matrix represents a bond. Each column represents a date on which one or more of the bonds has a cash flow. A 0 means the bond did not have a cash flow on that date. The dates associated with the columns are listed in `AllDates` returned as a datetime array.

```
[CFBondDate, AllDates, AllTF, IndByBond] = ...
cfport(CFlowAmounts, CFlowDates, TFactors)
```

```
CFBondDate = 2×7
```

```
-1.8000    2.0000    2.0000    2.0000         0 102.0000         0
-0.6694         0    2.5000         0    2.5000         0 102.5000
```

```
AllDates = 7×1 datetime
```

```
03-Aug-1999
15-Aug-1999
15-Dec-1999
15-Apr-2000
15-Jun-2000
15-Aug-2000
15-Dec-2000
```

```
AllTF = 7×1
```

```
0
0.0663
0.7322
1.3989
1.7322
```

```
2.0663
2.7322
```

```
IndByBond = 2×5
```

```
1    2    3    4    6
1    3    5    7   NaN
```

Input Arguments

CFlowAmounts — Cash flow amounts

matrix

Cash flow amounts, specified as number of bonds (NUMBONDS) by number of cash flows (NUMCFS) matrix with entries listing cash flow amounts corresponding to each date in CFlowDates.

Data Types: double

CFlowDates — Cash flow dates

datetime array | string array | date character vector

Cash flow dates, specified as an NUMBONDS-by-NUMCFS matrix with rows listing cash flow dates using a datetime array, string array, or date character vectors for each bond and padded with NaNs. If CFlowDates is a serial date number or a date character vector, AllDates is returned as an array of serial date numbers. If CFlowDates is a datetime array, then AllDates is returned as a datetime array.

To support existing code, cfport also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

TFactors — Time between settlement and the cash flow date

matrix

(Optional) Time between settlement and the cash flow date, specified as an NUMBONDS-by-NUMCFS matrix with entries listing the time between settlement and the cash flow date measured in semiannual coupon periods.

Data Types: double

Output Arguments

CFBondDate — Cash flows indexed by bond and by date

matrix

Cash flows indexed by bond and by date, returned as an NUMBONDS by number of dates (NUMDATES) matrix. Each row contains a bond's cash flow values at the indices corresponding to entries in AllDates. Other indices in the row contain zeros.

AllDates — List of all dates that have any cash flow from the bond portfolio

matrix

List of all dates that have any cash flow from the bond portfolio, returned as an NUMDATES-by-1 matrix. The AllDates matrix is expressed in datetime format (if CFflowDates is in datetime format).

AllTF — Time factors corresponding to the dates in AllDates

matrix

Time factors corresponding to the dates in AllDates, returned as an NUMDATES-by-1 matrix. If TFactors is not entered, AllTF contains the number of days from the first date in AllDates.

IndByBond — Indices by bond

matrix

Indices by bond, returned as an NUMBONDS-by-NUMCFS matrix. The *ith* row contains a list of indices into AllDates where the *ith* bond has cash flows. Since some bonds have more cash flows than others, the matrix is padded with NaNs.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although cfport supports serial date numbers, datetime values are recommended instead. The datetime data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to datetime values, use the datetime function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

cfamounts | cfplot | datetime

Topics

“Analyzing and Computing Cash Flows” on page 2-11

cfprice

Compute price for cash flow given yield to maturity

Syntax

```
Price = cfprice(CFlowAmounts,CFlowDates,Yield,Settle)
Price = cfprice( ___,Name,Value)
```

Description

`Price = cfprice(CFlowAmounts,CFlowDates,Yield,Settle)` computes a price given yield for a cash flow.

`Price = cfprice( ___,Name,Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

Examples

Compute the Price for a Cash Flow Given Yield to Maturity

Use `cfprice` to compute the price for a cash flow given yield to maturity.

Define data for the yield curve.

```
Settle = datetime(2003,7,1);
Yield = .05;
CFAmounts = [30;40;30];
CFDates = [datetime(2004,7,15) ; datetime(2005,7,15) ; datetime(2006,7,15)];
```

Compute the Price.

```
Price = cfprice(CFAmounts, CFDates, Yield, Settle)
```

```
Price = 3×1
```

```
28.4999
36.1689
25.8195
```

Input Arguments

CFlowAmounts — Cash flow amounts

vector

Cash flow amounts, specified as an NINST-by-MOSTCFS matrix. Each row is a list of cash flow values for one instrument. If an instrument has fewer than MOSTCFS cash flows, the end of the row is padded with NaNs.

Data Types: double

CFlowDates — Cash flow dates

datetime array | string array | date character vector

Cash flow dates, specified as an NINST-by-MOSTCFS matrix using a datetime array, string array, or date character vectors. Each entry contains the date of the corresponding cash flow in CFlowAmounts.

To support existing code, cfprice also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Yield — Yields

vector

Yields specified as an NINST-by-1 vector.

Data Types: double

Settle — Settlement date

datetime array | string array | date character vector

Settlement date, specified as an NINST-by-1 vector using a datetime array, string array, or date character vectors. The Settle date is the date on which the cash flows are priced.

To support existing code, cfprice also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Name-Value Pair Arguments

Specify optional pairs of arguments as Name1=Value1,...,NameN=ValueN, where Name is the argument name and Value is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Note An optional input of size NINST-by-1 is also acceptable as a single value applicable to all contracts. Single values are internally expanded to an array of size NINST-by-1.

Example: Price =
cfprice(CFlowAmounts,CFlowDates,Yield,Settle,'Basis',4,'CompoundingFrequency',4)

Basis — Day-count basis

0 (actual/actual) (default) | positive integers of the set [1...13] | vector of positive integers of the set [1...13]

Day-count basis, specified as the comma-separated pair consisting of 'Basis' and a positive integer using a NINST-by-1 vector.

- 0 = actual/actual

- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

CompoundingFrequency — Compounding frequency

SIA bases (0 - 7) and BUS/252 use 2, ICMA bases (8 - 12) use 1 (default) | integer with value of 1, 2, 3, 4, 6, or 12

Compounding frequency for yield calculation, specified as the comma-separated pair consisting of 'CompoundingFrequency' and a scalar or a NUMBONDS-by-1 or 1-by-NUMBONDS vector.

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note By default, SIA bases (0 - 7) and BUS/252 use a semiannual compounding convention and ICMA bases (8 - 12) use an annual compounding convention.

Data Types: `double`

Output Arguments

Price — Price of cash flows

vector

Price of cash flows, returned as an NINST-by-1 vector.

Version History

Introduced in R2012a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cfprice` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

See Also

`cfbyzero` | `cfyield` | `cfsread` | `datetime`

Topics

“Analyzing and Computing Cash Flows” on page 2-11

cfspread

Compute spread over yield curve for cash flow

Syntax

```
Spread = cfspread(RateSpec,Price,CFlowAmounts,CFlowDates,Settle)
Spread = cfspread( ___,Name,Value)
```

Description

`Spread = cfspread(RateSpec,Price,CFlowAmounts,CFlowDates,Settle)` computes spread over a yield curve for a cash flow.

`Spread = cfspread( ___,Name,Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

Examples

Compute Spread Over a Yield Curve for a Cash Flow

Use `cfspread` to compute the spread over a yield curve for a cash flow.

Define data for the yield curve.

```
Settle = datetime(2003,7,1) %datetime('01-Jul-2003');
```

```
Settle = datetime
01-Jul-2003
```

```
CurveDates = daysadd(Settle,360* [.25 .5 1 2 3 5 7 10 20],1);
ZeroRates = [.0089 .0096 .0107 .0130 .0166 .0248 .0306 .0356 .0454]';
```

Compute the `RateSpec`.

```
RateSpec = intenvset('StartDates', Settle, 'EndDates', CurveDates,...
'Rates', ZeroRates)
```

```
RateSpec = struct with fields:
    FinObj: 'RateSpec'
    Compounding: 2
    Disc: [9×1 double]
    Rates: [9×1 double]
    EndTimes: [9×1 double]
    StartTimes: [9×1 double]
    EndDates: [9×1 double]
    StartDates: 731763
    ValuationDate: 731763
    Basis: 0
    EndMonthRule: 1
```

Compute the spread.

```
Price = 98;
CFAmounts = [30;40;30];
CFDates = [datetime(2004,7,15); datetime(2005,7,15) ; datetime(2006,7,15)];

Spread = cfspread(RateSpec, Price, CFAmounts, CFDates, Settle)

Spread = 3×1
103 ×

    -8.7956
    -4.0774
    -3.7073
```

Input Arguments

RateSpec — Interest-rate specification for initial risk-free rate curve

structure

Interest-rate specification for initial risk-free rate curve, specified by the RateSpec obtained from `intenvset`. For information on the interest-rate specification, see `intenvset`.

Data Types: `struct`

Price — Price of cash flows

vector

Price of cash flows, specified as an NINST-by-1 vector.

Data Types: `double`

CFAmounts — Cash flow amounts

vector

Cash flow amounts, specified as an NINST-by-MOSTCFS matrix. Each row is a list of cash flow values for one instrument. If an instrument has fewer than MOSTCFS cash flows, the end of the row is padded with NaNs.

Data Types: `double`

CFDates — Cash flow dates

datetime array | string array | date character vector

Cash flow dates, specified as an NINST-by-MOSTCFS matrix using a datetime array, string array, or date character vectors. Each entry contains the date of the corresponding cash flow in CFAmounts.

To support existing code, `cfspread` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Settle — Settlement date

datetime array | string array | date character vector

Settlement date, specified as an NINST-by-1 vector using a datetime array, string array, or date character vectors. The `Settle` date is the date on which the cash flows are priced.

To support existing code, `cfsspread` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Note An optional input of size NINST-by-1 is also acceptable as a single value applicable to all contracts. Single values are internally expanded to an array of size NINST-by-1.

Example: `Spread = cfsspread(RateSpec,Price,CFlowAmounts,CFlowDates,Settle,'Basis',4)`

Basis — Day-count basis

0 (actual/actual) (default) | positive integers of the set [1...13] | vector of positive integers of the set [1...13]

Day-count basis, specified as the comma-separated pair consisting of `'Basis'` and a positive integer using a NINST-by-1 vector.

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

Output Arguments

Spread — Spread of cash flows over a zero curve

vector

Spread of cash flows over a zero curve, returned as an NINST-by-1 vector. The Spread is expressed in basis points.

Version History

Introduced in R2012a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cfspread` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`cfbyzero` | `cfyield` | `cfprice` | `datetime`

Topics

“Analyzing and Computing Cash Flows” on page 2-11

cfyield

Compute yield to maturity for cash flow given price

Syntax

```
Yield = cfyield(CFlowAmounts,CFlowDates,Price,Settle)
Yield = cfyield( ___,Name,Value)
```

Description

`Yield = cfyield(CFlowAmounts,CFlowDates,Price,Settle)` computes yield to maturity for a cash flow given price.

`Yield = cfyield( ___,Name,Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

Examples

Compute the Yield to Maturity for a Cash Flow When Given a Price

Use `cfyield` to compute yield to maturity for a cash flow when given a price.

Define data for the yield curve and price.

```
Settle = datetime(2003,7,1);
Price = 98;
CFlowAmounts = [30 40 30];
CFlowDates = [datetime(2004,7,15) ; datetime(2005,7,15) ; datetime(2006,7,15) ]';
```

Compute the Yield.

```
Yield = cfyield(CFlowAmounts, CFlowDates, Price, Settle)

Yield =
0.0099
```

Input Arguments

CFlowAmounts — Cash flow amounts

vector

Cash flow amounts, specified as an NINST-by-MOSTCFS matrix. Each row is a list of cash flow values for one instrument. If an instrument has fewer than MOSTCFS cash flows, the end of the row is padded with NaNs.

Data Types: double

CFlowDates — Cash flow dates

datetime array | string array | date character vector

Cash flow dates, specified as an NINST-by-MOSTCFS matrix using a datetime array, string array, or date character vectors. Each entry contains the date of the corresponding cash flow in CFflowAmounts.

To support existing code, cfyield also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Price — Prices

vector

Prices specified as an NINST-by-1 vector.

Data Types: double

Settle — Settlement date

datetime array | string array | date character vector

Settlement date, specified as an NINST-by-1 vector using a datetime array, string array, or date character vectors. The Settle date is the date on which the cash flows are priced.

To support existing code, cfyield also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Name-Value Pair Arguments

Specify optional pairs of arguments as Name1=Value1, ..., NameN=ValueN, where Name is the argument name and Value is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Note An optional input of size NINST-by-1 is also acceptable as a single value applicable to all contracts. Single values are internally expanded to an array of size NINST-by-1.

Example: Yield =

```
cfyield(CFAmounts,CFDates,Yield,Settle,'Basis',4,'CompoundingFrequency',4)
```

Basis — Day-count basis

0 (actual/actual) (default) | positive integers of the set [1...13] | vector of positive integers of the set [1...13]

Day-count basis, specified as the comma-separated pair consisting of 'Basis' and a positive integer using a NINST-by-1 vector.

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365

- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

CompoundingFrequency — Compounding frequency

SIA bases (0 - 7) and BUS/252 use 2, ICMA bases (8 - 12) use 1 (default) | integer with value of 1, 2, 3, 4, 6, or 12

Compounding frequency for yield calculation, specified as the comma-separated pair consisting of 'CompoundingFrequency' and a scalar or a NINST-by-1 or 1-by-NINST vector.

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note By default, SIA bases (0 - 7) and BUS/252 use a semiannual compounding convention and ICMA bases (8 - 12) use an annual compounding convention.

Data Types: double

Output Arguments

Yield — Yield for cash flows

vector

Yield for cash flows, returned as an NINST-by-1 vector.

Version History

Introduced in R2012a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cfyield` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`cfbyzero` | `cfspread` | `cfprice` | `datetime`

Topics

“Analyzing and Computing Cash Flows” on page 2-11

cftimes

Time factors corresponding to bond cash flow dates

Syntax

```
TFactors = cftimes(Settle,Maturity)
TFactors = cftimes(____,Name,Value)
```

Description

`TFactors = cftimes(Settle,Maturity)` determines the time factors corresponding to the cash flows of a bond or set of bonds.

`cftimes` computes the time factor of a cash flow, which is the difference between the settlement date and the cash flow date, in units of semiannual coupon periods. In computing time factors, use SIA actual/actual day count conventions for all time factor calculations.

`TFactors = cftimes(____,Name,Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

Examples

Compute the Time Factor of a Cash Flow

This example shows how to calculate a cash flow time factor.

```
Settle = datetime(1997,3,15);
Maturity = datetime(1999,9,1);
Period = 2;
TFactors = cftimes(Settle, Maturity, Period)
```

```
TFactors = 1×5
```

```
    0.9239    1.9239    2.9239    3.9239    4.9239
```

Input Arguments

Settle — Settlement date

datetime array | string array | date character vector

Settlement date, specified as an NINST-by-1 vector using a datetime array, string array, or date character vectors. `Settle` must be earlier than `Maturity`.

To support existing code, `cftimes` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date

datetime array | string array | date character vector

Maturity date, specified as an NINST-by-1 vector using a datetime array, string array, or date character vectors. The `Settle` date is the date on which the cash flows are priced.

To support existing code, `cftimes` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `TFactors = cftimes(Settle,Maturity,'Period',4)`

Period — Number of coupon payments per year

2 (default) | numeric with values 0, 1, 2, 3, 4, 6 or 12

Number of coupon payments per year, specified as the comma-separated pair consisting of 'Period' and a scalar or a NUNBONDS-by-1 or 1-by-NUNBONDS vector using the values: 0, 1, 2, 3, 4, 6, or 12.

Data Types: double

Basis — Day-count basis

0 (actual/actual) (default) | positive integers of the set [1...13] | vector of positive integers of the set [1...13]

Day-count basis, specified as the comma-separated pair consisting of 'Basis' and a positive integer using a NINST-by-1 vector.

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as the comma-separated pair consisting of 'EndMonthRule' and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector. This rule applies only when `Maturity` is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: `logical`

IssueDate — Bond issue date

`datetime` array | `string` array | `date` character vector

Bond Issue date, specified as the comma-separated pair consisting of 'IssueDate' and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime` array, `string` array, or `date` character vectors.

If you do not specify an `IssueDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cftimes` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

FirstCouponDate — Irregular or normal first coupon date

`datetime` array | `string` array | `date` character vector

Irregular or normal first coupon date, specified as the comma-separated pair consisting of 'FirstCouponDate' and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime` array, `string` array, or `date` character vectors.

If you do not specify a `FirstCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cftimes` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

LastCouponDate — Irregular or normal last coupon date

`datetime` array | `string` array | `date` character vector

Irregular or normal last coupon date, specified as the comma-separated pair consisting of 'LastCouponDate' and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime` array, `string` array, or `date` character vectors.

If you do not specify a `LastCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cftimes` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

StartDate — Forward starting date of payments

`datetime array` | `string array` | `date character vector`

Forward starting date of payments, specified as the comma-separated pair consisting of 'StartDate' and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector using a `datetime` array, `string array`, or `date character vectors`. The `StartDate` is when a bond actually starts (the date from which a bond cash flow is considered). To make an instrument forward-starting, specify this date as a future date.

If you do not specify a `StartDate`, the effective start date is the `Settle` date.

To support existing code, `cftimes` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

CompoundingFrequency — Compounding frequency for yield calculation

`SIA bases uses 2`, `ICMA bases uses 1 (default)` | integer with value of 1, 2, 3, 4, 6, or 12

Compounding frequency for yield calculation, specified as the comma-separated pair consisting of 'CompoundingFrequency' and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector.

- 1 — Annual compounding
- 2 — Semiannual compounding
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note By default, `SIA` bases (0-7) and `BUS/252` use a semiannual compounding convention and `ICMA` bases (8-12) use an annual compounding convention.

Data Types: `double`

DiscountBasis — Basis used to compute the discount factors for computing the yield

`SIA uses 0 (default)` | integers of the set `[0...13]` | vector of integers of the set `[0...13]`

Basis used to compute the discount factors for computing the yield, specified as the comma-separated pair consisting of 'DiscountBasis' and a scalar or a `NUMBONDS-by-1` or `1-by-NUMBONDS` vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365

- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If a SIA day-count basis is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the default behavior is for SIA bases to use the actual/actual day count to compute discount factors.

If an ICMA day-count basis or BUS/252 is defined in the `Basis` input argument and there is no value assigned for `DiscountBasis`, the specified bases from the `Basis` input argument are used.

Data Types: `double`

Output Arguments

TFactors — Time to cash flow

`matrix`

Time to cash flow, returned as an `NUMBONDS` rows. The number of columns is determined by the maximum number of cash flow payment dates required to hold the bond portfolio. NaNs are padded for bonds which have less than the maximum number of cash flow payment dates.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cftimes` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

y =

2021

There are no plans to remove support for serial date number inputs.

References

- [1] Krgin, Dragomir. *Handbook of Global Fixed Income Calculations*. John Wiley & Sons, 2002.
- [2] Mayle, Jan. "Standard Securities Calculations Methods: Fixed Income Securities Formulas for Analytic Measures." SIA, Vol 2, Jan 1994.
- [3] Stigum, Marcia, and Franklin Robinson. *Money Market and Bond Calculations*. McGraw-Hill, 1996.

See Also

accrfrac | cfdates | cfamounts | cpncount | cpndaten | cpndatenq | cpndatep | cpndatepq | cpndaysn | cpndaysp | date2time

Topics

"Analyzing and Computing Cash Flows" on page 2-11

chaikosc

Chaikin oscillator

Syntax

```
chosc = chaikosc(Data)
```

Description

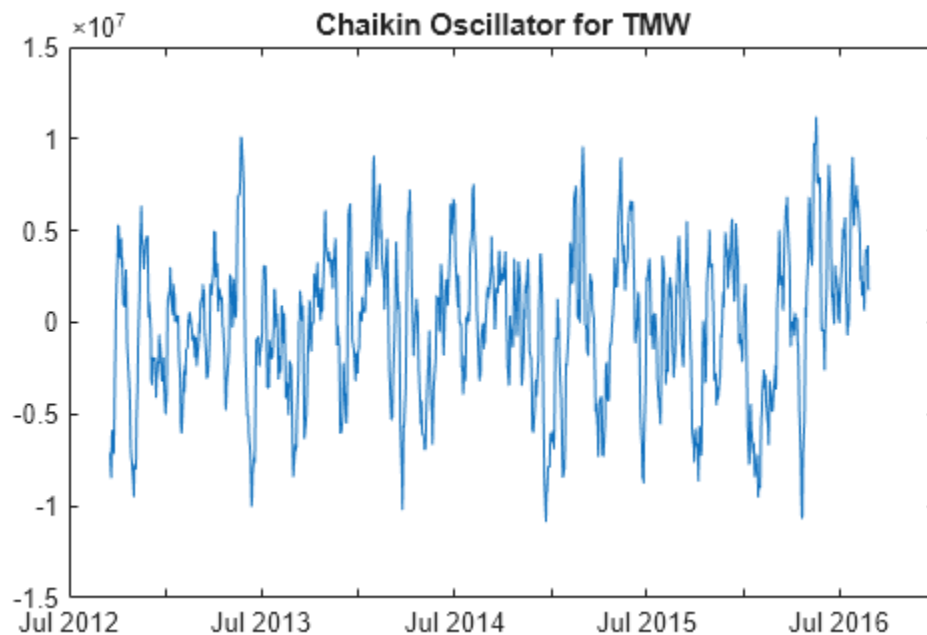
`chosc = chaikosc(Data)` calculates the Chaikin oscillator.

Examples

Calculate the Chaikin Oscillator for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat  
oscillator = chaikosc(TMW);  
plot(oscillator.Time, oscillator.ChaikinOscillator)  
title('Chaikin Oscillator for TMW')
```



Input Arguments

Data — Data with high, low, close, and volume information

matrix | table | timetable

Data with high, low, close, and volume information, specified as a matrix, table, or timetable. For matrix input, Data is an M-by-4 matrix of high, low, close prices and volume. Timetables and tables with M rows must contain variables named 'High', 'Low', 'Close', and 'Volume' (case insensitive).

Data Types: double | table | timetable

Output Arguments

chosc — Chaikin oscillator

matrix | table | timetable

Chaikin oscillator, returned with the same number of rows (M) and type (matrix, table, or timetable) as the input Data.

More About

Chaikin oscillator

The Chaikin oscillator is a technical analysis indicator developed that measures the momentum of the Accumulation/Distribution (A/D) line.

The Chaikin oscillator helps traders identify potential buy and sell signals by assessing the relationship between price and volume, thus providing insights into market sentiment.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the Data input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 91-94.

See Also

adline | timetable | table | chaikvolat

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

chaikvolat

Chaikin volatility

Syntax

```
volatility = chaikvolat(Data)  
volatility = chaikvolat( ___,Name,Value)
```

Description

`volatility = chaikvolat(Data)` calculates the Chaikin volatility from a data series of high and low stock prices.

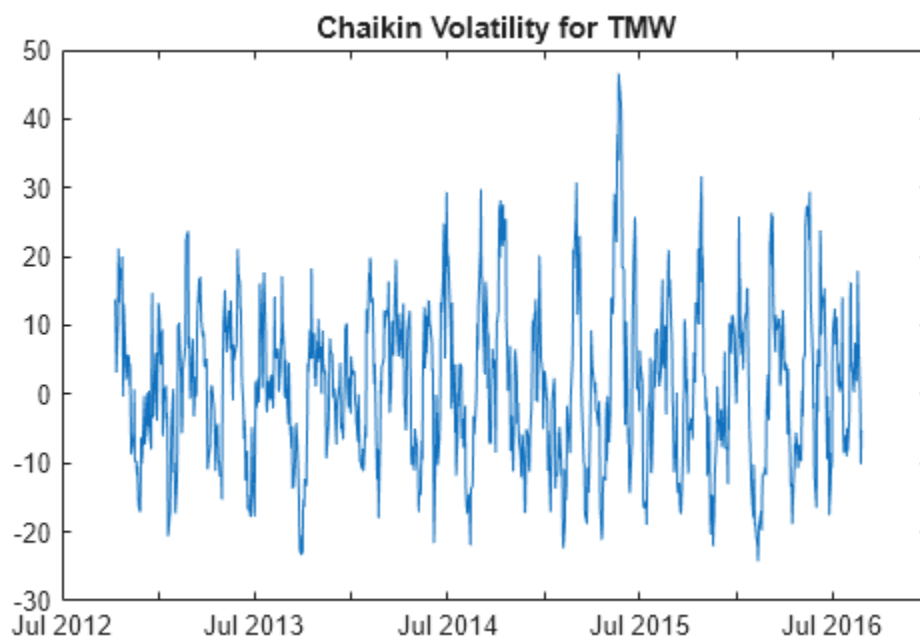
`volatility = chaikvolat( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Calculate the Chaikin Volatility for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat  
volatility = chaikvolat(TMW,'NumPeriods',14,'WindowSize',14);  
plot(volatility.Time,volatility.ChaikinVolatility)  
title('Chaikin Volatility for TMW')
```



Input Arguments

Data — Data with high, low, open, close information

matrix | table | timetable

Data with high, low, open, close information, specified as a vector, matrix, table, or timetable. For matrix input, **Data** is an M-by-2 matrix of high and low prices stored in the first and second columns. Timetables and tables with M rows must contain variables named 'High' and 'Low' (case insensitive).

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as **Name1=Value1, ..., NameN=ValueN**, where **Name** is the argument name and **Value** is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `volatility = chaikvolat(TMW,'NumPeriods',10,'WindowSize',10)`

NumPeriods — Period difference

10 (default) | positive integer

Period difference, specified as the comma-separated pair consisting of 'NumPeriods' and a scalar positive integer.

Data Types: double

WindowSize — Length of the exponential moving average in periods

10 (default) | positive integer

Length of the exponential moving average in periods, specified as the comma-separated pair consisting of 'WindowSize' and a scalar positive integer.

Data Types: double

Output Arguments

volatility — Chaikin volatility

matrix | table | timetable

Chaikin volatility, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input **Data**.

More About

Chaikin volatility

Chaikin volatility calculates the Chaikin's volatility from the series of high and low stock prices.

Chaikin volatility is calculated by taking the difference between the highest high and the lowest low over a specified period and then smoothing this difference using an exponential moving average

(EMA). By default, Chaikin's volatility values are based on a 10-period exponential moving average and 10-period difference.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the Data input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 304–305.

See Also

timetable | table | chaikosc

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

checkFeasibility

Check feasibility of input portfolios against portfolio object

Syntax

```
status = checkFeasibility(obj,pwgt)
```

Description

`status = checkFeasibility(obj,pwgt)` checks the feasibility of input portfolios against a portfolio object.

Use the `checkFeasibility` function with a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object to check the feasibility of input portfolios against a portfolio object. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

Examples

Determine If the Portfolio Is Feasible for a Portfolio Object

Given portfolio `p`, determine if `p` is feasible.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontier(p);

checkFeasibility(p, pwgt)

ans = 1×10 logical array

     1     1     1     1     1     1     1     1     1     1
```

Determine If the Portfolio Is Feasible for a PortfolioCVaR Object

Given portfolio `p`, determine if `p` is feasible.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
```

```

        0.00408 0.0289 0.0204 0.0119;
        0.00192 0.0204 0.0576 0.0336;
        0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt = estimateFrontier(p);

checkFeasibility(p, pwgt)

ans = 1×10 logical array

     1     1     1     1     1     1     1     1     1     1

```

Determine If the Portfolio Is Feasible for a PortfolioMAD Object

Given portfolio p, determine if p is feasible.

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

pwgt = estimateFrontier(p);

checkFeasibility(p, pwgt)

ans = 1×10 logical array

     1     1     1     1     1     1     1     1     1     1

```

Input Arguments

obj — Object for portfolio
object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

pwgt — Portfolios to check
`matrix`

Portfolios to check, specified as a `NumAssets`-by-`NumPorts` matrix.

Data Types: `double`

Output Arguments

status — Indicator if portfolio is feasible
`row vector`

Indicator if portfolio is feasible, returned as a row vector of `NumPorts` indicators that are `true` if portfolio is feasible and `false` otherwise.

Note By definition, any portfolio set must be nonempty and bounded. If the set is empty, no portfolios can be feasible. Use `estimateBounds` to test for nonempty and bounded sets.

Feasibility status is returned for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

More About

Feasibility

Feasibility in portfolio optimization ensures that the proposed investment strategies are practical and can be executed within the constraints set by the investor's preferences, risk tolerance, and regulatory requirements.

In portfolio optimization, feasible solutions are those that meet all specified constraints.

Tips

- You can also use dot notation to check the feasibility of input portfolios against a portfolio object.
`status = obj.checkFeasibility(pwgt);`
- The constraint tolerance to assess whether a constraint is satisfied is obtained from the hidden property `obj.defaultTolCon`.

Version History

Introduced in R2011a

See Also

`estimateBounds`

Topics

"Validate the Portfolio Problem for Portfolio Object" on page 4-94

"Validate the CVaR Portfolio Problem" on page 5-81

"Validate the MAD Portfolio Problem" on page 6-80

"Portfolio Optimization Examples Using Financial Toolbox" on page 4-161

"Portfolio Optimization Theory" on page 4-4

convert2sur

Convert multivariate normal regression model to seemingly unrelated regression (SUR) model

Syntax

```
DesignSUR = convert2sur(Design,Group)
```

Description

`DesignSUR = convert2sur(Design,Group)` converts a multivariate normal regression model into a seemingly unrelated regression model with a specified grouping of the data series.

Examples

Use convert2sur to Estimate Stock Alpha and Beta Values

This example shows a CAPM demonstration using 6 stocks and 60 months of simulated asset returns, where the model for each stock is $\text{AssetReturn} = \text{Alpha} * 1 + \text{CashReturn} + \text{Beta} * (\text{MarketReturn} - \text{CashReturn}) + \text{Noise}$ and the parameters to estimate are Alpha and Beta.

Using simulated data, where the Alpha estimate(s) are displayed in the first row(s) and the Beta estimate(s) are display in the second row(s).

```
Market = (0.1 - 0.04) + 0.17*randn(60, 1);
Asset = (0.1 - 0.04) + 0.35*randn(60, 6);

Design = cell(60, 1);
for i = 1:60
    Design{i} = repmat([ 1, Market(i) ], 6, 1);
end
```

Obtain the aggregate estimates for all stocks.

```
[Param, Covar] = mvnrmls(Asset, Design);

disp({'All 6 Assets Combined'});
    {'All 6 Assets Combined'}

disp(Param);

    0.0233
    0.1050
```

Estimate parameters for individual stocks using `convert2sur`

```
Group = 1:6;
DesignSUR = convert2sur(Design, Group);
[Param, Covar] = mvnrmls(Asset, DesignSUR);
Param = reshape(Param, 2, 6);

disp({ 'A', 'B', 'C', 'D', 'E', 'F' });
```

```

        {'A'}    {'B'}    {'C'}    {'D'}    {'E'}    {'F'}
disp(Param);

    0.0144    0.0270    0.0046    0.0419    0.0376    0.0291
    0.3264   -0.1716    0.3248   -0.0630   -0.0001    0.0637

```

Estimate parameters for pairs of stocks by forming groups.

```

disp({'A & B', 'C & D', 'E & F'});

    {'A & B'}    {'C & D'}    {'E & F'}

Group = { [1,2 ],[3,4],[5,6]};
DesignSUR = convert2sur(Design, Group);
[Param, Covar] = mvnrmls(Asset, DesignSUR);

Param = reshape(Param, 2, 3);

disp(Param);

    0.0186    0.0190    0.0334
    0.0988    0.1757    0.0293

```

Input Arguments

Design — Data series

matrix | cell array

Data series, specified as a matrix or a cell array that depends on the number of data series **NUMSERIES**.

- If **NUMSERIES** = 1, **convert2sur** returns the **Design** matrix.
- If **NUMSERIES** > 1, **Design** is a cell array with **NUMSAMPLES** cells, where each cell contains a **NUMSERIES**-by-**NUMPARAMS** matrix of known values.

Data Types: double | cell

Group — Grouping for data series

vector | cell array

Grouping for data series, specified using separate parameters for each group. Specify groups either by series or by groups:

- To identify groups by series, construct an index vector that has **NUMSERIES** elements. Element *i* = 1, ..., **NUMSERIES** in the vector, and has the index *j* = 1, ..., **NUMGROUPS** of the group in which series *i* is a member.
- To identify groups by groups, construct a cell array with **NUMGROUPS** elements. Each cell contains a vector with the indexes of the series that populate a given group.

In either case, the number of series is **NUMSERIES** and the number of groups is **NUMGROUPS**, with $1 \leq \text{NUMGROUPS} \leq \text{NUMSERIES}$.

Data Types: double | cell

Output Arguments

DesignSUR — Seemingly unrelated regression model with a specified grouping of the data series

matrix | cell array

Seemingly unrelated regression model with a specified grouping of the data series, returned as either a matrix or a cell array that depends on the value of **NUMSERIES**.

- If **NUMSERIES** = 1, **DesignSUR** = **Design**, which is a **NUMSAMPLES**-by-**NUMPARAMS** matrix.
- If **NUMSERIES** > 1 and **NUMGROUPS** groups are to be formed, **Design** is a cell array with **NUMSAMPLES** cells, where each cell contains a **NUMSERIES**-by-**(NUMGROUPS * NUMPARAMS)** matrix of known values.

The original collection of parameters that are common to all series are replicated to form collections of parameters for each group.

More About

Multivariate Normal Regression

Multivariate normal regression is a statistical modeling approach that extends traditional regression techniques to situations where the dependent variable is multivariate, meaning it consists of multiple correlated outcomes.

Multivariate normal regression is particularly useful in fields where researchers are interested in understanding the relationships between several response variables and one or more predictor variables.

Seemingly Unrelated Regression Model

The Seemingly Unrelated Regression (SUR) model is a statistical method used in econometrics to estimate multiple regression equations that are related to each other through their error terms.

The SUR model allows for the simultaneous estimation of these equations, taking into account the potential correlation between the error terms of different equations. This can lead to more efficient estimates compared to estimating each equation independently.

Version History

Introduced in R2006a

See Also

`ecmnfish` | `mvnrfish`

Topics

“Seemingly Unrelated Regression Without Missing Data” on page 9-17

“Multivariate Normal Linear Regression” on page 9-2

corr2cov

Convert standard deviation and correlation to covariance

Syntax

```
ExpCovariance = corr2cov(ExpSigma)
ExpCovariance = corr2cov( ___, ExpCorrC)
```

Description

`ExpCovariance = corr2cov(ExpSigma)` converts standard deviation and correlation to covariance.

`ExpCovariance = corr2cov( ___, ExpCorrC)` specifies options using one or more optional arguments in addition to the input arguments in the previous syntax.

Examples

Convert Standard Deviation and Correlation to Covariance

This example shows how to convert standard deviation and correlation to covariance.

```
ExpSigma = [0.5  2.0];
ExpCorrC = [1.0 -0.5
            -0.5  1.0];

ExpCovariance = corr2cov(ExpSigma, ExpCorrC)

ExpCovariance = 2×2

    0.2500    -0.5000
   -0.5000    4.0000
```

Input Arguments

ExpSigma — Standard deviations of each process

vector

Standard deviations of each process, specified as a vector of length *n* with the standard deviations of each process. *n* is the number of random processes.

Data Types: double

ExpCorrC — Correlation matrix

matrix

(Optional) Correlation matrix, specified as an n-by-n correlation coefficient matrix. A correlation coefficient is a statistic in which the covariance is scaled to a value between minus one (perfect negative correlation) and plus one (perfect positive correlation).

If `ExpCorrC` is not specified, the processes are assumed to be uncorrelated, and the identity matrix is used.

Data Types: double

Output Arguments

ExpCovariance — Covariance matrix

matrix

Covariance matrix, returned as an n-by-n covariance matrix, where n is the number of processes.

The (i,j) entry is the expectation of the i 'th fluctuation from the mean times the j 'th fluctuation from the mean.

$\text{ExpCov}(i,j) = \text{ExpCorrC}(i,j) * \text{ExpSigma}(i) * \text{ExpSigma}(j)$

Version History

Introduced before R2006a

See Also

`cov2corr` | `ewstats` | `nearcorr`

cov2corr

Convert covariance to standard deviation and correlation coefficient

Syntax

```
[ExpSigma,ExpCorrC] = cov2corr(ExpCovariance)
```

Description

[ExpSigma,ExpCorrC] = cov2corr(ExpCovariance) converts covariance to standard deviations and correlation coefficients.

Examples

Convert Covariance to Standard Deviations and Correlation Coefficients

This example shows how to convert a covariance matrix to standard deviations and correlation coefficients.

```
ExpCovariance = [0.25 -0.5
                 -0.5  4.0];
```

```
[ExpSigma, ExpCorrC] = cov2corr(ExpCovariance)
```

```
ExpSigma = 1×2
```

```
    0.5000    2.0000
```

```
ExpCorrC = 2×2
```

```
    1.0000   -0.5000
   -0.5000    1.0000
```

Input Arguments

ExpCovariance — Covariance matrix

matrix

Covariance matrix, specified as an n-by-n covariance matrix, where n is the number of random processes. For an example, see cov or ewstats.

Data Types: double

Output Arguments

ExpSigma — Standard deviation of each process

vector

Standard deviation of each process, returned as an 1-by-n vector.

The entries of `ExpCorrC` range from 1 (completely correlated) to -1 (completely anti-correlated). A value of 0 in the (i,j) entry indicates that the i 'th and j 'th processes are uncorrelated.

```
ExpSigma(i) = sqrt( ExpCovariance(i,i) );  
ExpCorrC(i,j) = ExpCovariance(i,j)/( ExpSigma(i)*ExpSigma(j) );
```

Data Types: double

ExpCorrC — Correlation coefficients

matrix

Correlation coefficients, returned as an n-by-n matrix.

Version History

Introduced before R2006a

See Also

`corr2cov` | `corrcoef` | `ewstats` | `nearcorr`

cpncount

Coupon payments remaining until maturity

Syntax

```
NumCouponsRemaining = cpncount(Settle,Maturity)
NumCouponsRemaining = cpncount( ___,Period,Basis,EndMonthRule,IssueDate,
FirstCouponDate,LastCouponDate)
```

Description

`NumCouponsRemaining = cpncount(Settle,Maturity)` returns the whole number of coupon payments between the `Settle` and `Maturity` dates for a coupon bond or set of bonds. Coupons falling on or before `Settle` are not counted, except for the `Maturity` payment which is always counted.

Required input arguments must be number of bonds, `NUMBONDS-by-1` or `1-by-NUMBONDS`, conforming vectors or scalars.

`NumCouponsRemaining = cpncount( ___,Period,Basis,EndMonthRule,IssueDate,FirstCouponDate,LastCouponDate)` returns the whole number of coupon payments between the `Settle` and `Maturity` dates for a coupon bond or set of bonds using optional input arguments.

Optional input arguments must be either `NUMBONDS-by-1` or `1-by-NUMBONDS` conforming vectors, scalars, or empty matrices.

Examples

Find Coupon Payments Remaining Until Maturity

This example shows how to find the coupon payments remaining until maturity.

```
NumCouponsRemaining = cpncount(datetime(1997,3,14), datetime(2000,11,30), 2, 0, 0)
```

```
NumCouponsRemaining =
8
```

Find Coupon Payments Remaining Until Maturity for Different Maturity Dates

This example shows how to find the coupon payments remaining until maturity, given three coupon bonds with different maturity dates and the same default arguments.

```
Maturity = [datetime(2000,9,30) ; datetime(2001,10,31) ; datetime(2002,11,30)];
NumCouponsRemaining = cpncount(datetime(1997,9,14), Maturity)
```

```
NumCouponsRemaining = 3×1
```

9
11

Input Arguments

Settle — Settlement date

datetime array | string array | date character vector

Settlement date, specified as a NBONDS-by-1 vector using a datetime array, string array, or date character vectors. **Settle** must be earlier than **Maturity**.

To support existing code, **cpncount** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date

datetime array | string array | date character vector

Maturity date, specified as a NBONDS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, **cpncount** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Period — Coupons per year of the bond

2 (semiannual) (default) | vector of positive integers from the set [1,2,3,4,6,12]

Coupons per year of the bond, specified as a vector of positive integers from the set [1,2,3,4,6,12].

Data Types: double

Basis — Day-count basis of the bond

0 (actual/actual) (default) | numeric with value 0 through 13 | vector of numerics with values 0 through 13

Day-count basis of the bond, specified as an integer with a value of 0 through 13 or an N-by-1 vector of integers with values of 0 through 13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)

- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

EndMonthRule — End-of-month rule flag for month having 30 or fewer days

1 (in effect) (default) | nonnegative integer [0, 1]

End-of-month rule flag for month having 30 or fewer days, specified as scalar nonnegative integer [0, 1] or a using an N-by-1 vector of values. This rule applies only when `Maturity` is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond’s coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond’s coupon payment date is always the last actual day of the month.

Data Types: `logical`

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond issue date, specified as a NBONDS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `cpncount` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

FirstCouponDate — Date when bond makes first coupon payment

datetime array | string array | date character vector

Date when a bond makes its first coupon payment, specified as a NBONDS-by-1 vector using a datetime array, string array, or date character vectors.

`FirstCouponDate` is used when a bond has an irregular first coupon period. When `FirstCouponDate` and `LastCouponDate` are both specified, `FirstCouponDate` takes precedence in determining the coupon payment structure. If you do not specify a `FirstCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cpncount` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

LastCouponDate — Last coupon date of bond before maturity date

datetime array | string array | date character vector

Last coupon date of a bond before maturity date, specified as a NBONDS-by-1 vector using a datetime array, string array, or date character vectors.

LastCouponDate is used when a bond has an irregular last coupon period. In the absence of a specified FirstCouponDate, a specified LastCouponDate determines the coupon structure of the bond. The coupon structure of a bond is truncated at the LastCouponDate, regardless of where it falls, and is followed only by the bond's maturity cash flow date. If you do not specify a LastCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, cpncount also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Output Arguments

NumCouponsRemaining — Whole number of coupon payments between the settlement and maturity dates for a coupon bond or set of bonds

vector

Whole number of coupon payments between the settlement and maturity dates for a coupon bond or set of bonds, returned as an NBONDS-by-1 vector.

Coupons falling on or before settlement are not counted, except for the maturity payment which is always counted.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although cpncount supports serial date numbers, datetime values are recommended instead. The datetime data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to datetime values, use the datetime function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

accrfrac | cfamounts | cfdates | cftimes | cpndaten | cpndatenq | cpndatep | cpndaysn | cpndaysp | cpnpersz | datetime | cpndatepq

Topics

"Pricing and Computing Yields for Fixed-Income Securities" on page 2-15

cpndaten

Next coupon date for fixed-income security

Syntax

```
NextCouponDate = cpndaten(Settle,Maturity)
NextCouponDate = cpndaten( ___,Period,Basis,EndMonthRule,IssueDate,
FirstCouponDate,LastCouponDate)
```

Description

`NextCouponDate = cpndaten(Settle,Maturity)` returns the next coupon date after the `Settle` date. This function finds the next coupon date whether or not the coupon structure is synchronized with the `Maturity` date.

Required input arguments must be number of bonds, `NUMBONDS-by-1` or `1-by-NUMBONDS`, conforming vectors or scalars.

`NextCouponDate = cpndaten( ___,Period,Basis,EndMonthRule,IssueDate,FirstCouponDate,LastCouponDate)` returns the next coupon date after the `Settle` date using optional input arguments.

Optional input arguments must be either `NUMBONDS-by-1` or `1-by-NUMBONDS` conforming vectors, scalars, or empty matrices.

If all the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are either strings or date character vectors, then `NextCouponDate` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

If any of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are datetime arrays, then `NextCouponDate` is returned as a datetime array.

Examples

Calculate the Next Coupon Date After the Settlement Date

Determine the `NextCouponDate` when using character vectors for input arguments.

```
NextCouponDate = cpndaten('14-Mar-1997', '30-Nov-2000', 2, 0, 0);
datestr(NextCouponDate)
```

```
ans =
'30-May-1997'
```

Determine the `NextCouponDate` when using a datetime array for input arguments.

```
NextCouponDate = cpndaten(datetime(1997,3,14),datetime(2000,11,30), 2, 0, 0)
```

```
NextCouponDate = datetime
    30-May-1997
```

Determine the `NextCouponDate` when using character vectors for input arguments and the optional argument for `EndMonthRule`.

```
NextCouponDate = cpndaten('14-Mar-1997', '30-Nov-2000', 2, 0, 1);
datestr(NextCouponDate)

ans =
'31-May-1997'
```

Determine the `NextCouponDate` when using an input vector for `Maturity`.

```
Maturity = ['30-Sep-2000'; '31-Oct-2000'; '30-Nov-2000'];
NextCouponDate = cpndaten('14-Mar-1997', Maturity);
datestr(NextCouponDate)

ans = 3×11 char array
'31-Mar-1997'
'30-Apr-1997'
'31-May-1997'
```

Input Arguments

Settle — Settlement date

datetime array | string array | date character vector

Settlement date, specified as a `NUMBONDS`-by-1 vector using a datetime array, string array, or date character vectors. `Settle` must be earlier than `Maturity`.

To support existing code, `cpndaten` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date

datetime array | string array | date character vector

Maturity date, specified as a `NUMBONDS`-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `cpndaten` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Period — Coupons per year of the bond

2 (semiannual) (default) | vector of positive integers from the set [1,2,3,4,6,12]

Coupons per year of the bond, specified as a vector of positive integers from the set [1,2,3,4,6,12].

Data Types: single | double

Basis — Day-count basis of the bond

0 (actual/actual) (default) | numeric with value 0 through 13 | vector of numerics with values 0 through 13

Day-count basis of the bond, specified as an integer with a value of 0 through 13 or an N-by-1 vector of integers with values of 0 through 13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `single` | `double`

EndMonthRule — End-of-month rule flag for month having 30 or fewer days

1 (in effect) (default) | nonnegative integer [0, 1]

End-of-month rule flag for month having 30 or fewer days, specified as scalar nonnegative integer [0, 1] or a using an N-by-1 vector of values. This rule applies only when `Maturity` is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond’s coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond’s coupon payment date is always the last actual day of the month.

Data Types: `logical`

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond issue date, specified as a `NUMBONDS`-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `cpnmaten` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

FirstCouponDate — Date when bond makes first coupon payment

datetime array | string array | date character vector

Date when a bond makes its first coupon payment, specified as a `NUMBONDS`-by-1 vector using a datetime array, string array, or date character vectors.

`FirstCouponDate` is used when a bond has an irregular first coupon period. When `FirstCouponDate` and `LastCouponDate` are both specified, `FirstCouponDate` takes precedence in determining the coupon payment structure. If you do not specify a `FirstCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cpndaten` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

LastCouponDate — Last coupon date of bond before maturity date

datetime array | string array | date character vector

Last coupon date of a bond before maturity date, specified as a `NUMBONDS`-by-1 vector using a datetime array, string array, or date character vectors.

`LastCouponDate` is used when a bond has an irregular last coupon period. In the absence of a specified `FirstCouponDate`, a specified `LastCouponDate` determines the coupon structure of the bond. The coupon structure of a bond is truncated at the `LastCouponDate`, regardless of where it falls, and is followed only by the bond's maturity cash flow date. If you do not specify a `LastCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cpndaten` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Output Arguments

NextCouponDate — Next coupon date after the settlement date

vector

Next coupon date after the settlement date, returned as an `NUMBONDS`-by-1 vector of next actual coupon dates after settlement. If settlement is a coupon date, this function never returns the settlement date. Instead, the actual coupon date strictly after settlement is returned, but not exceeding the maturity date. Thus, this function will always return the lesser of the actual maturity date and the next coupon payment date.

If all the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are either strings or date character vectors, then `NextCouponDate` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted date character vectors.

If any of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are datetime arrays, then `NextCouponDate` is returned as a datetime array.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cpndaten` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)  
  
y =  
  
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`accrfrac` | `cfamounts` | `cfdates` | `cftimes` | `cpncount` | `cpndatenq` | `cpndatep` | `cpndaysn` | `cpndaysp` | `cpnpersz` | `datetime` | `cpndatepq`

Topics

"Pricing and Computing Yields for Fixed-Income Securities" on page 2-15

cpndatenq

Next quasi-coupon date for fixed-income security

Syntax

```
NextQuasiCouponDate = cpndatenq(Settle,Maturity)
NextQuasiCouponDate = cpndatenq( ___,Period,Basis,EndMonthRule,IssueDate,
FirstCouponDate,LastCouponDate)
```

Description

`NextQuasiCouponDate = cpndatenq(Settle,Maturity)` determines the next quasi coupon date for a portfolio of `NUMBONDS` fixed income securities whether or not the first or last coupon is normal, short, or long. For zero coupon bonds, `cpndatenq` returns quasi coupon dates as if the bond had a semiannual coupon structure. Successive quasi coupon dates determine the length of the standard coupon period for the fixed income security of interest and do not necessarily coincide with actual coupon payment dates.

Required input arguments must be number of bonds, `NUMBONDS-by-1` or `1-by-NUMBONDS`, conforming vectors or scalars.

`NextQuasiCouponDate = cpndatenq( ___,Period,Basis,EndMonthRule,IssueDate,FirstCouponDate,LastCouponDate)` determines the next quasi coupon date for a portfolio of `NUMBONDS` fixed income securities whether or not the first or last coupon is normal, short, or long using optional input arguments.

Optional input arguments must be either `NUMBONDS-by-1` or `1-by-NUMBONDS` conforming vectors, scalars, or empty matrices.

If all the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are either strings or date character vectors, then `NextQuasiCouponDate` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

If any of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are datetime arrays, then `NextQuasiCouponDate` is returned as a datetime array.

Examples

Determine the Next Quasi Coupon Date for a Portfolio of Fixed-Income Securities

Given a pair of bonds with the following characteristics:

```
Settle = [datetime(1997,5,30), datetime(1997,12,10)];
Maturity = [datetime(2002,11,30), datetime(2004,6,10)];
```

Compute `NextCouponDate` for this pair of bonds.

```
NextCouponDate = cpndaten(Settle, Maturity)
```

```
NextCouponDate = 2×1 datetime
    31-May-1997
    10-Jun-1998
```

Compute the next quasi coupon dates for these two bonds.

```
NextQuasiCouponDate = cpndatenq(Settle, Maturity)
```

```
NextQuasiCouponDate = 2×1 datetime
    31-May-1997
    10-Jun-1998
```

Because no `FirstCouponDate` has been specified, the results are identical.

Now supply an explicit `FirstCouponDate` for each bond.

```
FirstCouponDate = [datetime(1997,11,30), datetime(1998,12,10)];
```

Compute the next coupon dates.

```
NextCouponDate = cpndaten(Settle, Maturity, 2, 0, 1, [],FirstCouponDate)
```

```
NextCouponDate = 2×1 datetime
    30-Nov-1997
    10-Dec-1998
```

The next coupon dates are identical to the specified first coupon dates.

Now recompute the next quasi coupon dates.

```
NextQuasiCouponDate = cpndatenq(Settle, Maturity, 2, 0, 1, [],FirstCouponDate)
```

```
NextQuasiCouponDate = 2×1 datetime
    31-May-1997
    10-Jun-1998
```

These results illustrate the distinction between actual coupon payment dates and quasi coupon dates. `FirstCouponDate` (and `LastCouponDate`, as well), when specified, is associated with an actual coupon payment and also serves as the synchronization date for determining all quasi coupon dates. Since each bond in this example pays semiannual coupons, and the first coupon date occurs more than six months after settlement, each will have an intermediate quasi coupon date before the actual first coupon payment occurs.

Input Arguments

Settle — Settlement date

`datetime` array | `string` array | `date` character vector

Settlement date, specified as a `NUMBONDS`-by-1 vector using a `datetime` array, `string` array, or `date` character vectors. `Settle` must be earlier than `Maturity`.

To support existing code, `cpndatenq` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date

datetime array | string array | date character vector

Maturity date, specified as a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, cpndatenq also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Period — Coupons per year of the bond

2 (semiannual) (default) | vector of positive integers from the set [1,2,3,4,6,12]

Coupons per year of the bond, specified as a vector of positive integers from the set [1,2,3,4,6,12].

Data Types: single | double

Basis — Day-count basis of the instrument

0 (actual/actual) (default) | numeric with value 0 through 13 | vector of numerics with values 0 through 13

Day-count basis of the instrument, specified as an integer with a value of 0 through 13 or an N-by-1 vector of integers with values of 0 through 13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: single | double

EndMonthRule — End-of-month rule flag for month having 30 or fewer days

1 (in effect) (default) | nonnegative integer [0,1]

End-of-month rule flag for month having 30 or fewer days, specified as a nonnegative integer [0, 1] using an N-by-1 vector of values. This rule applies only when **Maturity** is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond's coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond's coupon payment date is always the last actual day of the month.

Data Types: logical

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond issue date, specified as a **NUMBONDS**-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, **cpndatenq** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

FirstCouponDate — Date when bond makes first coupon payment

datetime array | string array | date character vector

Date when a bond makes its first coupon payment, specified as a **NUMBONDS**-by-1 vector using a datetime array, string array, or date character vectors.

FirstCouponDate is used when a bond has an irregular first coupon period. When **FirstCouponDate** and **LastCouponDate** are both specified, **FirstCouponDate** takes precedence in determining the coupon payment structure. If you do not specify a **FirstCouponDate**, the cash flow payment dates are determined from other inputs.

To support existing code, **cpndatenq** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

LastCouponDate — Last coupon date of bond before maturity date

datetime array | string array | date character vector

Last coupon date of a bond before maturity date, specified as a **NUMBONDS**-by-1 vector using a datetime array, string array, or date character vectors.

LastCouponDate is used when a bond has an irregular last coupon period. In the absence of a specified **FirstCouponDate**, a specified **LastCouponDate** determines the coupon structure of the bond. The coupon structure of a bond is truncated at the **LastCouponDate**, regardless of where it falls, and is followed only by the bond's maturity cash flow date. If you do not specify a **LastCouponDate**, the cash flow payment dates are determined from other inputs.

To support existing code, **cpndatenq** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Output Arguments

NextQuasiCouponDate — Next quasi coupon date for portfolio of NUMBONDS fixed income securities

vector

Next quasi coupon date for a portfolio of NUMBONDS fixed income securities, whether or not the first or last coupon is normal, short, or long, returned as a NUMBONDS-by-1 vector.

For zero coupon bonds, cpndatenq returns quasi coupon dates as if the bond had a semiannual coupon structure. Successive quasi coupon dates determine the length of the standard coupon period for the fixed income security of interest and do not necessarily coincide with actual coupon payment dates.

If all of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are either strings or date character vectors, then `NextQuasiCouponDate` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

If any of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are datetime arrays, then `NextQuasiCouponDate` is returned as a datetime array.

More About

Quasi Coupon

A quasi coupon is a term used in the context of certain fixed-income securities, particularly bonds, that do not pay traditional periodic interest payments (coupons) but instead provide returns in a different manner.

A quasi coupon can include returns that are contingent on certain conditions or linked to the performance of an underlying asset.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although cpndatenq supports serial date numbers, datetime values are recommended instead. The datetime data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to datetime values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

See Also

accrfrac | cfamounts | cfdates | cftimes | cpncount | cpndaten | cpndatep | cpndatepq |
cpndaysn | cpndaysp | cpnpersz | datetime

Topics

“Pricing and Computing Yields for Fixed-Income Securities” on page 2-15

cpndatepq

Previous quasi-coupon date for fixed-income security

Syntax

```
PreviousQuasiCouponDate = cpndatepq(Settle,Maturity)
PreviousQuasiCouponDate = cpndatepq(____,Period,Basis,EndMonthRule,IssueDate,
FirstCouponDate,LastCouponDate)
```

Description

`PreviousQuasiCouponDate = cpndatepq(Settle,Maturity)` determines the previous quasi-coupon date for a set of `NUMBONDS` fixed income securities. Prior quasi-coupon dates determine the length of the standard coupon period for the fixed income security of interest, and do not necessarily coincide with actual coupon payment dates. This function finds the previous quasi-coupon date for bonds with a coupon structure whose first or last period is either normal, short, or long.

Required input arguments must be number of bonds, `NUMBONDS-by-1` or `1-by-NUMBONDS`, conforming vectors or scalars.

`PreviousQuasiCouponDate = cpndatepq(____,Period,Basis,EndMonthRule,IssueDate,FirstCouponDate,LastCouponDate)`, using optional input arguments, determines the previous quasi-coupon date for a set of `NUMBONDS` fixed income securities.

Optional input arguments must be either `NUMBONDS-by-1` or `1-by-NUMBONDS` conforming vectors, scalars, or empty matrices.

If all the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are either strings or date character vectors, then `PreviousQuasiCouponDate` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

If any of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are datetime arrays, then `PreviousQuasiCouponDate` is returned as a datetime array.

Examples

Determine the Previous Quasi Coupon Date for a Portfolio of Fixed-Income Securities

Given a pair of bonds with the following characteristics:

```
Settle = [datetime(1997,5,30) ; datetime(1997,12,10)];
Maturity = [datetime(2002,11,30) ; datetime(2004,6,10)];
```

With no `FirstCouponDate` explicitly supplied, compute the `PreviousCouponDate` for this pair of bonds.

```
PreviousCouponDate = cpndatepq(Settle, Maturity)
```

```
PreviousCouponDate = 2×1 datetime
    30-Nov-1996
    10-Dec-1997
```

Note that since the settlement date for the second bond is also a coupon date, `cpndatep` returns this date as the previous coupon date.

Now establish a `FirstCouponDate` and `IssueDate` for this pair of bonds.

```
FirstCouponDate = [datetime(1997,11,30) ; datetime(1998,12,10)];
IssueDate = [datetime(1996,5,30) ; datetime(1996,12,10)];
```

Recompute the `PreviousCouponDate` for this pair of bonds.

```
PreviousCouponDate = cpndatep(Settle, Maturity, 2, 0, 1, IssueDate, FirstCouponDate)
```

```
PreviousCouponDate = 2×1 datetime
    30-May-1996
    10-Dec-1996
```

Since both of these bonds settled before the first coupon had been paid, `cpndatep` returns the `IssueDate` as the `PreviousCouponDate`.

Using the same data, compute `PreviousQuasiCouponDate`.

```
PreviousQuasiCouponDate = cpndatepq(Settle, Maturity, 2, 0, 1, IssueDate, FirstCouponDate)
```

```
PreviousQuasiCouponDate = 2×1 datetime
    30-Nov-1996
    10-Dec-1997
```

For the first bond the settlement date is not a normal coupon date. The `PreviousQuasiCouponDate` is the coupon date before or on the settlement date. Since the coupon structure is synchronized to `FirstCouponDate`, the previous quasi coupon date is 30-Nov-1996. `PreviousQuasiCouponDate` disregards `IssueDate` and `FirstCouponDate` in this case. For the second bond the settlement date (10-Dec-1997) occurs on a date when a coupon would normally be paid in the absence of an explicit `FirstCouponDate`. `cpndatepq` returns this date as `PreviousQuasiCouponDate`.

Input Arguments

Settle — Settlement date

datetime array | string array | date character vector

Settlement date, specified as a `NUMBONDS`-by-1 vector using a datetime array, string array, or date character vectors. `Settle` must be earlier than `Maturity`.

To support existing code, `cpndatepq` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date

datetime array | string array | date character vector

Maturity date, specified as a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, cpndatepq also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Period — Coupons per year of the bond

2 (semiannual) (default) | vector of positive integers from the set [1, 2, 3, 4, 6, 12]

Coupons per year of the bond, specified as a vector of positive integers from the set [1, 2, 3, 4, 6, 12].

Data Types: single | double

Basis — Day-count basis of the instrument

0 (actual/actual) (default) | numeric with value 0 through 13 | vector of numerics with values 0 through 13

Day-count basis of the instrument, specified as an integer with a value of 0 through 13 or an N-by-1 vector of integers with values of 0 through 13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: single | double

EndMonthRule — End-of-month rule flag for month having 30 or fewer days

1 (in effect) (default) | nonnegative integer [0, 1]

End-of-month rule flag for month having 30 or fewer days, specified as a nonnegative integer [0, 1] using an N-by-1 vector of values. This rule applies only when Maturity is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond’s coupon payment date is always the same numerical day of the month.

- 1 = Set rule on, meaning that a bond's coupon payment date is always the last actual day of the month.

Data Types: logical

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond issue date, specified as a `NUMBONDS`-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `cpndatepq` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

FirstCouponDate — Date when bond makes first coupon payment

datetime array | string array | date character vector

Date when a bond makes its first coupon payment, specified as a `NUMBONDS`-by-1 vector using a datetime array, string array, or date character vectors.

`FirstCouponDate` is used when a bond has an irregular first coupon period. When `FirstCouponDate` and `LastCouponDate` are both specified, `FirstCouponDate` takes precedence in determining the coupon payment structure. If you do not specify a `FirstCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cpndatepq` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

LastCouponDate — Last coupon date of bond before maturity date

datetime array | string array | date character vector

Last coupon date of a bond before maturity date, specified as a `NUMBONDS`-by-1 vector using a datetime array, string array, or date character vectors.

`LastCouponDate` is used when a bond has an irregular last coupon period. In the absence of a specified `FirstCouponDate`, a specified `LastCouponDate` determines the coupon structure of the bond. The coupon structure of a bond is truncated at the `LastCouponDate`, regardless of where it falls, and is followed only by the bond's maturity cash flow date. If you do not specify a `LastCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cpndatepq` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Output Arguments**PreviousQuasiCouponDate — Previous quasi coupon date for portfolio of `NUMBONDS` fixed income securities**

vector

Previous quasi coupon date for a portfolio of NUMBONDS fixed income securities, whether or not the first or last coupon is normal, short, or long, returned as a NUMBONDS-by-1 vector of previous quasi-coupon dates before settlement. If settlement is a coupon date, this function returns the settlement date.

If all the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are either strings or date character vectors, then `PreviousQuasiCouponDate` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted date character vectors.

If any of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are datetime arrays, then `PreviousQuasiCouponDate` is returned as a datetime array.

More About

Quasi-Coupon

A quasi-coupon is a term used in the context of certain fixed-income securities, particularly bonds, that do not pay traditional periodic interest payments (coupons) but instead provide returns in a different manner.

A quasi coupon can include returns that are contingent on certain conditions or linked to the performance of an underlying asset.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cpndatepq` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`accrfrac` | `cfamounts` | `cfdates` | `cftimes` | `cpncount` | `cpndaten` | `cpndatep` | `cpndatenq` | `cpndaysn` | `cpndaysp` | `cpnpersz` | `datetime`

Topics

"Pricing and Computing Yields for Fixed-Income Securities" on page 2-15

cpndatep

Previous coupon date for fixed-income security

Syntax

```
PreviousCouponDate = cpndatep(Settle,Maturity)
PreviousCouponDate = cpndatep(____,Period,Basis,EndMonthRule,IssueDate,
FirstCouponDate,LastCouponDate)
```

Description

`PreviousCouponDate = cpndatep(Settle,Maturity)` returns the previous coupon date on or before settlement for a portfolio of bonds. This function finds the previous coupon date whether or not the coupon structure is synchronized with the maturity date. For zero coupon bonds the previous coupon date is the issue date, if available. However, if the issue date is not supplied, the previous coupon date for zero coupon bonds is the previous quasi coupon date calculated as if the frequency is semiannual.

Required input arguments must be number of bonds, `NUMBONDS-by-1` or `1-by-NUMBONDS`, conforming vectors or scalars.

`PreviousCouponDate = cpndatep(____,Period,Basis,EndMonthRule,IssueDate,FirstCouponDate,LastCouponDate)` returns the previous coupon date on or before settlement for a portfolio of bonds.

Optional input arguments must be either `NUMBONDS-by-1` or `1-by-NUMBONDS` conforming vectors, scalars, or empty matrices.

If all the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are either strings or date character vectors, then `PreviousCouponDate` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted date character vectors.

If any of the inputs for `Settle`, `Maturity`, `IssueDate`, `FirstCouponDate`, and `LastCouponDate` are datetime arrays, then `PreviousCouponDate` is returned as a datetime array.

Examples

Calculate the Previous Coupon Date on or Before Settlement

Determine the `PreviousCouponDate` when using character vectors for input arguments.

```
PreviousCouponDate = cpndatep(datetime(1997,3,14),datetime(2000,6,30),2, 0, 0)
```

```
PreviousCouponDate = datetime
    30-Dec-1996
```

Determine the `PreviousCouponDate` when using datetime arrays for input arguments.

```
PreviousCouponDate = cpndatep(datetime(1997,3,14),datetime(2000,6,30),2, 0, 0)
```

```
PreviousCouponDate = datetime  
30-Dec-1996
```

Determine the `PreviousCouponDate` when using character vectors for input arguments and the optional argument for `EndMonthRule`.

```
PreviousCouponDate = cpndatep(datetime(1997,3,14),datetime(2000,6,30),2, 0, 1)
```

```
PreviousCouponDate = datetime  
31-Dec-1996
```

Determine the `PreviousCouponDate` when using an input vector for `Maturity`.

```
Maturity = [datetime(2000,4,30) ; datetime(2000,5,31) ; datetime(2000,6,30)];  
PreviousCouponDate = cpndatep(datetime(1997,3,14), Maturity)
```

```
PreviousCouponDate = 3×1 datetime  
31-Oct-1996  
30-Nov-1996  
31-Dec-1996
```

Input Arguments

Settle — Settlement date

`datetime` array | `string` array | date character vector

Settlement date, specified as a `NUMBONDS`-by-1 vector using a `datetime` array, `string` array, or date character vectors. `Settle` must be earlier than `Maturity`.

To support existing code, `cpndatep` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date

`datetime` array | `string` array | date character vector

Maturity date, specified as a `NUMBONDS`-by-1 vector using a `datetime` array, `string` array, or date character vectors.

To support existing code, `cpndatep` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Period — Coupons per year of the bond

2 (semiannual) (default) | vector of positive integers from the set [1,2,3,4,6,12]

Coupons per year of the bond, specified as a vector of positive integers from the set [1,2,3,4,6,12].

Data Types: `single` | `double`

Basis — Day-count basis of the instrument

0 (actual/actual) (default) | numeric with value 0 through 13 | vector of numerics with values 0 through 13

Day-count basis of the instrument, specified as an integer with a value of 0 through 13 or an N-by-1 vector of integers with values of 0 through 13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `single` | `double`

EndMonthRule — End-of-month rule flag for month having 30 or fewer days

1 (in effect) (default) | nonnegative integer [0, 1]

End-of-month rule flag for month having 30 or fewer days, specified as a nonnegative integer [0, 1] using an N-by-1 vector of values. This rule applies only when `Maturity` is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond’s coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond’s coupon payment date is always the last actual day of the month.

Data Types: `logical`

IssueDate — Bond issue date

datetime array | string array | date character vector

Bond issue date, specified as a `NUMBONDS`-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `cpndatep` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

FirstCouponDate — Date when bond makes first coupon payment

datetime array | string array | date character vector

Date when a bond makes its first coupon payment, specified as a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors.

FirstCouponDate is used when a bond has an irregular first coupon period. When FirstCouponDate and LastCouponDate are both specified, FirstCouponDate takes precedence in determining the coupon payment structure. If you do not specify a FirstCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, cpndatep also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

LastCouponDate — Last coupon date of bond before maturity date

datetime array | string array | date character vector

Last coupon date of a bond before maturity date, specified as a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors.

LastCouponDate is used when a bond has an irregular last coupon period. In the absence of a specified FirstCouponDate, a specified LastCouponDate determines the coupon structure of the bond. The coupon structure of a bond is truncated at the LastCouponDate, regardless of where it falls, and is followed only by the bond's maturity cash flow date. If you do not specify a LastCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, cpndatep also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Output Arguments**PreviousCouponDate — Previous coupon date on or before settlement for portfolio of bonds**

vector

Previous coupon date on or before settlement for portfolio of bonds, returned as an NUMBONDS-by-1 vector. If settlement is a coupon date, this function returns the settlement date. The actual coupon date strictly on or before settlement is returned, but not exceeding the issue date, if available. Thus, this function always returns the lesser of the actual issue date and the previous coupon payment date with respect to the settlement date.

If all the inputs for Settle, Maturity, IssueDate, FirstCouponDate, and LastCouponDate are either strings or date character vectors, then PreviousCouponDate is returned as a serial date number. Use the function datetime to convert serial date numbers to formatted datetime arrays.

If any of the inputs for Settle, Maturity, IssueDate, FirstCouponDate, and LastCouponDate are datetime arrays, then PreviousCouponDate is returned as a datetime array.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `cpndatep` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

See Also

`accrfrac` | `cfamounts` | `cfdates` | `cftimes` | `cpncount` | `cpndaten` | `cpndatenq` | `cpndatepq` | `cpndaysn` | `cpndaysp` | `cpnpersz` | `datetime`

Topics

“Pricing and Computing Yields for Fixed-Income Securities” on page 2-15

cpndaysn

Number of days to next coupon date

Syntax

```
NumDaysNext = cpndaysn(Settle,Maturity)
NumDaysNext = cpndaysn(____,Period,Basis,EndMonthRule,IssueDate,
FirstCouponDate,LastCouponDate)
```

Description

`NumDaysNext = cpndaysn(Settle,Maturity)` returns the number of days from the settlement date to the next coupon date for a bond or set of bonds. For zero coupon bonds coupon dates are computed as if the bonds have a semiannual coupon structure. `NumDaysNext` returns a `NUMBONDS-by-1` vector containing the number of days to the next coupon date.

Required input arguments must be number of bonds, `NUMBONDS-by-1` or `1-by-NUMBONDS`, conforming vectors or scalars.

`NumDaysNext = cpndaysn(____,Period,Basis,EndMonthRule,IssueDate,FirstCouponDate,LastCouponDate)` returns the number of days from the settlement date to the next coupon date for a bond or set of bonds using optional input arguments.

Optional input arguments must be either `NUMBONDS-by-1` or `1-by-NUMBONDS` conforming vectors, scalars, or empty matrices.

Examples

Calculate the Number of Days From Settlement Date to Next Coupon Date

Determine the `NumDaysNext` when using datetimes for input arguments.

```
NumDaysNext = cpndaysn(datetime(2000,9,14) , datetime(2001,6,30), 2, 0, 0)
```

```
NumDaysNext =
107
```

Determine the `NumDaysNext` when using character vectors for input arguments and the optional argument for `EndMonthRule`.

```
NumDaysNext = cpndaysn('14-Sep-2000', '30-Jun-2001', 2, 0, 1)
```

```
NumDaysNext =
108
```

Determine the `NumDaysNext` when using a datetime array for `Maturity`.

```
Maturity = [datetime(2001,4,30) ; datetime(2001,5,31) ; datetime(2001,6,30)];
NumDaysNext = cpndaysn(datetime(2000,9,14), Maturity)
```

```
NumDaysNext = 3×1
```

47
77
108

Input Arguments

Settle — Settlement date

datetime array | string array | date character vector

Settlement date, specified as a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors. **Settle** must be earlier than **Maturity**.

To support existing code, **cpndaysn** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date

datetime array | string array | date character vector

Maturity date, specified as a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, **cpndaysn** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Period — Coupons per year of the bond

2 (semiannual) (default) | vector of positive integers from the set [1,2,3,4,6,12]

Coupons per year of the bond, specified as a vector of positive integers from the set [1,2,3,4,6,12].

Data Types: single | double

Basis — Day-count basis of the instrument

0 (actual/actual) (default) | numeric with value 0 through 13 | vector of numerics with values 0 through 13

Day-count basis of the instrument, specified as an integer with a value of 0 through 13 or an N-by-1 vector of integers with values of 0 through 13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)

- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `single` | `double`

EndMonthRule — End-of-month rule flag for month having 30 or fewer days

1 (in effect) (default) | nonnegative integer [0, 1]

End-of-month rule flag for month having 30 or fewer days, specified as a nonnegative integer [0, 1] using an N-by-1 vector of values. This rule applies only when `Maturity` is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond’s coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond’s coupon payment date is always the last actual day of the month.

Data Types: `logical`

IssueDate — Bond issue date

`datetime` array | `string` array | `date` character vector

Bond issue date, specified as a `NUMBONDS`-by-1 vector using a `datetime` array, `string` array, or `date` character vectors.

To support existing code, `cpndaysn` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

FirstCouponDate — Date when bond makes first coupon payment

`datetime` array | `string` array | `date` character vector

Date when a bond makes its first coupon payment, specified as a `NUMBONDS`-by-1 vector using a `datetime` array, `string` array, or `date` character vectors.

`FirstCouponDate` is used when a bond has an irregular first coupon period. When `FirstCouponDate` and `LastCouponDate` are both specified, `FirstCouponDate` takes precedence in determining the coupon payment structure. If you do not specify a `FirstCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cpndaysn` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

LastCouponDate — Last coupon date of bond before maturity date

`datetime` array | `string` array | `date` character vector

Last coupon date of a bond before maturity date, specified as a `NUMBONDS-by-1` vector using a datetime array, string array, or date character vectors.

`LastCouponDate` is used when a bond has an irregular last coupon period. In the absence of a specified `FirstCouponDate`, a specified `LastCouponDate` determines the coupon structure of the bond. The coupon structure of a bond is truncated at the `LastCouponDate`, regardless of where it falls, and is followed only by the bond's maturity cash flow date. If you do not specify a `LastCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cpndaysn` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Output Arguments

NumDaysNext — Number of days from settlement date to next coupon date
vector

Number of days from settlement date to next coupon date, returned as an `NUMBONDS-by-1` vector. For zero coupon bonds coupon dates are computed as if the bonds have a semiannual coupon structure.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended
Not recommended starting in R2022b

Although `cpndaysn` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)  
  
y =  
  
2021
```

There are no plans to remove support for serial date number inputs.

See Also

`accrfrac` | `cfamounts` | `cfdates` | `cftimes` | `cpncount` | `cpndaten` | `cpndatep` | `cpndatenq` | `cpndaysp` | `cpnpersz` | `datetime` | `cpndatepq`

Topics

"Pricing and Computing Yields for Fixed-Income Securities" on page 2-15

cpndaysp

Number of days since previous coupon date

Syntax

```
NumDaysPrevious = cpndaysp(Settle,Maturity)
NumDaysPrevious = cpndaysp(____,Period,Basis,EndMonthRule,IssueDate,
FirstCouponDate,LastCouponDate)
```

Description

`NumDaysPrevious = cpndaysp(Settle,Maturity)` returns the number of days between the previous coupon date and the settlement date for a bond or set of bonds. When the coupon frequency is 0 (a zero coupon bond), the previous coupon date is calculated as if the frequency were semiannual. `NumDaysPrevious` returns a `NUMBONDS-by-1` vector containing the number of days from the previous coupon date to settlement.

Required input arguments must be number of bonds, `NUMBONDS-by-1` or `1-by-NUMBONDS`, conforming vectors or scalars.

`NumDaysPrevious = cpndaysp(____,Period,Basis,EndMonthRule,IssueDate,FirstCouponDate,LastCouponDate)` returns the number of days between the previous coupon date and the settlement date for a bond or set of bonds using optional input arguments.

Optional input arguments must be either `NUMBONDS-by-1` or `1-by-NUMBONDS` conforming vectors, scalars, or empty matrices.

Examples

Calculate the Number of Days Between Previous Coupon Date and Settlement Date

Determine the `NumDaysPrevious` when using datetimes for input arguments.

```
NumDaysPrevious = cpndaysp(datetime(2000,3,14),datetime(2001,6,30), 2, 0, 0)
```

```
NumDaysPrevious =
75
```

Determine the `NumDaysPrevious` when using character vectors for input arguments and the optional argument for `EndMonthRule`.

```
NumDaysPrevious = cpndaysp('14-Mar-2000','30-Jun-2001', 2, 0, 1)
```

```
NumDaysPrevious =
74
```

Determine the `NumDaysPrevious` when using a datetime array for `Maturity`.

```
Maturity = [datetime(2001,4,30) ; datetime(2001,5,31) ; datetime(2001,6,30)];
NumDaysPrevious = cpndaysp(datetime(2000,3,14), Maturity)
```

```
NumDaysPrevious = 3×1
```

```
135  
105  
74
```

Input Arguments

Settle — Settlement date

datetime array | string array | date character vector

Settlement date, specified as a `NUMBONDS`-by-1 vector using a datetime array, string array, or date character vectors. `Settle` must be earlier than `Maturity`.

To support existing code, `cpndaysp` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date

datetime array | string array | date character vector

Maturity date, specified as a `NUMBONDS`-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `cpndaysp` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Period — Coupons per year of the bond

2 (semiannual) (default) | vector of positive integers from the set [1,2,3,4,6,12]

Coupons per year of the bond, specified as a vector of positive integers from the set [1,2,3,4,6,12].

Data Types: single | double

Basis — Day-count basis of the instrument

0 (actual/actual) (default) | numeric with value 0 through 13 | vector of numerics with values 0 through 13

Day-count basis of the instrument, specified as an integer with a with value 0 through 13 or an N-by-1 vector of integers with values 0 through 13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)

- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `single` | `double`

EndMonthRule — End-of-month rule flag for month having 30 or fewer days

1 (in effect) (default) | nonnegative integer [0, 1]

End-of-month rule flag for month having 30 or fewer days, specified as a nonnegative integer [0, 1] using an N-by-1 vector of values. This rule applies only when `Maturity` is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond’s coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond’s coupon payment date is always the last actual day of the month.

Data Types: `logical`

IssueDate — Bond issue date

`datetime` array | `string` array | `date` character vector

Bond issue date, specified as a `NUMBONDS`-by-1 vector using a `datetime` array, `string` array, or `date` character vectors.

To support existing code, `cpndaysp` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

FirstCouponDate — Date when bond makes first coupon payment

`datetime` array | `string` array | `date` character vector

Date when a bond makes its first coupon payment, specified as a `NUMBONDS`-by-1 vector using a `datetime` array, `string` array, or `date` character vectors.

`FirstCouponDate` is used when a bond has an irregular first coupon period. When `FirstCouponDate` and `LastCouponDate` are both specified, `FirstCouponDate` takes precedence in determining the coupon payment structure. If you do not specify a `FirstCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cpndaysp` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

LastCouponDate — Last coupon date of bond before maturity date

datetime array | string array | date character vector

Last coupon date of a bond before maturity date, specified as a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors.

LastCouponDate is used when a bond has an irregular last coupon period. In the absence of a specified FirstCouponDate, a specified LastCouponDate determines the coupon structure of the bond. The coupon structure of a bond is truncated at the LastCouponDate, regardless of where it falls, and is followed only by the bond's maturity cash flow date. If you do not specify a LastCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, cpndaysp also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Output Arguments**NumDaysPrevious — Number of days between previous coupon date and settlement date**
vector

Number of days between the previous coupon date and the settlement date, returned as an NUMBONDS-by-1 vector. If the settlement date is a coupon date, this function always returns the settlement date.

When the coupon frequency is 0 (a zero coupon bond), the previous coupon date is calculated as if the frequency were semiannual.

Version History**Introduced before R2006a****R2022b: Serial date numbers not recommended***Not recommended starting in R2022b*

Although cpndaysp supports serial date numbers, datetime values are recommended instead. The datetime data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to datetime values, use the datetime function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

accrfrac | cfamounts | cfdates | cftimes | cpncount | cpndaten | cpndatenq | cpndatep | cpndaysn | cpndatepq | cpnpersz | datetime

Topics

"Pricing and Computing Yields for Fixed-Income Securities" on page 2-15

cpnpersz

Number of days in coupon period

Syntax

```
NumDaysPeriod = cpnpersz(Settle,Maturity)
NumDaysPeriod = cpnpersz( ___,Period,Basis,EndMonthRule,IssueDate,
FirstCouponDate,LastCouponDate)
```

Description

`NumDaysPeriod = cpnpersz(Settle,Maturity)` returns the number of days in the coupon period containing the settlement date. For zero coupon bonds, coupon dates are computed as if the bonds have a semiannual coupon structure. `NumDaysPeriod` returns a `NUMBONDS`-by-1 vector containing the number of days in the coupon period containing the settlement date.

.

Required input arguments must be number of bonds, `NUMBONDS`-by-1 or 1-by-`NUMBONDS`, conforming vectors or scalars.

`NumDaysPeriod = cpnpersz( ___,Period,Basis,EndMonthRule,IssueDate,FirstCouponDate,LastCouponDate)` returns the number of days in the coupon period containing the settlement date using optional input arguments.

Optional input arguments must be either `NUMBONDS`-by-1 or 1-by-`NUMBONDS` conforming vectors, scalars, or empty matrices.

Examples

Calculate the Number of Days in the Coupon Period Containing Settlement Date

Determine the `NumDaysPeriod` when using datetimes for input arguments.

```
NumDaysPeriod = cpnpersz(datetime(2000,9,14),datetime(2001,6,30), 2, 0, 0)
```

```
NumDaysPeriod =
183
```

Determine the `NumDaysPeriod` when using character vectors for input arguments and the optional argument for `EndMonthRule`.

```
NumDaysPeriod = cpnpersz('14-Sep-2000', '30-Jun-2001', 2, 0, 1)
```

```
NumDaysPeriod =
184
```

Determine the `NumDaysPeriod` when using an input vector for `Maturity`.

```
Maturity = [datetime(2001,6,30) ; datetime(2001,5,31) ; datetime(2001,6,30)];
NumDaysPeriod = cpnpersz('14-Sep-2000', Maturity)
```



```
NumDaysPeriod = 3×1
```

```
184
183
184
```

Input Arguments

Settle — Settlement date

datetime array | string array | date character vector

Settlement date, specified as a **NUMBONDS**-by-1 vector using a datetime array, string array, or date character vectors. **Settle** must be earlier than **Maturity**.

To support existing code, **cpnpersz** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date

datetime array | string array | date character vector

Maturity date, specified as a **NUMBONDS**-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, **cpnpersz** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Period — Coupons per year of the bond

2 (semiannual) (default) | vector of positive integers from the set [1,2,3,4,6,12]

Coupons per year of the bond, specified as a vector of positive integers from the set [1,2,3,4,6,12].

Data Types: single | double

Basis — Day-count basis of the instrument

0 (actual/actual) (default) | numeric with value 0 through 13 | vector of numerics with values 0 through 13

Day-count basis of the instrument, specified as an integer with a value of 0 through 13 or an N-by-1 vector of integers with values of 0 through 13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)

- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `single` | `double`

EndMonthRule — End-of-month rule flag for month having 30 or fewer days

1 (in effect) (default) | nonnegative integer [0, 1]

End-of-month rule flag for month having 30 or fewer days, specified as a nonnegative integer [0, 1] using an N-by-1 vector of values. This rule applies only when `Maturity` is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond’s coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond’s coupon payment date is always the last actual day of the month.

Data Types: `logical`

IssueDate — Bond issue date

`datetime` array | `string` array | `date` character vector

Bond issue date, specified as a `NUMBONDS`-by-1 vector using a `datetime` array, `string` array, or `date` character vectors.

To support existing code, `cpnpersz` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

FirstCouponDate — Date when bond makes first coupon payment

`datetime` array | `string` array | `date` character vector

Date when a bond makes its first coupon payment, specified as a `NUMBONDS`-by-1 vector using a `datetime` array, `string` array, or `date` character vectors.

`FirstCouponDate` is used when a bond has an irregular first coupon period. When `FirstCouponDate` and `LastCouponDate` are both specified, `FirstCouponDate` takes precedence in determining the coupon payment structure. If you do not specify a `FirstCouponDate`, the cash flow payment dates are determined from other inputs.

To support existing code, `cpnpersz` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

LastCouponDate — Last coupon date of bond before maturity date

datetime array | string array | date character vector

Last coupon date of a bond before maturity date, specified as a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors.

LastCouponDate is used when a bond has an irregular last coupon period. In the absence of a specified FirstCouponDate, a specified LastCouponDate determines the coupon structure of the bond. The coupon structure of a bond is truncated at the LastCouponDate, regardless of where it falls, and is followed only by the bond's maturity cash flow date. If you do not specify a LastCouponDate, the cash flow payment dates are determined from other inputs.

To support existing code, cpnpersz also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Output Arguments**NumDaysPeriod — Number of days in coupon period containing settlement date**

vector

Number of days in the coupon period containing the settlement date, returned as an NUMBONDS-by-1 vector. For zero coupon bonds, coupon dates are computed as if the bonds have a semiannual coupon structure.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended*Not recommended starting in R2022b*

Although cpnpersz supports serial date numbers, datetime values are recommended instead. The datetime data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to datetime values, use the datetime function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

accrfrac | cfamounts | cfdates | cftimes | cpncount | cpndaten | cpndatenq | cpndatep | cpndaysn | cpndatepq | cpndaysp | datetime

Topics

"Pricing and Computing Yields for Fixed-Income Securities" on page 2-15

createholidays

Create trading calendars

Syntax

```
createholidays(Filename,Codefile,InfoFile,TargetDir,IncludeWkds,Wprompt,  
NoGUI)
```

Description

`createholidays(Filename,Codefile,InfoFile,TargetDir,IncludeWkds,Wprompt, NoGUI)` programmatically generates the market-specific `holidays.m` files (from FinancialCalendar.com financial center holiday data) without displaying the interface.

Note To use `createholidays`, you must obtain data, codes, and info files from [https://swapsmonitor.com/ trading calendars](https://swapsmonitor.com/trading-calendars). The data files must be in the required MATLAB format.

Examples

Create holidays.m Files Using createholidays

Use `createholidays` to create `holidays*.m` files from `My_datafile.csv` in the folder `c:\work`. Weekends are included in the holidays list based on the input flag `INCLUDEWDKS = 1`

```
createholidays('FinancialCalendar\My_datafile.csv',...  
'FinancialCalendar\My_codesfile.csv',...  
'FinancialCalendar\My_infofile.csv','C:\work',1,1,1)
```

Input Arguments

Filename — Data file name

character vector

Data file name, specified using a character vector.

Data Types: char

Codefile — Code file name

character vector

Code file name, specified using a character vector.

Data Types: char

InfoFile — Info file name

character vector

Info file name, specified using a character vector.

Data Types: char

TargetDir — Target folder where to write new holidays.m files

character vector

Target folder where to write the new holidays.m files, specified using a character vector.

Data Types: `char`**IncludeWkds — Option to include weekends in holiday list**

numeric with value 0 or 1

Option to include weekends in the holiday list, specified using a numeric with value 0 or 1. Values are:

- 0 - Do not include weekends in the holiday list.
- 1 - Include weekends in the holiday list.

Data Types: `logical`**Wprompt — Option to prompt for file location for each holiday.m file that is created**

numeric with value 0 or 1

Option to prompt for the file location for each holiday.m file that is created, specified using a numeric with value 0 or 1. Values are:

- 0 - Do not prompt for the file location.
- 1 - Prompt for the file location.

Data Types: `logical`**NoGUI — Run createholidays without displaying Trading Calendars user interface**

numeric with value 0 or 1

Run createholidays without displaying the Trading Calendars user interface, specified using a numeric with value 0 or 1. Values are:

- 0 - Display the GUI.
- 1 - Do not display the GUI.

Data Types: `logical`

Version History

Introduced in R2007b

See Also

holidays

Topics

“Handle and Convert Dates” on page 2-2

“Trading Calendars User Interface” on page 12-2

“UICalendar User Interface” on page 12-4

cur2frac

Decimal currency values to fractional values

Syntax

```
Fraction = cur2frac(Decimal,Denominator)
```

Description

`Fraction = cur2frac(Decimal,Denominator)` converts decimal currency values to fractional values. `Fraction` is returned as a character vector.

Examples

Convert Decimal Currency Values to Fractional Values

This example shows how to convert decimal currency values to fractional values.

```
Fraction = cur2frac(12.125, 8)
```

```
Fraction =  
'12.1'
```

Input Arguments

Decimal — Decimal currency value

numeric decimal

Decimal currency value, specified as a scalar or vector using numeric decimal values.

Data Types: `double`

Denominator — Denominator of the fractions

numeric

Denominator of the fractions, specified as a scalar or vector using numeric values for the denominator.

Data Types: `double`

Output Arguments

Fraction — Fractional values

character vector | cell array of character vectors

Fractional values, returned as a character vector or cell array of character vectors.

Version History

Introduced before R2006a

See Also

`cur2str` | `frac2cur`

cur2str

Bank-formatted text

Syntax

`BankText = cur2str(Value,Digits)`

Description

`BankText = cur2str(Value,Digits)` returns the given value in bank format.

The output format for `BankText` is a numerical format with dollar sign prefix, two decimal places, and negative numbers in parentheses; for example, `($123.45)` and `$6789.01`. The standard MATLAB bank format uses two decimal places, no dollar sign, and a minus sign for negative numbers; for example, `-123.45` and `6789.01`.

Examples

Return Bank Formatted Text

Return bank formatted text for a negative numeric value.

```
BankText = cur2str(-826444.4456,3)
```

```
BankText =  
' ($826444.446) '
```

```
% Negative numbers are displayed in parentheses.
```

Input Arguments

Value — Value to be formatted

numeric value

Value to be formatted, specified as a numeric value.

Data Types: `double`

Digits — Number of significant digits

2 (default) | integer

Number of significant digits, specified as an integer. For example, when rounding off the value of 1 significant digit, on the right of the decimal point, `.1921212` is rounded off to `.2`:

```
cur2str(9990.1921212,1)
```

```
ans =
```

```
' $9990.2 '
```

A negative `Digits` rounds the value to the left of the decimal point.

Data Types: `double`

Output Arguments

BankText — Bank-formatted text

character vector

Bank-formatted text (`BankText`) is returned as a character vector with a leading dollar sign (\$). Negative numbers are displayed in parentheses.

Version History

Introduced before R2006a

See Also

`frac2cur` | `cur2frac`

date2time

Time and frequency from dates

Syntax

```
[TFactors,F] = date2time(Settle,Maturity)
[TFactors,F] = date2time( ___,Compounding,Basis,EndMonthRule)
```

Description

[TFactors,F] = date2time(Settle,Maturity) computes time factors appropriate to compounded rate quotes between the Settle and Maturity dates. date2time is the inverse of time2date.

[TFactors,F] = date2time(___,Compounding,Basis,EndMonthRule) computes time factors appropriate to compounded rate quotes between the Settle and Maturity dates using optional input arguments for Compounding, Basis, and EndMonthRule. date2time is the inverse of time2date.

Examples

Compute date2time Using an actual/actual Basis

To get the date2time period between '31-Jul-2015' and '30-Sep-2015' using an actual/actual basis:

```
date2time(datetime(2015,7,31),datetime(2015,9,30), 2, 0, 1)
```

```
ans =
0.3333
```

When using date2time quasi coupon, two quasi coupon dates are computed for a bond with a maturity corresponding to the Dates input. In this case, that would be "30-Sep-2015". Assuming that the compounding frequency is 2, the other quasi coupon date is six months prior to this date. Assuming the end of month rule is in place, then the other quasi coupon date is "31-Mar-2015". You can use these two dates to compute the total number of actual days in a period (which is 183). Given this, the fraction of time between the start and end date for the actual/actual basis is computed as follows.

(Actual Days between Start Date and End Date)/(Actual Number of Days between Quasi Coupon Dates)

There are 61 days between 31-Jul-2015 and 30-Sep-2015 and 183 days between the quasi coupon dates ("31-Mar-2015" and "30-Sep-2015") which leads to a final result of 61/183 or exactly 1/3.

Input Arguments

Settle — Settlement date

datetime scalar | string scalar | date character vector

Settlement date, specified as a scalar datetime, string, or date character vector.

To support existing code, `date2time` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date

`datetime array` | `string array` | `date character vector`

Maturity date, specified as a scalar or N-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `date2time` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Compounding — Rate at which input zero rates are compounded when annualized

2 (Semiannual compounding) (default) | scalar with numeric values of 0, 1, 2, 3, 4, 5, 6, 12, 365, -1

Rate at which input zero rates are compounded when annualized, specified as a scalar with numeric values of: 0, 1, 2, 3, 4, 5, 6, 12, 365, or -1. Allowed values are defined as:

- 0 — Simple interest (no compounding)
- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- 365 — Daily compounding
- -1 — Continuous compounding

The optional Compounding argument determines the formula for the discount factors (Disc):

- Compounding = 0 for simple interest
 - $\text{Disc} = 1 / (1 + Z * T)$, where T is time in years and simple interest assumes annual times $F = 1$.
- Compounding = 1, 2, 3, 4, 6, 12
 - $\text{Disc} = (1 + Z/F)^{-T}$, where F is the compounding frequency, Z is the zero rate, and T is the time in periodic units, for example, $T = F$ is one year.
- Compounding = 365
 - $\text{Disc} = (1 + Z/F)^{-T}$, where F is the number of days in the basis year and T is a number of days elapsed computed by basis.
- Compounding = -1
 - $\text{Disc} = \exp(-T*Z)$, where T is time in years.

Basis — Day-count basis

0 (actual/actual) (default) | numeric with value 0 through 13 | vector of numerics with values 0 through 13

Day-count basis, specified as an integer with a value 0 through 13 or a N-by-1 vector of integers with values 0 through 13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: single | double

EndMonthRule — End-of-month rule flag for month having 30 or fewer days

1 (in effect) (default) | nonnegative integer [0, 1]

End-of-month rule flag for month having 30 or fewer days, specified as scalar nonnegative integer [0, 1] or a using a N-by-1 vector of values. This rule applies only when **Maturity** is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a payment date is always the last actual day of the month.

Data Types: logical

Output Arguments**TFactors — Time factors**

vector

Time factors, appropriate to compounded rate quotes between **Settle** and **Maturity** dates, returned as a vector.

F — Compounding frequencies

scalar

Compounding frequencies, returned as a scalar.

More About

Difference Between `yearfrac` and `date2time`

The difference between `yearfrac` and `date2time` is that `date2time` counts full periods as a whole integer, even if the number of actual days in the periods are different. `yearfrac` does not count full periods.

For example,

```
yearfrac('1/1/2000', '1/1/2001', 9)
```

```
ans =
```

```
1.0167
```

`yearfrac` for Basis 9 (ACT/360 ICMA) calculates $366/360 = 1.0167$. So, even if the dates have the same month and date, with a difference of 1 in the year, the returned value may not be exactly 1. On the other hand, `date2time` calculates one full year period:

```
date2time('1/1/2000', '1/1/2001', 1, 9)
```

```
ans =
```

```
1
```

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `date2time` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
```

```
y = year(t)
```

```
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

See Also

`cftimes` | `disc2rate` | `rate2disc` | `time2date` | `yearfrac` | `datetime`

Topics

“Handle and Convert Dates” on page 2-2

dateaxis

Convert serial-date axis labels to calendar-date axis labels

Syntax

```
dateaxis(Tickaxis,DateForm,StartDate)
```

Description

`dateaxis(Tickaxis,DateForm,StartDate)` replaces axis tick labels with date labels.

Examples

Replace Axis Tick Labels with Date Labels

This example shows how to use `dateaxis` to replace axis tick labels with date labels on a graphic figure.

Convert the x-axis labels to an automatically determined date format.

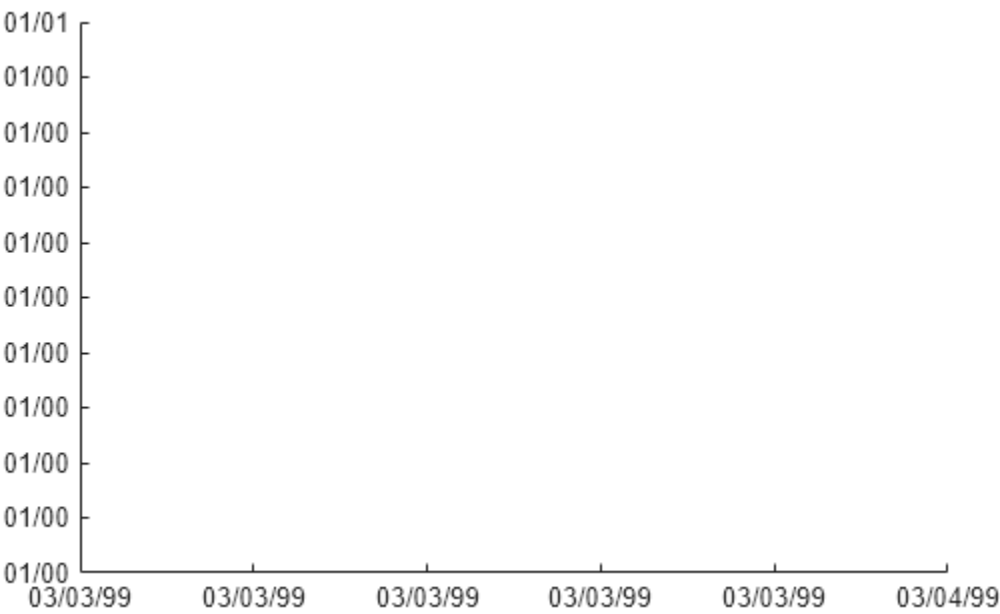
```
dateaxis('y', 6)
```

Convert the y-axis labels to the month/day format.

```
dateaxis('x', 2, datetime(1999,3,3))
```

Convert the x-axis labels to the month/day/year format. The minimum x-tick value is treated as March 3, 1999.

```
dateaxis('x', 2, datetime(1999,3,3))
```



Input Arguments

Tickaxis — Determines which axis tick labels, x, y, z to replace

'x' (default) | character vector with value 'x', 'y', or 'z'

(Optional) Determines which axis tick labels —x, y, or z— to replace, specified as a character vector.

Data Types: char

DateForm — Defines which date format to use

date format based on the span of the axis limits (default) | integer from 0 to 17

(Optional) Defines which date format to use, specified as an integer from 0 to 17.

If no **DateForm** argument is entered, this function determines the date format based on the span of the axis limits. For example, if the difference between the axis minimum and maximum is less than 15, the tick labels are converted to three-letter day-of-the-week abbreviations (**DateForm** = 8).

DateForm	Format	Description
0	01-Mar-1999 15:45:17	day-month-year hour:minute:second
1	01-mar-1999	day-month-year
2	03/01/99	month/day/year
3	Mar	month, three letters
4	M	month, single letter
5	3	month
6	03/01	month/day

DateForm	Format	Description
7	1	day of month
8	Wed	day of week, three letters
9	W	day of week, single letter
10	1999	year, four digits
11	99	year, two digits
12	Mar99	month year
13	15:45:17	hour:minute:second
14	03:45:17 PM	hour:minute:second AM or PM
15	15:45	hour:minute
16	03:45 PM	hour:minute AM or PM
17	95/03/01	year month day

Refer to the MATLAB `set` command for information on modifying the axis tick values and other axis parameters.

Data Types: double

StartDate — Assigns the date to the first axis tick value

lower axis limit converted to the appropriate date number (default) | datetime scalar | string scalar | date character vector

(Optional) Assigns the date to the first axis tick value, specified as a scalar datetime, string, or date character vector.

The default `StartDate` is the lower axis limit converted to the appropriate date number. For example, a tick value of 1 is converted to the date 01-Jan-0000. Entering `StartDate` as `'06-apr-1999'` assigns the date April 6, 1999 to the first tick value and the axis tick labels are set accordingly.

To support existing code, `dateaxis` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `dateaxis` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

`y =`

`2021`

There are no plans to remove support for serial date number inputs.

See Also

`bollinger` | `candle` | `highlow` | `movavg` | `pointfig` | `datetime`

datedisp

Display date entries

Syntax

```
datedisp(NumMat)
datedisp( ___,DateForm)
CharMat = datedisp(NumMat,DateForm)
```

Description

`datedisp(NumMat)` displays a matrix with the serial dates formatted as date character vectors, using a matrix with mixed numeric entries and serial date number entries. Integers between `datenum('01-Jan-1900')` and `datenum('01-Jan-2200')` are assumed to be serial date numbers, while all other values are treated as numeric entries.

`datedisp( ___,DateForm)`, using the optional argument `DateForm`, displays a matrix with the serial dates formatted as date character vectors, using a matrix with mixed numeric entries and serial date number entries. Integers between `datenum('01-Jan-1900')` and `datenum('01-Jan-2200')` are assumed to be serial date numbers, while all other values are treated as numeric entries.

`CharMat = datedisp(NumMat,DateForm)` returns `CharMat`, character array representing `NumMat`. If no output variable is assigned, the function prints the array to the display.

Examples

Display a Matrix with the Serial Dates Formatted as Date Character Vectors

This example shows how to display a matrix with the serial dates formatted as date character vectors.

```
NumMat = [730730, 0.03, 1200 730100;
          730731, 0.05, 1000 NaN];
```

```
datedisp(NumMat)
```

```
01-Sep-2000    0.03    1200    11-Dec-1998
02-Sep-2000    0.05    1000         NaN
```

Input Arguments

NumMat — Numeric matrix to display

serial date numbers

Numeric matrix to display, specified as serial date numbers.

Data Types: `double`

DateForm — Date format

scalar character vector to indicate format of text representing dates

Date format, specified as a scalar character vector to indicate the format of text representing dates.

Data Types: char

Output Arguments**CharMat — Character array representing NumMat**

array of date character vectors

Character array representing NumMat, returned as an array of date character vectors. If no output variable is assigned, the function prints the array to the display.

Version History

Introduced before R2006a

See Also

datetime

Topics

“Handle and Convert Dates” on page 2-2

datefind

Indices of dates in matrix

Syntax

```
Indices = datefind(Subset,Superset)
Indices = datefind( ___,Tolerance)
```

Description

`Indices = datefind(Subset,Superset)` returns a vector of indices to the date numbers in `Superset` that are present in `Subset`. If no date numbers match, `Indices = []`.

`Indices = datefind( ___,Tolerance)` returns a vector of indices to the date numbers in `Superset` that are present in `Subset`, plus the optional argument for `Tolerance`. If no date numbers match, `Indices = []`.

Examples

Return a Vector of Indices to Date Numbers

This example shows how to return a vector of indices to date numbers.

```
Superset = datetime(1999,7,1:31);
Subset = [datetime(1999,7,10) ; datetime(1999,7,20)];
Indices = datefind(Subset, Superset, 1)
```

```
Indices = 6×1
```

```
     9
    10
    11
    19
    20
    21
```

Input Arguments

Subset — Subset of dates to find matching dates

datetime array | string array | date character vector

Subset of dates to find matching dates in `Superset`, specified as a matrix using a datetime array, string array, or date character vectors.

`Subset` and `Superset` can be either be a datetime array, string array, or date character vectors. These types do not have to match. `datefind` determines the underlying date to match dates of different data types.

Note The elements of `Subset` must be contained in `Superset`, without repetition. `datefind` works with non-repeating sequences of dates.

Example: `Subset = [datetime(1997,7,10); datetime(1997,7,20)];`

To support existing code, `datefind` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Superset — Superset of dates

`datetime array` | `string array` | `date character vector`

Superset of dates, specified as a matrix of using a `datetime` array, `string` array, or `date` character vectors, whose elements are sought.

`Subset` and `Superset` can be either a `datetime` array, `string` array, or `date` character vectors. These types do not have to match. `datefind` determines the underlying date to match dates of different data types.

Note The elements of `Subset` must be contained in `Superset`, without repetition. `datefind` works with non-repeating sequences of dates.

Example: `Superset = datetime(1997,7,1:31);`

To support existing code, `datefind` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Tolerance — Tolerance for matching dates in Superset

`0` (default) | `positive integer` or `duration object`

Tolerance for matching dates (+/-) in `Superset`, specified as a `positive integer` or `duration object`.

Data Types: `single` | `double`

Output Arguments

Indices — Indices of dates in Superset that are present in Subset

`vector`

Indices of dates in `Superset` that are present in `Subset` (plus or minus the tolerance if defined using the optional argument `Tolerance`), returned as a `vector` of indices.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `datefind` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`datetime`

Topics

"Handle and Convert Dates" on page 2-2

datemnth

Date of day in future or past month

Syntax

```
TargetDate = datemnth(StartDate,NumberMonths)
TargetDate = datemnth( ____,DayFlag,Basis,EndMonthRule)
```

Description

`TargetDate = datemnth(StartDate,NumberMonths)` determines a date in a future or past month based on movement either forward or backward in time by a given number of months.

Any input can contain multiple values, but if so, all other inputs must contain the same number of values or a single value that applies to all. For example, if `StartDate` is an n -row datetime, then `NumberMonths` must be an N-by-1 vector of integers or a single integer. `TargetDate` is then an N-by-1 vector of datetimes.

If `StartDate` is a string or date character vector, `TargetDate` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

If `StartDate` is a datetime array, then `TargetDate` is returned as a datetime array.

`TargetDate = datemnth( ____,DayFlag,Basis,EndMonthRule)` determines a date in a future or past month based on movement either forward or backward in time by a given number of months, using optional input arguments for `DayFlag`, `Basis`, and `EndMonthRule`.

Examples

Determine the Dates of Days in a Future Month

Determine the `TargetDate` in a future month using a datetime for `StartDate`.

```
StartDate = datetime(1997,6,3);
NumberMonths = 6;
DayFlag = 0;
Basis = 0;
EndMonthRule = 1;
```

```
TargetDate = datemnth(StartDate, NumberMonths, DayFlag,Basis, EndMonthRule)
```

```
TargetDate = datetime
    03-Dec-1997
```

Determine the `TargetDate` in a future month using a vector for `NumberMonths`.

```
NumberMonths = [1; 3; 5; 7; 9];
TargetDate = datemnth(datetime(2001,1,31), NumberMonths)
```



```
TargetDate = 5x1 datetime
    28-Feb-2001
    30-Apr-2001
    30-Jun-2001
    31-Aug-2001
    31-Oct-2001
```

Input Arguments

StartDate — Start date

datetime array | string array | date character vector

Start date, specified as an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `datemnth` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

NumberMonths — Number of months in future (positive) or past (negative)

positive or negative integers

Number of months in future (positive) or past (negative), specified as an N-by-1 or 1-by-N vector containing positive or negative integers.

Data Types: `double`

DayFlag — Flag for how actual day number for target date in future or past month is determined

0 (day number should be the day in the future or past month corresponding to the actual day number of the start date) (default) | numeric with values 0, 1, or 2

Flag for how the actual day number for the target date in future or past month is determined, specified as an N-by-1 or 1-by-N vector using a numeric with values 0, 1, or 2.

Possible values are:

- 0 (default) = day number should be the day in the future or past month corresponding to the actual day number of the start date.
- 1 = day number should be the first day of the future or past month.
- 2 = day number should be the last day of the future or past month.

This flag has no effect if `EndMonthRule` is set to 1.

Data Types: `double`

Basis — Day-count basis of the instrument

0 (actual/actual) (default) | numeric with value 0 through 13 | vector of numerics with values 0 through 13

Day-count basis to be used when determining the past or future date, specified as a scalar value with an integer with value of 0 through 13, or an N-by-1 or 1-by-N vector of integers with values of 0 through 13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `single` | `double`

EndMonthRule — End-of-month rule flag for month having 30 or fewer days

0 (not in effect) (default) | nonnegative integer [0, 1]

End-of-month rule flag for month having 30 or fewer days, specified as a scalar with a nonnegative integer 0 or 1, or as an N-by-1 or 1-by-N vector of values 0 or 1.

- 0 = Ignore rule, meaning that rule is not in effect.
- 1 = Set rule on, meaning that if you are beginning on the last day of a month, and the month has 30 or fewer days, you will end on the last actual day of the future or past month regardless of whether that month has 28, 29, 30 or 31 days.

Note `EndOfMonthRule` *only applies* when you input a date that corresponds to the end of month as the `StartDate`. For example, if you enter 02/28/2023 as the `StartDate`, the `EndOfMonthRule` guarantees that the return date matches the last day of the month in the target output date.

```
datestr(datemnth('02/28/2023',4,1,0,1))
```

```
ans =
```

```
'30-Jun-2023'
```

`datemnth` returns the target date as the last day of June even though the `DayFlag` is specified to return the first day of June.

Data Types: `logical`

Output Arguments

TargetDate — Target date in the future or past month

vector

Target date in the future or past month, returned as an N-by-1 or 1-by-N vector.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `datemnth` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datetime");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`datevec` | `days360` | `days365` | `daysact` | `daysdif` | `wrkdydif` | `datetime`

Topics

“Handle and Convert Dates” on page 2-2

“Trading Calendars User Interface” on page 12-2

“UICalendar User Interface” on page 12-4

datewrkdy

Date of future or past workday

Syntax

```
EndDate = datewrkdy(StartDate, NumberWorkDays, NumberHolidays)
```

Description

`EndDate = datewrkdy(StartDate, NumberWorkDays, NumberHolidays)` returns the datetime of the date a given number of workdays before or after the start date.

Any input can contain multiple values, but if so, all other inputs must contain the same number of values or a single value that applies to all.

For example, if `StartDate` is an n -row datetime, then `NumberWorkDays` must be an N -by-1 vector of integers or a single integer. `EndDate` is then an N -by-1 vector of datetimes.

If `StartDate` is a string or date character vector, `EndDate` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

If `StartDate` is a datetime array, then `EndDate` is returned as a datetime array.

Examples

Determine the Date for a Future Workday

Determine the `EndDate` for a future workday using a date character vector for `StartDate`.

```
StartDate = '20-Dec-1994';  
NumberWorkDays = 16;  
NumberHolidays = 2;
```

```
EndDate = datewrkdy(StartDate, NumberWorkDays, NumberHolidays)
```

```
EndDate =  
728671
```

```
datestr(EndDate)
```

```
ans =  
'12-Jan-1995'
```

Determine the `EndDate` for a future workday using a datetime for `StartDate`.

```
EndDate = datewrkdy(datetime(2000,12,12), 16, 2)
```

```
EndDate = datetime  
04-Jan-2001
```

Determine the `EndDate` for future workdays using a vector for `NumberWorkDays`.

```

NumberWorkDays = [16; 20; 44];
EndDate = datewrkdy(datetime(2000,12,12), NumberWorkDays, 2)

EndDate = 3×1 datetime
    04-Jan-2001
    10-Jan-2001
    13-Feb-2001

```

Input Arguments

StartDate — Start date

datetime array | string array | date character vector

Start date, specified as an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, datewrkdy also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

NumberWorkDays — Number of work or business days in future (positive) or past (negative)

positive or negative integers

Number of work or business days in future (positive) or past (negative) that includes the starting date, specified as an N-by-1 or 1-by-N vector containing positive or negative integers.

NumberHolidays and NumberWorkDays must have the same sign.

Data Types: double

NumberHolidays — Number of holidays within NumberWorkDays

positive or negative integers

Number of holidays within NumberWorkDays, specified as positive or negative integers using an N-by-1 or 1-by-N containing values for the number of days movement in terms of holidays into the future (if positive) or past (if negative).

NumberHolidays and NumberWorkDays must have the same sign.

Data Types: double

Output Arguments

EndDate — Date of future or past workday

vector

Date of future or past workday, returned as an N-by-1 or 1-by-N vector for the future or past date.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `datenum` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`busdate` | `holidays` | `isbusday` | `wrkdydif` | `datetime`

Topics

"Handle and Convert Dates" on page 2-2

"Trading Calendars User Interface" on page 12-2

"UICalendar User Interface" on page 12-4

days252bus

Number of business days between dates

Syntax

```
NumberDays = days252bus(StartDate,EndDate)
NumberDays = days252bus(____HolidayVector)
```

Description

`NumberDays = days252bus(StartDate,EndDate)` computes the number of business days (that is, non-holiday or non-weekend) between the two input dates.

`NumberDays = days252bus(____HolidayVector)` adds an optional argument for `HolidayVector`. If a holiday vector is not optionally specified, then the `holidays.m` file is used to determine the holidays.

Examples

Computes the Number of Business Days Between Two Input Dates

This example shows how to compute the number of business days (i.e. non-holiday or non-weekend) between two dates using the `days252bus` convention.

```
NumberDays = days252bus(datetime(2009,1,1), datetime(2009,8,1))
```

```
NumberDays =
146
```

Input Arguments

StartDate — Start date

datetime array | string array | date character vector

Start date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `days252bus` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: double | char | string | datetime

EndDate — End date

datetime array | string array | date character vector

End date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `days252bus` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

HolidayVector — Holidays

`holidays.m` (default) | `datetime` array | `string` array | `date` character vector | `serial` date number

Holidays, specified as a scalar or an N-by-1 or 1-by-N vector using a `datetime` array, `string` array, or `date` character vectors.

To support existing code, `days252bus` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Output Arguments

NumberDays — Number of days between two dates

`vector`

Number of days between two dates, returned as a scalar or an N-by-1 or 1-by-N vector containing the number of days.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `days252bus` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`days360psa` | `daysact` | `daysdif` | `days365` | `datetime`

Topics

“Handle and Convert Dates” on page 2-2

days360

Days between dates based on 360-day year

Syntax

```
NumDays = days360(StartDate,EndDate)
```

Description

`NumDays = days360(StartDate,EndDate)` returns the number of days between `StartDate` and `EndDate` based on a 360-day year (that is, all months contain 30 days). If `EndDate` is earlier than `StartDate`, `NumDays` is negative.

Either input argument can contain multiple values, but if so, the other must contain the same number of values or a single value that applies to all. For example, if `StartDate` is an n -row datetime, then `EndDate` must be an N -by-1 vector of integers or a single integer. `NumDays` is then an N -by-1 vector of date numbers.

Examples

Determine the Number of Days Between Two Dates Based on a 360-Day Year

Determine the `NumDays` using date character vectors for `StartDate` and `EndDate`.

```
NumDays = days360('15-jan-2000', '15-mar-2000')
```

```
NumDays =  
60
```

Determine the `NumDays` using a datetime for `StartDate` and `EndDate`.

```
NumDays = days360(datetime(2000,1,15) , datetime(2000,3,15))
```

```
NumDays =  
60
```

Determine the `NumDays` using a datetime array for `EndDate`.

```
MoreDays = [datetime(2000,3,15) ; datetime(2000,4,15) ; datetime(2000,6,15)];  
NumDays = days360(datetime(2000,1,15), MoreDays)
```

```
NumDays = 3×1  
  
60  
90  
150
```

Input Arguments

StartDate — Start date

`datetime array` | `string array` | `date character vector`

Start date, specified as a scalar or an N-by-1 or 1-by-N vector using a `datetime` array, `string` array, or `date` character vectors.

To support existing code, `days360` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

EndDate — End date

`datetime array` | `string array` | `date character vector`

End date, specified as a scalar or an N-by-1 or 1-by-N vector using a `datetime` array, `string` array, or `date` character vectors.

To support existing code, `days360` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Output Arguments

NumDays — Number of days between two dates

`vector`

Number of days between two dates, returned as a scalar or an N-by-1 or 1-by-N vector containing the number of days.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `days360` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

References

[1] Addendum to Securities Industry Association, *Standard Securities Calculation Methods: Fixed Income Securities Formulas for Analytic Measures*. Vol. 2, Spring 1995.

See Also

days365 | daysact | daysdif | wrkdydif | yearfrac | datetime

Topics

"Handle and Convert Dates" on page 2-2

days360e

Days between dates based on 360-day year (European)

Syntax

```
NumDays = days360e(StartDate,EndDate)
```

Description

`NumDays = days360e(StartDate,EndDate)` returns the number of days between `StartDate` and `EndDate` based on a 360-day year (that is, all months contain 30 days). If `EndDate` is earlier than `StartDate`, `NumDays` is negative. This day count convention is used primarily in Europe. Under this convention, all months contain 30 days.

Either input argument can contain multiple values, but if so, the other must contain the same number of values or a single value that applies to all. For example, if `StartDate` is an n -row datetime, then `EndDate` must be an N -by-1 vector of integers or a single integer. `NumDays` is then an N -by-1 vector of date numbers.

Examples

Determine the Number of Days Between Two Dates Given a Basis of 30/360 Based on European Convention

Determine the `NumDays` using date character vectors for `StartDate` and `EndDate` for the month of January.

```
StartDate = '1-Jan-2002';  
EndDate = '1-Feb-2002';  
NumDays = days360e(StartDate, EndDate)
```

```
NumDays =  
30
```

Determine the `NumDays` in the month of January using datetimes for `StartDate` and `EndDate`.

```
NumDays = days360e(datetime(2002,1,1), datetime(2002,2,1))
```

```
NumDays =  
30
```

Determine the `NumDays` using a datetime array for `EndDate`.

```
MoreDays = [datetime(2000,3,15) ; datetime(2000,4,15) ; datetime(2000,6,15)];  
NumDays = days360e(datetime(2000,1,15), MoreDays)
```

```
NumDays = 3×1  
  
60  
90
```

Input Arguments

StartDate — Start date

datetime array | string array | date character vector

Start date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `days360e` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

EndDate — End date

datetime array | string array | date character vector

End date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `days360e` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Output Arguments

NumDays — Number of days between two dates given a basis of 30/360 based on European convention

vector

Number of days between two dates given a basis of 30/360 based on European convention, returned as a scalar or an N-by-1 or 1-by-N vector containing the number of days.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `days360e` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

y =

2021

There are no plans to remove support for serial date number inputs.

References

[1] Addendum to Securities Industry Association, *Standard Securities Calculation Methods: Fixed Income Securities Formulas for Analytic Measures*. Vol. 2, Spring 1995.

See Also

[days360](#) | [days360isda](#) | [days360psa](#) | [datetime](#)

Topics

“Handle and Convert Dates” on page 2-2

days360isda

Days between dates based on 360-day year (International Swap Dealer Association (ISDA) compliant)

Syntax

```
NumDays = days360isda(StartDate,EndDate)
```

Description

`NumDays = days360isda(StartDate,EndDate)` returns the number of days between `StartDate` and `EndDate` based on a 360-day year (that is, all months contain 30 days) and is International Swap Dealer Association (ISDA) compliant. If `EndDate` is earlier than `StartDate`, `NumDays` is negative. Under this convention, all months contain 30 days.

Either input argument can contain multiple values, but if so, the other must contain the same number of values or a single value that applies to all. For example, if `StartDate` is an n -row datetime, then `EndDate` must be an N -by-1 vector of integers or a single integer. `NumDays` is then an N -by-1 vector of date numbers.

Examples

Determine the Number of Days Between Two Dates Given a Basis of 30/360 Based on ISDA Compliance

Determine the `NumDays` using date character vectors for `StartDate` and `EndDate` for the month of January.

```
StartDate = '1-Jan-2002';
EndDate = '1-Feb-2002';
NumDays = days360isda(StartDate, EndDate)
```

```
NumDays =
    30
```

Determine the `NumDays` in the month of January using datetimes for `StartDate` and `EndDate`.

```
NumDays = days360isda(datetime(2002,1,1), datetime(2002,2,1))
```

```
NumDays =
    30
```

Determine the `NumDays` using a datetime array for `EndDate`.

```
MoreDays = [datetime(2000,3,15) ; datetime(2000,4,15) ; datetime(2000,6,15)];
NumDays = days360isda(datetime(2000,1,15), MoreDays)
```

```
NumDays = 3×1

    60
    90
```

150

Input Arguments

StartDate — Start date

datetime array | string array | date character vector

Start date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `days360isda` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

EndDate — End date

datetime array | string array | date character vector

End date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `days360isda` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Output Arguments

NumDays — Number of days between two dates given a basis of 30/360 based on European convention

vector

Number of days between two dates given a basis of 30/360 based on International Swap Dealer Association (ISDA) compliance, returned as a scalar or an N-by-1 or 1-by-N vector containing the number of days.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `days360isda` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```


y =

2021

There are no plans to remove support for serial date number inputs.

References

[1] Addendum to Securities Industry Association, *Standard Securities Calculation Methods: Fixed Income Securities Formulas for Analytic Measures*. Vol. 2, Spring 1995.

See Also

days360 | days360e | datetime

Topics

“Handle and Convert Dates” on page 2-2

days360psa

Days between dates based on 360-day year (Public Securities Association (PSA) compliant)

Syntax

```
NumDays = days360psa(StartDate,EndDate)
```

Description

`NumDays = days360psa(StartDate,EndDate)` returns the number of days between `StartDate` and `EndDate` based on a 360-day year (that is, all months contain 30 days) and is Public Securities Association (PSA) compliant. If `EndDate` is earlier than `StartDate`, `NumDays` is negative. Under this convention, all months contain 30 days.

Either input argument can contain multiple values, but if so, the other must contain the same number of values or a single value that applies to all. For example, if `StartDate` is an n -row datetime, then `EndDate` must be an N -by-1 vector of integers or a single integer. `NumDays` is then an N -by-1 vector of date numbers.

Examples

Determine the Number of Days Between Two Dates Given a Basis of 30/360 Based on PSA Compliance

Determine the `NumDays` using date character vectors for `StartDate` and `EndDate` for the month of January.

```
StartDate = '1-Jan-2002';  
EndDate = '1-Feb-2002';  
NumDays = days360psa(StartDate, EndDate)
```

```
NumDays =  
30
```

Determine the `NumDays` in the month of January using datetimes for `StartDate` and `EndDate`.

```
NumDays = days360psa(datetime(2002,1,1) , datetime(2002,2,1))
```

```
NumDays =  
30
```

Determine the `NumDays` using a datetime array for `EndDate`.

```
MoreDays = [datetime(2000,3,15) ; datetime(2000,4,15) ; datetime(2000,6,15)];  
NumDays = days360psa(datetime(2000,1,15), MoreDays)
```

```
NumDays = 3×1  
  
60  
90
```

Input Arguments

StartDate — Start date

datetime array | string array | date character vector

Start date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `days360psa` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

EndDate — End date

datetime array | string array | date character vector

End date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `days360psa` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Output Arguments

NumDays — Number of days between two dates given a basis of 30/360 based on European convention

vector

Number of days between two dates given a basis of 30/360 based on Public Securities Association (PSA) compliance, returned as a scalar or an N-by-1 or 1-by-N vector containing the number of days.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `days360psa` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

y =

2021

There are no plans to remove support for serial date number inputs.

References

[1] Addendum to Securities Industry Association, *Standard Securities Calculation Methods: Fixed Income Securities Formulas for Analytic Measures*. Vol. 2, Spring 1995.

See Also

[days360](#) | [days360e](#) | [days360isda](#) | [datetime](#)

Topics

“Handle and Convert Dates” on page 2-2

days365

Days between dates based on 365-day year

Syntax

```
NumDays = days365(StartDate,EndDate)
```

Description

`NumDays = days365(StartDate,EndDate)` returns the number of days between `StartDate` and `EndDate` based on a 365-day year. All months contain their actual number of days. February always contains 28 days.

If `EndDate` is earlier than `StartDate`, `NumDays` is negative. Under this convention, all months contain 30 days.

Either input argument can contain multiple values, but if so, the other must contain the same number of values or a single value that applies to all. For example, if `StartDate` is an n -row datetime, then `EndDate` must be an N -by-1 vector of integers or a single integer. `NumDays` is then an N -by-1 vector of date numbers.

Examples

Determine the Number of Days Between Two Dates Based on a 365-Day Year

Determine the `NumDays` using date character vectors for `StartDate` and `EndDate`.

```
NumDays = days365('15-jan-2000', '15-mar-2000')
```

```
NumDays =  
59
```

Determine the `NumDays` using datetimes for `StartDate` and `EndDate`.

```
NumDays = days365(datetime(2000,1,15) , datetime(2000,3,15))
```

```
NumDays =  
59
```

Determine the `NumDays` using a datetime array for `EndDate`.

```
MoreDays = [datetime(2000,3,15) ; datetime(2000,4,15) ; datetime(2000,6,15)];  
NumDays = days365('15-jan-2000', MoreDays)
```

```
NumDays = 3×1  
  
59  
90  
151
```

Input Arguments

StartDate — Start date

datetime array | string array | date character vector

Start date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `days365` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

EndDate — End date

datetime array | string array | date character vector

End date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `days365` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Output Arguments

NumDays — Number of days between two dates based on 365-day year

vector

Number of days between two dates based on a 365-day year, returned as a scalar or an N-by-1 or 1-by-N vector containing the number of days.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `days365` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

References

[1] Addendum to Securities Industry Association, *Standard Securities Calculation Methods: Fixed Income Securities Formulas for Analytic Measures*. Vol. 2, Spring 1995.

See Also

days360 | daysact | daysdif | wrkdydif | yearfrac | datetime

Topics

"Handle and Convert Dates" on page 2-2

daysact

Actual number of days between dates

Syntax

```
NumDays = daysact(StartDate)
NumDays = daysact( ___,EndDate)
```

Description

`NumDays = daysact(StartDate)` returns the actual number of days between the MATLAB base date and `StartDate`. In MATLAB, the base date 1 is 1-Jan-0000 A.D.

`NumDays = daysact( ___,EndDate)` returns the actual number of days between `StartDate` and the optional argument `EndDate`.

If `EndDate` is earlier than `StartDate`, `NumDays` is negative. Under this convention, all months contain 30 days.

Either input argument can contain multiple values, but if so, the other must contain the same number of values or a single value that applies to all. For example, if `StartDate` is an n -row datetime, then `EndDate` must be an N -by-1 vector of integers or a single integer. `NumDays` is then an N -by-1 vector of date numbers.

Examples

Determine the Number of Days Between Two Dates Based the Actual Number of Days

Determine the `NumDays` using date character vectors for `StartDate` and `EndDate`.

```
NumDays = daysact('7-sep-2002', '25-dec-2002')
```

```
NumDays =
109
```

Determine the `NumDays` using datetimes for `StartDate` and `EndDate`.

```
NumDays = daysact(datetime(2002,9,7) , datetime(2002,12,25))
```

```
NumDays =
109
```

Determine the `NumDays` using a datetime array for `EndDate`.

```
MoreDays = [datetime(2002,9,7) ; datetime(2002,10,22) ; datetime(2002,11,5)];
NumDays = daysact(MoreDays, '12/25/2002')
```

```
NumDays = 3×1

109
64
```


Input Arguments

StartDate — Start date

datetime array | string array | date character vector

Start date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `daysact` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

EndDate — End date

datetime array | string array | date character vector

End date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `daysact` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Output Arguments

NumDays — Number of days between two dates based actual number of days

vector

Number of days between two dates based on the actual number of days, returned as a scalar or an N-by-1 or 1-by-N vector containing the number of days.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `daysact` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

References

[1] Addendum to Securities Industry Association, *Standard Securities Calculation Methods: Fixed Income Securities Formulas for Analytic Measures*. Vol. 2, Spring 1995.

See Also

`datevec` | `days360` | `days365` | `daysdif` | `datetime`

Topics

“Handle and Convert Dates” on page 2-2

“Trading Calendars User Interface” on page 12-2

“UICalendar User Interface” on page 12-4

daysadd

Date away from starting date for any day-count basis

Syntax

```
NewDate = daysadd(StartDate,NumDays)
NewDate = daysadd( ____,Basis)
```

Description

`NewDate = daysadd(StartDate,NumDays)` returns a date `NewDate` number of days away from `StartDate`.

If `StartDate` is a string or date character vector, `NewDate` is returned as a serial date number.

If `StartDate` is a datetime array, then `NewDate` is returned as a datetime array.

`NewDate = daysadd( ____,Basis)` returns a date `NewDate` number of days away from `StartDate`, using the optional argument `Basis` for day-count.

If `StartDate` is a string or date character vector, `NewDate` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

If `StartDate` is a datetime array, then `NewDate` is returned as a datetime array.

Examples

Determine the Date for Given Number of Days Away From StartDate

Determine the `NewDate` using a date character vector for `StartDate`.

```
NewDate = daysadd('01-Feb-2004', 31)
```

```
NewDate =
732009
```

```
datestr(NewDate)
```

```
ans =
'03-Mar-2004'
```

Determine the `NewDate` using a datetime for `StartDate`.

```
NewDate = daysadd(datetime(2004,2,1), 31)
```

```
NewDate = datetime
03-Mar-2004
```

Determine the `NewDate` using a datetime array for `StartDate`.

```
MoreDays = [datetime(2002,9,7) ; datetime(2002,10,22) ; datetime(2002,11,5)];  
NewDate = daysadd(MoreDays, 31 ,2)
```

```
NewDate = 3×1 datetime  
08-Oct-2002  
22-Nov-2002  
06-Dec-2002
```

Input Arguments

StartDate — Start date

datetime array | string array | date character vector

Start date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `daysadd` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

NumDays — Number of days from StartDate

positive or negative integer

Number of days from `StartDate`, specified an N-by-1 or 1-by-N vector using positive or negative integers. Enter a negative integer for dates before start date.

Data Types: `double`

Basis — Day-count basis of the instrument

0 (actual/actual) (default) | numeric with value 0 through 13 | vector of numerics with values 0 through 13

Day-count basis of the instrument, specified as an integer with a value of 0 through 13 or a N-by-1 vector of integers with values of 0 through 13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)

- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note When using the 30/360 day-count basis, it is not always possible to find the exact date `NewDate` number of days away because of a known discontinuity in the method of counting days. A warning is displayed if this occurs.

Data Types: double

Output Arguments

NewDate — Date for given number of days away from StartDate

vector

Date for given number of days away from `StartDate`, returned as a scalar or an N-by-1 vector containing dates.

If `StartDate` is a string or date character vector, `NewDate` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

If `StartDate` is a datetime array, then `NewDate` is returned as a datetime array.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `daysadd` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] Stigum, Marcia L. and Franklin Robinson. *Money Market and Bond Calculations*. Richard D. Irwin, 1996, ISBN 1-55623-476-7

See Also

`daysdif` | `datetime`

Topics

“Handle and Convert Dates” on page 2-2

“Trading Calendars User Interface” on page 12-2

“UICalendar User Interface” on page 12-4

daysdif

Days between dates for any day-count basis

Syntax

```
NumDays = daysdif(StartDate,EndDate)
NumDays = daysdif(____,Basis)
```

Description

`NumDays = daysdif(StartDate,EndDate)` returns the number of days between dates `StartDate` and `EndDate`. The first date for `StartDate` is not included when determining the number of days between first and last date.

Any input argument can contain multiple values, but if so, the other inputs must contain the same number of values or a single value that applies to all. For example, if `StartDate` is an *n*-row cell array of character vectors, then `EndDate` must be an *n*-row cell array of character vectors or a single date character vector. `NumDays` is then an N-by-1 vector of numbers.

`NumDays = daysdif(____,Basis)` returns the number of days between dates `StartDate` and `EndDate` using the optional argument `Basis` for day-count. The first date for `StartDate` is not included when determining the number of days between first and last date.

Any input argument can contain multiple values, but if so, the other inputs must contain the same number of values or a single value that applies to all. For example, if `StartDate` is an *n*-row datetime array, then `EndDate` must be an *n*-row datetime array or a single datetime. `NumDays` is then an N-by-1 vector of numbers.

Examples

Determine the Number of Days Between StartDate and EndDate

Determine the `NumDays` using date character vectors for `StartDate` and `EndDate`.

```
NumDays = daysdif('3/1/99', '3/1/00', 1)

NumDays =
360
```

Determine the `NumDays` using datetimes for `StartDate` and `EndDate`.

```
NumDays = daysdif(datetime(1999,3,1),datetime(2000,3,1), 1)

NumDays =
360
```

Determine the `NumDays` using a datetime array for `EndDate`.

```
MoreDays = [datetime(2001,3,1) ; datetime(2002,3,1) ; datetime(2003,3,1)];
NumDays = daysdif(datetime(1998,3,1), MoreDays)
```

```
NumDays = 3×1  
    1096  
    1461  
    1826
```

Input Arguments

StartDate — Start date

datetime array | string array | date character vector

Start date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `daysdif` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

EndDate — End date

datetime array | string array | date character vector

End date, specified as a scalar or an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `daysdif` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Basis — Day-count basis of the instrument

0 (actual/actual) (default) | numeric with value 0 through 13 | vector of numerics with values 0 through 13

Day-count basis of the instrument, specified as an integer with a value of 0 through 13 or a N-by-1 vector of integers with values of 0 through 13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)

- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

Output Arguments

NumDays — Number of days between dates **StartDate** and **EndDate**

integer

Number of days between the **StartDate** and **EndDate**. **NumDays** is returned as an integer.

Note The first date for **StartDate** is not included when determining the number of days between first and last date.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although **daysdif** supports serial date numbers, **datetime** values are recommended instead. The **datetime** data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to **datetime** values, use the **datetime** function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

References

[1] Stigum, Marcia L. and Franklin Robinson. *Money Market and Bond Calculations*. Richard D. Irwin, 1996, ISBN 1-55623-476-7

See Also

dec2thirtytwo | datetime

Topics

“Handle and Convert Dates” on page 2-2

dec2thirtytwo

Decimal to thirty-second quotation

Syntax

```
[OutNumber,Fractions] = dec2thirtytwo(InNumber,Accuracy)
```

Description

[OutNumber,Fractions] = dec2thirtytwo(InNumber,Accuracy) changes a decimal price quotation for a bond or bond future to a fraction with a denominator of 32.

Examples

Convert Decimal to Thirty-Second Quotation

This example shows two bonds that are quoted with decimal prices of 101.78 and 102.96. These prices are converted to fractions with a denominator of 32.

```
InNumber = [101.78; 102.96];  
[OutNumber, Fractions] = dec2thirtytwo(InNumber)
```

```
OutNumber = 2×1
```

```
101  
102
```

```
Fractions = 2×1
```

```
25  
31
```

Input Arguments

InNumber — Input number

numeric decimal fraction

Input number, specified as an N-by-1 vector of numeric decimal fractions.

Data Types: `double`

Accuracy — Rounding

1 (round down to nearest thirty second) (default) | numeric with values 1, 2, 4 or 10

Rounding, specified as an N-by-1 vector of accuracy desired with numeric values of 1, 2, 4 or 10. The values are: 1, round down to nearest thirty second, 2 (nearest half), 4 (nearest quarter), or 10 (nearest decile).

Data Types: double

Output Arguments

OutNumber — Output number which is InNumber rounded downward to closest integer
numeric

Output number which is InNumber rounded downward to closest integer, returned as a numeric value.

Fractions — Fractional part in units of thirty-second
numeric

Fractional part in units of thirty-second, returned as a numeric value. The Fractions output conforms to accuracy as prescribed by the input Accuracy.

Version History

Introduced before R2006a

See Also

thirtytwo2dec

depfixdb

Fixed declining-balance depreciation schedule

Syntax

```
Depreciation = depfixdb(Cost,Salvage,Life,Period)
Depreciation = depfixdb(____,Month)
```

Description

`Depreciation = depfixdb(Cost,Salvage,Life,Period)` computes the fixed declining-balance depreciation for each `Period`.

`Depreciation = depfixdb(____,Month)` adds an optional argument.

Examples

Compute the Fixed Declining-Balance Depreciation

This example shows how to calculate the depreciation for the first five years for a car is purchased for \$11,000, with a salvage value of \$1500, and a lifetime of eight years.

```
Depreciation = depfixdb(11000, 1500, 8, 5)
```

```
Depreciation = 1×5
103 ×
```

```
2.4251    1.8904    1.4737    1.1488    0.8955
```

Input Arguments

Cost — Initial value of the asset

numeric

initial value of the asset, specified as a scalar numeric.

Data Types: double

Salvage — Salvage value of the asset

numeric

Salvage value of the asset, specified as a scalar numeric.

Data Types: double

Life — Life of the asset in years

numeric

life of the asset in years, specified as scalar numeric.

Data Types: double

Period — Number of years to calculate

integer

Number of years to calculate, specified as scalar integer.

Data Types: double

Month — Number of months in the first year of asset life

12 (default) | numeric

(Optional) Number of months in the first year of asset life, specified as a scalar numeric.

Data Types: double

Output Arguments

Depreciation — Depreciation

vector

Depreciation, returned as the fixed declining-balance depreciation for each Period.

More About

Fixed Declining-Balance Depreciation Schedule

A fixed declining-balance depreciation schedule is a method of calculating depreciation that applies a constant percentage rate to the book value of an asset at the beginning of each accounting period.

This approach results in a larger depreciation expense in the earlier years of the asset's life, gradually decreasing the expense in subsequent years. It is particularly useful for assets that are expected to lose value more quickly in their initial years of usage.

Version History

Introduced before R2006a

See Also

depgendb | deprdv | depsoyd | depstln

Topics

"Analyzing and Computing Cash Flows" on page 2-11

depgendb

General declining-balance depreciation schedule

Syntax

Depreciation = depgendb(Cost,Salvage,Life,Factor)

Description

Depreciation = depgendb(Cost,Salvage,Life,Factor) computes the declining-balance depreciation for each period.

Examples

Calculate the Declining-Balance Depreciation

A car is purchased for \$10,000 and is to be depreciated over five years. The estimated salvage value is \$1000. Using the double-declining-balance method, the function calculates the depreciation for each year and returns the remaining depreciable value at the end of the life of the car.

Define the depreciation.

```
Life = 5;  
Salvage = 0;  
Cost = 10000;  
Factor=2;
```

Use depgendb to calculate the depreciation.

```
Depreciation = depgendb(10000, 1000, 5, 2)
```

```
Depreciation = 1×5  
103 ×
```

```
4.0000    2.4000    1.4400    0.8640    0.2960
```

The large value returned at the final year is the sum of the depreciation over the life time and is equal to the difference between the Cost and Salvage. The value of the asset in the final year is computed as (Cost - Salvage) = Sum_Depreciation_Upto_Final_Year.

Input Arguments

Cost — Initial value of the asset

numeric

initial value of the asset, specified as a scalar numeric.

Data Types: double

Salvage — Salvage value of the asset

numeric

Salvage value of the asset, specified as a scalar numeric.

Data Types: `double`

Life — Number of periods over which the asset is depreciated

numeric

Number of periods over which the asset is depreciated, specified as a scalar numeric.

Data Types: `double`

Factor — Depreciation factor

numeric

Depreciation factor, specified as a scalar numeric. When `Factor = 2`, then the double-declining-balance method is used.

Data Types: `double`

Output Arguments**Depreciation — Depreciation**

vector

Depreciation, returned as the declining-balance depreciation for each period.

More About**General Declining-Balance Depreciation Schedule**

A general declining-balance depreciation schedule is a method of calculating depreciation that applies a fixed percentage rate to the book value of an asset at the beginning of each period.

This approach results in a decreasing amount of depreciation expense over the asset's useful life, reflecting the declining value of the asset as it ages. It is commonly used for assets that lose value more quickly in their earlier years.

Version History

Introduced before R2006a

See Also

`depfixedb` | `deprdv` | `depsoyd` | `depstln`

Topics

“Analyzing and Computing Cash Flows” on page 2-11

deprdv

Remaining depreciable value

Syntax

`Depreciation = deprdv(Cost, Salvage, Accum)`

Description

`Depreciation = deprdv(Cost, Salvage, Accum)` computes the remaining depreciable value for an asset.

Examples

Compute the Remaining Depreciable Value for an Asset

This example shows how to find the remaining depreciable value after six years for the cost of an asset for \$13,000 with a life of 10 years. The salvage value is \$1000.

```
Accum = depstln(13000, 1000, 10) * 6
```

```
Accum =  
7200
```

```
Value = deprdv(13000, 1000, 7200)
```

```
Value =  
4800
```

Input Arguments

Cost — Initial value of the asset

numeric

Initial value of the asset, specified as a scalar numeric.

Data Types: double

Salvage — Salvage value of the asset

numeric

Salvage value of the asset, specified as a scalar numeric.

Data Types: double

Accum — Accumulated depreciation of the asset for prior periods

numeric

Accumulated depreciation of the asset for prior periods, specified as a scalar numeric.

Data Types: double

Output Arguments

Depreciation — Depreciation

numeric

Depreciation, returned as the remaining depreciable value for an asset.

Version History

Introduced before R2006a

See Also

depfixdb | depgendb | depsoyd | depstln

Topics

“Analyzing and Computing Cash Flows” on page 2-11

depsoyd

Sum of years' digits depreciation

Syntax

`Sum = depsoyd(Cost, Salvage, Life)`

Description

`Sum = depsoyd(Cost, Salvage, Life)` computes the depreciation for an asset using the sum of years' digits method.

Examples

Compute the Depreciation for an Asset Using the Sum of Years' Digits Method

This example shows how to calculate the depreciation for an asset using the sum of years' digits method where the asset is \$13,000 with a life of 10 years. The salvage value of the asset is \$1000.

`Sum = depsoyd(13000, 1000, 10)'`

`Sum = 10×1
103 ×`

2.1818
1.9636
1.7455
1.5273
1.3091
1.0909
0.8727
0.6545
0.4364
0.2182

Input Arguments

Cost — Initial value of the asset

numeric

initial value of the asset, specified as a scalar numeric.

Data Types: `double`

Salvage — Salvage value of the asset

numeric

Salvage value of the asset, specified as a scalar numeric.

Data Types: double

Life — Depreciable life of the asset in years

numeric

Depreciable life of the asset in years, specified as a scalar numeric.

Data Types: double

Output Arguments**Sum — Depreciation values**

vector

Depreciation values, returned as a 1-by-Life vector of depreciation values with each element corresponding to a year of the asset's life.

More About**Sum of Years' Digits Method**

The Sum of Years' Digits (SYD) method is an accelerated depreciation method that allocates a larger portion of an asset's cost to the earlier years of its useful life and a smaller portion to the later years.

Version History

Introduced before R2006a

See Also

depxdb | depxdb | depxdb | depxdb

Topics

"Analyzing and Computing Cash Flows" on page 2-11

depstln

Straight-line depreciation schedule

Syntax

Depreciation = depstln(Cost,Salvage,Life)

Description

Depreciation = depstln(Cost,Salvage,Life) computes the straight-line depreciation for an asset.

Examples

Compute the Straight-Line Depreciation for an Asset

This example shows how to calculate the straight-line depreciation for an asset that costs \$13,000 with a life of 10 years. The salvage value of the asset is \$1000.

```
Depreciation = depstln(13000, 1000, 10)
```

```
Depreciation =  
1200
```

Input Arguments

Cost — Initial value of the asset

numeric

Initial value of the asset, specified as a scalar numeric.

Data Types: double

Salvage — Salvage value of the asset

numeric

Salvage value of the asset, specified as a scalar numeric.

Data Types: double

Life — Depreciable life of the asset in years

numeric

Depreciable life of the asset in years, specified as a scalar numeric.

Data Types: double

Output Arguments

Depreciation — Depreciation

numeric

Depreciation, returned as a straight-line depreciation for an asset.

More About

Straight-Line Depreciation Schedule

A straight-line depreciation schedule is a method of calculating depreciation that applies the same amount of depreciation expense each year until the asset is fully depreciated.

Version History

Introduced before R2006a

See Also

depfixedb | depgendb | deprdv | depsoyd

Topics

“Analyzing and Computing Cash Flows” on page 2-11

disc2zero

Zero curve given discount curve

Syntax

```
[ZeroRates, CurveDates] = disc2zero(DiscRates, CurveDates, Settle)
[ZeroRates, CurveDates] = disc2zero( ___, Name, Value)
```

Description

`[ZeroRates, CurveDates] = disc2zero(DiscRates, CurveDates, Settle)` returns a zero curve given a discount curve and its maturity dates. If either inputs for `CurveDates` or `Settle` are datetime arrays, the output `CurveDates` is returned as datetime arrays.

`[ZeroRates, CurveDates] = disc2zero( ___, Name, Value)` adds optional name-value pair arguments

Examples

Determine the Zero Curve Given a Discount Curve and Maturity Dates

Given the following discount factors `DiscRates` over a set of maturity dates `CurveDates`, and a settlement date `Settle`:

```
DiscRates = [0.9996
              0.9947
              0.9896
              0.9866
              0.9826
              0.9786
              0.9745
              0.9665
              0.9552
              0.9466];

CurveDates = [datetime(2000,11,6)
              datetime(2000,12,11)
              datetime(2001,1,15)
              datetime(2001,2,5)
              datetime(2001,3,4)
              datetime(2001,4,2)
              datetime(2001,4,30)
              datetime(2001,6,25)
              datetime(2001,9,4)
              datetime(2001,11,12)];

Settle = datetime(2000,11,3);
```

Set daily compounding for the output zero curve, on an actual/365 basis.

```
Compounding = 365;
Basis = 3;
```

Execute the function `disc2zero` which returns the zero curve `ZeroRates` at the maturity dates `CurveDates`.

```
[ZeroRates, CurveDates] = disc2zero(DiscRates, CurveDates, Settle, Compounding, Basis)
```

```
ZeroRates = 10×1
```

```
0.0487
0.0510
0.0523
0.0524
0.0530
0.0526
0.0530
0.0532
0.0549
0.0536
```

```
CurveDates = 10×1 datetime
```

```
06-Nov-2000
11-Dec-2000
15-Jan-2001
05-Feb-2001
04-Mar-2001
02-Apr-2001
30-Apr-2001
25-Jun-2001
04-Sep-2001
12-Nov-2001
```

For readability, `DiscRates` and `ZeroRates` are shown here only to the basis point. However, MATLAB® software computed them at full precision. If you enter `DiscRates` as shown, `ZeroRates` may differ due to rounding.

Input Arguments

DiscRates — Discount factors

decimal fraction

Discount factors, specified as a `NDATES-by-1` column vector of decimal fractions. In aggregate, the factors in `DiscRates` constitute a discount curve for the investment horizon represented by `CurveDates`.

Data Types: double

CurveDates — Maturity dates

datetime array | string array | date character vector

Maturity dates that correspond to the discount factors in `DiscRates`, specified as a `NDATES-by-1` column vector using a datetime array, string array, or date character vectors.

To support existing code, `disc2zero` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `datetime` | `string` | `char`

Settle — Common settlement date for discount rates in `DiscRates`

`datetime` scalar | `string` scalar | `date` character vector | `serial date number`

Common settlement date for the discount rates in `DiscRates`, specified as scalar `datetime`, `string`, or `date` character vector.

To support existing code, `disc2zero` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `datetime` | `string` | `char`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `[ZeroRates, CurveDates] = disc2zero(DiscRates, CurveDates, Settle, 'Compounding', 6, 'Basis', 9)`

Compounding — Rate at which output zero rates are compounded when annualized

2 (default) | numeric values: 0, 1, 2, 3, 4, 6, 12, 365, -1

Rate at which the output zero rates are compounded when annualized, specified as a numeric value. Allowed values are:

- 0 — Simple interest (no compounding)
- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- 365 — Daily compounding
- -1 — Continuous compounding

Data Types: `double`

Basis — Day-count basis used for annualizing output zero rates

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day-count basis used for annualizing the output zero rates, specified as a numeric value. Allowed values are:

- 0 = actual/actual

- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

Output Arguments

ZeroRates — Zero curve for investment horizon represented by CurveDates

vector

Zero curve for the investment horizon represented by `CurveDates`, returned as a `NDATES-by-1` column vector of decimal fractions. The zero rates are the yields to maturity on theoretical zero-coupon bonds.

CurveDates — Maturity dates that correspond to ZeroRates

vector

Maturity dates that correspond to the `ZeroRates`, returned as a `NDATES-by-1` column vector. This vector is the same as the input vector `CurveDates`, but the output is sorted by ascending maturity. If either inputs for `CurveDates` or `Settle` are datetime arrays, the output `CurveDates` is returned as datetime arrays.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `disc2zero` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

y =

2021

There are no plans to remove support for serial date number inputs.

R2017b: Name-value arguments added for Compounding and Basis

Behavior changed in R2017b

The specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the optional name-value pair inputs: `Compounding` and `Basis`.

See Also

`zero2disc` | `datetime` | `fwd2zero` | `prbyzero` | `pyld2zero` | `zbtprice` | `zbtyield` | `zero2disc` | `zero2fwd` | `zero2pyld`

Topics

“Term Structure of Interest Rates” on page 2-29

“Fixed-Income Terminology” on page 2-15

discrate

Bank discount rate of security

Syntax

```
DiscRate = discrate(Settle,Maturity,Face,Price)
DiscRate = discrate( ___,Basis)
```

Description

`DiscRate = discrate(Settle,Maturity,Face,Price)` computes the bank discount rate of a security. The bank discount rate normalizes by the face value of the security (for example, U. S. Treasury Bills) and understates the true yield earned by investors.

`DiscRate = discrate( ___,Basis)` adds an optional argument for `Basis`.

Examples

Compute the Bank Discount Rate of a Security

This example shows how to find the bank discount rate of a security.

```
DiscRate = discrate(datetime(2000,1,12),datetime(2000,6,25), 100, 97.74, 0)

DiscRate =
0.0501
```

Input Arguments

Settle — Settlement date

datetime scalar | string scalar | date character vector

Settlement date, specified as a scalar datetime, string, or date character vector.

To support existing code, `discrate` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: datetime | string | char

Maturity — Maturity date

datetime scalar | string scalar | date character vector

Maturity date, specified as scalar datetime, string, or date character vector.

To support existing code, `discrate` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Face — Redemption (par, face) value

numeric

Redemption (par, face) value, specified as a scalar numeric value.

Data Types: double

Price — Security price

numeric

Security price, specified as a scalar numeric value.

Data Types: double

Basis — Day-count basis of the instrument

0 (actual/actual) (default) | integer with value of 0 to 13

(Optional) Day-count basis of the instrument, specified as a scalar integer.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

Output Arguments**DiscRate — Discount rate of security**

decimal

Discount rate of security, returned as a decimal.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `discrate` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)  
  
y =  
  
    2021
```

There are no plans to remove support for serial date number inputs.

References

[1] Mayle. "*Standard Securities Calculation Methods*." Volumes I-II, 3rd edition. Formula 2.

See Also

`acrudisc` | `fvdisc` | `prdisc` | `ylddisc` | `datetime`

Topics

"Term Structure of Interest Rates" on page 2-29

"Fixed-Income Terminology" on page 2-15

ecmlsrmle

Least-squares regression with missing data

Syntax

```
[Parameters,Covariance,Resid,Info] = ecmlsrmle(Data,Design)
[Parameters,Covariance,Resid,Info] = ecmlsrmle( ___,MaxIterations,TolParam,
TolObj,Param0,Covar0,CovarFormat)
```

Description

`[Parameters,Covariance,Resid,Info] = ecmlsrmle(Data,Design)` estimates a least-squares regression model with missing data. The model has the form

$$Data_k \sim N(Design_k \times Parameters, Covariance)$$

for samples $k = 1, \dots, \text{NUMSAMPLES}$.

`ecmlsrmle` estimates a `NUMPARAMS`-by-1 column vector of model parameters called `Parameters`, and a `NUMSERIES`-by-`NUMSERIES` matrix of covariance parameters called `Covariance`.

`ecmlsrmle(Data,Design)` with no output arguments plots the log-likelihood function for each iteration of the algorithm.

`[Parameters,Covariance,Resid,Info] = ecmlsrmle( ___,MaxIterations,TolParam,TolObj,Param0,Covar0,CovarFormat)` estimates a least-squares regression model with missing data using optional arguments.

Input Arguments

Data — Data sample

matrix

Data sample, specified as an `NUMSAMPLES`-by-`NUMSERIES` matrix with `NUMSAMPLES` samples of a `NUMSERIES`-dimensional random vector. If a data sample has missing values, represented as NaNs, the sample is ignored. (Use `mvnrmle` to handle missing data.)

Data Types: double

Design — Model design

matrix | cell array of character vectors

Model design, specified as a matrix or a cell array that handles two model structures:

- If `NUMSERIES = 1`, `Design` is a `NUMSAMPLES`-by-`NUMPARAMS` matrix with known values. This structure is the standard form for regression on a single series.
- If `NUMSERIES ≥ 1`, `Design` is a cell array. The cell array contains either one or `NUMSAMPLES` cells. Each cell contains a `NUMSERIES`-by-`NUMPARAMS` matrix of known values.

If `Design` has a single cell, it is assumed to have the same `Design` matrix for each sample. If `Design` has more than one cell, each cell contains a `Design` matrix for each sample.

These points concern how `Design` handles missing data:

- Although `Design` should not have NaN values, ignored samples due to NaN values in `Data` are also ignored in the corresponding `Design` array.
- If `Design` is a 1-by-1 cell array, which has a single `Design` matrix for each sample, no NaN values are permitted in the array. A model with this structure must have `NUMSERIES ≥ Numparms` with `rank(Design{1}) = Numparms`.
- `ecmlsrml` is more strict than `mvnrmle` about the presence of NaN values in the `Design` array.

Data Types: `double` | `cell`

MaxIterations — Maximum number of iterations for the estimation algorithm

100 (default) | numeric

(Optional) Maximum number of iterations for the estimation algorithm, specified as a numeric. The default value is 100.

Data Types: `double`

TolParam — Convergence tolerance for estimation algorithm

`sqrt(eps)` (default) | numeric

(Optional) Convergence tolerance for estimation algorithm based on changes in model parameter estimates, specified as a numeric. The Default value is `sqrt(eps)` which is about 1.0e-8 for double precision. The convergence test for changes in model parameters is

$$\|Param_k - Param_{k-1}\| < TolParam \times (1 + \|Param_k\|)$$

where `Param` represents the output `Parameters`, and iteration $k = 2, 3, \dots$. Convergence is assumed when both the `TolParam` and `TolObj` conditions are satisfied. If both $TolParam \leq 0$ and $TolObj \leq 0$, do the maximum number of iterations (`MaxIterations`), whatever the results of the convergence tests.

Data Types: `double` | `table` | `timetable`

TolObj — Convergence tolerance for estimation algorithm based on changes in objective function

$eps \wedge 3/4$ (default) | numeric

(Optional) Convergence tolerance for estimation algorithm based on changes in the objective function, specified as a numeric. The default value is $eps \wedge 3/4$ which is about 1.0e-12 for double precision. The convergence test for changes in the objective function is

$$|Obj_k - Obj_{k-1}| < TolObj \times (1 + |Obj_k|)$$

for iteration $k = 2, 3, \dots$. Convergence is assumed when both the `TolParam` and `TolObj` conditions are satisfied. If both $TolParam \leq 0$ and $TolObj \leq 0$, do the maximum number of iterations (`MaxIterations`), whatever the results of the convergence tests.

Data Types: `double`

Param0 — User-supplied initial estimate for the parameters of regression model

zero vector (default) | vector

(Optional) User-supplied initial estimate for the parameters of the regression model, specified as an `Numparms`-by-1 column vector.

Data Types: double

Covar0 — User-supplied initial or known estimate for the covariance matrix of the regression residuals

identity matrix (default) | matrix

(Optional) User-supplied initial or known estimate for the covariance matrix of the regression residuals, specified as an NUMSERIES-by-NUMSERIES matrix.

For covariance-weighted least-squares calculations, this matrix corresponds with weights for each series in the regression. The matrix also serves as an initial guess for the residual covariance in the expectation conditional maximization (ECM) algorithm.

Data Types: double

CovarFormat — Format for covariance matrix

'full' (default) | character vector with value 'full' or 'diagonal'

(Optional) Format for the covariance matrix, specified as a character vector. The choices are:

- 'full' — This is the default method that computes the full covariance matrix.
- 'diagonal' — This forces the covariance matrix to be a diagonal matrix.

Data Types: char

Output Arguments

Parameters — Parameters of the regression model

vector

Parameters of the regression model, returned as an Numparms-by-1 column vector of estimates for the parameters of the regression model.

Covariance — Covariance of the regression model's residuals

matrix

Covariance of the regression model's residuals, returned as an NUMSERIES-by-NUMSERIES matrix of estimates for the covariance of the regression model's residuals.

Resid — Residuals from the regression

matrix

Residuals from the regression, returned as an NUMSAMPLES-by-NUMSERIES matrix of residuals from the regression.

Info — Structure containing additional information from the regression

structure

Structure containing additional information from the regression, returned as a structure. The structure has these fields:

- `Info.Obj` - A variable-extent column vector, with no more than `MaxIterations` elements, that contain each value of the objective function at each iteration of the estimation algorithm. The last value in this vector, `Obj(end)`, is the terminal estimate of the objective function. If you do least squares, the objective function is the least squares objective function.

- `Info.PrevParameters` – NUPARAMS-by-1 column vector of estimates for the model parameters from the iteration just before the terminal iteration.
- `Info.PrevCovariance` – NUMSERIES-by-NUMSERIES matrix of estimates for the covariance parameters from the iteration just before the terminal iteration.

Use the estimates in the output structure `Info` for diagnostic purposes.

More About

Least Squares Regression

Least squares regression is a statistical method used to estimate the relationships between variables.

Least squares regression is commonly employed in linear regression analysis to find the best-fitting line through a set of data points. The goal is to minimize the sum of the squares of the differences (residuals) between the observed values and the values predicted by the model.

Version History

Introduced in R2006a

References

- [1] Dempster A, P., N.M. Laird, and D. B. Rubin. "Maximum Likelihood from Incomplete Data via the EM Algorithm." *Journal of the Royal Statistical Society. Series B*, Vol. 39, No. 1, 1977, pp. 1-37.
- [2] Roderick J., A. Little, and Donald B. Rubin. *Statistical Analysis with Missing Data.*, 2nd Edition. John Wiley & Sons, Inc., 2002.
- [3] Sexton J. and Anders Rygh Swensen. "ECM Algorithms that Converge at the Rate of EM." *Biometrika*. Vol. 87, No. 3, 2000, pp. 651-662.
- [4] Xiao-Li Meng and Donald B. Rubin. "Maximum Likelihood Estimation via the ECM Algorithm." *Biometrika*. Vol. 80, No. 2, 1993, pp. 267-278.

See Also

`ecmlsrojb` | `ecmmvnrml` | `ecmmvnrml`

Topics

"Multivariate Normal Regression" on page 9-13
 "Least-Squares Regression" on page 9-14
 "Covariance-Weighted Least Squares" on page 9-14
 "Feasible Generalized Least Squares" on page 9-15
 "Seemingly Unrelated Regression" on page 9-16

ecmlsrobj

Log-likelihood function for least-squares regression with missing data

Syntax

```
Objective = ecmlsrobj(Data,Design,Parameters)
Objective = ecmlsrobj( ___,Covariance)
```

Description

`Objective = ecmlsrobj(Data,Design,Parameters)` computes a least-squares objective function based on current parameter estimates with missing data. **Objective** is a scalar that contains the least-squares objective function.

`Objective = ecmlsrobj( ___,Covariance)` computes a least-squares objective function based on current parameter estimates with missing data using an optional argument.

Input Arguments

Data — Data sample

matrix

Data sample, specified as an NUMSAMPLES-by-NUMSERIES matrix with NUMSAMPLES samples of a NUMSERIES-dimensional random vector. If a data sample has missing values, represented as NaNs. Only samples that are entirely NaNs are ignored. (To ignore samples with at least one NaN, use `mvnrmlc`.)

Data Types: double

Design — Model design

matrix | cell array of character vectors

Model design, specified as a matrix or a cell array that handles two model structures:

- If `NUMSERIES = 1`, **Design** is a NUMSAMPLES-by-NUMPARAMS matrix with known values. This structure is the standard form for regression on a single series.
- If `NUMSERIES ≥ 1`, **Design** is a cell array. The cell array contains either one or NUMSAMPLES cells. Each cell contains a NUMSERIES-by-NUMPARAMS matrix of known values.

If **Design** has a single cell, it is assumed to have the same **Design** matrix for each sample. If **Design** has more than one cell, each cell contains a **Design** matrix for each sample.

Data Types: double | cell

Parameters — Estimates for the parameters of regression model

vector

Estimates for the parameters of regression model, specified as an NUMPARAMS-by-1 column vector.

Data Types: double

Covariance — User-supplied estimate for covariance matrix of residuals of the regression vector

(Optional) User-supplied estimate for the covariance matrix of the residuals of the regression, specified as an NUPARAMS-by-1 column vector.

ecmlsrobj requires that Covariance be positive-definite.

Note that

```
ecmlsrobj(Data, Design, Parameters) = ecmmvnrobj(Data, Design, Parameters, IdentityMatrix)
```

where IdentityMatrix is a NUMSERIES-by-NUMSERIES identity matrix.

Data Types: double

Output Arguments

Objective — Least-squares objective function

scalar

Least-squares objective function, returned as scalar.

More About

Least Squares Regression

Least squares regression is a statistical method used to estimate the relationships between variables.

Least squares regression is commonly employed in linear regression analysis to find the best-fitting line through a set of data points. The goal is to minimize the sum of the squares of the differences (residuals) between the observed values and the values predicted by the model.

Version History

Introduced in R2006a

See Also

ecmlsrmle | mvnrmle

Topics

“Least-Squares Regression With Missing Data” on page 9-14

“Multivariate Normal Regression” on page 9-13

“Least-Squares Regression” on page 9-14

“Covariance-Weighted Least Squares” on page 9-14

“Feasible Generalized Least Squares” on page 9-15

“Seemingly Unrelated Regression” on page 9-16

ecmmvnrfish

Fisher information matrix for multivariate normal regression model

Syntax

```
Fisher] = ecmmvnrfish(Data,Design,Covariance)
Fisher = ecmmvnrfish( ___,Method,MatrixFormat,CovarFormat)
```

Description

`Fisher] = ecmmvnrfish(Data,Design,Covariance)` computes a Fisher information matrix based on current maximum likelihood or least-squares parameter estimates that account for missing data.

`Fisher` is a `NUMPARAMS`-by-`NUMPARAMS` Fisher information matrix or Hessian matrix. The size of `NUMPARAMS` depends on `MatrixFormat` and on current parameter estimates. If `MatrixFormat` = `'full'`,

`NUMPARAMS` = `NUMSERIES * (NUMSERIES + 3)/2`

If `MatrixFormat` = `'paramonly'`,

`NUMPARAMS` = `NUMSERIES`

Note `ecmmvnrfish` operates slowly if you calculate the full Fisher information matrix.

`Fisher = ecmmvnrfish( ___,Method,MatrixFormat,CovarFormat)` computes a Fisher information matrix based on current maximum likelihood or least-squares parameter estimates that account for missing data using optional arguments.

Input Arguments

Data — Data sample

matrix

Data sample, specified as an `NUMSAMPLES`-by-`NUMSERIES` matrix with `NUMSAMPLES` samples of a `NUMSERIES`-dimensional random vector. If a data sample has missing values, represented as NaNs, the sample is ignored. (Use `mvnrmle` to handle missing data.)

Data Types: `double`

Design — Model design

matrix | cell array of character vectors

Model design, specified as a matrix or a cell array that handles two model structures:

- If `NUMSERIES` = 1, `Design` is a `NUMSAMPLES`-by-`NUMPARAMS` matrix with known values. This structure is the standard form for regression on a single series.

- If `NUMSERIES` ≥ 1 , `Design` is a cell array. The cell array contains either one or `NUMSAMPLES` cells. Each cell contains a `NUMSERIES`-by-`NUMPARAMS` matrix of known values.

If `Design` has a single cell, it is assumed to have the same `Design` matrix for each sample. If `Design` has more than one cell, each cell contains a `Design` matrix for each sample.

Data Types: `double` | `cell`

Covariance — Estimates for the covariance of the residuals of the regression matrix

Estimates for the covariance of the residuals of the regression, specified as an `NUMSERIES`-by-`NUMSERIES` matrix.

Data Types: `double`

Method — Method of calculation for information matrix

`'hessian'` (default) | character vector with value `'hessian'` or `'fisher'`

(Optional) Method of calculation for the information matrix, specified as a character vector. The choices are:

- `'hessian'` — This is the default method using the expected Hessian matrix of the observed log-likelihood function. This method is recommended since the resultant standard errors incorporate the increased uncertainties due to missing data.
- `'fisher'` — This computes using the Fisher information matrix.

Data Types: `char`

MatrixFormat — Parameters to be included in Fisher information matrix

`'full'` (default) | character vector with value `'full'` or `'paramonly'`

(Optional) Parameters to be included in the Fisher information matrix, specified as a character vector. The choices are:

- `'full'` — This is the default method that computes the full Fisher information matrix for both model and covariance parameter estimates.
- `'paramonly'` — This computes only components of the Fisher information matrix associated with the model parameter estimates.

Data Types: `char`

CovarFormat — Format for covariance matrix

`'full'` (default) | character vector with value `'full'` or `'diagonal'`

(Optional) Format for the covariance matrix, specified as a character vector. The choices are:

- `'full'` — This is the default method that computes the full covariance matrix.
- `'diagonal'` — This forces the covariance matrix to be a diagonal matrix.

Data Types: `char`

Output Arguments

Fisher — Fisher information matrix

matrix

Fisher information matrix, returned as an Numparams-by-Numparams Fisher information matrix or Hessian matrix, depending on the optional input argument `Method`.

More About

Fisher Information Matrix

The Fisher information matrix is a key concept in statistics and information theory that measures the amount of information that an observable random variable carries about an unknown parameter upon which the probability of the random variable depends.

The Fisher information matrix is used to evaluate the efficiency of estimators and to derive confidence intervals.

Version History

Introduced in R2006a

See Also

`ecmnml` | `ecmnstd`

Topics

“Multivariate Normal Regression Without Missing Data” on page 9-13

“Multivariate Normal Regression” on page 9-13

“Least-Squares Regression” on page 9-14

“Covariance-Weighted Least Squares” on page 9-14

“Feasible Generalized Least Squares” on page 9-15

“Seemingly Unrelated Regression” on page 9-16

“Fisher Information” on page 9-4

“Multivariate Normal Linear Regression” on page 9-2

ecmmvnrml

Multivariate normal regression with missing data

Syntax

```
[Param,Covar] = ecmmvnrml(Data,Design)
[Param,Covar,Resid,Info] = ecmmvnrml(____,MaxIterations,TolParam,TolObj,
Param0,Covar0,CovarFormat)
```

Description

[Param,Covar] = ecmmvnrml(Data,Design) estimates a multivariate normal regression model with missing data. The model has the form

$$Data_k \sim N(Design_k \times Parameters, Covariance)$$

for samples $k = 1, \dots, \text{NUMSAMPLES}$.

[Param,Covar,Resid,Info] = ecmmvnrml(____,MaxIterations,TolParam,TolObj,Param0,Covar0,CovarFormat) adds an optional arguments for MaxIterations, TolParam, TolObj, Param0, Covar0, and CovarFormat.

Examples

Compute Multivariate Normal Regression With Missing Data

This example shows how to estimate a multivariate normal regression model with missing data.

First, load dates, total returns, and ticker symbols for the twelve stocks from the MAT-file.

```
load CAPMuniverse
whos Assets Data Dates
```

Name	Size	Bytes	Class	Attributes
Assets	1x14	1792	cell	
Data	1471x14	164752	double	
Dates	1471x1	11768	double	

```
Dates = datetime(Dates,'ConvertFrom','datenum');
```

The assets in the model have the following symbols, where the last two series are proxies for the market and the riskless asset.

```
Assets(1:14)
```

```
ans = 1x14 cell
      {'AAPL'}      {'AMZN'}      {'CSCO'}      {'DELL'}      {'EBAY'}      {'GOOG'}      {'HPQ'}      {'IBM'}
```

The data covers the period from January 1, 2000 to November 7, 2005 with daily total returns. Two stocks in this universe have missing values that are represented by NaNs. One of the two stocks had an IPO during this period and, consequently, has significantly less data than the other stocks.

Compute separate regressions for each stock, where the stocks with missing data have estimates that reflect their reduced observability.

```
[NumSamples, NumSeries] = size(Data);
NumAssets = NumSeries - 2;

StartDate = Dates(1);
EndDate = Dates(end);

Alpha = NaN(1, length(NumAssets));
Beta = NaN(1, length(NumAssets));
Sigma = NaN(1, length(NumAssets));
StdAlpha = NaN(1, length(NumAssets));
StdBeta = NaN(1, length(NumAssets));
StdSigma = NaN(1, length(NumAssets));
for i = 1:NumAssets
    % Set up separate asset data and design matrices
    TestData = zeros(NumSamples,1);
    TestDesign = zeros(NumSamples,2);

    TestData(:) = Data(:,i) - Data(:,14);
    TestDesign(:,1) = 1.0;
    TestDesign(:,2) = Data(:,13) - Data(:,14);

    % Estimate the multivariate normal regression for each asset separately.
    [Param, Covar] = ecmmvnrmls(TestData, TestDesign)
end

Param = 2×1

    0.0012
    1.2294

Covar =
    0.0010

Param = 2×1

    0.0006
    1.3661

Covar =
    0.0020

Param = 2×1

   -0.0002
    1.5653

Covar =
    8.8911e-04
```


Param = 2×1

-0.0000
1.2594

Covar =

6.4996e-04

Param = 2×1

0.0014
1.3441

Covar =

0.0014

Param = 2×1

0.0046
0.3742

Covar =

6.3272e-04

Param = 2×1

0.0001
1.3745

Covar =

6.5040e-04

Param = 2×1

-0.0000
1.0807

Covar =

2.8562e-04

Param = 2×1

0.0001
1.6002

Covar =

6.9146e-04

Param = 2×1

-0.0002
1.1765

```
Covar =  
3.7138e-04  
  
Param = 2×1  
  
    0.0000  
    1.5010
```

```
Covar =  
0.0010  
  
Param = 2×1  
  
    0.0001  
    1.6543
```

```
Covar =  
0.0015
```

Input Arguments

Data — Data

matrix

Data, specified as an NUMSAMPLES-by-NUMSERIES matrix with NUMSAMPLES samples of a NUMSERIES-dimensional random vector. Missing values are indicated by NaNs. Only samples that are entirely NaNs are ignored. (To ignore samples with at least one NaN, use `mvnrml`.)

Data Types: double

Design — Design model

matrix | cell array

Design model, specified as a matrix or a cell array that handles two model structures:

- If NUMSERIES = 1, Design is a NUMSAMPLES-by-NUMPARAMS matrix with known values. This structure is the standard form for regression on a single series.
- If NUMSERIES ≥ 1, Design is a cell array. The cell array contains either one or NUMSAMPLES cells. Each cell contains a NUMSERIES-by-NUMPARAMS matrix of known values.

If Design has a single cell, it is assumed to have the same Design matrix for each sample. If Design has more than one cell, each cell contains a Design matrix for each sample.

Data Types: double | cell

MaxIterations — Maximum number of iterations for the estimation algorithm

100 (default) | numeric

(Optional) Maximum number of iterations for the estimation algorithm, specified as a numeric.

Data Types: double

TolParam — Convergence tolerance for estimation algorithm based on changes in model parameter estimates

1.0e-8 (default) | numeric

(Optional) Convergence tolerance for estimation algorithm based on changes in model parameter estimates, specified as a numeric. The convergence test for changes in model parameters is

$$\|Param_k - Param_{k-1}\| < TolParam \times (1 + \|Param_k\|)$$

where *Param* represents the output *Parameters*, and iteration $k = 2, 3, \dots$. Convergence is assumed when both the *TolParam* and *TolObj* conditions are satisfied. If both $TolParam \leq 0$ and $TolObj \leq 0$, do the maximum number of iterations (*MaxIterations*), whatever the results of the convergence tests.

Data Types: double

TolObj — Convergence tolerance for estimation algorithm based on changes in objective function

1.0e-12 (default) | numeric

(Optional) Convergence tolerance for estimation algorithm based on changes in the objective function, specified as a numeric. The convergence test for changes in the objective function is

$$|Obj_k - Obj_{k-1}| < TolObj \times (1 + |Obj_k|)$$

for iteration $k = 2, 3, \dots$. Convergence is assumed when both the *TolParam* and *TolObj* conditions are satisfied. If both $TolParam \leq 0$ and $TolObj \leq 0$, do the maximum number of iterations (*MaxIterations*), whatever the results of the convergence tests.

Data Types: double

Param0 — Estimate for the parameters of regression model

[] (default) | vector

(Optional) Estimate for the parameters of the regression model, specified as an *NUMPARAMS*-by-1 column vector.

Data Types: double

Covar0 — Estimate for the covariance matrix of regression residuals

[] (default) | matrix

(Optional) Estimate for the covariance matrix of the regression residuals, specified as *NUMSERIES*-by-*NUMSERIES* matrix.

Data Types: double

CovarFormat — Format for the covariance matrix

'full' (default) | character vector

(Optional) Format for the covariance matrix, specified as a character vector. The choices are:

- 'full' — Compute the full covariance matrix.
- 'diagonal' — Force the covariance matrix to be a diagonal matrix.

Data Types: char

Output Arguments

Param — Estimates for parameters of the regression model

vector

Estimates for the parameters of the regression model, returned as a NUMPARAMS-by-1 column vector.

Covar — Estimates for the covariance of regression model's residuals

matrix

Estimates for the covariance of the regression model's residuals, returned as a NUMSERIES-by-NUMSERIES matrix.

Resid — Residuals from regression

matrix

Residuals from the regression, returned as a NUMSAMPLES-by-NUMSERIES matrix. For any missing values in `Data`, the corresponding residual is the difference between the conditionally imputed value for `Data` and the model, that is, the imputed residual.

Note The covariance estimate `Covariance` cannot be derived from the residuals.

Info — Additional information from regression

structure

Additional information from the regression, returned as a structure. The structure has these fields:

- `Info.Obj` — A variable-extent column vector, with no more than `MaxIterations` elements, that contain each value of the objective function at each iteration of the estimation algorithm. The last value in this vector, `Obj(end)`, is the terminal estimate of the objective function. If you do maximum likelihood estimation, the objective function is the log-likelihood function.
- `Info.PrevParameters` — NUMPARAMS-by-1 column vector of estimates for the model parameters from the iteration just prior to the terminal iteration. `Info.PrevCovariance` - NUMSERIES-by-NUMSERIES matrix of estimates for the covariance parameters from the iteration just prior to the terminal iteration.

More About

Multivariate Normal Regression

Multivariate normal regression is a statistical modeling approach that extends traditional regression techniques to situations where the dependent variable is multivariate, meaning it consists of multiple correlated outcomes.

Multivariate normal regression is particularly useful in fields where researchers are interested in understanding the relationships between several response variables and one or more predictor variables.

Version History

Introduced in R2006a

References

- [1] Little, Roderick J. A. and Donald B. Rubin. *Statistical Analysis with Missing Data*. 2nd Edition. John Wiley & Sons, Inc., 2002.
- [2] Meng, Xiao-Li and Donald B. Rubin. "Maximum Likelihood Estimation via the ECM Algorithm." *Biometrika*. Vol. 80, No. 2, 1993, pp. 267-278.
- [3] Sexton, Joe and Anders Rygh Swensen. "ECM Algorithms that Converge at the Rate of EM." *Biometrika*. Vol. 87, No. 3, 2000, pp. 651-662.
- [4] Dempster, A. P., N. M. Laird, and Donald B. Rubin. "Maximum Likelihood from Incomplete Data via the EM Algorithm." *Journal of the Royal Statistical Society. Series B*, Vol. 39, No. 1, 1977, pp. 1-37.

See Also

ecmmvnrobj | mvnrml

Topics

- "Multivariate Normal Regression With Missing Data" on page 9-14
- "Multivariate Normal Regression" on page 9-13
- "Least-Squares Regression" on page 9-14
- "Covariance-Weighted Least Squares" on page 9-14
- "Feasible Generalized Least Squares" on page 9-15
- "Seemingly Unrelated Regression" on page 9-16
- "Multivariate Normal Linear Regression" on page 9-2

ecmmvnrobj

Log-likelihood function for multivariate normal regression with missing data

Syntax

```
Objective = ecmmvnrobj(Data,Design,Parameters,Covariance)
Objective = ecmmvnrobj( ___,CovarFormat)
```

Description

`Objective = ecmmvnrobj(Data,Design,Parameters,Covariance)` computes a log-likelihood function based on current maximum likelihood parameter estimates with missing data. `Objective` is a scalar that contains the least-squares objective function.

`Objective = ecmmvnrobj( ___,CovarFormat)` computes a log-likelihood function based on current maximum likelihood parameter estimates with missing data using an optional argument.

Input Arguments

Data — Data sample

matrix

Data sample, specified as an `NUMSAMPLES`-by-`NUMSERIES` matrix with `NUMSAMPLES` samples of a `NUMSERIES`-dimensional random vector. If a data sample has missing values, represented as NaNs. Only samples that are entirely NaNs are ignored. (To ignore samples with at least one NaN, use `mvnrml`.)

Data Types: double

Design — Model design

matrix | cell array of character vectors

Model design, specified as a matrix or a cell array that handles two model structures:

- If `NUMSERIES = 1`, `Design` is a `NUMSAMPLES`-by-`NUMPARAMS` matrix with known values. This structure is the standard form for regression on a single series.
- If `NUMSERIES ≥ 1`, `Design` is a cell array. The cell array contains either one or `NUMSAMPLES` cells. Each cell contains a `NUMSERIES`-by-`NUMPARAMS` matrix of known values.

If `Design` has a single cell, it is assumed to have the same `Design` matrix for each sample. If `Design` has more than one cell, each cell contains a `Design` matrix for each sample.

Data Types: double | cell

Parameters — Estimates for the parameters of regression model

vector

Estimates for the parameters of regression model, specified as an `NUMPARAMS`-by-1 column vector.

Data Types: double

Covariance — Estimates for covariance matrix of residuals of the regression matrix

Estimates for the covariance matrix of the residuals of the regression, specified as an NUMSERIES-by-NUMSERIES matrix.

Data Types: double

CovarFormat — Format for covariance matrix

'full' (default) | character vector with value 'full' or 'diagonal'

(Optional) Format for the covariance matrix, specified as a character vector. The choices are:

- 'full' — This is the default method that computes the full covariance matrix.
- 'diagonal' — This forces the covariance matrix to be a diagonal matrix.

Data Types: char

Output Arguments

Objective — Least-squares objective function

scalar

Least-squares objective function, returned as scalar.

More About

Multivariate Normal Regression

Multivariate normal regression is a statistical modeling approach that extends traditional regression techniques to situations where the dependent variable is multivariate, meaning it consists of multiple correlated outcomes.

Multivariate normal regression is particularly useful in fields where researchers are interested in understanding the relationships between several response variables and one or more predictor variables.

Version History

Introduced in R2006a

See Also

ecmmvnrml | mvnrml | mvnrobj

Topics

“Multivariate Normal Regression With Missing Data” on page 9-14

“Portfolios with Missing Data” on page 9-21

“Multivariate Normal Regression” on page 9-13

“Least-Squares Regression” on page 9-14

“Covariance-Weighted Least Squares” on page 9-14

“Feasible Generalized Least Squares” on page 9-15

“Seemingly Unrelated Regression” on page 9-16

“Multivariate Normal Linear Regression” on page 9-2

ecmmvnstd

Evaluate standard errors for multivariate normal regression model

Syntax

```
[StdParameters,StdCovariance] = ecmmvnstd(Data,Design,Covariance)
[StdParameters,StdCovariance] = ecmmvnstd( ___,Method,CovarFormat)
```

Description

[StdParameters,StdCovariance] = ecmmvnstd(Data,Design,Covariance) evaluates standard errors for a multivariate normal regression model with missing data. The model has the form

$$Data_k \sim N(Design_k \times Parameters, Covariance)$$

for samples $k = 1, \dots, \text{NUMSAMPLES}$.

[StdParameters,StdCovariance] = ecmmvnstd(___,Method,CovarFormat) adds an optional arguments for Method and CovarFormat.

Examples

Compute Standard Errors for Multivariate Normal Regression

This example shows how to compute standard errors for a multivariate normal regression model.

First, load dates, total returns, and ticker symbols for the twelve stocks from the MAT-file.

```
load CAPMuniverse
whos Assets Data Dates
```

Name	Size	Bytes	Class	Attributes
Assets	1x14	1792	cell	
Data	1471x14	164752	double	
Dates	1471x1	11768	double	

```
Dates = datetime(Dates,'ConvertFrom','datenum');
```

The assets in the model have the following symbols, where the last two series are proxies for the market and the riskless asset.

```
Assets(1:14)
```

```
ans = 1x14 cell
      {'AAPL'}      {'AMZN'}      {'CSCO'}      {'DELL'}      {'EBAY'}      {'GOOG'}      {'HPQ'}      {'IBM'}
```

The data covers the period from January 1, 2000 to November 7, 2005 with daily total returns. Two stocks in this universe have missing values that are represented by NaNs. One of the two stocks had an IPO during this period and, consequently, has significantly less data than the other stocks.

```
[Mean,Covariance] = ecmmle(Data);
```

Compute separate regressions for each stock, where the stocks with missing data have estimates that reflect their reduced observability.

```
[NumSamples, NumSeries] = size(Data);  
NumAssets = NumSeries - 2;
```

```
StartDate = Dates(1);  
EndDate = Dates(end);
```

```
Alpha = NaN(1, length(NumAssets));  
Beta = NaN(1, length(NumAssets));  
Sigma = NaN(1, length(NumAssets));  
StdAlpha = NaN(1, length(NumAssets));  
StdBeta = NaN(1, length(NumAssets));  
StdSigma = NaN(1, length(NumAssets));
```

```
for i = 1:NumAssets  
    % Set up separate asset data and design matrices  
    TestData = zeros(NumSamples,1);  
    TestDesign = zeros(NumSamples,2);
```

```
    TestData(:) = Data(:,i) - Data(:,14);  
    TestDesign(:,1) = 1.0;  
    TestDesign(:,2) = Data(:,13) - Data(:,14);
```

```
    [Param, Covar] = ecmmvnrmls(TestData, TestDesign);
```

```
    % Estimate the sample standard errors for model parameters for each asset.  
    StdParam = ecmmvnrstd(TestData, TestDesign, Covar, 'hessian')
```

```
end
```

```
StdParam = 2×1
```

```
    0.0008  
    0.0715
```

```
StdParam = 2×1
```

```
    0.0012  
    0.1000
```

```
StdParam = 2×1
```

```
    0.0008  
    0.0663
```

```
StdParam = 2×1
```

```
    0.0007
```

0.0567

StdParam = 2×1

0.0010
0.0836

StdParam = 2×1

0.0014
0.2159

StdParam = 2×1

0.0007
0.0567

StdParam = 2×1

0.0004
0.0376

StdParam = 2×1

0.0007
0.0585

StdParam = 2×1

0.0005
0.0429

StdParam = 2×1

0.0008
0.0709

StdParam = 2×1

0.0010
0.0853

Input Arguments

Data — Data

matrix

Data, specified as an NUMSAMPLES-by-NUMSERIES matrix with NUMSAMPLES samples of a NUMSERIES-dimensional random vector. Missing values are indicated by NaNs. Only samples that are entirely NaNs are ignored. (To ignore samples with at least one NaN, use `mvnrml`.)

Data Types: `double`

Design — Design model

`matrix` | `cell array`

Design model, specified as a matrix or a cell array that handles two model structures:

- If `NUMSERIES` = 1, `Design` is a NUMSAMPLES-by-NUMPARAMS matrix with known values. This structure is the standard form for regression on a single series.
- If `NUMSERIES` ≥ 1, `Design` is a cell array. The cell array contains either one or NUMSAMPLES cells. Each cell contains a NUMSERIES-by-NUMPARAMS matrix of known values.

If `Design` has a single cell, it is assumed to have the same `Design` matrix for each sample. If `Design` has more than one cell, each cell contains a `Design` matrix for each sample.

Data Types: `double` | `cell`

Covariance — Estimates for covariance of regression residuals

`matrix`

Estimates for the covariance of the regression residuals, specified as an NUMSERIES-by-NUMSERIES matrix.

Data Types: `double`

Method — Method of calculation for the information matrix

`'hessian'` (default) | `character vector`

(Optional) Method of calculation for the information matrix, specified as a character vector defined as:

- `'hessian'` — The expected Hessian matrix of the observed log-likelihood function. This method is recommended since the resultant standard errors incorporate the increased uncertainties due to missing data.
- `'fisher'` — The Fisher information matrix.

Note If `Method` = `'fisher'`, to obtain more quickly just the standard errors of variance estimates without the standard errors of the covariance estimates, set `CovarFormat` = `'diagonal'` regardless of the form of the covariance matrix.

Data Types: `char`

CovarFormat — Format for the covariance matrix

`'full'` (default) | `character vector`

(Optional) Format for the covariance matrix, specified as a character vector. The choices are:

- `'full'` — Compute the full covariance matrix.
- `'diagonal'` — Force the covariance matrix to be a diagonal matrix.

Data Types: `char`

Output Arguments

StdParameters — Standard errors for each element of Parameters

vector

Standard errors for each element of Parameters, returned as an NUMPARAMS-by-1 column vector.

StdCovariance — Standard errors for each element of Covariance

matrix

Standard errors for each element of Covariance, returned as an NUMSERIES-by-NUMSERIES matrix.

More About

Multivariate Normal Regression

Multivariate normal regression is a statistical modeling approach that extends traditional regression techniques to situations where the dependent variable is multivariate, meaning it consists of multiple correlated outcomes.

Multivariate normal regression is particularly useful in fields where researchers are interested in understanding the relationships between several response variables and one or more predictor variables.

Version History

Introduced in R2006a

References

- [1] Little, Roderick J. A. and Donald B. Rubin. *Statistical Analysis with Missing Data*. 2nd Edition. John Wiley & Sons, Inc., 2002.

See Also

ecmmvnrmle

Topics

- "Multivariate Normal Regression" on page 9-13
- "Least-Squares Regression" on page 9-14
- "Covariance-Weighted Least Squares" on page 9-14
- "Feasible Generalized Least Squares" on page 9-15
- "Seemingly Unrelated Regression" on page 9-16
- "Multivariate Normal Regression With Missing Data" on page 9-14
- "Multivariate Normal Linear Regression" on page 9-2

ecmnfish

Fisher information matrix

Syntax

```
Fisher = ecmnfish(Data,Covariance)
Fisher = ecmnfish( ___,InvCovar,MatrixType)
```

Description

`Fisher = ecmnfish(Data,Covariance)` computes an NUMPARAMS-by-NUMPARAMS Fisher information matrix based on the current maximum likelihood parameter estimates.

Use `ecmnfish` after estimating the mean and covariance of `Data` with `ecmmle`.

`Fisher = ecmnfish( ___,InvCovar,MatrixType)` adds optional arguments for `InvCovar` and `MatrixType`.

Examples

Compute Fisher Information Matrix Based on Parameter Estimates for Data

This example shows how to compute the Fisher information matrix based on parameter estimates for `Data` for five years of daily total returns for 12 computer technology stocks, with six hardware and six software companies

```
load ecmtechdemo.mat
```

The time period for this data extends from April 19, 2000 to April 18, 2005. The sixth stock in `Assets` is Google (GOOG), which started trading on August 19, 2004. So, all returns before August 20, 2004 are missing and represented as NaNs. Also, Amazon (AMZN) had a few days with missing values scattered throughout the past five years.

```
[ECMMean, ECMCovar] = ecmmle(Data)
```

```
ECMMean = 12×1
```

```
    0.0008
    0.0008
   -0.0005
    0.0002
    0.0011
    0.0038
   -0.0003
   -0.0000
   -0.0003
   -0.0000
   -0.0003
    0.0004
    ⋮
```

ECMCovar = 12×12

0.0012	0.0005	0.0006	0.0005	0.0005	0.0003	0.0005	0.0003	0.0006	0.0006
0.0005	0.0024	0.0007	0.0006	0.0010	0.0004	0.0005	0.0003	0.0006	0.0006
0.0006	0.0007	0.0013	0.0007	0.0007	0.0003	0.0006	0.0004	0.0008	0.0008
0.0005	0.0006	0.0007	0.0009	0.0006	0.0002	0.0005	0.0003	0.0007	0.0007
0.0005	0.0010	0.0007	0.0006	0.0016	0.0006	0.0005	0.0003	0.0006	0.0006
0.0003	0.0004	0.0003	0.0002	0.0006	0.0022	0.0001	0.0002	0.0002	0.0002
0.0005	0.0005	0.0006	0.0005	0.0005	0.0001	0.0009	0.0003	0.0005	0.0005
0.0003	0.0003	0.0004	0.0003	0.0003	0.0002	0.0003	0.0005	0.0004	0.0004
0.0006	0.0006	0.0008	0.0007	0.0006	0.0002	0.0005	0.0004	0.0011	0.0011
0.0003	0.0004	0.0005	0.0004	0.0004	0.0001	0.0004	0.0003	0.0005	0.0005
0.0005	0.0006	0.0008	0.0005	0.0007	0.0003	0.0005	0.0004	0.0007	0.0007
0.0006	0.0012	0.0008	0.0007	0.0011	0.0016	0.0006	0.0004	0.0007	0.0007
:									

To evaluate the negative log-likelihood function for `ecmmle`, use `ecmnfish` based on the current maximum likelihood parameter estimates for `ECMCovar`.

`Fisher = ecmnfish(Data,ECMCovar)`

Fisher = 90×90
10⁷ ×

0.0001	0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	0.0000
0.0000	0.0001	-0.0000	0.0000	-0.0000	0.0001	0.0000	0.0000	0.0000	0.0000
-0.0000	-0.0000	0.0002	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000
-0.0000	0.0000	-0.0000	0.0003	-0.0000	0.0000	-0.0000	-0.0000	-0.0001	-0.0000
-0.0000	-0.0000	-0.0000	-0.0000	0.0001	-0.0000	-0.0000	-0.0000	0.0000	-0.0000
-0.0000	0.0001	-0.0000	0.0000	-0.0000	0.0002	0.0000	-0.0000	0.0000	0.0000
-0.0000	0.0000	-0.0000	-0.0000	-0.0000	0.0000	0.0002	-0.0001	-0.0000	0.0000
-0.0000	0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0001	0.0004	-0.0000	-0.0000
-0.0000	0.0000	-0.0000	-0.0001	0.0000	0.0000	-0.0000	-0.0000	0.0002	-0.0000
0.0000	0.0000	-0.0000	-0.0001	-0.0000	0.0001	0.0000	-0.0001	-0.0001	0.0000
-0.0000	0.0000	-0.0000	0.0000	-0.0000	0.0000	-0.0000	-0.0000	-0.0000	-0.0000
-0.0000	-0.0001	-0.0000	-0.0000	-0.0000	-0.0002	-0.0001	0.0000	-0.0000	-0.0000
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
:									

Input Arguments

Data — Data

matrix

Data, specified as an `NUMSAMPLES`-by-`NUMSERIES` matrix with `NUMSAMPLES` samples of a `NUMSERIES`-dimensional random vector. Missing values are indicated by NaNs.

Data Types: double

Covariance — Maximum likelihood parameter estimates for covariance of Data

matrix

Maximum likelihood parameter estimates for the covariance of the Data using the ECM algorithm, specified as a NUMSERIES-by-NUMSERIES matrix.

InvCovar — Cholesky decomposition of covariance matrix

[] (default) | matrix

(Optional) Inverse of covariance matrix, specified as a matrix using `inv` as:

`inv(Covariance)`

Data Types: double

MatrixType — Matrix format

'full' (default) | character vector

(Optional) Matrix format, specified as a character vector with a value of:

- 'full' — Computes the full Fisher information matrix.
- 'meanonly' — Computes only the components of the Fisher information matrix associated with the mean.

Data Types: char

Output Arguments

Fisher — Fisher information matrix

matrix

Fisher information matrix, returned as an NUMPARAMSNUMPARAMS matrix based on current parameter estimates, where $\text{NUMPARAMS} = \text{NUMSERIES} * (\text{NUMSERIES} + 3)/2$ if the `MatrixFormat` = 'full'. If the `MatrixFormat` = 'meanonly', then the $\text{NUMPARAMS} = \text{NUMSERIES}$.

More About

Fisher Information Matrix

The Fisher information matrix is a key concept in statistics and information theory that measures the amount of information that an observable random variable carries about an unknown parameter upon which the probability of the random variable depends.

The Fisher information matrix is used to evaluate the efficiency of estimators and to derive confidence intervals.

Version History

Introduced before R2006a

See Also

`ecmhess` | `ecmmle`

Topics

“Multivariate Normal Regression With Missing Data” on page 9-14

"Fisher Information" on page 9-4

ecmnhess

Hessian of negative log-likelihood function

Syntax

```
Hessian = ecmnhess(Data,Covariance)
Hessian = ecmnhess( ___,InvCovar,MatrixType)
```

Description

`Hessian = ecmnhess(Data,Covariance)` computes an NUMPARAMS-by-NUMPARAMS Hessian matrix of the observed negative log-likelihood function based on current parameter estimates.

Use `ecmnhess` after estimating the mean and covariance of `Data` with `ecmmle`.

`Hessian = ecmnhess( ___,InvCovar,MatrixType)` adds optional arguments for `InvCovar` and `MatrixType`.

Examples

Compute Hessian for Negative Log-Likelihood Function for Data

This example shows how to compute the Hessian for the negative log-likelihood function for five years of daily total return data for 12 computer technology stocks, with six hardware and six software companies

```
load ecmtechdemo.mat
```

The time period for this data extends from April 19, 2000 to April 18, 2005. The sixth stock in `Assets` is Google (GOOG), which started trading on August 19, 2004. So, all returns before August 20, 2004 are missing and represented as NaNs. Also, Amazon (AMZN) had a few days with missing values scattered throughout the past five years.

```
[ECMMean, ECMCovar] = ecmmle(Data)
```

```
ECMMean = 12×1
```

```
    0.0008
    0.0008
   -0.0005
    0.0002
    0.0011
    0.0038
   -0.0003
   -0.0000
   -0.0003
   -0.0000
   -0.0003
    0.0004
    ⋮
```

ECMCovar = 12×12

0.0012	0.0005	0.0006	0.0005	0.0005	0.0003	0.0005	0.0003	0.0006	0.0006
0.0005	0.0024	0.0007	0.0006	0.0010	0.0004	0.0005	0.0003	0.0006	0.0006
0.0006	0.0007	0.0013	0.0007	0.0007	0.0003	0.0006	0.0004	0.0008	0.0008
0.0005	0.0006	0.0007	0.0009	0.0006	0.0002	0.0005	0.0003	0.0007	0.0007
0.0005	0.0010	0.0007	0.0006	0.0016	0.0006	0.0005	0.0003	0.0006	0.0006
0.0003	0.0004	0.0003	0.0002	0.0006	0.0022	0.0001	0.0002	0.0002	0.0002
0.0005	0.0005	0.0006	0.0005	0.0005	0.0001	0.0009	0.0003	0.0005	0.0005
0.0003	0.0003	0.0004	0.0003	0.0003	0.0002	0.0003	0.0005	0.0004	0.0004
0.0006	0.0006	0.0008	0.0007	0.0006	0.0002	0.0005	0.0004	0.0011	0.0011
0.0003	0.0004	0.0005	0.0004	0.0004	0.0001	0.0004	0.0003	0.0005	0.0005
0.0005	0.0006	0.0008	0.0005	0.0007	0.0003	0.0005	0.0004	0.0007	0.0007
0.0006	0.0012	0.0008	0.0007	0.0011	0.0016	0.0006	0.0004	0.0007	0.0007
:									

To evaluate the negative log-likelihood function for `ecmmle`, use `ecmnhess` based on the current maximum likelihood parameter estimates for `ECMCovar`.

Hessian = `ecmnhess(Data,ECMCovar)`

Hessian = 90×90
10⁷ ×

0.0001	0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	0.0000
0.0000	0.0001	-0.0000	-0.0000	-0.0000	0.0000	-0.0000	0.0000	-0.0000	-0.0000
-0.0000	-0.0000	0.0002	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000
-0.0000	-0.0000	-0.0000	0.0003	-0.0000	0.0000	-0.0000	-0.0000	-0.0001	-0.0000
-0.0000	-0.0000	-0.0000	-0.0000	0.0001	-0.0000	-0.0000	-0.0000	0.0000	-0.0000
-0.0000	0.0000	-0.0000	0.0000	-0.0000	0.0000	0.0000	-0.0000	0.0000	0.0000
-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	0.0000	0.0002	-0.0000	-0.0000	-0.0000
-0.0000	0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	0.0004	-0.0000	-0.0000
-0.0000	-0.0000	-0.0000	-0.0001	0.0000	0.0000	-0.0000	-0.0000	0.0002	-0.0000
0.0000	-0.0000	-0.0000	-0.0001	-0.0000	0.0000	-0.0000	-0.0000	-0.0001	0.0000
-0.0000	0.0000	-0.0000	-0.0000	-0.0000	0.0000	-0.0000	-0.0000	-0.0000	-0.0000
-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
:									

Input Arguments

Data — Data

matrix

Data, specified as an `NUMSAMPLES`-by-`NUMSERIES` matrix with `NUMSAMPLES` samples of a `NUMSERIES`-dimensional random vector. Missing values are indicated by NaNs.

Data Types: double

Covariance — Maximum likelihood parameter estimates for covariance of Data

matrix

Maximum likelihood parameter estimates for the covariance of the Data using the ECM algorithm, specified as a NUMSERIES-by-NUMSERIES matrix.

InvCovar — Cholesky decomposition of covariance matrix

[] (default) | matrix

(Optional) Inverse of covariance matrix, specified as a matrix using `inv` as:

`inv(Covariance)`

Data Types: double

MatrixType — Matrix format

'full' (default) | character vector

(Optional) Matrix format, specified as a character vector with a value of:

- 'full' — Computes the full Hessian matrix.
- 'meanonly' — Computes only the components of the Hessian matrix associated with the mean.

Data Types: char

Output Arguments

Hessian — Hessian matrix

matrix

Hessian matrix, returned as an NUMPARAMSNUMPARAMS matrix of the observed log-likelihood function based on current parameter estimates, where $\text{NUMPARAMS} = \text{NUMSERIES} * (\text{NUMSERIES} + 3)/2$ if the `MatrixFormat` = 'full'. If the `MatrixFormat` = 'meanonly', then the `NUMPARAMS` = NUMSERIES.

More About

Hessian Matrix

The Hessian matrix is a square matrix of second-order partial derivatives of a scalar-valued function.

The Hessian matrix is a key concept in optimization, particularly in the context of multivariable calculus, and is used to study the local curvature of functions. The Hessian matrix provides important information about the behavior of functions near critical points, which are points where the gradient (first derivative) is zero.

Version History

Introduced before R2006a

See Also

`ecmnfish` | `ecmnml`

Topics

"Maximum Likelihood Estimation" on page 9-3

ecmninit

Initial mean and covariance

Syntax

```
[Mean,Covariance] = ecmninit(Data,InitMethod)
[Mean,Covariance] = ecmninit( ___,InitMethod)
```

Description

[Mean,Covariance] = ecmninit(Data,InitMethod) creates initial mean and covariance estimates for the function ecmnmle.

[Mean,Covariance] = ecmninit(___,InitMethod) adds an optional argument for InitMethod.

Examples

Compute Initial Mean and Covariance

This example shows how to compute the initial mean and covariance for five years of daily total return data for 12 computer technology stocks, with six hardware and six software companies

```
load ecmtechdemo.mat
```

The time period for this data extends from April 19, 2000 to April 18, 2005. The sixth stock in Assets is Google (GOOG), which started trading on August 19, 2004. So, all returns before August 20, 2004 are missing and represented as NaNs. Also, Amazon (AMZN) had a few days with missing values scattered throughout the past five years.

A naïve approach to the estimation of the mean and covariance for these 12 assets is to eliminate all days that have missing values for any of the 12 assets. Use the ecmninit function with the 'nanskip' option to do this.

```
[NaNMean, NaNCovar] = ecmninit(Data,'nanskip')
```

```
NaNMean = 12×1
```

```
    0.0054
   -0.0006
   -0.0006
    0.0002
   -0.0009
    0.0042
    0.0011
   -0.0005
    0.0002
    0.0001
    0.0009
    0.0010
```

```

:

NaNcovar = 12×12
10-3 ×

    0.7271    0.1003    0.0755    0.0585    0.1363    0.1030    0.0084    0.0741    0.0808    0.0
    0.1003    0.5958    0.1293    0.0919    0.2700    0.0554    0.0668    0.0548    0.1223    0.0
    0.0755    0.1293    0.2480    0.0841    0.0680    0.0322    0.0721    0.0632    0.1360    0.0
    0.0585    0.0919    0.0841    0.1414    0.0656   -0.0010    0.0386    0.0460    0.0617    0.0
    0.1363    0.2700    0.0680    0.0656    0.6223    0.2062    0.0797    0.0515    0.0850    0.0
    0.1030    0.0554    0.0322   -0.0010    0.2062    0.8376   -0.0103    0.0345    0.0236   -0.0
    0.0084    0.0668    0.0721    0.0386    0.0797   -0.0103    0.2462    0.0414    0.0881    0.0
    0.0741    0.0548    0.0632    0.0460    0.0515    0.0345    0.0414    0.1011    0.0561    0.0
    0.0808    0.1223    0.1360    0.0617    0.0850    0.0236    0.0881    0.0561    0.2642    0.0
    0.0407    0.0724    0.0562    0.0331    0.0436   -0.0034    0.0268    0.0321    0.0647    0.0
    0.0889    0.1252    0.0808    0.0499    0.1155    0.0069    0.0406    0.0494    0.1102    0.0
    0.1219    0.2317    0.1014    0.0528    0.2515    0.2788    0.0621    0.0548    0.1094    0.0
:

```

Input Arguments

Data — Data

matrix

Data, specified as an NUMSAMPLES-by-NUMSERIES matrix with NUMSAMPLES samples of a NUMSERIES-dimensional random vector. Missing values are indicated by NaNs.

Data Types: double

InitMethod — Initialization methods to compute initial estimates for mean and covariance of data

'nanskip' (default) | character vector

(Optional) Initialization methods to compute the initial estimates for the mean and covariance of data, specified as a character vector. The initialization methods are:

- 'nanskip' — Skip all records with NaNs.
- 'twostage' — Estimate mean. Fill NaNs with the mean. Then estimate the covariance.
- 'diagonal' — Form a diagonal covariance.

Data Types: char

Output Arguments

Mean — Initial estimate of mean of Data

vector

Initial estimate of the mean of the Data, returned as a NUMSERIES-by-1 column vector.

Covariance — Initial estimate of covariance of Data

matrix

Initial estimate of covariance of the Data, returned as a NUMSERIES-by-NUMSERIES matrix.

Algorithms

Model

The general model is

$$Z \sim N(\text{Mean}, \text{Covariance}),$$

where each row of **Data** is an observation of Z .

Each observation of Z is assumed to be iid (independent, identically distributed) multivariate normal, and missing values are assumed to be missing at random (MAR).

Initialization Methods

This routine has three initialization methods that cover most cases, each with its advantages and disadvantages.

nanskip

The **nanskip** method works well with small problems (fewer than 10 series or with monotone missing data patterns). It skips over any records with **NaNs** and estimates initial values from complete-data records only. This initialization method tends to yield fastest convergence of the ECM algorithm. This routine switches to the **twostage** method if it determines that significant numbers of records contain **NaN**.

twostage

The **twostage** method is the best choice for large problems (more than 10 series). It estimates the mean for each series using all available data for each series. It then estimates the covariance matrix with missing values treated as equal to the mean rather than as **NaNs**. This initialization method is robust but tends to result in slower convergence of the ECM algorithm.

diagonal

The **diagonal** method is a worst-case approach that deals with problematic data, such as disjoint series and excessive missing data (more than 33% missing data). Of the three initialization methods, this method causes the slowest convergence of the ECM algorithm.

Version History

Introduced before R2006a

See Also

ecmmle

Topics

“Portfolios with Missing Data” on page 9-21

“Mean and Covariance Estimation” on page 9-4

ecmmle

Mean and covariance of incomplete multivariate normal data

Syntax

```
ecmmle(Data)
[Mean,Covariance] = ecmmle(Data)
[Mean,Covariance] = ecmmle( ___,InitMethod,MaxIterations,Tolerance,Mean0,
Covar0)
```

Description

`ecmmle(Data)` with no output arguments, this mode displays the convergence of the ECM algorithm in a plot by estimating objective function values for each iteration of the ECM algorithm until termination.

`[Mean,Covariance] = ecmmle(Data)` estimates the mean and covariance of a data set (`Data`). If the data set has missing values, this routine implements the ECM algorithm of Meng and Rubin [2] with enhancements by Sexton and Swensen [3]. ECM stands for a conditional maximization form of the EM algorithm of Dempster, Laird, and Rubin [4].

`[Mean,Covariance] = ecmmle( ___,InitMethod,MaxIterations,Tolerance,Mean0,Covar0)` adds an optional arguments for `InitMethod`, `MaxIterations`, `Tolerance`, `Mean0`, and `Covar0`.

Examples

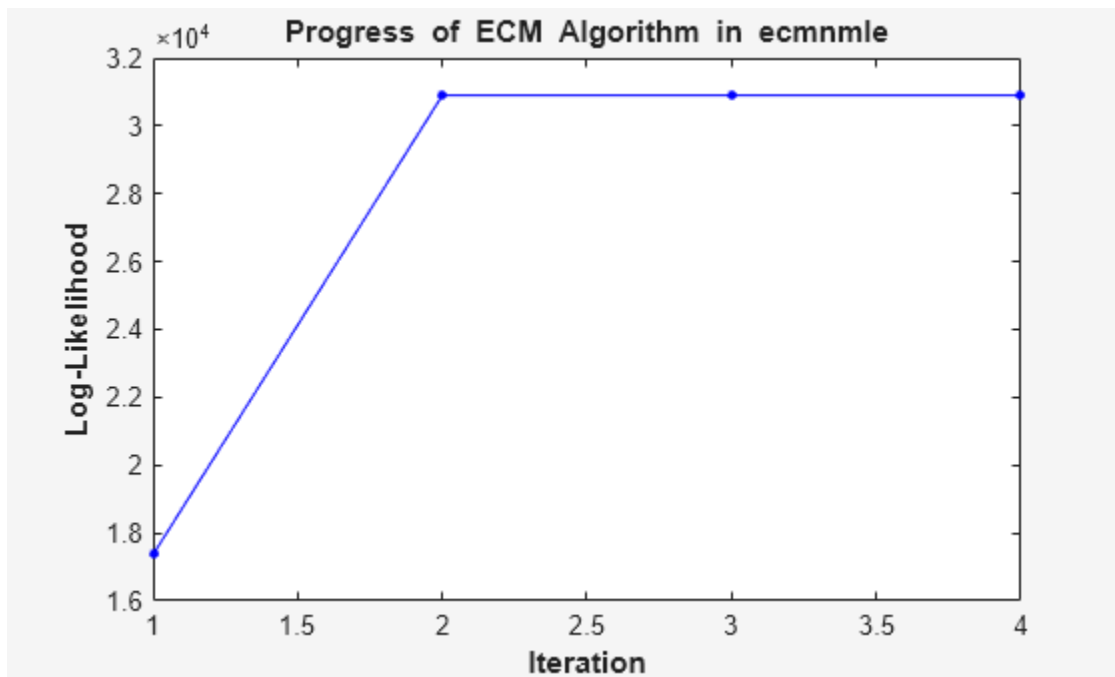
Compute Mean and Covariance of Incomplete Multivariate Normal Data

This example shows how to compute the mean and covariance of incomplete multivariate normal data for five years of daily total return data for 12 computer technology stocks, with six hardware and six software companies

```
load ecmtechdemo.mat
```

The time period for this data extends from April 19, 2000 to April 18, 2005. The sixth stock in `Assets` is Google (GOOG), which started trading on August 19, 2004. So, all returns before August 20, 2004 are missing and represented as NaNs. Also, Amazon (AMZN) had a few days with missing values scattered throughout the past five years.

```
ecmmle(Data)
```

```
ans = 12×1
```

```
0.0008
0.0008
-0.0005
0.0002
0.0011
0.0038
-0.0003
-0.0000
-0.0003
-0.0000
-0.0003
0.0004
⋮
```

This plot shows that, even with almost 87% of the Google data being NaN values, the algorithm converges after only four iterations.

```
[Mean,Covariance] = ecmmle(Data)
```

```
Mean = 12×1
```

```
0.0008
0.0008
-0.0005
0.0002
0.0011
0.0038
-0.0003
-0.0000
-0.0003
-0.0000
```

```

-0.0003
0.0004
:

Covariance = 12×12

0.0012    0.0005    0.0006    0.0005    0.0005    0.0003    0.0005    0.0003    0.0006    0.
0.0005    0.0024    0.0007    0.0006    0.0010    0.0004    0.0005    0.0003    0.0006    0.
0.0006    0.0007    0.0013    0.0007    0.0007    0.0003    0.0006    0.0004    0.0008    0.
0.0005    0.0006    0.0007    0.0009    0.0006    0.0002    0.0005    0.0003    0.0007    0.
0.0005    0.0010    0.0007    0.0006    0.0016    0.0006    0.0005    0.0003    0.0006    0.
0.0003    0.0004    0.0003    0.0002    0.0006    0.0022    0.0001    0.0002    0.0002    0.
0.0005    0.0005    0.0006    0.0005    0.0005    0.0001    0.0009    0.0003    0.0005    0.
0.0003    0.0003    0.0004    0.0003    0.0003    0.0002    0.0003    0.0005    0.0004    0.
0.0006    0.0006    0.0008    0.0007    0.0006    0.0002    0.0005    0.0004    0.0011    0.
0.0003    0.0004    0.0005    0.0004    0.0004    0.0001    0.0004    0.0003    0.0005    0.
0.0005    0.0006    0.0008    0.0005    0.0007    0.0003    0.0005    0.0004    0.0007    0.
0.0006    0.0012    0.0008    0.0007    0.0011    0.0016    0.0006    0.0004    0.0007    0.
:

```

Input Arguments

Data — Data

matrix

Data, specified as an NUMSAMPLES-by-NUMSERIES matrix with NUMSAMPLES samples of a NUMSERIES-dimensional random vector. Missing values are indicated by NaNs.

Data Types: double

InitMethod — Initialization methods to compute initial estimates for mean and covariance of data

'nanskip' (default) | character vector

(Optional) Initialization methods to compute the initial estimates for the mean and covariance of data, specified as a character vector. The initialization methods are:

- 'nanskip' — Skip all records with NaNs.
- 'twostage' — Estimate mean. Fill NaNs with the mean. Then estimate the covariance.
- 'diagonal' — Form a diagonal covariance.

Note If you supply Mean0 and Covar0, InitMethod is not executed.

Data Types: char

MaxIterations — Maximum number of iterations

50 (default) | numeric

(Optional) Maximum number of iterations for the expectation conditional maximization (ECM) algorithm, specified as a numeric.

Data Types: double

Tolerance — Convergence tolerance

1.0e-8 (default) | numeric

(Optional) Convergence tolerance for the ECM algorithm, specified as a numeric. If $\text{Tolerance} \leq 0$, perform maximum iterations specified by `MaxIterations` and do not evaluate the objective function at each step unless in display mode.

Data Types: double

Mean0 — Estimate for the mean

[] (default) | matrix

(Optional) Estimate for the mean, specified as a NUMSERIES-by-1 column vector. If you leave `Mean0` unspecified ([]), the method specified by `InitMethod` is used. If you specify `Mean0`, you must also specify `Covar0`.

Data Types: double

Covar0 — Estimate for the covariance

[] (default) | matrix

(Optional) Estimate for the covariance, specified as a NUMSERIES-by-NUMSERIES matrix, where the input matrix must be positive-definite. If you leave `Covar0` unspecified ([]), the method specified by `InitMethod` is used. If you specify `Covar0`, you must also specify `Mean0`.

Data Types: double

Output Arguments**Mean — Maximum likelihood parameter estimates for mean of Data**

vector

Maximum likelihood parameter estimates for the mean of the `Data` using ECM algorithm, returned as a NUMSERIES-by-1 column vector.

Covariance — Maximum likelihood parameter estimates for covariance of Data

matrix

Maximum likelihood parameter estimates for the covariance of the `Data` using ECM algorithm, returned as a NUMSERIES-by-NUMSERIES matrix.

Algorithms**Model**

The general model is

$$Z \sim N(\text{Mean}, \text{Covariance}),$$

where each row of `Data` is an observation of Z .

Each observation of Z is assumed to be iid (independent, identically distributed) multivariate normal, and missing values are assumed to be missing at random (MAR). See Little and Rubin [1] for a precise definition of MAR.

This routine estimates the mean and covariance from given data. If data values are missing, the routine implements the ECM algorithm of Meng and Rubin [2] with enhancements by Sexton and Swensen [3].

If a record is empty (every value in a sample is NaN), this routine ignores the record because it contributes no information. If such records exist in the data, the number of nonempty samples used in the estimation is $\leq \text{NumSamples}$.

The estimate for the covariance is a biased maximum likelihood estimate (MLE). To convert to an unbiased estimate, multiply the covariance by $\text{Count}/(\text{Count} - 1)$, where Count is the number of nonempty samples used in the estimation.

Requirements

This routine requires consistent values for `NUMSAMPLES` and `NUMSERIES` with `NUMSAMPLES > NUMSERIES`. It must have enough nonmissing values to converge. Finally, it must have a positive-definite covariance matrix. Although the references provide some necessary and sufficient conditions, general conditions for existence and uniqueness of solutions in the missing-data case, do not exist. The main failure mode is an ill-conditioned covariance matrix estimate. Nonetheless, this routine works for most cases that have less than 15% missing data (a typical upper bound for financial data).

Initialization Methods

This routine has three initialization methods that cover most cases, each with its advantages and disadvantages. The ECM algorithm always converges to a minimum of the observed negative log-likelihood function. If you override the initialization methods, you must ensure that the initial estimate for the covariance matrix is positive-definite.

The following is a guide to the supported initialization methods.

- **nanskip** — The **nanskip** method works well with small problems (fewer than 10 series or with monotone missing data patterns). It skips over any records with NaNs and estimates initial values from complete-data records only. This initialization method tends to yield fastest convergence of the ECM algorithm. This routine switches to the **twostage** method if it determines that significant numbers of records contain NaN.
- **twostage** — The **twostage** method is the best choice for large problems (more than 10 series). It estimates the mean for each series using all available data for each series. It then estimates the covariance matrix with missing values treated as equal to the mean rather than as NaNs. This initialization method is robust but tends to result in slower convergence of the ECM algorithm.
- **diagonal** —

The **diagonal** method is a worst-case approach that deals with problematic data, such as disjoint series and excessive missing data (more than 33% of data missing). Of the three initialization methods, this method causes the slowest convergence of the ECM algorithm. If problems occur with this method, use display mode to examine convergence and modify either `MaxIterations` or `Tolerance`, or try alternative initial estimates with `Mean0` and `Covar0`. If all else fails, try

```
Mean0 = zeros(NumSeries);  
Covar0 = eye(NumSeries,NumSeries);
```

Given estimates for mean and covariance from this routine, you can estimate standard errors with the companion routine `ecmnstd`.

Convergence

The ECM algorithm does not work for all patterns of missing values. Although it works in most cases, it can fail to converge if the covariance becomes singular. If this occurs, plots of the log-likelihood function tend to have a constant upward slope over many iterations as the log of the negative determinant of the covariance goes to zero. In some cases, the objective fails to converge due to machine precision errors. No general theory of missing data patterns exists to determine these cases. An example of a known failure occurs when two time series are proportional wherever both series contain nonmissing values.

Version History

Introduced before R2006a

References

- [1] Little, Roderick J. A. and Donald B. Rubin. *Statistical Analysis with Missing Data*. 2nd Edition. John Wiley & Sons, Inc., 2002.
- [2] Meng, Xiao-Li and Donald B. Rubin. "Maximum Likelihood Estimation via the ECM Algorithm." *Biometrika*. Vol. 80, No. 2, 1993, pp. 267-278.
- [3] Sexton, Joe and Anders Rygh Swensen. "ECM Algorithms that Converge at the Rate of EM." *Biometrika*. Vol. 87, No. 3, 2000, pp. 651-662.
- [4] Dempster, A. P., N. M. Laird, and Donald B. Rubin. "Maximum Likelihood from Incomplete Data via the EM Algorithm." *Journal of the Royal Statistical Society. Series B*, Vol. 39, No. 1, 1977, pp. 1-37.

See Also

ecmnfish | ecmnhess | ecmninit | ecmnobj | ecmnstd

Topics

"Multivariate Normal Regression With Missing Data" on page 9-14
 "Portfolios with Missing Data" on page 9-21
 "Mean and Covariance Estimation" on page 9-4

ecmnobj

Multivariate normal negative log-likelihood function

Syntax

```
Objective = ecmnobj(Data,Mean,Covariance)
Objective = ecmnobj( ___,CholCovariance)
```

Description

`Objective = ecmnobj(Data,Mean,Covariance)` evaluates the negative log-likelihood function for `ecmmle`.

Use `ecmnobj` after estimating the mean and covariance of `Data` with `ecmmle`.

`Objective = ecmnobj( ___,CholCovariance)` adds an optional argument for `CholCovariance`.

Examples

Compute Value of the Observed Negative Log-Likelihood Function for Data

This example shows how to compute the value of the observed negative log-likelihood function for five years of daily total return data for 12 computer technology stocks, with six hardware and six software companies

```
load ecmtechdemo.mat
```

The time period for this data extends from April 19, 2000 to April 18, 2005. The sixth stock in `Assets` is Google (GOOG), which started trading on August 19, 2004. So, all returns before August 20, 2004 are missing and represented as NaNs. Also, Amazon (AMZN) had a few days with missing values scattered throughout the past five years.

```
[ECMMean, ECMCovar] = ecmmle(Data)
```

```
ECMMean = 12×1
```

```
    0.0008
    0.0008
   -0.0005
    0.0002
    0.0011
    0.0038
   -0.0003
   -0.0000
   -0.0003
   -0.0000
   -0.0003
    0.0004
      ⋮
```

ECMCovar = 12×12

0.0012	0.0005	0.0006	0.0005	0.0005	0.0003	0.0005	0.0003	0.0006	0.0006
0.0005	0.0024	0.0007	0.0006	0.0010	0.0004	0.0005	0.0003	0.0006	0.0006
0.0006	0.0007	0.0013	0.0007	0.0007	0.0003	0.0006	0.0004	0.0008	0.0008
0.0005	0.0006	0.0007	0.0009	0.0006	0.0002	0.0005	0.0003	0.0007	0.0007
0.0005	0.0010	0.0007	0.0006	0.0016	0.0006	0.0005	0.0003	0.0006	0.0006
0.0003	0.0004	0.0003	0.0002	0.0006	0.0022	0.0001	0.0002	0.0002	0.0002
0.0005	0.0005	0.0006	0.0005	0.0005	0.0001	0.0009	0.0003	0.0005	0.0005
0.0003	0.0003	0.0004	0.0003	0.0003	0.0002	0.0003	0.0005	0.0004	0.0004
0.0006	0.0006	0.0008	0.0007	0.0006	0.0002	0.0005	0.0004	0.0011	0.0011
0.0003	0.0004	0.0005	0.0004	0.0004	0.0001	0.0004	0.0003	0.0005	0.0005
0.0005	0.0006	0.0008	0.0005	0.0007	0.0003	0.0005	0.0004	0.0007	0.0007
0.0006	0.0012	0.0008	0.0007	0.0011	0.0016	0.0006	0.0004	0.0007	0.0007
:	:	:	:	:	:	:	:	:	:

To evaluate the negative log-likelihood function for `ecmmle`, use `ecmnobj` based on the current maximum likelihood parameter estimates.

```
Objective = ecmnobj(Data,ECMMean,ECMCovar)
```

```
Objective =  
-3.0898e+04
```

Input Arguments

Data — Data

matrix

Data, specified as an NUMSAMPLES-by-NUMSERIES matrix with NUMSAMPLES samples of a NUMSERIES-dimensional random vector. Missing values are indicated by NaNs.

Data Types: double

Mean — Maximum likelihood parameter estimates for mean of Data

vector

Maximum likelihood parameter estimates for the mean of the Data using the ECM algorithm, specified as a NUMSERIES-by-1 column vector.

Covariance — Maximum likelihood parameter estimates for covariance of Data

matrix

Maximum likelihood parameter estimates for the covariance of the Data using the ECM algorithm, specified as a NUMSERIES-by-NUMSERIES matrix.

cholCovariance — Cholesky decomposition of covariance matrix

[] (default) | matrix

(Optional) Cholesky decomposition of covariance matrix, specified as a matrix using `chol` as:

```
chol(Covariance)
```

Data Types: double

Output Arguments

Objective — Value of the observed negative log-likelihood function over Data

numeric

Value of the observed negative log-likelihood function over the Data, returned as a numeric value.

Version History

Introduced before R2006a

See Also

chol | ecmnml

Topics

“Multivariate Normal Regression Without Missing Data” on page 9-13

“Multivariate Normal Regression With Missing Data” on page 9-14

ecmnstd

Standard errors for mean and covariance of incomplete data

Syntax

```
[StdMean,StdCovar] = ecmnstd(Data,Mean,Covariance)
[StdMean,StdCovar] = ecmnstd( ___,Method)
```

Description

[StdMean,StdCovar] = ecmnstd(Data,Mean,Covariance) computes standard errors for mean and covariance of incomplete data.

Use ecmnstd after estimating the mean and covariance of Data with ecmmle. If the mean and distinct covariance elements are treated as the parameter θ in a complete-data maximum-likelihood estimation, then as the number of samples increases, θ attains asymptotic normality such that

$$\theta - E[\theta] \sim N(0, I^{-1}(\theta)),$$

where $E[\theta]$ is the mean and $I(\theta)$ is the Fisher information matrix.

With missing data, the Hessian $H(\theta)$ is a good approximation for the Fisher information (which can only be approximated when data is missing).

[StdMean,StdCovar] = ecmnstd(___,Method) adds an optional argument for Method.

Examples

Compute Standard Errors for Mean and Covariance of Incomplete Data

This example shows how to compute the standard errors for mean and covariance of incomplete data for five years of daily total return data for 12 computer technology stocks, with six hardware and six software companies

```
load ecmtechdemo.mat
```

The time period for this data extends from April 19, 2000 to April 18, 2005. The sixth stock in Assets is Google (GOOG), which started trading on August 19, 2004. So, all returns before August 20, 2004 are missing and represented as NaNs. Also, Amazon (AMZN) had a few days with missing values scattered throughout the past five years.

```
[ECMMean, ECMCovar] = ecmmle(Data)
```

```
ECMMean = 12×1
```

```
    0.0008
    0.0008
   -0.0005
    0.0002
    0.0011
```

```

0.0038
-0.0003
-0.0000
-0.0003
-0.0000
-0.0003
0.0004
:
```

```
ECMCovar = 12×12
```

```

0.0012  0.0005  0.0006  0.0005  0.0005  0.0003  0.0005  0.0003  0.0006  0.
0.0005  0.0024  0.0007  0.0006  0.0010  0.0004  0.0005  0.0003  0.0006  0.
0.0006  0.0007  0.0013  0.0007  0.0007  0.0003  0.0006  0.0004  0.0008  0.
0.0005  0.0006  0.0007  0.0009  0.0006  0.0002  0.0005  0.0003  0.0007  0.
0.0005  0.0010  0.0007  0.0006  0.0016  0.0006  0.0005  0.0003  0.0006  0.
0.0003  0.0004  0.0003  0.0002  0.0006  0.0022  0.0001  0.0002  0.0002  0.
0.0005  0.0005  0.0006  0.0005  0.0005  0.0001  0.0009  0.0003  0.0005  0.
0.0003  0.0003  0.0004  0.0003  0.0003  0.0002  0.0003  0.0005  0.0004  0.
0.0006  0.0006  0.0008  0.0007  0.0006  0.0002  0.0005  0.0004  0.0011  0.
0.0003  0.0004  0.0005  0.0004  0.0004  0.0001  0.0004  0.0003  0.0005  0.
0.0005  0.0006  0.0008  0.0005  0.0007  0.0003  0.0005  0.0004  0.0007  0.
0.0006  0.0012  0.0008  0.0007  0.0011  0.0016  0.0006  0.0004  0.0007  0.
:
```

To evaluate the impact of the estimation error and, in particular, the effect of missing data, use `ecmstd` to calculate standard errors. Although it is possible to estimate the standard errors for both the mean and covariance, the standard errors for the mean estimates alone are usually the main quantities of interest.

```
StdMeanF = ecmstd(Data,ECMMean,ECMCovar, 'fisher')
```

```
StdMeanF = 12×1
```

```

0.0010
0.0014
0.0010
0.0009
0.0011
0.0013
0.0009
0.0006
0.0009
0.0007
0.0010
0.0012
:
```

Calculate standard errors that use the data-generated Hessian matrix (which accounts for the possible loss of information due to missing data) with the option `'hessian'`.

```
StdMeanH = ecmstd(Data,ECMMean,ECMCovar, 'hessian')
```

```
StdMeanH = 12×1
```

```

0.0010
0.0014
0.0010
0.0009
0.0011
0.0021
0.0009
0.0006
0.0009
0.0007
0.0010
0.0012
:

```

The difference in the standard errors shows the increase in uncertainty of estimation of asset expected returns due to missing data. To view the differences:

Assets

```

Assets = 1×12 cell
        {'AAPL'}    {'AMZN'}    {'CSCO'}    {'DELL'}    {'EBAY'}    {'GOOG'}    {'HPQ'}    {'IBM'}

```

StdMeanH'

```

ans = 1×12
        0.0010    0.0014    0.0010    0.0009    0.0011    0.0021    0.0009    0.0006    0.0009    0.

```

StdMeanF'

```

ans = 1×12
        0.0010    0.0014    0.0010    0.0009    0.0011    0.0013    0.0009    0.0006    0.0009    0.

```

StdMeanH' - StdMeanF'

```

ans = 1×12
10-3 ×
        -0.0000    0.0021    -0.0000    -0.0000    -0.0000    0.7742    -0.0000    -0.0000    -0.0000    -0.

```

The two assets with missing data, AMZN and GOOG, are the only assets to have differences due to missing information.

Input Arguments

Data — Data

matrix

Data, specified as an NUMSAMPLES-by-NUMSERIES matrix with NUMSAMPLES samples of a NUMSERIES-dimensional random vector. Missing values are indicated by NaNs.

Data Types: double

Mean — Maximum likelihood parameter estimates for mean of Data

vector

Maximum likelihood parameter estimates for the mean of the Data using the ECM algorithm, specified as a NUMSERIES-by-1 column vector.

Covariance — Maximum likelihood parameter estimates for covariance of Data

matrix

Maximum likelihood parameter estimates for the covariance of the Data using the ECM algorithm, specified as a NUMSERIES-by-NUMSERIES matrix.

Method — Method of estimation for standard error calculations

'hessian' (default) | character vector

(Optional) Method of estimation for standard error calculations, specified as a character vector. The estimation methods are:

- 'hessian' — The Hessian of the observed negative log-likelihood function. This method is recommended since the resultant standard errors incorporate the increase uncertainties due to missing data. In particular, standard errors calculated with the Hessian are generally larger than standard errors calculated with the Fisher information matrix.
- 'fisher' — The Fisher information matrix.

Data Types: char

Output Arguments**StdMean — Standard errors of estimates for each element of Mean vector**

vector

Standard errors of estimates for each element of Mean vector, returned as a NUMSERIES-by-1 column vector.

StdCovar — Standard errors of estimates for each element of Covariance matrix

matrix

Standard errors of estimates for each element of Covariance matrix, returned as a NUMSERIES-by-NUMSERIES matrix.

Version History

Introduced before R2006a

See Also

ecmmle

Topics

"Portfolios with Missing Data" on page 9-21

"Mean and Covariance Estimation" on page 9-4

effrr

Effective rate of return

Syntax

```
Return = effrr(Rate,NumPeriods)
```

Description

`Return = effrr(Rate,NumPeriods)` calculates the annual effective rate of return. Compounding continuously returns `Return` equivalent to $(e^{\text{Rate}} - 1)$.

Examples

Compute the Annual Effective Rate of Return

This example shows how to find the effective annual rate of return based on an annual percentage rate of 9% compounded monthly.

```
Return = effrr(0.09, 12)
```

```
Return =  
0.0938
```

Input Arguments

Rate — Annual percentage rate

scalar numeric decimal

Annual percentage rate, specified as a scalar numeric decimal.

Data Types: double

NumPeriods — Number of compounding periods per year

scalar integer

Number of compounding periods per year, specified as a scalar integer.

Data Types: double

Output Arguments

Return — Annual effective rate of return

scalar numeric decimal

Annual effective rate of return, returned as a scalar numeric decimal.

More About

Effective Rate of Return

The effective rate of return (ERR) is a measure of the actual return on an investment, taking into account the effects of compounding over a specific period.

The ERR provides a more accurate representation of the profitability of an investment compared to nominal interest rates, which do not consider compounding effects. The effective rate of return is particularly useful for comparing different investment options with varying compounding periods.

Version History

Introduced before R2006a

See Also

`nomrr`

Topics

“Analyzing and Computing Cash Flows” on page 2-11

elpm

Compute expected lower partial moments for normal asset returns

Syntax

```
elpm(Mean,Sigma)
elpm(Mean,Sigma,MAR)
elpm(Mean,Sigma,MAR,Order)
Moment = elpm(MeanSigmaMAROrder)
```

Description

`elpm(Mean,Sigma)` compute expected lower partial moments (`elpm`) relative to a default value of `MAR` for each asset in a `NUMORDERS`-by-`NUMSERIES` matrix.

`elpm(Mean,Sigma,MAR)` computes expected lower partial moments (`elpm`) relative to a `MAR` for each asset in a `NUMORDERS`-by-`NUMSERIES` matrix.

`elpm(Mean,Sigma,MAR,Order)` computes expected lower partial moments (`elpm`) relative to a `MAR` and `Order` for each asset in a `NUMORDERS`-by-`NUMSERIES` matrix.

`Moment = elpm(MeanSigmaMAROrder)` computes expected lower partial moments (`elpm`) relative to a default value of `MAR` for each asset in a `NUMORDERS`-by-`NUMSERIES` matrix `Moment`.

Examples

Compute Expected Lower Partial Moments

This example shows how to compute expected lower partial moments based on the mean and standard deviations of normally distributed asset returns. The `elpm` function works with the mean and standard deviations for multiple assets and multiple orders.

```
load FundMarketCash
Returns = tick2ret(TestData);
MAR = mean>Returns(:,3))

MAR =
0.0017

Mean = mean>Returns)

Mean = 1×3
    0.0038    0.0030    0.0017

Sigma = std>Returns, 1)

Sigma = 1×3
```

```
0.0229    0.0389    0.0009
```

Assets

```
Assets = 1×3 cell
    {'Fund'}    {'Market'}    {'Cash'}
```

```
ELPM = elpm(Mean, Sigma, MAR, [0 1 2])
```

```
ELPM = 3×3
```

```
0.4647    0.4874    0.5000
0.0082    0.0149    0.0004
0.0002    0.0007    0.0000
```

Based on the moments of each asset, the expected values for lower partial moments imply better than expected performance for the fund and market and worse than expected performance for cash. The `elpm` function works with either degenerate or nondegenerate normal random variables. For example, if cash were truly riskless, its standard deviation would be 0. You can examine the difference in average shortfall.

```
RisklessCash = elpm(Mean(3), 0, MAR, 1)
```

```
RisklessCash =
0
```

Input Arguments

Mean — Mean returns

vector

Mean returns, specified as a `NUMSERIES` vector with mean returns for a collection of `NUMSERIES` assets.

Data Types: `double`

Sigma — Standard deviation of returns

vector

Standard deviation of returns, specified as a `NUMSERIES` vector with standard deviation of returns for a collection of `NUMSERIES` assets.

Data Types: `double`

MAR — Minimum acceptable return

0 (default) | numeric

(Optional) Minimum acceptable return, specified as a scalar numeric. MAR is a cutoff level of return such that all returns above MAR contribute nothing to the lower partial moment.

Data Types: `double`

Order — Moment orders

0 (default) | scalar numeric | vector

(Optional) Moment orders, specified as either a scalar or a NUMORDERS vector of nonnegative integer moment orders. If no order specified, the default `Order = 0`, which is the shortfall probability. The `elpm` function does not work for negative or a noninteger `Order`.

Data Types: `double`

Output Arguments

Moment — Expected lower partial moments

matrix

Expected Lower partial moments, returned as a NUMORDERS-by-NUMSERIES matrix of expected lower partial moments with NUMORDERS Orders and NUMSERIES series, that is, each row contains expected lower partial moments for a given Order. The output Moment for the lower partial moment represents the moments of asset returns that fall below a minimum acceptable level of return.

Note To compute upper partial moments, reverse the signs of both the input Mean and MAR (do not reverse the signs of either Sigma or the output). This function computes expected lower partial moments with the mean and standard deviation of normally distributed asset returns. To compute sample lower partial moments from asset returns which have no distributional assumptions, use `lpm`.

More About

Lower Partial Moments

Use *lower partial moments* to examine what is colloquially known as “downside risk.”

The main idea of the lower partial moment framework is to model moments of asset returns that fall below a minimum acceptable level of return. To compute lower partial moments from data, use `lpm` to calculate lower partial moments for multiple asset return series and for multiple moment orders. To compute expected values for lower partial moments under several assumptions about the distribution of asset returns, use `elpm` to calculate lower partial moments for multiple assets and for multiple orders.

Version History

Introduced in R2006b

References

- [1] Bawa, V.S. "Safety-First, Stochastic Dominance, and Optimal Portfolio Choice." *Journal of Financial and Quantitative Analysis*. Vol. 13, No. 2, June 1978, pp. 255-271.
- [2] Harlow, W.V. "Asset Allocation in a Downside-Risk Framework." *Financial Analysts Journal*. Vol. 47, No. 5, September/October 1991, pp. 28-40.
- [3] Harlow, W.V. and K. S. Rao. "Asset Pricing in a Generalized Mean-Lower Partial Moment Framework: Theory and Evidence." *Journal of Financial and Quantitative Analysis*. Vol. 24, No. 3, September 1989, pp. 285-311.

[4] Sortino, F.A. and Robert van der Meer. "Downside Risk." *Journal of Portfolio Management*. Vol. 17, No. 5, Spring 1991, pp. 27-31.

See Also

lpm

Topics

"Performance Metrics Overview" on page 7-2

emaxdrawdown

Compute expected maximum drawdown for Brownian motion

Syntax

`ExpDrawdown = emaxdrawdown(Mu,Sigma,T)`

Description

`ExpDrawdown = emaxdrawdown(Mu,Sigma,T)` computes the expected maximum drawdown for a Brownian motion for each time period in T using the following equation:

$$dX(t) = \mu dt + \sigma dW(t).$$

If the Brownian motion is geometric with the stochastic differential equation

$$dS(t) = \mu_0 S(t) dt + \sigma_0 S(t) dW(t)$$

then use Ito's lemma with $X(t) = \log(S(t))$ such that

$$\mu = \mu_0 - 0.5\sigma_0^2,$$

$$\sigma = \sigma_0$$

converts it to the form used here.

Examples

Compute Expected Maximum Drawdown

This example shows how to use log-return moments of a fund to compute the expected maximum drawdown (EMaxDD) and then compare it with the realized maximum drawdown (MaxDD).

```
load FundMarketCash
logReturns = log(TestData(2:end,:)) ./ TestData(1:end - 1,:);
Mu = mean(logReturns(:,1));
Sigma = std(logReturns(:,1),1);
T = size(logReturns,1);
```

```
MaxDD = maxdrawdown(TestData(:,1),'geometric')
```

```
MaxDD =
0.1813
```

```
EMaxDD = emaxdrawdown(Mu, Sigma, T)
```

```
EMaxDD =
0.1545
```

The drawdown observed in this time period is above the expected maximum drawdown. There is no contradiction here. The expected maximum drawdown is not an upper bound on the maximum losses from a peak, but an estimate of their average, based on a geometric Brownian motion assumption.

Input Arguments

Mu — Drift term of a Brownian motion with drift

numeric

Drift term of a Brownian motion with drift., specified as a scalar numeric.

Data Types: double

Sigma — Diffusion term of a Brownian motion with drift

numeric

Diffusion term of a Brownian motion with drift, specified as a scalar numeric.

Data Types: double

T — A time period of interest

numeric | vector

A time period of interest, specified as a scalar numeric or vector.

Data Types: double

Output Arguments

ExpDrawdown — Expected maximum drawdown

numeric

Expected maximum drawdown, returned as a numeric. ExpDrawdown is computed using an interpolation method. Values are accurate to a fraction of a basis point. Maximum drawdown is nonnegative since it is the change from a peak to a trough.

Note To compare the actual results from maxdrawdown with the expected results of emaxdrawdown, set the Format input argument of maxdrawdown to either of the nondefault values ('arithmetic' or 'geometric'). These are the only two formats that emaxdrawdown supports.

Version History

Introduced in R2006b

References

- [1] Malik, M. I., Amir F. Atiya, Amrit Pratap, and Yaser S. Abu-Mostafa. "On the Maximum Drawdown of a Brownian Motion." *Journal of Applied Probability*. Vol. 41, Number 1, March 2004, pp. 147-161.

See Also

maxdrawdown

Topics

"Performance Metrics Overview" on page 7-2

estimateCustomObjectivePortfolio

Estimate optimal portfolio for user-defined objective function for Portfolio object

Syntax

```
[pwgt,pbuy,psell,exitflag] = estimateCustomObjectivePortfolio(obj,fun)
[pwgt,pbuy,psell,exitflag] = estimateCustomObjectivePortfolio( ___,Name=Value)
```

Description

[pwgt,pbuy,psell,exitflag] = estimateCustomObjectivePortfolio(obj,fun) estimates optimal portfolio with a user-defined objective function for a Portfolio object. For details on using estimateCustomObjectivePortfolio, see “Solver Guidelines for Custom Objective Problems Using Portfolio Objects” on page 4-119.

[pwgt,pbuy,psell,exitflag] = estimateCustomObjectivePortfolio(___,Name=Value) adds optional name-value arguments.

Examples

Solve Continuous Problems Using Custom Objectives

This example shows how to use estimateCustomObjectivePortfolio to solve a portfolio problem with a custom objective. You define the constraints for portfolio problems using functions for the Portfolio object and then you specify the objective function as an input to estimateCustomObjectivePortfolio. The objective function must be continuous (and preferably smooth).

Create Portfolio Object

Find the portfolio that solves the problem:

$$\begin{aligned} \min_x \quad & x^T x \\ \text{s.t.} \quad & \sum_i x_i = 1 \\ & x \geq 0 \end{aligned}$$

Create a Portfolio object and set the default constraints using setDefaultConstraints. In this case, the portfolio problem does not involve the mean or covariance of the assets returns. Therefore, you do not need to define the assets moments to create the Portfolio object. You need only to define the number of assets in the problem.

```
% Create a Portfolio object
p = Portfolio(NumAssets=6);
% Define the constraints of the portfolio methods
p = setDefaultConstraints(p); % Long-only, fully invested weights
```

Define Objective Function

Define a function handle for the objective function $x^T x$.

```
% Define the objective function
objFun = @(x) x'*x;
```

Solve Portfolio Problem

Use `estimateCustomObjectivePortfolio` to compute the solution to the problem.

```
% Solve portfolio problem
wMin = estimateCustomObjectivePortfolio(p,objFun)

wMin = 6×1

    0.1667
    0.1667
    0.1667
    0.1667
    0.1667
    0.1667
```

The `estimateCustomObjectivePortfolio` function automatically assumes that the objective sense is to minimize. You can change that default behavior by using the `estimateCustomObjectivePortfolio` name-value argument `ObjectiveSense='maximize'`.

Add Gross or Net Return Constraints for Problems with Custom Objectives

This example shows how to use `estimateCustomObjectivePortfolio` to solve a portfolio problem with a custom objective and a return constraint. You define the constraints for portfolio problems, other than the return constraint, using functions for the `Portfolio` object and then you specify the objective function as an input to `estimateCustomObjectivePortfolio`. The objective function must be continuous (and preferably smooth). To specify a return constraint, you use the `TargetReturn` name-value argument.

Create Portfolio Object

Find the portfolio that solves the problem:

$$\begin{aligned} \min_x \quad & x^T x \\ \text{s.t.} \quad & \sum_i x_i = 1 \\ & x \geq 0 \end{aligned}$$

Create a `Portfolio` object and set the default constraints using `setDefaultConstraints`.

```
% Create a Portfolio object
load('SixStocks.mat')
p = Portfolio(AssetMean=AssetMean,AssetCovar=AssetCovar);
% Define the constraints of the portfolio methods
p = setDefaultConstraints(p); % Long-only, fully invested weights
```

Define Objective Function

Define a function handle for the objective function $x^T x$.

```
% Define the objective function
objFun = @(x) x'*x;
```

Add Return Constraints

When using the `Portfolio` object, you can use `estimateFrontierByReturn` to add a return constraint to the portfolio. However, when using the `estimateCustomObjectivePortfolio` function with a `Portfolio` object, you must add return constraints by using the `TargetReturn` name-value argument with a return target value.

The `Portfolio` object supports two types of return constraints: gross return and net return. The type of return constraint that is added to the portfolio problem is implicitly defined by whether you provide buy or sell costs to the `Portfolio` object using `setCosts`. If no buy or sell costs are present, the added return constraint is a gross return constraint. Otherwise, a net return constraint is added.

Gross Return Constraint

The gross portfolio return constraint for a portfolio is

$$r_0 + (\mu - r_0)^T x \geq \mu_0,$$

where r_0 is the risk-free rate (with 0 as default), μ is the mean of assets returns, and μ_0 is the target return.

Since buy or sell costs are not needed to add a gross return constraint, the `Portfolio` object does not need to be modified before using `estimateCustomObjectivePortfolio`.

```
% Set a return target
ret0 = 0.03;
% Solve portfolio problem with a gross return constraint
wGross = estimateCustomObjectivePortfolio(p,objFun, ...
    TargetReturn=ret0)
```

```
wGross = 6×1
```

```
0.1377
0.1106
0.1691
0.1829
0.1179
0.2818
```

The return constraint is not added to the `Portfolio` object. In other words, the `Portfolio` properties are not modified by adding a gross return constraint in `estimateCustomObjectivePortfolio`.

Net Return Constraint

The net portfolio return constraint for a portfolio is

$$r_0 + (\mu - r_0)^T x - c_B^T \max\{0, x - x_0\} - c_S^T \max\{0, x_0 - x\} \geq \mu_0,$$

where r_0 is the risk-free rate (with 0 as default), μ is the mean of assets returns, c_B is the proportional buy cost, c_S is the proportional sell cost, x_0 is the initial portfolio, and μ_0 is the target return.

To add net return constraints to the portfolio problem, you must use `setCosts` with the `Portfolio` object. If the `Portfolio` object has either of these costs, `estimateCustomObjectivePortfolio` automatically assumes that any added return constraint is a net return constraint.

```
% Add buy and sell costs to the Portfolio object
buyCost = 0.002;
sellCost = 0.001;
initPort = zeros(p.NumAssets,1);
p = setCosts(p,buyCost,sellCost,initPort);
% Solve portfolio problem with a net return constraint
% wNet = estimateCustomObjectivePortfolio(p,objFun, ...
%     TargetReturn=ret0)
```

As with the gross return constraint, the net return constraint is not added to the `Portfolio` object properties, however the `Portfolio` object is modified by the addition of buy or sell costs. Adding buy or sell costs to the `Portfolio` object does not affect any constraints besides the return constraint, but these costs do affect the solution of maximum return problems because the solution is the maximum *net* return instead of the maximum *gross* return.

Solve Portfolio Problems with Custom Objectives and Cardinality Constraints

This example shows how to use `estimateCustomObjectivePortfolio` to solve a portfolio problem with a custom objective and cardinality constraints. You define the constraints for portfolio problems using functions for the `Portfolio` object and then you specify the objective function as an input to `estimateCustomObjectivePortfolio`. For problems with cardinality constraints or continuous bounds, the objective function must be continuous and convex.

Create Portfolio Object

Find a long-only, fully weighted portfolio with half the assets that minimizes the tracking error to the equally weighted portfolio. Furthermore, if an asset is present in the portfolio, at least 10% should be invested in that asset. The portfolio problem is as follows:

$$\begin{aligned} \min_x \quad & (x - x_0)^T \Sigma (x - x_0) \\ \text{s.t.} \quad & \sum_i x_i = 1, \\ & \sum_i \#(x_i \neq 0) = 3, \\ & x_i \geq 0.1 \text{ or } x_i = 0 \end{aligned}$$

Create a `Portfolio` object and set the assets moments.

```
load('SixStocks.mat')
p = Portfolio(AssetMean=AssetMean,AssetCovar=AssetCovar);
nAssets = size(AssetMean,1);
```

Define the `Portfolio` constraints.

```
% Fully invested portfolio
p = setBudget(p,1,1);
% Cardinality constraint
p = setMinMaxNumAssets(p,3,3);
% Conditional bounds
p = setBounds(p,0.1,[],BoundType="conditional");
```

Define Objective Function

Define a function handle for the objective function.

```
% Define the objective function
EWP = 1/nAssets*ones(nAssets,1);
trackingError = @(x) (x-EWP)'*p.AssetCovar*(x-EWP);
```

Solve Portfolio Problem

Use `estimateCustomObjectivePortfolio` to compute the solution to the problem.

```
% Solve portfolio problem
wMinTE = estimateCustomObjectivePortfolio(p,trackingError)
```

```
wMinTE = 6×1
```

```
0.1795
0.3507
0
0.4698
0
0
```

Using ObjectiveBound and InitialPoint with Custom Objective Portfolios

This example shows how to use the name-value arguments `ObjectiveBound` and `InitialPoint` with `estimateCustomObjectivePortfolio`.

Load the returns data in `CAPMuniverse.mat`. Then, create a standard mean-variance `Portfolio` object with default constraints, which is a long-only portfolio whose weights sum to 1. For this example, you can define the feasible region of weights X as

$$X = \left\{ x \mid \sum_{i=1}^n x_i = 1, x_i \geq 0 \right\}.$$

```
% Load data
load CAPMuniverse

% Create a mean-variance Portfolio object with default constraints
p = Portfolio(AssetList=Assets(1:12));
p = estimateAssetMoments(p,Data(:,1:12));
p = setDefaultConstraints(p);

% Add conditional bounds
pInt = setBounds(p,0.1,[],BoundType='cond');
```

Using ObjectiveBound

Use the `ObjectiveBound` name-value argument to provide an initial bound for the objective function. Providing an initial bound saves computation time because the solver does not have to solve an initial NLP relaxation to obtain the initial bound. Set up the MINLP solver, then test its performance when you do not provide an initial bound.

```
% Minimize tracking error without a lower bound
wTE = 1/p.NumAssets*ones(p.NumAssets,1);
trackingError = @(w) (w-wTE)'*p.AssetCovar*(w-wTE);
pInt2 = setSolverMINLP(pInt,'OuterApproximation',...
    ExtendedFormulation=true);
s = tic;
wTE_NoBound = estimateCustomObjectivePortfolio(pInt2,trackingError);
timeTE_NoBound = toc(s)

timeTE_NoBound =
8.6102
```

Since the lowest value that can be achieved by the tracking error is 0, you can use that as the lower bound with the `ObjectiveBound` name-value argument. Test the solver's performance when you provide this lower bound. The solver is much faster when you provide one.

```
% Minimize tracking error with a lower bound
s = tic;
wTE_WithBound = estimateCustomObjectivePortfolio(pInt2,trackingError,ObjectiveBound=0);
timeTE_WithBound = toc(s)

timeTE_WithBound =
2.0617
```

Because you are minimizing the objective function, the bound that you provide to `estimateCustomObjectivePortfolio` must be a lower bound.

If you want to maximize the objective, then when you set `ObjectiveSense` to 'maximize', the `ObjectiveBound` is the upper bound of the objective function. Maximize the Sharpe ratio using `ObjectiveBound`.

```
% Maximize Sharpe ratio with upper bound
sharpeRatio = @(x) (p.AssetMean'*x)/sqrt(x'*p.AssetCovar*x);
wSR = estimateCustomObjectivePortfolio(pInt,sharpeRatio,...
    ObjectiveSense='maximize',ObjectiveBound=1);
```

Note that the `ObjectiveBound` name-value argument is ignored in continuous problems.

Using InitialPoint

Use the `InitialPoint` name-value argument to provide an initial point to the `Portfolio` solvers at the beginning of the iterations. You can use this name-value argument in continuous problems with nonconvex objectives to search the feasible region to find different local minima.

Set up a risk parity portfolio with constraints, group the constraints, then estimate the portfolio without and with specifying an initial point. Compare the risk parities of these portfolios.

```
% Risk parity portfolio with constraints
sigma = [5;5;7;10;15;15;15;18]/100;
rho = [ 1    0.8  0.6 -0.2 -0.1 -0.2 -0.2 -0.2;
```

```

    0.8 1 0.4 -0.2 -0.2 -0.1 -0.2 -0.2;
    0.6 0.4 1 0.5 0.3 0.2 0.2 0.3;
    -0.2 -0.2 0.5 1 0.6 0.6 0.5 0.6;
    -0.1 -0.2 0.3 0.6 1 0.9 0.7 0.7;
    -0.2 -0.1 0.2 0.6 0.9 1 0.6 0.7;
    -0.2 -0.2 0.2 0.5 0.7 0.6 1 0.7;
    -0.2 -0.2 0.3 0.6 0.7 0.7 0.7 1];
covariance = corr2cov(sigma,rho);
riskParity = @(x) sum((x.*(covariance*x)/(x'*covariance*x)-1).^2);

% Group the constraints
p = Portfolio(NumAssets=8);
p = setDefaultConstraints(p);
G = [0 0 0 0 1 1 1 1];
p = setGroups(p,G,0.3,[]);

% Solve the problem
w = estimateCustomObjectivePortfolio(p,riskParity);
riskParity(w)

ans =
6.1365

% Use the InitialPoint name-value argument
x0 = [0.7; zeros(3,1); 0.3; zeros(3,1)];
w2 = estimateCustomObjectivePortfolio(p,riskParity,InitialPoint=x0);
riskParity(w2)

ans =
6.1667

```

The objective function is different for different initial portfolios, because the objective function used to compute the risk parity portfolios is nonconvex.

Using different initial points in mixed-integer problems should not return different values of the objective function because the objective function should always be convex. That is, there is only one value for the minimum of the objective function.

Add Tracking Error Constraints to a Custom Objective Problem

This example shows how to use `estimateCustomObjectivePortfolio` to compare the weight assignment of the minimum risk portfolio with a constrained equally weighted portfolio. The weights can be either zero or greater than 15% and must sum to 1. You define the constraints for portfolio problems using functions for the `Portfolio` object and then you specify the objective function as an input to `estimateCustomObjectivePortfolio`. The objective function must be continuous (and preferably smooth).

Create Portfolio Object

Create a `Portfolio` object and set the bounds and budget using `setBounds` and `setBudget`.

```

% Define initial data
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;

```

```

0 0.0119 0.0336 0.1225 ];

% Create Portfolio object
p = Portfolio(AssetMean=m,AssetCovar=C);

% Weight bounds
p1 = setBounds(p,0.15,1,BoundType='conditional');
% Weights must sum to one
p1 = setBudget(p1,1,1);

Add a tracking error constraint using setTrackingError to bound deviations from the max Sharpe
ratio portfolio to 3%.

% Compute max Sharpe ratio portfolio
wSR = estimateMaxSharpeRatio(p1,Method='iterative');

% Set tracking error constraint
p1 = setTrackingError(p1,0.03,wSR);

```

Solve and compare the minimum risk portfolio and the constrained equally weighted portfolio.

```

% Compute max return portfolio
wMin = estimateFrontierLimits(p1,'min');

% Compute equally weighted portfolio by minimizing the
% Herfindahl-Hirshman index
HH = @(w) w'*w;
wHH = estimateCustomObjectivePortfolio(p1,HH);

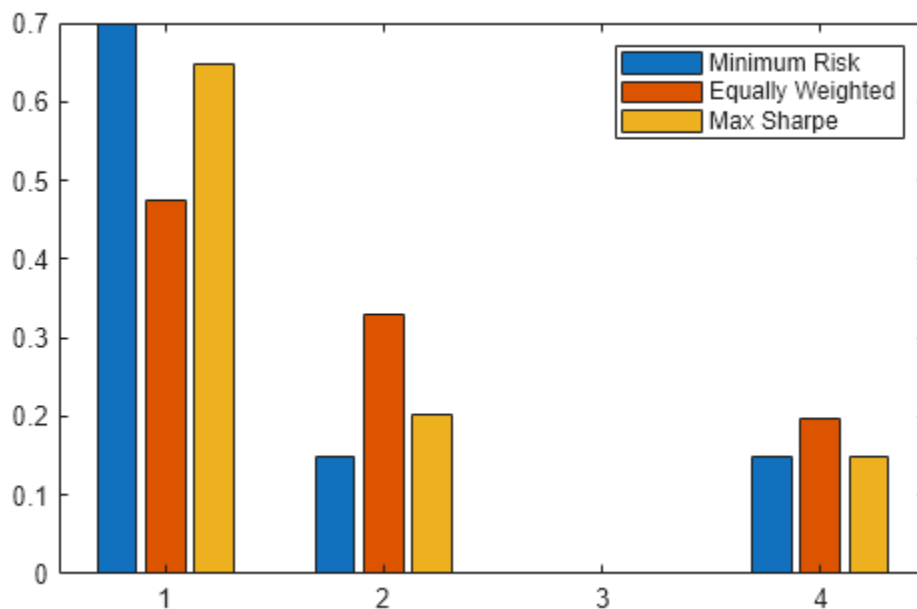
```

Use a bar plot to observe the difference between the portfolios.

```

figure
bar([wMin wHH wSR])
legend('Minimum Risk','Equally Weighted','Max Sharpe')

```



Notice that the weights of the minimum risk portfolio are closer to the weights of the max Sharpe ratio than the weights of the constrained equally weighted portfolio (EWP). This happens because the priority of the constrained EWP is to make the weights as even as possible while still satisfying the constraints. Thus, the weights of the EWP are closer to each other than the rest of the portfolios.

Add Risk Constraint to a Custom Objective Problem

This example shows how to use `estimateCustomObjectivePortfolio` to define a risk-constrained long-only, fully-invested portfolio and then compare the maximum return portfolio against a minimum tracking error portfolio, where the tracking portfolio is the equally-weighted portfolio. You define the constraints for portfolio problems using functions for the `Portfolio` object and then you specify the objective function as an input to `estimateCustomObjectivePortfolio`. The objective function must be continuous (and preferably smooth).

Create Portfolio Object

Create a `Portfolio` object using the default constraints and then define the `targetRisk`.

```
% Define initial data
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

% Create Portfolio object
p = Portfolio(AssetMean=m,AssetCovar=C);

% Set long-only, fully-invested constraints
p2 = setDefaultConstraints(p);

% Define maximum risk of the portfolio
targetRisk = 0.1;
```

To compute the maximum return portfolio subject to a risk constraint, use `estimateFrontierByRisk`.

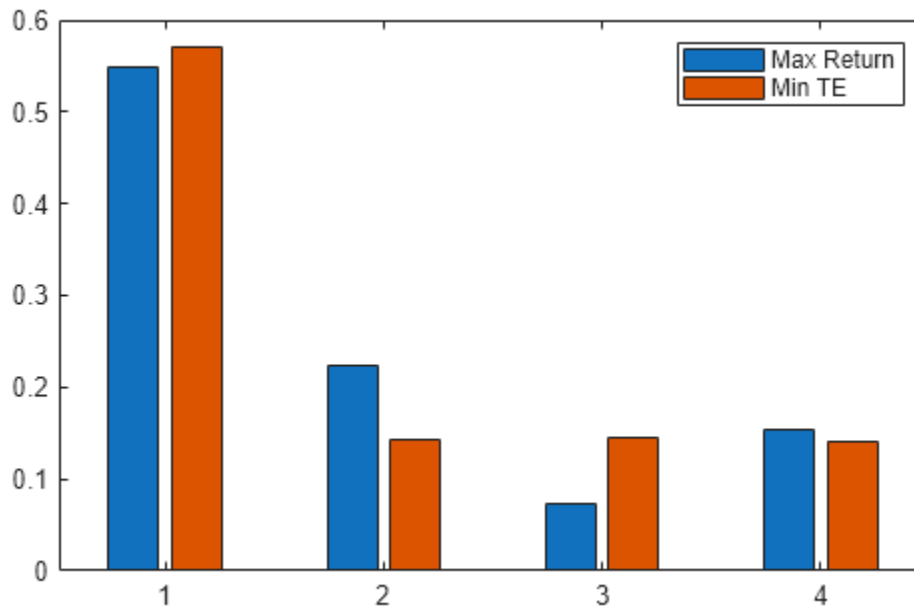
```
% Compute the max return risk constrained portfolio
wMax = estimateFrontierByRisk(p2,targetRisk);
```

Compute the minimum tracking error portfolio subject to a risk constraint using `estimateCustomObjectivePortfolio`, where the tracking portfolio is the equally weighted portfolio.

```
% Compute the min tracking error portfolio with a risk constraint
x0 = 1/4*ones(4,1);
TE = @(w) (w-x0)'*p2.AssetCovar*(w-x0);
wTE = estimateCustomObjectivePortfolio(p2,TE,...
    TargetVolatility=targetRisk);
```

Use a bar plot to observe the difference between the portfolios.

```
figure
bar([wMax wTE])
legend('Max Return','Min TE')
```



Notice that the weights of the minimum tracking portfolio tries to keep the weights of the assets more evenly weighted. However, in order to satisfy the risk constraint, the minimum tracking error portfolio still allocates a larger weight to the less risky asset.

Input Arguments

obj — Object for portfolio optimization

Portfolio object

Object for portfolio optimization, specified using a `Portfolio` object. When using a `Portfolio` object in the custom objective workflow, you do not need to specify both the mean and covariance. However, the custom objective workflow requires that you specify either the mean, the covariance, or the number of assets.

Note If no initial portfolio is specified in `obj.InitPort`, the initial portfolio is assumed to be 0 so that `pbuy = max(0, pwgt)` and `psell = max(0, -pwgt)`. If no tracking portfolio is specified in `obj.TrackingPort`, the tracking portfolio is assumed to be 0.

Data Types: object

fun — Function handle that defines objective function

function handle

Function handle that defines the objective function, specified using a function handle in terms of the portfolio weights.

Note The objective function must be continuous and defined using only the portfolio weights as variables. If the portfolio problem has cardinality constraints and/or conditional bounds using

`setMinMaxNumAssets` or `setBounds`, the objective function must also be convex. For more information, see “Role of Convexity in Portfolio Problems” on page 4-157.

Data Types: `function_handle`

Name-Value Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `pwgt = estimateCustomObjectivePortfolio(p,fun,ObjectiveSense="maximize",TargetReturn=0.05)`

ObjectiveSense — Sense of optimization

`"minimize"` (default) | string with value `"mimimize"` or `"maximize"` | character vector with value `'mimimize'` or `'maximize'`

Sense of the optimization, specified as `ObjectiveSense` and a string or character vector with one of the following values:

- `"minimize"` — The solution minimizes the objective function.
- `"maximize"` — The solution maximizes the objective function.

Data Types: `string` | `char`

TargetReturn — Target value for portfolio return

`[]` (default) | scalar

Target value for portfolio return, specified as `TargetReturn` and a scalar numeric value. For an example, see “Add Gross or Net Return Constraints for Problems with Custom Objectives” on page 15-917.

Data Types: `double`

TargetVolatility — Target value for portfolio standard deviation

`[]` (default) | nonnegative scalar

Target value for portfolio standard deviation, specified as `TargetVolatility` and a nonnegative scalar numeric value. For an example, see “Add Risk Constraint to a Custom Objective Problem” on page 15-924.

Data Types: `double`

ObjectiveBound — User-supplied objective function bound

`[]` (default) | numeric

User-supplied objective function bound, specified as `ObjectiveBound` and a numeric value. `ObjectiveBound` is useful to speed up solvers.

Note If `ObjectiveSense` is `'minimize'`, then `ObjectiveBound` should be a lower bound of the objective function. If `ObjectiveSense` is `'maximize'`, then `ObjectiveBound` should be an upper

bound of the objective function. If `ObjectiveBound` is not specified, a numerical lower bound is computed.

Data Types: double

InitialPoint — Weights allocation to initialize solver

`[]` (default) | vector

Weights allocation to initialize solver, specified as `InitialPoint` and a `NumAssets-by-1` vector. `InitialPoint` is useful for continuous portfolio problems that may have many local minima. By specifying a starting point, you can make the algorithm search different local minima.

Note If `InitialPoint` is not specified, it is set to the long-only, fully-invested, equally-weighted portfolio.

Data Types: double

Output Arguments

pwgt — Optimal weight allocation of portfolio problem

vector

Optimal weight allocation of the portfolio problem, returned as a `NumAssets-by-1` vector.

pbuy — Purchases relative to initial portfolio to achieve optimal weight allocation of portfolio problem

vector

Purchases relative to initial portfolio to achieve the optimal weight allocation of the portfolio problem, returned as a `NumAssets-by-1` vector.

psell — Sales relative to initial portfolio to achieve optimal weight allocation of the portfolio problem

vector

Sales relative to initial portfolio to achieve the optimal weight allocation of the portfolio problem, returned as a `NumAssets` vector.

exitflag — Reason solver stopped

enumeration variable | integer

Reason the solver stopped, returned as an enumeration variable or integer. There are two types of `exitflag` output. If the problem is continuous, the `exitflag` output is an enumeration variable. If the problem is mixed-integer, then the `exitflag` output is an integer.

Version History

Introduced in R2022b

R2024a: Support tracking error constraints

`estimateCustomObjectivePortfolio` supports tracking error constraints when using `setTrackingError` with the `Portfolio` object. For an example, see “Add Tracking Error Constraints to a Custom Objective Problem” on page 15-922.

R2024a: Added Name-Value Arguments for TargetVolatility for risk constraints

`estimateCustomObjectivePortfolio` added a name-value argument for `TargetVolatility` for risk constraints. For an example, see “Add Risk Constraint to a Custom Objective Problem” on page 15-924.

R2023b: Added Name-Value Arguments for ObjectiveBound and InitialPoint

`estimateCustomObjectivePortfolio` added name-value arguments for `ObjectiveBound` and `InitialPoint`.

References

[1] Cornuejols, G. and Reha Tütüncü. *Optimization Methods in Finance*. Cambridge University Press, 2007.

See Also

`estimatePortSharpeRatio` | `estimateFrontier` | `estimateFrontierByReturn` | `estimateFrontierByRisk`

Topics

“Diversify Portfolios Using Custom Objective” on page 4-331
“Portfolio Optimization Using Social Performance Measure” on page 4-264
“Portfolio Optimization Against a Benchmark” on page 4-202
“Solve Tracking Error Portfolio Problems” on page 4-343
“Solve Problem for Minimum Tracking Error with Net Return Constraint” on page 4-349
“Solve Robust Portfolio Maximum Return Problem with Ellipsoidal Uncertainty” on page 4-351
“Risk Parity or Budgeting with Constraints” on page 4-357
“Single Period Goal-Based Wealth Management” on page 4-362
“Adding Constraints to Satisfy UCITS Directive” on page 4-457
“Solver Guidelines for Custom Objective Problems Using Portfolio Objects” on page 4-119
“Portfolio Optimization Theory” on page 4-4
“Role of Convexity in Portfolio Problems” on page 4-157
“Choose MINLP Solvers for Portfolio Problems” on page 4-140
“Troubleshooting `estimateCustomObjectivePortfolio`” on page 4-148
“`solveContinuousCustomObjProb` or `solveMICustomObjProb` Errors” on page 4-146

estimateAssetMoments

Estimate mean and covariance of asset returns from data

Syntax

```
obj = estimateAssetMoments(obj,AssetReturns)
obj = estimateAssetMoments( ___,Name,Value)
```

Description

`obj = estimateAssetMoments(obj,AssetReturns)` estimates mean and covariance of asset returns from data for a `Portfolio` object. For details on the workflow, see “Portfolio Object Workflow” on page 4-18.

`obj = estimateAssetMoments( ___,Name,Value)` estimates mean and covariance of asset returns from data for a `Portfolio` object with additional options for one or more `Name,Value` pair arguments.

Examples

Estimate Mean and Covariance of Asset Returns from Data for a Portfolio Object

To illustrate using the `estimateAssetMoments` function, generate random samples of 120 observations of asset returns for four assets from the mean and covariance of asset returns in the variables `m` and `C` with the `portsim` function. The default behavior `portsim` creates simulated data with estimated mean and covariance identical to the input moments `m` and `C`. In addition to a return series created by the `portsim` function in the variable `X`, a price series is created in the variable `Y`:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;
X = portsim(m', C, 120);
Y = ret2tick(X);
```

Given asset returns and prices in the variables `X` and `Y` from above, the following examples demonstrate equivalent ways to estimate asset moments for the `Portfolio` object. A `Portfolio` object is created in `p` with the moments of asset returns set directly in the `Portfolio` object and a second `Portfolio` object is created in `q` to obtain the mean and covariance of asset returns from asset return data in `X` using the `estimateAssetMoments` function.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
```

```

C = C/12;

X = portsim(m', C, 120);
p = Portfolio('mean',m,'covar',C);
q = Portfolio;
q = estimateAssetMoments(q, X);

[passetmean, passetcovar] = getAssetMoments(p)

```

```
passetmean = 4×1
```

```

0.0042
0.0083
0.0100
0.0150

```

```
passetcovar = 4×4
```

```

0.0005    0.0003    0.0002         0
0.0003    0.0024    0.0017    0.0010
0.0002    0.0017    0.0048    0.0028
         0    0.0010    0.0028    0.0102

```

```
[qassetmean, qassetcovar] = getAssetMoments(q)
```

```
qassetmean = 4×1
```

```

0.0042
0.0083
0.0100
0.0150

```

```
qassetcovar = 4×4
```

```

0.0005    0.0003    0.0002   -0.0000
0.0003    0.0024    0.0017    0.0010
0.0002    0.0017    0.0048    0.0028
-0.0000    0.0010    0.0028    0.0102

```

Notice how either approach yields the same moments. The default behavior of the `estimateAssetMoments` function is to work with asset returns. If, instead, you have asset prices, such as in the variable `Y`, the `estimateAssetMoments` function accepts a parameter name `'DataFormat'` with a corresponding value set to `'prices'` to indicate that the input to the method is in the form of asset prices and not returns (the default parameter value for `'DataFormat'` is `'returns'`). The following example compares direct assignment of moments in the Portfolio object `p` with estimated moments from asset price data in `Y` in the Portfolio object `q`:

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

```

```

X = portsim(m', C, 120);
Y = ret2tick(X);

p = Portfolio('mean',m,'covar',C);

q = Portfolio;
q = estimateAssetMoments(q, Y, 'dataformat', 'prices');

[passetmean, passetcovar] = getAssetMoments(p)

passetmean = 4×1

    0.0042
    0.0083
    0.0100
    0.0150

passetcovar = 4×4

    0.0005    0.0003    0.0002         0
    0.0003    0.0024    0.0017    0.0010
    0.0002    0.0017    0.0048    0.0028
         0    0.0010    0.0028    0.0102

[qassetmean, qassetcovar] = getAssetMoments(q)

qassetmean = 4×1

    0.0042
    0.0083
    0.0100
    0.0150

qassetcovar = 4×4

    0.0005    0.0003    0.0002   -0.0000
    0.0003    0.0024    0.0017    0.0010
    0.0002    0.0017    0.0048    0.0028
   -0.0000    0.0010    0.0028    0.0102

```

Estimate Mean and Covariance of Asset Returns from Timetable Data for a Portfolio Object

To illustrate using the `estimateAssetMoments` function with `AssetReturns` data continued in a `timetable` object, use the `CAPMuniverse.mat` which contains a `timetable` object (`AssetTimeTable`) for returns data.

```

load CAPMuniverse
AssetsTimeTable.Properties;
head(AssetsTimeTable,5)

```

Time	AAPL	AMZN	CSCO	DELL	EBAY	GOOG	HP

03-Jan-2000	0.088805	0.1742	0.008775	-0.002353	0.12829	NaN	0.0
04-Jan-2000	-0.084331	-0.08324	-0.05608	-0.08353	-0.093805	NaN	-0.0
05-Jan-2000	0.014634	-0.14877	-0.003039	0.070984	0.066875	NaN	-0.0
06-Jan-2000	-0.086538	-0.060072	-0.016619	-0.038847	-0.012302	NaN	-0.0
07-Jan-2000	0.047368	0.061013	0.0587	-0.037708	-0.000964	NaN	0.0

Notice that GOOG has missing data (NaN), because it was not listed before Aug 2004. The `estimateAssetMoments` function has a name-value pair argument `'MissingData'` that indicates with a Boolean value whether to use the missing data capabilities of Financial Toolbox™ software. The default value for `'MissingData'` is false which removes all samples with NaN values. If, however, `'MissingData'` is set to true, `estimateAssetMoments` uses the ECM algorithm to estimate asset moments.

```
r = Portfolio;
r = estimateAssetMoments(r,AssetsTimeTable,'dataformat','returns','missingdata',true);
```

In addition, the `estimateAssetMoments` function also extracts asset names or identifiers from a timetable object when the name-value argument `'GetAssetList'` set to true (its default value is false). If the `'GetAssetList'` value is true, the timetable column identifiers are used to set the `AssetList` property of the Portfolio object. To show this, the formation of the Portfolio object `r` is repeated with the `'GetAssetList'` flag set to true.

```
r = estimateAssetMoments(r,AssetsTimeTable,'GetAssetList',true);
disp(r.AssetList')
```

```
{'AAPL' }
{'AMZN' }
{'CSCO' }
{'DELL' }
{'EBAY' }
{'GOOG' }
{'HPQ' }
{'IBM' }
{'INTC' }
{'MSFT' }
{'ORCL' }
{'YHOO' }
{'MARKET'}
{'CASH' }
```

Estimate Mean and Covariance of Asset Returns from Data for a Portfolio Object with Integrality Constraints

Create a Portfolio object for three assets.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
AssetMean = AssetMean/12
```

```
AssetMean = 3×1
```

```
0.0008
```

```
0.0004
0.0011
```

```
AssetCovar = AssetCovar/12
```

```
AssetCovar = 3×3
10-3 ×
```

```
0.2705    0.0192    0.3503
0.0192    0.0416    0.0160
0.3503    0.0160    0.6367
```

```
X = portsim(AssetMean', AssetCovar, 120);
```

```
p = Portfolio('AssetMean',AssetMean, 'AssetCovar', AssetCovar);
p = setDefaultConstraints(p);
```

Use `setBounds` with semi-continuous constraints to set $x_i=0$ or $0.02 \leq x_i \leq 0.5$ for all $i=1,\dots,\text{NumAssets}$.

```
p = setBounds(p, 0.02, 0.5, 'BoundType', 'Conditional', 'NumAssets', 3);
```

When working with a `Portfolio` object, the `setMinMaxNumAssets` function enables you to set up cardinality constraints for a long-only portfolio. This sets the cardinality constraints for the `Portfolio` object, where the total number of allocated assets satisfying the nonzero semi-continuous constraints are between `MinNumAssets` and `MaxNumAssets`. By setting `MinNumAssets=MaxNumAssets=2`, only two of the three assets are invested in the portfolio.

```
p = setMinMaxNumAssets(p, 2, 2);
```

Use `estimateAssetMoments` to estimate mean and covariance of asset returns from data for a `Portfolio` object.

```
p = estimateAssetMoments(p, X);
[passetmean, passetcovar] = getAssetMoments(p)
```

```
passetmean = 3×1
```

```
0.0008
0.0004
0.0011
```

```
passetcovar = 3×3
10-3 ×
```

```
0.2705    0.0192    0.3503
0.0192    0.0416    0.0160
0.3503    0.0160    0.6367
```

The `estimateAssetMoments` function uses the MINLP solver to solve this problem. Use the `setSolverMINLP` function to configure the `SolverType` and options.

```
p.solverOptionsMINLP
```

```
ans = struct with fields:
```

```
MaxIterations: 1000
```

```
AbsoluteGapTolerance: 1.0000e-07
RelativeGapTolerance: 1.0000e-05
NonlinearScalingFactor: 1000
ObjectiveScalingFactor: 1000
    Display: 'off'
    CutGeneration: 'basic'
MaxIterationsInactiveCut: 30
    ActiveCutTolerance: 1.0000e-07
IntMainSolverOptions: [1x1 optim.options.Intlinprog]
NumIterationsEarlyIntegerConvergence: 30
    ExtendedFormulation: 0
    NumInnerCuts: 10
    NumInitialOuterCuts: 10
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a `Portfolio` object. For more information on creating a portfolio object, see

- `Portfolio`

Data Types: object

AssetReturns — Matrix, table, or timetable that contains asset price data that can be converted to asset returns

matrix | table | timetable

Matrix, table, or timetable that contains asset price data that can be converted to asset returns, specified by a `NumSamples`-by-`NumAssets` matrix.

AssetReturns data can be:

- `NumSamples`-by-`NumAssets` matrix.
- Table of `NumSamples` prices or returns at a given periodicity for a collection of `NumAssets` assets
- Timetable object with `NumSamples` observations and `NumAssets` time series

Use the optional `DataFormat` argument to convert `AssetReturns` input data that is asset prices into asset returns. Be careful when using asset price data because portfolio optimization usually requires total returns and not simply price returns.

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `p = estimateAssetMoments(p,Y,'dataformat','prices')`

DataFormat — Flag to convert input data as prices into returns

'Returns' (default) | character vector with values 'Returns' or 'Prices'

Flag to convert input data as prices into returns, specified as the comma-separated pair consisting of 'DataFormat' and a character vector with the values:

- 'Returns' — Data in AssetReturns contains asset total returns.
- 'Prices' — Data in AssetReturns contains asset total return prices.

Data Types: `char`

MissingData — Flag indicating whether to use ECM algorithm or exclude samples with NaN values

false (default) | logical with value true or false

Flag indicating whether to use ECM algorithm or excludes samples with NaN values, specified as the comma-separated pair consisting of 'MissingData' and a logical with a value of true or false.

To handle time series with missing data (indicated with NaN values), the MissingData flag either uses the ECM algorithm to obtain maximum likelihood estimates in the presences of NaN values or excludes samples with NaN values. Since the default is false, it is necessary to specify MissingData as true to use the ECM algorithm.

Acceptable values for MissingData are:

- false — Do not use ECM algorithm to handle NaN values (exclude NaN values).
- true — Use ECM algorithm to handle NaN values.

For more information on the ECM algorithm, see `ecmmle` and “Multivariate Normal Regression” on page 9-2.

Data Types: `logical`

GetAssetList — Flag indicating which asset names to use for asset list

false (default) | logical with value true or false

Flag indicating which asset names to use for the asset list, specified as the comma-separated pair consisting of 'GetAssetList' and a logical with a value of true or false. Acceptable values for GetAssetList are:

- false — Do not extract or create asset names.
- true — Extract or create asset names from a table or timetable object.

If a table or timetable is passed into this function using the AssetReturns argument and the GetAssetList flag is true, the column names from the table or timetable object are used as asset names in `obj.AssetList`.

If a matrix is passed and the GetAssetList flag is true, default asset names are created based on the AbstractPortfolio property `defaultforAssetList`, which is 'Asset'.

If the GetAssetList flag is false, no action occurs, which is the default behavior.

Data Types: `logical`

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a `Portfolio` object. For more information on creating a portfolio object, see

- `Portfolio`

Tips

You can also use dot notation to estimate the mean and covariance of asset returns from data.

```
obj = obj.estimateAssetMoments(AssetReturns);
```

Version History

Introduced in R2011a

R2018a: Support for timetable input for AssetReturns

Behavior changed in R2018a

Using a `fints` object for the `AssetReturns` argument of `estimateAssetMoments` is not recommended. Use `timetable` instead for financial time series. For more information, see “Convert Financial Time Series Objects (`fints`) to Timetables” on page 11-2.

See Also

`Portfolio` | `estimateBounds` | `portsim`

Topics

“Asset Returns and Moments of Asset Returns Using Portfolio Object” on page 4-42

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

“Portfolio Optimization Theory” on page 4-4

estimateBounds

Estimate global lower and upper bounds for set of portfolios

Syntax

```
[glb,gub,isbounded] = estimateBounds(obj)
[glb,gub,isbounded] = estimateBounds(obj,obtainExactBounds)
```

Description

[glb,gub,isbounded] = estimateBounds(obj) estimates global lower and upper bounds for set of portfolios for Portfolio, PortfolioCVaR, or PortfolioMAD objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

Note The estimateBounds function does not consider cardinality or semicontinuous constraints. For more information, see “Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects” on page 4-81.

[glb,gub,isbounded] = estimateBounds(obj,obtainExactBounds) estimates global lower and upper bounds for set of portfolios with an additional option specified for obtainExactBounds.

Examples

Create an Unbounded Portfolio for a Portfolio Object

Create an unbounded portfolio set.

```
p = Portfolio('AInequality', [1 -1; 1 1 ], 'bInequality', 0);
[lb, ub, isbounded] = estimateBounds(p)
```

```
lb = 2×1
```

```
    -Inf
    -Inf
```

```
ub = 2×1
```

```
     0
    Inf
```

```
isbounded = logical
     0
```

The `estimateBounds` function returns (possibly infinite) bounds and sets the `isbounded` flag to `false`. The result shows which assets are unbounded so that you can apply bound constraints as necessary.

Create an Unbounded Portfolio for a PortfolioCvAR Object

Create an unbounded portfolio set.

```
p = PortfolioCvAR('AInequality', [1 -1; 1 1 ], 'bInequality', 0);
[lb, ub, isbounded] = estimateBounds(p)

lb = 2×1

    -Inf
    -Inf

ub = 2×1

     0
    Inf

isbounded = logical
         0
```

The `estimateBounds` function returns (possibly infinite) bounds and sets the `isbounded` flag to `false`. The result shows which assets are unbounded so that you can apply bound constraints as necessary.

Create an Unbounded Portfolio for a PortfolioMAD Object

Create an unbounded portfolio set.

```
p = PortfolioMAD('AInequality', [1 -1; 1 1 ], 'bInequality', 0);
[lb, ub, isbounded] = estimateBounds(p)

lb = 2×1

    -Inf
    -Inf

ub = 2×1

     0
    Inf

isbounded = logical
         0
```

The `estimateBounds` function returns (possibly infinite) bounds and sets the `isbounded` flag to `false`. The result shows which assets are unbounded so that you can apply bound constraints as necessary.

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

obtainExactBounds — Flag to specify whether to solve for all bounds or to accept specified bounds whenever available

`true` (default) | logical

Flag to specify whether to solve for all bounds or to accept specified bounds whenever available, specified as a logical with values of `true` or `false`. If bounds are known, set `obtainExactBounds` to `false` to accept known bounds. The default for `obtainExactBounds` is `true`.

Data Types: `logical`

Output Arguments

glb — Global lower bounds for portfolio set

vector

Global lower bounds for portfolio set, returned as vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

gub — Global upper bounds for portfolio set

vector

Global upper bounds for portfolio set, returned as vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

isbounded — Indicator for whether portfolio set is empty, bounded, or unbounded

logical

Indicator for whether portfolio set is empty (`[]`), bounded (`true`), or unbounded (`false`), returned as a logical.

Note By definition, any portfolio set must be nonempty and bounded:

- If the set is empty, `isbounded = []`.

- If the set is nonempty and unbounded, `isbounded = false`.
 - If the set is nonempty and bounded, `isbounded = true`.
 - If the set is empty, `glb` and `gub` are set to NaN vectors.
-

An `isbounded` value is returned for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

More About

Upper and Lower Bounds

Upper and lower bounds in portfolio optimization are critical constraints that help define the limits of asset allocation, ensuring that the resulting portfolio aligns with the investor's goals for risk tolerance.

Tips

- You can also use dot notation to estimate the global lower and upper bounds for a given set of portfolios.

```
[glb, gub, isbounded] = obj.estimateBounds;
```
- Estimated bounds are accurate in most cases to within $1.0e-8$. If you intend to use these bounds directly in a portfolio object, ensure that if you impose such bound constraints, a lower bound of 0 is probably preferable to a lower bound of, for example, $1.0e-10$ for portfolio weights.

Version History

Introduced in R2011a

See Also

`checkFeasibility`

Topics

“Validate the Portfolio Problem for Portfolio Object” on page 4-94

“Validate the CVaR Portfolio Problem” on page 5-81

“Validate the MAD Portfolio Problem” on page 6-80

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

“Portfolio Optimization Theory” on page 4-4

estimateFrontier

Estimate specified number of optimal portfolios on the efficient frontier

Syntax

```
[pwgt,pbuy,psell] = estimateFrontier(obj)
[pwgt,pbuy,psell] = estimateFrontier(obj,NumPorts)
```

Description

[pwgt,pbuy,psell] = estimateFrontier(obj) estimates the specified number of optimal portfolios on the efficient frontier for Portfolio, PortfolioCVaR, or PortfolioMAD objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

[pwgt,pbuy,psell] = estimateFrontier(obj,NumPorts) estimates the specified number of optimal portfolios on the efficient frontier with an additional option specified for NumPorts.

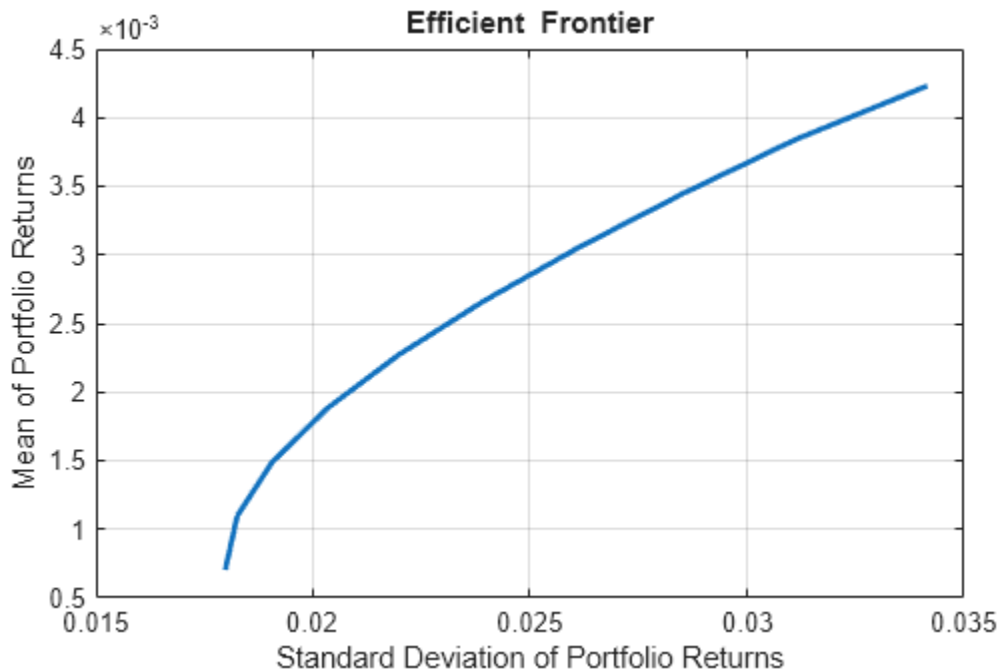
Examples

Create a Portfolio Object and Determine Efficient Portfolios

Create efficient portfolios:

```
load CAPMuniverse

p = Portfolio('AssetList',Assets(1:12));
p = estimateAssetMoments(p, Data(:,1:12),'missingdata',true);
p = setDefaultConstraints(p);
plotFrontier(p);
```



```

pwgt = estimateFrontier(p, 5);

pnames = cell(1,5);
for i = 1:5
    pnames{i} = sprintf('Port%d',i);
end

Blotter = dataset([pwgt],pnames,'obsnames',p.AssetList);

disp(Blotter);

```

	Port1	Port2	Port3	Port4	Port5
AAPL	0.017926	0.058247	0.097816	0.12955	0
AMZN	0	0	0	0	0
CSCO	0	0	0	0	0
DELL	0.0041906	0	0	0	0
EBAY	0	0	0	0	0
GOOG	0.16144	0.35678	0.55228	0.75116	1
HPQ	0.052566	0.032302	0.011186	0	0
IBM	0.46422	0.36045	0.25577	0.11928	0
INTC	0	0	0	0	0
MSFT	0.29966	0.19222	0.082949	0	0
ORCL	0	0	0	0	0
YHOO	0	0	0	0	0

Create a Portfolio Object with BoundType and MaxNumAssets Constraints and Determine Efficient Portfolios

Create a Portfolio object for 12 stocks based on CAPMuniverse.mat.


```
load CAPMuniverse
p0 = Portfolio('AssetList',Assets(1:12));
p0 = estimateAssetMoments(p0, Data(:,1:12), 'missingdata', true);
p0 = setDefaultConstraints(p0);
```

Use `setMinNumAssets` to define a maximum number of 3 assets.

```
p1 = setMinNumAssets(p0, [], 3);
```

Use `setBounds` to define a lower and upper bound and a `BoundType` of 'Conditional'.

```
p1 = setBounds(p1, 0.1, 0.5, 'BoundType', 'Conditional');
pwgt = estimateFrontier(p1, 5);
```

The following table shows that the optimized allocations only have maximum 3 assets invested, and small positions less than 0.1 are avoided.

```
result = table(p0.AssetList', pwgt)
```

```
result=12x2 table
    Var1                pwgt
```

{ 'AAPL' }	0	0	0	0.14232	0
{ 'AMZN' }	0	0	0	0	0
{ 'CSCO' }	0	0	0	0	0
{ 'DELL' }	0	0	0	0	0
{ 'EBAY' }	0	0	0	0	0.5
{ 'GOOG' }	0.16891	0.29534	0.42177	0.5	0.5
{ 'HPQ' }	0	0	0	0	0
{ 'IBM' }	0.49968	0.43657	0.37326	0.35768	0
{ 'INTC' }	0	0	0	0	0
{ 'MSFT' }	0.3314	0.2681	0.20496	0	0
{ 'ORCL' }	0	0	0	0	0
{ 'YHOO' }	0	0	0	0	0

The `estimateFrontier` function uses the MINLP solver to solve this problem. Use the `setSolverMINLP` function to configure the `SolverType` and options.

```
p1.solverTypeMINLP
```

```
ans =
'OuterApproximation'
```

```
p1.solverOptionsMINLP
```

```
ans = struct with fields:
    MaxIterations: 1000
    AbsoluteGapTolerance: 1.0000e-07
    RelativeGapTolerance: 1.0000e-05
    NonlinearScalingFactor: 1000
    ObjectiveScalingFactor: 1000
    Display: 'off'
    CutGeneration: 'basic'
    MaxIterationsInactiveCut: 30
    ActiveCutTolerance: 1.0000e-07
    IntMainSolverOptions: [1x1 optim.options.Intlinprog]
    NumIterationsEarlyIntegerConvergence: 30
```

```

ExtendedFormulation: 0
NumInnerCuts: 10
NumInitialOuterCuts: 10

```

Construct a PortfolioCVaR Object and Determine Efficient Portfolios

Create efficient portfolios:

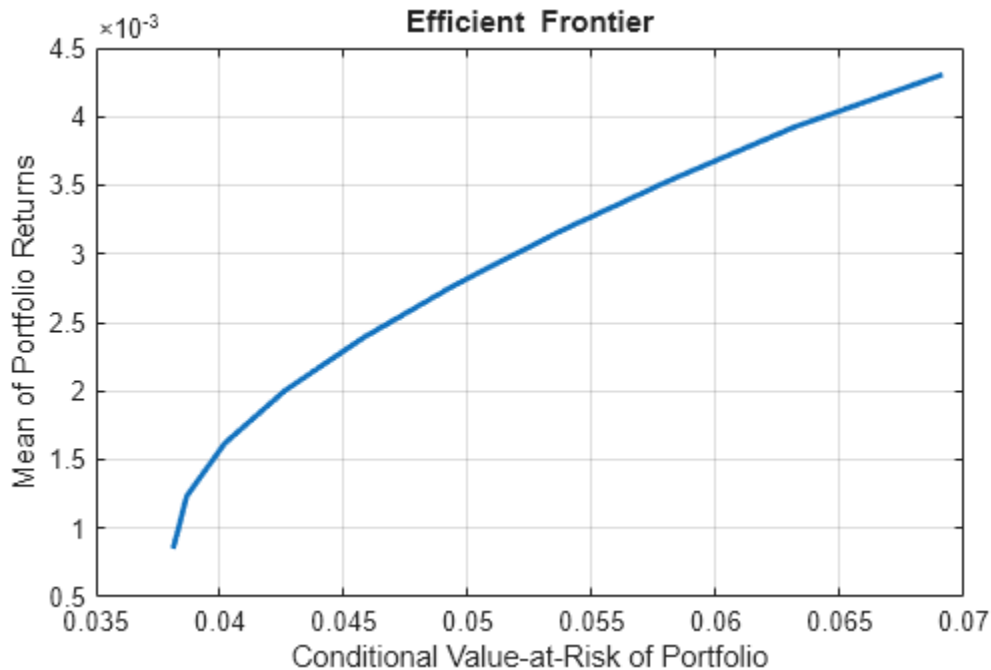
```

load CAPMuniverse

p = PortfolioCVaR('AssetList',Assets(1:12));
p = simulateNormalScenariosByData(p, Data(:,1:12), 20000 , 'missingdata',true);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

plotFrontier(p);

```



```

pwgt = estimateFrontier(p, 5);

pnames = cell(1,5);
for i = 1:5
    pnames{i} = sprintf('Port%d',i);
end

Blotter = dataset([pwgt],pnames,'obsnames',p.AssetList);

disp(Blotter);

```

	Port1	Port2	Port3	Port4	Port5
AAPL	0.010223	0.073393	0.11939	0.13137	0

AMZN	0	0	0	0	0
CSCO	0	0	0	0	0
DELL	0.02301	0	0	0	0
EBAY	0	0	0	0	0
GOOG	0.20389	0.38068	0.56253	0.75919	1
HPQ	0.041396	0.009472	0	0	0
IBM	0.44369	0.36472	0.26247	0.10944	0
INTC	0	0	0	0	0
MSFT	0.27779	0.17174	0.055611	0	0
ORCL	0	0	0	0	0
YHOO	0	0	0	0	0

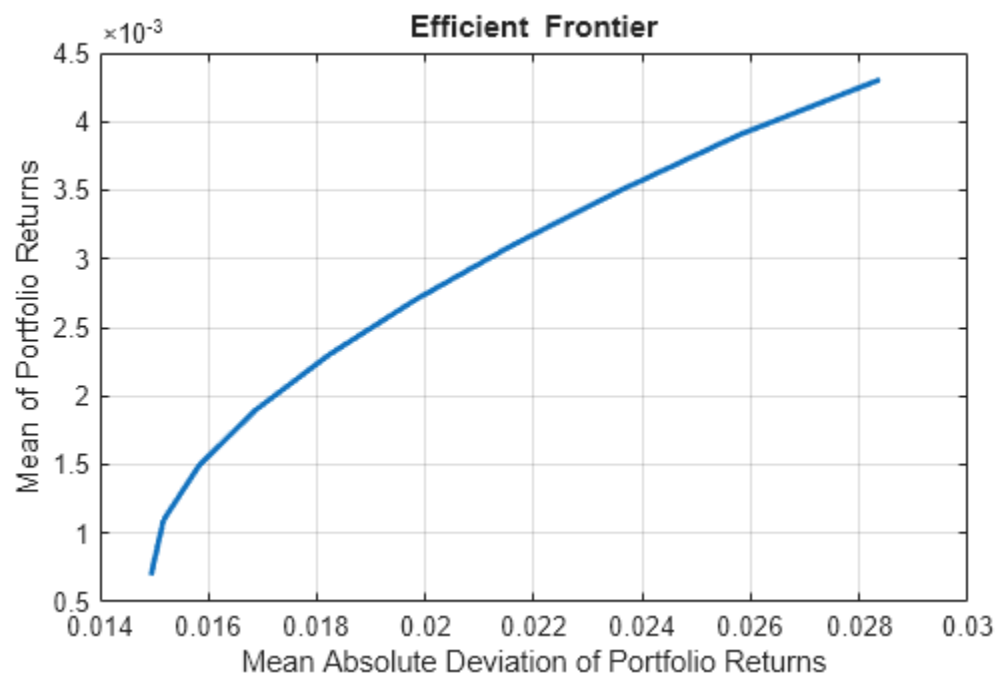
Create a PortfolioMAD Object and Determine Efficient Portfolios

Create efficient portfolios:

```
load CAPMuniverse

p = PortfolioMAD('AssetList',Assets(1:12));
p = simulateNormalScenariosByData(p, Data(:,1:12), 20000 , 'missingdata',true);
p = setDefaultConstraints(p);

plotFrontier(p);
```



```
pwgt = estimateFrontier(p, 5);

pnames = cell(1,5);
for i = 1:5
    pnames{i} = sprintf('Port%d',i);
end
```

```
Blotter = dataset([{pwgt},pnames], 'obsnames', p.AssetList);
disp(Blotter);
```

	Port1	Port2	Port3	Port4	Port5
AAPL	0.029643	0.075874	0.11335	0.13405	0
AMZN	0	0	0	0	0
CSCO	0	0	0	0	0
DELL	0.0086367	0	0	0	0
EBAY	0	0	0	0	0
GOOG	0.16177	0.35217	0.54489	0.74913	1
HPQ	0.056891	0.023419	0	0	0
IBM	0.45916	0.37921	0.29376	0.11682	0
INTC	0	0	0	0	0
MSFT	0.2839	0.16933	0.048005	0	0
ORCL	0	0	0	0	0
YHOO	0	0	0	0	0

Obtain the Default Number of Efficient Portfolios for a Portfolio Object

Obtain the default number of efficient portfolios over the entire range of the efficient frontier.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
```

```
p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontier(p);
disp(pwgt);
```

0.8891	0.7215	0.5540	0.3865	0.2190	0.0515	0	0	0
0.0369	0.1289	0.2209	0.3129	0.4049	0.4969	0.4049	0.2314	0.0579
0.0404	0.0567	0.0730	0.0893	0.1056	0.1219	0.1320	0.1394	0.1468
0.0336	0.0929	0.1521	0.2113	0.2705	0.3297	0.4630	0.6292	0.7953

Obtain Purchases and Sales for Portfolios on the Efficient Frontier for a Portfolio Object

Starting from the initial portfolio, the `estimateFrontier` function returns purchases and sales to get from your initial portfolio to each efficient portfolio on the efficient frontier. Given an initial portfolio in `pwgt0`, you can obtain purchases and sales.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
```

```
p = Portfolio;
```

```

p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateFrontier(p);

```

```
display(pwgt);
```

```
pwgt = 4×10
```

0.8891	0.7215	0.5540	0.3865	0.2190	0.0515	0	0	0
0.0369	0.1289	0.2209	0.3129	0.4049	0.4969	0.4049	0.2314	0.0579
0.0404	0.0567	0.0730	0.0893	0.1056	0.1219	0.1320	0.1394	0.1468
0.0336	0.0929	0.1521	0.2113	0.2705	0.3297	0.4630	0.6292	0.7953

```
display(pbuy);
```

```
pbuy = 4×10
```

0.5891	0.4215	0.2540	0.0865	0	0	0	0	0
0	0	0	0.0129	0.1049	0.1969	0.1049	0	0
0	0	0	0	0	0	0	0	0
0	0	0.0521	0.1113	0.1705	0.2297	0.3630	0.5292	0.6953

```
display(psell);
```

```
psell = 4×10
```

0	0	0	0	0.0810	0.2485	0.3000	0.3000	0.3000
0.2631	0.1711	0.0791	0	0	0	0	0.0686	0.2421
0.1596	0.1433	0.1270	0.1107	0.0944	0.0781	0.0680	0.0606	0.0532
0.0664	0.0071	0	0	0	0	0	0	0

Obtain the Default Number of Efficient Portfolios for a PortfolioCVaR Object

Obtain the default number of efficient portfolios over the entire range of the efficient frontier.

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

```

```
pwgt = estimateFrontier(p);
```

```
disp(pwgt);
```

0.8445	0.6841	0.5148	0.3534	0.1897	0.0303	0	0	0
0.0609	0.1429	0.2302	0.3171	0.3987	0.4742	0.3524	0.1803	0
0.0458	0.0640	0.0945	0.1081	0.1340	0.1590	0.1738	0.1918	0.2211
0.0488	0.1090	0.1606	0.2215	0.2776	0.3365	0.4738	0.6280	0.7789

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Obtain Purchases and Sales for Portfolios on the Efficient Frontier for a PortfolioCVaR Object

Starting from the initial portfolio, the `estimateFrontier` function returns purchases and sales to get from your initial portfolio to each efficient portfolio on the efficient frontier. Given an initial portfolio in `pwgt0`, you can obtain purchases and sales.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);
p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateFrontier(p);
```

```
display(pwgt);
```

```
pwgt = 4×10
```

0.8445	0.6841	0.5148	0.3534	0.1897	0.0303	0	0	0
0.0609	0.1429	0.2302	0.3171	0.3987	0.4742	0.3524	0.1803	0
0.0458	0.0640	0.0945	0.1081	0.1340	0.1590	0.1738	0.1918	0.2211
0.0488	0.1090	0.1606	0.2215	0.2776	0.3365	0.4738	0.6280	0.7789

```
display(pbuy);
```

```
pbuy = 4×10
```

0.5445	0.3841	0.2148	0.0534	0	0	0	0	0
0	0	0	0.0171	0.0987	0.1742	0.0524	0	0

```

0 0 0 0 0 0 0 0 0.0211
0 0.0090 0.0606 0.1215 0.1776 0.2365 0.3738 0.5280 0.6789 0.9

```

display(psell);

psell = 4×10

```

0 0 0 0 0.1103 0.2697 0.3000 0.3000 0.3000 0.3
0.2391 0.1571 0.0698 0 0 0 0 0.1197 0.3000 0.3
0.1542 0.1360 0.1055 0.0919 0.0660 0.0410 0.0262 0.0082 0 0.2
0.0512 0 0 0 0 0 0 0 0 0

```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Obtain the Default Number of Efficient Portfolios for a PortfolioMAD Object

Obtain the default number of efficient portfolios over the entire range of the efficient frontier.

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

pwgt = estimateFrontier(p);

disp(pwgt);

```

```

0.8817 0.7150 0.5488 0.3811 0.2173 0.0503 0 0 0
0.0435 0.1290 0.2130 0.2987 0.3821 0.4668 0.3614 0.1751 0
0.0385 0.0600 0.0826 0.1061 0.1242 0.1477 0.1780 0.2101 0.2267
0.0363 0.0960 0.1556 0.2141 0.2764 0.3352 0.4605 0.6148 0.7733 1.0

```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Obtain Purchases and Sales for Portfolios on the Efficient Frontier for a PortfolioMAD Object

Starting from the initial portfolio, the `estimateFrontier` function returns purchases and sales to get from your initial portfolio to each efficient portfolio on the efficient frontier. Given an initial portfolio in `pwgt0`, you can obtain purchases and sales.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);
p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

pwgt0 = [ 0.3; 0.3; 0.2; 0.1 ];
p = setInitPort(p, pwgt0);
[pwgt, pbuy, psell] = estimateFrontier(p);
```

```
display(pwgt);
```

```
pwgt = 4×10
```

0.8817	0.7150	0.5488	0.3811	0.2173	0.0503	0	0	0	
0.0435	0.1290	0.2130	0.2987	0.3821	0.4668	0.3614	0.1751	0	
0.0385	0.0600	0.0826	0.1061	0.1242	0.1477	0.1780	0.2101	0.2267	
0.0363	0.0960	0.1556	0.2141	0.2764	0.3352	0.4605	0.6148	0.7733	1.0

```
display(pbuy);
```

```
pbuy = 4×10
```

0.5817	0.4150	0.2488	0.0811	0	0	0	0	0	
0	0	0	0	0.0821	0.1668	0.0614	0	0	
0	0	0	0	0	0	0	0.0101	0.0267	
0	0	0.0556	0.1141	0.1764	0.2352	0.3605	0.5148	0.6733	0.9

```
display(psell);
```

```
psell = 4×10
```

0	0	0	0	0.0827	0.2497	0.3000	0.3000	0.3000	0.3
0.2565	0.1710	0.0870	0.0013	0	0	0	0.1249	0.3000	0.3
0.1615	0.1400	0.1174	0.0939	0.0758	0.0523	0.0220	0	0	0.2
0.0637	0.0040	0	0	0	0	0	0	0	

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using Portfolio, PortfolioCvAR, or PortfolioMAD object. For more information on creating a portfolio object, see

- Portfolio
- PortfolioCvAR
- PortfolioMAD

Data Types: object

NumPorts — Number of points to obtain on efficient frontier

value from hidden property defaultNumPorts (default value is 10) (default) | scalar integer

Number of points to obtain on the efficient frontier, specified as a scalar integer.

Note If no value is specified for NumPorts, the default value is obtained from the hidden property defaultNumPorts (default value is 10). If NumPorts = 1, this function returns the portfolio specified by the hidden property defaultFrontierLimit (current default value is 'min').

Data Types: double

Output Arguments

pwgt — Optimal portfolios on efficient frontier with specified number of portfolios spaced equally from minimum to maximum portfolio return

matrix

Optimal portfolios on the efficient frontier with specified number of portfolios spaced equally from minimum to maximum portfolio return, returned as a NumAssets-by-NumPorts matrix. pwgt is returned for a Portfolio, PortfolioCvAR, or PortfolioMAD input object (obj).

pbuy — Purchases relative to initial portfolio for optimal portfolios on efficient frontier

matrix

Purchases relative to an initial portfolio for optimal portfolios on the efficient frontier, returned as NumAssets-by-NumPorts matrix.

Note If no initial portfolio is specified in obj.InitPort, that value is assumed to be 0 such that pbuy = max(0, pwgt) and psell = max(0, -pwgt).

pbuy is returned for a Portfolio, PortfolioCvAR, or PortfolioMAD input object (obj).

psell — Sales relative to initial portfolio for optimal portfolios on efficient frontier

matrix

Sales relative to an initial portfolio for optimal portfolios on the efficient frontier, returned as a NumAssets-by-NumPorts matrix.

Note If no initial portfolio is specified in `obj.InitPort`, that value is assumed to be 0 such that `pbuy = max(0, pwgt)` and `psell = max(0, -pwgt)`.

`psell` is returned for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

More About

Efficient Frontier

The efficient frontier is a key concept in modern portfolio theory (MPT) that represents a set of optimal portfolios offering the highest expected return for a given level of risk, or the lowest risk for a given level of expected return.

The efficient frontier illustrates the tradeoff between risk (often measured as the standard deviation of portfolio returns) and expected return. Investors aim to construct portfolios that lie on this frontier to maximize their returns for a given level of risk. Portfolios that lie on the efficient frontier are considered optimal because they provide the best possible expected return for their level of risk. Portfolios that lie below the frontier are suboptimal, as they do not provide sufficient return for the amount of risk taken. The efficient frontier emphasizes the importance of diversification. By combining different assets with varying returns and risks, investors can reduce the overall risk of the portfolio without sacrificing expected returns.

Tips

- You can also use dot notation to estimate the specified number of optimal portfolios over the entire efficient frontier.

```
[pwgt, pbuy, psell] = obj.estimateFrontier(NumPorts);
```

- When introducing transaction costs and turnover constraints to the `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object, the portfolio optimization objective contains a term with an absolute value. For more information on how Financial Toolbox handles such cases algorithmically, see “References” on page 15-952.

Version History

Introduced in R2011a

References

[1] Cornuejols, G., and R. Tutuncu. *Optimization Methods in Finance*. Cambridge University Press, 2007.

See Also

`estimateFrontierByReturn` | `estimateFrontierByRisk` | `estimateFrontierLimits` | `setBounds` | `setMinMaxNumAssets`

Topics

- "Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object" on page 4-98
- "Estimate Efficient Frontiers for Portfolio Object" on page 4-122
- "Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object" on page 5-85
- "Estimate Efficient Frontiers for PortfolioCVaR Object" on page 5-105
- "Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object" on page 6-84
- "Estimate Efficient Frontiers for PortfolioMAD Object" on page 6-105
- "Portfolio Optimization Examples Using Financial Toolbox" on page 4-161
- "Bond Portfolio Optimization Using Portfolio Object" on page 10-30
- "Portfolio Optimization Theory" on page 4-4
- "Choose MINLP Solvers for Portfolio Problems" on page 4-140

estimateFrontierByReturn

Estimate optimal portfolios with targeted portfolio returns

Syntax

```
[pwgt,pbuy,psell] = estimateFrontierByReturn(obj,TargetReturn)
```

Description

[pwgt,pbuy,psell] = estimateFrontierByReturn(obj,TargetReturn) estimates optimal portfolios with targeted portfolio returns for Portfolio, PortfolioCVaR, or PortfolioMAD objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

Examples

Obtain the Portfolio for Targeted Portfolio Returns for a Portfolio Object

To obtain efficient portfolios that have targeted portfolio returns, the estimateFrontierByReturn function accepts one or more target portfolio returns and obtains efficient portfolios with the specified returns. Assume you have a universe of four assets where you want to obtain efficient portfolios with target portfolio returns of 6%, 9%, and 12%.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontierByReturn(p, [0.06, 0.09, 0.12]);

display(pwgt);

pwgt = 4×3

    0.8772    0.5032    0.1293
    0.0434    0.2488    0.4541
    0.0416    0.0780    0.1143
    0.0378    0.1700    0.3022
```

Use Portfolio Object to Solve for Portfolio Weights to Achieve Min/Max Objective

To obtain efficient portfolios for a `Portfolio` object that has targeted portfolio returns, `estimateFrontierByReturn` accepts one or more target portfolio returns and obtains efficient portfolios with the specified returns.

Load the stock price data and set portfolio constraints so that the total weight of the portfolio must equal 1 and the maximum allocation to for any one stock is 50%, while the minimum is 5%.

```
T = readtable('dowPortfolio.xlsx');
```

	D a t e s	D J	A A	A I G	A X P	B A	C	C A T	D D	D I S	G E	G M	H D	H O N	H P Q	I B M	I N T C	J N J	J P M	K O	M C D	M M M	M O R	M S F T	P F E	P G	T	U T X	V Z	W M T	X O M
1	1/3/2006	1.04e+04	2.8720	6.8410	5.1330	6.8600	4.5260	5.5680	4.0680	2.4180	3.7680	1.7820	3.9714	3.6135	2.8378	8.0137	2.4580	5.9080	3.7880	3.2720	7.5330	5.2730	3.0619	2.2166	2.2138	5.6300	2.2700	5.4900	2.6790	4.9000	5.6640
2	1/4/2006	1.08e+04	2.8890	6.8510	5.1340	6.8600	4.5290	5.5680	4.0680	2.4170	3.7650	1.7820	3.9709	3.6130	2.8388	8.0109	2.4580	5.9060	3.7811	3.2745	7.5308	5.2700	3.0620	2.2188	2.2188	5.6300	2.2700	5.4900	2.6790	4.9000	5.6640
3	1/5/2006	1.08e+04	2.9120	6.8600	5.1573	6.8600	4.5290	5.5680	4.0680	2.4170	3.7650	1.7820	3.9709	3.6130	2.8388	8.0109	2.4580	5.9060	3.7811	3.2745	7.5308	5.2700	3.0620	2.2188	2.2188	5.6300	2.2700	5.4900	2.6790	4.9000	5.6640

	D a t e s	D J l	A A	A I G	A X P	B A	C	C A T	D D	D I S	G E	G M	H D	H O N	H P Q	I B M	I N T C	J N J	J P M	K O	M C D	M M M	M O	M R K	M S F T	P F E	P G	T	U T X	V Z	W M T	X O M
4	1/6/2006	1.09500e+04	290200	688900	517500	674650	484300	405500	245200	339700	196100	389600	365300	298000	829600	252800	601000	379400	332500	754700	529500	310800	262600	231600	562400	232100	545800	280100	445600	575700		
5	1/9/2006	1.193700e+04	293700	683500	530400	674390	403200	247800	336100	392800	112000	396300	301700	814400	257600	603800	355000	396800	758400	531100	315800	262100	231600	566700	232100	545800	280200	445000	574000	754000		
6	1/10/2006	1.184180e+04	294800	691880	528800	673300	405700	220000	254900	333000	207000	407390	313000	821000	251000	609000	386100	397000	753700	530400	312700	263500	237000	564500	231600	542400	280000	445000	574000	759900		
7	1/11/2006	1.180050e+04	290050	696000	529000	683900	408800	388700	258300	336600	206100	414900	308900	821100	251000	599000	385200	397000	752200	533100	336900	263000	236000	566500	233600	544100	280000	458000	573800	783800		

	D a t e s	D J	A A	A I G	A X P	B A	C	C A T	D D	D I S	G E	G M	H D	H O N	H P Q	I B M	I N T C	J N J	J P M	K O	M C D	M M M	M O	M R K	M S F T	P F E	P G	T	U T X	V Z	W M T	X O M
8	1/12/2006	1.0962e+04	276800	690400	526000	6790200	450300	6013000	3802000	2541000	3325000	1976000	4105000	357000	8161000	2496000	537000	395000	387000	3396000	7457000	5323000	3148000	2649000	229000	5602000	2392400	539000	2869000	443000	577000	
9	113/2006	1.0981e+04	2788100	685000	527000	6792400	4503000	6078600	3750000	2537000	3325000	1920000	4030000	3583000	8122000	2478000	537000	3937000	387000	3365000	7489000	5329000	3140000	2659000	2299000	5649000	2399000	537000	2810000	475000	5906000	
10	117/2006	1.0897e+04	2797000	6784000	5203000	66933000	4475000	6075000	3715000	2515000	3320000	1860000	4010000	3560000	8105000	2452000	5374000	3961000	387000	3399000	7495000	5385000	3160000	2643000	2263000	5625000	2393000	5410000	2812000	466000	5961000	
11	118/2006	1.0885e+04	2781000	6742000	5184000	6658000	4414000	6044000	3754000	2497000	3308000	1898000	4020000	3571000	8130000	2472000	5361000	3991000	386000	3379000	7447000	5389000	3099000	26138000	2266000	5654000	2391000	5408000	2783000	488000	5878000	

	D a t e s	D J l	A A	A I G	A X P	B A	C	C A T	D D	D I S	G E	G M	H D	H O N	H P Q	I B M	I N T C	J N J	J P M	K O	M C D	M M M	M O	M R K	M S F T	P F E	P G	T	U T X	V Z	W M T	X O M
1 2	1/1 9/2 0006	1.0 883 01e+04	28.3300	66.9200	51.6200	64.2000	40200	60600	37.6900	26.0000	32.9100	19.3000	39.8300	35.8800	31.7700	81.1400	21.5300	59.6000	37.0300	39.0100	34.3600	73.8100	52.6800	31.2600	26.32700	56.3900	23.1800	54.1000	28.0700	44.9000	44.9000	59.5700
1 3	1/2 0/2 0006	1.0 767 0e+04	27.6700	65.5900	64.7900	41.9500	58.9800	37.3400	25.7900	31.7000	18.9000	38.7000	34.5700	31.2800	79.4500	20.9100	58.2800	36.0700	38.2100	35.0100	72.2180	52.1800	31.2000	25.7300	55.7400	23.0100	53.4600	27.6400	43.7100	37.1000	58.6300	
1 4	1/2 3/2 0006	1.0 868 30e+04	28.0300	65.4600	50.5300	62.3400	42.2300	59.3700	35.2900	21.6300	10.6600	32.9000	38.2900	34.8800	75.0000	20.5200	56.6800	36.2800	38.6000	34.8600	72.6500	52.0900	30.8000	25.71900	55.5500	22.7400	52.9400	27.6900	43.9500	39.2800		
1 5	1/2 4/2 0006	1.0 871 60e+04	28.1300	65.8900	51.9900	62.5200	43.5300	59.0900	37.0900	25.7600	31.3200	19.7300	39.2300	35.1500	79.9100	20.4500	56.91300	36.8900	35.0000	71.2100	51.7400	30.0100	25.76400	55.7700	22.9100	52.7600	24.8600	27.6000	44.1000	39.0500		

	D a t e s	D J	A A	A I G	A X P	B A	C	C A T	D D	D I S	G E	G M	H D	H O N	H P Q	I B M	I N T C	J N J	J P M	K O	M C D	M M M	M O	M R K	M S F T	P F E	P G	T	U T X	V Z	W M T	X O M
1 6	1/25/2006	1.0710e+04	2.8571000	6.4677000	5.1977000	6.5190000	4.2500000	6.0350000	3.2000000	2.5120000	3.1480000	2.2400000	3.8700000	3.4790000	3.1641000	7.0380000	2.0380000	5.3048000	3.9210000	3.7426000	7.0490000	5.1490000	3.1140000	2.5761000	2.3174000	5.6357000	2.3440000	5.4400000	2.8030000	4.6500000	5.8620000	
:																																

Setup a portfolio of ten stocks.

```
symbol = T.Properties.VariableNames(3:12)';
dailyReturn = tick2ret(T{:,3:12});
p = Portfolio('AssetList',symbol,'RiskFreeRate',0.01/252);
```

Estimate the mean and variance of portfolio based on daily returns using estimateAssetMoments.

```
p = estimateAssetMoments(p,dailyReturn);
```

Set the total weight of portfolio to 1 where the LowerBudget = 1 and the UpperBudget = 1.

```
p = setBudget(p,1,1)
```

```
p =
```

Portfolio with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: 3.9683e-05
AssetMean: [10x1 double]
AssetCovar: [10x10 double]
TrackingError: []
TrackingPort: []
Turnover: []
BuyTurnover: []
SellTurnover: []
Name: []
NumAssets: 10
AssetList: {'AA' 'AIG' 'AXP' 'BA' 'C' 'CAT' 'DD' 'DIS' 'GE' 'GM'}
InitPort: []
AInequality: []
bInequality: []
AEquality: []
bEquality: []
LowerBound: []
UpperBound: []
LowerBudget: 1
UpperBudget: 1
GroupMatrix: []
```

```
LowerGroup: []
UpperGroup: []
GroupA: []
GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: []
```

Set the maximum allocation for any one security to 50%, and set the minimum for any one security to 5%.

```
p.LowerBound = repmat(0.05,[10,1]);
p.UpperBound = repmat(0.5,[10,1]);
```

Calculate the portfolio weights for the maximum Sharpe ratio using `estimateMaxSharpeRatio`.

```
w1 = estimateMaxSharpeRatio(p)
```

```
w1 = 10×1
```

```
0.0500
0.0500
0.0500
0.0500
0.0500
0.0500
0.0500
0.0500
0.4357
0.0500
0.1643
```

Estimate moments of portfolio returns using `estimatePortMoments`.

```
[risk1, ret1] = estimatePortMoments(p,w1)
```

```
risk1 =
0.0090
```

```
ret1 =
0.0013
```

Calculate portfolio weights given a target return.

```
TargetReturn = 0.001
```

```
TargetReturn =
1.0000e-03
```

```
TargetReturn = 1.0000e-03
```

```
TargetReturn =
1.0000e-03
```

Use `estimateFrontierByReturn` to estimate optimal portfolios with targeted portfolio returns.

```
w2 = estimateFrontierByReturn(p,TargetReturn)
```

```
w2 = 10×1
```

```
0.0500
0.0500
0.0500
0.0500
0.1668
0.0500
0.0870
0.2949
0.1105
0.0908
```

Estimate moments of portfolio returns using `estimatePortMoments`, where `risk2` is estimates for standard deviations of portfolio returns for each portfolio and `ret2` is estimates for means of portfolio returns for each portfolio.

```
[risk2,ret2] = estimatePortMoments(p,w2)
```

```
risk2 =
0.0076
```

```
ret2 =
1.0000e-03
```

Obtain Portfolios with Targeted Portfolio Returns for a Portfolio Object with BoundType, MinNumAsset, and MaxNumAsset Constraints

When any one, or any combination of the constraints from 'Conditional' BoundType, MinNumAssets, and MaxNumAssets are active, the portfolio problem is formulated as mixed integer programming problem and the MINLP solver is used.

Create a Portfolio object for three assets.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setDefaultConstraints(p);
```

Use `setBounds` with semicontinuous constraints to set $x_i = 0$ or $0.02 \leq x_i \leq 0.5$ for all $i = 1, \dots, \text{NumAssets}$.

```
p = setBounds(p, 0.02, 0.7, 'BoundType', 'Conditional', 'NumAssets', 3);
```

When working with a Portfolio object, the `setMinMaxNumAssets` function enables you to set up the limits on the number of assets invested (as known as cardinality) constraints. This sets the total number of allocated assets satisfying the Bound constraints that are between `MinNumAssets` and `MaxNumAssets`. By setting `MinNumAssets = MaxNumAssets = 2`, only two of the three assets are invested in the portfolio.

```
p = setMinMaxNumAssets(p, 2, 2);
```

Use `estimateFrontierByReturn` to estimate optimal portfolios with targeted portfolio returns.

```
[pwgt, pbuy, psell] = estimateFrontierByReturn(p,[ 0.0072321, 0.0119084 ])
```

```
pwgt = 3×2
```

```
    0    0.5000
0.6922    0
0.3078    0.5000
```

```
pbuy = 3×2
```

```
    0    0.5000
0.6922    0
0.3078    0.5000
```

```
psell = 3×2
```

```
    0    0
    0    0
    0    0
```

The `estimateFrontierByReturn` function uses the MINLP solver to solve this problem. Use the `setSolverMINLP` function to configure the `SolverType` and options.

```
p.solverTypeMINLP
```

```
ans =
'OuterApproximation'
```

```
p.solverOptionsMINLP
```

```
ans = struct with fields:
    MaxIterations: 1000
    AbsoluteGapTolerance: 1.0000e-07
    RelativeGapTolerance: 1.0000e-05
    NonlinearScalingFactor: 1000
    ObjectiveScalingFactor: 1000
    Display: 'off'
    CutGeneration: 'basic'
    MaxIterationsInactiveCut: 30
    ActiveCutTolerance: 1.0000e-07
    IntMainSolverOptions: [1×1 optim.options.Intlinprog]
    NumIterationsEarlyIntegerConvergence: 30
    ExtendedFormulation: 0
    NumInnerCuts: 10
    NumInitialOuterCuts: 10
```

Obtain the Portfolio for Targeted Portfolio Returns for a PortfolioCVaR Object

To obtain efficient portfolios that have targeted portfolio returns, the `estimateFrontierByReturn` function accepts one or more target portfolio returns and obtains efficient portfolios with the

specified returns. Assume you have a universe of four assets where you want to obtain efficient portfolios with target portfolio returns of 7%, 10%, and 13%.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

rng(11);

p = PortfolioCVaR;
p = simulateNormalScenariosByMoments(p, m, C, 2000);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt = estimateFrontierByReturn(p, [0.07 0.10, 0.13]);

display(pwgt);

pwgt = 4×3

    0.7371    0.3072         0
    0.1504    0.3919    0.4396
    0.0286    0.1011    0.1360
    0.0839    0.1999    0.4244
```

The function `rng(seed)` is used to reset the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Obtain the Portfolio for Targeted Portfolio Returns for a PortfolioMAD Object

To obtain efficient portfolios that have targeted portfolio returns, the `estimateFrontierByReturn` function accepts one or more target portfolio returns and obtains efficient portfolios with the specified returns. Assume you have a universe of four assets where you want to obtain efficient portfolios with target portfolio returns of 7%, 10%, and 13%.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

rng(11);

p = PortfolioMAD;
p = simulateNormalScenariosByMoments(p, m, C, 2000);
p = setDefaultConstraints(p);

pwgt = estimateFrontierByReturn(p, [0.07 0.10, 0.13]);

display(pwgt);

pwgt = 4×3
```

```
0.7436    0.3151    0
0.1357    0.3829    0.4425
0.0326    0.0936    0.1319
0.0880    0.2083    0.4255
```

The function `rng(seed)` is used to reset the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

TargetReturn — Target values for portfolio return

vector

Target values for portfolio return, specified as a `NumPorts` vector.

Note `TargetReturn` specifies target returns for portfolios on the efficient frontier. If any `TargetReturn` values are outside the range of returns for efficient portfolios, the `TargetReturn` is replaced with the minimum or maximum efficient portfolio return, depending upon whether the target return is below or above the range of efficient portfolio returns.

Data Types: `double`

Output Arguments

pwgt — Optimal portfolios on efficient frontier with specified target returns

matrix

Optimal portfolios on the efficient frontier with specified target returns from `TargetReturn`, returned as a `NumAssets`-by-`NumPorts` matrix. `pwgt` is returned for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

pbuy — Purchases relative to initial portfolio for optimal portfolios on efficient frontier

matrix

Purchases relative to an initial portfolio for optimal portfolios on the efficient frontier, returned as `NumAssets`-by-`NumPorts` matrix.

Note If no initial portfolio is specified in `obj.InitPort`, that value is assumed to be 0 such that $p_{buy} = \max(0, p_{wgt})$ and $p_{sell} = \max(0, -p_{wgt})$.

`pbuy` is returned for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

psell — Sales relative to initial portfolio for optimal portfolios on efficient frontier matrix

Sales relative to an initial portfolio for optimal portfolios on the efficient frontier, returned as a `NumAssets-by-NumPorts` matrix.

Note If no initial portfolio is specified in `obj.InitPort`, that value is assumed to be 0 such that $p_{buy} = \max(0, p_{wgt})$ and $p_{sell} = \max(0, -p_{wgt})$.

`psell` is returned for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Tips

You can also use dot notation to estimate optimal portfolios with targeted portfolio returns.

```
[pwgt, pbuy, psell] = obj.estimateFrontierByReturn(TargetReturn);
```

Version History

Introduced in R2011a

See Also

`estimateFrontier` | `estimateFrontierByRisk` | `estimateFrontierLimits` | `setBounds` | `setMinMaxNumAssets`

Topics

- “Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98
- “Estimate Efficient Frontiers for Portfolio Object” on page 4-122
- “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
- “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
- “Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
- “Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
- “Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
- “Black-Litterman Portfolio Optimization Using Financial Toolbox” on page 4-222
- “Portfolio Optimization Using Factor Models” on page 4-231
- “Portfolio Optimization Theory” on page 4-4
- “Choose MINLP Solvers for Portfolio Problems” on page 4-140

estimateFrontierByRisk

Estimate optimal portfolios with targeted portfolio risks

Syntax

```
[pwgt,pbuy,psell] = estimateFrontierByRisk(obj,TargetRisk)
```

```
[pwgt,pbuy,psell] = estimateFrontierByRisk( ___,Name,Value)
```

Description

[pwgt,pbuy,psell] = estimateFrontierByRisk(obj,TargetRisk) estimates optimal portfolios with targeted portfolio risks for Portfolio, PortfolioCVaR, or PortfolioMAD objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

[pwgt,pbuy,psell] = estimateFrontierByRisk(___,Name,Value) adds name-optional name-value pair arguments for Portfolio or PortfolioMAD objects.

Examples

Obtain Portfolios with Targeted Portfolio Risks for a Portfolio Object

To obtain efficient portfolios that have targeted portfolio risks, the estimateFrontierByRisk function accepts one or more target portfolio risks and obtains efficient portfolios with the specified risks. Assume you have a universe of four assets where you want to obtain efficient portfolios with target portfolio risks of 12%, 14%, and 16%. This example uses the default 'direct' method to estimate the optimal portfolios with targeted portfolio risks.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontierByRisk(p, [0.12, 0.14, 0.16]);

display(pwgt);

pwgt = 4×3

    0.3984    0.2659    0.1416
    0.3064    0.3791    0.4474
    0.0882    0.1010    0.1131
    0.2071    0.2540    0.2979
```


Use Portfolio Object to Solve for Portfolio Weights to Achieve Min/Max Objective

To obtain efficient portfolios for a `Portfolio` object that has targeted portfolio risks, `estimateFrontierByRisk` accepts one or more target portfolio risks and obtains efficient portfolios with the specified risks.

Load the stock price data and set portfolio constraints so that the total weight of the portfolio must equal 1 and the maximum allocation to for any one stock is 50%, while the minimum is 5%.

```
T = readtable('dowPortfolio.xlsx');
```

	D a t e s	D J I	A A	A I G	A X P	B A	C	C A T	D D	D I S	G E	G M	H D	H O N	H P Q	I B M	I N T C	J N J	J P M	K O	M C D	M M M	M O	M R K	M S F T	P F E	P G	T	U T X	V Z	W M T	X O M
1	1/3/2006	1.084e+04	2.8720	6.8410	5.1330	6.8600	4.5260	5.5860	4.6800	2.4180	3.7680	1.3720	3.6140	3.8150	2.8070	8.0130	2.4570	5.9880	3.7780	3.8970	3.2720	7.5930	5.2730	3.0610	2.2110	2.6380	5.2700	2.7000	5.4940	2.6790	4.7900	5.6640
2	1/4/2006	1.088e+04	2.8890	6.8510	5.1340	6.8600	4.5290	5.5960	4.6800	2.4760	3.8560	1.3900	3.6950	3.9180	2.8090	8.0300	2.4990	5.9900	3.7960	3.9110	3.3010	7.5940	5.2650	3.0680	2.2380	2.6880	5.2700	2.7000	5.4870	2.6810	4.7800	5.6940
3	1/5/2006	1.091e+04	2.8910	6.8600	5.1570	6.8600	4.5980	5.6980	4.6800	2.4910	3.9470	1.4370	3.7970	4.0560	2.8750	8.0500	2.5250	5.9740	3.7670	3.9050	3.3050	7.5850	5.2520	3.1530	2.3400	2.6900	5.2700	2.7000	5.4920	2.6910	4.9000	5.8500

	D a t e s	D J l	A A	A I G	A X P	B A	C	C A T	D D	D I S	G E	G M	H D	H O N	H P Q	I B M	I N T C	J N J	J P M	K O	M C D	M M M	M O	M R K	M S F T	P F E	P G	T	U T X	V Z	W M T	X O M
4	1/6/2006	1.09500e+04	290200	688900	517500	674650	484300	405500	245200	339700	196100	389600	365300	298000	829600	252800	60100	379400	332500	754700	529500	310800	262600	231600	562400	232100	545800	280100	445600	575700		
5	1/9/2006	1.193700e+04	293700	685700	530400	674390	403000	403200	247800	336120	193800	390370	801600	254800	638500	396600	758400	531800	315800	261600	566700	233000	542000	281200	440000	574000	280000	440000	574000			
6	1/10/2006	1.184180e+04	294800	691800	528800	673300	405200	402000	253900	334390	207300	406370	301300	821000	254900	604900	386100	397000	753700	530400	312700	263500	277000	564500	233100	542400	280000	440000	579900			
7	1/11/2006	1.180050e+04	290050	696000	529000	683000	409800	388700	258300	336600	206100	414900	308000	821190	259100	595800	387200	395000	752200	533100	339000	266300	230600	566500	233400	541000	280000	458000	583800			

	D a t e s	D J	A A	A I G	A X P	B A	C	C A T	D D	D I S	G E	G M	H D	H O N	H P Q	I B M	I N T C	J N J	J P M	K O	M C D	M M	M O	M R K	M S F T	P F E	P G	T	U T X	V Z	W M T	X O M
8	1/12/2006	1.076800e+04	27.6800	69.0400	52.6000	67.9020	45.0300	60.1300	38.0200	25.4100	33.2500	19.7600	41.0500	35.7700	30.5700	81.6100	24.9600	59.6300	37.8700	39.5000	33.9600	74.5700	53.2300	31.4800	26.4900	56.0200	23.2400	53.9000	28.6900	44.4300	57.7700	
9	1/13/2006	1.078800e+04	27.8810	68.5400	52.7000	67.9200	44.0300	60.7860	37.8600	25.4700	33.3500	19.2000	40.4300	35.8300	31.4200	81.2200	24.7800	59.6400	37.9300	33.6500	74.3800	53.2900	31.4000	26.4900	56.9900	23.2700	53.9000	28.7500	44.1000	57.7000		
10	1/17/2006	1.078970e+04	27.7900	67.8400	52.0300	66.9330	44.7000	60.8550	37.7500	25.1550	33.2000	18.6100	40.5160	35.2000	31.0500	81.0500	24.5200	59.7400	37.6410	33.7700	73.9900	52.8500	31.1600	26.3400	56.2300	23.2500	53.3100	28.4100	43.6600	59.6100		
11	1/18/2006	1.078810e+04	27.8810	67.4200	51.8400	66.5800	44.1000	60.4400	37.5440	24.9700	33.0800	18.9900	40.4200	35.1100	30.7100	81.3000	24.7200	59.6200	37.9100	34.1600	74.0700	52.8900	30.9900	26.1800	56.5400	23.3100	54.0800	27.8300	43.8800	58.7800		

	D a t e s	D J l	A A	A I G	A X P	B A	C	C A T	D D	D I S	G E	G M	H D	H O N	H P Q	I B M	I N T C	J N J	J P M	K O	M C D	M M M	M O	M R K	M S F T	P F E	P G	T	U T X	V Z	W M T	X O M
1 2	1/1 9/2 0006	1.0 883 01e+04	28.3300	66.9200	51.6400	64.2200	40.6000	37.669000	26.0000	32.910000	19.380000	39.830000	35.870000	31.740000	81.130000	21.530000	59.600000	37.030000	39.430000	73.810000	52.680000	31.260000	26.327000	56.390000	23.180000	54.410000	28.070000	44.900000	44.900000	59.570000		
1 3	1/2 0/2 0006	1.0 76700e+04	27.6700	65.5900	64.7900	41.9500	58.9800	37.340000	25.790000	31.870000	19.000000	38.760000	34.280000	79.450000	20.910000	58.280000	36.070000	38.210000	72.210000	52.180000	31.200000	25.730000	55.740000	23.010000	53.460000	27.640000	43.710000	37.100000	58.630000			
1 4	1/2 3/2 0006	1.0 86380e+04	28.0300	65.4600	55.3000	62.3400	42.3000	57.370000	25.290000	31.630000	20.660000	38.290000	34.780000	79.500000	20.520000	56.680000	36.280000	38.600000	72.650000	52.090000	30.800000	25.719000	55.550000	22.750000	52.940000	27.690000	43.950000	39.280000				
1 5	1/2 4/2 0006	1.0 871600e+04	28.13900	65.89200	51.92000	62.53000	42.50000	57.909000	25.760000	31.320000	21.730000	39.030000	35.250000	79.915000	20.450000	56.903000	38.980000	35.000000	71.210000	51.740000	30.760000	25.641000	55.770000	22.960000	54.860000	27.600000	44.410000	39.050000				

	D a t e s	D J	A A	A I G	A X P	B A	C	C A T	D D	D I S	G E	G M	H D	H O N	H P Q	I B M	I N T C	J N J	J P M	K O	M C D	M M M	M O	M R K	M S F T	P F E	P G	T	U T X	V Z	W M T	X O M
1 6	1/25/2006	1.0710e+04	2.8571000	6.4677000	5.1977000	6.5190000	4.2500000	6.0350000	3.2000000	2.5120000	3.1480000	2.2470000	3.3790000	3.4640000	7.9100000	2.0380000	5.6080000	3.6480000	3.9200000	7.4060000	5.1490000	3.1760000	2.5410000	2.3710000	5.6350000	2.3470000	5.4400000	2.8030000	4.6500000	5.8320000		
:																																

Setup a portfolio of ten stocks.

```
symbol = T.Properties.VariableNames(3:12)';
dailyReturn = tick2ret(T{:,3:12});
p = Portfolio('AssetList',symbol,'RiskFreeRate',0.01/252);
```

Estimate the mean and variance of portfolio based on daily returns using estimateAssetMoments.

```
p = estimateAssetMoments(p,dailyReturn);
```

Set the total weight of portfolio to 1 where the LowerBudget = 1 and the UpperBudget = 1.

```
p = setBudget(p,1,1)
```

p =

Portfolio with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: 3.9683e-05
AssetMean: [10x1 double]
AssetCovar: [10x10 double]
TrackingError: []
TrackingPort: []
Turnover: []
BuyTurnover: []
SellTurnover: []
Name: []
NumAssets: 10
AssetList: {'AA' 'AIG' 'AXP' 'BA' 'C' 'CAT' 'DD' 'DIS' 'GE' 'GM'}
InitPort: []
AInequality: []
bInequality: []
AEquality: []
bEquality: []
LowerBound: []
UpperBound: []
LowerBudget: 1
UpperBudget: 1
GroupMatrix: []
```

```
LowerGroup: []
UpperGroup: []
GroupA: []
GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: []
```

Set the maximum allocation for any one security to 50%, and set the minimum for any one security to 5%.

```
p.LowerBound = repmat(0.05,[10,1]);
p.UpperBound = repmat(0.5,[10,1]);
```

Calculate the portfolio weights for the maximum Sharpe ratio using `estimateMaxSharpeRatio`.

```
w1 = estimateMaxSharpeRatio(p)
```

```
w1 = 10×1
```

```
0.0500
0.0500
0.0500
0.0500
0.0500
0.0500
0.0500
0.0500
0.4357
0.0500
0.1643
```

Estimate moments of portfolio returns using `estimatePortMoments`.

```
[risk1, ret1] = estimatePortMoments(p,w1)
```

```
risk1 =
0.0090
```

```
ret1 =
0.0013
```

Calculate portfolio weights given a target risk.

```
TargetRisk = 0.01
```

```
TargetRisk =
0.0100
```

Use `estimateFrontierByRisk` to estimate optimal portfolios with targeted portfolio risks.

```
w3 = estimateFrontierByRisk(p,TargetRisk)
```

```
w3 = 10×1
```

```

0.0500
0.0500
0.0500
0.0500
0.0500
0.0500
0.0500
0.3419
0.0500
0.2581

```

Estimate moments of portfolio returns using `estimatePortMoments`, where `risk3` is estimates for standard deviations of portfolio returns for each portfolio and `ret3` is estimates for means of portfolio returns for each portfolio.

```

[risk3,ret3] = estimatePortMoments(p,w3)

risk3 =
0.0100

ret3 =
0.0013

```

Obtain Portfolios with Targeted Portfolio Risks for a Portfolio Object with BoundType, MinNumAsset, and MaxNumAsset Constraints

When any one, or any combination of the constraints from 'Conditional' BoundType, MinNumAssets, and MaxNumAssets are active, the portfolio problem is formulated as mixed integer programming problem and the MINLP solver is used.

Create a Portfolio object for three assets.

```

AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setDefaultConstraints(p);

```

Use `setBounds` with semicontinuous constraints to set $x_i = 0$ or $0.02 \leq x_i \leq 0.5$ for all $i = 1, \dots, \text{NumAssets}$.

```
p = setBounds(p, 0.02, 0.7, 'BoundType', 'Conditional', 'NumAssets', 3);
```

When working with a Portfolio object, the `setMinMaxNumAssets` function enables you to set up the limits on the number of assets invested (as known as cardinality) constraints. This sets the total number of allocated assets satisfying the Bound constraints that are between `MinNumAssets` and `MaxNumAssets`. By setting `MinNumAssets = MaxNumAssets = 2`, only two of the three assets are invested in the portfolio.

```
p = setMinMaxNumAssets(p, 2, 2);
```

Use `estimateFrontierByRisk` to estimate optimal portfolios with targeted portfolio risks.

```
[pwgt, pbuy, psell] = estimateFrontierByRisk(p,[0.0324241, 0.0694534 ])
```

```
pwgt = 3×2
```

```
    0.0000    0.5000  
    0.6907    0.0000  
    0.3093    0.5000
```

```
pbuy = 3×2
```

```
    0.0000    0.5000  
    0.6907    0.0000  
    0.3093    0.5000
```

```
psell = 3×2
```

```
    0    0  
    0    0  
    0    0
```

The `estimateFrontierByRisk` function uses the MINLP solver to solve this problem. Use the `setSolverMINLP` function to configure the `SolverType` and options.

```
p.solverTypeMINLP
```

```
ans =  
'OuterApproximation'
```

```
p.solverOptionsMINLP
```

```
ans = struct with fields:  
    MaxIterations: 1000  
    AbsoluteGapTolerance: 1.0000e-07  
    RelativeGapTolerance: 1.0000e-05  
    NonlinearScalingFactor: 1000  
    ObjectiveScalingFactor: 1000  
    Display: 'off'  
    CutGeneration: 'basic'  
    MaxIterationsInactiveCut: 30  
    ActiveCutTolerance: 1.0000e-07  
    IntMainSolverOptions: [1×1 optim.options.Intlinprog]  
    NumIterationsEarlyIntegerConvergence: 30  
    ExtendedFormulation: 0  
    NumInnerCuts: 10  
    NumInitialOuterCuts: 10
```

Obtain Portfolios with Targeted Portfolio Risks for a Portfolio Object Using the Direct Method and Solver Options

To obtain efficient portfolios that have targeted portfolio risks, the `estimateFrontierByRisk` function accepts one or more target portfolio risks and obtains efficient portfolios with the specified risks. Assume you have a universe of four assets where you want to obtain efficient portfolios with

target portfolio risks of 12%, 14%, and 16%. This example uses the default 'direct' method to estimate the optimal portfolios with targeted portfolio risks. The 'direct' method uses `fmincon` to solve the optimization problem that maximizes portfolio return, subject to the target risk as the quadratic nonlinear constraint. `setSolver` specifies the `solverType` and `SolverOptions` for `fmincon`.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);

p = setSolver(p, 'fmincon', 'Display', 'off', 'Algorithm', 'sqp', ...
    'SpecifyObjectiveGradient', true, 'SpecifyConstraintGradient', true, ...
    'ConstraintTolerance', 1.0e-8, 'OptimalityTolerance', 1.0e-8, 'StepTolerance', 1.0e-8);

pwgt = estimateFrontierByRisk(p, [0.12, 0.14, 0.16]);

display(pwgt);

pwgt = 4×3

    0.3984    0.2659    0.1416
    0.3064    0.3791    0.4474
    0.0882    0.1010    0.1131
    0.2071    0.2540    0.2979
```

Obtain Portfolios with Targeted Portfolio Risks for a PortfolioCVaR Object

To obtain efficient portfolios that have targeted portfolio risks, the `estimateFrontierByRisk` function accepts one or more target portfolio risks and obtains efficient portfolios with the specified risks. Assume you have a universe of four assets where you want to obtain efficient portfolios with target portfolio risks of 12%, 20%, and 30%.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

rng(11);

p = PortfolioCVaR;
p = simulateNormalScenariosByMoments(p, m, C, 2000);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt = estimateFrontierByRisk(p, [0.12, 0.20, 0.30]);

display(pwgt);
```

```
pwgt = 4×3

    0.5363    0.1387         0
    0.2655    0.4991    0.3830
    0.0570    0.1239    0.1461
    0.1412    0.2382    0.4709
```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Obtain Portfolios with Targeted Portfolio Risks for a PortfolioMAD Object

To obtain efficient portfolios that have targeted portfolio risks, the `estimateFrontierByRisk` function accepts one or more target portfolio risks and obtains efficient portfolios with the specified risks. Assume you have a universe of four assets where you want to obtain efficient portfolios with target portfolio risks of 12%, 20%, and 25%. This example uses the default 'direct' method to estimate the optimal portfolios with targeted portfolio risks.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

rng(11);

p = PortfolioMAD;
p = simulateNormalScenariosByMoments(p, m, C, 2000);
p = setDefaultConstraints(p);

pwgt = estimateFrontierByRisk(p, [0.12, 0.20, 0.25]);

display(pwgt);

pwgt = 4×3

    0.1611         0         0
    0.4774    0.2137    0.0047
    0.1126    0.1384    0.1200
    0.2488    0.6480    0.8753
```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Obtain Portfolios with Targeted Portfolio Risks for a PortfolioMAD Object Using the Direct Method and Solver Options

To obtain efficient portfolios that have targeted portfolio risks, the `estimateFrontierByRisk` function accepts one or more target portfolio risks and obtains efficient portfolios with the specified risks. Assume you have a universe of four assets where you want to obtain efficient portfolios with

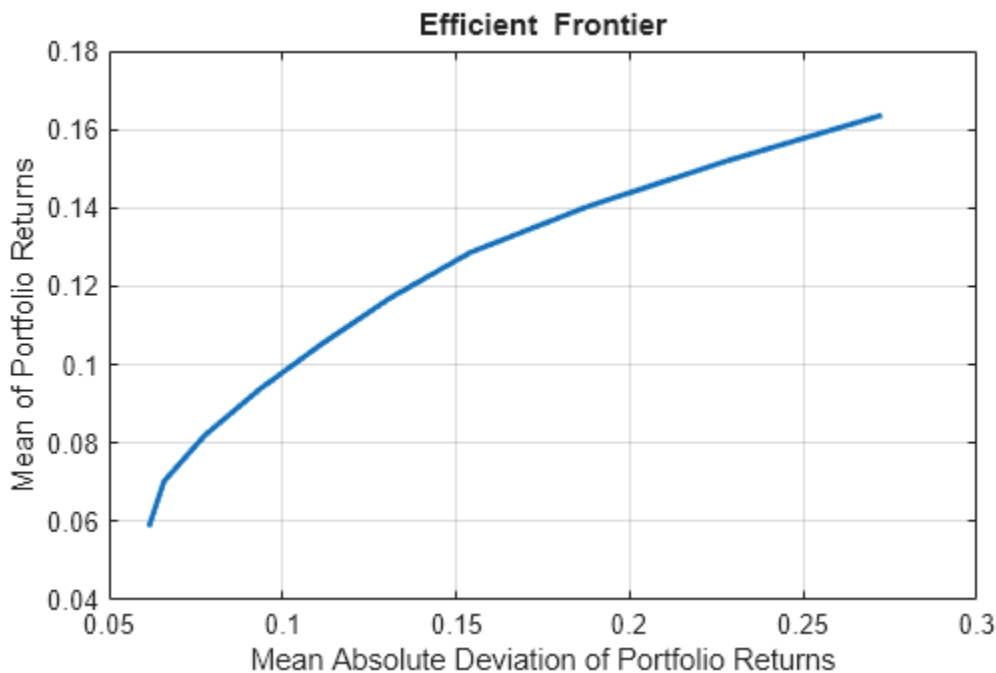
target portfolio risks of 12%, 20%, and 25%. This example uses the default 'direct' method to estimate the optimal portfolios with targeted portfolio risks. The 'direct' method uses `fmincon` to solve the optimization problem that maximizes portfolio return, subject to the target risk as the quadratic nonlinear constraint. `setSolver` specifies the `solverType` and `SolverOptions` for `fmincon`.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

rng(11);

p = PortfolioMAD;
p = simulateNormalScenariosByMoments(p, m, C, 2000);
p = setDefaultConstraints(p);

p = setSolver(p, 'fmincon', 'Display', 'off', 'Algorithm', 'sqp', ...
    'SpecifyObjectiveGradient', true, 'SpecifyConstraintGradient', true, ...
    'ConstraintTolerance', 1.0e-8, 'OptimalityTolerance', 1.0e-8, 'StepTolerance', 1.0e-8);
plotFrontier(p);
```



```
pwgt = estimateFrontierByRisk(p, [0.12 0.20, 0.25]);
```

```
display(pwgt);
```

```
pwgt = 4×3
```

```
0.1613    0.0000    0.0000
0.4777    0.2139    0.0037
0.1118    0.1381    0.1214
```

0.2492 0.6480 0.8749

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Note If no initial portfolio is specified in `obj.InitPort`, it is assumed to be 0 so that `pbuy = max(0,pwgt)` and `psell = max(0,-pwgt)`. For more information on setting an initial portfolio, see `setInitPort`.

Data Types: `object`

TargetRisk — Target values for portfolio risk

vector

Target values for portfolio risk, specified as a `NumPorts` vector.

Note If any `TargetRisk` values are outside the range of risks for efficient portfolios, the target risk is replaced with the minimum or maximum efficient portfolio risk, depending on whether the target risk is below or above the range of efficient portfolio risks.

Data Types: `double`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1,...,NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[pwgt,pbuy,psell] = estimateFrontierByRisk(p,'method','direct')`

Method — Method to estimate frontier by risk for Portfolio or PortfolioMAD objects

'direct' (default) | character vector with value 'direct' or 'iterative'

Method to estimate frontier by risk for `Portfolio` or `PortfolioMAD` objects, specified as the comma-separated pair consisting of 'Method' and a character vector with one of the following values:

- **'direct'** — Construct one optimization problem to maximize the portfolio return with target risk as the nonlinear constraint, and solve it directly using `fmincon`, instead of iteratively exploring the efficient frontier. For an example of using the **'direct'** option, see “Obtain Portfolios with Targeted Portfolio Risks for a Portfolio Object Using the Direct Method and Solver Options” on page 15-974 and “Obtain Portfolios with Targeted Portfolio Risks for a PortfolioMAD Object Using the Direct Method and Solver Options” on page 15-976.
- **'iterative'** — One-dimensional optimization using `fminbnd` to find the portfolio return between min and max return on the frontier that minimizes the difference between the actual risk and target risk. Then the portfolio weights are obtained by solving a frontier by return problem. Consequently, the **'iterative'** method is slower than the **'direct'**.

Data Types: `char`

Output Arguments

pwgt — Optimal portfolios on efficient frontier with specified target risks

matrix

Optimal portfolios on the efficient frontier with specified target returns from `TargetRisk`, returned as a `NumAssets-by-NumPorts` matrix. `pwgt` is returned for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

pbuy — Purchases relative to initial portfolio for optimal portfolios on efficient frontier

matrix

Purchases relative to an initial portfolio for optimal portfolios on the efficient frontier, returned as `NumAssets-by-NumPorts` matrix.

Note If no initial portfolio is specified in `obj.InitPort`, that value is assumed to be 0 such that `pbuy = max(0,pwgt)` and `psell = max(0,-pwgt)`.

`pbuy` is returned for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

psell — Sales relative to initial portfolio for optimal portfolios on efficient frontier

matrix

Sales relative to an initial portfolio for optimal portfolios on the efficient frontier, returned as a `NumAssets-by-NumPorts` matrix.

Note If no initial portfolio is specified in `obj.InitPort`, that value is assumed to be 0 such that `pbuy = max(0,pwgt)` and `psell = max(0,-pwgt)`.

`psell` is returned for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Tips

You can also use dot notation to estimate optimal portfolios with targeted portfolio risks.

```
[pwgt,pbuy,psell] = obj.estimateFrontierByRisk(TargetRisk);
```

or

```
[pwgt,pbuy,psell] = obj.estimateFrontierByRisk(TargetRisk,Name,Value);
```

Version History

Introduced in R2011a

See Also

[estimateFrontier](#) | [estimateFrontierByReturn](#) | [estimateFrontierLimits](#) | [setInitPort](#)
| [rng](#) | [setBounds](#) | [setMinMaxNumAssets](#)

Topics

"Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object" on page 4-98
"Estimate Efficient Frontiers for Portfolio Object" on page 4-122
"Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object" on page 5-85
"Estimate Efficient Frontiers for PortfolioCVaR Object" on page 5-105
"Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object" on page 6-84
"Estimate Efficient Frontiers for PortfolioMAD Object" on page 6-105
"Portfolio Optimization Examples Using Financial Toolbox" on page 4-161
"Black-Litterman Portfolio Optimization Using Financial Toolbox" on page 4-222
"Portfolio Optimization Using Factor Models" on page 4-231
"Add Risk Constraint to a Custom Objective Problem" on page 15-924
"Portfolio Optimization Theory" on page 4-4

estimateFrontierLimits

Estimate optimal portfolios at endpoints of efficient frontier

Syntax

```
[pwgt,pbuy,psell] = estimateFrontierLimits(obj)
[pwgt,pbuy,psell] = estimateFrontierLimits(obj,Choice)
```

Description

[pwgt,pbuy,psell] = estimateFrontierLimits(obj) estimates optimal portfolios at endpoints of efficient frontier for Portfolio, PortfolioCVaR, or PortfolioMAD objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

[pwgt,pbuy,psell] = estimateFrontierLimits(obj,Choice) estimates optimal portfolios at endpoints of efficient frontier with an additional option specified for the Choice argument.

Examples

Obtain Endpoint Portfolios for a Portfolio Object

Given portfolio p, the estimateFrontierLimits function obtains the endpoint portfolios.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
```

```
p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontierLimits(p);
```

```
disp(pwgt);
```

```
0.8891      0
0.0369      0
0.0404      0
0.0336      1.0000
```

Obtain Endpoint Portfolios for a Portfolio Object with BoundType, MinNumAsset, and MaxNumAsset Constraints

When any one, or any combination of the constraints from 'Conditional' BoundType, MinNumAssets, and MaxNumAssets are active, the portfolio problem is formulated as mixed integer programming problem and the MINLP solver is used.

Create a Portfolio object for three assets.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setDefaultConstraints(p);
```

Use setBounds with semicontinuous constraints to set $x_i = 0$ or $0.02 \leq x_i \leq 0.5$ for all $i = 1, \dots, \text{NumAssets}$.

```
p = setBounds(p, 0.02, 0.7, 'BoundType', 'Conditional', 'NumAssets', 3);
```

When working with a Portfolio object, the setMinMaxNumAssets function enables you to set up the limits on the number of assets invested (as known as cardinality) constraints. This sets the total number of allocated assets satisfying the Bound constraints that are between MinNumAssets and MaxNumAssets. By setting MinNumAssets = MaxNumAssets = 2, only two of the three assets are invested in the portfolio.

```
p = setMinMaxNumAssets(p, 2, 2);
```

Use estimateFrontierLimits to estimate the optimal portfolios at endpoints of the efficient frontier.

```
[pwgt, pbuy, psell] = estimateFrontierLimits(p, 'Both')
```

```
pwgt = 3×2
```

```
    0.3000    0.3000
    0.7000         0
         0    0.7000
```

```
pbuy = 3×2
```

```
    0.3000    0.3000
    0.7000         0
         0    0.7000
```

```
psell = 3×2
```

```
         0         0
         0         0
         0         0
```

The estimateFrontierLimits function uses the MINLP solver to solve this problem. Use the setSolverMINLP function to configure the SolverType and options.


```

p.solverTypeMINLP

ans =
'OuterApproximation'

p.solverOptionsMINLP

ans = struct with fields:
    MaxIterations: 1000
    AbsoluteGapTolerance: 1.0000e-07
    RelativeGapTolerance: 1.0000e-05
    NonlinearScalingFactor: 1000
    ObjectiveScalingFactor: 1000
    Display: 'off'
    CutGeneration: 'basic'
    MaxIterationsInactiveCut: 30
    ActiveCutTolerance: 1.0000e-07
    IntMainSolverOptions: [1x1 optim.options.Intlinprog]
    NumIterationsEarlyIntegerConvergence: 30
    ExtendedFormulation: 0
    NumInnerCuts: 10
    NumInitialOuterCuts: 10

```

Obtain Endpoint Portfolios for a PortfolioCVar Object

Given portfolio `p`, the `estimateFrontierLimits` function obtains the endpoint portfolios.

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVar;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt = estimateFrontierLimits(p);

disp(pwgt);

    0.8445         0
    0.0609         0
    0.0458         0
    0.0488    1.0000

```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Obtain Endpoint Portfolios for a PortfolioMAD Object

Given portfolio `p`, the `estimateFrontierLimits` function obtains the endpoint portfolios.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

pwgt = estimateFrontierLimits(p);

disp(pwgt);

    0.8817         0
    0.0435         0
    0.0385         0
    0.0363    1.0000
```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: object

Choice — Indicator for which portfolios to obtain at extreme ends of efficient frontier

[] (default) | character vector with values 'Both', 'Min', 'Max' | string with values "Both", "Min", "Max"

(Optional) Indicator which portfolios to obtain at the extreme ends of the efficient frontier, specified as a character vector with values 'Both' or "Both", 'Min' or "Min", or 'Max' or "Max". The options for a Choice action are:

- [] — Compute both minimum-risk and maximum-return portfolios.
- 'Both' or "Both" — Compute both minimum-risk and maximum-return portfolios.
- 'Min' or "Min" — Compute minimum-risk portfolio only.
- 'Max' or "Max" — Compute maximum-return portfolio only.

Data Types: char | string

Output Arguments

pwgt — Optimal portfolios at endpoints of efficient frontier

matrix

Optimal portfolios at the endpoints of the efficient frontier TargetReturn, returned as a NumAssets-by-NumPorts matrix. pwgt is returned for a Portfolio, PortfolioCVaR, or PortfolioMAD input object (obj).

pbuy — Purchases relative to an initial portfolio for optimal portfolios at endpoints of efficient frontier

matrix

Purchases relative to an initial portfolio for optimal portfolios at the endpoints of the efficient frontier, returned as NumAssets-by-NumPorts matrix.

Note If no initial portfolio is specified in obj.InitPort, that value is assumed to be 0 such that $pbuy = \max(0, pwgt)$ and $psell = \max(0, -pwgt)$.

pbuy is returned for a Portfolio, PortfolioCVaR, or PortfolioMAD input object (obj).

psell — Sales relative to an initial portfolio for optimal portfolios at endpoints of efficient frontier

matrix

Sales relative to an initial portfolio for optimal portfolios on the efficient frontier, returned as a NumAssets-by-NumPorts matrix.

Note If no initial portfolio is specified in obj.InitPort, that value is assumed to be 0 such that $pbuy = \max(0, pwgt)$ and $psell = \max(0, -pwgt)$.

psell is returned for Portfolio, PortfolioCVaR, or PortfolioMAD input object (obj).

More About

Efficient Frontier

The efficient frontier is a key concept in modern portfolio theory (MPT) that represents a set of optimal portfolios offering the highest expected return for a given level of risk, or the lowest risk for a given level of expected return.

The efficient frontier illustrates the tradeoff between risk (often measured as the standard deviation of portfolio returns) and expected return. Investors aim to construct portfolios that lie on this frontier to maximize their returns for a given level of risk. Portfolios that lie on the efficient frontier are considered optimal because they provide the best possible expected return for their level of risk. Portfolios that lie below the frontier are suboptimal, as they do not provide sufficient return for the amount of risk taken. The efficient frontier emphasizes the importance of diversification. By combining different assets with varying returns and risks, investors can reduce the overall risk of the portfolio without sacrificing expected returns.

Tips

You can also use dot notation to estimate the optimal portfolios at the endpoints of the efficient frontier.

```
[pwgt, pbuy, psell] = obj.estimateFrontierLimits(Choice);
```

Version History

Introduced in R2011a

See Also

`estimateFrontier` | `estimateFrontierByReturn` | `estimateFrontierByRisk` | `rng` | `setBounds` | `setMinMaxNumAssets`

Topics

“Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98

“Estimate Efficient Frontiers for Portfolio Object” on page 4-122

“Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85

“Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105

“Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84

“Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

“Portfolio Optimization Theory” on page 4-4

“Choose MINLP Solvers for Portfolio Problems” on page 4-140

estimateMaxSharpeRatio

Estimate efficient portfolio to maximize Sharpe ratio for Portfolio object

Syntax

```
[pwgt,pbuy,psell] = estimateMaxSharpeRatio(obj)
[pwgt,pbuy,psell] = estimateMaxSharpeRatio( ___,Name,Value)
```

Description

[pwgt,pbuy,psell] = estimateMaxSharpeRatio(obj) estimates efficient portfolio to maximize Sharpe ratio for Portfolio object. For details on the workflow, see “Portfolio Object Workflow” on page 4-18.

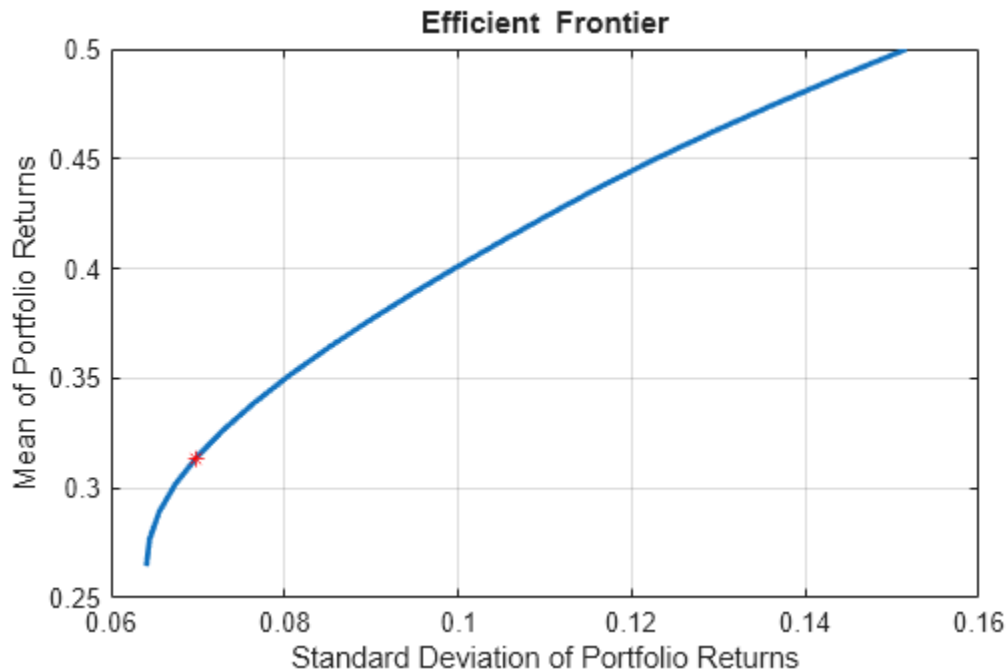
[pwgt,pbuy,psell] = estimateMaxSharpeRatio(___,Name,Value) adds optional name-value pair arguments.

Examples

Estimate Efficient Portfolio that Maximizes the Sharpe Ratio for a Portfolio Object

Estimate the efficient portfolio that maximizes the Sharpe ratio. The estimateMaxSharpeRatio function maximizes the Sharpe ratio among portfolios on the efficient frontier. This example uses the default 'direct' method to estimate the maximum Sharpe ratio. For more information on the 'direct' method, see “Algorithms” on page 15-996.

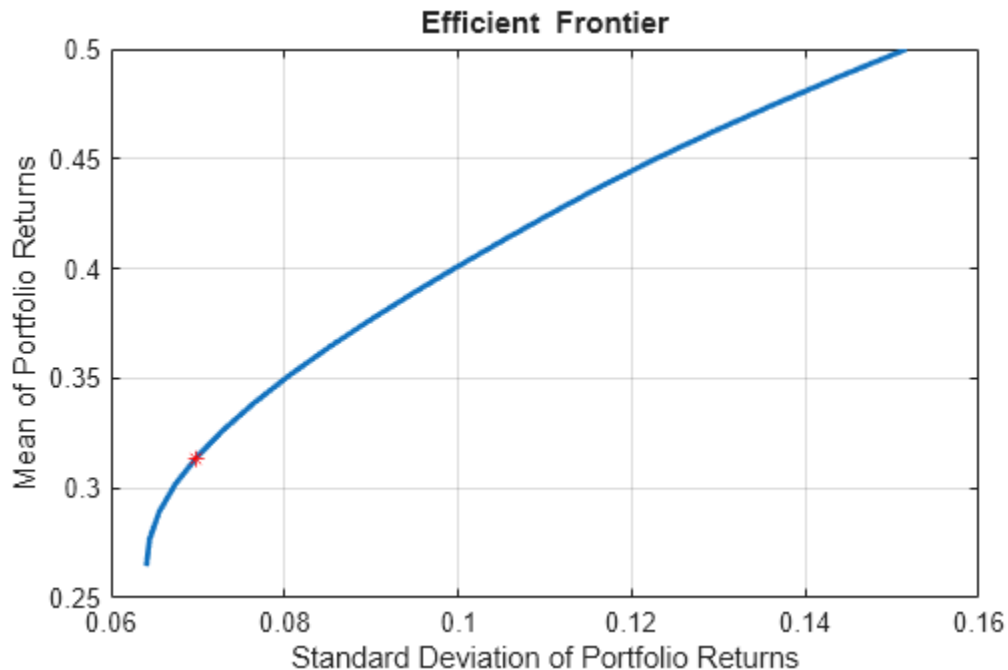
```
p = Portfolio('AssetMean',[0.3, 0.1, 0.5], 'AssetCovar',...
[0.01, -0.010, 0.004; -0.010, 0.040, -0.002; 0.004, -0.002, 0.023]);
p = setDefaultConstraints(p);
plotFrontier(p, 20);
weights = estimateMaxSharpeRatio(p);
[risk, ret] = estimatePortMoments(p, weights);
hold on
plot(risk,ret,'*r');
```



Estimate Efficient Portfolio that Maximizes the Sharpe Ratio for a Portfolio Object Using Solver Options with the Direct Method

Estimate the efficient portfolio that maximizes the Sharpe ratio. The `estimateMaxSharpeRatio` function maximizes the Sharpe ratio among portfolios on the efficient frontier. This example uses the 'direct' method for a `Portfolio` object (`p`) that does not specify a tracking error and only uses linear constraints. The `setSolver` function is used to control the `SolverType` and `SolverOptions`. In this case, the `SolverType` is `quadprog`. For more information on the 'direct' method, see "Algorithms" on page 15-996.

```
p = Portfolio('AssetMean',[0.3, 0.1, 0.5], 'AssetCovar',...
[0.01, -0.010, 0.004; -0.010, 0.040, -0.002; 0.004, -0.002, 0.023]);
p = setDefaultConstraints(p);
plotFrontier(p, 20);
p = setSolver(p,'quadprog','Display','off','ConstraintTolerance',1.0e-8,'OptimalityTolerance',1.0e-8);
weights = estimateMaxSharpeRatio(p);
[risk, ret] = estimatePortMoments(p, weights);
hold on
plot(risk,ret,'*r');
```



Estimate Efficient Portfolio that Maximizes the Sharpe Ratio for a Portfolio Object With Tracking Error Using the Direct Method and Solver Options

Estimate the efficient portfolio that maximizes the Sharpe ratio. The `estimateMaxSharpeRatio` function maximizes the Sharpe ratio among portfolios on the efficient frontier. This example uses the 'direct' method for a `Portfolio` object (`p`) that specifies a tracking error uses nonlinear constraints. The `setSolver` function is used to control the `SolverType` and `SolverOptions`. In this case `fmincon` is the `SolverType`.

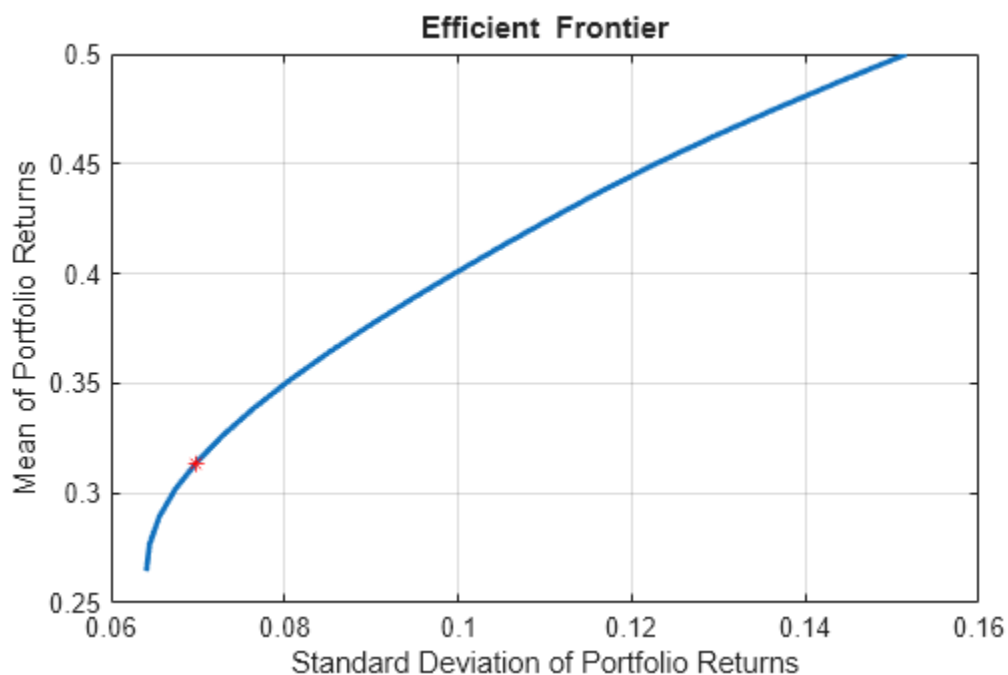
```
p = Portfolio('AssetMean',[0.3, 0.1, 0.5], 'AssetCovar',...
[0.01, -0.010, 0.004; -0.010, 0.040, -0.002; 0.004, -0.002, 0.023],'lb', 0,'budget', 1);
plotFrontier(p, 20);

p = setSolver(p, 'fmincon', 'Display', 'off', 'Algorithm', 'sqp', ...
'SpecifyObjectiveGradient', true, 'SpecifyConstraintGradient', true, ...
'ConstraintTolerance', 1.0e-8, 'OptimalityTolerance', 1.0e-8, 'StepTolerance', 1.0e-8);

weights = estimateMaxSharpeRatio(p);

te = 0.08;
p = setTrackingError(p,te,weights);

[risk, ret] = estimatePortMoments(p,weights);
hold on
plot(risk,ret,'*r');
```



Estimate Efficient Portfolio that Maximizes the Sharpe Ratio for a Portfolio Object With a Risk-Free Asset Using the Direct and Iterative Method

The `estimateMaxSharpeRatio` function maximizes the Sharpe ratio among portfolios on the efficient frontier. In the case of Portfolio with a risk-free asset, there are multiple efficient portfolios that maximize the Sharpe ratio on the capital asset line. Because of the nature of 'direct' and 'iterative' methods, the portfolio weights (`pwgts`) output from each of these methods might be different, but the Sharpe ratio is the same. This example demonstrates the scenario where the `pwgts` are different and the Sharpe ratio is the same.

```
load BlueChipStockMoments

mret = MarketMean;
mrsk = sqrt(MarketVar);
cret = CashMean;
crsk = sqrt(CashVar);

p = Portfolio('AssetList', AssetList, 'RiskFreeRate', CashMean);
p = setAssetMoments(p, AssetMean, AssetCovar);

p = setInitPort(p, 1/p.NumAssets);
[ersk, eret] = estimatePortMoments(p, p.InitPort);

p = setDefaultConstraints(p);
pwgt = estimateFrontier(p, 20);
[prsk, pret] = estimatePortMoments(p, pwgt);
pwgtshpr_fully = estimateMaxSharpeRatio(p, 'Method', 'direct');
[riskshpr_fully, retshpr_fully] = estimatePortMoments(p, pwgtshpr_fully);

q = setBudget(p, 0, 1);
```

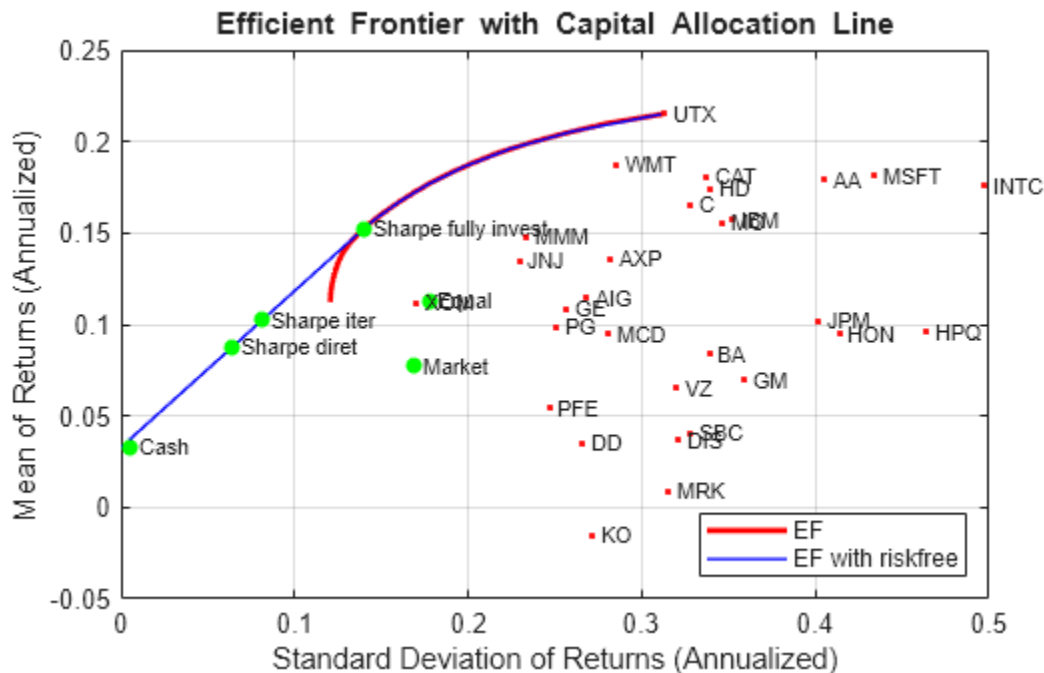


```
qwgt = estimateFrontier(q, 20);
[qrsk, qret] = estimatePortMoments(q, qwgt);
```

Plot the efficient frontier with a tangent line (0 to 1 cash).

```
pwgtshpr_direct = estimateMaxSharpeRatio(q, 'Method', 'direct');
pwgtshpr_iter = estimateMaxSharpeRatio(q, 'Method', 'iterative'); % Default for 'TolX' is 1e-8
[riskshpr_diret, retshpr_diret] = estimatePortMoments(q, pwgtshpr_direct);
[riskshpr_iter, retshpr_iter] = estimatePortMoments(q, pwgtshpr_iter);

clf;
portfolioexamples_plot('Efficient Frontier with Capital Allocation Line', ...
    {'line', prsk, pret, {'EF'}, '-r', 2}, ...
    {'line', qrsk, qret, {'EF with riskfree'}, '-b', 1}, ...
    {'scatter', [mrsk, crsk, ersk, riskshpr_fully, riskshpr_diret, riskshpr_iter], ...
    [mret, cret, eret, retshpr_fully, retshpr_diret, retshpr_iter], {'Market', 'Cash', 'Equal', ...
    {'scatter', sqrt(diag(p.AssetCovar)), p.AssetMean, p.AssetList, '.r'}});
```



When a risk-free asset is not available to the portfolio, or in other words, the portfolio is fully invested, the efficient frontier is curved, corresponding to the red line in the above figure. Therefore, there is a unique (risk, return) point that maximizes the Sharpe ratio, which the 'iterative' and 'direct' methods will both find. If the portfolio is allowed to invest in a risk-free asset, part of the red efficient frontier line is replaced by the capital allocation line, resulting in the efficient frontier of a portfolio with a risk-free investment (blue line). All the (risk, return) points on the straight blue line share the same Sharpe ratio. Also, it is likely that the 'iterative' and 'direct' methods end up with different points, therefore there are different portfolio allocations.

When using the 'iterative' method, you can use an optional 'TolX' name-value argument. TolX is a termination tolerance that is related to the possible return levels of the efficient frontier. If the selected TolX value is large compared to the range of returns, then the accuracy of the solution is poor. TolX should be a number smaller than $0.01 * (\text{maxReturn} - \text{minReturn})$.

```
maxReturn = max(qret); % Max return portfolio
minReturn = min(qret); % Min return portfolio
display(0.01*(maxReturn-minReturn))

1.5193e-04
```

The purpose of increasing the termination tolerance is to speed up the convergence of the 'iterative' algorithm. However, as previously mentioned, the accuracy of the solution will decrease. You can see this in the table that follows.

```
pwgtshpr_iter_largerTol = estimateMaxSharpeRatio(q, 'Method', 'iterative',...
'TolX', 1e-4);
display(table(pwgtshpr_iter, pwgtshpr_iter_largerTol,...
'VariableNames', {'Default TolX = 1e-8', 'TolX = 1e-4'}))

30x2 table
```

Default TolX = 1e-8	TolX = 1e-4
0	0
0	0
0	0
0	0
0	0
0	0
0	0
0	0
0	0
0	0
0	0
0	0
0	0
0.015518	0.015523
0.052455	0.052473
0	0
0	0
0	0
0.090304	0.090335
0.081535	0.081562
0	0
0.026085	0.026094
0	0
0.043451	0.043466
0	0
0.034986	0.034998
0	0
0.12846	0.1285
0.10976	0.10979

In the table, the value of the weights varies slightly with respect to the weights obtained with the default tolerance, as expected.

Estimate Efficient Portfolio that Maximizes the Sharpe Ratio for a Portfolio Object with Semicontinuous and Cardinality Constraints

Create a Portfolio object for three assets.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setDefaultConstraints(p);
```

Use `setBounds` with semicontinuous constraints to set $x_i = 0$ or $0.02 \leq x_i \leq 0.5$ for all $i = 1, \dots, \text{NumAssets}$.

```
p = setBounds(p, 0.02, 0.5, 'BoundType', 'Conditional', 'NumAssets', 3);
```

When working with a Portfolio object, the `setMinNumAssets` function enables you to set up cardinality constraints for a long-only portfolio. This sets the cardinality constraints for the Portfolio object, where the total number of allocated assets satisfying the nonzero semi-continuous constraints are between `MinNumAssets` and `MaxNumAssets`. By setting `MinNumAssets = MaxNumAssets = 2`, only two of the three assets are invested in the portfolio.

```
p = setMinNumAssets(p, 2, 2);
```

Use `estimateMaxSharpeRatio` to estimate efficient portfolio to maximize Sharpe ratio.

```
weights = estimateMaxSharpeRatio(p, 'Method', 'iterative')
```

```
weights = 3×1
```

```
    0
0.5000
0.5000
```

The `estimateMaxSharpeRatio` function uses the MINLP solver to solve this problem. Use the `setSolverMINLP` function to configure the `SolverType` and options.

```
p.solverOptionsMINLP
```

```
ans = struct with fields:
```

```
    MaxIterations: 1000
    AbsoluteGapTolerance: 1.0000e-07
    RelativeGapTolerance: 1.0000e-05
    NonlinearScalingFactor: 1000
    ObjectiveScalingFactor: 1000
    Display: 'off'
    CutGeneration: 'basic'
    MaxIterationsInactiveCut: 30
    ActiveCutTolerance: 1.0000e-07
    IntMainSolverOptions: [1×1 optim.options.Intlinprog]
    NumIterationsEarlyIntegerConvergence: 30
    ExtendedFormulation: 0
    NumInnerCuts: 10
    NumInitialOuterCuts: 10
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a `Portfolio` object.

Note The risk-free rate is obtained from the property `RiskFreeRate` in the `Portfolio` object. If you leave the `RiskFreeRate` unset, it is assumed to be 0. If the max return of portfolio is less than the `RiskFreeRate`, the solution is set as `pwgt` at max return and the resulting Sharpe ratio will be negative.

For more information on creating a portfolio object, see

- `Portfolio`

Data Types: `object`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[pwgt,pbuy,psell] = estimateMaxSharpeRatio(p,'Method','iterative')`

Method — Method to estimate Sharpe ratio

'direct' (default) | character vector with value 'direct' or 'iterative'

Method to estimate Sharpe ratio, specified as the comma-separated pair consisting of 'Method' and a character vector with one of the following values:

- 'direct' — Transform the Sharpe ratio function into a quadratic one and solve one optimization problem directly, instead of iteratively exploring the efficient frontier. The 'direct' option either uses the solver `quadprog` (for a problem with linear constraints) or `fmincon` (for a problem with nonlinear constraints). For an example of using the 'direct' option, see “Estimate Efficient Portfolio that Maximizes the Sharpe Ratio for a Portfolio Object Using Solver Options with the Direct Method” on page 15-988 and “Estimate Efficient Portfolio that Maximizes the Sharpe Ratio for a Portfolio Object With Tracking Error Using the Direct Method and Solver Options” on page 15-989.
- 'iterative' — One-dimensional optimization using `fminbnd` to find the portfolio that maximizes the Sharpe ratio by iteratively exploring the efficient frontier.

Note If you are using `estimateMaxSharpeRatio` with a `Portfolio` object with semicontinuous and cardinality constraints specified by `setBounds` and `setMinMaxNumAssets`, you can only use the 'iterative' method.

Data Types: `char`

TolX — Tolerance on the step size for fminbnd convergence when using 'iterative' method

1e-8 (default) | positive scalar

Tolerance on the step size for fminbnd convergence when using 'iterative' method, specified as the comma-separated pair consisting of 'TolX' and a positive scalar. The step size is related to the return levels achieved on the efficient frontier and tried by fminbnd. If the specified TolX is large compared to the range of returns, the accuracy of the solution is poor. TolX should be a number smaller than $0.01 * (\text{maxReturn} - \text{minReturn})$.

Note If the 'direct' method is selected, TolX is ignored.

Data Types: double

Output Arguments**pwgt — Portfolio on efficient frontier with maximum Sharpe ratio**

vector

Portfolio on the efficient frontier with a maximum Sharpe ratio, returned as a NumAssets vector.

pbuy — Purchases relative to initial portfolio for portfolio on efficient frontier with maximum Sharpe ratio

vector

Purchases relative to an initial portfolio for a portfolio on the efficient frontier with a maximum Sharpe ratio, returned as a NumAssets vector.

pbuy is returned for a Portfolio input object (obj).

psell — Sales relative to initial portfolio for portfolio on efficient frontier with maximum Sharpe ratio

vector

Sales relative to an initial portfolio for a portfolio on the efficient frontier with maximum Sharpe ratio, returned as a NumAssets vector.

psell is returned for a Portfolio input object (obj).

More About**Sharpe Ratio**

The Sharpe ratio is the ratio of the difference between the mean of portfolio returns and the risk-free rate divided by the standard deviation of portfolio returns.

The estimateMaxSharpeRatio function maximizes the Sharpe ratio among portfolios on the efficient frontier.

Tips

You can also use dot notation to estimate an efficient portfolio that maximizes the Sharpe ratio.

```
[pwgt,pbuy,psell] = obj.estimateMaxSharpeRatio;
```

Algorithms

The maximization of the Sharpe ratio is accomplished by either using the 'direct' or 'iterative' method. For the 'direct' method, consider the following scenario. To maximize the Sharpe ratio is to:

$$\text{Maximize } \frac{\mu^T x - r_f}{\sqrt{x^T C x}}, \text{ s.t. } \sum x_i = 1, \quad 0 \leq x_i \leq 1,$$

where μ and C are the mean and covariance matrix, and r_f is the risk-free rate.

If $\mu^T x - r_f \leq 0$ for all x the portfolio that maximizes the Sharpe ratio is the one with maximum return.

If $\mu^T x - r_f > 0$, let $t = \frac{1}{\mu^T x - r_f}$

and $y = tx$ (Cornuejols [1] section 8.2). Then after some substitutions, you can transform the original problem into the following form,

$$\text{Minimize } y^T C y, \text{ s.t. } \sum y_i = t, \quad t > 0, \quad 0 \leq y_i \leq t, \quad \mu^T y - r_f t = 1.$$

Only one optimization needs to be solved, hence the name “direct”. The portfolio weights can be recovered by $x^* = y^* / t^*$.

For the 'iterative' method, the idea is to iteratively explore the portfolios at different return levels on the efficient frontier and locate the one with maximum Sharpe ratio. Therefore, multiple optimization problems are solved during the process, instead of only one in the 'direct' method. Consequently, the 'iterative' method is slow compared to 'direct' method.

Version History

Introduced in R2011b

References

[1] Cornuejols, G. and Reha Tütüncü. *Optimization Methods in Finance*. Cambridge University Press, 2007.

See Also

`estimatePortSharpeRatio` | `estimateFrontier` | `estimateFrontierByReturn` | `estimateFrontierByRisk` | `setBounds` | `setMinMaxNumAssets`

Topics

“Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98

“Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects” on page 4-81

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

“Bond Portfolio Optimization Using Portfolio Object” on page 10-30

“Portfolio Optimization Theory” on page 4-4

“Choose MINLP Solvers for Portfolio Problems” on page 4-140

estimatePortSharpeRatio

Estimate Sharpe ratio of given portfolio weights for Portfolio object

Syntax

```
psharpe = estimatePortSharpeRatio(obj,pwgt)
```

Description

`psharpe = estimatePortSharpeRatio(obj,pwgt)` estimates the Sharpe ratio of given portfolio weights for a `Portfolio` object. For details on the workflow, see “Portfolio Object Workflow” on page 4-18.

Examples

Estimate Sharpe Ratios of the Given Portfolio Weights

This example shows how to find efficient portfolios that satisfy the target returns and then find the Sharpe ratios corresponding to each of the portfolios.

To obtain efficient portfolios that have targeted portfolio returns, the `estimateFrontierByReturn` function accepts one or more target portfolio returns and obtains efficient portfolios with the specified returns. Assume you have a universe of four assets where you want to obtain efficient portfolios with target portfolio returns of 6%, 9%, and 12%. Use `estimatePortSharpeRatio` to obtain the Sharpe ratio for the collection of portfolios (`pwgt`).

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontierByReturn(p, [0.06, 0.09, 0.12]);

display(pwgt);

pwgt = 4×3

    0.8772    0.5032    0.1293
    0.0434    0.2488    0.4541
    0.0416    0.0780    0.1143
    0.0378    0.1700    0.3022
```

`pwgt` is a `NumAssets`-by-`NumPorts` matrix where `NumAssets` is the number of asset in the universe and `NumPorts` is the number of portfolios in the collection of portfolios.

```
psharpe = estimatePortSharpeRatio(p,pwgt)
```



```
psharpe = 3×1

    0.7796
    0.8519
    0.7406
```

psharpe is a NumPorts-by-1 vector, where NumPorts is the number of portfolios in the collection of portfolios.

Estimate Sharpe Ratios of the Given Portfolio Weights Portfolio Object with Integrality Constraints

Create a Portfolio object for three assets.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setDefaultConstraints(p);
```

Use setBounds with semi-continuous constraints to set $x_i=0$ or $0.02 \leq x_i \leq 0.5$ for all $i=1,\dots,\text{NumAssets}$.

```
p = setBounds(p, 0.02, 0.5, 'BoundType', 'Conditional', 'NumAssets', 3);
```

When working with a Portfolio object, the setMinMaxNumAssets function enables you to set up cardinality constraints for a long-only portfolio. This sets the cardinality constraints for the Portfolio object, where the total number of allocated assets satisfying the nonzero semi-continuous constraints are between MinNumAssets and MaxNumAssets. By setting MinNumAssets=MaxNumAssets=2, only two of the three assets are invested in the portfolio.

```
p = setMinMaxNumAssets(p, 2, 2);
```

Use estimatePortSharpeRatio to estimate Sharpe ratio of given portfolio weights for a Portfolio object.

```
pwgt = estimateFrontierByReturn(p, [ 0.0072321, 0.0119084 ]);
psharpe = estimatePortSharpeRatio(p,pwgt)
```

```
psharpe = 2×1

    0.2230
    0.1715
```

The estimatePortSharpeRatio function uses the MINLP solver to solve this problem. Use the setSolverMINLP function to configure the SolverType and options.

```
p.solverOptionsMINLP
```

```
ans = struct with fields:
    MaxIterations: 1000
    AbsoluteGapTolerance: 1.0000e-07
```

```
RelativeGapTolerance: 1.0000e-05
NonlinearScalingFactor: 1000
ObjectiveScalingFactor: 1000
    Display: 'off'
    CutGeneration: 'basic'
MaxIterationsInactiveCut: 30
    ActiveCutTolerance: 1.0000e-07
IntMainSolverOptions: [1x1 optim.options.Intlinprog]
NumIterationsEarlyIntegerConvergence: 30
    ExtendedFormulation: 0
    NumInnerCuts: 10
    NumInitialOuterCuts: 10
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a `Portfolio` object.

Note The risk-free rate is obtained from the property `RiskFreeRate` in the `Portfolio` object. If you leave the `RiskFreeRate` unset, it is assumed to be 0. For more information on creating a portfolio object, see `Portfolio`.

Data Types: `object`

pwgt — Collection of portfolios

matrix

Collection of portfolios, specified as a `NumAssets`-by-`NumPorts` matrix where `NumAssets` is the number of assets in the universe and `NumPorts` is the number of portfolios in the collection of portfolios.

Data Types: `double`

Output Arguments

psharpe — Sharpe ratios of the given portfolios

vector

Sharpe ratios of the given portfolios, returned as a `NumPorts`-by-1 vector.

More About

Sharpe Ratio

The Sharpe ratio is the ratio of the difference between the mean of portfolio returns and the risk-free rate divided by the standard deviation of portfolio returns for each portfolio in `pwgt`.

`estimatePortSharpeRatio` computes the Sharpe ratio with mean and standard deviation (which is the square-root of variance) of portfolio returns.

Tips

You can also use dot notation to estimate the Sharpe ratio of given portfolio weights.

```
psharpe = obj.estimatePortSharpeRatio(pwgt);
```

Version History

Introduced in R2018a

See Also

[estimateMaxSharpeRatio](#) | [estimateFrontier](#) | [estimateFrontierByReturn](#) | [estimateFrontierByRisk](#)

Topics

“Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

“Portfolio Optimization Theory” on page 4-4

estimatePortMoments

Estimate moments of portfolio returns for Portfolio object

Syntax

```
[prsk,pret] = estimatePortMoments(obj,pwgt)
```

Description

[prsk,pret] = estimatePortMoments(obj,pwgt) estimate moments of portfolio returns for a Portfolio object. For details on the workflow, see “Portfolio Object Workflow” on page 4-18.

The estimate of port moments is specific to mean-variance portfolio optimization and computes the mean and standard deviation (which is the square-root of variance) of portfolio returns.

Examples

Identify the Range of Risks and Returns for Efficient Portfolios for a Portfolio Object

Given portfolio p, use the estimatePortMoments function to show the range of risks and returns for efficient portfolios.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontierLimits(p);

[prsk, pret] = estimatePortMoments(p, pwgt);
disp([prsk, pret]);

    0.0769    0.0590
    0.3500    0.1800
```

Identify the Range of Risks and Returns for Efficient Portfolios Portfolio Object with Integrality Constraints

Create a Portfolio object for three assets.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
```

```
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setDefaultConstraints(p);
```

Use `setBounds` with semi-continuous constraints to set $x_i=0$ or $0.02 \leq x_i \leq 0.5$ for all $i=1,\dots,\text{NumAssets}$.

```
p = setBounds(p, 0.02, 0.5, 'BoundType', 'Conditional', 'NumAssets', 3);
```

When working with a `Portfolio` object, the `setMinMaxNumAssets` function enables you to set up cardinality constraints for a long-only portfolio. This sets the cardinality constraints for the `Portfolio` object, where the total number of allocated assets satisfying the nonzero semi-continuous constraints are between `MinNumAssets` and `MaxNumAssets`. By setting `MinNumAssets=MaxNumAssets=2`, only two of the three assets are invested in the portfolio.

```
p = setMinMaxNumAssets(p, 2, 2);
```

Use `estimatePortMoments` to estimate moments of portfolio returns for a `Portfolio` object.

```
pwgt = estimateFrontierLimits(p);
[prsk, pret] = estimatePortMoments(p, pwgt)
```

```
prsk = 2×1
```

```
0.0324
0.0695
```

```
pret = 2×1
```

```
0.0072
0.0119
```

The `estimatePortMoments` function uses the MINLP solver to solve this problem. Use the `setSolverMINLP` function to configure the `SolverType` and options.

```
p.solverOptionsMINLP
```

```
ans = struct with fields:
```

```

    MaxIterations: 1000
    AbsoluteGapTolerance: 1.0000e-07
    RelativeGapTolerance: 1.0000e-05
    NonlinearScalingFactor: 1000
    ObjectiveScalingFactor: 1000
        Display: 'off'
    CutGeneration: 'basic'
MaxIterationsInactiveCut: 30
    ActiveCutTolerance: 1.0000e-07
    IntMainSolverOptions: [1×1 optim.options.Intlinprog]
NumIterationsEarlyIntegerConvergence: 30
    ExtendedFormulation: 0
        NumInnerCuts: 10
    NumInitialOuterCuts: 10
```

Calculate Portfolio Weights For a Target Risk Level

The Portfolio object is able to find an efficient portfolio with respect to a specified target risk.

Load Portfolio

Load a vector of expected returns and a covariance matrix for 30 stocks.

```
load StockStats
```

Create Portfolio Object with Default Constraints

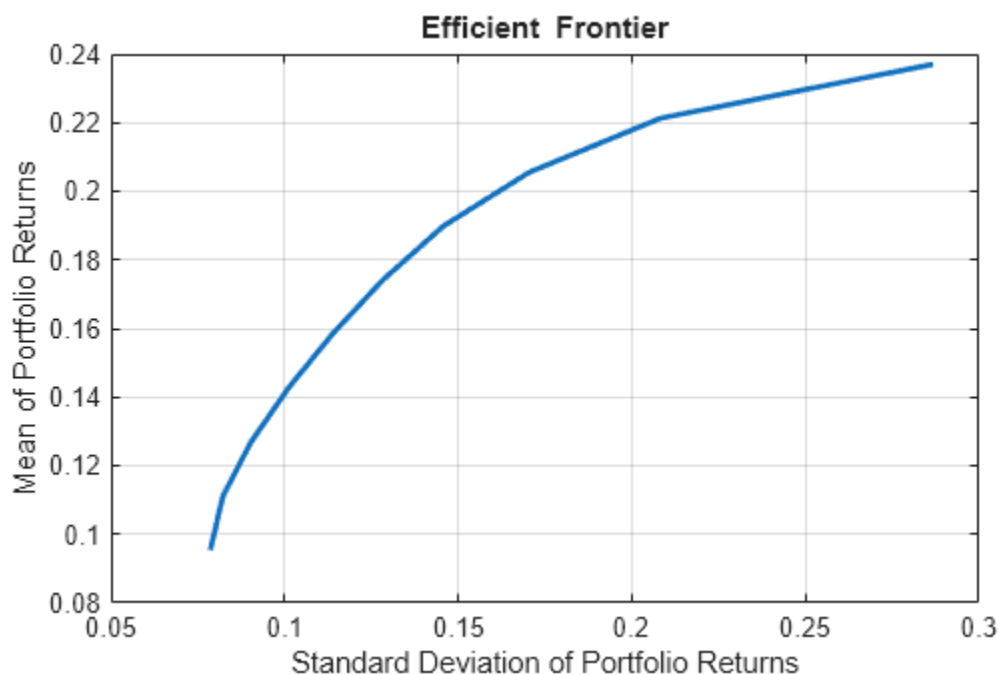
The default constraints for a Portfolio object are that it is a long-only portfolio and that it is 100% invested. Many other constraint types are possible.

```
P = Portfolio('mean',expRet,'covar',expCov);
P = setDefaultConstraints(P);
```

Plot Full Efficient Frontier

Use the plotFrontier function with the Portfolio object to visualize the full frontier. Other functions exist that allow you to probe into particular portfolios along the frontier.

```
P.plotFrontier
```



Capture Upper and Lower Bounds of Portfolio Risks and Returns

It is useful to know what are the upper and lower limits of the portfolio moments along the efficient frontier. This information allows you to determine what are feasible targets. Use the estimateFrontierLimits function with the Portfolio object to identify the weights at these extremes. Then you can use the estimatePortMoments function Portfolio object to find the limiting moments.

```

P_Weights1 = P.estimateFrontierLimits;
[P_risklimits, P_returnlimits] = P.estimatePortMoments(P_Weights1)

P_risklimits = 2×1

    0.0786
    0.2868

P_returnlimits = 2×1

    0.0954
    0.2370

```

Estimate Efficient Portfolio with Target Return

Select a target return for the portfolio somewhere in the feasible region. You can estimate its makeup with `estimateFrontierByRisk` and then confirm its moments using `estimatePortMoments`.

```

targetReturn = 0.15;

P_Weights2 = P.estimateFrontierByReturn(targetReturn);
[P_risk2, P_return2] = P.estimatePortMoments(P_Weights2)

P_risk2 =
    0.1068

P_return2 =
    0.1500

```

The return matches the `targetReturn` value and the risk is in agreement with the efficient frontier plot.

Estimate Efficient Portfolio with Target Risk

The `Portfolio` object is able to find an efficient portfolio with a specified target risk.

```

targetRisk = 0.2;

P_Weights3 = P.estimateFrontierByRisk(targetRisk);
[P_risk3, P_return3] = P.estimatePortMoments(P_Weights3)

P_risk3 =
    0.2000

P_return3 =
    0.2182

```

Another useful feature of the `estimateFrontierByReturn` and `estimateFrontierByRisk` functions is what happens if you specify an infeasible (too high or too low) target. In this case, these functions provide a warning message to suggest the best solution.

Input Arguments

obj — Object for portfolio
object

Object for portfolio, specified using a `Portfolio` object. For more information on creating a portfolio object, see `Portfolio`.

Data Types: `object`

pwgt — Collection of portfolios
`matrix`

Collection of portfolios, specified as a `NumAssets`-by-`NumPorts` matrix where `NumAssets` is the number of assets in the universe and `NumPorts` is the number of portfolios in the collection of portfolios.

Data Types: `double`

Output Arguments

prsk — Estimates for standard deviations of portfolio returns for each portfolio in pwgt
`vector`

Estimates for standard deviations of portfolio returns for each portfolio in `pwgt`, returned as a `NumPorts` vector.

`prsk` is returned for a `Portfolio` input object (`obj`). If the asset returns provided to the `Portfolio` object are daily returns, then the output (`prsk`) for the standard deviation of the portfolio(s) is also in daily terms.

pret — Estimates for means of portfolio returns for each portfolio in pwgt
`vector`

Estimates for means of portfolio returns for each portfolio in `pwgt`, returned as a `NumPorts` vector.

`pret` is returned for a `Portfolio` input object (`obj`). If the asset returns provided to the `Portfolio` object are daily returns, then the output (`pret`) for returns is also in daily terms.

More About

Moments of Portfolio Returns

The moments of portfolio returns are statistical measures that provide insights into the characteristics of the return distribution of an investment portfolio.

The moments of portfolio returns provide information on:

- The variance and standard deviation to help investors gauge the risk associated with their portfolio.
- A mean return that provides a benchmark for evaluating portfolio performance.

Tips

You can also use dot notation to estimate the moments of portfolio returns.

```
[prsk, pret] = obj.estimatePortMoments(pwgt);
```


Version History

Introduced in R2011a

See Also

`estimatePortReturn` | `estimatePortRisk`

Topics

"Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object" on page 4-98

"Portfolio Optimization Examples Using Financial Toolbox" on page 4-161

"Portfolio Optimization Theory" on page 4-4

estimatePortReturn

Estimate mean of portfolio returns

Syntax

```
pret = estimatePortReturn(obj,pwgt)
```

Description

`pret = estimatePortReturn(obj,pwgt)` estimates the mean of portfolio returns (as the proxy for portfolio return) for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

Examples

Estimate the Mean of Portfolio Returns for a Portfolio Object

Given portfolio `p`, use the `estimatePortReturn` function to estimate the mean of portfolio returns.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
```

```
p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontierLimits(p);
pret = estimatePortReturn(p, pwgt);
disp(pret)
```

```
0.0590
0.1800
```

Estimate the Mean of Portfolio Returns for a Portfolio Object with Integrality Constraints

Create a `Portfolio` object for three assets.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setDefaultConstraints(p);
```

Use `setBounds` with semi-continuous constraints to set $x_i=0$ or $0.02 \leq x_i \leq 0.5$ for all $i=1,\dots,\text{NumAssets}$.

```
p = setBounds(p, 0.02, 0.5, 'BoundType', 'Conditional', 'NumAssets', 3);
```

When working with a `Portfolio` object, the `setMinMaxNumAssets` function enables you to set up cardinality constraints for a long-only portfolio. This sets the cardinality constraints for the `Portfolio` object, where the total number of allocated assets satisfying the nonzero semi-continuous constraints are between `MinNumAssets` and `MaxNumAssets`. By setting `MinNumAssets=MaxNumAssets=2`, only two of the three assets are invested in the portfolio.

```
p = setMinMaxNumAssets(p, 2, 2);
```

Use `estimatePortReturn` to estimate the mean of portfolio returns for a `Portfolio` object.

```
pwgt = estimateFrontierLimits(p);
pret = estimatePortReturn(p, pwgt)
```

```
pret = 2×1
```

```
0.0072
```

```
0.0119
```

The `estimatePortReturn` function uses the MINLP solver to solve this problem. Use the `setSolverMINLP` function to configure the `SolverType` and options.

```
p.solverOptionsMINLP
```

```
ans = struct with fields:
```

```

    MaxIterations: 1000
  AbsoluteGapTolerance: 1.0000e-07
  RelativeGapTolerance: 1.0000e-05
  NonlinearScalingFactor: 1000
  ObjectiveScalingFactor: 1000
           Display: 'off'
    CutGeneration: 'basic'
MaxIterationsInactiveCut: 30
   ActiveCutTolerance: 1.0000e-07
  IntMainSolverOptions: [1×1 optim.options.Intlinprog]
NumIterationsEarlyIntegerConvergence: 30
    ExtendedFormulation: 0
           NumInnerCuts: 10
    NumInitialOuterCuts: 10
```

Estimate the Mean of Portfolio Returns for a PortfolioCVaR Object

Given portfolio `p`, use the `estimatePortReturn` function to estimate the mean of portfolio returns.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
```

```
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt = estimateFrontierLimits(p);
pret = estimatePortReturn(p, pwgt);
disp(pret)

    0.0050
    0.0154
```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Estimate the Mean of Portfolio Returns for a PortfolioMAD Object

Given portfolio `p`, use the `estimatePortReturn` function to estimate the mean of portfolio returns.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

pwgt = estimateFrontierLimits(p);
pret = estimatePortReturn(p, pwgt);
disp(pret)

    0.0048
    0.0154
```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVar`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVar`
- `PortfolioMAD`

Data Types: object

pwgt — Collection of portfolios

matrix

Collection of portfolios, specified as a `NumAssets`-by-`NumPorts` matrix, where `NumAssets` is the number of assets in the universe and `NumPorts` is the number of portfolios in the collection of portfolios.

Data Types: double

Output Arguments

pret — Estimates for means of portfolio returns for each portfolio in pwgt

vector

Estimates for means of portfolio returns for each portfolio in `pwgt`, returned as a `NumPorts` vector.

`pret` is returned for a `Portfolio`, `PortfolioCVar`, or `PortfolioMAD` input object (`obj`).

Note Depending on whether costs have been set, the portfolio return is either gross or net portfolio returns. For information on setting costs, see `setCosts`.

Tips

You can also use dot notation to estimate the mean of portfolio returns (as the proxy for portfolio return).

```
pret = obj.estimatePortReturn(pwgt);
```

Version History

Introduced in R2011a

See Also

`estimatePortRisk` | `estimateFrontierByReturn` | `estimateFrontierByRisk` | `rng`

Topics

“Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98

“Estimate Efficient Frontiers for Portfolio Object” on page 4-122
“Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
“Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
“Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84
“Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105
“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
“Portfolio Optimization Theory” on page 4-4

estimatePortRisk

Estimate portfolio risk according to risk proxy associated with corresponding object

Syntax

```
prsk = estimatePortRisk(obj,pwgt)
```

Description

`prsk = estimatePortRisk(obj,pwgt)` estimates portfolio risk according to the risk proxy associated with the corresponding object (`obj`) for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

Examples

Standard Deviation of Portfolio Returns as the Proxy for Portfolio Risk for a Portfolio Object

Given portfolio `p`, use the `estimatePortRisk` function to show the standard deviation of portfolio returns for each portfolio in `pwgt`.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = setAssetMoments(p, m, C);
p = setDefaultConstraints(p);
pwgt = estimateFrontierLimits(p);
prsk = estimatePortRisk(p, pwgt);
disp(prsk)

    0.0769
    0.3500
```

Standard Deviation of Portfolio Returns as the Proxy for Portfolio Risk for a Portfolio Object with Integrality Constraints

Create a `Portfolio` object for three assets.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
```

```
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setDefaultConstraints(p);
```

Use `setBounds` with semi-continuous constraints to set $x_i=0$ or $0.02 \leq x_i \leq 0.5$ for all $i=1,\dots,\text{NumAssets}$.

```
p = setBounds(p, 0.02, 0.5, 'BoundType', 'Conditional', 'NumAssets', 3);
```

When working with a `Portfolio` object, the `setMinMaxNumAssets` function enables you to set up cardinality constraints for a long-only portfolio. This sets the cardinality constraints for the `Portfolio` object, where the total number of allocated assets satisfying the nonzero semi-continuous constraints are between `MinNumAssets` and `MaxNumAssets`. By setting `MinNumAssets=MaxNumAssets=2`, only two of the three assets are invested in the portfolio.

```
p = setMinMaxNumAssets(p, 2, 2);
```

Use `estimatePortRisk` to estimate the portfolio risk according to a risk proxy associated with a `Portfolio` object.

```
pwgt = estimateFrontierLimits(p);
prsk = estimatePortRisk(p, pwgt)
```

```
prsk = 2×1
```

```
0.0324
0.0695
```

The `estimatePortRisk` function uses the MINLP solver to solve this problem. Use the `setSolverMINLP` function to configure the `SolverType` and options.

```
p.solverOptionsMINLP
```

```
ans = struct with fields:
```

```

    MaxIterations: 1000
  AbsoluteGapTolerance: 1.0000e-07
  RelativeGapTolerance: 1.0000e-05
  NonlinearScalingFactor: 1000
  ObjectiveScalingFactor: 1000
           Display: 'off'
    CutGeneration: 'basic'
  MaxIterationsInactiveCut: 30
    ActiveCutTolerance: 1.0000e-07
  IntMainSolverOptions: [1×1 optim.options.Intlinprog]
  NumIterationsEarlyIntegerConvergence: 30
    ExtendedFormulation: 0
           NumInnerCuts: 10
    NumInitialOuterCuts: 10
```

Conditional Value-at-Risk of Portfolio Returns as the Proxy for Portfolio Risk for a PortfolioCVaR Object

Given a portfolio `pwgt`, use the `estimatePortRisk` function to show the conditional value-at-risk (CVaR) of portfolio returns for each portfolio.


```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt = estimateFrontierLimits(p);
prsk = estimatePortRisk(p, pwgt);
disp(prsk)

    0.0407
    0.1911

```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Mean-Absolute Deviation Returns as the Proxy for Portfolio Risk for a PortfolioMAD Object

Given a portfolio `pwgt`, use the `estimatePortRisk` function to show the mean-absolute deviation of portfolio returns for each portfolio.

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

pwgt = estimateFrontierLimits(p);
prsk = estimatePortRisk(p, pwgt);
disp(prsk)

    0.0177
    0.0809

```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

pwgt — Collection of portfolios

matrix

Collection of portfolios, specified as a `NumAssets`-by-`NumPorts` matrix, where `NumAssets` is the number of assets in the universe and `NumPorts` is the number of portfolios in the collection of portfolios.

Data Types: `double`

Output Arguments

prsk — Estimates for portfolio risk according to the risk proxy associated with the corresponding object (obj) for each portfolio in pwgt

vector

Estimates for portfolio risk according to the risk proxy associated with the corresponding object (`obj`) for each portfolio in `pwgt`, returned as a `NumPorts` vector.

`prsk` is returned for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Tips

You can also use dot notation to estimate portfolio risk according to the risk proxy associated with the corresponding object (`obj`).

```
prsk = obj.estimatePortRisk(pwgt);
```

Version History

Introduced in R2011a

See Also

`estimatePortStd` | `estimatePortVaR` | `estimateFrontierByReturn` | `estimateFrontierByRisk` | `rng`

Topics

- "Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object" on page 4-98
- "Estimate Efficient Frontiers for Portfolio Object" on page 4-122
- "Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object" on page 5-85
- "Estimate Efficient Frontiers for PortfolioCVaR Object" on page 5-105
- "Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object" on page 6-84
- "Estimate Efficient Frontiers for PortfolioMAD Object" on page 6-105
- "Portfolio Optimization Examples Using Financial Toolbox" on page 4-161
- "Portfolio Optimization Theory" on page 4-4

estimatePortStd

Estimate standard deviation of portfolio returns

Syntax

```
pstd = estimatePortStd(obj,pwgt)
```

Description

`pstd = estimatePortStd(obj,pwgt)` estimate standard deviation of portfolio returns for `PortfolioCVaR` or `PortfolioMAD` objects. For details on the workflows, see “PortfolioCVaR Object Workflow” on page 5-16 and “PortfolioMAD Object Workflow” on page 6-16.

Examples

Estimate Standard Deviations for Portfolio Returns for a PortfolioCVaR Object

Given a portfolio `pwgt`, use the `estimatePortStd` function to show the standard deviation of portfolio returns.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt = estimateFrontierLimits(p);

pstd = estimatePortStd(p, pwgt);
disp(pstd)

    0.0223
    0.1010
```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Estimate Standard Deviations for Portfolio Returns for a PortfolioMAD Object

Given a portfolio `pwgt`, use the `estimatePortStd` function to show the standard deviation of portfolio returns.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

pwgt = estimateFrontierLimits(p);

pstd = estimatePortStd(p, pwgt);
disp(pstd)

    0.0222
    0.1010
```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a `PortfolioCVaR` or `PortfolioMAD` object.

For more information on creating a `PortfolioCVaR` or `PortfolioMAD` object, see

- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

pwgt — Collection of portfolios

matrix

Collection of portfolios, specified as a `NumAssets`-by-`NumPorts` matrix, where `NumAssets` is the number of assets in the universe and `NumPorts` is the number of portfolios in the collection of portfolios.

Data Types: `double`

Output Arguments

pstd — Estimates for standard deviations of portfolio returns for each portfolio in **pwgt** vector

Estimates for standard deviations of portfolio returns for each portfolio in **pwgt**, returned as a **NumPorts** vector.

Tips

You can also use dot notation to estimate the standard deviation of portfolio returns.

```
pstd = obj.estimatePortStd(pwgt);
```

Version History

Introduced in R2012b

See Also

[estimatePortReturn](#) | [estimateFrontierByReturn](#) | [estimateFrontierByRisk](#) | [estimatePortVaR](#) | [rng](#)

Topics

“Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105

“Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105

“Portfolio Optimization Theory” on page 4-4

External Websites

Getting Started with Portfolio Optimization (4 min 13 sec)

CVaR Portfolio Optimization (4 min 56 sec)

estimatePortVaR

Estimate value-at-risk for PortfolioCVaR object

Syntax

```
pvar = estimatePortVaR(obj,pwgt)
```

Description

`pvar = estimatePortVaR(obj,pwgt)` estimates value-at-risk for a `PortfolioCVaR` object where the probability level used is from the `PortfolioCVaR` property `ProbabilityLevel`. For details on the workflow, see “PortfolioCVaR Object Workflow” on page 5-16.

Examples

Estimate Value-at-Risk for a PortfolioCVaR Object

Given a portfolio `pwgt`, use the `estimatePortVaR` function to estimate the value-at-risk of portfolio.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

pwgt = estimateFrontierLimits(p);

pvar = estimatePortVaR(p, pwgt);
disp(pvar)

    0.0314
    0.1483
```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a PortfolioCVaR object.

For more information on creating a PortfolioCVaR object, see

- PortfolioCVaR

Data Types: object

pwgt — Collection of portfolios

matrix

Collection of portfolios, specified as a NumAssets-by-NumPorts matrix, where NumAssets is the number of assets in the universe and NumPorts is the number of portfolios in the collection of portfolios.

Data Types: double

Output Arguments

pvar — Estimates for value-at-risk of portfolio returns for each portfolio in pwgt

vector

Estimates for value-at-risk of portfolio returns for each portfolio in pwgt, returned as a NumPorts vector.

More About

Value-at-Risk

Value-at-Risk (VaR) estimates the maximum potential loss that a portfolio could face over a specified time period, given normal market conditions and a specified confidence level.

Tips

You can also use dot notation to estimate the value-at-risk of PortfolioCVaR object.

```
pvar = obj.estimatePortVaR(pwgt);
```

Version History

Introduced in R2012b

See Also

estimatePortStd | setProbabilityLevel | rng

Topics

“Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105

“Conditional Value-at-Risk” on page 5-5

External Websites

Getting Started with Portfolio Optimization (4 min 13 sec)

CVaR Portfolio Optimization (4 min 56 sec)

estimateScenarioMoments

Estimate mean and covariance of asset return scenarios

Syntax

```
[ScenarioMean, ScenarioCovar] = estimateScenarioMoments(obj)
```

Description

[ScenarioMean, ScenarioCovar] = estimateScenarioMoments(obj) estimates mean and covariance of asset return scenarios for PortfolioCVaR or PortfolioMAD objects. For details on the workflows, see “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

Examples

Estimate Mean and Covariance of Asset Return Scenarios for a PortfolioCVaR Object

Given PortfolioCVaR object p, use the estimatePortRisk function to estimate mean and covariance of asset return scenarios.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

[ScenarioMean, ScenarioCovar] = estimateScenarioMoments(p)

ScenarioMean = 4×1

    0.0039
    0.0082
    0.0102
    0.0154

ScenarioCovar = 4×4

    0.0005    0.0003    0.0001   -0.0001
```

```

0.0003    0.0024    0.0017    0.0010
0.0001    0.0017    0.0048    0.0028
-0.0001    0.0010    0.0028    0.0102

```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Estimate Mean and Covariance of Asset Return Scenarios for a PortfolioMAD Object

Given PortfolioMAD object `p`, use the `estimatePortRisk` function to estimate mean and covariance of asset return scenarios.

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

[ScenarioMean, ScenarioCovar] = estimateScenarioMoments(p)

ScenarioMean = 4×1

    0.0039
    0.0082
    0.0102
    0.0154

```

```

ScenarioCovar = 4×4

    0.0005    0.0003    0.0001   -0.0001
    0.0003    0.0024    0.0017    0.0010
    0.0001    0.0017    0.0048    0.0028
   -0.0001    0.0010    0.0028    0.0102

```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Input Arguments

obj — Object for portfolio
object

Object for portfolio, specified using a `PortfolioCVaR` or `PortfolioMAD` object.

For more information on creating a `PortfolioCVaR` or `PortfolioMAD` object, see

- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

Output Arguments

ScenarioMean — Estimate for mean of scenarios

`[]` (default) | `vector`

Estimate for mean of scenarios, returned as a `NumPorts` vector or `[]`.

Note If no scenarios are associated with the specified object, both `ScenarioMean` and `ScenarioCovar` are set to empty `[]`.

ScenarioCovar — Estimate for covariance of scenarios

`[]` (default) | `matrix`

Estimate for covariance of scenarios, returned as a `NumAssets`-by-`NumAssets` matrix or `[]`.

Note If no scenarios are associated with the specified object, both `ScenarioMean` and `ScenarioCovar` are set to empty `[]`.

Tips

You can also use dot notation to estimate the mean and covariance of asset return scenarios for a portfolio.

```
[ScenarioMean, ScenarioCovar] = obj.estimateScenarioMoments
```

Version History

Introduced in R2012b

See Also

`setScenarios` | `estimatePortRisk` | `simulateNormalScenariosByMoments` | `rng`

Topics

“Asset Returns and Scenarios Using `PortfolioCVaR` Object” on page 5-36

“Asset Returns and Scenarios Using `PortfolioMAD` Object” on page 6-35

External Websites

Getting Started with Portfolio Optimization (4 min 13 sec)

CVaR Portfolio Optimization (4 min 56 sec)

ewstats

Expected return and covariance from return time series

Syntax

```
[ExpReturn,ExpCovariance,NumEffObs] = ewstats(RetSeries)
[ExpReturn,ExpCovariance,NumEffObs] = ewstats( ___,DecayFactor,WindowLength)
```

Description

[ExpReturn,ExpCovariance,NumEffObs] = ewstats(RetSeries) computes estimated expected returns (ExpReturn), estimated covariance matrix (ExpCovariance), and the number of effective observations (NumEffObs). These outputs are maximum likelihood estimates which are biased.

[ExpReturn,ExpCovariance,NumEffObs] = ewstats(___,DecayFactor,WindowLength) adds optional input arguments for DecayFactor and WindowLength.

Examples

Compute Estimated Expected Returns and Estimated Covariance Matrix

This example shows how to compute the estimated expected returns and the estimated covariance matrix.

```
RetSeries = [ 0.24 0.08
              0.15 0.13
              0.27 0.06
              0.14 0.13 ];
```

```
DecayFactor = 0.98;
```

```
[ExpReturn, ExpCovariance] = ewstats(RetSeries, DecayFactor)
```

```
ExpReturn = 1×2
```

```
    0.1995    0.1002
```

```
ExpCovariance = 2×2
```

```
    0.0032   -0.0017
   -0.0017    0.0010
```

Input Arguments

RetSeries — Return series
matrix

Return series, specified the number of observations (NUMOBS) by number of assets (NASSETS) matrix of equally spaced incremental return observations. The first row is the oldest observation, and the last row is the most recent.

Data Types: double

DecayFactor — Controls how much less each observation is weighted than its successor

1 (default) | numeric

(Optional) Controls how much less each observation is weighted than its successor, specified as a numeric value. The k th observation back in time has weight DecayFactor^k . DecayFactor must lie in the range: $0 < \text{DecayFactor} \leq 1$.

The default value of 1 is the equally weighted linear moving average model (BIS).

Data Types: double

WindowLength — Number of recent observations in computation

NUMOBS (default) | numeric

(Optional) Number of recent observations in the computation, specified as a numeric value.

Data Types: double

Output Arguments

ExpReturn — Estimated expected returns

vector

Estimated expected returns, returned as a 1-by-NASSETS vector.

ExpCovariance — Estimated covariance matrix

matrix

Estimated covariance matrix, returned as a NASSETS-by-NASSETS matrix.

The standard deviations of the asset return processes are defined as

$$\text{STDVec} = \text{sqrt}(\text{diag}(\text{ExpCovariance}))$$

The correlation matrix is

$$\text{CorrMat} = \text{ExpCovariance} ./ (\text{STDVec} * \text{STDVec}')$$

NumEffObs — Number of effective observations

numeric

NumEffObs is the number of effective observations where

$$\text{NumEffObs} = \frac{1 - \text{DecayFactor}^{\text{WindowLength}}}{1 - \text{DecayFactor}}$$

A smaller DecayFactor or WindowLength emphasizes recent data more strongly but uses less of the available data set.

Algorithms

For a return series $r(1), \dots, r(n)$, where (n) is the most recent observation, and w is the decay factor, the expected returns (ExpReturn) are calculated by

$$E(r) = \frac{(r(n) + wr(n-1) + w^2r(n-2) + \dots + w^{n-1}r(1))}{NumEffObs}$$

where the number of effective observations NumEffObs is defined as

$$NumEffObs = 1 + w + w^2 + \dots + w^{n-1} = \frac{1 - w^n}{1 - w}$$

$E(r)$ is the weighed average of $r(n), \dots, r(1)$. The unnormalized weights are $w, w^2, \dots, w^{(n-1)}$. The unnormalized weights do not sum up to 1, so NumEffObs rescales the unnormalized weights. After rescaling, the normalized weights (which sum up to 1) are used for averaging. When $w = 1$, then NumEffObs = n , which is the number of observations. When $w < 1$, NumEffObs is still interpreted as the sample size, but it is less than n due to the down-weight on the observations of the remote past.

Note The ewstats function may give slightly different results from the RiskMetrics® approach for determining expected return and covariance from a time series. This is because ewstats calculates NumEffObs by directly summing the unnormalized weights, while RiskMetrics® uses an approximation. Additionally, RiskMetrics® assumes a mean of 0 in the return series when calculating the covariance, while ewstats uses the calculated ExpReturn output.

Version History

Introduced before R2006a

See Also

cov | mean | cov2corr

Topics

"Portfolio Optimization Functions" on page 3-3

fanplot

Plot combined historical and forecast data to visualize possible outcomes

Syntax

```
fanplot(historical, forecast)
fanplot( ___, Name, Value)

fanplot(ax, historical, forecast)
fanplot( ___, Name, Value)

h = fanplot(ax, historical, forecast)
[h, ax] = fanplot( ___, Name, Value)
```

Description

`fanplot(historical, forecast)` generates a fan chart. In time series analysis, a fan chart is a chart that joins a simple line chart for observed past data with ranges for possible values of future data. The historical data and possible future data are joined with a line showing a central estimate or most likely value for the future outcomes.

`fanplot` supports three plotting scenarios:

- Matching — This scenario occurs when the time period perfectly matches for historical and forecast data.
- Backtest — This scenario occurs when there are overlaps between historical and forecast data.
- Gap — This scenario occurs when there are NaN values in the historical or forecast data.

`fanplot( ___, Name, Value)` generates a fan chart using optional name-value pair arguments.

`fanplot(ax, historical, forecast)` generates a fan chart using an optional `ax` argument.

`fanplot( ___, Name, Value)` generates a fan chart using optional name-value pair arguments.

`h = fanplot(ax, historical, forecast)` generates a fan chart and returns the figure handle `h`. In time series analysis, a fan chart is a chart that joins a simple line chart for observed past data with ranges for possible values of future data. The historical data and possible future data are joined with a line showing a central estimate or most likely value for the future outcomes.

`fanplot` supports three plotting scenarios:

- Matching — This scenario occurs when the time period perfectly matches for historical and forecast data.
- Backtest — This scenario occurs when there are overlaps between historical and forecast data.
- Gap — This scenario occurs when there are NaN values in the historical or forecast data.

`[h, ax] = fanplot( ___, Name, Value)` generates a fan chart and returns the figure handle `h` and axis (`ax`) using optional name-value pair arguments.

Examples

Create a Fan Plot Using Cell Array Data

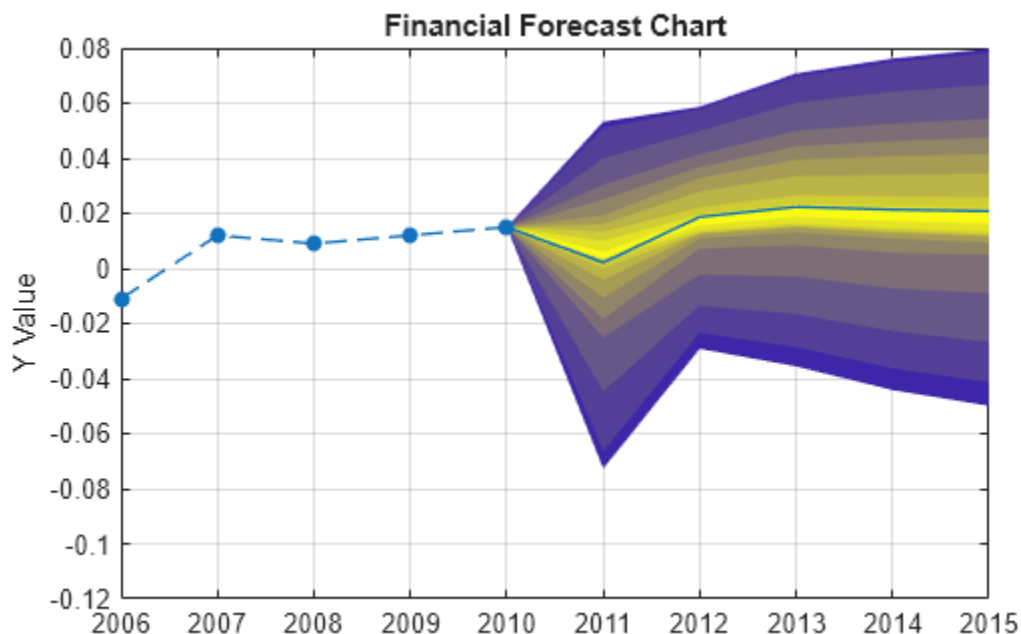
Define the data inputs for `historical` as a 5-by-2 cell array and `forecast` as a 5-by-21 cell array with 20 observations.

```
historical = {2006    -0.0110
              2007     0.0120
              2008     0.0090
              2009     0.0120
              2010     0.0150};
```

```
forecast = {2011  0.0203    -0.0155    0.0311    -0.0026    0.0035    0.0049    0.0020
            0.0533     0.0139    0.0037    -0.0727    -0.0291   -0.0058    0.0183
            2012  0.0430    -0.0094    0.0587    0.0095    0.0185    0.0205    0.0172
            0.0141     0.0337    0.0187    0.0132    -0.0292    0.0048    0.0400
            2013  0.0518    -0.0116    0.0708    0.0112    0.0221    0.0246    0.0209
            0.0168     0.0405    0.0224    0.0157   -0.0356    0.0056    0.0481
            2014  0.0546    -0.0171    0.0762    0.0088    0.0210    0.0239    0.0193
            0.0151     0.0419    0.0214    0.0139   -0.0442    0.0024    0.0500
            2015  0.0565    -0.0207    0.0797    0.0072    0.0203    0.0234    0.0183
            0.0139     0.0428    0.0207    0.0126   -0.0499    0.0026    0.0522}
```

Generate the fan plot.

```
fanplot (historical, forecast);
```



The dotted points are the historical lines and the filled lines indicate the mean for the forecasts. This fanplot represents a matching scenario where the time period perfectly matches for the historical and forecast data.

Create a Fan Plot Using Matrix Data

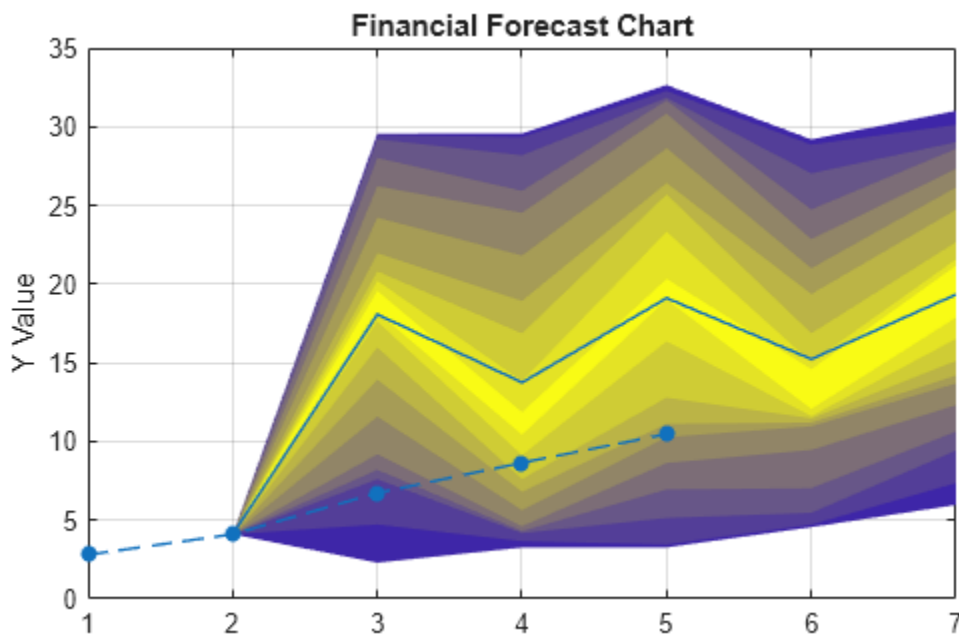
Define the data inputs for `historical` as a 5-by-2 matrix and `forecast` as a 5-by-21 matrix with 20 observations.

```
historical = [ 1.0000    2.8046 ;
              2.0000    4.1040 ;
              3.0000    6.7292 ;
              4.0000    8.6486 ;
              5.0000   10.4747 ];
```

```
forecast = [ 3.0000    28.9874    18.3958    19.6376    29.5627    8.3462    7.1502    25.3845    2
            20.8557    27.0691    23.0803    20.7885    18.0205    17.2294    10.0197    29
            4.0000    4.8933    27.2659    7.2206    24.4703    10.5895    15.0212    29.1137    6
            8.0007    18.7114    19.1691    24.5963    4.2835    4.0676    3.2612    29
            5.0000    20.9732    19.7069    11.6862    25.7018    31.8940    7.2664    19.2113    10
            31.7996    3.6419    3.2695    27.1422    10.5487    32.6529    18.8370    6
            6.0000    11.0069    29.1965    4.5551    20.2627    10.9209    15.2675    28.5359    11
            23.9532    18.4804    25.5484    4.8747    8.0036    11.5329    11.6807    23
            7.0000    5.9699    11.1486    26.0449    13.4619    21.1196    28.8068    26.2525    10
            21.2390    29.2396    18.4828    28.3945    21.9342    14.4642    17.2613    15
```

Generate the fan plot and return the figure handle (h) and axis object (ax).

```
[h,ax] = fanplot(historical, forecast)
```



```

h =
  Figure (1) with properties:

    Number: 1
    Name: ''
    Color: [1 1 1]
    Position: [161 244 560 336.9343]
    Units: 'pixels'

  Show all properties

ax =
  Axes (Financial Forecast Chart) with properties:

    XLim: [1 7]
    YLim: [0 35]
    XScale: 'linear'
    YScale: 'linear'
    GridLineStyle: '-'
    Position: [0.1300 0.1100 0.7750 0.8150]
    Units: 'normalized'

  Show all properties

```

The dotted points are the historical lines and the filled lines indicate the mean for the forecasts. This fanplot represents a backtest scenario where there is an overlap between the historical and forecast data.

Create Fan Plot Using Cell Array Data and Customize the Plot With Name-Value Pair Arguments

Define the data inputs for `historical` as a 5-by-2 cell array and `forecast` as a 5-by-21 cell array with 20 observations.

```

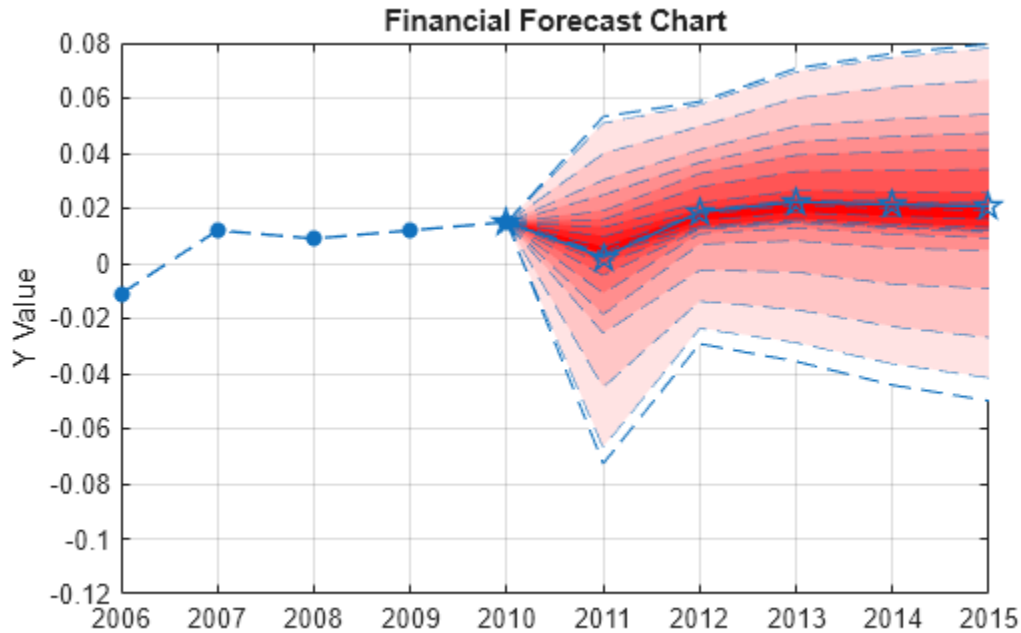
historical = {2006    -0.0110
              2007     0.0120
              2008     0.0090
              2009     0.0120
              2010     0.0150};

forecast = {2011  0.0203    -0.0155    0.0311    -0.0026    0.0035    0.0049    0.0020
            0.0533     0.0139    0.0037    -0.0727    -0.0291    -0.0058    0.0183
            2012  0.0430    -0.0094    0.0587    0.0095    0.0185    0.0205    0.0172
            0.0141     0.0337    0.0187    0.0132    -0.0292    0.0048    0.0400
            2013  0.0518    -0.0116    0.0708    0.0112    0.0221    0.0246    0.0209
            0.0168     0.0405    0.0224    0.0157    -0.0356    0.0056    0.0481
            2014  0.0546    -0.0171    0.0762    0.0088    0.0210    0.0239    0.0193
            0.0151     0.0419    0.0214    0.0139    -0.0442    0.0024    0.0500
            2015  0.0565    -0.0207    0.0797    0.0072    0.0203    0.0234    0.0185
            0.0139     0.0428    0.0207    0.0126    -0.0499    0.0026    0.0522}

```

Generate the fan plot using name-value pair arguments to customize the presentation.

```
fanplot(historical,forecast,'FanFaceColor',[1 1 1;1 0 0],'FanLineStyle','--','ForecastMarker','p')
```



Create a Fan Plot Using Table Data

Create table of historical dates and data.

```
historicalDates = datetime(2006:2010,1,1)';
historicalData = [-0.0110;0.0120;0.0090;0.0120;0.0150];
historical = table(historicalDates,historicalData,'VariableNames',{'Dates','Data'});
```

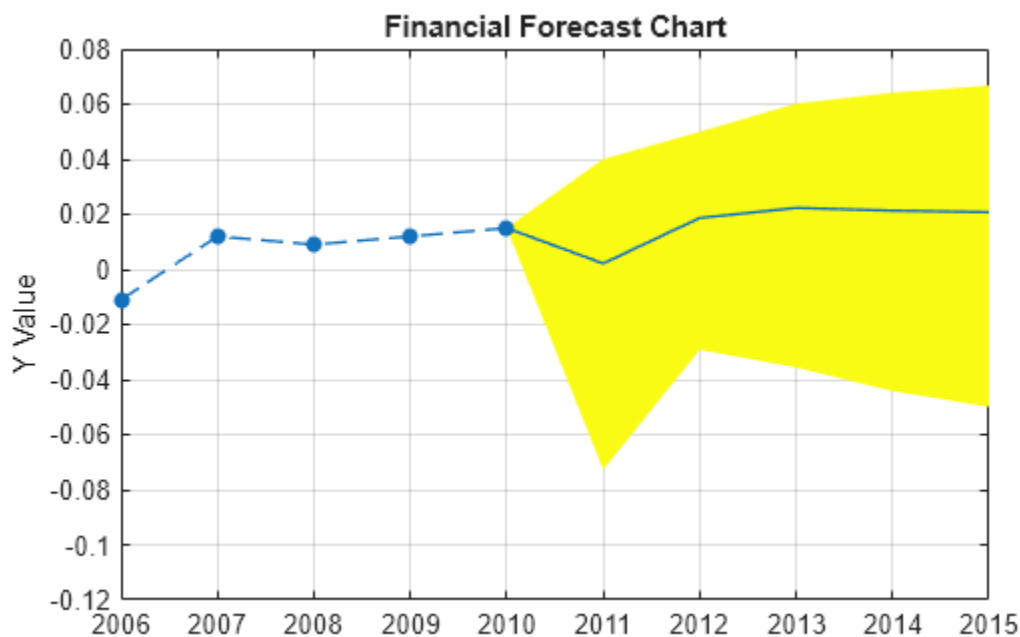
Create table of forecast dates and data.

```
forecastDates = datetime(2011:2015,1,1)';
forecastData =
    [0.0203    -0.0155    0.0311   -0.0026    0.0035    0.0049    0.0026
      0.0533     0.0139    0.0037   -0.0727   -0.0291   -0.0058    0.0183
      0.0430   -0.0094    0.0587    0.0095    0.0185    0.0205    0.0172
      0.0141    0.0337    0.0187    0.0132   -0.0292    0.0048    0.0400
      0.0518   -0.0116    0.0708    0.0112    0.0221    0.0246    0.0205
      0.0168    0.0405    0.0224    0.0157   -0.0356    0.0056    0.0482
      0.0546   -0.0171    0.0762    0.0088    0.0210    0.0239    0.0193
      0.0151    0.0419    0.0214    0.0139   -0.0442    0.0024    0.0506
      0.0565   -0.0207    0.0797    0.0072    0.0203    0.0234    0.0185
      0.0139    0.0428    0.0207    0.0126   -0.0499    0.0026    0.0522]
```

```
forecast = [table(forecastDates,'VariableName',{'Dates'}),array2table(forecastData)];
```

Plot the data using fanplot and use the Quantiles name-value argument to find the 90th percentile.

```
fanplot(historical,forecast,Quantiles=0.9);
```



Input Arguments

historical — Historical dates and data

matrix | cell array | table | timetable

Historical dates and data, specified as an N-by-2 matrix, cell array, table, or timetable where the first column is the date, and the second column is the data associated for that date. N indicates the number of dates. By using the cell array format for the input, you can make the first column datetime and produce the same plot as would serial date numbers or date character vectors. For example:

```
historical(:,1) = num2cell(datetime(2006:2010,1,1));
forecast(:,1) = num2cell(datetime(2011:2015,1,1));
fanplot (historical, forecast);
```

Data Types: cell | double | table | timetable

forecast — Forecast dates and data

matrix | cell array of character vectors | table | timetable

Forecast dates and data, specified as an N-by-M matrix, cell array, table, or timetable where the first column is the date, and the second to the last columns are the data observations. N indicates the number of the dates and (M - 1) is the number for data observations. By using the cell array format for the input, you can make the first column datetime and produce the same plot as would serial date numbers or date character vectors. For example:

```
historical(:,1) = num2cell(datetime(2006:2010,1,1));
forecast(:,1) = num2cell(datetime(2011:2015,1,1));
fanplot (historical, forecast);
```

Data Types: cell | double | table | timetable

ax — Valid axis object

object

(Optional) Valid axis object, specified as an `ax` object that is created using `axes`. The plot will be created in the axes specified by the optional `ax` argument instead of in the current axes (`gca`). The optional argument `ax` can precede any of the input argument combinations.

Data Types: `object`**Name-Value Pair Arguments**

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example:

```
fanplot(historical,forecast,'NumQuantiles',14,'FanLineColor','blue','HistoricalLineWidth',1.8,'ForecastLineColor','red')
```

NumQuantiles — Number of quantiles to display

20 (default) | positive integer

Number of quantiles to display in fan chart, specified as a positive integer.

Note `NumQuantiles` is ignored if `Quantiles` is specified.

Data Types: `double`**Quantiles — Quantiles to display**Use values from `NumQuantiles` (default) | nonnegative decimal

Quantiles to display in fan chart, specified as a nonnegative decimal value between 0 and 1.

Note If `Quantiles` is specified, then `NumQuantiles` is ignored.

Data Types: `double`**FanLineStyle — Style of the lines separating fans**

'none' (default) | character vector

Style of the lines separating fans, specified as a character vector. For more information on supported character vectors for line styles, see `Primitive Line`.

Data Types: `char`**FanLineColor — Color of lines separating fans**

axes dependent (default) | character vector for color or RGB triplet

Color of lines separating fans, specified as a character vector for color or an RGB triplet. For more information on supported color character vectors, see `Primitive Line`.

Data Types: double | char

FanFaceColor — Color of each fan

[1 1 0;1 0 0] (yellow to red) (default) | matrix

Color of each fan, specified as an N-by-3 matrix controlling the color of each fan, where each row is an RGB triplet. There are three possible values of N:

- When $N = \text{NumQuantiles}$, the color of each fan is specified by the corresponding row in the matrix.
- When $N = \text{ceil}(\text{NumQuantiles}/2)$, the specified colors represent the bottom half of the fans. The colors of the top half are determined by reversing the order of these colors. For more information, see `ceil`.
- When $N = 2$, the colors in the bottom half of the fan are a linear interpolation between the two specified colors. The pattern is reversed for the top half.

Data Types: double

HistoricalMarker — Marker symbol of historical line

'o' (default) | character vector

Marker symbol of historical line, specified as a character vector. For more information on supported character vectors for markers, see `Primitive Line`.

Data Types: char

HistoricalMarkerSize — Marker size of historical line

5 (default) | positive value in point units

Marker size of historical line, specified as a positive value in point units.

Data Types: double | char

HistoricalMarkerFaceColor — Marker fill color of historical line

axes dependent (default) | character vector with a value of 'none', 'auto', color identifier, or RGB triplet

Marker fill color of historical line, specified as a character vector with a value of 'none', 'auto', a character vector for color, or an RGB triplet. For more information on supported character vectors for color, see `Primitive Line`.

Data Types: double | char

HistoricalMarkerEdgeColor — Marker outline color of historical line

axes dependent (default) | character vector with a value of 'none', 'auto', color identifier, or RGB triplet

Marker outline color of historical line, specified as a character vector with a value of 'none', 'auto', a character vector for color, or an RGB triplet. For more information on supported character vectors for color, see `Primitive Line`.

Data Types: double | char

HistoricalLineColor — Color of historical line

axes dependent (default) | character vector with a value of 'none', color identifier, or RGB triplet

Color of historical line, specified as a character vector with a value of 'none', a character vector for color, or an RGB triplet. For more information on supported character vectors for color, see Primitive Line.

Data Types: double | char

HistoricalLineStyle — Style of historical line

'--' (default) | character vector

Style of historical line, specified as a character vector. For more information on supported character vectors for line styles, see Primitive Line.

Data Types: char

HistoricalLineWidth — Width of historical line

1 (default) | positive value in point units

Width of historical line, specified as a positive value in point units.

Data Types: double

ForecastMarker — Marker symbol of forecast line

'none' (default) | character vector

Marker symbol of forecast line, specified as a character vector. For more information on supported character vectors for marker symbols, see Primitive Line.

Data Types: char

ForecastMarkerSize — Marker size of forecast line

5 (default) | positive value in point units

Marker size of forecast line, specified as a positive value in point units.

Data Types: double

ForecastMarkerFaceColor — Marker fill color of forecast line

'none' (default) | character vector with a value of 'none', 'auto', color identifier, or RGB triplet

Marker fill color of forecast line, specified as a character vector with a value of 'none', 'auto', a character vector for color, or an RGB triplet. For more information on supported character vectors for color, see Primitive Line.

Data Types: double | char

ForecastMarkerEdgeColor — Marker outline color of forecast line

'auto' (default) | character vector with a value of 'none', 'auto', color identifier, or RGB triplet

Marker outline color of forecast line, specified as a character vector with a value of 'none', 'auto', character vector for color, or an RGB triplet. For more information on supported character vectors for color, see Primitive Line.

Data Types: double | char

ForecastLineColor — Color of forecast line

axes dependent (default) | character vector with a value of 'none', color identifier, or RGB triplet

Color of forecast line, specified as a character vector with a value of 'none', a character vector for color, or an RGB triplet. For more information on supported character vectors for color, see Primitive Line.

Data Types: double | char

ForecastLineStyle — Style of forecast line

' - ' (default) | character vector

Style of forecast line, specified as a character vector. For more information on supported character vectors for line styles, see Primitive Line.

Data Types: char

ForecastLineWidth — Width of forecast line

1 (default) | positive value in point units

Width of forecast line, specified as a positive value in point units.

Data Types: double

Output Arguments

h — Figure handle for fanplot

handle object

Figure handle for the fanplot, returned as handle object.

ax — Axis object

axis object

Axis object, returned as an ax object.

Version History

Introduced in R2014b

R2025a: Added Name-Value Argument for Quantiles

Use the name-value argument for Quantiles to specify quantiles displayed in the fan chart.

R2025a: Additional output argument for ax

You can specify an additional output argument for ax that retunes an axis object.

See Also

bollinger | candle | highlow | linebreak | movavg | pointfig | renko | volarea | priceandvol | datetime | ceil | timetable

fbusdate

First business date of month

Syntax

```
Date = fbusdate(Year,Month)
Date = fbusdate( ___,Holiday,Weekend,outputType)
```

Description

`Date = fbusdate(Year,Month)` returns the serial date number for the first business date of the given year and month.

`Year` and `Month` can contain multiple values. If one contains multiple values, the other must contain the same number of values or a single value that applies to all. For example, if `Year` is a 1-by-N vector of integers, then `Month` must be a 1-by-N vector of integers or a single integer. `Date` is then a 1-by-N vector of datetimes or serial date numbers.

Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

`Date = fbusdate( ___,Holiday,Weekend,outputType)` returns the serial date number for the first business date of the given year and month using optional input arguments. The optional argument `Holiday` specifies nontrading days.

If neither `Holiday` nor `outputType` are specified, `Date` is returned as a serial date number. If `Holiday` is specified, but not `outputType`, then the type of the holiday variable controls the type of date. If `Holiday` is a string or date character vector, then `Date` is returned as a serial date number.

Examples

Return a Serial Date Number for the First Business Date

This example shows how to return serial date numbers for the first business date, given year and month.

```
Date = fbusdate(2001, 11)
```

```
Date =
731156
```

```
datestr(Date)
```

```
ans =
'01-Nov-2001'
```

```
Year = [2002 2003 2004];
Date = fbusdate(Year, 11)
```

```
Date = 1×3
```

731521 731888 732252

```
datestr(Date)
```

```
ans = 3×11 char array
'01-Nov-2002'
'03-Nov-2003'
'01-Nov-2004'
```

Return a Serial Date Number for the First Business Date Using the Weekend Argument

This example shows how to return serial date numbers for the first business date, given year and month, and also indicate that Saturday is a business day by setting the `Weekend` argument. March 1, 2003, is a Saturday. Use `fbusdate` to check that this Saturday is actually the first business day of the month.

```
Weekend = [1 0 0 0 0 0 0];
Date = datestr(fbusdate(2003, 3, [], Weekend))

Date =
'01-Mar-2003'
```

Return a datetime array for Date for the First Business Date Using the outputType Argument

This example shows how to return a datetime array for `Date` using an `outputType` of `'datetime'`.

```
Date = fbusdate(2001, 11, [], [], 'datetime')

Date = datetime
01-Nov-2001
```

Input Arguments

Year — Year to determine occurrence of weekday

4-digit integer | vector of 4-digit integers

Year to determine occurrence of weekday, specified as a 4-digit integer or vector of 4-digit integers.

Data Types: `double`

Month — Month to determine occurrence of weekday

integer with value 1 through 12 | vector of integers with values 1 through 12

Month to determine occurrence of weekday, specified as an integer or vector of integers with values 1 through 12.

Data Types: `double`

Holiday — Holidays and nontrading-day dates

non-trading day vector is determined by the routine `holidays` (default) | datetime array | string array | date character vector

Holidays and nontrading-day dates, specified as vector using a datetime array, string array, date character vectors, or serial date numbers.

Note The `holidays` function supplies the default vector.

All dates in `Holiday` must be the same format: either datetimes, strings, or date character vectors.

If `Holiday` is a datetime array, then `Date` is returned as a datetime array. If `outputType` is specified, then its value determines the output type of `Date`. This overrides any influence of `Holiday`.

To support existing code, `fbusdate` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Weekend — Weekend days

`[1 0 0 0 0 0 1]` (Saturday and Sunday form the weekend) (default) | vector of length 7, containing 0 and 1, where 1 indicates weekend days

Weekend days, specified as a vector of length 7, containing 0 and 1, where 1 indicates weekend days and the first element of this vector corresponds to Sunday.

Data Types: `double`

outputType — Year to determine days

'`datenum`' (default) | character vector with values '`datenum`' or '`datetime`'

A character vector specified as either '`datenum`' or '`datetime`'. The output `Date` is in serial date format if '`datenum`' is specified, or datetime format if '`datetime`' is specified. By default the output `Date` is in serial date format, or match the format of `Holiday`, if specified.

Data Types: `char`

Output Arguments

Date — Date for the first business date of given year and month

serial date number | datetime array

Date for the first business date of a given year and month, returned as a datetime or serial date number.

If neither `Holiday` nor `outputType` are specified, `Date` is returned as a serial date number. If `Holiday` is specified, but not `outputType`, then the type of the holiday variable controls the type of date:

- If `Holiday` is a string or date character vector, then `Date` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.
- If `Holiday` is a datetime array, then `Date` is returned as a datetime array.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although fbusdate supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`busdate` | `holidays` | `isbusday` | `lbusdate` | `datetime`

Topics

“Handle and Convert Dates” on page 2-2

“Trading Calendars User Interface” on page 12-2

“UICalendar User Interface” on page 12-4

floatdiscmargin

Discount margin for floating-rate bond

Syntax

```
Margin = floatdiscmargin(Price, SpreadSettle, Maturity, RateInfo,  
LatestFloatingRate)  
Margin = floatdiscmargin( ___, Name, Value)
```

Description

Margin = floatdiscmargin(Price, SpreadSettle, Maturity, RateInfo, LatestFloatingRate) calculates the discount margin or zero discount margin for a floating-rate bond.

The input RateInfo determines whether the discount margin or the zero discount margin is calculated. Principal schedules are supported using Principal.

Margin = floatdiscmargin(___, Name, Value) adds optional name-value pair arguments.

Examples

Compute the Zero Discount Margin Using a Yield Curve

Use floatdiscmargin to compute the discount margin and zero discount margin for a floating-rate note.

Define data for the floating-rate note.

```
Price = 99.99;  
Spread = 50;  
Settle = datetime(2011,1,20);  
Maturity = datetime(2012,1,15);  
LatestFloatingRate = 0.05;  
StubRate = 0.049;  
SpotRate = 0.05;  
Reset = 4;  
Basis = 2;
```

Compute the discount margin.

```
dMargin = floatdiscmargin(Price, Spread, Settle, Maturity, ...  
[StubRate, SpotRate], LatestFloatingRate, 'Reset', Reset, 'Basis', Basis, ...  
'AdjustCashFlowsBasis', true)  
  
dMargin =  
48.4810
```

Usually you want to set AdjustCashFlowsBasis to true, so cash flows are calculated with adjustments on accrual amounts.

Create an annualized zero-rate term structure to calculate the zero discount margin.

```
Rates = [0.0500;
          0.0505;
          0.0510;
          0.0520];
StartDates = [datetime(2011,1,20);
              datetime(2011,4,15);
              datetime(2011,7,15);
              datetime(2011,11,15)];
EndDates = [datetime(2011,4,15);
            datetime(2011,7,15);
            datetime(2011,11,15);
            datetime(2012,1,15)];
ValuationDate = datetime(2011,1,20);
RateSpec = intenvset('Compounding', Reset, 'Rates', Rates,...
                    'StartDates', StartDates, 'EndDates', EndDates,...
                    'ValuationDate', ValuationDate, 'Basis', Basis);
```

Calculate the zero discount margin using the previous yield curve.

```
dMargin = floatdiscmargin(Price, Spread, Settle, Maturity, ...
                          RateSpec, LatestFloatingRate, 'Reset', Reset, 'Basis', Basis, ...
                          'AdjustCashFlowsBasis', true)
```

```
dMargin =
46.0689
```

Input Arguments

Price — Bond prices where discount margin is to be computed

matrix

Bond prices where discount margin is to be computed, specified as a NINST-by-1 matrix.

Note The spread is calculated against the clean price (the function internally does not add the accrued interest to the price specified by the Price input). If the spread is required against the dirty price, the price of a bond that includes the accrued interest, you must supply the dirty price for the Price input.

Data Types: double

Spread — Number of basis points over the reference rate

numeric

Number of basis points over the reference rate, specified as a NINST-by-1 matrix.

Data Types: double

Settle — Settlement date of the floating-rate bonds

datetime array | string array | date character vector

Settlement date of the floating-rate bonds, specified as a scalar or a NINST-by-1 vector using a datetime array, string array, or date character vectors. If supplied as a NINST-by-1 vector of dates, all settlement dates must be the same (only a single settlement date is supported)

To support existing code, `floatdiscmargin` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date of the floating-rate bond

datetime array | string array | date character vector

Maturity date of the floating-rate bond, specified as a NINST-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `floatdiscmargin` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

RateInfo — Interest-rate information

numeric

interest-rate information, specified as NINST-by-2 vector where the:

- First column is the stub rate between the settlement date and the first coupon rate.
- Second column is the reference rate for the term of the floating coupons (for example, the 3-month LIBOR from settlement date for a bond with a `Reset` of 4).

Note If the `RateInfo` argument is an annualized zero-rate term structure created by `intenvset`, the zero discount margin is calculated.

Data Types: `double`

LatestFloatingRate — Rate for next floating payment set at last reset date

numeric

Rate for the next floating payment set at the last reset date, specified as NINST-by-1 vector.

Data Types: `double`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `Margin = floatdiscmargin(Price,Spread,Settle,Maturity,RateInfo,LatestFloatingRate,'Reset',2,'Basis',5)`

Reset — Frequency of payments per year

1 (default) | numeric

Frequency of payments per year, specified as NINST-by-1 vector.

Data Types: double

Basis — Day-count basis used for time factor calculations

0 (actual/actual) (default) | integers of the set [0 . . . 13] | vector of integers of the set [0 . . . 13]

Day-count basis used for time factor calculations, specified as a NINST-by-1 vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

Principal — Notional principal amounts

100 (default) | numeric

Notional principal amounts, specified as a NINST-by-1 vector or a NINST-by-1 cell array where each element is a NUMDATES-by-2 cell array where the first column is dates and the second column is the associated principal amount. The date indicates the last day that the principal value is valid.

Data Types: double | cell

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as a NINST-by-1 vector. This rule applies only when Maturity is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: logical

AdjustCashFlowsBasis — Adjust cash flows according to accrual amount

0 (not in effect) (default) | nonnegative integer 0 or 1

Adjusts cash flows according to the accrual amount, specified as a NINST-by-1 vector of logicals.

Note Usually you want to set `AdjustCashFlowsBasis` to 1, so cash flows are calculated with adjustments on accrual amounts. The default is set to 0 to be consistent with `floatbyzero`.

Data Types: logical

Holidays — Dates for holidays

`holidays.m` used (default) | datetime array | string array | date character vector | serial date number

Dates for holidays, specified as NHOLIDAYS-by-1 vector using a datetime array, string array, or date character vectors. Holidays are used in computing business days.

To support existing code, `floatdiscmargin` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

BusinessDayConvention — Business day conventions

'actual' (default) | character vector with values 'actual', 'follow', 'modifiedfollow', 'previous' or 'modifiedprevious'

Business day conventions, specified as a NINST-by-1 cell array of character vectors of business day conventions to be used in computing payment dates. The selection for business day convention determines how nonbusiness days are treated. Nonbusiness days are defined as weekends plus any other date that businesses are not open (for example, statutory holidays). Values are:

- 'actual' — Nonbusiness days are effectively ignored. Cash flows that fall on non-business days are assumed to be distributed on the actual date.
- 'follow' — Cash flows that fall on a nonbusiness day are assumed to be distributed on the following business day.
- 'modifiedfollow' — Cash flows that fall on a non-business day are assumed to be distributed on the following business day. However if the following business day is in a different month, the previous business day is adopted instead.
- 'previous' — Cash flows that fall on a nonbusiness day are assumed to be distributed on the previous business day.
- 'modifiedprevious' — Cash flows that fall on a nonbusiness day are assumed to be distributed on the previous business day. However if the previous business day is in a different month, the following business day is adopted instead.

Data Types: char | cell

Output Arguments**Margin — Discount margin**

numeric

Discount margin, returned as a NINST-by-1 vector of the discount margin if `RateInfo` is specified as a NINST-by-2 vector of stub and spot rates.

If `RateInfo` is specified as an annualized zero rate term structure created by `intenvset`, `Margin` is returned as a NINST-by-NCURVES matrix of the zero discount margin.

More About

Discount Margin

The discount margin is the spread between the expected yield on a floating-rate security and the benchmark interest rate, adjusted for the expected cash flows of the security.

Version History

Introduced in R2012b

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `floatdiscmargin` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] Fabozzi, Frank J., Mann, Steven V. *Floating-Rate Securities*. John Wiley and Sons, New York, 2000.
- [2] Fabozzi, Frank J., Mann, Steven V. *Introduction to Fixed Income Analytics: Relative Value Analysis, Risk Measures and Valuation*. John Wiley and Sons, New York, 2010.
- [3] O'Kane, Dominic, Sen, Saurav. "Credit Spreads Explained." Lehman Brothers Fixed Income Quantitative Research, March 2004.

See Also

`floatmargin` | `floatbyzero` | `bndspread` | `intenvset` | `datetime`

Topics

"Fixed-Income Terminology" on page 2-15

floatmargin

Margin measures for floating-rate bond

Syntax

```
[Margin,AdjPrice] = floatmargin(Price,SpreadSettle,Maturity)
[Margin,AdjPrice] = floatmargin( ____,Name,Value)
```

Description

[Margin,AdjPrice] = floatmargin(Price,SpreadSettle,Maturity) calculates margin measures for a floating-rate bond.

Use floatmargin to calculate the following types of margin measures for a floating-rate bond:

- Spread for life
- Adjusted simple margin
- Adjusted total margin

To calculate the discount margin or zero discount margin, see floatdiscmargin.

[Margin,AdjPrice] = floatmargin(____,Name,Value) adds optional name-value pair arguments.

Examples

Compute Margin Measures for a Floating-Rate Bond

Use floatmargin to compute margin measures for spreadforlife, adjustedsimple, and adjustedtotal for a floating-rate note.

Define data for the floating-rate note.

```
Price = 99.99;
Spread = 50;
Settle = datetime(2011,1,20);
Maturity = datetime(2012,1,15);
LatestFloatingRate = 0.05;
StubRate = 0.049;
SpotRate = 0.05;
Reset = 4;
Basis = 2;
```

Calculate spreadforlife.

```
Margin = floatmargin(Price, Spread, Settle, Maturity, 'Reset', Reset, 'Basis', Basis)
```

```
Margin =
51.0051
```

Calculate adjustedsimple margin.

```
[Margin, AdjPrice] = floatmargin(Price, Spread, Settle, Maturity, ...
'SpreadType', 'adjustedsimple', 'RateInfo', [StubRate, SpotRate], ...
'LatestFloatingRate', LatestFloatingRate, 'Reset', Reset, 'Basis', Basis)
```

```
Margin =
53.2830
```

```
AdjPrice =
99.9673
```

Calculate adjustedtotal margin.

```
[Margin, AdjPrice] = floatmargin(Price, Spread, Settle, Maturity, ...
'SpreadType', 'adjustedtotal', 'RateInfo', [StubRate, SpotRate], ...
'LatestFloatingRate', LatestFloatingRate, 'Reset', Reset, 'Basis', Basis)
```

```
Margin =
53.4463
```

```
AdjPrice =
99.9673
```

Input Arguments

Price — Bond prices where spreads are to be computed

matrix

Bond prices where spreads are to be computed, specified as a NINST-by-1 matrix.

Data Types: double

Spread — Number of basis points over the reference rate

numeric

Number of basis points over the reference rate, specified as a NINST-by-1 matrix.

Data Types: double

Settle — Settlement date of the floating-rate bonds

datetime array | string array | date character vector

Settlement date of the floating-rate bonds, specified as a scalar or a NINST-by-1 vector using a datetime array, string array, or date character vectors. If supplied as a NINST-by-1 vector of dates, all settlement dates must be the same (only a single settlement date is supported)

To support existing code, floatmargin also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date of the floating-rate bond

datetime array | string array | date character vector

Maturity date of the floating-rate bond, specified as a NINST-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `floatmargin` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

```
Example: [Margin,AdjPrice] = floatmargin(Price,Spread,Settle,Maturity,  
'SpreadType','adjustedtotal','RateInfo',  
[StubRate,SpotRate],'LatestFloatingRate',.0445,'Reset',2,'Basis',5)
```

SpreadType — Type of spread to calculate

`spreadforlife` (default) | `adjustedsimple` | `adjustedtotal`

Type of spread to calculate, specified by type, specified as `spreadforlife`, `adjustedsimple`, or `adjustedtotal`.

Note If the `SpreadType` is `spreadforlife` (default), then the name-value arguments `LatestFloatingRate` and `RateInfo` are not used. If the `SpreadType` is `adjustedsimple` or `adjustedtotal`, then the name-value arguments `LatestFloatingRate` and `RateInfo` must be specified.

Data Types: `double`

LatestFloatingRate — Rate for next floating payment set at last reset date

`numeric`

Rate for the next floating payment set at the last reset date, specified as `NINST-by-1` vector.

Note This rate must be specified for a `SpreadType` of `adjustedsimple` and `adjustedtotal`.

Data Types: `double`

RateInfo — Interest-rate information

`numeric`

interest-rate information, specified as `NINST-by-2` vector where the:

- First column is the stub rate between the settlement date and the first coupon rate.
- Second column is the reference rate for the term of the floating coupons (for example, the 3-month LIBOR from settlement date for a bond with a `Reset` of 4).

Note The `RateInfo` must be specified for `SpreadType` of `adjustedsimple` and `adjustedtotal`.

Data Types: double

Reset — Frequency of payments per year

1 (default) | numeric

Frequency of payments per year, specified as NINST-by-1 vector.

Data Types: double

Basis — Day-count basis used for time factor calculations

0 (actual/actual) (default) | integers of the set [0 . . . 13] | vector of integers of the set [0 . . . 13]

Day-count basis used for time factor calculations, specified as a NINST-by-1 vector. Values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

Principal — Notional principal amounts

100 (default) | numeric

Notional principal amounts, specified as NINST-by-1 vector.

Data Types: double

EndMonthRule — End-of-month rule flag

1 (in effect) (default) | nonnegative integer 0 or 1

End-of-month rule flag, specified as a NINST-by-1 vector. This rule applies only when **Maturity** is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a bond coupon payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a bond coupon payment date is always the last actual day of the month.

Data Types: logical

Holidays — Dates for holidays

`holidays.m` used (default) | datetime array | string array | date character vector

Dates for holidays, specified as `NHOLIDAYS-by-1` vector using a datetime array, string array, or date character vectors. Holidays are used in computing business days.

To support existing code, `floatmargin` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

BusinessDayConvention — Business day conventions

'actual' (default) | character vector with values 'actual', 'follow', 'modifiedfollow', 'previous' or 'modifiedprevious'

Business day conventions, specified as a `NINST-by-1` cell array of character vectors of business day conventions to be used in computing payment dates. The selection for business day convention determines how nonbusiness days are treated. Nonbusiness days are defined as weekends plus any other date that businesses are not open (for example, statutory holidays). Values are:

- 'actual' — Nonbusiness days are effectively ignored. Cash flows that fall on non-business days are assumed to be distributed on the actual date.
- 'follow' — Cash flows that fall on a nonbusiness day are assumed to be distributed on the following business day.
- 'modifiedfollow' — Cash flows that fall on a non-business day are assumed to be distributed on the following business day. However if the following business day is in a different month, the previous business day is adopted instead.
- 'previous' — Cash flows that fall on a nonbusiness day are assumed to be distributed on the previous business day.
- 'modifiedprevious' — Cash flows that fall on a nonbusiness day are assumed to be distributed on the previous business day. However if the previous business day is in a different month, the following business day is adopted instead.

Data Types: `char` | `cell`

Output Arguments**Margin — Spreads for floating-rate bond**

numeric

Spreads for the floating-rate bond, returned as a `NINST-by-1` vector.

AdjPrice — Adjusted price used to calculate spreads for SpreadType of adjustedsimple and adjustedtotal

numeric

Adjusted price used to calculate spreads for `SpreadType` of `adjustedsimple` and `adjustedtotal`, returned as a `NINST-by-1` vector.

Version History

Introduced in R2012b

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `floatmargin` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] Fabozzi, Frank J., Mann, Steven V. *Floating-Rate Securities*. John Wiley and Sons, New York, 2000.
- [2] Fabozzi, Frank J., Mann, Steven V. *Introduction to Fixed Income Analytics: Relative Value Analysis, Risk Measures and Valuation*. John Wiley and Sons, New York, 2010.

See Also

`floatdiscmargin` | `floatbyzero` | `bndspread` | `datetime`

Topics

“Fixed-Income Terminology” on page 2-15

frac2cur

Fractional currency value to decimal value

Syntax

```
Decimal = frac2cur(Fraction,Denominator)
```

Description

`Decimal = frac2cur(Fraction,Denominator)` converts a fractional currency value to a decimal value. `Fraction` is the fractional currency value input as a character vector, and `Denominator` is the denominator of the fraction.

Examples

Convert a Fractional Currency Value to a Decimal Value

This example shows how to convert a fractional currency value to a decimal value.

```
Decimal = frac2cur('12.1', 8)
```

```
Decimal =  
12.1250
```

Input Arguments

Fraction — Fractional currency values

character vector | cell array of character vectors

Fractional currency values, specified as a character vector or cell array of character vectors.

Data Types: `char` | `cell`

Denominator — Denominator of the fractions

numeric

Denominator of the fractions, specified as a scalar or vector using numeric values for the denominator.

Data Types: `double`

Output Arguments

Decimal — Decimal currency value

numeric decimal

Decimal currency value, returned as a scalar or vector with numeric decimal values.

Data Types: `double`

Version History

Introduced before R2006a

See Also

cur2frac | cur2str

frontier

Rolling efficient frontier

Syntax

```
[PortWts,AllMean,AllCovariance] = frontier(Universe,Window,Offset,NumPorts)
[PortWts,AllMean,AllCovariance] = frontier( ___,ActiveMap,Conset,NumNonNan)
```

Description

[PortWts,AllMean,AllCovariance] = frontier(Universe,Window,Offset,NumPorts) generates a surface of efficient frontiers showing how asset allocation influences risk and return over time.

Note An alternative for portfolio optimization is to use the `Portfolio` object for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

[PortWts,AllMean,AllCovariance] = frontier(___,ActiveMap,Conset,NumNonNan) specifies options using one or more optional arguments in addition to the input arguments in the previous syntax.

Input Arguments

Universe — Total return data for a group of securities

array

Total return data for a group of securities, specified as a number of observations (NUMOBS) by number of assets plus one (NASSETS + 1) time series array. Each row represents an observation. Column 1 contains MATLAB serial date numbers. The remaining columns contain the total return data for each security.

Data Types: double

Window — Number of data periods used to calculate each frontier

positive integer

Number of data periods used to calculate each frontier, specified as a positive integer value.

Data Types: double

Offset — Increment in number of periods between each frontier

numeric

Increment in number of periods between each frontier, specified as a numeric value.

Data Types: double

NumPorts — Number of portfolios to calculate on each frontier

positive integer

Number of portfolios to calculate on each frontier, specified as a positive integer.

Data Types: double

ActiveMap — Indicates if an asset is part of the Universe on the corresponding date

NUMOBS-by-NASSETS matrix of 1's (all assets active on all dates) (default) | matrix

(Optional) Indicates if an asset is part of the Universe on the corresponding date, specified as a number of observations (NUMOBS) by number of assets (NASSETS) matrix with Boolean elements corresponding to the Universe.

Data Types: double

Conset — Constraint matrix for a portfolio of asset investments

matrix

Constraint matrix for a portfolio of asset investments, specified using portcons with the 'Default' constraint type. This single constraint matrix is applied to each frontier..

Data Types: double

NumNonNaN — Minimum number of non-NaN points for each active asset in each window of data

Window - NASSETS (default) | numeric value

Minimum number of non-NaN points for each active asset in each window of data needed to perform the optimization, specified as a numeric value.

Data Types: double

Output Arguments**PortWts — Weights allocated to each asset**

matrix

Weights allocated to each asset., returned as a number of curves (NCURVES)-by-1 cell array, where each element is a NPORTS-by-NASSETS matrix of weights.

AllMean — Expected asset returns used to generate each curve on the surface

vector

Expected asset returns used to generate each curve on the surface, returned as an NCURVES-by-1 cell array, where each element is a 1-by-NASSETS vector of the expected asset returns.

AllCovariance — Covariance matrix used to generate each curve on the surface

vector

Covariance matrix used to generate each curve on the surface, returned as an NCURVES-by-1 cell array, where each element is a NASSETS-by-NASSETS vector.

More About

Efficient Frontier

The efficient frontier is a key concept in modern portfolio theory (MPT) that represents a set of optimal portfolios offering the highest expected return for a given level of risk, or the lowest risk for a given level of expected return.

The efficient frontier illustrates the tradeoff between risk (often measured as the standard deviation of portfolio returns) and expected return. Investors aim to construct portfolios that lie on this frontier to maximize their returns for a given level of risk. Portfolios that lie on the efficient frontier are considered optimal because they provide the best possible expected return for their level of risk. Portfolios that lie below the frontier are suboptimal, as they do not provide sufficient return for the amount of risk taken. The efficient frontier emphasizes the importance of diversification. By combining different assets with varying returns and risks, investors can reduce the overall risk of the portfolio without sacrificing expected returns.

Version History

Introduced before R2006a

See Also

`portcons` | `portopt` | `portstats`

Topics

“Portfolio Construction Examples” on page 3-5

“Portfolio Selection and Risk Aversion” on page 3-7

“Active Returns and Tracking Error Efficient Frontier” on page 3-27

“Analyzing Portfolios” on page 3-2

“Portfolio Optimization Functions” on page 3-3

fvdisc

Future value of discounted security

Syntax

```
FutureVal = fvdisc(Settle,Maturity,Price,Discount)
FutureVal = fvdisc( ____,Basis)
```

Description

`FutureVal = fvdisc(Settle,Maturity,Price,Discount)` finds the amount received at maturity for a fully vested security.

`FutureVal = fvdisc( ____,Basis)` specifies options using an optional argument in addition to the input arguments in the previous syntax.

Examples

Find the Amount Received at Maturity for a Fully-Vested Security Using datetime Inputs

This example shows how to use `datetime` inputs to find the amount received at maturity for a fully-vested security, using the following data.

```
Settle = datetime(2001,2,15);
Maturity = datetime(2001,5,15);
Price = 100;
Discount = 0.0575;
Basis = 2;
```

```
FutureVal = fvdisc(Settle, Maturity, Price, Discount, Basis)
```

```
FutureVal =
101.4420
```

Input Arguments

Settle — Settlement date

`datetime` scalar | `string` scalar | `date` character vector

Settlement date, specified as a scalar `datetime`, `string`, or `date` character vector.

To support existing code, `fvdisc` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date

`datetime` scalar | `string` scalar | `date` character vector

Maturity date, specified as a scalar datetime, string, or date character vector.

To support existing code, `fvdisc` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Price — Price (present value) of security

scalar numeric

Price (present value) of the security, specified as a scalar numeric.

Data Types: `double`

Discount — Bank discount rate of security

scalar decimal

Bank discount rate of security, specified as a scalar decimal.

Data Types: `double`

Basis — Day-count basis of instrument

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day-count basis of the instrument, specified as a scalar integer using one of the following values:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

Output Arguments

FutureVal — Amount received at maturity for fully vested security

scalar numeric

Amount received at maturity for a fully vested security, returned as a scalar numeric.

Data Types: double

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although fvdisc supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

References

[1] Jan Mayle. *Standard Securities Calculation Methods*. Securities Industry Assn, Volumes I-II, 3rd edition, 1994

See Also

`acrudisc` | `discrate` | `prdisc` | `ylddisc` | `datetime`

Topics

“Analyzing and Computing Cash Flows” on page 2-11

fvfix

Future value with fixed periodic payments

Syntax

```
FutureVal = fvfix(Rate,NumPeriods,Payment)
FutureVal = fvfix( ____,PresentVal,Due)
```

Description

`FutureVal = fvfix(Rate,NumPeriods,Payment)` returns the future value of a series of equal payments.

`FutureVal = fvfix( ____,PresentVal,Due)` specifies options using one or more optional argument in addition to the input arguments in the previous syntax.

Examples

Return the Future Value of a Series of Equal Payments

This example shows how to compute the future value of a series of equal payments using a savings account that has a starting balance of \$1500. \$200 is added at the end of each month for 10 years and the account pays 9% interest compounded monthly.

```
FutureVal = fvfix(0.09/12, 12*10, 200, 1500, 0)
```

```
FutureVal =  
4.2380e+04
```

Input Arguments

Rate — Interest-rate per period

scalar numeric decimal

Interest-rate per period, specified as a scalar numeric decimal.

Data Types: double

NumPeriods — Number of payment periods

scalar numeric

Number of payment periods, specified as a scalar numeric.

Data Types: double

Payment — Payment per period

scalar numeric

Payment per period, specified as a scalar numeric.

Data Types: double

PresentVal — Present value

0 (default) | scalar numeric

(Optional) Present value, specified as a scalar numeric.

Data Types: double

Due — When payments are due

0 (end of period) (default) | scalar integer with value of 0 or 1

(Optional) When payments are due, specified as a scalar integer with value of 0 (end of period) or 1 (beginning of period).

Data Types: double

Output Arguments**FutureVal — Future value of series of equal payments**

scalar numeric

Future value of a series of equal payments, returned as a scalar numeric.

Version History

Introduced before R2006a

References

[1] Jan Mayle. *Standard Securities Calculation Methods*. Securities Industry Assn, Volumes I-II, 3rd edition, 1994

See Also

fvvar | pvfix | pvvar

Topics

“Analyzing and Computing Cash Flows” on page 2-11

fvvar

Future value of varying cash flow

Syntax

```
FutureVal = fvvar(CashFlow,Rate)
FutureVal = fvvar(____,CFDates)
```

Description

`FutureVal = fvvar(CashFlow,Rate)` returns the future value of a varying cash flow.

`FutureVal = fvvar(____,CFDates)` specifies options using an optional argument in addition to the input arguments in the previous syntax.

Examples

Calculate Future Value of Varying Cash Flows

This cash flow represents the yearly income from an initial investment of \$10,000. The annual interest rate is 2%.

- Year 1 — \$2000
- Year 2 — \$1500
- Year 3 — \$3000
- Year 4 — \$3800
- Year 5 — \$5000

For the future value of this regular (periodic) cash flow:

```
FutureVal = fvvar([-10000 2000 1500 3000 3800 5000], 0.02)
```

```
FutureVal =  
4.7131e+03
```

An investment of \$10,000 returns this irregular cash flow. The original investment and its date are included. The periodic interest rate is 3%.

- -\$1000 — January 12, 2000
- \$2500 — February 14, 2001
- \$2000 — March 3, 2001
- \$3000 — June 14, 2001
- \$4000 — December 1, 2001

To calculate the future value of this irregular (nonperiodic) cash flow:

```
CashFlow = [-10000, 2500, 2000, 3000, 4000];
```

```
CFDates = [datetime(2000,1,12)
           datetime(2001,2,14)
           datetime(2001,3,3)
           datetime(2001,6,14)
           datetime(2001,12,1)];

FutureVal = fvvar(CashFlow, 0.03, CFDates)

FutureVal =
1.0731e+03
```

Input Arguments

CashFlow — Varying cash flows

vector

Varying cash flows, specified as a vector.

Note You must include the initial investment as the initial cash flow value (a negative number).

.

Data Types: double

Rate — Periodic interest rate

scalar numeric decimal

Periodic interest rate, specified as a scalar numeric decimal.

Data Types: double

CFDates — Dates on which the cash flows occur

assumes CashFlow contains regular (periodic) cash flows (default) | datetime array | string array | date character vector | serial date number

(Optional) Dates on which the cash flows occur, specified as a vector using a datetime array, string array, or date character vectors.

Note Use CFDates for irregular (nonperiodic) cash flows.

To support existing code, fvvar also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Output Arguments

FutureVal — Future value of a varying cash flow

scalar numeric

Future value of a varying cash flow, returned as a scalar numeric.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `fvvar` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

References

[1] Jan Mayle. *Standard Securities Calculation Methods*. Securities Industry Assn, Volumes I-II, 3rd edition, 1994

See Also

`irr` | `fvfix` | `pvfix` | `pvvar` | `payuni` | `datetime`

Topics

"Analyzing and Computing Cash Flows" on page 2-11

fwd2zero

Zero curve given forward curve

Note In R2017b, the specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the new optional name-value pair inputs: `InputCompounding`, `InputBasis`, `OutputCompounding`, and `OutputBasis`.

Syntax

```
[ZeroRates, CurveDates] = fwd2zero(ForwardRates, CurveDates, Settle)
[ZeroRates, CurveDates] = fwd2zero( ___, Name, Value)
```

Description

`[ZeroRates, CurveDates] = fwd2zero(ForwardRates, CurveDates, Settle)` returns a zero curve given an implied forward rate curve and its maturity dates. If both inputs for `CurveDates` and `Settle` are strings or date character vectors, `CurveDates` is returned as serial date numbers. However, if either of the inputs for `CurveDates` and `Settle` are a datetime array, `CurveDates` is returned as a datetime array.

`[ZeroRates, CurveDates] = fwd2zero( ___, Name, Value)` adds optional name-value pair arguments

Examples

Compute the Zero Curve Given the Forward Curve Using datetime Inputs

This example shows how to use `datetime` inputs compute the zero curve, given an implied forward rate curve over a set of maturity dates, a settlement date, and a compounding rate.

```
ForwardRates = [0.0469
                0.0519
                0.0549
                0.0535
                0.0558
                0.0508
                0.0560
                0.0545
                0.0615
                0.0486];

CurveDates = [datetime(2000,11,6)
              datetime(2000,12,11)
              datetime(2001,1,15)
              datetime(2001,2,5)
              datetime(2001,3,4)
              datetime(2001,4,2)]
```

```
        datetime(2001,4,30)
        datetime(2001,6,25)
        datetime(2001,9,4)
        datetime(2001,11,12)];

Settle = datetime(2000,11,3);

InputCompounding = 1;
InputBasis = 2;
OutputCompounding = 1;
OutputBasis = 2;
[ZeroRates, CurveDates] = fwd2zero(ForwardRates, CurveDates,...
Settle,'InputCompounding',1,'InputBasis',2,'OutputCompounding',1,'OutputBasis',2)

ZeroRates = 10×1

    0.0469
    0.0515
    0.0531
    0.0532
    0.0538
    0.0532
    0.0536
    0.0539
    0.0556
    0.0543

CurveDates = 10×1 datetime
    06-Nov-2000
    11-Dec-2000
    15-Jan-2001
    05-Feb-2001
    04-Mar-2001
    02-Apr-2001
    30-Apr-2001
    25-Jun-2001
    04-Sep-2001
    12-Nov-2001
```

Input Arguments

ForwardRates — Annualized implied forward rates

decimal fraction

Annualized implied forward rates, specified as a (NUMBONDS)-by-1 vector using decimal fractions. In aggregate, the rates in **ForwardRates** constitute an implied forward curve for the investment horizon represented by **CurveDates**. The first element pertains to forward rates from the settlement date to the first curve date.

Data Types: double

CurveDates — Maturity dates

datetime array | string array | date character vector

Maturity dates, specified as a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors, that correspond to the ForwardRates.

To support existing code, fwd2zero also accepts serial date numbers as inputs, but they are not recommended.

Data Types: datetime | string | char

Settle — Common settlement date for ForwardRates

datetime scalar | string scalar | date character vector

Common settlement date for ForwardRates, specified as scalar datetime, string, or date character vector.

To support existing code, fwd2zero also accepts serial date numbers as inputs, but they are not recommended.

Data Types: datetime | string | char

Name-Value Pair Arguments

Specify optional pairs of arguments as Name1=Value1,...,NameN=ValueN, where Name is the argument name and Value is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: [ZeroRates, CurveDates] =
fwd2zero(ForwardRates, CurveDates, Settle, 'InputCompounding', 3, 'InputBasis', 5, 'OutputCompounding', 4, 'OutputBasis', 5)

InputCompounding — Compounding frequency of input forward rates

2 (default) | numeric values: 0, 1, 2, 3, 4, 6, 12, 365, -1

Compounding frequency of input forward rates, specified with allowed values:

- 0 — Simple interest (no compounding)
- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- 365 — Daily compounding
- -1 — Continuous compounding

Note If InputCompounding is not specified, then InputCompounding is assigned the value specified for OutputCompounding. If either InputCompounding or OutputCompounding are not specified, the default is 2

Data Types: double

InputBasis — Day-count basis of input forward rates

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day count basis of input forward rates, specified as a numeric value. Allowed values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If `InputBasis` is not specified, then `InputBasis` is assigned the value specified for `OutputBasis`. If either `InputBasis` or `OutputBasis` are not specified, the default is 0 (actual/actual) for both.

Data Types: `double`

OutputCompounding — Compounding frequency of output zero rates

2 (default) | numeric values: 0, 1, 2, 3, 4, 6, 12, 365, -1

Compounding frequency of output zero rates, specified with the allowed values:

- 0 — Simple interest (no compounding)
- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- 365 — Daily compounding
- -1 — Continuous compounding

Note If `OutputCompounding` is not specified, then `OutputCompounding` is assigned the value specified for `InputCompounding`. If either `InputCompounding` or `OutputCompounding` are not specified, the default is 2 (semiannual) for both.

Data Types: double

OutputBasis — Day-count basis of output zero rates

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day count basis of output zero rates, specified as a numeric value. Allowed values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If `OutputBasis` is not specified, then `OutputBasis` is assigned the value specified for `InputBasis`. If either `InputBasis` or `OutputBasis` are not specified, the default is 0 (actual/actual) for both.

Data Types: double

Output Arguments

ZeroRates — Zero curve for investment horizon represented by CurveDates

numeric

Zero curve for the investment horizon represented by `CurveDates`, returned as a `NUMBONDS-by-1` vector of decimal fractions. In aggregate, the rates in `ZeroRates` constitute a zero curve for the investment horizon represented by `CurveDates`.

CurveDates — Maturity dates that correspond to ZeroRates

datetime | serial date number

Maturity dates that correspond to the `ZeroRates`, returned as a `NUMBONDS`-by-1 vector of maturity dates that correspond to the zero rates in `ZeroRates`. This vector is the same as the input vector `CurveDates`, but is sorted by ascending maturity.

If both inputs for `CurveDates` and `Settle` are strings or date character vectors, `CurveDates` is returned as serial date numbers. Use the function `datetime` to convert serial date numbers to formatted datetime arrays. However, if either of the inputs for `CurveDates` and `Settle` are a datetime array, `CurveDates` is returned as a datetime array.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `fwd2zero` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`zero2fwd` | `prbyzero` | `pyld2zero` | `zbtprice` | `zbtyield` | `zero2disc` | `zero2fwd` | `zero2pyld` | `datetime`

Topics

"Term Structure of Interest Rates" on page 2-29

"Fixed-Income Terminology" on page 2-15

geom2arith

Geometric to arithmetic moments of asset returns

Syntax

```
[ma,Ca] = geom2arith(mg,Cg)
[ma,Ca] = geom2arith( ___,t)
```

Description

[ma,Ca] = geom2arith(mg,Cg) transforms moments associated with a continuously compounded geometric Brownian motion into equivalent moments associated with a simple Brownian motion with a possible change in periodicity.

[ma,Ca] = geom2arith(___,t) adds an optional argument t.

Examples

Obtain Geometric to Arithmetic Moments of Asset Returns

This example shows several variations of using geom2arith.

Given geometric mean m and covariance C of monthly total returns, obtain annual arithmetic mean ma and covariance Ca. In this case, the output period (1 year) is 12 times the input period (1 month) so that the optional input t = 12.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
[ma, Ca] = geom2arith(m, C, 12)
```

ma = 4×1

```
    0.5508
    1.0021
    1.0906
    1.4802
```

Ca = 4×4

```
    0.0695    0.0423    0.0196         0
    0.0423    0.2832    0.1971    0.1095
    0.0196    0.1971    0.5387    0.3013
         0    0.1095    0.3013    1.0118
```

Given annual geometric mean m and covariance C of asset returns, obtain monthly arithmetic mean ma and covariance Ca . In this case, the output period (1 month) is $1/12$ times the input period (1 year) so that the optional input $t = 1/12$.

```
[ma, Ca] = geom2arith(m, C, 1/12)
```

```
ma = 4×1
```

```
0.0038  
0.0070  
0.0076  
0.0103
```

```
Ca = 4×4
```

```
0.0005    0.0003    0.0001         0  
0.0003    0.0020    0.0014    0.0008  
0.0001    0.0014    0.0037    0.0021  
         0    0.0008    0.0021    0.0070
```

Given geometric mean m and covariance C of monthly total returns, obtain quarterly arithmetic return moments. In this case, the output is 3 of the input periods so that the optional input $t = 3$.

```
[ma, Ca] = geom2arith(m, C, 3)
```

```
ma = 4×1
```

```
0.1377  
0.2505  
0.2726  
0.3701
```

```
Ca = 4×4
```

```
0.0174    0.0106    0.0049         0  
0.0106    0.0708    0.0493    0.0274  
0.0049    0.0493    0.1347    0.0753  
         0    0.0274    0.0753    0.2530
```

Input Arguments

mg — Continuously compounded or geometric mean of asset returns

vector

Continuously compounded or geometric mean of asset returns, specified as an n -vector.

Data Types: `double`

Cg — Continuously compounded or geometric covariance of asset returns

matrix

Continuously compounded or geometric covariance of asset returns, specified as an n -by- n symmetric, positive semidefinite matrix. If C_g is not a symmetric positive semidefinite matrix, use `nearcorr` to create a positive semidefinite matrix for a correlation matrix.

Data Types: `double`

t — Target period of arithmetic moments in terms of periodicity of geometric moments

1 (default) | scalar positive numeric

(Optional) Target period of geometric moments in terms of periodicity of arithmetic moments, specified as a scalar positive numeric.

Data Types: `double`

Output Arguments

ma — Arithmetic mean of asset returns over the target period

vector

Arithmetic mean of asset returns over the target period (t), returned as an n -vector.

Ca — Arithmetic covariance of asset returns over the target period

matrix

Arithmetic covariance of asset returns over the target period (t), returned as an n -by- n matrix.

Algorithms

Geometric returns over period t_G are modeled as multivariate lognormal random variables with moments

$$E[Y] = 1 + m_G$$

and

$$\text{cov}(Y) = C_G$$

Arithmetic returns over period t_A are modeled as multivariate normal random variables with moments

$$E[X] = m_A$$

$$\text{cov}(X) = C_A$$

Given $t = t_A / t_G$, the transformation from geometric to arithmetic moments is

$$C_{Aij} = t \log \left(1 + \frac{C_{Gij}}{(1 + m_{G_i})(1 + m_{G_j})} \right)$$

$$m_{A_i} = t \log(1 + m_{G_i}) - \frac{1}{2} C_{Aii}$$

For $i, j = 1, \dots, n$.

Note If $t = 1$, then $\mathbf{X} = \log(\mathbf{Y})$.

This function requires that the input mean must satisfy $1 + m_g > 0$ and that the input covariance C_g must be a symmetric, positive, semidefinite matrix.

The functions `geom2arith` and `arith2geom` are complementary so that, given m , C , and t , the sequence

```
[ma,Ca] = geom2arith(m,C,t);  
[mg,Cg] = arith2geom(ma,Ca,1/t);
```

yields $m_g = m$ and $C_g = C$.

Version History

Introduced before R2006a

See Also

`arith2geom` | `nearcorr`

getAssetMoments

Obtain mean and covariance of asset returns from Portfolio object

Syntax

```
[AssetMean,AssetCovar] = getAssetMoments(obj)
```

Description

Use the `getAssetMoments` function with a `Portfolio` object to obtain mean and covariance of asset returns.

For details on the workflow, see “Portfolio Object Workflow” on page 4-18.

`[AssetMean,AssetCovar] = getAssetMoments(obj)` obtains mean and covariance of asset returns for a `Portfolio` object.

Examples

Obtain Asset Moment Properties for a Portfolio Object

Given the mean and covariance of asset returns in the variables `m` and `C`, the asset moment properties can be set and then obtained using the `getAssetMoments` function:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

p = Portfolio;
p = setAssetMoments(p, m, C);
[assetmean, assetcovar] = getAssetMoments(p)
```

```
assetmean = 4×1
```

```
0.0042
0.0083
0.0100
0.0150
```

```
assetcovar = 4×4
```

```
0.0005    0.0003    0.0002         0
0.0003    0.0024    0.0017    0.0010
0.0002    0.0017    0.0048    0.0028
         0    0.0010    0.0028    0.0102
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a `Portfolio` object. For more information on creating a portfolio object, see

- `Portfolio`

Data Types: `object`

Output Arguments

AssetMean — Mean of asset returns

vector

Mean of asset returns, returned as a vector.

AssetCovar — Covariance of asset returns

matrix

Covariance of asset returns, returned as a matrix.

Tips

You can also use dot notation to obtain the mean and covariance of asset returns from a `Portfolio` object:

```
[AssetMean, AssetCovar] = obj.getAssetMoments;
```

Version History

Introduced in R2011a

See Also

`setAssetMoments`

Topics

“Asset Returns and Moments of Asset Returns Using Portfolio Object” on page 4-42

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

“Portfolio Optimization Theory” on page 4-4

getBounds

Obtain bounds for portfolio weights from portfolio object

Syntax

```
[LowerBound,UpperBound] = getBounds(obj)
```

Description

Use the `getBounds` function with a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object to obtain bounds for portfolio weights from portfolio objects.

For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`[LowerBound,UpperBound] = getBounds(obj)` obtains bounds for portfolio weights from portfolio objects.

Examples

Obtain Values for Lower and Upper Bounds for a Portfolio Object

Given portfolio `p` with the default constraints set, obtain the values for `LowerBound` and `UpperBound`.

```
p = Portfolio;
p = setDefaultConstraints(p, 5);
[LowerBound, UpperBound] = getBounds(p)
```

```
LowerBound = 5×1
```

```
0
0
0
0
0
```

```
UpperBound =
```

```
[]
```

Obtain Values for Lower and Upper Bounds for a PortfolioCVaR Object

Given a `PortfolioCVaR` object `p` with the default constraints set, obtain the values for `LowerBound` and `UpperBound`.

```
p = PortfolioCVaR;  
p = setDefaultConstraints(p, 5);  
[LowerBound, UpperBound] = getBounds(p)
```

```
LowerBound = 5×1
```

```
0  
0  
0  
0  
0
```

```
UpperBound =
```

```
[]
```

Obtain Values for Lower and Upper Bounds for a PortfolioMAD Object

Given a PortfolioMAD object `p` with the default constraints set, obtain the values for `LowerBound` and `UpperBound`.

```
p = PortfolioMAD;  
p = setDefaultConstraints(p, 5);  
[LowerBound, UpperBound] = getBounds(p)
```

```
LowerBound = 5×1
```

```
0  
0  
0  
0  
0
```

```
UpperBound =
```

```
[]
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: object

Output Arguments

LowerBound — Lower-bound weight for each asset

vector

Lower-bound weight for each asset, returned as a vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`). For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

UpperBound — Upper-bound weight for each asset

vector

Upper-bound weight for each asset, returned as a vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`). For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

More About

Upper and Lower Bounds

Upper and lower bounds in portfolio optimization are critical constraints that help define the limits of asset allocation, ensuring that the resulting portfolio aligns with the investor's goals for risk tolerance.

Tips

You can also use dot notation to obtain bounds for portfolio weights from portfolio objects.

```
[LowerBound, UpperBound] = obj.getBounds;
```

Version History

Introduced in R2011a

See Also

`setBounds`

Topics

“Working with ‘Simple’ Bound Constraints Using Portfolio Object” on page 4-62

“Working with ‘Simple’ Bound Constraints Using PortfolioCVaR Object” on page 5-55

“Working with ‘Simple’ Bound Constraints Using PortfolioMAD Object” on page 6-54

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

getBudget

Obtain budget constraint bounds from portfolio object

Syntax

```
[LowerBudget,UpperBudget] = getBudget(obj)
```

Description

Use the `getBudget` function with a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object to obtain budget constraint bounds from portfolio objects.

For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`[LowerBudget,UpperBudget] = getBudget(obj)` obtains budget constraint bounds from portfolio objects.

Examples

Obtain Values for Lower and Upper Budgets for a Portfolio Object

Given portfolio `p` with the default constraints set, obtain the values for `LowerBudget` and `UpperBudget`.

```
p = Portfolio;
p = setDefaultConstraints(p, 5);
[LowerBudget, UpperBudget] = getBudget(p)
```

```
LowerBudget =
1
```

```
UpperBudget =
1
```

Obtain Values for Lower and Upper Budgets for a PortfolioCVaR Object

Given a `PortfolioCVaR` object `p` with the default constraints set, obtain the values for `LowerBudget` and `UpperBudget`.

```
p = PortfolioCVaR;
p = setDefaultConstraints(p, 5);
[LowerBudget, UpperBudget] = getBudget(p)
```

```
LowerBudget =
1
```

```
UpperBudget =  
1
```

Obtain Values for Lower and Upper Budgets for a PortfolioMAD Object

Given a PortfolioMAD object `p` with the default constraints set, obtain the values for `LowerBudget` and `UpperBudget`.

```
p = PortfolioMAD;  
p = setDefaultConstraints(p, 5);  
[LowerBudget, UpperBudget] = getBudget(p)
```

```
LowerBudget =  
1
```

```
UpperBudget =  
1
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

Output Arguments

LowerBudget — Lower-bound weight for each asset

scalar

Lower bound for budget constraint, returned as a scalar for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

UpperBudget — Upper bound for budget constraint

scalar

Upper bound for budget constraint, returned as a scalar for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

More About

Budget Constraint Bounds

Budget constraint bounds are the limitations placed on the total amount of capital that can be allocated to different assets within a portfolio.

Budget constraint bounds ensure that the total investment does not exceed the available capital and that the portfolio adheres to the investor's financial guidelines.

Tips

You can also use dot notation to obtain the budget constraint bounds from portfolio objects.

```
[LowerBudget, UpperBudget] = obj.getBudget;
```

Version History

Introduced in R2011a

See Also

setBudget | setConditionalBudget

Topics

“Working with Budget Constraints Using Portfolio Object” on page 4-65

“Working with Budget Constraints Using PortfolioCVaR Object” on page 5-58

“Working with Budget Constraints Using PortfolioMAD Object” on page 6-57

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

getCosts

Obtain buy and sell transaction costs from portfolio object

Syntax

```
[BuyCost,SellCost] = getCosts(obj)
```

Description

Use the `getCosts` function with a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object to obtain buy and sell transaction costs from portfolio objects.

For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`[BuyCost,SellCost] = getCosts(obj)` obtains buy and sell transaction costs from portfolio objects.

Examples

Obtain Buy and Sell Costs for a Portfolio Object

Given portfolio `p` with the costs set, obtain the values for `BuyCost` and `SellCost`.

```
p = Portfolio;  
p = setCosts(p, 0.001, 0.001, 5);  
[BuyCost, SellCost] = getCosts(p)
```

```
BuyCost =  
1.0000e-03
```

```
SellCost =  
1.0000e-03
```

Obtain Buy and Sell Costs for a PortfolioCVaR Object

Given a `PortfolioCVaR` object `p` with the costs set, obtain the values for `BuyCost` and `SellCost`.

```
p = PortfolioCVaR;  
p = setCosts(p, 0.001, 0.001, 5);  
[BuyCost, SellCost] = getCosts(p)
```

```
BuyCost =  
1.0000e-03
```

```
SellCost =  
1.0000e-03
```

Obtain Buy and Sell Costs for a PortfolioMAD Object

Given a PortfolioMAD object `p` with the costs set, obtain the values for `BuyCost` and `SellCost`.

```
p = PortfolioMAD;
p = setCosts(p, 0.001, 0.001, 5);
[BuyCost, SellCost] = getCosts(p)
```

```
BuyCost =
1.0000e-03
```

```
SellCost =
1.0000e-03
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

Output Arguments

BuyCost — Proportional transaction cost to purchase each asset

vector

Proportional transaction cost to purchase each asset, returned as a vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

SellCost — Proportional transaction cost to sell each asset

vector

Proportional transaction cost to sell each asset, returned as a vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

More About

Transaction Costs

Transaction costs are the expenses incurred when buying or selling securities within an investment portfolio.

Transaction costs can significantly impact the overall performance of an investment strategy, and understanding them is crucial for effective portfolio management.

Tips

You can also use dot notation to obtain the buy and sell transaction costs from portfolio objects.

```
[BuyCost, SellCost] = obj.getCosts;
```

Version History

Introduced in R2011a

See Also

setCosts

Topics

“Working with Transaction Costs” on page 4-54

“Working with Transaction Costs” on page 5-47

“Working with Transaction Costs” on page 6-46

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

getEquality

Obtain equality constraint arrays from portfolio object

Syntax

```
[AEquality,bEquality] = getEquality(obj)
```

Description

Use the `getEquality` function with a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object to obtain equality constraint arrays from portfolio objects.

For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`[AEquality,bEquality] = getEquality(obj)` obtains equality constraint arrays from portfolio objects.

Examples

Obtain Equality Constraints for a Portfolio Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets are exactly 50% of your portfolio. Given a `Portfolio` object `p`, set the linear equality constraints and obtain the values for `AEquality` and `bEquality`:

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = Portfolio;
p = setEquality(p, A, b);
[AEquality, bEquality] = getEquality(p)
```

```
AEquality = 1×5
```

```
1      1      1      0      0
```

```
bEquality =
0.5000
```

Obtain Equality Constraints for a PortfolioCVaR Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets are 50% of your portfolio. Given a `PortfolioCVaR` object `p`, set the linear equality constraints and obtain the values for `AEquality` and `bEquality`:

```
A = [ 1 1 1 0 0 ];
b = 0.5;
```

```
p = PortfolioCVaR;  
p = setEquality(p, A, b);  
[AEquality, bEquality] = getEquality(p)
```

```
AEquality = 1×5
```

```
    1    1    1    0    0
```

```
bEquality =  
0.5000
```

Obtain Equality Constraints for a PortfolioMAD Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets are 50% of your portfolio. Given a PortfolioMAD object `p`, set the linear equality constraints and obtain the values for `AEquality` and `bEquality`:

```
A = [ 1 1 1 0 0 ];  
b = 0.5;  
p = PortfolioMAD;  
p = setEquality(p, A, b);  
[AEquality, bEquality] = getEquality(p)
```

```
AEquality = 1×5
```

```
    1    1    1    0    0
```

```
bEquality =  
0.5000
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

Output Arguments

AEquality — Matrix to form linear equality constraints

matrix

Matrix to form linear equality constraints, returned as a matrix for a `Portfolio`, `PortfolioCvAR`, or `PortfolioMAD` input object (obj).

bEquality — Vector to form linear equality constraints

vector

Vector to form linear equality constraints, returned as a vector for a `Portfolio`, `PortfolioCvAR`, or `PortfolioMAD` input object (obj).

More About

Equality Constraint

An equality constraint requires certain conditions to be met exactly, rather than allowing for a range of values.

Tips

You can also use dot notation to obtain the equality constraint arrays from portfolio objects.

```
[AEquality, bEquality] = obj.getEquality;
```

Version History

Introduced in R2011a

See Also

setEquality

Topics

“Working with Linear Equality Constraints Using Portfolio Object” on page 4-75

“Working with Linear Equality Constraints Using PortfolioCvAR Object” on page 5-68

“Working with Linear Equality Constraints Using PortfolioMAD Object” on page 6-67

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCvAR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

getGroupRatio

Obtain group ratio constraint arrays from portfolio object

Syntax

```
[GroupA,GroupB,LowerRatio,UpperRatio] = getGroupRatio(obj)
```

Description

Use the `getGroupRatio` function with a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object to obtain group ratio constraint arrays from portfolio objects.

For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`[GroupA,GroupB,LowerRatio,UpperRatio] = getGroupRatio(obj)` obtains equality constraint arrays from portfolio objects.

Examples

Obtain Group Ratio Constraints for a Portfolio Object

Suppose you want to make sure that the ratio of financial to nonfinancial companies in your portfolios never goes above 50%. Assume you have 6 assets with 3 financial companies (assets 1-3) and 3 nonfinancial companies (assets 4-6). After setting group ratio constraints, obtain the values for `GroupA`, `GroupB`, `LowerRatio`, and `UpperRatio`.

```
GA = [ true true true false false false ];    % financial companies
GB = [ false false false true true true ];    % nonfinancial companies
p = Portfolio;
p = setGroupRatio(p, GA, GB, [], 0.5);
[GroupA, GroupB, LowerRatio, UpperRatio] = getGroupRatio(p)
```

```
GroupA = 1×6
```

```
    1    1    1    0    0    0
```

```
GroupB = 1×6
```

```
    0    0    0    1    1    1
```

```
LowerRatio =
```

```
    []
```

```
UpperRatio =
```

```
    0.5000
```


Obtain Group Ratio Constraints for a PortfolioCVaR Object

Suppose you want to ensure that the ratio of financial to nonfinancial companies in your portfolios never exceeds 50%. Assume you have six assets with three financial companies (assets 1-3) and three nonfinancial companies (assets 4-6). After setting group ratio constraints, obtain the values for GroupA, GroupB, LowerRatio, and UpperRatio.

```
GA = [ true true true false false false ]; % financial companies
GB = [ false false false true true true ]; % nonfinancial companies
p = PortfolioCVaR;
p = setGroupRatio(p, GA, GB, [], 0.5);
[GroupA, GroupB, LowerRatio, UpperRatio] = getGroupRatio(p)
```

```
GroupA = 1×6
```

```
    1    1    1    0    0    0
```

```
GroupB = 1×6
```

```
    0    0    0    1    1    1
```

```
LowerRatio =
```

```
    []
```

```
UpperRatio =
```

```
    0.5000
```

Obtain Group Ratio Constraints for a PortfolioMAD Object

Suppose you want to ensure that the ratio of financial to nonfinancial companies in your portfolios never exceeds 50%. Assume you have six assets with three financial companies (assets 1-3) and three nonfinancial companies (assets 4-6). After setting group ratio constraints, obtain the values for GroupA, GroupB, LowerRatio, and UpperRatio.

```
GA = [ true true true false false false ]; % financial companies
GB = [ false false false true true true ]; % nonfinancial companies
p = PortfolioMAD;
p = setGroupRatio(p, GA, GB, [], 0.5);
[GroupA, GroupB, LowerRatio, UpperRatio] = getGroupRatio(p)
```

```
GroupA = 1×6
```

```
    1    1    1    0    0    0
```

```
GroupB = 1×6
```

```
    0    0    0    1    1    1
```

```
LowerRatio =  
  
    []  
  
UpperRatio =  
0.5000
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: object

Output Arguments

GroupA — Matrix that forms base groups for comparison

matrix

Matrix that forms base groups for comparison, returned as a matrix for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (obj).

GroupB — Matrix that forms comparison groups

matrix

Matrix that forms comparison groups, returned as a matrix `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (obj).

LowerRatio — Lower bound for ratio of GroupB groups to GroupA groups

vector

Lower bound for ratio of GroupB groups to GroupA groups, returned as a vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (obj).

UpperRatio — Upper bound for ratio of GroupB groups to GroupA groups

vector

Upper bound for ratio of GroupB groups to GroupA groups, returned as a vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (obj).

More About

Group Ratio Constraints

Group ratio constraints specify that the total weight or allocation of assets within a particular group must satisfy certain conditions relative to the total portfolio.

Tips

You can also use dot notation to obtain group ratio constraint arrays from portfolio objects.

```
[GroupA, GroupB, LowerRatio, UpperRatio] = obj.getGroupRatio;
```

Version History

Introduced in R2011a

See Also

setGroupRatio

Topics

“Working with Group Ratio Constraints Using Portfolio Object” on page 4-72

“Working with Group Constraints Using PortfolioCVaR Object” on page 5-62

“Working with Group Constraints Using PortfolioMAD Object” on page 6-61

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

getGroups

Obtain group constraint arrays from portfolio object

Syntax

```
[GroupMatrix, LowerGroup, UpperGroup] = getGroups(obj)
```

Description

Use the `getGroups` function with a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object to obtain group constraint arrays from portfolio objects.

For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`[GroupMatrix, LowerGroup, UpperGroup] = getGroups(obj)` obtains group constraint arrays from portfolio objects.

Examples

Obtain Group Constraints for a Portfolio Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets constitute no more than 30% of your portfolio. Given a `Portfolio` object `p` with the group constraints set, obtain the values for `GroupMatrix`, `LowerGroup`, and `UpperGroup`.

```
G = [ true true true false false ];  
p = Portfolio;  
p = setGroups(p, G, [], 0.3);  
[GroupMatrix, LowerGroup, UpperGroup] = getGroups(p)
```

```
GroupMatrix = 1x5
```

```
    1    1    1    0    0
```

```
LowerGroup =
```

```
    []
```

```
UpperGroup =  
    0.3000
```

Obtain Group Constraints for a PortfolioCVaR Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets constitute at most 30% of your portfolio. Given a PortfolioCVaR object `p` with the group constraints set, obtain the values for `GroupMatrix`, `LowerGroup`, and `UpperGroup`.

```
G = [ true true true false false ];
p = PortfolioCVaR;
p = setGroups(p, G, [], 0.3);
[GroupMatrix, LowerGroup, UpperGroup] = getGroups(p)
```

```
GroupMatrix = 1x5
    1    1    1    0    0
```

```
LowerGroup =
    []
```

```
UpperGroup =
    0.3000
```

Obtain Group Constraints for a PortfolioMAD Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets constitute at most 30% of your portfolio. Given a PortfolioMAD object `p` with the group constraints set, obtain the values for `GroupMatrix`, `LowerGroup`, and `UpperGroup`.

```
G = [ true true true false false ];
p = PortfolioMAD;
p = setGroups(p, G, [], 0.3);
[GroupMatrix, LowerGroup, UpperGroup] = getGroups(p)
```

```
GroupMatrix = 1x5
    1    1    1    0    0
```

```
LowerGroup =
    []
```

```
UpperGroup =
    0.3000
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- Portfolio
- PortfolioCVaR
- PortfolioMAD

Data Types: object

Output Arguments

GroupMatrix — Group constraint matrix

matrix

Group constraint matrix, returned as a matrix for a Portfolio, PortfolioCVaR, or PortfolioMAD input object (obj).

LowerGroup — Lower bound for group constraints

vector

Lower bound for group constraints, returned as a vector for a Portfolio, PortfolioCVaR, or PortfolioMAD input object (obj).

UpperGroup — Upper bound for group constraints

vector

Upper bound for group constraints, returned as a vector for a Portfolio, PortfolioCVaR, or PortfolioMAD input object (obj).

More About

Group Constraints

Group constraints specify that the total weight or allocation of assets within a particular group must satisfy certain conditions relative to the total portfolio.

Group constraints help maintain diversification by preventing excessive concentration in specific asset groups. They also align the portfolio with the investor's risk tolerance and investment strategy.

Tips

You can also use dot notation to obtain the group constraint arrays from portfolio objects.

```
[GroupMatrix, LowerGroup, UpperGroup] = obj.getGroups;
```

Version History

Introduced in R2011a

See Also

setGroups

Topics

“Working with Group Constraints Using Portfolio Object” on page 4-69

“Working with Group Constraints Using PortfolioCVaR Object” on page 5-62
“Working with Group Constraints Using PortfolioMAD Object” on page 6-61
“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9
Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8
Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

getScenarios

Obtain scenarios from portfolio object

Syntax

```
Y = getScenarios(obj)
```

Description

Use the `getScenarios` function with a `PortfolioCVaR` or `PortfolioMAD` objects to obtain scenarios.

For details on the workflows, see “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`Y = getScenarios(obj)` obtains scenarios for `PortfolioCVaR` or `PortfolioMAD` objects.

Examples

Obtain Scenarios for a CVaR Portfolio Object

For a given `PortfolioCVaR` object `p`, display the defined scenarios.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 10);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

Y = getScenarios(p)

Y = 10×4

-0.0056    0.0440    0.1186    0.0488
-0.0368   -0.0753    0.0087    0.1124
 0.0025    0.0856    0.0484    0.1404
 0.0318    0.0826    0.0377    0.0404
 0.0013   -0.0561   -0.1466   -0.0621
 0.0035    0.0310   -0.0183    0.1225
```



```

-0.0519  -0.1634  -0.0526   0.1528
 0.0029  -0.1163  -0.0627  -0.0760
 0.0192  -0.0182  -0.1243  -0.1346
 0.0440   0.0189   0.0098   0.0821

```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Obtain Scenarios for a MAD Portfolio Object

For a given PortfolioMAD object `p`, display the defined scenarios.

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

rng(11);

AssetScenarios = mvnrnd(m, C, 10);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

Y = getScenarios(p)

Y = 10x4

-0.0056   0.0440   0.1186   0.0488
-0.0368  -0.0753   0.0087   0.1124
 0.0025   0.0856   0.0484   0.1404
 0.0318   0.0826   0.0377   0.0404
 0.0013  -0.0561  -0.1466  -0.0621
 0.0035   0.0310  -0.0183   0.1225
-0.0519  -0.1634  -0.0526   0.1528
 0.0029  -0.1163  -0.0627  -0.0760
 0.0192  -0.0182  -0.1243  -0.1346
 0.0440   0.0189   0.0098   0.0821

```

The function `rng(seed)` resets the random number generator to produce the documented results. It is not necessary to reset the random number generator to simulate scenarios.

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a PortfolioCVaR or PortfolioMAD object.

For more information on creating a `PortfolioCVaR` or `PortfolioMAD` object, see

- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

Output Arguments

Y — Scenarios matrix

`matrix`

Scenarios matrix, returned as a `NumScenarios`-by-`NumAssets` matrix for a `PortfolioCVaR` or `PortfolioMAD` object.

Tips

You can also use dot notation to obtain scenarios from a `PortfolioCVaR` or `PortfolioMAD` object.

```
Y = obj.getScenarios;
```

Version History

Introduced in R2012b

See Also

`setScenarios` | `rng`

Topics

“Asset Returns and Scenarios Using `PortfolioCVaR` Object” on page 5-36

“Asset Returns and Scenarios Using `PortfolioMAD` Object” on page 6-35

External Websites

Getting Started with Portfolio Optimization (4 min 13 sec)

CVaR Portfolio Optimization (4 min 56 sec)

getInequality

Obtain inequality constraint arrays from portfolio object

Syntax

```
[AInequality,bInequality] = getInequality(obj)
```

Description

Use the `getInequality` function with a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object to obtain inequality constraint arrays from portfolio objects.

For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`[AInequality,bInequality] = getInequality(obj)` obtains equality constraint arrays from portfolio objects.

Examples

Obtain Inequality Constraints for a Portfolio Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets are no more than 50% of your portfolio. Given a `Portfolio` object `p`, set the linear inequality constraints and then obtain values for `AInequality` and `bInequality`.

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = Portfolio;
p = setInequality(p, A, b);
[AInequality, bInequality] = getInequality(p)
```

```
AInequality = 1x5
```

```
    1    1    1    0    0
```

```
bInequality =
0.5000
```

Obtain Inequality Constraints for a PortfolioCVaR Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets constitute at most 50% of your portfolio. Given a `PortfolioCVaR` object `p`, set the linear inequality constraints and then obtain values for `AInequality` and `bInequality`.

```
A = [ 1 1 1 0 0 ];
b = 0.5;
```

```
p = PortfolioCVaR;
p = setInequality(p, A, b);
[AInequality, bInequality] = getInequality(p)

AInequality = 1×5

    1    1    1    0    0

bInequality =
0.5000
```

Obtain Inequality Constraints for a PortfolioMAD Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets constitute at most 50% of your portfolio. Given a PortfolioMAD object `p`, set the linear inequality constraints and then obtain values for `AInequality` and `bInequality`.

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = PortfolioMAD;
p = setInequality(p, A, b);
[AInequality, bInequality] = getInequality(p)

AInequality = 1×5

    1    1    1    0    0

bInequality =
0.5000
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

Output Arguments

AInequality — Matrix to form linear inequality constraints

matrix

Matrix to form linear inequality constraints, returned as a matrix for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (obj).

bInequality — Vector to form linear inequality constraints

vector

Vector to form linear inequality constraints, returned as a vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (obj).

More About

Inequality Constraint

An inequality constraint specifies a range of acceptable values rather than requiring an exact value.

Use inequality constraints to impose limits on certain parameters of the portfolio to ensure that the portfolio adheres to specific guidelines or preferences set by the investor or portfolio manager.

Tips

You can also use dot notation to obtain the inequality constraint arrays from portfolio objects.

```
[AInequality, bInequality] = obj.getInequality;
```

Version History

Introduced in R2011a

See Also

`setInequality`

Topics

“Working with Linear Inequality Constraints Using Portfolio Object” on page 4-78

“Working with Linear Inequality Constraints Using PortfolioCVaR Object” on page 5-70

“Working with Linear Inequality Constraints Using PortfolioMAD Object” on page 6-69

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

getOneWayTurnover

Obtain one-way turnover constraints from portfolio object

Syntax

```
[BuyTurnover,SellTurnover] = getOneWayTurnover(obj)
```

Description

Use the `getOneWayTurnover` function with a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object to obtain one-way turnover constraints from portfolio objects.

For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

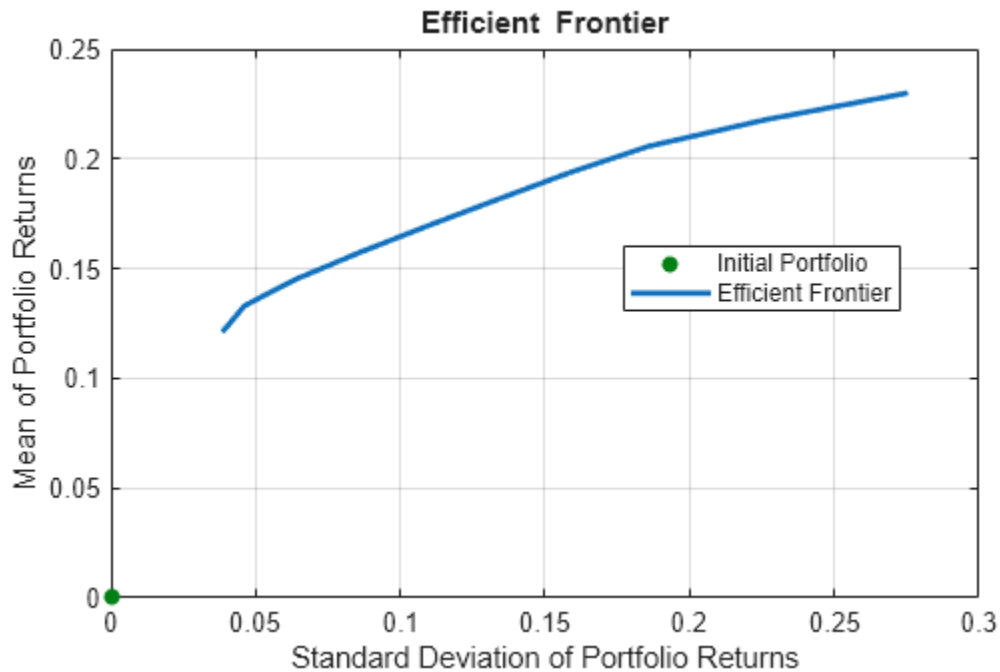
`[BuyTurnover,SellTurnover] = getOneWayTurnover(obj)` obtain one-way turnover constraints from portfolio objects.

Examples

Obtain One-Way Turnover Costs for a Portfolio Object

Set one-way turnover costs.

```
p = Portfolio('AssetMean',[0.1, 0.2, 0.15], 'AssetCovar',...  
[ 0.005, -0.010, 0.004; -0.010, 0.040, -0.002; 0.004, -0.002, 0.023]);  
p = setBudget(p, 1, 1);  
p = setOneWayTurnover(p, 1.3, 0.3, 0);    %130-30 portfolio  
plotFrontier(p);
```



Obtain one-way turnover costs.

```
[BuyTurnover, SellTurnover] = getOneWayTurnover(p)
```

```
BuyTurnover =  
1.3000
```

```
SellTurnover =  
0.3000
```

Obtain One-Way Turnover Costs for a PortfolioCVaR Object

Set one-way turnover costs and obtain the buy and sell turnover values.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];  
C = [ 0.0064 0.00408 0.00192 0;  
      0.00408 0.0289 0.0204 0.0119;  
      0.00192 0.0204 0.0576 0.0336;  
      0 0.0119 0.0336 0.1225 ];  
m = m/12;  
C = C/12;  
  
AssetScenarios = mvnrnd(m, C, 20000);  
  
p = PortfolioCVaR;  
p = setScenarios(p, AssetScenarios);  
p = setDefaultConstraints(p);  
p = setProbabilityLevel(p, 0.95);  
  
p = setBudget(p, 1, 1);  
p = setOneWayTurnover(p, 1.3, 0.3, 0); %130-30 portfolio
```

```
[BuyTurnover,SellTurnover] = getOneWayTurnover(p)
```

```
BuyTurnover =  
1.3000
```

```
SellTurnover =  
0.3000
```

Obtain One-Way Turnover Costs for a PortfolioMAD Object

Set one-way turnover costs and obtain the buy and sell turnover values.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];  
C = [ 0.0064 0.00408 0.00192 0;  
      0.00408 0.0289 0.0204 0.0119;  
      0.00192 0.0204 0.0576 0.0336;  
      0 0.0119 0.0336 0.1225 ];  
m = m/12;  
C = C/12;  
  
AssetScenarios = mvnrnd(m, C, 20000);  
  
p = PortfolioMAD;  
p = setScenarios(p, AssetScenarios);  
p = setDefaultConstraints(p);  
  
p = setBudget(p, 1, 1);  
p = setOneWayTurnover(p, 1.3, 0.3, 0); %130-30 portfolio  
  
[BuyTurnover,SellTurnover] = getOneWayTurnover(p)  
  
BuyTurnover =  
1.3000  
  
SellTurnover =  
0.3000
```

Input Arguments

obj — Object for portfolio
object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: object

Output Arguments

BuyTurnover — Turnover constraint on purchases

scalar

Turnover constraint on purchases, returned as a scalar for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

SellTurnover — Turnover constraint on sales

scalar

Turnover constraint on sales, returned as a scalar for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

More About

One-way Turnover Constraint

One-way turnover constraint is a specific measure of trading activity in a portfolio that captures the extent to which assets are bought or sold, but only considers the purchases or sales in one direction (either buys or sells).

One-way turnover constraints ensure that estimated optimal portfolios differ from an initial portfolio by no more than specified amounts according to whether the differences are purchases or sales.

The constraints take the form

$$1^T \max\{0, x - x_0\} \leq \tau_B$$

$$1^T \max\{0, x_0 - x\} \leq \tau_S$$

with

- x — The portfolio (*NumAssets* vector)
- x_0 — Initial portfolio (*NumAssets* vector)
- τ_B — Upper-bound for turnover constraint on purchases (scalar)
- τ_S — Upper-bound for turnover constraint on sales (scalar)

Specify one-way turnover constraints using the following properties in a supported portfolio object: `BuyTurnover` for τ_B , `SellTurnover` for τ_S , and `InitPort` for x_0 .

Note The average turnover constraint (which is set using `setTurnover`) is not just the combination of the one-way turnover constraints with the same value for the constraint.

Tips

You can also use dot notation to get the one-way turnover constraint for portfolio objects.

```
[BuyTurnover, SellTurnover] = obj.getOneWayTurnover
```

Version History

Introduced in R2011a

See Also

setOneWayTurnover | setTurnover

Topics

"Working with One-Way Turnover Constraints Using Portfolio Object" on page 4-88

"Working with One-Way Turnover Constraints Using PortfolioCVaR Object" on page 5-78

"Working with One-Way Turnover Constraints Using PortfolioMAD Object" on page 6-77

"Portfolio Optimization Examples Using Financial Toolbox" on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

hhigh

Highest high

Syntax

```
values = hhigh(Data)
values = hhigh( ___,Name,Value)
```

Description

`values = hhigh(Data)` generates a vector of highest high values from the series of high prices for the past n periods.

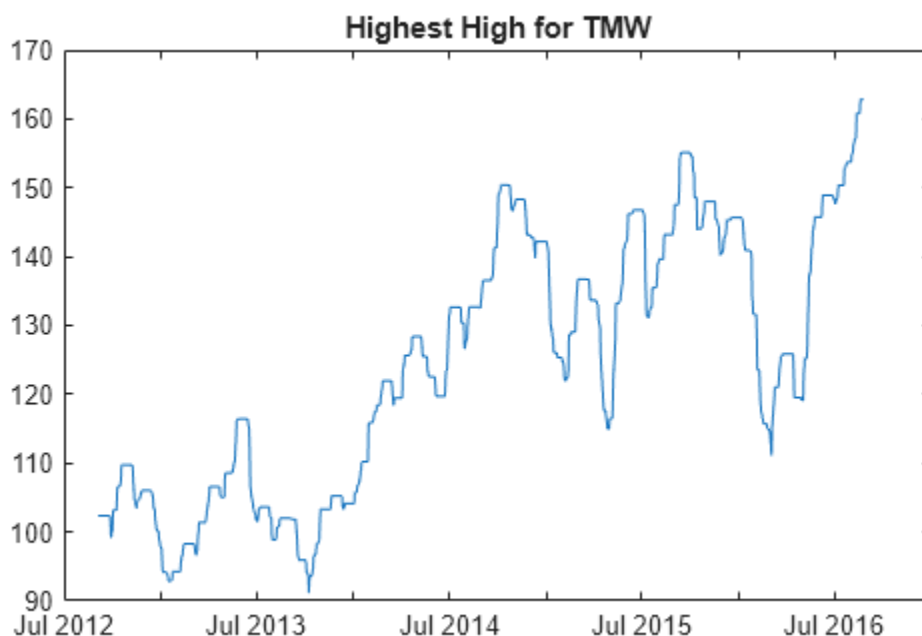
`values = hhigh( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Calculate the Highest High for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat
values = hhigh(TMW);
plot(values.Time,values.HighestHigh)
title('Highest High for TMW')
```



Input Arguments

Data — Data for high prices

matrix | table | timetable

Data for high prices, specified as a matrix, table, or timetable. Timetables and tables with *M* rows must contain a variable named 'High' (case insensitive).

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as *Name1=Value1, ..., NameN=ValueN*, where *Name* is the argument name and *Value* is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `values = hhigh(TMW_HIGH, 'NumPeriods', 10)`

NumPeriods — Moving window for the highest high calculation

14 (default) | positive integer

Moving window for the highest high calculation, specified as the comma-separated pair consisting of 'NumPeriods' and a scalar positive integer.

Data Types: double

Output Arguments

values — Highest high series

matrix | table | timetable

Highest high series, returned with the same number of rows (*M*) and the same type (matrix, table, or timetable) as the input *Data*.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the *Data* input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The *Data* input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995.

See Also

timetable | table | llow

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

highlow

High, low, open, close chart

Syntax

```
highlow(Data)
highlow(Data,Color)
h = highlow(ax,___)
```

Description

`highlow(Data)` displays a highlow chart from a series of opening, high, low, and closing prices of a security. The plots are vertical lines whose top is the high, bottom is the low, open is a left tick, and close is a right tick.

`highlow(Data,Color)` adds an optional argument for `Color`.

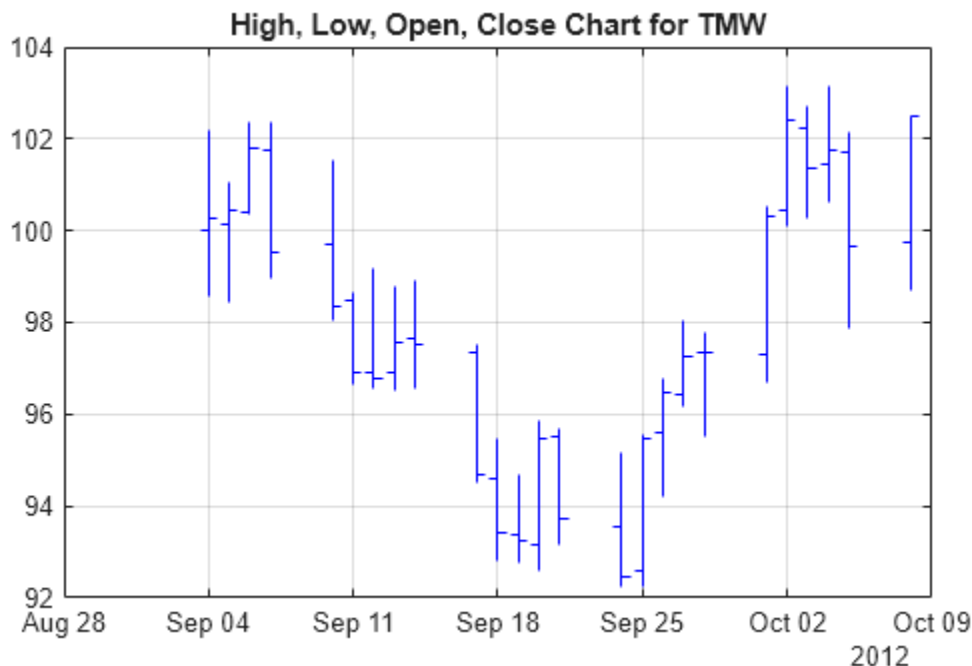
`h = highlow(ax,___)` adds an optional argument for `ax`.

Examples

Generate a Highlow Chart for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock. The highlow chart plots the price data using blue lines.

```
load SimulatedStock.mat
range = 1:25;
highlow(TMW(range,:), 'b');
title('High, Low, Open, Close Chart for TMW')
```



Input Arguments

Data — Data for opening, high, low, and closing prices

matrix | table | timetable

Data for opening, high, low, and closing prices, specified as a matrix, table, or timetable. For matrix input, **Data** is an M-by-4 matrix of opening, high, low, and closing prices stored in the corresponding columns. Timetables and tables with M rows must contain variables named 'Open', 'High', 'Low', and 'Close' (case insensitive).

Data Types: double | table | timetable

Color — Three element color vector

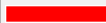
background color of figure window (default) | color vector [R G B] | string

(Optional) Three element color vector, specified as a [R G B] color vector or a string specifying the color name. The default color differs depending on the background color of the figure window.

RGB triplets and hexadecimal color codes are useful for specifying custom colors.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range [0, 1]; for example, [0.4 0.6 0.7].
- A hexadecimal color code is a character vector or a string scalar that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Thus, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and hexadecimal color codes.

Color Name	Short Name	RGB Triplet	Hexadecimal Color Code	Appearance
"red"	"r"	[1 0 0]	"#FF0000"	
"green"	"g"	[0 1 0]	"#00FF00"	
"blue"	"b"	[0 0 1]	"#0000FF"	
"cyan"	"c"	[0 1 1]	"#00FFFF"	
"magenta"	"m"	[1 0 1]	"#FF00FF"	
"yellow"	"y"	[1 1 0]	"#FFFF00"	
"black"	"k"	[0 0 0]	"#000000"	
"white"	"w"	[1 1 1]	"#FFFFFF"	

This table lists the default color palettes for plots in the light and dark themes.

Palette	Palette Colors
"gem" — Light theme default Most plots use these colors by default.	
"glow" — Dark theme default	

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Data Types: `double` | `string`

ax — Valid axis object

current axes (`ax = gca`) (default) | axes object

(Optional) Valid axis object, specified as an axes object. The highlow plot is created in the axes specified by `ax` instead of in the current axes (`ax = gca`). The option `ax` can precede any of the input argument combinations.

Data Types: `object`

Output Arguments

h — Graphic handle of the figure

handle object

Graphic handle of the figure, returned as a handle object.

Version History

Introduced before R2006a

R2022b: Support for negative price data

The Data input accepts negative prices.

R2018a: Support for matrix, time, or timetable input for data

Behavior changed in R2018a

highlow accepts Data input as a matrix, timetable, or table.

See Also

timetable | table | movavg | pointfig | kagi | linebreak | priceandvol | renko | volarea | candle

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

holdings2weights

Portfolio holdings into weights

Syntax

```
Weights = holdings2weights(Holdings,Prices)
Weights = holdings2weights( ___,Budget)
```

Description

`Weights = holdings2weights(Holdings,Prices)` converts portfolio holdings into portfolio weights. The weights must satisfy a budget constraint such that the weights sum to `Budget` for each portfolio.

`Weights = holdings2weights( ___,Budget)` converts portfolio holdings into portfolio weights using an optional argument for `Budget`.

Input Arguments

Holdings — Holdings value

matrix

Holdings value, returned as an NPORTS-by-NASSETS matrix that contains the holdings of NPORTS portfolios that contain NASSETS assets.

Note

- `Holdings` may be negative to indicate a short position, but the overall portfolio weights must satisfy a nonzero budget constraint.
 - The weights in each portfolio sum to the `Budget` value (which is 1 if `Budget` is unspecified.)
-

Data Types: double

Prices — Asset prices

vector

Asset prices, specified as an NASSETS vector.

Data Types: double

Budget — Budget constraints

1 (default) | scalar numeric | vector

(Optional) Budget constraints, specified as a scalar or an NPORTS vector of nonzero budget constraints.

Data Types: double

Output Arguments

Weights — Portfolio weights

matrix

Portfolio weights, returned as an NPORTS-by-NASSETS matrix containing the normalized weights of NPORTS portfolios containing NASSETS assets.

Data Types: double

Version History

Introduced before R2006a

See Also

weights2holdings

holidays

Holidays and nontrading days

Syntax

```
H = holidays
H = holidays(StartDate,EndDate)
H = holidays( ___,AltHolidays)
```

Description

`H = holidays` returns a vector or datetime array corresponding to all holidays and nontrading days.

`H = holidays(StartDate,EndDate)` returns a vector or datetime array corresponding to the holidays and nontrading days between `StartDate` and `EndDate`, inclusive.

`H = holidays( ___,AltHolidays)` returns a vector or datetime array corresponding to the alternate list of holidays and nontrading days.

Examples

Determine Holidays for a Given StartDate and EndDate

Create a vector of serial date numbers corresponding to all holidays and nontrading dates between a specified `StartDate` and `EndDate`:

```
H = holidays('jan 1 2001', 'jun 23 2001')
H = 5×1
```

```
730852
730866
730901
730954
730999
```

```
datestr(H)
```

```
ans = 5×11 char array
'01-Jan-2001'
'15-Jan-2001'
'19-Feb-2001'
'13-Apr-2001'
'28-May-2001'
```

Alternatively, using a datetime array for `StartDate` and `EndDate` returns a datetime array for `H`.

```
H = holidays(datetime(2001,1,1),datetime(2001,6,23))
```

```
H = 5×1 datetime
    01-Jan-2001
    15-Jan-2001
    19-Feb-2001
    13-Apr-2001
    28-May-2001
```

Input Arguments

StartDate — Start date

datetime array | string array | date character vector

Start date, specified using a scalar or vector using a datetime array, string array, or date character vectors.

To support existing code, `holidays` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

EndDate — End date

datetime array | string array | date character vector

End date, specified scalar or vector using a datetime array, string array, or date character vectors.

To support existing code, `holidays` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

AltHolidays — Alternate list of holidays and nontrading days

datetime array | string array | date character vector

Alternate list of holidays and nontrading days, specified scalar or vector using a datetime array, string array, or date character vectors.

To support existing code, `holidays` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Output Arguments

H — Dates corresponding to all holidays and nontrading days

vector | datetime

Dates corresponding to all holidays and nontrading days, returned as a vector or a datetime array of dates.

Note If `StartDate`, `EndDate`, and `AltHolidays` are all either strings or date character vectors, `H` is returned as serial date numbers. Use the function `datetime` to convert serial date numbers to formatted datetime arrays. If either `StartDate`, `EndDate`, or `AltHolidays` are datetime arrays, `H` is returned as a datetime array.

More About

holidays

The `holidays` function is based on a modern five-day workweek.

This function contains all holidays and special nontrading days for the New York Stock Exchange from January 1, 1885 to December 31, 2070.

Since the New York Stock Exchange was open on Saturdays before September 29, 1952, exact closures from 1885 to 2070 include Saturday trading days. To capture these dates, use the function `nyseclosures`. The results from `holidays` and `nyseclosures` are identical if the `WorkWeekFormat` in `nyseclosures` is `'Modern'`.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `holidays` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`busdate` | `createholidays` | `isbusday` | `lbusdate` | `nyseclosures` | `datetime`

Topics

“Handle and Convert Dates” on page 2-2

“Trading Calendars User Interface” on page 12-2

“UICalendar User Interface” on page 12-4

inforatio

Calculate information ratio for one or more assets

Syntax

```
inforatio(Asset,Benchmark)
[Ratio,TE] = inforatio(Asset,Benchmark)
```

Description

`inforatio(Asset,Benchmark)` computes the information ratio for each asset relative to the Benchmark.

`[Ratio,TE] = inforatio(Asset,Benchmark)` computes the information ratio and tracking error for each asset relative to the Benchmark.

Examples

Compute Information Ratio

This example show how to calculate the information ratio using `inforatio` with example data, where the mean return of the market series is used as the return of the benchmark.

You can use `inforatio` to compute the information ratio for the given asset return data and the riskless asset return.

```
load FundMarketCash
Returns = tick2ret(TestData);
Benchmark = Returns(:,2);
InfoRatio = inforatio>Returns, Benchmark)
```

```
InfoRatio = 1×3
```

```
    0.0432    NaN   -0.0315
```

Since the market series has no risk relative to itself, the information ratio for the second series is undefined (which is represented as `NaN` in MATLAB®).

Use Information Ratio to Compute Tracking Error

This example show how to calculate the tracking error using `inforatio` with example data, where the mean return of the market series is used as the return of the benchmark.

Given an asset or portfolio of assets and a benchmark, the relative standard deviation of returns between the asset or portfolio of assets and the benchmark is called tracking error.

```
load FundMarketCash
Returns = tick2ret(TestData);
Benchmark = Returns(:,2);
[InfoRatio, TrackingError] = inforatio>Returns, Benchmark)

InfoRatio = 1×3

    0.0432    NaN   -0.0315

TrackingError = 1×3

    0.0187    0    0.0390
```

Tracking error, also known as active risk, measures the volatility of active returns. Tracking error is a useful measure of performance relative to a benchmark since it is in units of asset returns. For example, the tracking error of 1.87% for the fund relative to the market in this example is reasonable for an actively managed, large-cap value fund.

Input Arguments

Asset — Asset returns

matrix

Asset returns, specified as a `NUMSAMPLES` × `NUMSERIES` matrix with `NUMSAMPLES` observations of asset returns for `NUMSERIES` asset return series.

Data Types: `double`

Benchmark — Returns for a benchmark asset

vector

Returns for a benchmark asset, specified as a `NUMSAMPLES` vector of returns for a benchmark asset. The periodicity must be the same as the periodicity of `Asset`. For example, if `Asset` is monthly data, then `Benchmark` should be monthly returns.

Data Types: `double`

Output Arguments

Ratio — Information ratios

vector

Information ratios, returned as a `1` × `NUMSERIES` row vector of information ratios for each series in `Asset`. Any series in `Asset` with a tracking error of `0` has a `NaN` value for its information ratio.

TE — Tracking errors

vector

Tracking errors, returned as a `1` × `NUMSERIES` row vector of tracking errors, that is, the standard deviation of `Asset` relative to `Benchmark` returns, for each series.

Note `NaN` values in the data are ignored. If the `Asset` and `Benchmark` series are identical, the information ratio is `NaN` since the tracking error is `0`. The information ratio and the Sharpe ratio of an

Asset versus a riskless Benchmark (a Benchmark with standard deviation of returns equal to 0) are equivalent. This equivalence is not necessarily true if the Benchmark is risky.

More About

Information Ratio

The information ratio quantifies the excess return of a portfolio (or investment) over its benchmark, relative to the volatility (or tracking error) of that excess return.

Version History

Introduced in R2006b

References

- [1] Grinold, R. C. and Ronald N. Kahn. *Active Portfolio Management*. 2nd. Edition. McGraw-Hill, 2000.
- [2] Treynor, J. and Fischer Black. "How to Use Security Analysis to Improve Portfolio Selection."
Journal of Business. Vol. 46, No. 1, January 1973, pp. 66-86.

See Also

portalpha | sharpe

Topics

"Performance Metrics Overview" on page 7-2

irr

Internal rate of return

Syntax

```
Return = irr(CashFlow)
[Return, AllRates] = irr(____)
```

Description

`Return = irr(CashFlow)` calculates the internal rate of return for a series of periodic cash flows.

`irr` uses the following conventions:

- If one or more internal rates of returns (warning if multiple) are strictly positive rates, `Return` sets to the minimum.
- If there is no strictly positive rate of returns, but one or multiple (warning if multiple) returns are nonpositive rates, `Return` sets to the maximum.
- If no real-valued rates exist, `Return` sets to NaN (no warnings).

`[Return, AllRates] = irr(____)` calculates the internal rate of return and a vector of all internal rates for a series of periodic cash flows.

Examples

Find a Single and Multiple Internal Rates of Return

Find the internal rate of return for a simple investment with a unique positive rate of return. The initial investment is \$100,000 and the following cash flows represent the yearly income from the investment

Year 1 \$10,000

Year 2 \$20,000

Year 3 \$30,000

Year 4 \$40,000

Year 5 \$50,000

Calculate the internal rate of return on the investment:

```
Return = irr([-100000 10000 20000 30000 40000 50000])
```

```
Return =  
0.1201
```

If the cash flow payments were monthly, then the resulting rate of return is multiplied by 12 for the annual rate of return.

Find Multiple Internal Rates of Return

Find the internal rate of return for multiple rates of return. The project has the following cash flows and a market rate of 10%.

```
CashFlow = [-1000 6000 -10900 5800]
```

```
CashFlow = 1×4
```

```
    -1000    6000   -10900    5800
```

```
[Return, AllRates] = irr(CashFlow)
```

```
Return =  
1.0000
```

```
AllRates = 3×1
```

```
   -0.0488  
    1.0000  
   2.0488
```

The rates of return in `AllRates` are -4.88%, 100%, and 204.88%. Though some rates are lower and some higher than the market rate, based on the work of Hazen, any rate gives a consistent recommendation on the project. However, you can use a present value analysis in these kinds of situations. To check the present value of the project, use `pvvar`:

```
PV = pvvar(CashFlow,0.10)
```

```
PV =  
-196.0932
```

The second argument is the 10% market rate. The present value is -196.0932, negative, so the project is undesirable.

Input Arguments

CashFlow — Stream of periodic cash flows

vector | matrix

Stream of periodic cash flows, specified as a vector or matrix. The first entry in `CashFlow` is the initial investment. If `CashFlow` is a matrix, `irr` handles each column of `CashFlow` as a separate cash-flow stream.

Data Types: double

Output Arguments

Return — Internal rate of return

vector

Internal rate of return associated to `CashFlow`, returned as a vector whose entry `j` is an internal rate of return for column `j` in `CashFlow`

AllRates — All the internal rates of return

serial date number | matrix

All the internal rates of return associated with `CashFlow`, returned as a matrix with the same number of columns as `CashFlow` and one less row. Also, column `j` in `AllRates` contains all the rates of return associated to column `j` in `CashFlow` (including complex-valued rates).

More About**Internal Rate of Return**

Internal rate of return is a financial metric used to evaluate the profitability of an investment. It represents the discount rate at which the net present value (NPV) of all cash flows (both incoming and outgoing) from an investment equals zero.

Version History

Introduced before R2006a

References

- [1] Brealey and Myers. *Principles of Corporate Finance*. McGraw-Hill Higher Education, Chapter 5, 2003.
- [2] Hazen G. "A New Perspective on Multiple Internal Rates of Return." *The Engineering Economist*. Vol. 48-1, 2003, pp. 31-51.

See Also

effrr | mirr | nomrr | xirr | pvvar

Topics

- "Interest Rates/Rates of Return" on page 2-11
- "Analyzing and Computing Cash Flows" on page 2-11
- "Performance Metrics Overview" on page 7-2

isbusday

True for dates that are business days

Syntax

```
Busday = isbusday(Date)
Busday = isbusday( ___,Holiday,Weekend)
```

Description

`Busday = isbusday(Date)` returns logical true (1) if `Date` is a business day and logical false (0) otherwise.

`Busday = isbusday( ___,Holiday,Weekend)`, using optional input arguments, returns logical true (1) if `Date` is a business day, and logical false (0) otherwise.

Examples

Determine If a Given Date Is a Business Day

Determine if `Date` is a business day.

```
Busday = isbusday('16 jun 2001')
```

```
Busday = logical
        0
```

Determine if a `Date` vector are business days.

```
Date = ['15 feb 2001'; '16 feb 2001'; '17 feb 2001'];
Busday = isbusday(Date)
```

```
Busday = 3×1 logical array
```

```
    1
    1
    0
```

Determine if a `Date` vector are business days using a datetime array.

```
Date = [datetime(2001,2,15); datetime(2001,2,16) ; datetime(2001,2,17)];
Busday = isbusday(Date)
```

```
Busday = 3×1 logical array
```

```
    1
    1
    0
```

Set June 21, 2003 (a Saturday) as a business day.

```
Weekend = [1 0 0 0 0 0 0];  
isbusday(datetime(2003,6,21), [], Weekend)  
  
ans = logical  
     1
```

If the second argument, `Holiday`, is empty (`[]`), the default `Holidays` vector (generated with `holidays` and then associated to the NYSE calendar) is used.

Input Arguments

Date — Date being checked

datetime array | string array | date character vector

Date being checked, specified as a scalar or a vector using a datetime array, string array, or date character vectors. `Date` can contain multiple dates, but they must all be in the same format. Dates are assumed to be whole date numbers or date stamps with no fractional or time values.

To support existing code, `isbusday` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Holiday — Holidays and nontrading-day dates

non-trading day vector is determined by the routine `holidays` (default) | datetime array | string array | date character vector

Holidays and nontrading-day dates, specified as a vector using a datetime array, string array, or date character vectors.

All dates in `Holiday` must be the same format: either datetimes, strings, date character vectors, or serial date numbers. The `holidays` function supplies the default vector.

To support existing code, `isbusday` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Weekend — Weekend days

[1 0 0 0 0 0 1] (Saturday and Sunday form the weekend) (default) | vector of length 7, containing 0 and 1, where 1 indicates weekend days

Weekend days, specified as a vector of length 7, containing 0 and 1, where 1 indicates weekend days and the first element of this vector corresponds to Sunday.

Data Types: `double`

Output Arguments

Busday — Logical true if a business day

logical 0 or 1

Logical true if a business day, returned as a logical true (1) if `Date` is a business day and logical false (0) otherwise.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `isbusday` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`busdate` | `fbusdate` | `holidays` | `lbusdate` | `datetime`

Topics

“Handle and Convert Dates” on page 2-2

“Trading Calendars User Interface” on page 12-2

“UICalendar User Interface” on page 12-4

kagi

Kagi chart

Syntax

```
kagi(Data)
h = kagi(ax,Data)
```

Description

`kagi(Data)` plots a Kagi chart from a series of prices.

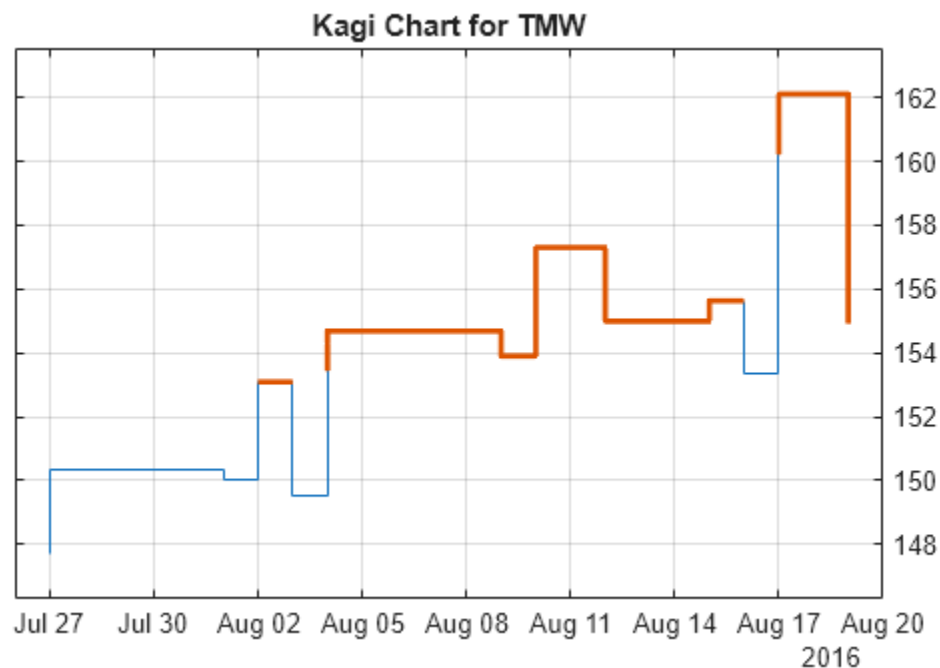
`h = kagi(ax,Data)` adds an optional argument for `ax`.

Examples

Generate a Kagi Chart for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock. This Kagi chart is for closing prices of the stock TMW for the most recent 21 days. Note that the variable name of asset price is be renamed to 'Price' (case insensitive).

```
load SimulatedStock.mat
TMW.Properties.VariableNames{'Close'} = 'Price';
kagi(TMW(end-20:end,:))
title('Kagi Chart for TMW')
```



Input Arguments

Data — Data for a series of prices

matrix | table | timetable

Data for a series of prices, specified as a matrix, table, or timetable. Timetables and tables with M rows must contain a variable named 'Price' (case insensitive).

Data Types: double | table | timetable

ax — Valid axis object

current axes (ax = gca) (default) | axes object

(Optional) Valid axis object, specified as an axes object. The kagi plot is created in the axes specified by ax instead of in the current axes (ax = gca). The option ax can precede any of the input argument combinations.

Data Types: object

Output Arguments

h — Graphic handle of the figure

handle object

Graphic handle of the figure, returned as a handle object.

More About

Kagi Chart

A Kagi chart represents price movements of an asset over time, focusing on the direction of price rather than the time intervals.

Kagi charts consist of a series of vertical lines that change direction based on price movements. The lines are drawn based on price changes and not on time intervals. The chart starts with a vertical line that represents the initial price level. If the price rises by a specified amount (called the "reversal amount"), a new line is drawn in the upward direction. Conversely, if the price falls by the same reversal amount, a line is drawn downward. The reversal amount is a key parameter in Kagi charts. It determines how much the price must change before a new line is drawn. This helps filter out minor price fluctuations and focuses on significant price movements.

Version History

Introduced in R2008a

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

R2018a: Support for matrix, time, or timetable input for data

Behavior changed in R2018a

`kagi` accepts Data input as a matrix, `timetable`, or `table`.

The syntax for `kagi` has changed. Previously, when using `table` input, the first column of dates could be serial date numbers, date character vectors, or datetime arrays, and you were required to have specific number of columns.

When using `table` input, the new syntax for `kagi` supports:

- No need for time information. If you want to pass in date information, use `timetable` input.
- No requirement of specific number of columns. However, you must provide valid column names. `kagi` must contain a column named 'price' (case insensitive).

See Also

`timetable` | `table` | `movavg` | `pointfig` | `highlow` | `linebreak` | `priceandvol` | `renko` | `volarea` | `candle`

Topics

"Use Timetables in Finance" on page 11-7

"Convert Financial Time Series Objects (fints) to Timetables" on page 11-2

lbusdate

Last business date of month

Syntax

```
Date = lbusdate(Year,Month)
Date = lbusdate( ___,Holiday,Weekend,outputType)
```

Description

`Date = lbusdate(Year,Month)` returns the serial date number for the last business date of the given year and month.

`Year` and `Month` can contain multiple values. If one contains multiple values, the other must contain the same number of values or a single value that applies to all. For example, if `Year` is a 1-by-N vector of integers, then `Month` must be a 1-by-N vector of integers or a single integer. `Date` is then a 1-by-N vector of serial date numbers.

Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

`Date = lbusdate( ___,Holiday,Weekend,outputType)` returns the serial date number for the last business date of the given year and month using optional input arguments. The optional argument `Holiday` specifies nontrading days.

If neither `Holiday` nor `outputType` are specified, `Date` is returned as a serial date number. If `Holiday` is specified, but not `outputType`, then the type of the holiday variable controls the type of date. If `Holiday` is a string or date character vector, then `Date` is returned as a serial date number.

Examples

Determine the Last Business Date of a Given Year and Month

Determine the `Date` using an input argument for `Year` and `Month`.

```
Date = lbusdate(2001, 5)
```

```
Date =
731002
```

```
datestr(Date)
```

```
ans =
'31-May-2001'
```

Determine the `Date` using the optional input argument for `outputType`.

```
Date = lbusdate(2001, 11,[],[], 'datetime')
```

```
Date = datetime
30-Nov-2001
```

Indicate that Saturday is a business day by appropriately setting the **Weekend** argument. May 31, 2003, is a Saturday. Use `lbusdate` to check that this Saturday is actually the last business day of the month.

```
Weekend = [1 0 0 0 0 0 0];  
Date = lbusdate(2003, 5, [], Weekend, 'datetime')  
  
Date = datetime  
      31-May-2003
```

Input Arguments

Year — Year to determine occurrence of weekday

4-digit integer | vector of 4-digit integers

Year to determine occurrence of weekday, specified as a 4-digit integer or vector of 4-digit integers.

Data Types: `double`

Month — Month to determine occurrence of weekday

integer with value 1 through 12 | vector of integers with values 1 through 12

Month to determine occurrence of weekday, specified as an integer or vector of integers with values 1 through 12.

Data Types: `double`

Holiday — Holidays and nontrading-day dates

non-trading day vector is determined by the routine `holidays` (default) | datetime array | string array | date character vector

Holidays and nontrading-day dates, specified as a vector using a datetime array, string array, or date character vectors.

All dates in `Holiday` must be the same format: either datetimes, strings, or date character vectors. The `holidays` function supplies the default vector.

If `Holiday` is a datetime array, then `Date` is returned as a datetime array. If `outputType` is specified, then its value determines the output type of `Date`. This overrides any influence of `Holiday`.

To support existing code, `lbusdate` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Weekend — Weekend days

[1 0 0 0 0 0 1] (Saturday and Sunday form the weekend) (default) | vector of length 7, containing 0 and 1, where 1 indicates weekend days

Weekend days, specified as a vector of length 7, containing 0 and 1, where 1 indicates weekend days and the first element of this vector corresponds to Sunday.

Data Types: `double`

outputType — Year to determine days

'datenum' (default) | character vector with values 'datenum' or 'datetime'

A character vector specified as either 'datenum' or 'datetime'. The output `Date` is in serial date format if 'datenum' is specified, or datetime format if 'datetime' is specified. By default the output `Date` is in serial date format, or match the format of `Holiday`, if specified.

Data Types: char

Output Arguments**Date — Date for the last business date of given year and month**

datetime | serial date number

Date for the last business date of a given year and month, returned as a datetime or serial date number.

If neither `Holiday` nor `outputType` are specified, `Date` is returned as a serial date number. If `Holiday` is specified, but not `outputType`, then the type of the holiday variable controls the type of date:

- If `Holiday` is a string or date character vector, then `Date` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.
- If `Holiday` is a datetime array, then `Date` is returned as a datetime array.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `lbusdate` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`busdate` | `fbusdate` | `holidays` | `isbusday` | `datetime`

Topics

“Handle and Convert Dates” on page 2-2

"Trading Calendars User Interface" on page 12-2
"UICalendar User Interface" on page 12-4

lifetableconv

Convert life table series into life tables with forced termination

Syntax

```
[qx,lx,dx] = lifetableconv(x0,lx0)
```

```
[qx,lx,dx] = lifetableconv(x0,y0,y0type)
```

Description

`[qx,lx,dx] = lifetableconv(x0,lx0)` converts life table with ages `x0` and survival counts `lx0` into life tables with termination.

`[qx,lx,dx] = lifetableconv(x0,y0,y0type)` converts life table with ages `x0` and series `y0`, specified by the optional argument `y0type`, into life tables with termination.

Examples

Convert Life Table Series into Life Tables with Forced Termination

Load the life table data file.

```
load us_lifetable_2009
```

Convert life table series into life tables with forced termination.

```
[qx,lx,dx] = lifetableconv(x,lx);
display(qx(1:20,:))
```

```

0.0064    0.0070    0.0057
0.0004    0.0004    0.0004
0.0003    0.0003    0.0002
0.0002    0.0002    0.0002
0.0002    0.0002    0.0001
0.0001    0.0002    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0002    0.0002    0.0002
0.0003    0.0004    0.0002
0.0004    0.0005    0.0002
0.0005    0.0006    0.0003
0.0005    0.0007    0.0003
0.0006    0.0009    0.0004
0.0007    0.0010    0.0004
```

```
display(lx(1:20,:))
```

```
1.0e+05 *
```

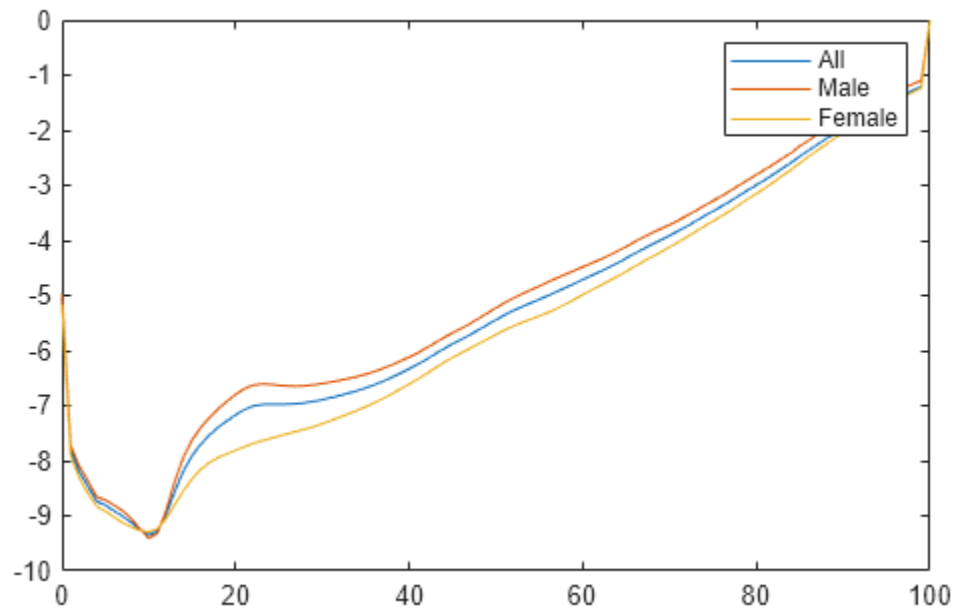
1.0000	1.0000	1.0000
0.9936	0.9930	0.9943
0.9932	0.9926	0.9939
0.9930	0.9923	0.9937
0.9927	0.9920	0.9935
0.9926	0.9919	0.9933
0.9924	0.9917	0.9932
0.9923	0.9916	0.9931
0.9922	0.9914	0.9930
0.9921	0.9913	0.9929
0.9920	0.9912	0.9928
0.9919	0.9911	0.9927
0.9918	0.9910	0.9926
0.9917	0.9909	0.9925
0.9915	0.9907	0.9923
0.9912	0.9903	0.9921
0.9908	0.9898	0.9919
0.9904	0.9892	0.9916
0.9899	0.9885	0.9913
0.9892	0.9876	0.9909

```
display(dx(1:20,:))
```

637.2266	698.8750	572.6328
40.4062	43.9297	36.7188
27.1875	30.0938	24.1406
20.7656	23.0781	18.3359
15.9141	17.2109	14.5625
14.8672	16.3125	13.3516
13.3672	14.7891	11.8750
12.1328	13.3828	10.8203
10.8125	11.6094	9.9844
9.4609	9.5781	9.3438
8.6172	8.1328	9.1172
9.2656	8.8359	9.7188
12.5938	13.5078	11.6328
19.1016	22.9844	15.0234
27.6719	35.5938	19.3516
36.6328	48.5703	24.0547
45.0156	60.7109	28.4844
53.1406	72.8906	32.2812
60.8984	85.1172	35.2578
68.3438	97.2266	37.6875

Plot the `qx` series and display the legend. The series `qx` is the conditional probability that a person at age `x` will die between age `x` and the next age in the series.

```
plot(x, log(qx))  
legend(series)
```

Convert the Life Table dx Series After Fitting and Generating the Life Table Series

Load the life table data file.

```
load us_lifetable_2009
```

Calibrate life table from survival data with the default Heligman-Pollard parametric model.

```
a = lifetablefit(x,lx)
```

```
a = 8×3
```

0.0005	0.0006	0.0004
0.0592	0.0819	0.0192
0.1452	0.1626	0.1048
0.0007	0.0011	0.0007
6.2841	6.7638	1.1037
24.1388	24.2895	53.1831
0.0000	0.0000	0.0000
1.0971	1.0987	1.1100

Generate life table series from the calibrated mortality model.

```
qx = lifetablegen((0:120),a);  
display(qx(1:20,:))
```

0.0063	0.0069	0.0057
0.0005	0.0006	0.0004
0.0002	0.0003	0.0002
0.0002	0.0002	0.0002

```

0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0001    0.0001    0.0001
0.0002    0.0002    0.0001
0.0002    0.0002    0.0002
0.0002    0.0003    0.0002
0.0003    0.0004    0.0002
0.0004    0.0005    0.0002
0.0005    0.0006    0.0003
0.0006    0.0008    0.0003
0.0007    0.0009    0.0003

```

Convert life table series into life tables with forced termination.

```

[~,~,dx] = lifetableconv((0:120),qx,'qx');
display(dx(1:20,:))

```

```

630.9948  686.9480  571.6097
 48.7926   55.1030   40.9874
 24.8018   26.3779   23.6167
 17.0832   17.5878   17.0317
 13.6181   13.8188   13.6142
 11.8662   12.0076   11.6313
 10.9782   11.1573   10.4904
 10.5996   10.8605    9.9488
 10.5757   10.9396    9.8952
 10.8789   11.3612   10.2718
 11.6084   12.2508   11.0419
 12.9921   13.9271   12.1764
 15.3480   16.8834   13.6482
 18.9938   21.6792   15.4301
 24.1393   28.7663   17.4944
 30.8006   38.3213   19.8134
 38.7714   50.1487   22.3603
 47.6533   63.6907   25.1098
 56.9299   78.1270   28.0385
 66.0574   92.5259   31.1257

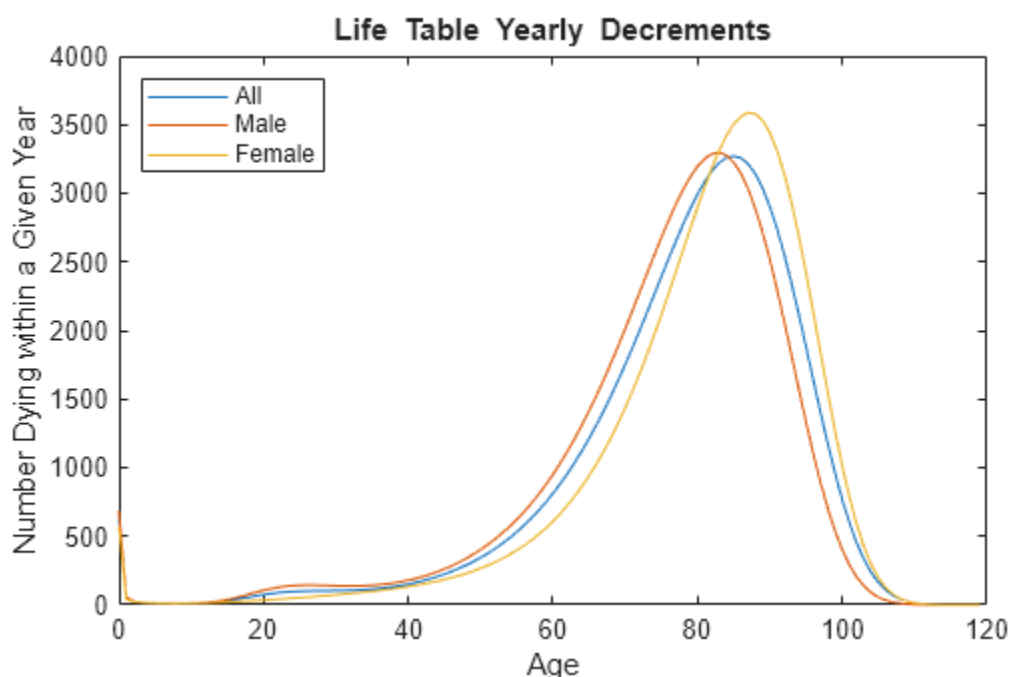
```

Plot the dx series and display the legend. The series dx is the number of people who die out of 100,000 alive at birth between age x and the next age in the series.

```

plot((0:119),dx(1:end-1,:));
legend(series, 'location', 'northwest');
title('\bfLife Table Yearly Decrements');
xlabel('Age');
ylabel('Number Dying within a Given Year');

```



Input Arguments

x0 — Increasing ages for raw data

vector of nonnegative integer values

Increasing ages for raw data, specified as nonnegative integer values in an **N0** vector.

The vector of ages x must contain nonnegative integer values. If the input series is the discrete survival function l_x , then the starting age need only be nonnegative. Otherwise, the starting age must be 0.

Data Types: double

lx0 — Collection of num standardized survivor series

matrix

Collection of num standardized survivor series, specified as an **N0**-by-num matrix. The input l_{x0} series is the number of people alive at age x , given 100,000 alive at birth. Values of 0 or NaN in the input table l_{x0} are ignored.

Data Types: double

y0 — Collection of num life table series to be converted

matrix

Collection of num life table series to be converted, specified as an **N0**-by-num matrix. The default $y0$ series is l_{x0} .

Data Types: double

y0type — Type of mortality series for input y0 with default 'lx'

'lx' (default) | character vector with values 'qx', 'lx', 'dx'

(Optional) Type of mortality series for input y_0 , specified as a character vector with one of the following values:

- 'qx' — Input is a table of discrete hazards (qx).
- 'lx' — Input is a table of discrete survival counts (lx).
- 'dx' — Input is a table of discrete decrements (dx).

Whereas the output series have forced termination, the input series (y_0) can have one of several types of termination:

- Natural termination runs out to the last person so that lx goes to 0, qx goes to 1, and dx goes to 0. For more information, see “Natural Termination” on page 15-1147.
- Truncated termination stops at a terminal age so that lx is positive, qx is less than 1, and dx is positive. Any ages after the terminal age are NaN values. For more information, see “Truncated Termination” on page 15-1148.

Data Types: char

Output Arguments

qx — Discrete hazard function

matrix

Discrete hazard function, returned as an N0-by-num matrix with forced termination. For more information, see “Forced Termination” on page 15-1147.

The series qx is the conditional probability that a person at age x will die between age x and the next age in the series.

lx — Discrete survival function

matrix

Discrete survival function, returned as an N0-by-num matrix with forced termination. For more information, see “Forced Termination” on page 15-1147.

The series lx is the number of people alive at age x, given 100,000 alive at birth.

dx — Discrete decrements function

matrix

Discrete decrements function, returned as an N0-by-num matrix with forced termination. For more information, see “Forced Termination” on page 15-1147.

The series dx is the number of people who die out of 100,000 alive at birth, between age x and the next age in the series.

More About

Life Table

A life table is a statistical table used in demography and actuarial science to summarize the mortality experience of a population.

A life table provides a detailed breakdown of the probability of death and survival at various ages, allowing for the analysis of life expectancy and the assessment of the health status of a population.

Forced Termination

Most modern life tables have “forced” termination. Forced termination means that the last row of the life table applies for all persons with ages on or after the last age in the life table.

This sample illustrates forced termination.

United States Life Tables 2009

x	l_x WM	l_x WF	l_x BM	l_x BF	q_x WM	q_x WF	q_x BM	q_x BF
93	9533	18368	6615	15685	0.219553	0.177156	0.200605	0.157858
94	7440	15114	5288	13209	0.23871	0.194852	0.214448	0.170868
95	5664	12169	4154	10952	0.258475	0.213411	0.229177	0.184624
96	4200	9572	3202	8930	0.27881	0.232971	0.24391	0.198992
97	3029	7342	2421	7153	0.299439	0.253337	0.259397	0.214036
98	2122	5482	1793	5622	0.320452	0.27417	0.274958	0.229634
99	1442	3979	1300	4331	0.341886	0.295803	0.291538	0.245671
100+	949	2802	921	3267	1	1	1	1

In this case, the last row of the life table applies for all persons aged 100 or older. Specifically, q_x probabilities are ${}_1q_x$ for ages less than 100 and, technically, ${}_∞q_x$ for age 100.

Forced termination has terminal age values that apply to all ages after the terminal age so that l_x is positive, q_x is 1, and dx is positive. Ages after the terminal age are NaN values, although l_x and dx can be 0 and q_x can be 1 for input series. Forced termination is triggered by a naturally terminating series, the last age in a truncated series, or the first NaN value in a series.

Natural Termination

Before 1970, life tables were often published with data that included all ages for which persons associated with a given series were still alive. Tables in this form have “natural” termination. In natural termination, the last row of the life table for each series counts the deaths or probabilities of deaths of the last remaining person at the corresponding age. Tables in this form can be problematic due to the granularity of the data and the fact that groups of series can terminate at distinct ages. Natural termination is illustrated in the following sample of the last few years of a life table.

United States Life Tables 1940

x	l_x WM	l_x WF	l_x BM	l_x BF	q_x WM	q_x WF	q_x BM	q_x BF
103	14	29	59	234	0.428571	0.413793	0.389831	0.333333
104	8	17	36	156	0.5	0.470588	0.416667	0.358974
105	4	9	21	100	0.5	0.444444	0.47619	0.38
106	2	5	11	62	0.5	0.4	0.454545	0.403226
107	1	3	6	37	0	0.666667	0.5	0.432432
108	1	1	3	21	1	0	0.666667	0.47619
109		1	1	11		1	1	0.454545
110				6				0.5
111				3				0.666667
112				1				0
113				1				1
114								

This form for life tables poses a number of issues that go beyond the obvious statistical issues. First, the l_x table on the left terminates at ages 108, 109, 109, and 113 for the four series in the table.

Technically, the numbers after these ages are 0, but can also be NaN values because no person is alive after these terminating ages. Second, the probabilities q_x on the right terminate with fluctuating probabilities that go from 0 to 1 in some cases. In this case, however, all probabilities are ${}_1q_x$ probabilities (unlike the forced termination probabilities). You can argue that the probabilities after the ages of termination can be 1 (anyone alive at this age is expected to die in the next year), 0 (the age lies outside the support of the probability distribution), or NaN values.

Truncated Termination

Truncated termination occurs with truncation of life tables at an arbitrary age. For example, from 1970–1990, United States life tables truncated at age 85. This format is problematic because life table probabilities must either terminate with probability 1 (forced termination) or discard data that exceeds the terminating age. This sample of the last few years of a life table illustrates truncated termination. The raw data for this table is the l_x series. The q_x series is derived from this series.

United States Life Tables 1980

x	l_x WM	l_x WF	l_x BM	l_x BF	q_x WM	q_x WF	q_x BM	q_x BF
78	38558	61100	25481	45776	0.078064	0.045221	0.082689	0.054789
79	35548	58337	23374	43268	0.084477	0.050037	0.088303	0.059351
80	32545	55418	21310	40700	0.091504	0.055542	0.094603	0.064619
81	29567	52340	19294	38070	0.099165	0.061922	0.101741	0.070607
82	26635	49099	17331	35382	0.107528	0.06935	0.109746	0.077356
83	23771	45694	15429	32645	0.116613	0.078194	0.118608	0.084822
84	20999	42121	13599	29876	0.126339	0.081788	0.128392	0.093018
85	18346	38676	11853	27097				

This life table format poses problems for termination because, for example, over 27% of the population for the fourth l_x series is still alive at age 85. To claim that the probability of dying for all ages after age 85 is 100% might be true but is uninformative. Notwithstanding the statistical issues, however, tables in this form are completed by forced termination.

Version History

Introduced in R2015a

References

- [1] Arias, E. "United States Life Tables." *National Vital Statistics Reports, U.S. Department of Health and Human Services*. Vol. 62, No. 7, 2009.

See Also

`lifetablefit` | `lifetablegen`

Topics

"Case Study for Life Tables Analysis" on page 2-46

"About Life Tables" on page 2-44

lifetablefit

Calibrate life table from survival data with parametric models

Syntax

```
[a,elx] = lifetablefit(x,lx)
[a,elx] = lifetablefit( ___,lifemodel,objtype,interpmethod,a0)
```

Description

`[a,elx] = lifetablefit(x,lx)` calibrates a life table, `x`, from survival data, `lx`, using parametric models.

`[a,elx] = lifetablefit( ___,lifemodel,objtype,interpmethod,a0)` calibrates a life table, `x`, from survival data, `lx`, using parametric models using optional arguments for `lifemodel`, `objtype`, `interpmethod`, and `a0`.

Examples

Calibrate Life Table from Survival Data Using a Heligman-Pollard Parametric Model

Load the life table data file.

```
load us_lifetable_2009
```

Calibrate the life table from survival data using the default heligman-pollard parametric model.

```
[a,elx] = lifetablefit(x,lx);
display(a)
```

```
a = 8×3
```

```
    0.0005    0.0006    0.0004
    0.0592    0.0819    0.0192
    0.1452    0.1626    0.1048
    0.0007    0.0011    0.0007
    6.2841    6.7638    1.1037
   24.1388   24.2895   53.1831
    0.0000    0.0000    0.0000
    1.0971    1.0987    1.1100
```

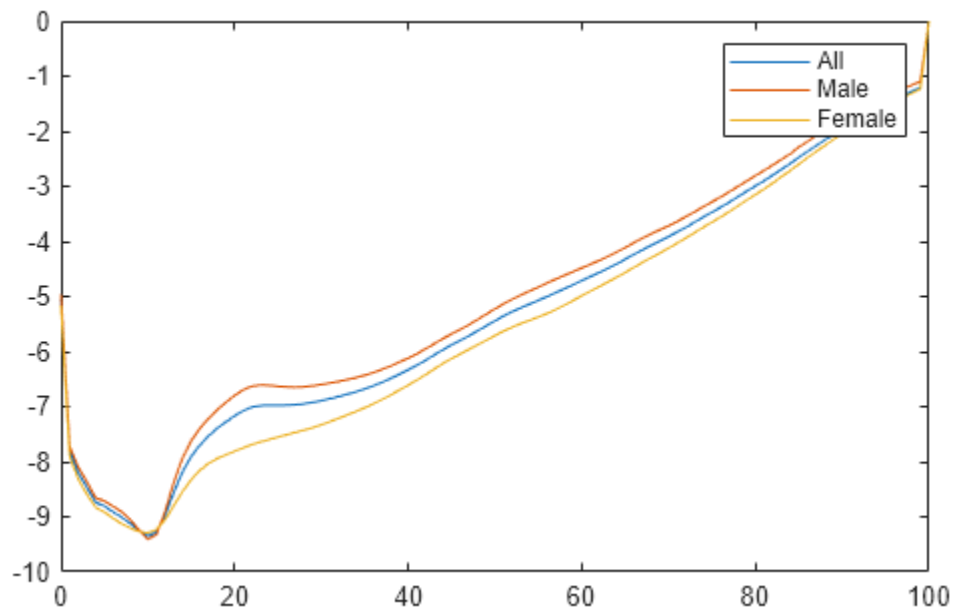
```
display(elx(1:20,:))
```

```
1.0e+05 *
    1.0000    1.0000    1.0000
    0.9937    0.9931    0.9943
    0.9932    0.9926    0.9939
    0.9930    0.9923    0.9936
    0.9928    0.9921    0.9935
```

0.9926	0.9920	0.9933
0.9925	0.9919	0.9932
0.9924	0.9918	0.9931
0.9923	0.9917	0.9930
0.9922	0.9916	0.9929
0.9921	0.9914	0.9928
0.9920	0.9913	0.9927
0.9919	0.9912	0.9926
0.9917	0.9910	0.9924
0.9915	0.9908	0.9923
0.9913	0.9905	0.9921
0.9910	0.9901	0.9919
0.9906	0.9896	0.9917
0.9901	0.9890	0.9914
0.9895	0.9882	0.9912

Plot the `qx` series and display the legend. The series `qx` is the conditional probability that a person at age `x` will die between age `x` and the next age in the series

```
plot(x, log(qx))
legend(series)
```



Input Arguments

x — Increasing ages for raw data

vector of nonnegative integers

Increasing ages for raw data, specified as a `N` vector for nonnegative integers.

Data Types: `double`

lx — Collection of num discrete survival counts

matrix

Collection of num discrete survival counts, specified as an N-by-num matrix. The input lx series is the number of people alive at age x, given 100,000 alive at birth. Values of 0 or NaN in the input table lx are ignored.

Data Types: double

lifemodel — Parametric mortality model type

'heligman-pollard' (default) | character vector with values 'heligman-pollard', 'heligman-pollard-2', 'heligman-pollard-3', 'gompertz', 'makeham', 'siler'

(Optional) Parametric mortality model type, specified as a character vector with one of the following values:

- 'heligman-pollard' — Eight-parameter Heligman-Pollard model (version 1), specified in terms of the discrete hazard function:

$$\frac{q(x)}{1 - q(x)} = A^{(x+B)^C} + D \exp\left(-E \left(\log \frac{x}{F}\right)^2\right) + GH^X$$

for ages $x \geq 0$, with parameters $A, B, C, D, E, F, G, H \geq 0$.

- 'heligman-pollard-2' — Eight-parameter Heligman-Pollard model (version 2), specified in terms of the discrete hazard function:

$$\frac{q(x)}{1 - q(x)} = A^{(x+B)^C} + D \exp\left(-E \left(\log \frac{x}{F}\right)^2\right) + \frac{GH^X}{1 + GH^X}$$

for ages $x \geq 0$, with parameters $A, B, C, D, E, F, G, H \geq 0$.

- 'heligman-pollard-3' — Eight-parameter Heligman-Pollard model (version 3), specified in terms of the discrete hazard function:

$$q(x) = A^{(x+B)^C} + D \exp\left(-E \left(\log \frac{x}{F}\right)^2\right) + GH^X$$

for ages $x \geq 0$, with parameters $A, B, C, D, E, F, G, H \geq 0$.

- 'gompertz' — Two-parameter Gompertz model, specified in terms of the continuous hazard function:

$$h(x) = A \exp(Bx)$$

for ages $x \geq 0$, with parameters $A, B \geq 0$.

- 'makeham' — Three-parameter Gompertz-Makeham model, specified in terms of the continuous hazard function:

$$h(x) = A \exp(Bx) + C$$

for ages $x \geq 0$, with parameters $A, B, C \geq 0$.

- 'siler' — Five-parameter Siler model, specified in terms of the continuous hazard function:

$$h(x) = A \exp(Bx) + C + D \exp(-Ex)$$

for ages $x \geq 0$, with parameters $A, B, C, D, E \geq 0$.

Data Types: char

objtype — Objective for nonlinear least-squares estimation

'ratio' (default) | character vector with values 'ratio' 'logratio'

(Optional) Objective for nonlinear least-squares estimation, specified as a character vector with the following values:

- 'ratio' — Given raw data q_x and model estimates $\hat{q}_x$ for $x = 1, \dots, N$, the first objective (which is the preferred objective) has the form

$$\Phi = \sum_{x=1}^N \left(1 - \frac{\hat{q}_x}{q_x}\right)^2$$

- 'logratio' — Given raw data q_x and model estimates $\hat{q}_x$ for $x = 1, \dots, N$, the second objective has the form

$$\Phi = \sum_{x=1}^N (\log(\hat{q}_x) - \log(q_x))^2$$

Data Types: char

interpmethod — Interpolation method to use for abridged life table inputs

'cubic' (default) | character vector with values 'cubic', 'linear', 'none'

(Optional) Interpolation method to use for abridged life table inputs, specified as a character vector with the following values:

- 'cubic' — Cubic interpolation that uses 'pchip' method in `interp1`.
- 'linear' — Linear interpolation.
- 'none' — No interpolation.

Note If the ages in x are not consecutive years and interpolation is set to 'none', then the estimates for the parameters are suitable only for the age vector x .

If you use the parameter estimates to compute life table values for arbitrary years, interpolate using the default 'cubic' method.

Interpolation with abridged life tables forms internal interpolated full life tables, which usually improves model fits.

Data Types: char

a0 — Initial parameter estimate to be applied to all series

vector

(Optional) Initial parameter estimate to be applied to all series, specified as a `numparam` vector. This vector must conform to the number of parameters in the model specified using the `lifemodel` argument.

Data Types: double

Output Arguments

a — Parameter estimates for each num series

matrix

Parameter estimates for each num series, returned as a numparam-by-num matrix.

e_{lx} — Estimated collection of num standardized survivor series

matrix

Estimated collection of num standardized survivor series, returned as an N-by-num matrix. The e_{lx} output series is the number of people alive at age x, given 100,000 alive at birth. Values of 0 or NaN in the input table lx are ignored.

More About

Life Table

A life table is a statistical table used in demography and actuarial science to summarize the mortality experience of a population.

A life table provides a detailed breakdown of the probability of death and survival at various ages, allowing for the analysis of life expectancy and the assessment of the health status of a population.

Version History

Introduced in R2015a

References

- [1] Arias, E. "United States Life Tables." *National Vital Statistics Reports, U.S. Department of Health and Human Services*. Vol. 62, No. 7, 2009.
- [2] Carriere, F. "Parametric Models for Life Tables." *Transactions of the Society of Actuaries*. Vol. 44, 1992, pp. 77-99.
- [3] Gompertz, B. "On the Nature of the Function Expressive of the Law of Human Mortality, and on a New Mode of Determining the Value of Life Contingencies." *Philosophical Transactions of the Royal Society*. Vol. 115, 1825, pp. 513-582.
- [4] Heligman, L. M. A., and J. H. Pollard. "The Age Pattern of Mortality." *Journal of the Institute of Actuaries* Vol. 107, Pt. 1, 1980, pp. 49-80.
- [5] Makeham, W. M. "On the Law of Mortality and the Construction of Annuity Tables." *Journal of the Institute of Actuaries* Vol. 8, 1860, pp. 301-310.
- [6] Siler, W. "A Competing-Risk Model for Animal Mortality." *Ecology* Vol. 60, pp. 750-757, 1979.
- [7] Siler, W. "Parameters of Mortality in Human Populations with Widely Varying Life Spans." *Statistics in Medicine* Vol. 2, 1983, pp. 373-380.

See Also

lifetableconv | lifetablegen

Topics

“Case Study for Life Tables Analysis” on page 2-46

“About Life Tables” on page 2-44

lifetablegen

Generate life table series from calibrated mortality model

Syntax

```
[qx, lx, dx] = lifetablegen(x, a)
[qx, lx, dx] = lifetablegen(x, a, lifemodel)
```

Description

[qx, lx, dx] = lifetablegen(x, a) generates a life table series from a calibrated mortality model.

[qx, lx, dx] = lifetablegen(x, a, lifemodel) generates a life table series from a calibrated mortality model using the optional argument for lifemodel.

Examples

Generate Life Table Series from a Calibrated Mortality Model for Heligman-Pollard

Load the life table data file.

```
load us_lifetable_2009
```

Calibrate the life table from survival data using the default heligman-pollard parametric model.

```
a = lifetablefit(x, lx)
```

```
a = 8×3
```

0.0005	0.0006	0.0004
0.0592	0.0819	0.0192
0.1452	0.1626	0.1048
0.0007	0.0011	0.0007
6.2841	6.7638	1.1037
24.1388	24.2895	53.1831
0.0000	0.0000	0.0000
1.0971	1.0987	1.1100

Generate a life table series from the calibrated mortality model.

```
qx = lifetablegen(x, a);
display(qx(1:20,:))
```

0.0063	0.0069	0.0057
0.0005	0.0006	0.0004
0.0002	0.0003	0.0002
0.0002	0.0002	0.0002
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001

```

0.0001  0.0001  0.0001
0.0001  0.0001  0.0001
0.0001  0.0001  0.0001
0.0001  0.0001  0.0001
0.0001  0.0001  0.0001
0.0001  0.0001  0.0001
0.0002  0.0002  0.0001
0.0002  0.0002  0.0002
0.0002  0.0003  0.0002
0.0003  0.0004  0.0002
0.0004  0.0005  0.0002
0.0005  0.0006  0.0003
0.0006  0.0008  0.0003
0.0007  0.0009  0.0003

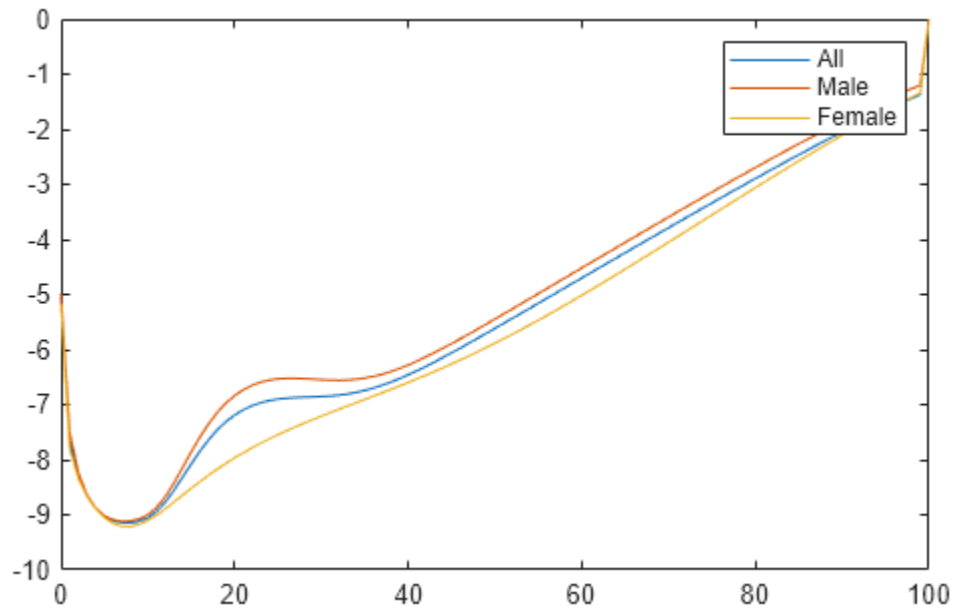
```

Plot the `qx` series and display the legend. The series `qx` is the conditional probability that a person at age `x` will die between age `x` and the next age in the series.

```

plot(x, log(qx))
legend(series)

```



Prepare Life Table Data Using the `qx` Series and Generate the `qx` Life Table

Load the life table data file.

```
load us_lifetable_2009
```

Convert the life table series into life tables with forced termination.

```
[~, lx] = lifetableconv(x, qx, 'qx');
```

Calibrate the life table from survival data using the default heligman-pollard parametric model.

```
a = lifetablefit(x, lx)
```

```
a = 8×3
```

0.0005	0.0006	0.0004
0.0592	0.0819	0.0192
0.1452	0.1626	0.1048
0.0007	0.0011	0.0007
6.2853	6.7635	1.1033
24.1385	24.2898	53.2008
0.0000	0.0000	0.0000
1.0971	1.0987	1.1100

Generate a life table series from the calibrated mortality model.

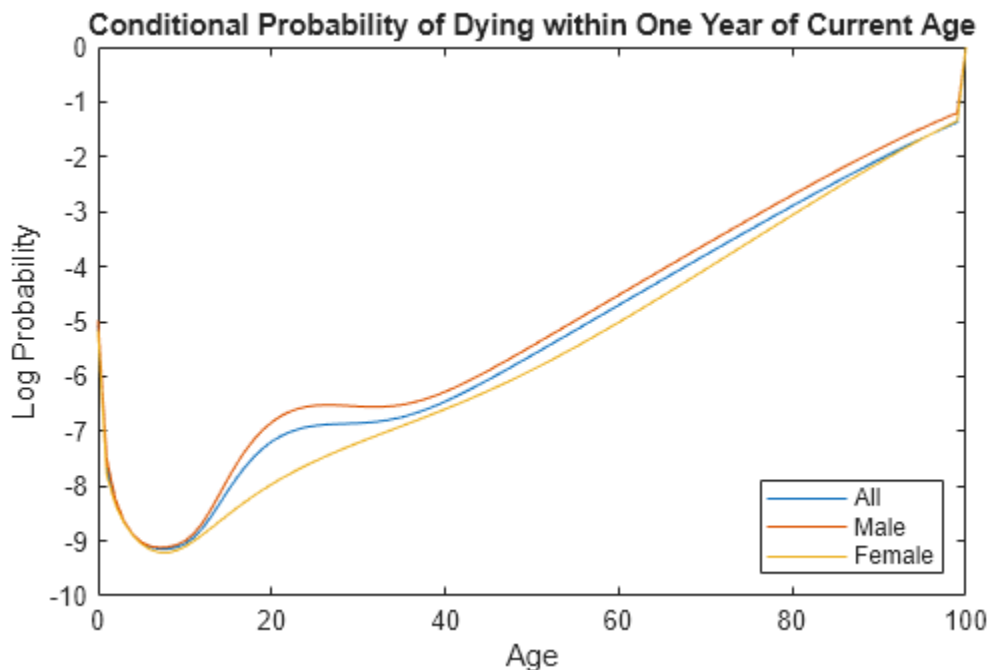
```
qx = lifetablegen((0:100), a)
```

```
qx = 101×3
```

0.0063	0.0069	0.0057
0.0005	0.0006	0.0004
0.0002	0.0003	0.0002
0.0002	0.0002	0.0002
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0002	0.0002	0.0001
0.0002	0.0002	0.0002
0.0002	0.0003	0.0002
⋮		

Plot the qx series and display the legend. The series qx is the conditional probability that a person at age x will die between age x and the next age in the series.

```
plot((0:100), log(qx));
legend(series, 'location', 'southeast');
title('Conditional Probability of Dying within One Year of Current Age');
xlabel('Age');
ylabel('Log Probability');
```



Input Arguments

x — Increasing ages for raw data

vector of nonnegative integers

Increasing ages for raw data, specified as a N vector of nonnegative integer values. The ages must start at 0 (birth).

Data Types: double

a — Model parameters for num models

matrix

Model parameters for num models, specified as a numparam-by-num matrix, where the number of parameters (numparam) depends on the model specified using the lifemodel argument.

Data Types: double

lifemodel — Parametric mortality model type

'heligman-pollard' (default) | character vector with values 'heligman-pollard', 'heligman-pollard-2', 'heligman-pollard-3', 'gompertz', 'makeham', 'siler'

(Optional) Parametric mortality model type, specified as a character vector with one of the following values:

- 'heligman-pollard' — Eight-parameter Heligman-Pollard model (version 1), specified in terms of the discrete hazard function:

$$\frac{q(x)}{1 - q(x)} = A^{(x+B)^C} + D \exp(-E(\log \frac{x}{F})^2) + GH^X$$

for ages $x \geq 0$, with parameters $A, B, C, D, E, F, G, H \geq 0$.

- 'heligman-pollard-2' — Eight-parameter Heligman-Pollard model (version 2), specified in terms of the discrete hazard function:

$$\frac{q(x)}{1 - q(x)} = A^{(x+B)^C} + D \exp(-E(\log \frac{x}{F})^2) + \frac{GH^x}{1 + GH^x}$$

for ages $x \geq 0$, with parameters $A, B, C, D, E, F, G, H \geq 0$.

- 'heligman-pollard-3' — Eight-parameter Heligman-Pollard model (version 3), specified in terms of the discrete hazard function:

$$q(x) = A^{(x+B)^C} + D \exp(-E(\log \frac{x}{F})^2) + GH^x$$

for ages $x \geq 0$, with parameters $A, B, C, D, E, F, G, H \geq 0$.

- 'gompertz' — Two-parameter Gompertz model, specified in terms of the continuous hazard function:

$$h(x) = A \exp(Bx)$$

for ages $x \geq 0$, with parameters $A, B \geq 0$.

- 'makeham' — Three-parameter Gompertz-Makeham model, specified in terms of the continuous hazard function:

$$h(x) = A \exp(Bx) + C$$

for ages $x \geq 0$, with parameters $A, B, C \geq 0$.

- 'siler' — Five-parameter Siler model, specified in terms of the continuous hazard function:

$$h(x) = A \exp(Bx) + C + D \exp(-Ex)$$

for ages $x \geq 0$, with parameters $A, B, C, D, E \geq 0$.

Data Types: char

Output Arguments

qx — Conditional probabilities of dying for N ages and num series

matrix

Conditional probabilities of dying for N ages and num series, returned as an N-by-num matrix. The series qx is the conditional probability that a person at age x will die between age x and the next age in the series. For the last age, qx represents probabilities or counts for all ages after the last age.

The last row of the N-by-num output for qx is the values for all ages on or after the last age in x (due to "Forced Termination" on page 15-1160). Therefore, the last row of qx contains 1 (100% probability of dying on or after the last age).

lx — Survival counts for N ages and num series

matrix

Survival counts for N ages and num series, returned as an N-by-num matrix. The series lx is the number of people alive at age x, given 100,000 alive at birth.

dx — Decrement counts for N ages and num series

matrix

Decrement counts for N ages and num series, returned as an N-by-num matrix. The series dx is the number of people out of 100,000 alive at birth who die between age x and the next age in the series. For the last age, dx represent probabilities or counts for all ages after the last age.

The last row of the N-by-num output for dx are values for all ages on or after the last age in x (due to “Forced Termination” on page 15-1160). Therefore, the last row of dx contains the remaining count of 100,000 people alive at birth who have not died by the last age.

More About**Life Table**

A life table is a statistical table used in demography and actuarial science to summarize the mortality experience of a population.

A life table provides a detailed breakdown of the probability of death and survival at various ages, allowing for the analysis of life expectancy and the assessment of the health status of a population.

Forced Termination

Most modern life tables have “forced” termination. Forced termination means that the last row of the life table applies for all persons with ages on or after the last age in the life table.

This sample illustrates forced termination.

United States Life Tables 2009

x	l_x WM	l_x WF	l_x BM	l_x BF	q_x WM	q_x WF	q_x BM	q_x BF
93	9533	18368	6615	15685	0.219553	0.177156	0.200605	0.157858
94	7440	15114	5288	13209	0.23871	0.194852	0.214448	0.170868
95	5664	12169	4154	10952	0.258475	0.213411	0.229177	0.184624
96	4200	9572	3202	8930	0.27881	0.232971	0.24391	0.198992
97	3029	7342	2421	7153	0.299439	0.253337	0.259397	0.214036
98	2122	5482	1793	5622	0.320452	0.27417	0.274958	0.229634
99	1442	3979	1300	4331	0.341886	0.295803	0.291538	0.245671
100+	949	2802	921	3267	1	1	1	1

In this case, the last row of the life table applies for all persons aged 100 or older. Specifically, q_x probabilities are ${}_1q_x$ for ages less than 100 and, technically, ${}_{\infty}q_x$ for age 100.

Forced termination has terminal age values that apply to all ages after the terminal age so that l_x is positive, q_x is 1, and dx is positive. Ages after the terminal age are NaN values, although l_x and dx can be 0 and q_x can be 1 for input series. Forced termination is triggered by a naturally terminating series, the last age in a truncated series, or the first NaN value in a series.

Version History

Introduced in R2015a

References

- [1] Arias, E. "United States Life Tables." *National Vital Statistics Reports, U.S. Department of Health and Human Services*. Vol. 62, No. 7, 2009.
- [2] Carriere, F. "Parametric Models for Life Tables." *Transactions of the Society of Actuaries*. Vol. 44, 1992, pp. 77-99.
- [3] Gompertz, B. "On the Nature of the Function Expressive of the Law of Human Mortality, and on a New Mode of Determining the Value of Life Contingencies." *Philosophical Transactions of the Royal Society*. Vol. 115, 1825, pp. 513-582.
- [4] Heligman, L. M. A., and J. H. Pollard. "The Age Pattern of Mortality." *Journal of the Institute of Actuaries* Vol. 107, Pt. 1, 1980, pp. 49-80.
- [5] Makeham, W. M. "On the Law of Mortality and the Construction of Annuity Tables." *Journal of the Institute of Actuaries* Vol. 8, 1860, pp. 301-310.
- [6] Siler, W. "A Competing-Risk Model for Animal Mortality." *Ecology* Vol. 60, pp. 750-757, 1979.
- [7] Siler, W. "Parameters of Mortality in Human Populations with Widely Varying Life Spans." *Statistics in Medicine* Vol. 2, 1983, pp. 373-380.

See Also

lifetablefit | lifetableconv

Topics

"Case Study for Life Tables Analysis" on page 2-46

"About Life Tables" on page 2-44

linebreak

Line break chart

Syntax

```
linebreak(Data)  
h = linebreak(ax,Data)
```

Description

`linebreak(Data)` plots the asset data, in a line break chart.

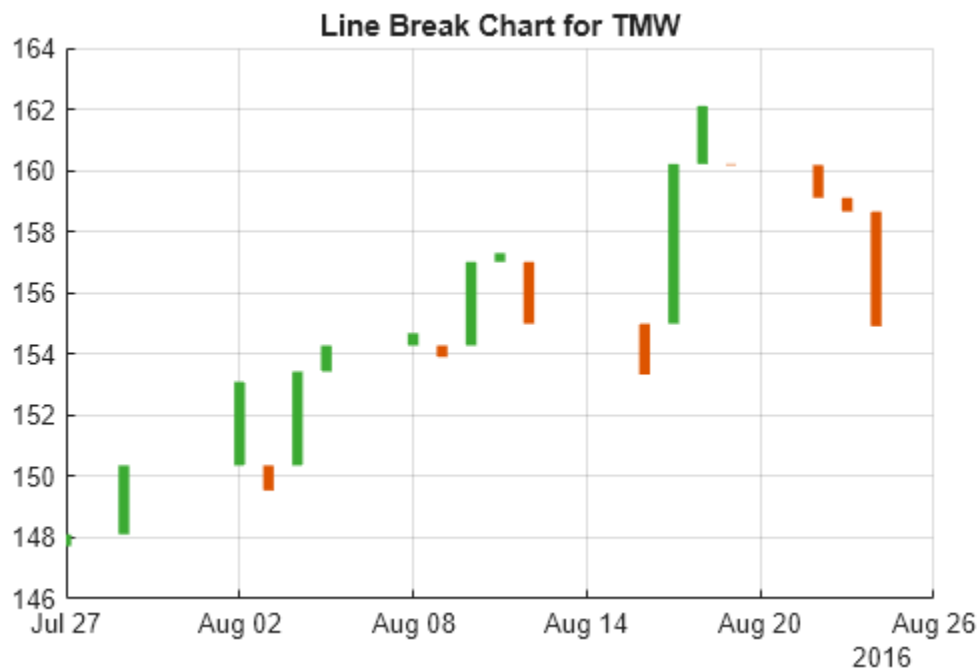
`h = linebreak(ax,Data)` adds an optional argument for `ax`.

Examples

Generate a Line Break Chart for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock. This Linebreak chart is for closing prices of the stock TMW for the most recent 21 days. Note that the variable name of asset price is be renamed to 'Price' (case insensitive).

```
load SimulatedStock.mat  
TMW.Properties.VariableNames{'Close'} = 'Price';  
linebreak(TMW(end-20:end,:))  
title('Line Break Chart for TMW')
```



Input Arguments

Data — Data for a series of prices

matrix | table | timetable

Data for a series of prices, specified as a matrix, table, or timetable. Timetables and tables with M rows must contain a variable named 'Price' (case insensitive).

Data Types: double | table | timetable

ax — Valid axis object

current axes (ax = gca) (default) | axes object

(Optional) Valid axis object, specified as an axes object. The linebreak plot is created in the axes specified by ax instead of in the current axes (ax = gca). The option ax can precede any of the input argument combinations.

Data Types: object

Output Arguments

h — Graphic handle of the figure

handle object

Graphic handle of the figure, returned as a handle object.

More About

Line Break Chart

A line break chart is a type of financial chart used to represent price movements of an asset over time, focusing on price action rather than time intervals.

The line break chart is particularly useful for identifying trends and reversals in price movements without the noise of time-based charts, such as candlestick or bar charts. Line break charts help traders and analysts make decisions based on significant price changes. A line break chart contains a series of vertical lines that represent price movements. Each line is drawn based on a specific price threshold rather than a fixed time interval. The chart consists of a series of "lines" that change direction based on price movements. A new line is drawn when the price moves a certain amount beyond the previous line.

Version History

Introduced in R2008a

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

R2018a: Support for matrix, time, or timetable input for data

Behavior changed in R2018a

`linebreak` accepts Data input as a matrix, `timetable`, or `table`.

The syntax for `linebreak` has changed. Previously, when using `table` input, the first column of dates could be serial date numbers, date character vectors, or datetime arrays, and you were required to have specific number of columns.

When using `table` input, the new syntax for `linebreak` supports:

- No need for time information. If you want to pass in date information, use `timetable` input.
- No requirement of specific number of columns. However, you must provide valid column names. `linebreak` must contain a column named 'price' (case insensitive).

See Also

`timetable` | `table` | `movavg` | `pointfig` | `highlow` | `kagi` | `priceandvol` | `renko` | `volarea` | `candle`

Topics

"Use Timetables in Finance" on page 11-7

"Convert Financial Time Series Objects (fints) to Timetables" on page 11-2

lloow

Lowest low

Syntax

```
values = lloow(Data)  
values = lloow( ___,Name,Value)
```

Description

`values = lloow(Data)` generates a vector of lowest low values from the series of low prices for the past n periods.

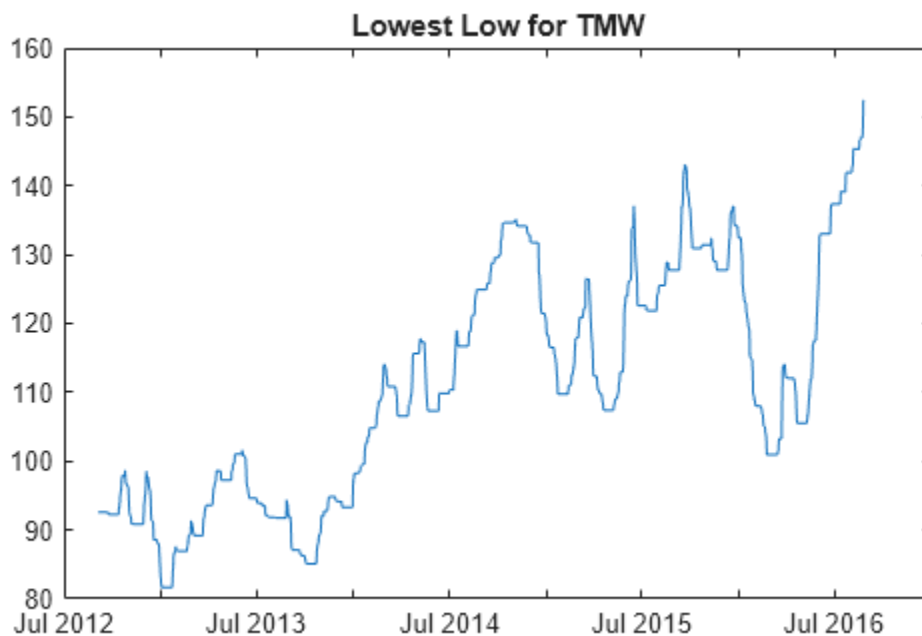
`values = lloow( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Calculate the Lowest Low for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat  
values = lloow(TMW);  
plot(values.Time,values.LowestLow)  
title('Lowest Low for TMW')
```



Input Arguments

Data — Data for low prices

matrix | table | timetable

Data for low prices, specified as a matrix, table, or timetable. Timetables and tables with M rows must contain a variable named 'Low' (case insensitive).

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `values = llow(TMW_LOW, 'NumPeriods', 10)`

NumPeriods — Moving window for the lowest low calculation

14 (default) | positive integer

Moving window for the lowest low calculation, specified as the comma-separated pair consisting of 'NumPeriods' and a scalar positive integer.

Data Types: double

Output Arguments

values — Lowest low series

matrix | table | timetable

Lowest low series, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input Data.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the Data input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995.

See Also

timetable | table | hhigh

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

lpm

Compute sample lower partial moments of data

Syntax

```
lpm(Data)
lpm(Data,MAR,Order)
Moment = lpm(Data,MAR,Order)
```

Description

`lpm(Data)` computes lower partial moments for asset returns `Data` relative to a default value for MAR for each asset in a `NUMORDERS` $\times$ `NUMSERIES` matrix and a default value for `Order`.

`lpm(Data,MAR,Order)` computes lower partial moments for asset returns `Data` relative to MAR for each asset in a `NUMORDERS` $\times$ `NUMSERIES` matrix.

`Moment = lpm(Data,MAR,Order)` computes lower partial moments for asset returns `Data` relative to MAR for each asset in a `NUMORDERS` $\times$ `NUMSERIES` matrix `Moment`.

Examples

Compute Lower Partial Moments

This example shows how to compute the zero-order, first-order, and second-order lower partial moments for the three time series, where the mean of the third time series is used to compute MAR (minimum acceptable return) with the so-called risk-free rate.

```
load FundMarketCash
Returns = tick2ret(TestData);
Assets

Assets = 1×3 cell
    {'Fund'}    {'Market'}    {'Cash'}
```



```
MAR = mean>Returns(:,3))

MAR =
0.0017

LPM = lpm>Returns, MAR, [0 1 2])

LPM = 3×3

    0.4333    0.4167    0.6167
    0.0075    0.0140    0.0004
    0.0003    0.0008    0.0000
```

The first row of LPM contains zero-order lower partial moments of the three series. The fund and market index fall below MAR about 40% of the time and cash returns fall below its own mean about 60% of the time.

The second row contains first-order lower partial moments of the three series. The fund and market have large average shortfall returns relative to MAR by 75 and 140 basis points per month. On the other hand, cash underperforms MAR by about only four basis points per month on the downside.

The third row contains second-order lower partial moments of the three series. The square root of these quantities provides an idea of the dispersion of returns that fall below the MAR. The market index has a much larger variation on the downside when compared to the fund.

Input Arguments

Data — Asset returns

matrix

Asset returns, specified as a NUMSAMPLES-by-NUMSERIES matrix with NUMSAMPLES observations of NUMSERIES asset returns.

Data Types: double

MAR — Minimum acceptable return

0 (default) | numeric

(Optional) Minimum acceptable return, specified as a scalar numeric. MAR is a cutoff level of return such that all returns above MAR contribute nothing to the lower partial moment.

Data Types: double

Order — Moment orders

0 (default) | scalar numeric | vector

(Optional) Moment orders, specified as either a scalar or a NUMORDERS vector of nonnegative integer moment orders. If no order specified, the default Order = 0, which is the shortfall probability. Although the lpm function works for noninteger orders and, in some cases, for negative orders, this falls outside customary usage.

Data Types: double

Output Arguments

Moment — Lower partial moments

matrix

Lower partial moments, returned as a NUMORDERS x NUMSERIES matrix of lower partial moments with NUMORDERS Orders and NUMSERIES series, that is, each row contains lower partial moments for a given order.

Note To compute upper partial moments, reverse the signs of both Data and MAR (do not reverse the sign of the output). The lpm function computes sample lower partial moments from data. To compute expected lower partial moments for multivariate normal asset returns with a specified mean

and covariance, use `elpm`. With `lpm`, you can compute various investment ratios such as Omega ratio, Sortino ratio, and Upside Potential ratio, where:

- $\text{Omega} = \text{lpm}(-\text{Data}, -\text{MAR}, 1) / \text{lpm}(\text{Data}, \text{MAR}, 1)$
 - $\text{Sortino} = (\text{mean}(\text{Data}) - \text{MAR}) / \sqrt{\text{lpm}(\text{Data}, \text{MAR}, 2)}$
 - $\text{Upside} = \text{lpm}(-\text{Data}, -\text{MAR}, 1) / \sqrt{\text{lpm}(\text{Data}, \text{MAR}, 2)}$
-

More About

Lower Partial Moments

Use *lower partial moments* to examine what is colloquially known as “downside risk.”

The main idea of the lower partial moment framework is to model moments of asset returns that fall below a minimum acceptable level of return. To compute lower partial moments from data, use `lpm` to calculate lower partial moments for multiple asset return series and for multiple moment orders. To compute expected values for lower partial moments under several assumptions about the distribution of asset returns, use `elpm` to calculate lower partial moments for multiple assets and for multiple orders.

Version History

Introduced in R2006b

References

- [1] Bawa, V.S. "Safety-First, Stochastic Dominance, and Optimal Portfolio Choice." *Journal of Financial and Quantitative Analysis*. Vol. 13, No. 2, June 1978, pp. 255-271.
- [2] Harlow, W.V. "Asset Allocation in a Downside-Risk Framework." *Financial Analysts Journal*. Vol. 47, No. 5, September/October 1991, pp. 28-40.
- [3] Harlow, W.V. and K. S. Rao. "Asset Pricing in a Generalized Mean-Lower Partial Moment Framework: Theory and Evidence." *Journal of Financial and Quantitative Analysis*. Vol. 24, No. 3, September 1989, pp. 285-311.
- [4] Sortino, F.A. and Robert van der Meer. "Downside Risk." *Journal of Portfolio Management*. Vol. 17, No. 5, Spring 1991, pp. 27-31.

See Also

`elpm`

Topics

“Performance Metrics Overview” on page 7-2

macd

Moving Average Convergence/Divergence (MACD)

Syntax

```
[MACDLine,SignalLine] = macd(Data)
```

Description

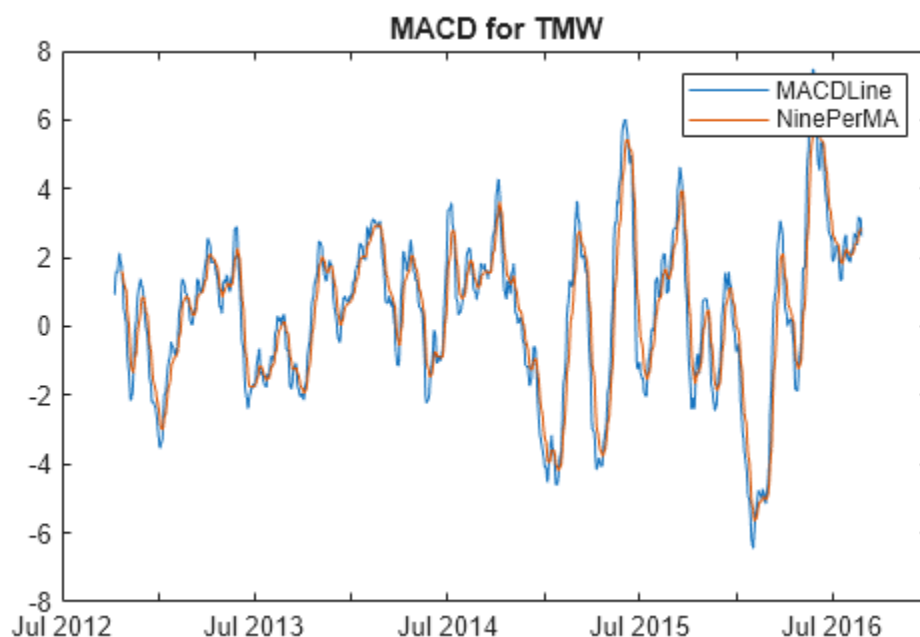
[MACDLine,SignalLine] = macd(Data) calculates the Moving Average Convergence/Divergence (MACD) line from the series of data and the nine-period exponential moving average from the MACDLine.

Examples

Calculate the Moving Average Convergence/Divergence for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat  
[MACDLine, signalLine]= macd(TMW);  
plot(MACDLine.Time,MACDLine.Close,signalLine.Time,signalLine.Close);  
legend('MACDLine','NinePerMA')  
title('MACD for TMW')
```



Input Arguments

Data — Data with high, low, open, close information

matrix | table | timetable

Data with high, low, open, close information, specified as a matrix, table, or timetable. For matrix input, **Data** is an M-by-4 matrix of high, low, opening, and closing prices. Timetables and tables with M rows must contain variables named 'High', 'Low', 'Open', and 'Close' (case insensitive).

Data Types: double | table | timetable

Output Arguments

MACDLine — MACD series

matrix | table | timetable

MACD series, returned with the same number of rows (M) and type (matrix, table, or timetable) as the input **Data**. The MACDLine is calculated by subtracting the 26-period (7.5%) exponential

SignalLine — Nine-period exponential series

matrix | table | timetable

Nine-period exponential series, returned with the same number of rows (M) and type (matrix, table, or timetable) as the input **Data**. The nine-period (20%) exponential moving average of the MACDLine is used as the SignalLine.

More About

MACD

The MACD is calculated by subtracting the 26-period (7.5%) exponential moving average from the 12-period (15%) moving average.

The nine-period (20%) exponential moving average of the MACD line is used as the "signal" line. When the two lines are plotted, they can give you indications on when to buy or sell a stock, when overbought or oversold is occurring, and when the end of trend may occur. For example, when the MACD and the 20-day moving average line have crossed and the MACD line becomes below the other line, it is time to sell.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the **Data** input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The **Data** input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 166-168.

See Also

adline | timetable | table | willad

Topics

"Use Timetables in Finance" on page 11-7

"Convert Financial Time Series Objects (fints) to Timetables" on page 11-2

maxdrawdown

Compute maximum drawdown for one or more price series

Syntax

```
MaxDD = maxdrawdown(Data)
MaxDD = maxdrawdown( ____, Format)
[MaxDD,MaxDDIndex] = maxdrawdown( ____ )
```

Description

`MaxDD = maxdrawdown(Data)` computes maximum drawdown for each series in an N-vector `MaxDD` and identifies start and end indexes of maximum drawdown periods for each series in a 2-by-N matrix `MaxDDIndex`.

`MaxDD = maxdrawdown( ____, Format)` adds an optional argument for `Format`.

`[MaxDD,MaxDDIndex] = maxdrawdown( ____ )` adds an optional output for `MaxDDIndex`.

Examples

Calculate Maximum Drawdown

Calculate the maximum drawdown (`MaxDD`) using example data with a fund, market, and cash series:

```
load FundMarketCash
MaxDD = maxdrawdown(TestData)
```

```
MaxDD = 1×3
```

```
    0.1658    0.3381         0
```

The maximum drop in the given time period was 16.58% for the fund series, and 33.81% for the market series. There was no decline in the cash series, as expected, because the cash account never loses value.

`maxdrawdown` also returns the indices (`MaxDDIndex`) of the maximum drawdown intervals for each series in an optional output argument.

```
[MaxDD, MaxDDIndex] = maxdrawdown(TestData)
```

```
MaxDD = 1×3
```

```
    0.1658    0.3381         0
```

```
MaxDDIndex = 2×3
```

```
     2     2   NaN
```


18 18 NaN

The first two series experience their maximum drawdowns from the second to the 18th month in the data. The indices for the third series are NaNs because it never has a drawdown.

The 16.58% value loss from month 2 to month 18 for the fund series is verified using the reported indices.

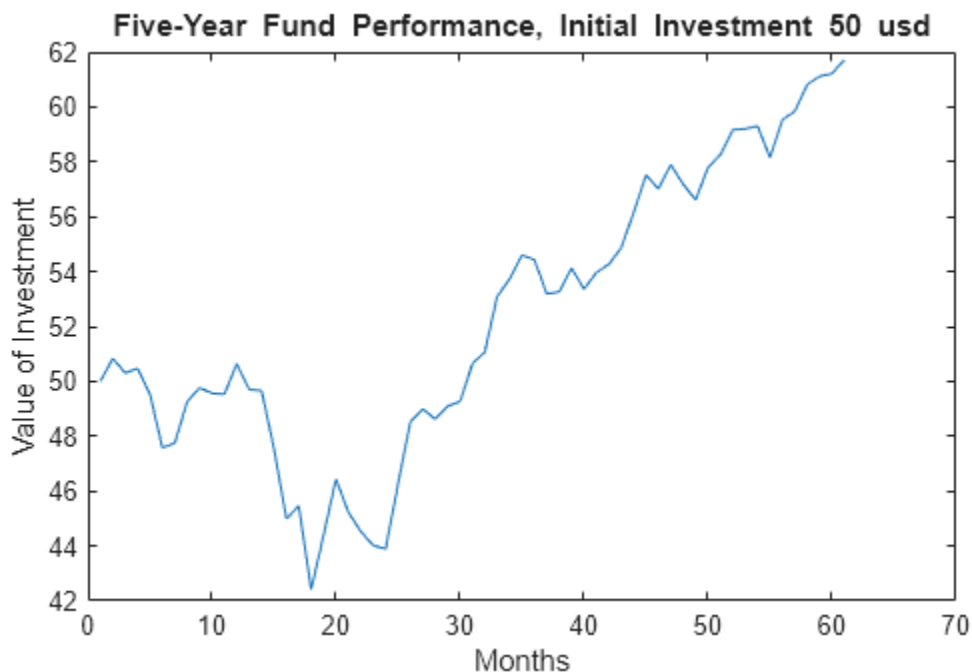
```
Start = MaxDDIndex(1,:);
End = MaxDDIndex(2,:);
(TestData(Start(1),1) - TestData(End(1),1))/TestData(Start(1),1)
```

```
ans =
0.1658
```

```
ans = 0.1658
```

Although the maximum drawdown is measured in terms of returns, `maxdrawdown` can measure the drawdown in terms of absolute drop in value, or in terms of log-returns. To contrast these alternatives more clearly, you can work with the fund series, assuming an initial investment of 50 dollars:

```
Fund50 = 50*TestData(:,1);
plot(Fund50);
title('\bfFive-Year Fund Performance, Initial Investment 50 usd');
xlabel('Months');
ylabel('Value of Investment');
```



First, compute the standard maximum drawdown, which coincides with the results above because returns are independent of the initial amounts invested.

```
MaxDD50Ret = maxdrawdown(Fund50)
```

```
MaxDD50Ret =  
0.1658
```

Next, compute the maximum drop in value, using the 'arithmetic' argument.

```
[MaxDD50Arith, Ind50Arith] = maxdrawdown(Fund50, 'arithmetic')
```

```
MaxDD50Arith =  
8.4285
```

```
Ind50Arith = 2×1
```

```
    2  
   18
```

The value of this investment was \$50.84 in month 2, but by month 18 the value was down to \$42.41, a drop of \$8.43. This is the largest loss in dollar value from a previous high in the given time period. In this case, the maximum drawdown period, from the 2nd to 18th month, is the same independently of whether drawdown is measured as return or as dollar value loss.

```
[MaxDD50LogRet, Ind50LogRet] = maxdrawdown(Fund50, 'geometric')
```

```
MaxDD50LogRet =  
0.1813
```

```
Ind50LogRet = 2×1
```

```
    2  
   18
```

Note, the last measure is equivalent to finding the arithmetic maximum drawdown for the log of the series.

```
MaxDD50LogRet2 = maxdrawdown(log(Fund50), 'arithmetic')
```

```
MaxDD50LogRet2 =  
0.1813
```

Input Arguments

Data — Total return price series

matrix

Total return price series, specified as a T-by-N matrix with T samples of N total return price series.

Data Types: double

Format — (Optional) Format of Data

character vector with value of 'return', 'arithmetic', or 'geometric'

Format of Data, specified as character vector with the following possible values:

- 'return' (default) — Maximum drawdown in terms of maximum percentage drop from a peak.
- 'arithmetic'

— Maximum drawdown of an arithmetic Brownian motion with drift (differences of data from peak to trough) using the equation

$$dX(t) = \mu dt + \sigma dW(t).$$

- 'geometric'

— Maximum drawdown of a geometric Brownian motion with drift (differences of log of data from peak to trough) using the equation

$$dS(t) = \mu_0 S(t) dt + \sigma_0 S(t) dW(t)$$

Data Types: char

Output Arguments

MaxDD — Maximum drawdown

vector

Maximum drawdown, returned as a 1-by-N vector with maximum drawdown for each of N time series.

Note

- Drawdown is the percentage drop in total returns from the start to the end of a period. If the total equity time series is increasing over an entire period, drawdown is 0. Otherwise, it is a positive number. Maximum drawdown is an ex-ante proxy for downside risk that computes the largest drawdown over all intervals of time that can be formed within a specified interval of time.
 - Maximum drawdown is sensitive to quantization error.
-

MaxDDIndex — Start and end indexes for each maximum drawdown period for each total equity time series

vector

Start and end indexes for each maximum drawdown period for each total equity time series, returned as a 2-by-N vector of start and end indexes. The first row of the vector contains the start indexes and the second row contains the end indexes of each maximum drawdown period.

More About

Maximum Drawdown

Maximum drawdown (MDD) is a key risk metric to measure the largest peak-to-trough decline in the value of an investment portfolio, mutual fund, or financial asset over a specified period.

MDD quantifies the potential loss an investor could experience during a downturn, providing insights into the risk and volatility associated with an investment.

Version History

Introduced in R2006b

References

- [1] Christian S. Pederson and Ted Rudholm-Alfvén. "Selecting a Risk-Adjusted Shareholder Performance Measure." *Journal of Asset Management*. Vol. 4, No. 3, 2003, pp. 152-172.

See Also

emaxdrawdown

Topics

"Performance Metrics Overview" on page 7-2

medprice

Median price from the series of high and low prices

Syntax

```
MedianPrice = medprice(Data)
```

Description

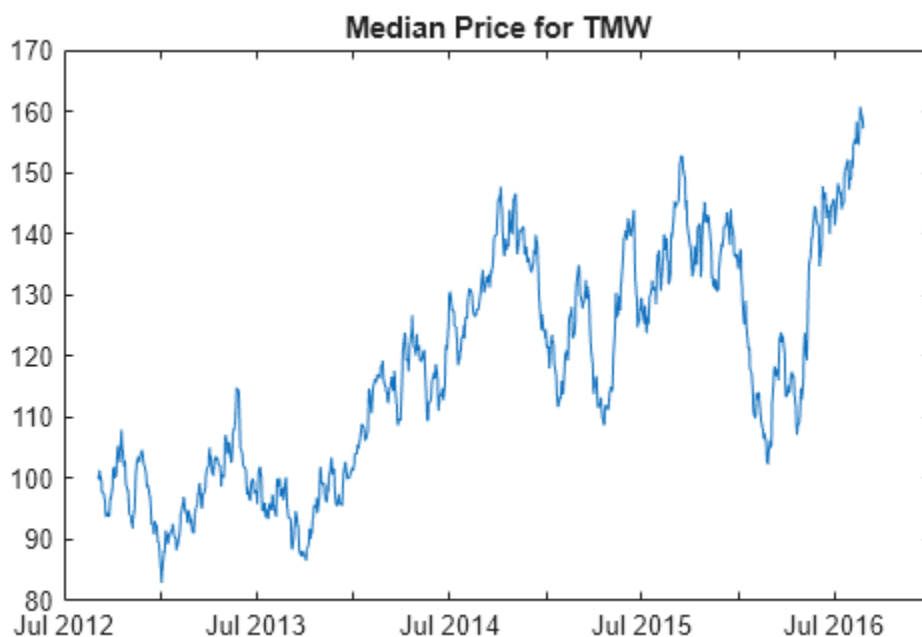
`MedianPrice = medprice(Data)` calculates the median prices from the series of high and low prices. The median price is the average of the high and low prices for each period.

Examples

Calculate the Median Prices for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat
MedianPrice = medprice(TMW);
plot(MedianPrice.Time,MedianPrice.MedianPrice)
title('Median Price for TMW')
```



Input Arguments

Data — Data for high and low prices

matrix | table | timetable

Data for high and low prices, specified as a matrix, table, or timetable. For matrix input, **Data** is an M-by-2 matrix of high and low prices stored in the corresponding columns. The columns can be in either order because the median price is the average of the high and low prices for each period. Timetables and tables with M rows must contain a variable named 'High' and 'Low' (case insensitive).

Data Types: double | table | timetable

Output Arguments

MedianPrice — Median price series

matrix | table | timetable

Median price series, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input **Data**.

Version History

Introduced before R2006a

R2023a: **fints** support removed for **Data** input argument

Behavior changed in R2023a

fints object support for the **Data** input argument is removed.

R2022b: **Support for negative price data**

Behavior changed in R2022b

The **Data** input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 177–178.

See Also

timetable | table | wclose | typprice

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

mirr

Modified internal rate of return

Syntax

`Return = mirr(CashFlow,FinRate,Reinvest)`

Description

`Return = mirr(CashFlow,FinRate,Reinvest)` calculates the modified internal rate of return for a series of periodic cash flows. This function calculates only positive rates of return; for nonpositive rates of return, `Return = 0`.

Examples

Compute Modified Internal Rate of Return

This cash flow represents the yearly income from an initial investment of \$100,000. The finance rate is 9% and the reinvestment rate is 12%.

Year 1 \$20,000

Year 2 (\$10,000)

Year 3 \$30,000

Year 4 \$38,000

Year 5 \$50,000

To calculate the modified internal rate of return on the investment:

```
Return = mirr([-100000 20000 -10000 30000 38000 50000], 0.09,0.12)
```

```
Return =  
0.0832
```

Input Arguments

CashFlow — Cash flow

vector | matrix

Cash flow, specified as a vector or matrix. The first entry is the initial investment. If `CashFlow` is entered as a matrix, each column is treated as a separate cash flow.

Data Types: double

FinRate — Finance rate for negative cash flow values

decimal

Finance rate for negative cash flow values, specified as a decimal.

Data Types: `double`

Reinvest — Reinvestment rate for positive cash flow values

`decimal`

Reinvestment rate for positive cash flow values, specified as a decimal.

Data Types: `double`

Output Arguments

Return — Modified internal rate of return

`numeric`

Modified internal rate of return, returned as a scalar or vector.

More About

Modified Internal Rate of Return

The Modified Internal Rate of Return (MIRR) is the interest rate that equates the present value of a project's negative cash flows (outflows) to the future value of its positive cash flows (inflows), using specific financing and reinvestment rates.

Version History

Introduced before R2006a

References

- [1] Brealey and Myers. *Principles of Corporate Finance*. McGraw-Hill Higher Education, Chapter 5, 2003.
- [2] Hazen G. "A New Perspective on Multiple Internal Rates of Return." *The Engineering Economist*. Vol. 48-1, 2003, pp. 31-51.

See Also

`annurate` | `effrr` | `irr` | `nomrr` | `pvvar` | `xirr`

Topics

"Analyzing and Computing Cash Flows" on page 2-11

movavg

Moving average of a financial time series

Syntax

```
ma = movavg(Data,type,windowSize)
ma = movavg( ___,initialPoints)

ma = movavg(Data,type,weights)
ma = movavg( ___,initialPoints)
```

Description

`ma = movavg(Data,type,windowSize)` computes the moving average (MA) of a financial time series.

`ma = movavg( ___,initialPoints)` adds an optional argument for `initialPoints`.

`ma = movavg(Data,type,weights)` computes the moving average (MA) of a financial time series using a 'custom' type and weights.

`ma = movavg( ___,initialPoints)` adds an optional argument for `initialPoints`.

Examples

Calculate the Moving Average for a Data Series

Load the file `SimulatedStock.mat` and compute the moving average using simulated closing price data.

```
load SimulatedStock.mat
type = 'linear';
windowSize = 14;
ma = movavg(TMW_CLOSE,type,windowSize)
```

```
ma = 1000×1
```

```
100.2500
100.3433
100.8700
100.4916
99.9937
99.3603
98.8769
98.6364
98.4348
97.8491
97.1689
96.5502
96.2904
95.8138
```

```
95.1923
:
```

Calculate the Leading and Lagging Moving Averages for a Data Series

Load the file `SimulatedStock.mat` and compute the moving average using simulated closing price data.

```
load SimulatedStock.mat
type = 'linear';
mashort_term=movavg(TMW_CLOSE,type,3) % Short-term moving average

mashort_term = 1000×1
```

```
100.2500
100.3580
101.0900
100.4300
99.3183
97.8217
97.0833
97.1950
97.4133
96.1133
94.5267
93.5517
94.3917
94.2317
93.3850
:
```

```
malong_term=movavg(TMW_CLOSE,type,20) % Long-term moving average

malong_term = 1000×1
```

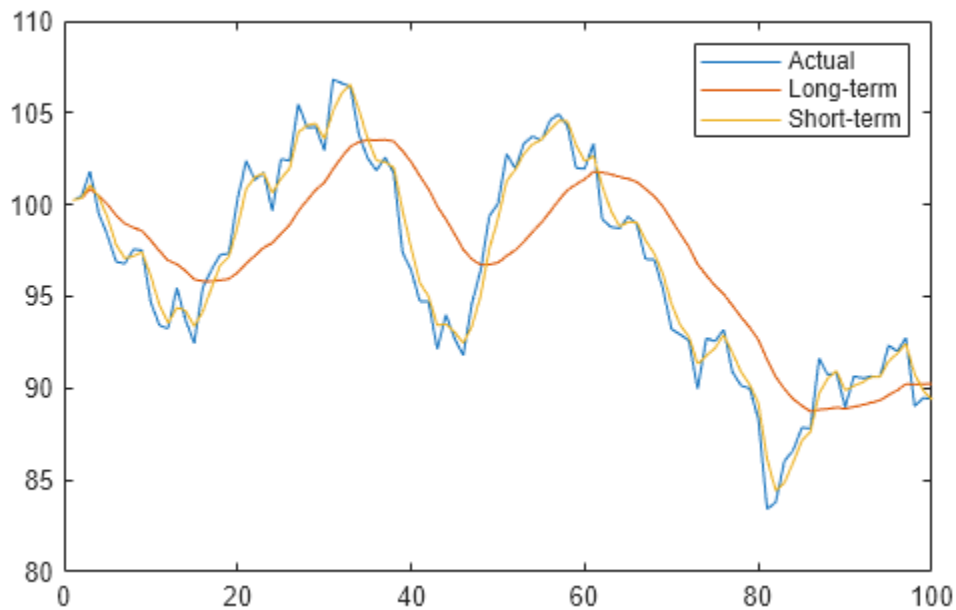
```
100.2500
100.3423
100.8574
100.4943
100.0198
99.4230
98.9728
98.7509
98.5688
98.0554
97.4737
96.9605
96.7584
96.3950
95.9392
:
```

Plot the long-term and short-term moving averages.

```

plot(TMW_CLOSE(1:100))
hold on
plot(malong_term(1:100))
plot(mashort_term(1:100))
hold off
legend('Actual', 'Long-term', 'Short-term')

```



Input Arguments

Data — Data for a financial series

matrix | table | timetable

Data for a financial series, specified as a column-oriented matrix, table, or timetable. Timetables and tables must contain variables with only a numeric type.

Data Types: double | table | timetable

type — Type of moving average to compute

character vector with value of 'simple', 'square-root', 'linear', 'square', 'exponential', 'triangular', 'modified', or 'custom' | string with value of "simple", "square-root", "linear", "square", "exponential", "triangular", "modified", or "custom"

Type of moving average to compute, specified as a character vector or string with an associated value. The types of moving average are:

- "simple" — The simple moving average (SMA) is calculated by summing up a specified number of data points and dividing the sum by the number of data points. The SMA gives equal importance to each data point within the specified period. The resulting value represents the average price over the specified period.

- "square-root" — In the square root method, the weights assigned to each data point decrease in a square root fashion as the data points move further back in time. The most recent data point has the highest weight, and the weights decrease progressively for older data points.
- "linear" — The linear method assigns weights to each data point that decrease in a linear fashion as the data points move further back in time. This method can be useful for capturing trends and price movements while minimizing the impact of outliers or extreme values.
- "square" — The square method assigns weights to each data point that decrease in a square fashion as the data points move further back in time. The most recent data point has the highest weight, and the weights decrease progressively for older data points.
- "exponential" — An exponential moving average (EMA) places a greater weight and significance on the most recent data points. The exponential moving average is also referred to as the exponentially weighted moving average. An exponentially weighted moving average reacts more significantly to recent price changes than a simple moving average (SMA), which applies an equal weight to all observations in the period.
- "triangular" — The triangular moving average (TMA) is a variation of the SMA that assigns weights to the data points in a triangular pattern. This method aims to give more weight to the data points in the middle of the moving average period and less weight to the data points at the beginning and end of the period.
- "modified" — A modified moving average (MMA) is an indicator that shows the average value of a security's price over a period of time. It is also known as the running moving average (RMA) or smoothed moving average (SMMA). Modified moving averages are similar to simple moving averages, but all subsequent points are calculated by first adding the new price and then subtracting the last average from the resulting sum.
- "custom" — A moving average using a custom method is a type of moving average that uses a custom measure of weights to calculate the average, instead of just taking the close price of each bar as most other moving averages do. The custom measure calculates the difference between the lowest lows and highest highs over different data points.

Data Types: `char` | `string`

windowSize — Number of observations of the input series to include in moving average
positive integer

Number of observations of the input series to include in moving average, specified as a scalar positive integer. The observations include (windowSize - 1) previous data points and the current data point.

Note The `windowSize` argument applies only to moving averages whose type is 'simple', 'square-root', 'linear', 'square', 'exponential', 'triangular', or 'modified'.

Data Types: `double`

weights — Custom weights used to compute moving average
vector

Custom weights used to compute the moving average, specified as a vector.

Note The length of weights (N) determines the size of the moving average window (`windowSize`). The `weights` argument applies only to a 'custom' type of moving average.

To compute moving average with custom weights, the weights (w) are first normalized such that they sum to one:

$$W(i) = w(i)/\text{sum}(w), \text{ for } i = 1, 2, \dots, N$$

The normalized weights (W) are then used to form the N -point weighted moving average (y) of the input Data (x):

$$y(t) = W(1)*x(t) + W(2)*x(t-1) + \dots + W(N)*x(t-N)$$

The initial moving average values within the window size are then adjusted according to the method specified in the name-value pair argument `initialPoints`.

Data Types: `double`

initialPoints — Indicates how moving average is calculated at initial points

'shrink' (default) | character vector with values of 'shrink', 'fill', or 'zero' | string with values of "shrink", "fill", or "zero"

(Optional) Indicates how the moving average is calculated at initial points (before there is enough data to fill the window), specified as a character vector or string using one of the following values:

- 'shrink' - Initializes the moving average such that the initial points include only observed data
- 'zero' - Initializes the initial points with 0
- 'fill' - Fills initial points with NaNs

Note The `initialPoints` argument applies to all type specifications except for the 'exponential' and 'modified' options.

Data Types: `char` | `string`

Output Arguments

ma — Moving average series

`matrix` | `table` | `timetable`

Moving average series, returned with the same number of rows (M) and the same type (`matrix`, `table`, or `timetable`) as the input Data.

More About

Moving Average

A moving average is a statistical calculation used to analyze data points by creating averages of different subsets of the complete data set.

Moving averages are commonly used to smooth out price data over a specified period, helping to identify trends and reduce noise in price movements.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument*Behavior changed in R2023a*

fints object support for the Data input argument is removed.

R2018a: Lead and Lag input arguments not supported*Behavior changed in R2018a*

The syntax for movavg has changed. There is no longer support for the input arguments Lead and Lag, only a single windowSize is supported, and there is only one output argument (ma). If you want to compute the leading and lagging moving averages, you need to run movavg twice and adjust the windowSize.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 184-192.

See Also

timetable | table | bollinger | candle | dateaxis | highlow | pointfig

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

mvnrfish

Fisher information matrix for multivariate normal or least-squares regression

Syntax

```
Fisher = mvnrfish(Data,Design,Covariance)
Fisher = mvnrfish( ___,MatrixFormat,CovarFormat)
```

Description

`Fisher = mvnrfish(Data,Design,Covariance)` computes a Fisher information matrix based on current maximum likelihood or least-squares parameter estimates.

Fisher is a TOTALPARAMS-by-TOTALPARAMS Fisher information matrix. The size of TOTALPARAMS depends on MatrixFormat and on current parameter estimates. If `MatrixFormat = 'full'`,

$$\text{TOTALPARAMS} = \text{NUMPARAMS} + \text{NUMSERIES} * (\text{NUMSERIES} + 1)/2$$

If `MatrixFormat = 'paramonly'`,

$$\text{TOTALPARAMS} = \text{NUMPARAMS}$$

Note mvnrfish operates slowly if you calculate the full Fisher information matrix.

`Fisher = mvnrfish( ___,MatrixFormat,CovarFormat)` computes a Fisher information matrix based on current maximum likelihood or least-squares parameter estimates using optional arguments.

Input Arguments

Data — Data sample

matrix

Data sample, specified as an NUMSAMPLES-by-NUMSERIES matrix with NUMSAMPLES samples of a NUMSERIES-dimensional random vector. If a data sample has missing values, represented as NaNs, the sample is ignored. (Use `mvnrmle` to handle missing data.)

Data Types: double

Design — Model design

matrix | cell array of character vectors

Model design, specified as a matrix or a cell array that handles two model structures:

- If `NUMSERIES = 1`, Design is a NUMSAMPLES-by-NUMPARAMS matrix with known values. This structure is the standard form for regression on a single series.
- If `NUMSERIES ≥ 1`, Design is a cell array. The cell array contains either one or NUMSAMPLES cells. Each cell contains a NUMSERIES-by-NUMPARAMS matrix of known values.

If `Design` has a single cell, it is assumed to have the same `Design` matrix for each sample. If `Design` has more than one cell, each cell contains a `Design` matrix for each sample.

Data Types: `double` | `cell`

Covariance — Estimates for the covariance of the residuals of the regression matrix

Estimates for the covariance of the residuals of the regression, specified as an `NUMSERIES`-by-`NUMSERIES` matrix.

Data Types: `double`

MatrixFormat — Parameters to be included in Fisher information matrix

`'full'` (default) | character vector with value `'full'` or `'paramonly'`

(Optional) Parameters to be included in the Fisher information matrix, specified as a character vector. The choices are:

- `'full'` — This is the default method that computes the full Fisher information matrix for both model and covariance parameter estimates.
- `'paramonly'` — This computes only components of the Fisher information matrix associated with the model parameter estimates.

Data Types: `char`

CovarFormat — Format for covariance matrix

`'full'` (default) | character vector with value `'full'` or `'diagonal'`

(Optional) Format for the covariance matrix, specified as a character vector. The choices are:

- `'full'` — This is the default method that computes the full covariance matrix.
- `'diagonal'` — This forces the covariance matrix to be a diagonal matrix.

Data Types: `char`

Output Arguments

Fisher — Fisher information matrix

matrix

Fisher information matrix, returned as an `TOTALPARAMS`-by-`TOTALPARAMS` matrix.

More About

Fisher Information Matrix

The Fisher information matrix is a key concept in statistics and information theory that measures the amount of information that an observable random variable carries about an unknown parameter upon which the probability of the random variable depends.

The Fisher information matrix is used to evaluate the efficiency of estimators and to derive confidence intervals.

Version History

Introduced in R2006a

See Also

`mvnrstd` | `mvnrml`

Topics

- "Multivariate Normal Regression With Missing Data" on page 9-14
- "Multivariate Normal Regression Without Missing Data" on page 9-13
- "Least-Squares Regression With Missing Data" on page 9-14
- "Least-Squares Regression Without Missing Data" on page 9-14
- "Fisher Information" on page 9-4
- "Multivariate Normal Linear Regression" on page 9-2
- "Least-Squares Regression" on page 9-4

mvnrmle

Multivariate normal regression (ignore missing data)

Syntax

```
[Parameters,Covariance,Resid,Info] = mvnrmle(Data,Design)
[Parameters,Covariance,Resid,Info] = mvnrmle( ___,MaxIterations,TolParam,
TolObj,Covar0,CovarFormat)
```

Description

[Parameters,Covariance,Resid,Info] = mvnrmle(Data,Design) estimates a multivariate normal regression model without missing data. The model has the form

$$Data_k \sim N(Design_k \times Parameters, Covariance)$$

for samples $k = 1, \dots, \text{NUMSAMPLES}$.

mvnrmle estimates a NUMPARAMS-by-1 column vector of model parameters called **Parameters**, and a NUMSERIES-by-NUMSERIES matrix of covariance parameters called **Covariance**.

mvnrmle(Data, Design) with no output arguments plots the log-likelihood function for each iteration of the algorithm.

[Parameters,Covariance,Resid,Info] = mvnrmle(___,MaxIterations,TolParam,TolObj,Covar0,CovarFormat) estimates a multivariate normal regression model without missing data using optional arguments.

Input Arguments

Data — Data sample

matrix

Data sample, specified as an NUMSAMPLES-by-NUMSERIES matrix with NUMSAMPLES samples of a NUMSERIES-dimensional random vector. If a data sample has missing values, represented as NaNs, the sample is ignored. (Use mvnrmle to handle missing data.)

Data Types: double

Design — Model design

matrix | cell array of character vectors

Model design, specified as a matrix or a cell array that handles two model structures:

- If NUMSERIES = 1, Design is a NUMSAMPLES-by-NUMPARAMS matrix with known values. This structure is the standard form for regression on a single series.
- If NUMSERIES ≥ 1, Design is a cell array. The cell array contains either one or NUMSAMPLES cells. Each cell contains a NUMSERIES-by-NUMPARAMS matrix of known values.

If Design has a single cell, it is assumed to have the same Design matrix for each sample. If Design has more than one cell, each cell contains a Design matrix for each sample.

These points concern how `Design` handles missing data:

- Although `Design` should not have NaN values, ignored samples due to NaN values in `Data` are also ignored in the corresponding `Design` array.
- If `Design` is a 1-by-1 cell array, which has a single `Design` matrix for each sample, no NaN values are permitted in the array. A model with this structure must have `NUMSERIES ≥ Numparms` with `rank(Design{1}) = Numparms`.
- Two functions for handling missing data, `ecmmvnrmle` and `ecmlsrmle`, are stricter about the presence of NaN values in `Design`.

Data Types: `double` | `cell`

MaxIterations — Maximum number of iterations for the estimation algorithm

100 (default) | numeric

(Optional) Maximum number of iterations for the estimation algorithm, specified as a numeric. The default value is 100.

Data Types: `double`

TolParam — Convergence tolerance for estimation algorithm

`sqrt(eps)` (default) | numeric

(Optional) Convergence tolerance for estimation algorithm based on changes in model parameter estimates, specified as a numeric. The Default value is `sqrt(eps)` which is about 1.0e-8 for double precision. The convergence test for changes in model parameters is

$$\|Param_k - Param_{k-1}\| < TolParam \times (1 + \|Param_k\|)$$

where `Param` represents the output Parameters, and iteration $k = 2, 3, \dots$. Convergence is assumed when both the `TolParam` and `TolObj` conditions are satisfied. If both $TolParam \leq 0$ and $TolObj \leq 0$, do the maximum number of iterations (`MaxIterations`), whatever the results of the convergence tests.

Data Types: `double` | `table` | `timetable`

TolObj — Convergence tolerance for estimation algorithm based on changes in objective function

$eps \wedge 3/4$ (default) | numeric

(Optional) Convergence tolerance for estimation algorithm based on changes in the objective function, specified as a numeric. The default value is $eps \wedge 3/4$ which is about 1.0e-12 for double precision. The convergence test for changes in the objective function is

$$|Obj_k - Obj_{k-1}| < TolObj \times (1 + |Obj_k|)$$

for iteration $k = 2, 3, \dots$. Convergence is assumed when both the `TolParam` and `TolObj` conditions are satisfied. If both $TolParam \leq 0$ and $TolObj \leq 0$, do the maximum number of iterations (`MaxIterations`), whatever the results of the convergence tests.

Data Types: `double`

Covar0 — User-supplied initial or known estimate for the covariance matrix of the regression residuals

identity matrix (default) | matrix

(Optional) user-supplied initial or known estimate for the covariance matrix of the regression residuals, specified as an NUMSERIES-by-NUMSERIES matrix.

Data Types: double

CovarFormat — Format for covariance matrix

'full' (default) | character vector with value 'full' or 'diagonal'

(Optional) Format for the covariance matrix, specified as a character vector. The choices are:

- 'full' — This is the default method that computes the full covariance matrix.
- 'diagonal' — This forces the covariance matrix to be a diagonal matrix.

Data Types: char

Output Arguments

Parameters — Parameters of the regression model

vector

Parameters of the regression model, returned as an NUMPARAMS-by-1 column vector of estimates for the parameters of the regression model.

Covariance — Covariance of the regression model's residuals

matrix

Covariance of the regression model's residuals, returned as an NUMSERIES-by-NUMSERIES matrix of estimates for the covariance of the regression model's residuals.

Resid — Residuals from the regression

matrix

Residuals from the regression, returned as an NUMSAMPLES-by-NUMSERIES matrix of residuals from the regression. For any row with missing values in `Data`, the corresponding row of residuals is represented as all NaN missing values, since this routine ignores rows with NaN values.

Info — Structure containing additional information from the regression

structure

Structure containing additional information from the regression, returned as a structure. The structure has these fields:

- `Info.Obj` – A variable-extent column vector, with no more than `MaxIterations` elements, that contain each value of the objective function at each iteration of the estimation algorithm. The last value in this vector, `Obj(end)`, is the terminal estimate of the objective function. If you do maximum likelihood estimation, the objective function is the log-likelihood function.
- `Info.PrevParameters` – NUMPARAMS-by-1 column vector of estimates for the model parameters from the iteration just before the terminal iteration.
- `Info.PrevCovariance` – NUMSERIES-by-NUMSERIES matrix of estimates for the covariance parameters from the iteration just before the terminal iteration.

Use the estimates in the output structure `Info` for diagnostic purposes.

More About

Multivariate Normal Regression

Multivariate normal regression is a statistical modeling approach that extends traditional regression techniques to situations where the dependent variable is multivariate, meaning it consists of multiple correlated outcomes.

Multivariate normal regression is particularly useful in fields where researchers are interested in understanding the relationships between several response variables and one or more predictor variables.

Version History

Introduced in R2006a

References

- [1] Roderick J. A. Little and Donald B. Rubin. *Statistical Analysis with Missing Data.*, 2nd Edition. John Wiley & Sons, Inc., 2002.
- [2] Xiao-Li Meng and Donald B. Rubin. "Maximum Likelihood Estimation via the ECM Algorithm." *Biometrika*. Vol. 80, No. 2, 1993, pp. 267-278.

See Also

ecmmvnrmle | mvnrstd | mvnrobj | mvregress

Topics

- "Multivariate Normal Regression" on page 9-13
- "Least-Squares Regression" on page 9-14
- "Covariance-Weighted Least Squares" on page 9-14
- "Feasible Generalized Least Squares" on page 9-15
- "Seemingly Unrelated Regression" on page 9-16
- "Multivariate Normal Regression With Missing Data" on page 9-14
- "Multivariate Normal Regression Without Missing Data" on page 9-13
- "Capital Asset Pricing Model with Missing Data" on page 9-34
- "Multivariate Normal Linear Regression" on page 9-2

mvnrobj

Log-likelihood function for multivariate normal regression without missing data

Syntax

```
Objective = mvnrobj(Data,Design,Parameters,Covariance)
Objective = mvnrobj( ___,CovarFormat)
```

Description

`Objective = mvnrobj(Data,Design,Parameters,Covariance)` computes the log-likelihood function based on current maximum likelihood parameter estimates without missing data. `Objective` is a scalar that contains the log-likelihood function.

`Objective = mvnrobj( ___,CovarFormat)` computes the log-likelihood function based on current maximum likelihood parameter estimates without missing data using an optional argument.

Input Arguments

Data — Data sample

matrix

Data sample, specified as a NUMSAMPLES-by-NUMSERIES matrix with NUMSAMPLES samples of a NUMSERIES-dimensional random vector. If a data sample has missing values, represented as NaNs, the sample is ignored. (Use `ecmmvnrmle` to handle missing data.)

Data Types: double

Design — Model design

matrix | cell array of character vectors

Model design, specified as a matrix or a cell array that handles two model structures:

- If `NUMSERIES = 1`, `Design` is a NUMSAMPLES-by-NUMPARAMS matrix with known values. This structure is the standard form for regression on a single series.
- If `NUMSERIES ≥ 1`, `Design` is a cell array. The cell array contains either one or NUMSAMPLES cells. Each cell contains a NUMSERIES-by-NUMPARAMS matrix of known values.

If `Design` has a single cell, it is assumed to have the same `Design` matrix for each sample. If `Design` has more than one cell, each cell contains a `Design` matrix for each sample.

Data Types: double | cell

Parameters — Estimates for the parameters of regression model

vector

Estimates for the parameters of regression model, specified as a NUMPARAMS-by-1 column vector.

Data Types: double

Covariance — Estimates for the covariance of the residuals of the regression matrix

Estimates for the covariance of the residuals of the regression, specified as an NUMSERIES-by-NUMSERIES matrix.

Data Types: double

CovarFormat — Format for covariance matrix

'full' (default) | character vector with value 'full' or 'diagonal'

(Optional) Format for the covariance matrix, specified as a character vector. The choices are:

- 'full' — This is the default method that computes the full covariance matrix.
- 'diagonal' — This forces the covariance matrix to be a diagonal matrix.

Data Types: char

Output Arguments

Objective — Log-likelihood function

scalar

Log-likelihood function, returned as scalar.

More About

Multivariate Normal Regression

Multivariate normal regression is a statistical modeling approach that extends traditional regression techniques to situations where the dependent variable is multivariate, meaning it consists of multiple correlated outcomes.

Multivariate normal regression is particularly useful in fields where researchers are interested in understanding the relationships between several response variables and one or more predictor variables.

Version History

Introduced in R2006a

See Also

ecmmvnrml | ecmmvnrobj | mvnrml

Topics

“Multivariate Normal Regression Without Missing Data” on page 9-13

“Multivariate Normal Regression” on page 9-13

“Least-Squares Regression” on page 9-14

“Covariance-Weighted Least Squares” on page 9-14

“Feasible Generalized Least Squares” on page 9-15

“Seemingly Unrelated Regression” on page 9-16

“Multivariate Normal Linear Regression” on page 9-2

mvnrstd

Evaluate standard errors for multivariate normal regression model

Syntax

```
[StdParameters,StdCovariance] = mvnrstd(Data,Design,Covariance)
[StdParameters,StdCovariance] = mvnrstd( ___,CovarFormat)
```

Description

`[StdParameters,StdCovariance] = mvnrstd(Data,Design,Covariance)` evaluates standard errors for a multivariate normal regression model without missing data. The model has the form

$$Data_k \sim N(Design_k \times Parameters, Covariance)$$

for samples $k = 1, \dots, NUMSAMPLES$.

`mvnrstd` computes two outputs:

- `StdParameters` is a `NUMPARAMS`-by-1 column vector of standard errors for each element of `Parameters`, the vector of estimated model parameters.
- `StdCovariance` is a `NUMSERIES`-by-`NUMSERIES` matrix of standard errors for each element of `Covariance`, the matrix of estimated covariance parameters.

Note `mvnrstd` operates slowly when you calculate the standard errors associated with the covariance matrix `Covariance`.

`[StdParameters,StdCovariance] = mvnrstd( ___,CovarFormat)` computes the log-likelihood function based on current maximum likelihood parameter estimates without missing data using an optional argument.

Input Arguments

Data — Data sample

matrix

Data sample, specified as an `NUMSAMPLES`-by-`NUMSERIES` matrix with `NUMSAMPLES` samples of a `NUMSERIES`-dimensional random vector. If a data sample has missing values, represented as NaNs, the sample is ignored. (Use `ecmmvnrml` to handle missing data.)

Data Types: double

Design — Model design

matrix | cell array of character vectors

Model design, specified as a matrix or a cell array that handles two model structures:

- If `NUMSERIES = 1`, `Design` is a `NUMSAMPLES`-by-`NUMPARAMS` matrix with known values. This structure is the standard form for regression on a single series.

- If `NUMSERIES` $\geq$ 1, `Design` is a cell array. The cell array contains either one or `NUMSAMPLES` cells. Each cell contains a `NUMSERIES`-by-`NUMPARAMS` matrix of known values.

If `Design` has a single cell, it is assumed to have the same `Design` matrix for each sample. If `Design` has more than one cell, each cell contains a `Design` matrix for each sample.

Data Types: `double` | `cell`

Covariance — Estimates for the covariance of the residuals of the regression matrix

Estimates for the covariance of the residuals of the regression, specified as an `NUMSERIES`-by-`NUMSERIES` matrix.

Data Types: `double`

CovarFormat — Format for covariance matrix

'full' (default) | character vector with value 'full' or 'diagonal'

(Optional) Format for the covariance matrix, specified as a character vector. The choices are:

- 'full' — This is the default method that computes the full covariance matrix.
- 'diagonal' — This forces the covariance matrix to be a diagonal matrix.

Data Types: `char`

Output Arguments

StdParameters — standard errors for each element of Parameters, the vector of estimated model parameters

vector

Standard errors for each element of `Parameters`, the vector of estimated model parameters, returned as an `NUMPARAMS`-by-1 column vector.

StdCovariance — standard errors for each element of Covariance, the matrix of estimated covariance parameters

matrix

Standard errors for each element of `Covariance`, the matrix of estimated covariance parameters, returned as an `NUMSERIES`-by-`NUMSERIES` matrix.

More About

Multivariate Normal Regression

Multivariate normal regression is a statistical modeling approach that extends traditional regression techniques to situations where the dependent variable is multivariate, meaning it consists of multiple correlated outcomes.

Multivariate normal regression is particularly useful in fields where researchers are interested in understanding the relationships between several response variables and one or more predictor variables.

Version History

Introduced in R2006a

References

[1] Roderick J. A. Little and Donald B. Rubin. *Statistical Analysis with Missing Data.*, 2nd Edition. John Wiley & Sons, Inc., 2002.

See Also

`ecmmvnrml` | `ecmmvnrstd` | `mvnrml`

Topics

“Multivariate Normal Regression Without Missing Data” on page 9-13

“Multivariate Normal Regression” on page 9-13

“Least-Squares Regression” on page 9-14

“Covariance-Weighted Least Squares” on page 9-14

“Feasible Generalized Least Squares” on page 9-15

“Seemingly Unrelated Regression” on page 9-16

“Multivariate Normal Linear Regression” on page 9-2

negvolidx

Negative volume index

Syntax

```
volume = negvolidx(Data)  
volume = negvolidx( ___,Name,Value)
```

Description

`volume = negvolidx(Data)` calculates the negative volume index from the series of closing stock prices and trade volume.

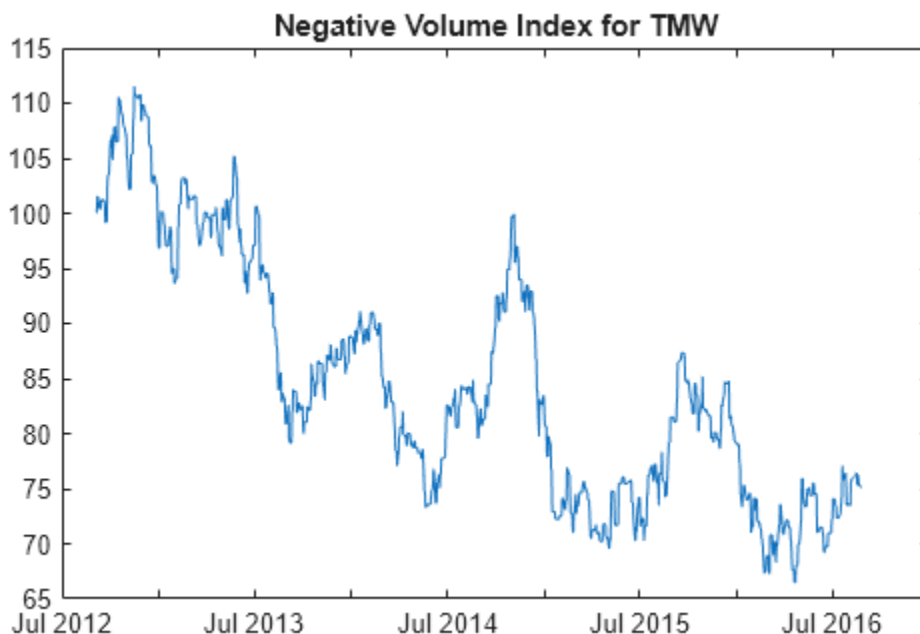
`volume = negvolidx( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Calculate the Negative Volume Index for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat  
volume = negvolidx(TMW);  
plot(volume.Time,volume.NegativeVolume)  
title('Negative Volume Index for TMW')
```



Input Arguments

Data — Data with closing prices and trade volume

matrix | table | timetable

Data with closing prices and trade volume, specified as a matrix, table, or timetable. For matrix input, **Data** is an M-by-2 with closing prices and trade volume stored in the first and second columns. Timetables and tables with M rows must contain variables named 'Close' and 'Volume' (case insensitive).

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as **Name1=Value1, ..., NameN=ValueN**, where **Name** is the argument name and **Value** is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `volume = negvolidx(TMW,'InitialValue',500)`

InitialValue — Initial value for negative volume index

100 (default) | positive integer

Initial value for negative volume index, specified as the comma-separated pair consisting of 'InitialValue' and a scalar positive integer.

Data Types: double

Output Arguments

volume — Negative volume index

matrix | table | timetable

Negative volume index, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input **Data**.

More About

Negative Volume Index

A negative volume index is a technical analysis indicator used to assess the strength of a trend based on the volume of trading activity shows the days when the trading volume of a particular security is substantially lower than other days.

The negative volume index is particularly useful for identifying potential market reversals and confirming trends by analyzing how price changes relate to changes in volume.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument*Behavior changed in R2023a*

fints object support for the Data input argument is removed.

R2022b: Support for negative price data*Behavior changed in R2022b*

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 193-194.

See Also

timetable | table | onbalvol | posvolidx

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

nomrr

Nominal rate of return

Syntax

```
Return = nomrr(Rate,NumPeriods)
```

Description

`Return = nomrr(Rate,NumPeriods)` calculates the nominal rate of return.

Examples

Calculate the Nominal Rate of Return

This example shows how to calculate the nominal rate of return based on an effective annual percentage rate of 9.38% compounded monthly.

```
Return = nomrr(0.0938, 12)
```

```
Return =  
0.0900
```

Input Arguments

Rate — Effective annual percentage rate

decimal

Effective annual percentage rate, specified as a decimal.

Data Types: double

NumPeriods — Number of compounding periods per year

scalar numeric

Number of compounding periods per year, specified as a scalar numeric.

Data Types: double

Output Arguments

Return — Nominal rate of return

numeric

Nominal rate of return, returned as a numeric.

More About

Nominal Rate of Return

The nominal rate of return is the percentage increase in the value of an investment over a specific period, not adjusted for inflation.

The nominal rate of return represents the raw return on an investment, which includes any income generated (such as interest, dividends, or capital gains) without taking into account the erosion of purchasing power due to inflation.

Version History

Introduced before R2006a

See Also

[effrr](#) | [irr](#) | [mirr](#) | [taxedrr](#) | [xirr](#)

Topics

“Analyzing and Computing Cash Flows” on page 2-11

nyseclosures

New York Stock Exchange closures from 1885 to 2070

Syntax

```
Closures = nyseclosures  
[Closures,SatTransition] = nyseclosures(StartDate,EndDate,WorkWeekFormat)
```

Description

`Closures = nyseclosures` returns a vector of dates for all known or anticipated closures from January 1, 1885 to December 31, 2070.

Since the New York Stock Exchange was open on Saturdays before September 29, 1952, exact closures from 1885 to 1952 are based on a 6-day workweek. `nyseclosures` contains all holiday and special non-trading days for the New York Stock Exchange from 1885 through 2070 based on a six-day work week (always closed on Sundays).

`[Closures,SatTransition] = nyseclosures(StartDate,EndDate,WorkWeekFormat)`, using optional input arguments, returns a vector of dates corresponding to market closures between `StartDate` and `EndDate`, inclusive.

Since the New York Stock Exchange was open on Saturdays before September 29, 1952, exact closures from 1885 to 1952 are based on a 6-day workweek. `nyseclosures` contains all holiday and special non-trading days for the New York Stock Exchange from 1885 through 2070 based on a six-day work week (always closed on Sundays). Use `WorkWeekFormat` to modify the list of dates.

Examples

Find NYSE Closures

Find the NYSE closures for 1899:

```
datestr(nyseclosures('1-jan-1899','31-dec-1899'),'dd-mmm-yyyy ddd')
```

```
ans = 16×15 char array  
    '02-Jan-1899 Mon'  
    '11-Feb-1899 Sat'  
    '13-Feb-1899 Mon'  
    '22-Feb-1899 Wed'  
    '31-Mar-1899 Fri'  
    '29-May-1899 Mon'  
    '30-May-1899 Tue'  
    '03-Jul-1899 Mon'  
    '04-Jul-1899 Tue'  
    '04-Sep-1899 Mon'  
    '29-Sep-1899 Fri'  
    '30-Sep-1899 Sat'  
    '07-Nov-1899 Tue'  
    '25-Nov-1899 Sat'
```



```
'30-Nov-1899 Thu'
'25-Dec-1899 Mon'
```

Find the NYSE closures for 1899 using a datetime array:

```
[Closures,SatTransition] = nyseclosures(datetime(1899,1,1),datetime(1899,6,30))
```

```
Closures = 7×1 datetime
    02-Jan-1899
    11-Feb-1899
    13-Feb-1899
    22-Feb-1899
    31-Mar-1899
    29-May-1899
    30-May-1899
```

```
SatTransition = datetime
    29-Sep-1952
```

Find the NYSE closure dates using the 'Archaic' value for WorkWeekFormat:

```
nyseclosures(datetime(1952,9,1),datetime(1952,10,31),'a')
```

```
ans = 10×1 datetime
    01-Sep-1952
    06-Sep-1952
    13-Sep-1952
    20-Sep-1952
    27-Sep-1952
    04-Oct-1952
    11-Oct-1952
    13-Oct-1952
    18-Oct-1952
    25-Oct-1952
```

The exchange was closed on Saturdays for much of 1952 before the official transition to a 5-day workweek.

Input Arguments

StartDate — Start date

start of default date range, January 1, 1885 (default) | datetime array | string array | date character vector

Start date, specified as a scalar or a vector using a datetime array, string array, or date character vectors.

To support existing code, `nyseclosures` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

EndDate — End date

end of default date range, December 31, 2070 (default) | datetime array | string array | date character vector

End date, specified as a scalar or a vector using a datetime array, string array, or date character vectors.

To support existing code, `nyseclosures` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

WorkWeekFormat — Method to handle the workweek

'Implicit' (default) | date character vector with values 'Modern', 'Implicit', or 'Archaic'

Method to handle the workweek, specified using a date character vector with values 'Modern', 'Implicit', or 'Archaic'. This function accepts the first letter for each method as input and is not case-sensitive. Acceptable values are:

- 'Modern' — 5-day workweek with all Saturday trading days removed.
- 'Implicit' — 6-day workweek until 1952 and 5-day week afterward (no need to exclude Saturdays).
- 'Archaic' — 6-day workweek throughout and Saturdays treated as closures after 1952.

Data Types: char

Output Arguments

Closures — Market closures between StartDate and EndDate, inclusive

vector

Market closures between the `StartDate` and `EndDate`, inclusive, returned as a vector of dates.

If `StartDate` or `EndDate` are all either strings or date character vectors, both `Closures` and `SatTransition` are returned as serial date numbers. Use the function `datetime` to convert serial date numbers to formatted datetime arrays. If either `StartDate` or `EndDate` are datetime arrays, both `Closures` and `SatTransition` are returned as datetime arrays.

If both `StartDate` and `EndDate` are not specified or are empty, `Closures` contains all known or anticipated closures from January 1, 1885 to December 31, 2070 based on a `WorkWeekFormat` of 'implicit'.

SatTransition — Date of transition for New York Stock Exchange from 6-day workweek to 5-day workweek

serial date number | datetime array

Date of transition for the New York Stock Exchange from a 6-day workweek to a 5-day workweek, returned as the date September 29, 1952 (serial date number 713226).

If `StartDate` or `EndDate` are all either strings or date character vectors, both `Closures` and `SatTransition` are returned as serial date numbers. Use the function `datetime` to convert serial date numbers to formatted datetime arrays. If either `StartDate` or `EndDate` are datetime arrays, both `Closures` and `SatTransition` are returned as datetime arrays.

More About

holidays

The `holidays` function is based on a modern 5-day workweek.

This function contains all holidays and special nontrading days for the New York Stock Exchange from January 1, 1885 to December 31, 2070.

Since the New York Stock Exchange was open on Saturdays before September 29, 1952, exact closures from 1885 to 2070 should include Saturday trading days. To capture these dates, use `nyseclosures`. The results from `holidays` and `nyseclosures` are identical if the `WorkWeekFormat` in `nyseclosures` is 'Modern'.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `nyseclosures` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datetime");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`busdate` | `createholidays` | `fbusdate` | `isbusday` | `lbusdate` | `holidays` | `datetime`

Topics

“Handle and Convert Dates” on page 2-2

“Trading Calendars User Interface” on page 12-2

“UICalendar User Interface” on page 12-4

onbalvol

On-Balance Volume (OBV)

Syntax

```
volume = onbalvol(Data)
```

Description

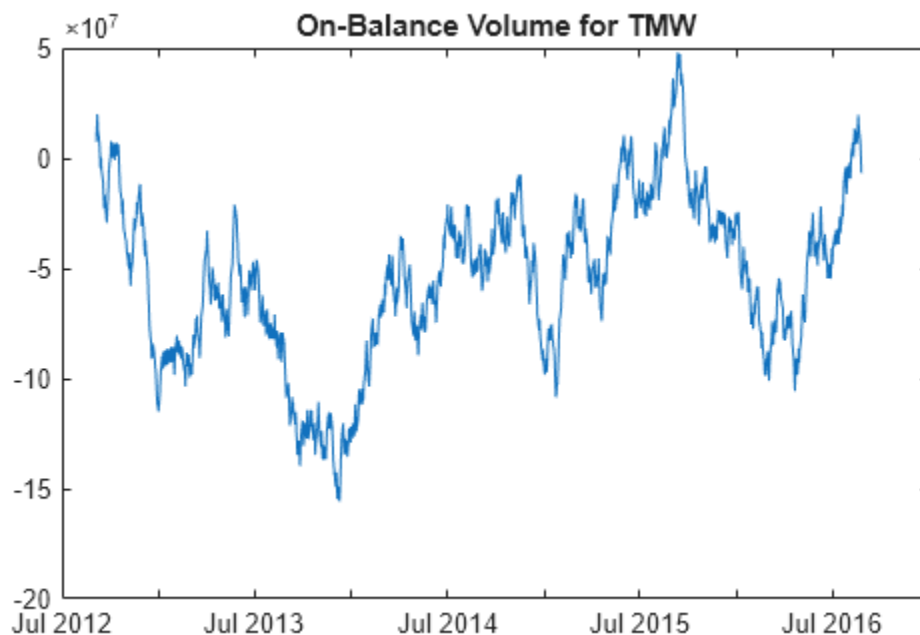
`volume = onbalvol(Data)` calculates the On-Balance Volume from the series of closing stock prices and trade volume.

Examples

Calculate the On-Balance Volume for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat
volume = onbalvol(TMW);
plot(volume.Time, volume.OnBalanceVolume)
title('On-Balance Volume for TMW')
```



Input Arguments

Data — Data for closing prices and trade volume

matrix | table | timetable

Data for closing prices and trade volume, specified as a matrix, table, or timetable. For matrix input, Data is an M-by-2 matrix of closing prices and trade volume stored in the first and second columns. Timetables and tables with M rows must contain variables named 'Close' and 'Volume' (case insensitive).

Data Types: double | table | timetable

Output Arguments

volume — On-Balance Volume

matrix | table | timetable

On-Balance Volume, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input Data.

More About

On-Balance Volume

The on-balance volume (OBV) is a technical analysis indicator that uses volume flow to predict changes in stock price.

The concept behind OBV is that volume precedes price movement; therefore, when a security's volume increases significantly without a corresponding price change, it can signal that a price movement is forthcoming.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the Data input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 207-209.

See Also

timetable | table | negvolidx | posvolidx

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

opprofit

Option profit

Syntax

```
Profit = opprofit(AssetPrice,Strike,Cost,PosFlag,OptType)
```

Description

`Profit = opprofit(AssetPrice,Strike,Cost,PosFlag,OptType)` returns the profit of an option.

Examples

Calculate the Profit of an Option

This example shows how to return the profit of an option. For example, consider buying (going long on) a call option with a strike price of \$90 on an underlying asset with a current price of \$100 for a cost of \$4.

```
Profit = opprofit(100, 90, 4, 0, 0)
```

```
Profit =  
6
```

Input Arguments

AssetPrice — Asset price

numeric

Asset price, specified as a scalar or a NINST-by-1 vector.

Data Types: double

Strike — Strike or exercise price

numeric

Strike or exercise price, specified as a scalar or a NINST-by-1 vector.

Data Types: double

Cost — Cost of option

numeric

Cost of the option, specified as a scalar or a NINST-by-1 vector.

Data Types: double

PosFlag — Option position

0 = long | 1 = short

Option position, specified as a scalar or a NINST-by-1 vector using the values 0 (long) or 1 (short).

Data Types: `logical`

OptType — Option type

0 = call option | 1 = put option

Option type, specified as a scalar or a NINST-by-1 vector using the values 0 (call option) or 1 (put option).

Data Types: `logical`

Output Arguments**Profit — Option profit**

vector

Option profit, returned as a scalar or a NINST-by-1 vector.

Version History

Introduced before R2006a

See Also

`binprice` | `blsprice`

Topics

“Pricing and Analyzing Equity Derivatives” on page 2-39

“Greek-Neutral Portfolios of European Stock Options” on page 10-14

“Plotting Sensitivities of an Option” on page 10-25

“Plotting Sensitivities of a Portfolio of Options” on page 10-27

payadv

Periodic payment given number of advance payments

Syntax

`Payment = payadv(Rate,NumPeriods,PresentValue,FutureValue,Advance)`

Description

`Payment = payadv(Rate,NumPeriods,PresentValue,FutureValue,Advance)` computes the periodic payment given a number of advance payments.

Examples

Compute the Periodic Payment

This example shows how to compute the periodic payment, given a number of advance payments. For example, the present value of a loan is \$1000.00 and it will be paid in full in 12 months. The annual interest rate is 10% and three payments are made at closing time.

```
Payment = payadv(0.1/12, 12, 1000, 0, 3)
```

```
Payment =  
85.9389
```

Input Arguments

Rate — Lending or borrowing rate per period

decimal

Lending or borrowing rate per period, specified as a decimal. The Rate must be greater than or equal to 0.

Data Types: double

NumPeriods — Number of periods in the life of the instrument

integer

Number of periods in the life of the instrument, specified as an integer.

Data Types: double

PresentValue — Present value of instrument

numeric

Present value of the instrument, specified as a numeric.

Data Types: double

FutureValue — Future value or target value attained after NumPeriods periods

numeric

Future value or target value to be attained after NumPeriods periods, specified as a numeric.

Data Types: double

Advance — Number of advance payments

integer

Number of advance payments, specified as an integer. If the payments are made at the beginning of the period, add 1 to Advance.

Data Types: double

Output Arguments**Payment — Periodic payment**

numeric

Periodic payment, returned as the periodic payment given a number of advance payments.

Version History

Introduced before R2006a

See Also

amortize | payodd | payper

Topics

“Analyzing and Computing Cash Flows” on page 2-11

payodd

Payment of loan or annuity with odd first period

Syntax

`Payment = payodd(Rate,NumPeriods,PresentValue,FutureValue,Days)`

Description

`Payment = payodd(Rate,NumPeriods,PresentValue,FutureValue,Days)` computes the payment for a loan or annuity with an odd first period.

Examples

Compute the Payment for a Loan or Annuity With an Odd First Period

This example shows how to return the payment for a loan or annuity with an odd first period. For example, consider a two-year loan for \$4000 that has an annual interest rate of 11% and the first payment will be made in 36 days.

```
Payment = payodd(0.11/12, 24, 4000, 0, 36)
```

```
Payment =  
186.7731
```

Input Arguments

Rate — Interest rate per period

decimal

Interest rate per period, specified as a decimal.

Data Types: double

NumPeriods — Number of periods in the life of the instrument

integer

Number of periods in the life of the instrument, specified as an integer.

Data Types: double

PresentValue — Present value of instrument

numeric

Present value of the instrument, specified as a numeric.

Data Types: double

FutureValue — Future value or target value attained after NumPeriods periods

numeric

Future value or target value to be attained after NumPeriods periods, specified as a numeric.

Data Types: double

Days — Actual number of days until first payment is made

integer

Actual number of days until the first payment is made, specified as an integer.

Data Types: double

Output Arguments

Payment — Payment

numeric

Payment, returns the payment for a loan or annuity with an odd first period.

Version History

Introduced before R2006a

See Also

amortize | payadv | payper

Topics

“Analyzing and Computing Cash Flows” on page 2-11

payper

Periodic payment of loan or annuity

Syntax

```
Payment = payper(Rate,NumPeriods,PresentValue)
```

```
Payment = payper( ___,FutureValue,Due)
```

Description

`Payment = payper(Rate,NumPeriods,PresentValue)` returns the periodic payment of a loan or annuity.

`Payment = payper( ___,FutureValue,Due)` adds optional arguments.

Examples

Compute the Periodic Payment of a Loan or Annuity

This example shows how to find the monthly payment for a three-year loan of \$9000 with an annual interest rate of 11.75%.

```
Payment = payper(0.1175/12, 36, 9000, 0, 0)
```

```
Payment =  
297.8553
```

Input Arguments

Rate — Interest rate per period

decimal

Interest rate per period, specified as a decimal.

Data Types: double

NumPeriods — Number of payment periods in life of instrument

integer

Number of payment periods in the life of instrument, specified as an integer.

Data Types: double

PresentValue — Present value of the instrument

numeric

Present value of the instrument, specified as a numeric.

Data Types: double

FutureValue — Future value or target value attained after NumPeriods periods

0 (default) | numeric

(Optional) Future value or target value to be attained after NumPeriods periods, specified as a numeric.

Data Types: double

Due — Indicator for when payments are due

0 (default) | logical with value of 1 or 0

(Optional) Indicator for when payments are due, specified as a logical with a value of 0 = end of period (default), or 1 = beginning of period.

Data Types: logical

Output Arguments**Payment — Periodic payment**

numeric

Periodic payment, returns the periodic payment of a loan or annuity.

More About**Annuity**

An annuity is a series of payments over a period of time.

The payments are usually in equal amounts and usually at regular intervals such as quarterly, semiannually, or annually.

Version History

Introduced before R2006a

See Also

fvfix | amortize | payadv | payodd | pvfix

Topics

“Analyzing and Computing Cash Flows” on page 2-11

payuni

Uniform payment equal to varying cash flow

Syntax

Series = payuni(CashFlow,Rate)

Description

Series = payuni(CashFlow,Rate) computes the uniform series value of a varying cash flow.

Examples

Calculate the Uniform Series Value

This example shows how to calculate the uniform series value using `payuni`.

The following cash flow represents the yearly income from an initial investment of \$10,000. The annual interest rate is 8%.

Year 1 - \$2000

Year 2 - \$1500

Year 3 - \$3000

Year 4 - \$3800

Year 5 - \$5000

To calculate the uniform series value:

```
Series = payuni([-10000 2000 1500 3000 3800 5000], 0.08)
```

```
Series =  
429.6296
```

Input Arguments

CashFlow — Cash flows

vector

Cash flows, specified as a vector of varying cash flows. Include the initial investment as the initial cash flow value (a negative number).

Data Types: double

Rate — Periodic interest rate

decimal

Periodic interest rate, specified as a decimal.

Data Types: `double`

Output Arguments

Series — Uniform series

`numeric`

Uniform series, returned as the value of a varying cash flow.

Version History

Introduced before R2006a

See Also

`fvfix` | `pvfix` | `fvvar` | `irr` | `pvvar`

Topics

“Analyzing and Computing Cash Flows” on page 2-11

pcalims

Linear inequalities for individual asset allocation

Syntax

```
[A,b] = pcalims(AssetMin,AssetMax)
[A,b] = pcalims( ___,NumAssets)
```

Description

As an alternative to `pcalims`, use the `Portfolio` object (`Portfolio`) for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

`[A,b] = pcalims(AssetMin,AssetMax)` specifies the lower and upper bounds of portfolio allocations in each of `NumAssets` available asset investments. `pcalims` specifies the lower and upper bounds of portfolio allocations in each of `NASSETS` available asset investments.

Note If `pcalims` is called with fewer than two output arguments, the function returns `A` concatenated with `b` `[A,b]`.

`[A,b] = pcalims( ___,NumAssets)` specifies options using an optional argument in addition to the input arguments in the previous syntax.

Examples

Compute Linear Inequalities for Individual Asset Allocation

Set the minimum weight in every asset to 0 (no short-selling), and set the maximum weight of IBM stock to 0.5 and CSCO to 0.8, while letting the maximum weight in INTC float.

Minimum weight:

- IBM — 0
- INTC — 0
- CSCO — 0

Maximum weight:

- IBM — 0.5
- INTC —
- CSCO — 0.8

`AssetMin = 0`

```
AssetMin =  
0  
  
AssetMax = [0.5 NaN 0.8]  
  
AssetMax = 1×3  
    0.5000    NaN    0.8000  
  
[A,b] = pcalims(AssetMin, AssetMax)  
  
A = 5×3  
    1     0     0  
    0     0     1  
   -1     0     0  
    0    -1     0  
    0     0    -1  
  
b = 5×1  
    0.5000  
    0.8000  
     0  
     0  
     0
```

Portfolio weights of 50% in IBM and 50% in INTC satisfy the constraints

Set the minimum weight in every asset to 0 and the maximum weight to 1.

Minimum weight:

- IBM — 0
- INTC — 0
- CSCO — 0

Maximum weight:

- IBM — 1
- INTC — 1
- CSCO — 1

```
AssetMin = 0
```

```
AssetMin =  
0
```

```
AssetMax = 1
```

```
AssetMax =  
1
```

```
NumAssets = 3
```

```
NumAssets =
3
```

```
[A,b] = pcalims(AssetMin, AssetMax, NumAssets)
```

```
A = 6×3
```

```

1      0      0
0      1      0
0      0      1
-1     0      0
0     -1      0
0      0     -1
```

```
b = 6×1
```

```

1
1
1
0
0
0
```

Portfolio weights of 50% in IBM and 50% in INTC satisfy the constraints.

Input Arguments

AssetMin — Minimum allocations in each asset

scalar numeric | vector

Minimum allocations in each asset, specified as a scalar numeric or NASSETS vector. NaN indicates no constraint.

Data Types: double

AssetMax — Maximum allocations in each asset

scalar numeric | vector

Maximum allocations in each asset, specified as a scalar numeric or NASSETS vector. NaN indicates no constraint.

Data Types: double

NumAssets — Number of assets

length of AssetMin or AssetMax (default) | scalar numeric

(Optional) Number of assets, specified as a scalar numeric.

Data Types: double

Output Arguments

A — Lower bound

matrix

Lower bound, returned as a matrix such that $A * PortWts' \leq b$, where `PortWts` is a 1-by-NASSETS vector of asset allocations.

b — Upper bound

vector

Upper bound, returned as a vector such that $A * PortWts' \leq b$, where `PortWts` is a 1-by-NASSETS vector of asset allocations.

Version History

Introduced before R2006a

See Also

`pcgcomp` | `pcglims` | `portstats` | `pcpval` | `portcons` | `portopt` | `Portfolio`

Topics

“Portfolio Construction Examples” on page 3-5

“Portfolio Selection and Risk Aversion” on page 3-7

“Active Returns and Tracking Error Efficient Frontier” on page 3-27

“Analyzing Portfolios” on page 3-2

“Portfolio Optimization Functions” on page 3-3

pcgcomp

Linear inequalities for asset group comparison constraints

Syntax

```
[A,b] = pcgcomp(GroupA,AtoBmin,AtoBmax,GroupB)
```

Description

As an alternative to `pcgcomp`, use the Portfolio object (`Portfolio`) for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using Portfolio objects, see “Portfolio Object Workflow” on page 4-18.

`[A,b] = pcgcomp(GroupA,AtoBmin,AtoBmax,GroupB)` specifies that the ratio of allocations in one group to allocations in another group is at least `AtoBmin` to 1 and at most `AtoBmax` to 1. Comparisons can be made between an arbitrary number of group pairs `NGROUPS` comprising subsets of `NASSETS` available investments.

If `pcgcomp` is called with fewer than two output arguments, the function returns `A` concatenated with `b` `[A,b]`.

Examples

Linear Inequalities for Asset Group-to-Group Comparison Constraints

Use the following assets and groupings.

Asset	INTC	XOM	RD
Region	North America	North America	Europe
Sector	Technology	Energy	Energy

Group	Min. Exposure	Max. Exposure
North America	0.30	0.75
Europe	0.10	0.55
Technology	0.20	0.50
Energy	0.20	0.80

Make the North American energy sector compose exactly 20% of the North American investment.

```
%
%      INTC  XOM  RD
GroupA = [ 0    1    0 ]; % North American Energy
GroupB = [ 1    1    0 ]; % North America
```

```
AtoBmin = 0.20;
AtoBmax = 0.20;
```

```
[A,b] = pcgcomp(GroupA, AtoBmin, AtoBmax, GroupB)
```

A = 2×3

0.2000	-0.8000	0
-0.2000	0.8000	0

b = 2×1

0
0

Portfolio weights of 40% for INTC, 10% for XOM, and 50% for RD satisfy the constraints.

Input Arguments

GroupA — Grouping A

vector

Grouping A, specified as a number of groups (NGROUPS) by number of assets (NASSETS) vector of groups to compare. Each row specifies a group. For a specific group, $\text{Group}(i, j) = 1$ if the group contains asset j ; otherwise, $\text{Group}(i, j) = 0$.

Data Types: double

AtoBmin — Minimum ratios

scalar | vector

Minimum ratios, specified as a scalar or NGROUPS-long vectors of minimum ratios of allocations in GroupA to allocations in GroupB. NaN indicates no constraint between the two groups. Scalar bounds are applied to all group pairs. The total number of assets allocated to GroupA divided by the total number of assets allocated to GroupB is $\geq \text{AtoBmin}$ and $\leq \text{AtoBmax}$.

Data Types: double

AtoBmax — Maximum ratios

scalar | vector

Maximum ratios, specified as a scalar or NGROUPS-long vectors of maximum ratios of allocations in GroupA to allocations in GroupB. NaN indicates no constraint between the two groups. Scalar bounds are applied to all group pairs. The total number of assets allocated to GroupA divided by the total number of assets allocated to GroupB is $\geq \text{AtoBmin}$ and $\leq \text{AtoBmax}$.

Data Types: double

GroupB — Grouping B

vector

Grouping B, specified as a number of groups (NGROUPS) by number of assets (NASSETS) vector of groups to compare. Each row specifies a group. For a specific group, $\text{Group}(i, j) = 1$ if the group contains asset j ; otherwise, $\text{Group}(i, j) = 0$.

Data Types: double

Output Arguments

A — Lower bound

matrix

Lower bound, returned as a matrix such that $A * \text{PortWts}' \leq b$, where `PortWts` is a 1-by-NASSETS vector of asset allocations.

b — Upper bound

vector

Upper bound, returned as a vector such that $A * \text{PortWts}' \leq b$, where `PortWts` is a 1-by-NASSETS vector of asset allocations.

Version History

Introduced before R2006a

See Also

`pcalims` | `pcglims` | `pcpval` | `portcons` | `portopt` | `Portfolio`

Topics

“Portfolio Construction Examples” on page 3-5

“Portfolio Selection and Risk Aversion” on page 3-7

“Active Returns and Tracking Error Efficient Frontier” on page 3-27

“Analyzing Portfolios” on page 3-2

“Portfolio Optimization Functions” on page 3-3

pcglims

Linear inequalities for asset group minimum and maximum allocation

Syntax

```
[A,b] = pcglims(Groups,GroupMin,GroupMax)
```

Description

As an alternative to `pcglims`, use the Portfolio object (`Portfolio`) for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using Portfolio objects, see “Portfolio Object Workflow” on page 4-18.

`[A,b] = pcglims(Groups,GroupMin,GroupMax)` specifies minimum and maximum allocations to groups of assets. Bounds can be specified for an arbitrary number of groups `NGROUPS`, made up as subsets of the `NASSETS` investments.

If `pcglims` is called with fewer than two output arguments, the function returns `A` concatenated with `b` `[A,b]`.

Examples

Linear Inequalities for Asset Group Minimum and Maximum Allocation

Use the following assets and groupings.

Asset	INTC	XOM	RD
Region	North America	North America	Europe
Sector	Technology	Energy	Energy

Group	Min. Exposure	Max. Exposure
North America	0.30	0.75
Europe	0.10	0.55
Technology	0.20	0.50
Energy	0.50	0.50

Set the minimum and maximum investment in various groups.

```
%
Groups = [ INTC  XOM  RD ; % North America
           1    1    0 ; % Europe
           0    0    1 ; % Technology
           1    0    0 ; % Energy
           0    1    1 ];

GroupMin = [0.30
            0.10
            0.20
```



```

        0.50];

GroupMax = [0.75
            0.55
            0.50
            0.50];

[A,b] = pcglims(Groups, GroupMin, GroupMax)

A = 8×3

    -1    -1     0
     0     0    -1
    -1     0     0
     0    -1    -1
     1     1     0
     0     0     1
     1     0     0
     0     1     1

b = 8×1

   -0.3000
   -0.1000
   -0.2000
   -0.5000
    0.7500
    0.5500
    0.5000
    0.5000

```

Portfolio weights of 50% in INTC, 25% in XOM, and 25% in RD satisfy the constraints.

Input Arguments

Groups — Grouping

vector

Grouping, specified as a number of groups (NGROUPS) by number of assets (NASSETS) vector of which assets belong to which group. Each row specifies a group. For a specific group, $\text{Group}(i,j) = 1$ if the group contains asset j ; otherwise, $\text{Group}(i,j) = 0$.

Data Types: double

GroupMin — Minimum allocations

scalar | vector

Minimum allocations, specified as a scalar or NGROUPS-long vectors of minimum combined allocations in each group. NaN indicates no constraint. Scalar bounds are applied to all groups.

Data Types: double

GroupMax — Maximum allocations

scalar | vector

Maximum allocations, specified as a scalar or `NGROUPS`-long vectors maximum combined allocations in each group. `NaN` indicates no constraint. Scalar bounds are applied to all groups.

Data Types: `double`

Output Arguments

A — Lower bound

matrix

Lower bound, returned as a matrix such that $A * \text{PortWts}' \leq b$, where `PortWts` is a 1-by-`NASSETS` vector of asset allocations.

b — Upper bound

vector

Upper bound, returned as a vector such that $A * \text{PortWts}' \leq b$, where `PortWts` is a 1-by-`NASSETS` vector of asset allocations.

Version History

Introduced before R2006a

See Also

`pcalims` | `pcgcomp` | `pcpval` | `portcons` | `portopt` | `Portfolio`

Topics

“Portfolio Construction Examples” on page 3-5

“Portfolio Selection and Risk Aversion” on page 3-7

“Active Returns and Tracking Error Efficient Frontier” on page 3-27

“Analyzing Portfolios” on page 3-2

“Portfolio Optimization Functions” on page 3-3

pcpval

Linear inequalities for fixing total portfolio value

Syntax

```
[A,b] = pcpval(PortValue,NumAssets)
```

Description

`[A,b] = pcpval(PortValue,NumAssets)` scales the total value of a portfolio of `NumAssets` assets to `PortValue`. All portfolio weights, bounds, return, and risk values except `ExpReturn` and `ExpCovariance` (see `portopt`) are in terms of `PortValue`.

Note As an alternative to `pcpval`, use the `Portfolio` object (`Portfolio`) for mean-variance portfolio optimization. The `Portfolio` object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

Examples

Scale the Value of a Portfolio

Scale the value of a portfolio of three assets that are equal to 1, so all return values are rates and all weight values are in fractions of the portfolio.

```
PortValue = 1;
NumAssets = 3;
```

```
[A,b] = pcpval(PortValue, NumAssets)
```

```
A = 2×3
```

```
    1    1    1
   -1   -1   -1
```

```
b = 2×1
```

```
    1
   -1
```

Portfolio weights of 40%, 10%, and 50% in the three assets satisfy the constraints.

Input Arguments

PortValue — Total value of asset portfolio

numeric

Total value of asset portfolio, specified as a scalar numeric representing the sum of the allocations in all assets. `PortValue = 1` specifies weights as fractions of the portfolio and return and risk numbers as rates instead of value.

Data Types: `double`

NumAssets — Number of available asset investments

numeric

Number of available asset investments, specified as a scalar numeric.

Data Types: `double`

Output Arguments

A — Asset allocations

matrix

Asset allocations, returned as a matrix such that $A \cdot \text{PortWts}' \leq b$, where `PortWts` is a 1-by-`NASSETS` vector of asset allocations.

b — Asset allocations

vector

Asset allocations, returned as a vector such that $A \cdot \text{PortWts}' \leq b$, where `PortWts` is a 1-by-`NASSETS` vector of asset allocations.

Note If `pcpval` is called with fewer than two output arguments, returns `A` and `b` are concatenated together:

```
Cons = [A, b];  
Cons = pcpval(PortValue, NumAssets)
```

Version History

Introduced before R2006a

See Also

`pcalims` | `pcgcomp` | `pcglims` | `portcons` | `portopt` | `Portfolio`

Topics

“Portfolio Construction Examples” on page 3-5
“Portfolio Selection and Risk Aversion” on page 3-7
“Active Returns and Tracking Error Efficient Frontier” on page 3-27
“Analyzing Portfolios” on page 3-2
“Portfolio Optimization Functions” on page 3-3

periodicreturns

Periodic total returns from total return prices

Syntax

```
TotalReturn = periodicreturns(TotalReturnPrices)
TotalReturn = periodicreturns( ___,Period)
```

Description

`TotalReturn = periodicreturns(TotalReturnPrices)` calculates the daily total returns from a daily total return price series.

`TotalReturn = periodicreturns( ___,Period)` specifies options using one or more optional arguments in addition to the input arguments in the previous syntax.

Examples

Compute TotalReturn Using datetime Input for TotalReturnPrices

Compute `TotalReturn` returned as a table using `datetime` input in a table for `TotalReturnPrices`.

```
Dates = datetime(2015,1,1:10)';
Prices = [100:10:190]';
TotalReturnPrices = table(Dates,Prices);
TotalReturn = periodicreturns(TotalReturnPrices)
```

```
TotalReturn=9×2 table
      Dates      Prices
      _____  _____
    02-Jan-2015      0.1
    03-Jan-2015    0.090909
    04-Jan-2015    0.083333
    05-Jan-2015    0.076923
    06-Jan-2015    0.071429
    07-Jan-2015    0.066667
    08-Jan-2015     0.0625
    09-Jan-2015    0.058824
    10-Jan-2015    0.055556
```

Input Arguments

TotalReturnPrices — Total return prices for a given security
matrix | table

Total return prices for a given security, specified as an `NUMOBS-by-NASSETS + 1` matrix where Column 1 contains MATLAB serial date numbers. The remaining columns contain total return price data.

If you specify `TotalReturnPrices` as a table, the first column of the table represents the dates (as either a datetime array, string array, date character vectors, or serial date numbers) while the other columns represent the returns data. If a table is used, `TotalReturn` is returned as a table.

Note Although input returns can have dates in either ascending or descending order, output total returns in `TotalReturn` have dates in ascending order, with the earliest date in the first row of `TotalReturn`, and the most recent date in the last row of `TotalReturn`.

Data Types: `double` | `datetime` | `string` | `char` | `table`

Period — Periodicity flag used to compute total returns

`'d'` (default) | character vector | numeric

(Optional) Periodicity flag used to compute total returns, specified as one of the following values:

- `'d'` = daily values (default)
- `'w'` = weekly values
- `'m'` = monthly values
- `n` = rolling return periodic values, where `n` is an integer

Data Types: `char` | `double`

Output Arguments

TotalReturn — Total return values

matrix | table

Total return values, returned as a `P-by-N` matrix or table consisting of either Dates in column 1 and daily return values in the remaining columns or period-end dates in column 1 and monthly return values in the remaining columns. The format of `TotalReturn` matches the format of the input `TotalReturnPrices`.

Version History

Introduced before R2006a

See Also

`rollingreturns`

Topics

“Portfolio Construction Examples” on page 3-5

“Portfolio Optimization Functions” on page 3-3

plotFrontier

Plot efficient frontier

Syntax

```
[prsk,pret] = plotFrontier(obj)

[prsk,pret] = plotFrontier(obj,NumPortfolios)
[prsk,pret] = plotFrontier(obj,PortfolioWeights)
[prsk,pret] = plotFrontier(obj,PortfolioRisks,PortfolioReturns)

[prsk,pret] = plotFrontier(ax,obj, __ )
[prsk,pret] = plotFrontier(obj, __ ,Name=Value)

[prsk,pret,h] = plotFrontier( __ )
```

Description

`[prsk,pret] = plotFrontier(obj)` estimates the efficient frontier with default number of 10 portfolios on the frontier, and plots the corresponding efficient frontier for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`[prsk,pret] = plotFrontier(obj,NumPortfolios)` estimates the efficient frontier with a specified number of portfolios (`NumPortfolios`) on the frontier, and plots the corresponding efficient frontier.

`[prsk,pret] = plotFrontier(obj,PortfolioWeights)` estimates efficient portfolio risks and returns with `PortfolioWeights`, and plots the efficient frontier with those portfolios. This syntax assumes that you provide valid efficient portfolio weights as input. `PortWeights` is a `NumAssets-by-NumPortfolios` matrix.

`[prsk,pret] = plotFrontier(obj,PortfolioRisks,PortfolioReturns)` plots the efficient frontier with the given risks and returns. This syntax assumes that you provide valid inputs for efficient portfolio risks and returns. `PortfolioRisks` and `PortfolioReturns` are vectors with the same size.

Note `plotFrontier` handles multiple input formats as described above. Given an asset universe with `NumAssets` assets and an efficient frontier with `NumPortfolios` portfolios, remember that portfolio weights are `NumAsset-by-NumPortfolios` matrices and that portfolio risks and returns are `NumPortfolios-by-1` column vectors.

`[prsk,pret] = plotFrontier(ax,obj, __ )` displays the efficient frontier in the target axes (`ax`). Axes can be specified as an optional first input argument in any of the previous syntaxes.

`[prsk,pret] = plotFrontier(obj, __ ,Name=Value)` specifies the axes where the efficient frontier is displayed using the name-value argument `Parent`.

`[prsk,pret,h] = plotFrontier( __ )` plots the efficient frontier and additionally returns a graphics object or an array of graphic objects (h). Use h to modify properties of the efficient frontier after creating it.

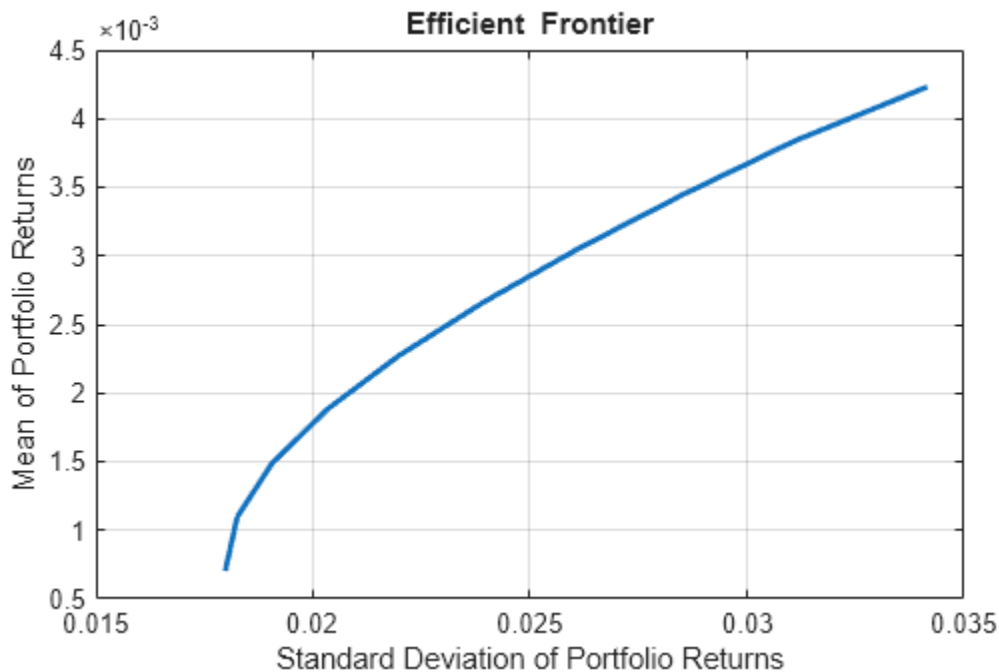
Examples

Plot the Efficient Frontier for the Portfolio Object

Given a portfolio p, plot the efficient frontier.

```
load CAPMuniverse

p = Portfolio('AssetList',Assets(1:12));
p = estimateAssetMoments(p, Data(:,1:12),'missingdata',true);
p = setDefaultConstraints(p);
plotFrontier(p);
```



Display Plot for Efficient Frontier for Portfolio and PortfolioCVar Objects Using Axes Object

Given a portfolio p, plot the efficient frontier and specify an axes object using the Parent name-value argument.

```
% Define the Portfolio object.
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
```



```

    0 0.0119 0.0336 0.1225 ];
p = Portfolio(AssetMean=m,AssetCovar=C);
p = setDefaultConstraints(p);

```

Plot 20 efficient portfolios for the Portfolio object on the left y-axis using the Parent name-value argument.

```

fig = figure;
ax = newplot(fig);
yyaxis(ax,'left')
[~,~,h1] = plotFrontier(p,20,Parent=ax);

```

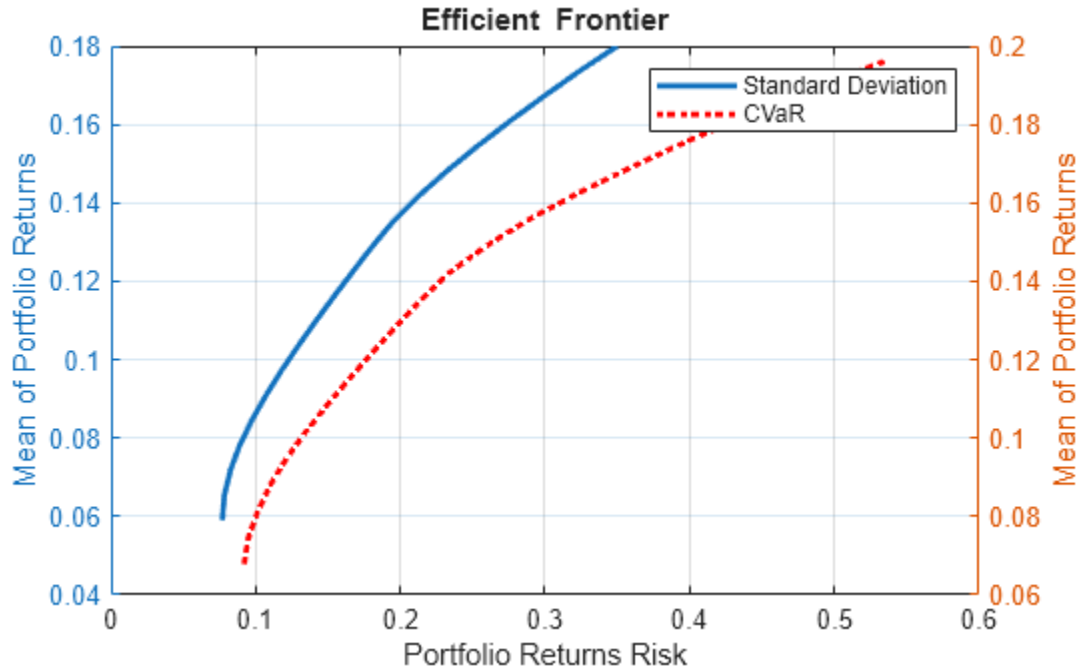
Plot 20 efficient portfolios for the associated PortfolioCVaR object on the right y-axis using the Parent name-value argument.

```

% Create PortfolioCVaR object
AssetScenarios = mvnrnd(m,C,1000);
pCVaR = PortfolioCVaR(Scenarios=AssetScenarios,ProbabilityLevel=0.95);
pCVaR = setDefaultConstraints(pCVaR);

% Plot
yyaxis(ax,'right')
[~,~,h2] = plotFrontier(pCVaR,20,Parent=ax);
h2.Color = 'r';
h2.LineStyle = ':';
xlabel(ax,'Portfolio Returns Risk')
legend('Standard Deviation','CVaR')

```



Display Plot for Efficient Frontier Using Axes Optional First Input

Given a portfolio `p`, plot the efficient frontier and specify an axes object using an optional first input argument.

```
% Define the Portfolio object.
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
p = Portfolio(AssetMean=m,AssetCovar=C);
p = p.setDefaultConstraints;

% Add an initial portfolio to a copy of the Portfolio object.
pInit = p;
pInit.InitPort = 1/4*ones(4,1);
```

Plot both Portfolio objects efficient frontiers using different axes.

```
figure
ax1 = axes('Position',[0.1 0.1 0.7 0.7]);
ax2 = axes('Position',[0.65 0.65 0.28 0.28]);
```

Plot the efficient frontier for the Portfolio object without the initial portfolio (`p`) in `ax2`.

```
[~,~,h] = plotFrontier(ax2,p)

h =
  Line (Efficient Frontier) with properties:
          Color: [0.0660 0.4430 0.7450]
        LineStyle: '-'
        LineWidth: 2
          Marker: 'none'
        MarkerSize: 6
    MarkerFaceColor: 'none'
          XData: [0.0769 0.0831 0.0994 0.1217 0.1474 0.1750 0.2068 0.2487 0.2968 0.3500]
          YData: [0.0590 0.0725 0.0859 0.0994 0.1128 0.1262 0.1397 0.1531 0.1666 0.1800]

Show all properties
```

Plot the efficient frontiers for the Portfolio object with initial portfolio (`pInit`) in `ax1`.

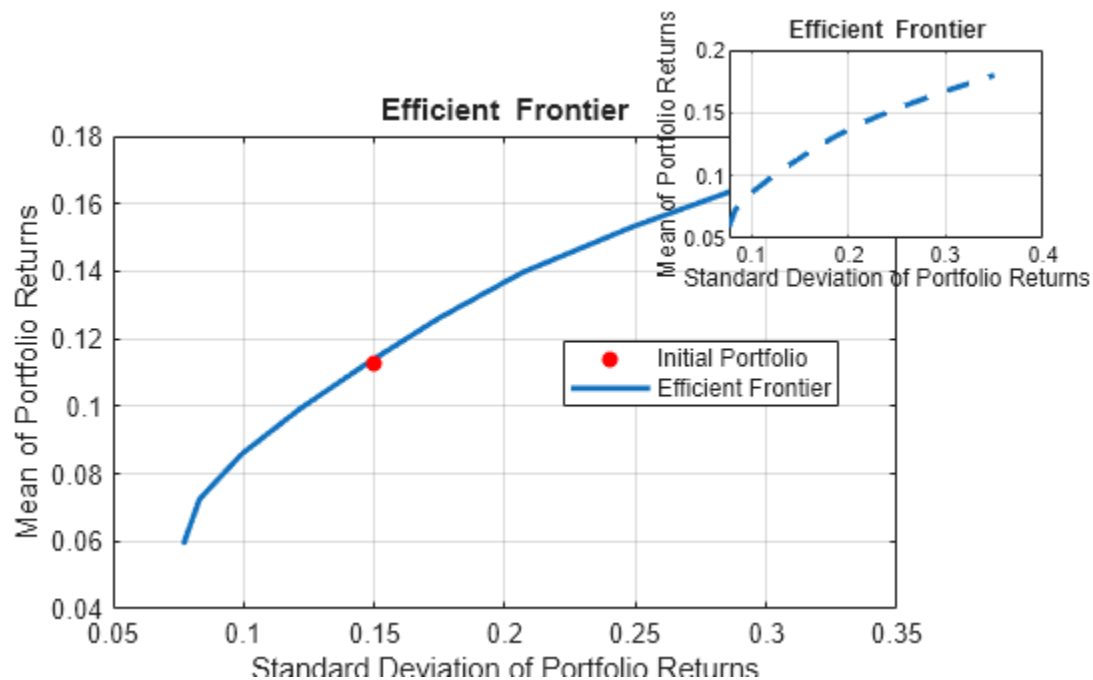
```
[~,~,hInit] = plotFrontier(ax1,pInit)

hInit =
  2x1 graphics array:

  Scatter    (Initial Portfolio)
  Line       (Efficient Frontier)
```

Modify the plot properties.

```
h.LineStyle = '--';
hInit(1).MarkerFaceColor = 'r';
```



Plot the Efficient Frontier for the Portfolio Object with BoundType and MaxNumAssets Constraints

Create a Portfolio object for 12 stocks based on CAPMuniverse.mat.

```
load CAPMuniverse
p0 = Portfolio('AssetList',Assets(1:12));
p0 = estimateAssetMoments(p0, Data(:,1:12),'missingdata',true);
p0 = setDefaultConstraints(p0);
```

Use setMinMaxNumAssets to define a maximum number of 3 assets.

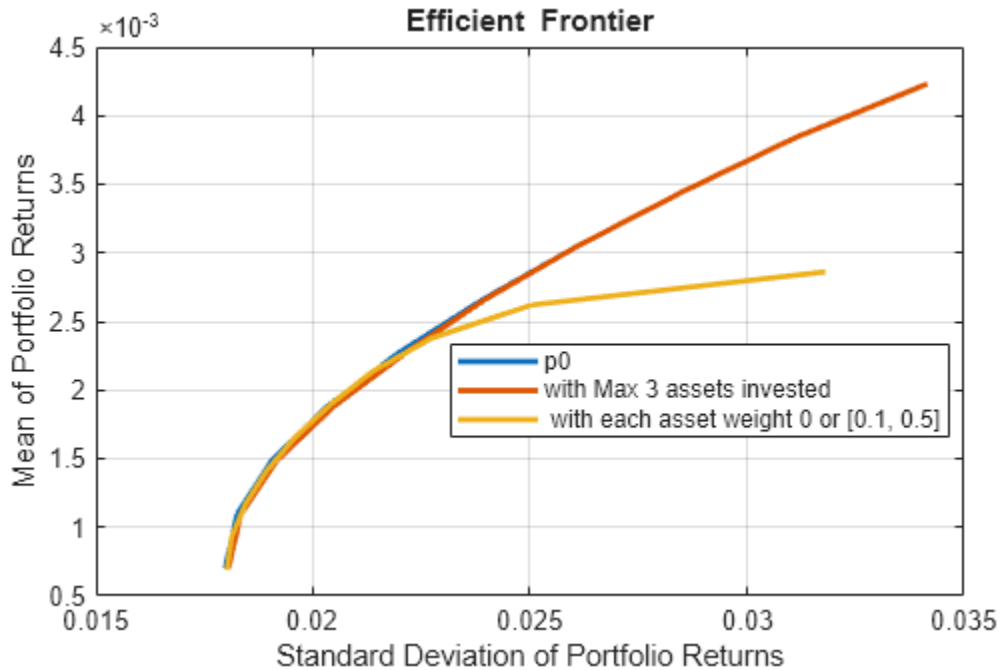
```
pWithMaxNumAssets = setMinMaxNumAssets(p0, [], 3);
```

Use setBounds to define a lower and upper bound and a BoundType of 'Conditional'.

```
pWithConditionalBound = setBounds(p0, 0.1, 0.5,'BoundType', 'Conditional');
```

Use plotFrontier to compare the different portfolio objects.

```
figure;
plotFrontier(p0); hold on;
plotFrontier(pWithMaxNumAssets); hold on;
plotFrontier(pWithConditionalBound); hold off;
legend('p0', 'with Max 3 assets invested', ' with each asset weight 0 or [0.1, 0.5]', 'location'
```



Define a target return and use `estimateFrontierByReturn` to compare the three portfolio objects.

```
targetRetn = 2.0e-3;
pwgt0 = estimateFrontierByReturn(p0, targetRetn);
pwgtWithMaxNumAssets = estimateFrontierByReturn(pWithMaxNumAssets, targetRetn);
pwgtConditionalBound = estimateFrontierByReturn(pWithConditionalBound, targetRetn);
```

The following table shows the final allocation for specified target return among the three portfolio objects. You can see that the small positions in 'AAPL' and 'HPQ' are avoided in `pwgtConditionalBound`, and only three assets are invested in `pwgtWithMaxNumAssets`.

```
result = table(p0.AssetList', pwgt0, pwgtWithMaxNumAssets, pwgtConditionalBound)
```

```
result=12x4 table
      Var1      pwgt0      pwgtWithMaxNumAssets      pwgtConditionalBound
-----
{'AAPL'}      0.076791      -5.3776e-17      0.1
{'AMZN'}           0           0           0
{'CSCO'}           0           0           0
{'DELL'}           0           0           0
{'EBAY'}           0           0           0
{'GOOG'}      0.44841      0.47297      0.44255
{'HPQ'}       0.022406           0           0
{'IBM'}       0.31139      0.34763      0.31592
{'INTC'}           0           0           0
{'MSFT'}      0.14101      0.1794      0.14153
{'ORCL'}           0           0           0
{'YHOO'}           0           0           0
```

Plot the Efficient Frontier for the PortfolioCVaR Object

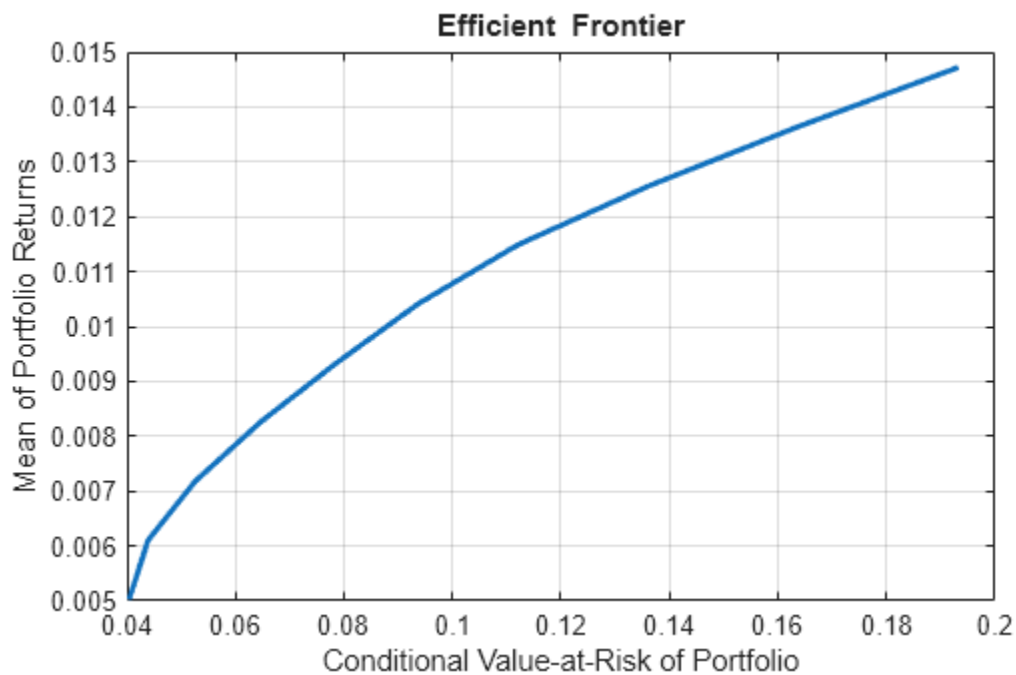
Given a PortfolioCVaR p, plot the efficient frontier.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

plotFrontier(p);
```



Plot Efficient Frontier for PortfolioMAD Object

Given a PortfolioMAD p, plot the efficient frontier.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
```

```

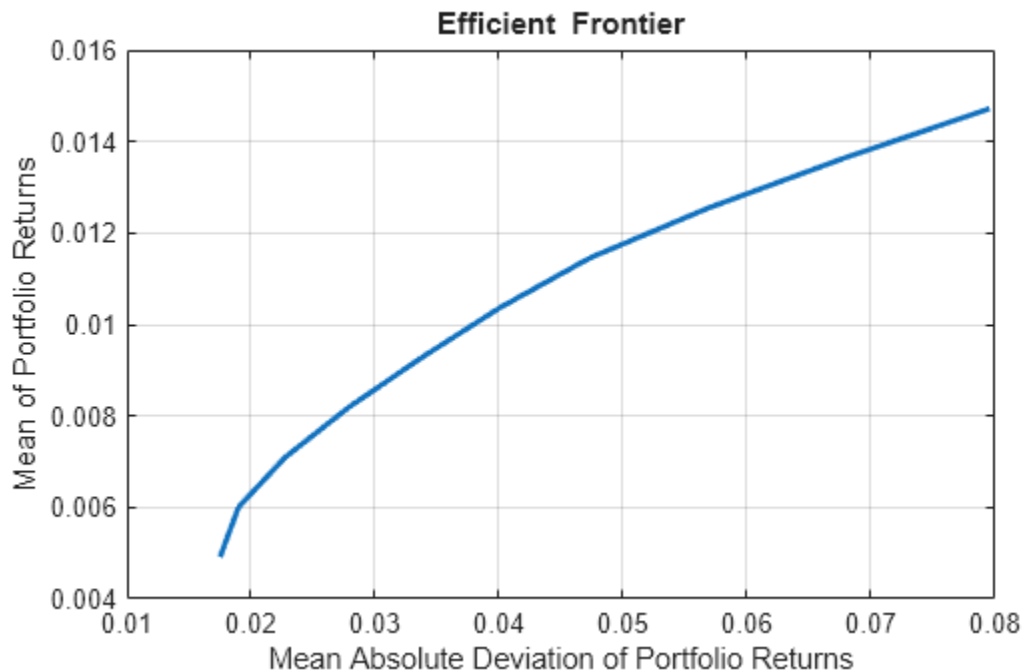
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

plotFrontier(p);

```



Input Arguments

ax — Axes object in which to display the efficient frontier plot

Axes object

(Optional) Axes object in which to display the efficient frontier plot, specified as an Axes object that you create using `axes`. The optional argument `ax` can precede any of the input argument combinations. The efficient frontier plot is displayed on the axes specified by the optional `ax` argument. If you do not specify the axes, MATLAB plots into the current axes or it creates an axes object if one does not exist. For an example of using an Axes object with `plotFrontier`, see “Display Plot for Efficient Frontier Using Axes Optional First Input” on page 15-1239.

Data Types: object

obj — Object for portfolio optimization

Portfolio object | PortfolioCVaR object | PortfolioMAD object

Object for portfolio optimization, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- Portfolio
- PortfolioCVaR
- PortfolioMAD

Data Types: object

NumPortfolios — Number of portfolios to plot on efficient frontier

10 (default) | scalar integer

(Optional) Number of portfolios to plot on efficient frontier, specified as a scalar integer.

Note If no value is specified for NumPortfolios, the default value is obtained from the hidden property defaultNumPorts (default value is 10). If NumPortfolios = 1, this function returns the portfolio specified by the hidden property defaultFrontierLimit (current default value is 'min').

Data Types: double

PortfolioRisks — Standard deviations of portfolio returns for each portfolio

column vector

(Optional) Standard deviations of portfolio returns for each portfolio, specified as a NumPortfolios-by-1 column vector.

Note PortfolioRisks and PortfolioReturns must be vectors with the same size. The risks and returns must be valid values associated to portfolios on the efficient frontier.

If portfolio risks and returns are inputs, make sure that risks come first in the calling sequence. In addition, if portfolio risks and returns are not sorted in ascending order, this method performs the sort. On output, the sorted moments are returned.

Data Types: double

PortfolioReturns — Means of portfolio returns for each portfolio

column vector

(Optional) Means of portfolio returns for each portfolio, specified as a NumPortfolios-by-1 column vector.

Note PortfolioRisks and PortfolioReturns must be vectors with the same size. The risks and returns must be valid values associated to portfolios on the efficient frontier.

If portfolio risks and returns are inputs, make sure that risks come first in the calling sequence. In addition, if portfolio risks and returns are not sorted in ascending order, this method performs the sort. On output, the sorted moments are returned.

Data Types: double

PortfolioWeights — Optimal portfolios on the efficient frontier

matrix

(Optional) Optimal portfolios on the efficient frontier, specified as a NumAssets-by-NumPortfolios matrix.

Note PortfolioWeights assumes that you provide valid portfolios on the efficient frontier.

Data Types: double

Name-Value Arguments

Specify required pairs of arguments as Name1=Value1,...,NameN=ValueN, where Name is the argument name and Value is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: [prsk,pret]=plotFrontier(p,Parent=ax)

Parent — Axes object in which to display the efficient frontier plot

Axes object

Axes object in which to display the efficient frontier plot, specified as an Axes object. The efficient frontier plot is displayed on the axes specified by the Parent name-value argument. If you do not specify the axes, MATLAB plots into the current axes or it creates an axes object if one does not exist. For an example of using an Axes object with plotFrontier, see “Display Plot for Efficient Frontier for Portfolio and PortfolioCVaR Objects Using Axes Object” on page 15-1238.

Data Types: object

Output Arguments

prsk — Estimated efficient portfolio risks

vector

Estimated efficient portfolio risks, returned as a NumPortfolios-by-1 vector for a Portfolio, PortfolioCVaR, or PortfolioMAD input object (obj).

pret — Estimated efficient portfolio returns

vector

Estimated efficient portfolio returns, returned as a NumPortfolios-by-1 vector for a Portfolio, PortfolioCVaR, or PortfolioMAD input object (obj).

h — Figure handle for efficient frontier plot

graphics vector

Figure handle for the efficient frontier plot, returned as an array of graphics (Line or Scatter) objects. Use the elements in h to access and modify properties of the efficient frontier plot.

Note

- If the portfolio object has a name in the Name property, the name is displayed as the title of the plot. Otherwise, the plot is labeled “Efficient Frontier.”
 - If the portfolio object has an initial portfolio in the InitPort property, the initial portfolio is plotted and labeled.
-

More About

Efficient Frontier

The efficient frontier is a key concept in modern portfolio theory (MPT) that represents a set of optimal portfolios offering the highest expected return for a given level of risk, or the lowest risk for a given level of expected return.

The efficient frontier illustrates the tradeoff between risk (often measured as the standard deviation of portfolio returns) and expected return. Investors aim to construct portfolios that lie on this frontier to maximize their returns for a given level of risk. Portfolios that lie on the efficient frontier are considered optimal because they provide the best possible expected return for their level of risk. Portfolios that lie below the frontier are suboptimal, as they do not provide sufficient return for the amount of risk taken. The efficient frontier emphasizes the importance of diversification. By combining different assets with varying returns and risks, investors can reduce the overall risk of the portfolio without sacrificing expected returns.

Tips

You can also use dot notation to plot the efficient frontier.

```
[prsk, pret] = obj.plotFrontier;
```

Version History

Introduced in R2011a

R2024a: Support for axes in which to display efficient frontier plot

When using `plotFrontier`, you can specify an `Axes` object in which to display the efficient frontier plot by using either an optional `ax` input argument or a `Parent` name-value argument.

See Also

`estimateFrontier` | `estimateFrontierByReturn` | `estimateFrontierByRisk` | `estimateFrontierLimits`

Topics

“Plotting the Efficient Frontier for a Portfolio Object” on page 4-125

“Plotting the Efficient Frontier for a PortfolioCVaR Object” on page 5-109

“Plotting the Efficient Frontier for a PortfolioMAD Object” on page 6-108

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

pointfig

Point and figure chart

Syntax

```
pointfig(Data)
h = pointfig(ax,Data)
```

Description

`pointfig(Data)` plots a point and figure chart from a series of prices of a security. Upward price movements are plotted as X's and downward price movements are plotted as O's.

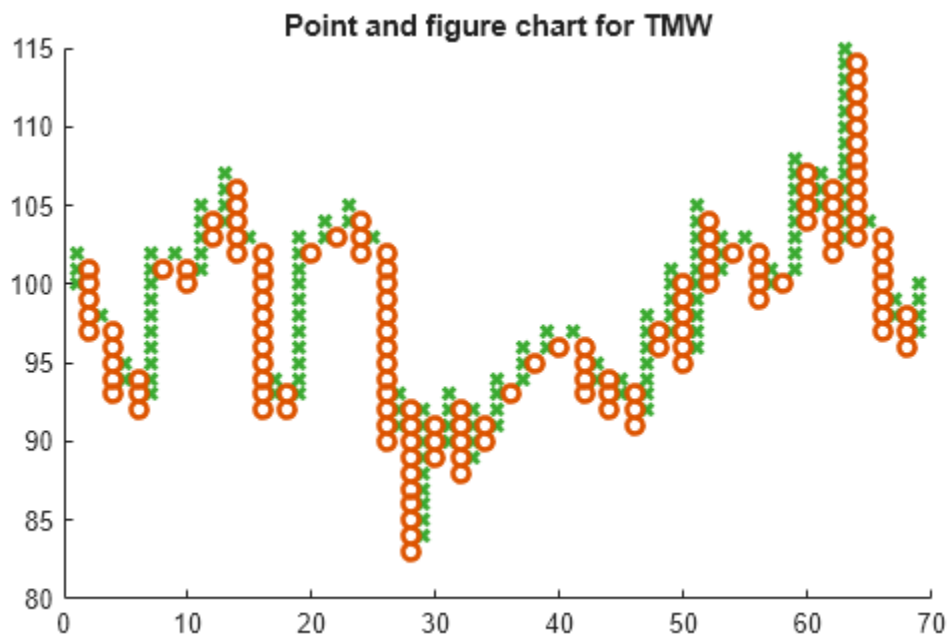
`h = pointfig(ax,Data)` adds an optional argument for `ax`.

Examples

Generate a Point and Figure Chart for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock. This Point and Figure chart is for closing prices of the stock TMW for the most recent 21 days. Note that the variable name of asset price is be renamed to 'Price' (case insensitive).

```
load SimulatedStock.mat
TMW.Properties.VariableNames{'Close'} = 'Price';
pointfig(TMW(1:200,:))
title('Point and figure chart for TMW')
```



Input Arguments

Data — Data for a series of prices

matrix | table | timetable

Data for a series of prices, specified as a matrix, table, or timetable. Timetables and tables with M rows must contain a variable named 'Price' (case insensitive).

Data Types: double | table | timetable

ax — Valid axis object

current axes (ax = gca) (default) | axes object

(Optional) Valid axis object, specified as an axes object. The point and figure plot is created in the axes specified by ax instead of in the current axes (ax = gca). The option ax can precede any of the input argument combinations.

Data Types: object

Output Arguments

h — Graphic handle of the figure

handle object

Graphic handle of the figure, returned as a handle object.

More About

Point and Figure Chart

The point and figure chart is a type of financial chart used in technical analysis to represent price movements of an asset without considering the time dimension.

Unlike traditional charts that plot price changes over time (like candlestick or line charts), point and figure charts focus solely on significant price movements, allowing traders to identify trends, reversals, and support and resistance levels more clearly. When the price rises and reaches the designated box size, an "X" is added to the current column. If the price subsequently falls by the reversal amount, a new column of "O"s is started. Conversely, if the price falls and reaches the box size, an "O" is added. If the price rises by the reversal amount, a new column of "X"s is started.

Version History

Introduced in R2006a

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

R2018a: Support for matrix, time, or timetable input for data

Behavior changed in R2018a

pointfig accepts Data input as a matrix, timetable, or table.

See Also

`timetable` | `table` | `movavg` | `linebreak` | `highlow` | `kagi` | `priceandvol` | `renko` | `volarea` | `candle`

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

portalloc

Optimal capital allocation to efficient frontier portfolios

Syntax

```
[RiskyRisk,RiskyReturn,RiskyWts,RiskyFraction,OverallRisk,OverallReturn] =
portalloc(PortRisk,PortReturn,PortWts,RisklessRate)
```

```
[RiskyRisk,RiskyReturn,RiskyWts,RiskyFraction,OverallRisk,OverallReturn] =
portalloc( ___,BorrowRate,RiskAversion)
```

```
portalloc(PortRisk,PortReturn,PortWts,RisklessRate,BorrowRate,RiskAversion)
```

Description

[RiskyRisk,RiskyReturn,RiskyWts,RiskyFraction,OverallRisk,OverallReturn] = portalloc(PortRisk,PortReturn,PortWts,RisklessRate) calculates the optimal risky portfolio and the optimal allocation of funds between that risky portfolio of NASSETS and the risk-free asset.

Note An alternative for portfolio optimization is to use the `Portfolio` object for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

[RiskyRisk,RiskyReturn,RiskyWts,RiskyFraction,OverallRisk,OverallReturn] = portalloc(___,BorrowRate,RiskAversion) specifies options using one or more optional arguments in addition to the input arguments in the previous syntax.

portalloc(PortRisk,PortReturn,PortWts,RisklessRate,BorrowRate,RiskAversion) when invoked without any output arguments, a graph of the optimal capital allocation decision is displayed.

Examples

Compute the Optimal Risky Portfolio

This example shows how to compute the optimal risky portfolio by generating the efficient frontier from the asset data and then finding the optimal risky portfolio and allocate capital. The risk-free investment return is 8%, and the borrowing rate is 12%.

```
ExpReturn = [0.1 0.2 0.15];
```

```
ExpCovariance = [0.005    -0.010    0.004
                 -0.010    0.040   -0.002
                  0.004   -0.002    0.023];
```

```
[PortRisk, PortReturn, PortWts] = portopt(ExpReturn,...
```

```
ExpCovariance);

RisklessRate = 0.08;
BorrowRate   = 0.12;
RiskAversion = 3;

[RiskyRisk, RiskyReturn, RiskyWts, RiskyFraction, ...
OverallRisk, OverallReturn] = portalloc(PortRisk, PortReturn, ...
PortWts, RisklessRate, BorrowRate, RiskAversion)

RiskyRisk =
0.1283

RiskyReturn =
0.1788

RiskyWts = 1×3

    0.0265    0.6023    0.3712

RiskyFraction =
1.1898

OverallRisk =
0.1527

OverallReturn =
0.1899
```

Input Arguments

PortRisk — Standard deviation of each risky asset efficient frontier portfolio

vector

Standard deviation of each risky asset efficient frontier portfolio, specified as an NPORTS-by-1 vector.

Data Types: double

PortReturn — Expected return of each risky asset efficient frontier portfolio

vector

Expected return of each risky asset efficient frontier portfolio, specified an NPORTS-by-1 vector.

Data Types: double

PortWts — Weights allocated to each asset

numeric

Weights allocated to each asset, specified as an NPORTS by number of assets (NASSETS) matrix of weights allocated to each asset. Each row represents an efficient frontier portfolio of risky assets. Total of all weights in a portfolio is 1.

Data Types: double

RisklessRate — Risk-free lending rate

scalar decimal

Risk-free lending rate, specified as a scalar decimal.

Data Types: double

BorrowRate — Borrowing rate

NaN (default) | scalar decimal

(Optional) Borrowing rate, specified as a scalar decimal. If borrowing is not desired, or not an option, set the BorrowRate to NaN (which is the default value).

Data Types: double

RiskAversion — Coefficient of investor's degree of risk aversion

3 (default) | scalar numeric

(Optional) Coefficient of investor's degree of risk aversion, specified as a scalar numeric. Higher numbers indicate greater risk aversion. Typical coefficients range from 2.0 through 4.0. The default value of RiskAversion is 3.

Note Consider that a less risk-averse investor would be expected to accept much greater risk and, consequently, a more risk-averse investor would accept less risk for a given level of return. Therefore, making the RiskAversion argument higher reflects the risk-return tradeoff in the data.

Data Types: double

Output Arguments

RiskyRisk — Standard deviation of the optimal risky portfolio

numeric

Standard deviation of the optimal risky portfolio, returned as a scalar.

RiskyReturn — Expected return of the optimal risky portfolio

numeric

Expected return of the optimal risky portfolio, returned as a scalar.

RiskyWts — Weights allocated to the optimal risky portfolio

vector

Weights allocated to the optimal risky portfolio, returned a 1-by-NASSETS vector. The total of all weights in the portfolio is 1.

RiskyFraction — Fraction of the complete portfolio allocated to the risky portfolio

numeric

Fraction of the complete portfolio (that is, the overall portfolio including risky and risk-free assets) allocated to the risky portfolio, returned as a scalar.

OverallRisk — Standard deviation of the optimal overall portfolio

numeric

Standard deviation of the optimal overall portfolio, returned as a scalar.

OverallReturn — Expected rate of return of the optimal overall portfolio

numeric

Expected rate of return of the optimal overall portfolio, returned as a scalar.

More About**Efficient Frontier**

The efficient frontier is a key concept in modern portfolio theory (MPT) that represents a set of optimal portfolios offering the highest expected return for a given level of risk, or the lowest risk for a given level of expected return.

The efficient frontier illustrates the tradeoff between risk (often measured as the standard deviation of portfolio returns) and expected return. Investors aim to construct portfolios that lie on this frontier to maximize their returns for a given level of risk. Portfolios that lie on the efficient frontier are considered optimal because they provide the best possible expected return for their level of risk. Portfolios that lie below the frontier are suboptimal, as they do not provide sufficient return for the amount of risk taken. The efficient frontier emphasizes the importance of diversification. By combining different assets with varying returns and risks, investors can reduce the overall risk of the portfolio without sacrificing expected returns.

Version History

Introduced before R2006a

References

[1] Bodie, Z., Kane, A., and A. Marcus. *Investments*. McGraw-Hill Education, 2013.

See Also

portrand | portstats

Topics

“Portfolio Construction Examples” on page 3-5

“Portfolio Optimization Functions” on page 3-3

portfolioRiskContribution

Compute individual asset risk contribution to overall portfolio volatility

Syntax

```
riskCont = portfolioRiskContribution(portWeights,Sigma)
riskCont = portfolioRiskContribution( ___,Name=Value)
```

Description

`riskCont = portfolioRiskContribution(portWeights,Sigma)` computes individual asset risk contribution to overall portfolio volatility and returns `riskCont` as a matrix of risk contributions.

`riskCont = portfolioRiskContribution( ___,Name=Value)` specifies an option using a name-value argument in addition to the input arguments in the previous syntax.

Examples

Compute Individual Asset Risk Contribution

Use `portfolioRiskContribution` to compute the risk contribution per asset with respect to the portfolio total risk.

Assume the returns covariance matrix is given by the following values.

```
Sigma = [0.0100    0.0075    0.0100    0.0150
          0.0075    0.0225    0.0150    0.0225
          0.0100    0.0150    0.0400    0.0450
          0.0150    0.0225    0.0450    0.0900];
```

The `portWeights` are the following values:

```
portWeights = [0.4101; 0.2734; 0.1899; 0.1266];
```

Use `portfolioRiskContribution` to compute the percent of risk contribution per asset.

```
riskCont = portfolioRiskContribution(portWeights,Sigma)
```

```
riskCont = 4×1
```

```
    0.2500
    0.2500
    0.2500
    0.2500
```

The default is to compute the relative risk contribution. However, you can compute the absolute risk contribution by using the name-value argument `RiskContributionType="absolute"`. Use `portfolioRiskContribution` to compute each asset's relative risk contribution.

```
riskCont = portfolioRiskContribution(portWeights,Sigma,RiskContributionType="relative")
```

```

riskCont = 4×1

    0.2500
    0.2500
    0.2500
    0.2500

```

Compute Individual Asset Risk Contribution When Returns Contain NaN Values

Use `portfolioRiskContribution` to compute the risk contribution of each asset with respect to the portfolio total risk when the assets returns used to compute the variance have NaN values.

Assume the returns covariance matrix is given by the following values.

```

load('CAPMuniverse.mat','AssetsTimeTable')
Sigma = cov(AssetsTimeTable{:,1:12},'partialrows')

```

```

Sigma = 12×12

```

```

    0.0012    0.0005    0.0005    0.0005    0.0005    0.0001    0.0004    0.0003    0.0006    0.0006
    0.0005    0.0023    0.0007    0.0005    0.0009    0.0001    0.0005    0.0003    0.0006    0.0006
    0.0005    0.0007    0.0012    0.0006    0.0007    0.0000    0.0006    0.0004    0.0007    0.0007
    0.0005    0.0005    0.0006    0.0009    0.0006    0.0000    0.0005    0.0003    0.0006    0.0006
    0.0005    0.0009    0.0007    0.0006    0.0017    0.0002    0.0005    0.0003    0.0005    0.0005
    0.0001    0.0001    0.0000    0.0000    0.0002    0.0006    -0.0000    0.0000    0.0000    0.0000
    0.0004    0.0005    0.0006    0.0005    0.0005    -0.0000    0.0009    0.0003    0.0005    0.0005
    0.0003    0.0003    0.0004    0.0003    0.0003    0.0000    0.0003    0.0004    0.0003    0.0003
    0.0006    0.0006    0.0007    0.0006    0.0005    0.0000    0.0005    0.0003    0.0010    0.0010
    0.0003    0.0004    0.0005    0.0004    0.0004    0.0000    0.0003    0.0002    0.0005    0.0005
    0.0005    0.0006    0.0008    0.0005    0.0007    0.0000    0.0005    0.0004    0.0006    0.0006
    0.0006    0.0011    0.0008    0.0006    0.0010    0.0002    0.0005    0.0004    0.0006    0.0006
    ⋮

```

The risk parity portfolio is subject to the following weights for all the assets.

```

portWeights = [0.1; 0.1; 0.1; 0.03; 0.1; 0.1; 0.1; 0.05; 0.1; 0.1; 0.1; 0.1];

```

Use `portfolioRiskContribution` to compute the percentage risk contribution per asset.

```

riskCont = portfolioRiskContribution(portWeights,Sigma)

```

```

riskCont = 12×1

```

```

    0.0911
    0.1261
    0.1103
    0.0251
    0.1197
    0.0203
    0.0805
    0.0258
    0.0978
    0.0660
    0.1035

```

```
0.1338
:
```

The default values returned by `portfolioRiskContribution` are the relative risk contribution of the individual assets to the overall portfolio risk. Also, you can compute the absolute risk contribution by using the name-value argument `RiskContributionType="absolute"`. Use `portfolioRiskContribution` to compute each asset's absolute risk contribution.

```
riskCont = portfolioRiskContribution(portWeights,Sigma,RiskContributionType="absolute")
```

```
riskCont = 12×1
```

```
0.0023
0.0031
0.0027
0.0006
0.0030
0.0005
0.0020
0.0006
0.0024
0.0016
0.0026
0.0033
:
```

Compute Individual Asset Risk Contribution When portWeights Is Matrix

Use `portfolioRiskContribution` to compute the risk contribution per asset with respect to the portfolio total risk when `portWeights` is a matrix.

Assume the returns covariance matrix is given by the following values.

```
Sigma = [0.0100    0.0075    0.0100    0.0150
          0.0075    0.0225    0.0150    0.0225
          0.0100    0.0150    0.0400    0.0450
          0.0150    0.0225    0.0450    0.0900];
```

The matrix of `portWeights` is the values:

```
portWeights = [0.25    0.10    0.10828;
               0.25    0.20    0.17197;
               0.25    0.30    0.28026;
               0.25    0.40    0.43949];
```

Use `portfolioRiskContribution` to compute the percent of risk contribution per asset.

```
riskCont = portfolioRiskContribution(portWeights,Sigma)
```

```
riskCont = 4×3
```

```
0.1083    0.0308    0.0322
0.1720    0.1005    0.0816
0.2803    0.2735    0.2455
```

```
0.4395    0.5952    0.6407
```

The default is to compute the relative risk contribution. However, you can compute the absolute risk contribution by using the name-value argument `RiskContributionType="absolute"`. Use `portfolioRiskContribution` to compute each asset's absolute risk contribution.

```
riskCont = portfolioRiskContribution(portWeights,Sigma,RiskContributionType="absolute")
```

```
riskCont = 4×3
```

```
0.0170    0.0060    0.0064
0.0269    0.0194    0.0162
0.0439    0.0528    0.0488
0.0688    0.1149    0.1274
```

Input Arguments

portWeights — Portfolio weights

matrix

Portfolio weights, specified using an `NumAssets`-by-`NumPortfolios` matrix. `portWeights` must be nonempty, numeric, finite, and real. `portWeights` can be negative and does not need to sum to 1. `portWeights` and `Sigma` must have the same number of `NumAssets`.

Data Types: double

Sigma — Covariance matrix of returns

positive semidefinite covariance matrix

Covariance matrix of returns, specified using an `NumAssets`-by-`NumAssets` positive semidefinite covariance matrix. `Sigma` and `portWeights` must have the same number of `NumAssets`.

Note If `Sigma` is not a positive symmetric positive semidefinite matrix, use `nearcorr` to create a positive semidefinite matrix.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `riskCont =`

```
portfolioRiskContribution(portWeights,Sigma,RiskContributionType="relative")
```

RiskContributionType — Type of risk contribution to compute

"relative" (relative risk contribution) (default) | character vector with value of 'relative' or 'absolute' | string with value of "relative" or "absolute"

Type of risk contribution to compute, specified as `RiskContributionType` and a scalar character vector or string:

- "relative" — Computes the relative contribution.
- "absolute" — Computes the absolute risk contribution.

Data Types: `char` | `string`

Output Arguments

riskCont — Risk contributions

`matrix`

Risk contributions, returned as a matrix. The default values returned by `portfolioRiskContribution` are the relative contribution of the assets to the portfolio. However, `portfolioRiskContribution` can also return the absolute risk contribution by passing the name-value argument `RiskContributionType` with a value of "absolute".

Version History

Introduced in R2022a

See Also

`riskBudgetingPortfolio`

Topics

"Risk Budgeting Portfolio" on page 4-286

"Backtest Using Risk-Based Equity Indexation" on page 4-291

"Create Hierarchical Risk Parity Portfolio" on page 4-296

riskBudgetingPortfolio

Compute risk budgeting portfolios

Syntax

```
pwgt = riskBudgetingPortfolio(Sigma)
pwgt = riskBudgetingPortfolio(Sigma,budget)
pwgt = riskBudgetingPortfolio( ___,Name=Value)
[pwgt,exitflag] = riskBudgetingPortfolio( ___,Name=Value)
```

Description

`pwgt = riskBudgetingPortfolio(Sigma)` computes risk budgeting portfolios for long-only, fully invested, risk budgeting portfolios. When a `budget` input argument is not provided, the budget is set the same for all assets.

A risk budgeting portfolio is an allocation strategy that focuses on the allocation of risk without considering the returns. The goal of this strategy is to ensure that each asset in a portfolio contributes a given target risk level to the overall portfolio volatility.

`pwgt = riskBudgetingPortfolio(Sigma,budget)` computes risk budgeting portfolios for the long-only, fully invested, risk budgeting portfolios associated with the target risk budgets (`Budget`).

`pwgt = riskBudgetingPortfolio( ___,Name=Value)` specifies options using name-value arguments in addition to the input arguments in the previous syntaxes.

`[pwgt,exitflag] = riskBudgetingPortfolio( ___,Name=Value)` add an output argument for `exitflag` and specifies options using name-value arguments in addition to the input arguments in the previous syntax.

Examples

Compute Risk Budgeting Portfolio

Use `riskBudgetingPortfolio` to compute long-only, fully invested, risk budgeting portfolios.

Assume the returns covariance matrix is given by the following values.

```
Sigma = [0.0100    0.0075    0.0100    0.0150
          0.0075    0.0225    0.0150    0.0225
          0.0100    0.0150    0.0400    0.0450
          0.0150    0.0225    0.0450    0.0900];
```

When a `budget` input argument is not provided, by default, the `riskBudgetingPortfolio` function computes a risk parity portfolio. The `riskBudgetingPortfolio` default is to consider that the risk contribution is the same for all assets and the weights are long-only and fully invested.

```
pwgt = riskBudgetingPortfolio(Sigma)

pwgt = 4×1
```

```

0.4101
0.2734
0.1899
0.1266

```

However, if a vector is passed for the optional input `budget`, the `riskBudgetingPortfolio` function computes the long-only, fully invested, risk budgeting portfolios that achieve the target risk contributions specified in `budget`.

```
pwgt = riskBudgetingPortfolio(Sigma, [0.1;0.2;0.3;0.4])
```

```
pwgt = 4×1
```

```

0.2292
0.2800
0.2633
0.2275

```

Compute Risk Budgeting Portfolio When Returns Contain NaN Values

Use `riskBudgetingPortfolio` to compute the risk budgeting portfolio when the asset returns contain NaN values.

Assume the returns covariance matrix is given by the following values.

```
load('CAPMuniverse.mat','AssetsTimeTable')
Sigma = cov(AssetsTimeTable{:,1:12},'partialrows')
```

```
Sigma = 12×12
```

```

0.0012    0.0005    0.0005    0.0005    0.0005    0.0001    0.0004    0.0003    0.0006    0.0006    0.0006    0.0006
0.0005    0.0023    0.0007    0.0005    0.0009    0.0001    0.0005    0.0003    0.0006    0.0006    0.0006    0.0006
0.0005    0.0007    0.0012    0.0006    0.0007    0.0000    0.0006    0.0004    0.0007    0.0007    0.0007    0.0007
0.0005    0.0005    0.0006    0.0009    0.0006    0.0000    0.0005    0.0003    0.0006    0.0006    0.0006    0.0006
0.0005    0.0009    0.0007    0.0006    0.0017    0.0002    0.0005    0.0003    0.0005    0.0005    0.0005    0.0005
0.0001    0.0001    0.0000    0.0000    0.0002    0.0006    -0.0000    0.0000    0.0000    0.0000    0.0000    0.0000
0.0004    0.0005    0.0006    0.0005    0.0005    -0.0000    0.0009    0.0003    0.0003    0.0005    0.0005    0.0005
0.0003    0.0003    0.0004    0.0003    0.0003    0.0000    0.0003    0.0004    0.0004    0.0003    0.0003    0.0003
0.0006    0.0006    0.0007    0.0006    0.0005    0.0000    0.0005    0.0003    0.0010    0.0010    0.0010    0.0010
0.0003    0.0004    0.0005    0.0004    0.0004    0.0000    0.0003    0.0002    0.0005    0.0005    0.0005    0.0005
0.0005    0.0006    0.0008    0.0005    0.0007    0.0000    0.0005    0.0004    0.0006    0.0006    0.0006    0.0006
0.0006    0.0011    0.0008    0.0006    0.0010    0.0002    0.0005    0.0004    0.0006    0.0006    0.0006    0.0006
⋮

```

If only the covariance matrix is provided, the `riskBudgetingPortfolio` default is to consider that the risk contribution is the same for all assets and the weights are long-only and fully invested. Basically, the default is to obtain the risk parity portfolio.

```
pwgt = riskBudgetingPortfolio(Sigma)
```

```
pwgt = 12×1
```

```
0.0718
0.0576
0.0610
0.0757
0.0574
0.1940
0.0817
0.1178
0.0676
0.0981
0.0656
0.0516
:
```

However, if a vector is passed for the optional input budget, then the risk budgeting portfolio is computed using the default ccd algorithm.

```
pwgt = riskBudgetingPortfolio(Sigma,[0.1; 0.04; 0.2; 0.05; 0.1; 0.1; 0.1; 0.05; 0.1; 0.03; 0.1; 0.03])
```

```
pwgt = 12×1
```

```
0.0871
0.0315
0.1415
0.0482
0.0717
0.2235
0.0995
0.0757
0.0829
0.0386
0.0793
0.0204
:
```

Compute Risk Budgeting Portfolio When budget Is Matrix

This example shows how to use `riskBudgetingPortfolio` to compute the risk budgeting portfolio when the budget values are a matrix.

Assume the returns covariance matrix is given by the following values.

```
Sigma = [0.0100    0.0075    0.0100    0.0150;
          0.0075    0.0225    0.0150    0.0225;
          0.0100    0.0150    0.0400    0.0450;
          0.0150    0.0225    0.0450    0.0900];
```

If only the covariance matrix is provided, the `riskBudgetingPortfolio` default is to consider that the risk contribution is the same for all assets and the weights are long-only and fully invested. Basically, the default is to obtain the risk parity portfolio.

```
pwgt = riskBudgetingPortfolio(Sigma)
```



```
pwgt = 4×1
```

```
0.4101
0.2734
0.1899
0.1266
```

If a matrix is passed for the optional input `budget`, then the risk budgeting portfolio is computed.

```
B = [0.25    0.10    0.10828;
      0.25    0.20    0.17197;
      0.25    0.30    0.28026;
      0.25    0.40    0.43949];
pwgt = riskBudgetingPortfolio(Sigma,B)

pwgt = 4×3
```

```
0.4101    0.2292    0.2500
0.2734    0.2800    0.2500
0.1899    0.2633    0.2500
0.1266    0.2275    0.2500
```

Input Arguments

Sigma — Covariance matrix of returns

positive semidefinite covariance matrix

Covariance matrix of returns, specified using an `NumAssets`-by-`NumAssets` positive semidefinite covariance matrix.

`Sigma` and `budget` must have the same number of `NumAssets`.

Note If `Sigma` is not a symmetric positive semidefinite matrix, use `nearcorr` to create a positive semidefinite matrix.

Data Types: double

budget — Target risk budget for each asset

matrix

Target risk budget for each asset, specified using an `NumAssets`-by-`NumPortfolios` matrix. `budget` must be nonnegative and all columns must sum to 1 because the values for `budget` represent the percentage of risk that each asset contributes of the total portfolio risk.

`budget` and `Sigma` must have the same number of `NumAssets`.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `pwgt = riskBudgetingPortfolio(Sigma, [0.1;0.2;0.3;0.4],Algorithm="newton",BudgetTolerance=1e6)`

Algorithm — Optimization algorithm

"ccd" (default) | character vector with value 'ccd' or 'newton' | string with value "ccd" or "newton"

Optimization algorithm, specified as `Algorithm` and a character vector or string:

- "ccd" — Cyclical coordinate descent algorithm
- "newton" — Newton algorithm with backtracking line-search

Data Types: char | string

BudgetTolerance — Deviation tolerance from target budget

1e-8 (default) | positive numeric

Deviation tolerance from target budget, specified as `BudgetTolerance` and a scalar positive numeric.

Data Types: double

MaxIterations — Maximum number of iterations allowed for the Algorithm

1e6 for "ccd" or 1e3 for "newton" (default) | positive integer

Maximum number of iterations allowed for the `Algorithm`, specified as `MaxIterations` and a scalar positive integer.

Data Types: double

Output Arguments

pwgt — Portfolio weights

matrix

Portfolio weights, returned as a matrix in which each column represents a portfolio.

exitflag — Indication of why the algorithm stopped

vector

Indication of why the algorithm stopped, returned as a 1-by-`NumPortfolios` vector with one of the following values:

- 1 — The weights of the portfolio achieve the target risk budget up to `BudgetTolerance` accuracy.
- 0 — The number of iterations exceeded `MaxIterations`.
- -1 — The solver cannot find a descent direction. This value applies only to the "newton" option for the `Algorithm`.

Version History

Introduced in R2022a

See Also

`portfolioRiskContribution`

Topics

“Risk Budgeting Portfolio” on page 4-286

“Backtest Using Risk-Based Equity Indexation” on page 4-291

“Create Hierarchical Risk Parity Portfolio” on page 4-296

portalalpha

Compute risk-adjusted alphas and returns for one or more assets

Syntax

```
portalalpha(Asset,Benchmark)
portalalpha(Asset,Benchmark,Cash)
[Alpha,RAReturn] = portalalpha(Asset,Benchmark,Cash,Choice)
```

Description

`portalalpha(Asset,Benchmark)` computes risk-adjusted alphas.

Note An alternative for portfolio optimization is to use the `Portfolio` object for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

`portalalpha(Asset,Benchmark,Cash)` computes risk-adjusted alphas using the optional argument `Cash`.

`[Alpha,RAReturn] = portalalpha(Asset,Benchmark,Cash,Choice)` computes risk-adjusted alphas and returns for one or more methods specified by `Choice`.

Examples

Calculate the Risk-Adjusted Return

This example shows how to calculate the risk-adjusted return using `portalalpha` and compare it with the fund and market's mean returns.

Use the example data with a fund, a market, and a cash series.

```
load FundMarketCash
Returns = tick2ret(TestData);
Fund = Returns(:,1);
Market = Returns(:,2);
Cash = Returns(:,3);
MeanFund = mean(Fund)

MeanFund =
0.0038

MeanMarket = mean(Market)

MeanMarket =
0.0030
```

```

[MM, aMM] = portalalpha(Fund, Market, Cash, 'MM')

MM =
0.0022

aMM =
0.0052

[GH1, aGH1] = portalalpha(Fund, Market, Cash, 'gh1')

GH1 =
0.0013

aGH1 =
0.0025

[GH2, aGH2] = portalalpha(Fund, Market, Cash, 'gh2')

GH2 =
0.0022

aGH2 =
0.0052

[SML, aSML] = portalalpha(Fund, Market, Cash, 'sml')

SML =
0.0013

aSML =
0.0025

```

Since the fund's risk is much less than the market's risk, the risk-adjusted return of the fund is much higher than both the nominal fund and market returns.

Input Arguments

Asset — Asset returns

matrix

Asset returns, specified as a `NUMSAMPLES` × `NUMSERIES` matrix with `NUMSAMPLES` observations of asset returns for `NUMSERIES` asset return series.

Data Types: double

Benchmark — Returns for a benchmark asset

vector

Returns for a benchmark asset, specified as a `NUMSAMPLES` vector of returns for a benchmark asset. The periodicity must be the same as the periodicity of `Asset`. For example, if `Asset` is monthly data, then `Benchmark` should be monthly returns.

Data Types: double

Cash — Riskless asset

0 (default) | numeric | vector

(Optional) Riskless asset, specified as either a scalar return for a riskless asset or a vector of asset returns to be a proxy for a “riskless” asset. In either case, the periodicity must be the same as the

periodicity of `Asset`. For example, if `Asset` is monthly data, then `Cash` must be monthly returns. If no value is supplied, the default value for `Cash` returns is 0.

Data Types: `double`

Choice — Computed measures

'xs' (default) | character vector | cell array of character vectors

(Optional) Computed measures, specified as a character vector or cell array of character vectors to indicate one or more measures to be computed from among various risk-adjusted alphas and return measures. The number of choices selected in `Choice` is `NUMCHOICES`. The list of choices is given in the following table:

Code	Description
'xs'	Excess Return (no risk adjustment)
'sml'	Security Market Line — The security market line shows that the relationship between risk and return is linear for individual securities (that is, increased risk = increased return).
'capm'	Jensen's Alpha — Risk-adjusted performance measure that represents the average return on a portfolio or investment, above or below that predicted by the capital asset pricing model (CAPM), given the portfolio's or investment's beta and the average market return.
'mm'	Modigliani & Modigliani — Measures the returns of an investment portfolio for the amount of risk taken relative to some benchmark portfolio.
'gh1'	Graham-Harvey 1 — Performance measure developed by John Graham and Campbell Harvey. The idea is to lever a fund's portfolio to exactly match the volatility of the S&P 500. The difference between the fund's levered return and the S&P 500 return is the performance measure.
'gh2'	Graham-Harvey 2 — In this measure, the idea is to lever up or down the fund's recommended investment strategy (using a Treasury bill), so that the strategy has the same volatility as the S&P 500.
'all'	Compute all measures.

`Choice` is specified by using the code from the table (for example, to select the Modigliani & Modigliani measure, `Choice = 'mm'`). A single choice is either a character vector or a scalar cell array with a single code from the table.

Multiple choices can be selected with a cell array of character vectors for choice codes (for example, to select both Graham-Harvey measures, `Choice = {'gh1', 'gh2'}`). To select all choices, specify `Choice = 'all'`. If no value is supplied, the default choice is to compute the excess return with `Choice = 'xs'`. `Choice` is not case-sensitive.

Data Types: `char` | `cell`

Output Arguments

Alpha — Risk-adjusted alphas

matrix

Risk-adjusted alphas, returned as an `NUMCHOICES`-by-`NUMSERIES` matrix of risk-adjusted alphas for each series in `Asset` with each row corresponding to a specified measure in `Choice`.

RAReturn — Risk-adjusted returns

matrix

Risk-adjusted returns, returned as an NUMCHOICES-by-NUMSERIES matrix of risk-adjusted returns for each series in Asset with each row corresponding to a specified measure in Choice.

Note NaN values in the data are ignored and, if NaNs are present, some results could be unpredictable. Although the alphas are comparable across measures, risk-adjusted returns depend on whether the Asset or Benchmark is levered or unlevered to match its risk with the alternative. If Choice = 'all', the order of rows in Alpha and RAReturn follows the order in the table. In addition, Choice = 'all' overrides all other choices.

More About

Risk-Adjusted Alpha

Risk-adjusted alpha measures the return of an investment or portfolio above (or below) a benchmark, adjusted for the risk taken.

Version History

Introduced in R2006b

References

- [1] Graham, J. R. and Campbell R. Harvey. "Market Timing Ability and Volatility Implied in Investment Newsletters' Asset Allocation Recommendations." *Journal of Financial Economics*. Vol. 42, 1996, pp. 397-421.
- [2] Lintner, J. "The Valuation of Risk Assets and the Selection of Risky Investments in Stocks Portfolios and Capital Budgets." *Review of Economics and Statistics*. Vol. 47, No. 1, February 1965, pp. 13-37.
- [3] Modigliani, F. and Leah Modigliani. "Risk-Adjusted Performance: How to Measure It and Why." *Journal of Portfolio Management*. Vol. 23, No. 2, Winter 1997, pp. 45-54.
- [4] Mossin, J. "Equilibrium in a Capital Asset Market." *Econometrica*. Vol. 34, No. 4, October 1966, pp. 768-783.
- [5] Sharpe, W.F., "Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk." *Journal of Finance*. Vol. 19, No. 3, September 1964, pp. 425-442.

See Also

inforatio | sharpe

Topics

"Performance Metrics Illustration" on page 7-3
 "Performance Metrics Overview" on page 7-2

portcons

Portfolio constraints

Syntax

ConSet = portcons(ConstType,consttype_values)

Description

ConSet = portcons(ConstType,consttype_values) generates a matrix of constraints, using linear inequalities, for a portfolio of asset investments. The inequalities are of the type $A*Wts' \leq b$, where Wts is the matrix of weights. The matrix ConSet is defined as $ConSet = [A \ b]$.

Note An alternative for portfolio optimization is to use the Portfolio object for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using Portfolio objects, see “Portfolio Object Workflow” on page 4-18.

Examples

Generate Matrix of Constraints for Portfolio of Asset Investments

Constrain a portfolio of three assets:

Asset	IBM	HPQ	XOM
Group	A	A	B
Minimum Weight	0	0	0
Maximum Weight	0.5	0.9	0.8

```
NumAssets = 3;
PVal = 1; % Scale portfolio value to 1.
AssetMin = 0;
AssetMax = [0.5 0.9 0.8];
GroupA = [1 1 0];
GroupB = [0 0 1];
AtoBmax = 1.5 % Value of assets in Group A at most 1.5 times value

AtoBmax =
1.5000

% in group B.

ConSet = portcons('PortValue', PVal, NumAssets, 'AssetLims',...
AssetMin, AssetMax, NumAssets, 'GroupComparison', GroupA, NaN,...
AtoBmax, GroupB)

ConSet = 9x4
```



```

1.0000    1.0000    1.0000    1.0000
-1.0000   -1.0000   -1.0000   -1.0000
1.0000     0        0        0.5000
0        1.0000     0        0.9000
0         0        1.0000    0.8000
-1.0000     0        0        0
0       -1.0000     0        0
0         0       -1.0000     0
1.0000    1.0000   -1.5000     0

```

For instance, one possible solution for portfolio weights that satisfy the constraints is 30% in IBM, 30% in HPQ, and 40% in XOM.

Input Arguments

ConstType — Constraint type

character vector with value 'Default', 'PortValue', 'AssetLims', 'GroupLims', 'GroupComparison', or 'Custom'

Constraint type, specified as a character vector defined as follows:

Constraint Type	Description	Values
'Default'	All allocations are ≥ 0 ; no short selling allowed. Combined value of portfolio allocations normalized to 1.	NumAssets (required). Scalar representing number of assets in portfolio.
'PortValue'	Fix total value of portfolio to PVal.	PVal (required). Scalar representing total value of portfolio. NumAssets (required). Scalar representing number of assets in portfolio. See pcpval.
'AssetLims'	Minimum and maximum allocation per asset.	AssetMin (required). Scalar or vector of length NASSETS, specifying minimum allocation per asset. AssetMax (required). Scalar or vector of length NASSETS, specifying maximum allocation per asset. NumAssets (optional). See pcalims.

Constraint Type	Description	Values
'GroupLims'	Minimum and maximum allocations to asset group.	Groups (required). NGROUPS-by-NASSETS matrix specifying which assets belong to each group. GroupMin (required). Scalar or a vector of length NGROUPS, specifying minimum combined allocations in each group. GroupMax (required). Scalar or a vector of length NGROUPS, specifying maximum combined allocations in each group. See <code>pcglims</code>.
'GroupComparison'	Group-to-group comparison constraints.	GroupA (required). NGROUPS-by-NASSETS matrix specifying first group in the comparison. AtoBmin (required). Scalar or vector of length NGROUPS specifying minimum ratios of allocations in GroupA to allocations in GroupB. AtoBmax (required). Scalar or vector of length NGROUPS specifying maximum ratios of allocations in GroupA to allocations in GroupB. GroupB (required). NGROUPS-by-NASSETS matrix specifying second group in the comparison. See <code>pcgcomp</code>.
'Custom'	Custom linear inequality constraints $A * PortWts' \leq b$.	A (required). NCONSTRAINTS-by-NASSETS matrix, specifying weights for each asset in each inequality equation. b (required). Vector of length NCONSTRAINTS specifying the right-hand sides of the inequalities. Note For more information using Custom, see “Specifying Group Constraints” on page 3-24.

Note You can specify multiple 'ConstType' arguments as `ConSet = portcons('ConstType1',consttype_value1,'ConstType2',consttype_value2,'ConstTypeN',consttype_valueN)`.

Data Types: char

Output Arguments

ConSet — Constraints
matrix

Constraints, returned as a matrix. ConSet is defined as $\text{ConSet} = [A \ b]$. A is a matrix and b a vector such that $A \cdot \text{Wts}' \leq b$ sets the value, where Wts is the matrix of weights.

Version History

Introduced before R2006a

See Also

[pcalims](#) | [pcgcomp](#) | [pcglims](#) | [pcpval](#) | [portopt](#) | [Portfolio](#)

Topics

“Portfolio Construction Examples” on page 3-5
“Portfolio Selection and Risk Aversion” on page 3-7
“Active Returns and Tracking Error Efficient Frontier” on page 3-27
“Analyzing Portfolios” on page 3-2
“Portfolio Optimization Functions” on page 3-3

Portfolio

Create Portfolio object for mean-variance portfolio optimization and analysis

Description

Use the `Portfolio` function to create a `Portfolio` object for mean-variance portfolio optimization.

The main workflow for portfolio optimization is to create an instance of a `Portfolio` object that completely specifies a portfolio optimization problem and to operate on the `Portfolio` object using supported functions to obtain and analyze efficient portfolios. For details on this workflow, see “Portfolio Object Workflow” on page 4-18.

You can use the `Portfolio` object in several ways. To set up a portfolio optimization problem in a `Portfolio` object, the simplest syntax is:

```
p = Portfolio;
```

This syntax creates a `Portfolio` object, `p`, such that all object properties are empty.

The `Portfolio` object also accepts collections of name-value pair arguments for properties and their values. The `Portfolio` object accepts inputs for properties with the general syntax:

```
p = Portfolio('property1',value1,'property2',value2, ... );
```

If a `Portfolio` object exists, the syntax permits the first (and only the first argument) of the `Portfolio` object to be an existing object with subsequent name-value pair arguments for properties to be added or modified. For example, given an existing `Portfolio` object in `p`, the general syntax is:

```
p = Portfolio(p,'property1',value1,'property2',value2, ... );
```

Input argument names are not case-sensitive, but must be completely specified. In addition, several properties can be specified with alternative argument names (see “Shortcuts for Property Names” on page 15-1293). The `Portfolio` object tries to detect problem dimensions from the inputs and, once set, subsequent inputs can undergo various scalar or matrix expansion operations that simplify the overall process to formulate a problem. In addition, a `Portfolio` object is a value object so that, given portfolio `p`, the following code creates two objects, `p` and `q`, that are distinct:

```
q = Portfolio(p, ...)
```

After creating a `Portfolio` object, you can use the associated object functions to set portfolio constraints, analyze the efficient frontier, and validate the portfolio model.

For more detailed information on the theoretical basis for mean-variance optimization, see “Portfolio Optimization Theory” on page 4-4.

Creation

Syntax

```
p = Portfolio
```

```
p = Portfolio(Name,Value)
```

```
p = Portfolio(p,Name,Value)
```

Description

`p = Portfolio` creates an empty `Portfolio` object for mean-variance portfolio optimization and analysis. You can then add elements to the `Portfolio` object using the supported "add" and "set" functions. For more information, see "Creating the Portfolio Object" on page 4-25.

`p = Portfolio(Name,Value)` creates a `Portfolio` object (`p`) and sets Properties on page 15-1275 using name-value pairs. For example, `p = Portfolio('AssetList',Assets(1:12))`. You can specify multiple name-value pairs.

`p = Portfolio(p,Name,Value)` creates a `Portfolio` object (`p`) using a previously created `Portfolio` object `p` and sets Properties on page 15-1275 using name-value pairs. You can specify multiple name-value pairs.

Input Arguments

p — Previously constructed Portfolio object

object

Previously constructed `Portfolio` object, specified using `Portfolio`.

Output Arguments

p — Portfolio optimization

`Portfolio` object

Portfolio optimization, returned as a `Portfolio` object.

Properties

Setting Up the Portfolio Object

AssetList — Names or symbols of assets in universe

[] (default) | cell array of character vectors | string array

Names or symbols of assets in the universe, specified as a cell array of character vectors or a string array.

Data Types: `cell` | `string`

InitPort — Initial portfolio

[] (default) | vector

Initial portfolio, specified as a vector.

Data Types: `double`

Name — Name for instance of Portfolio object

[] (default) | character vector | string

Name for instance of the `Portfolio` object, specified as a character vector or a string.

Data Types: `char` | `string`

NumAssets — Number of assets in the universe

[] (default) | positive integer scalar

Number of assets in the universe, specified as a positive integer scalar.

Data Types: double

Portfolio Object Constraints**AEquality — Linear equality constraint matrix**

[] (default) | matrix

Linear equality constraint matrix, specified as a matrix. For more information, see “Linear Equality Constraints” on page 4-10.

Data Types: double

AInequality — Linear inequality constraint matrix

[] (default) | matrix

Linear inequality constraint matrix, specified as a matrix. For more information, see “Linear Inequality Constraints” on page 4-9.

Data Types: double

bEquality — Linear equality constraint vector

[] (default) | vector

Linear equality constraint vector, specified as a vector. For more information, see “Linear Equality Constraints” on page 4-10.

Data Types: double

bInequality — Linear inequality constraint vector

[] (default) | vector

Linear inequality constraint vector, specified as a vector. For more information, see “Linear Inequality Constraints” on page 4-9.

Data Types: double

GroupA — Group A weights to be bounded by weights in group B

[] (default) | matrix

Group A weights to be bounded by weights in group B, specified as a matrix. For more information, see “Group Constraints” on page 4-12.

Data Types: double

GroupB — Group B weights

[] (default) | matrix

Group B weights, specified as a matrix. For more information, see “Group Constraints” on page 4-12.

Data Types: double

GroupMatrix — Group membership matrix

[] (default) | matrix

Group membership matrix, specified as a matrix. For more information, see “Group Ratio Constraints” on page 4-13.

Data Types: double

LowerBound — Lower-bound constraint

[] (default) | vector

Lower-bound constraint, specified as a vector. For more information, see “‘Simple’ Bound Constraints” on page 4-10 and “‘Conditional’ Bound Constraints” on page 4-11.

Data Types: double

LowerBudget — Lower-bound budget constraint

[] (default) | scalar

Lower-bound budget constraint, specified as a scalar. For more information, see “Budget Constraints” on page 4-11.

Data Types: double

ConditionalBudgetThreshold — Weights threshold above which the asset is accounted for in conditional budget

[] (default) | scalar | vector

Weights threshold above which the asset is accounted for in conditional budget, specified as a scalar or a NumAssets-by-1 vector. For more information, see “Working with Conditional Budget Constraints Using Portfolio Object” on page 4-67.

Data Types: double

LowerGroup — Lower-bound group constraint

[] (default) | vector

Lower-bound group constraint, specified as a vector. For more information, see “Group Constraints” on page 4-12.

Data Types: double

LowerRatio — Minimum ratio of allocations between Groups A and B

[] (default) | vector

Minimum ratio of allocations between GroupA and GroupB, specified as a vector. For more information, see “Group Ratio Constraints” on page 4-13.

Data Types: double

TrackingError — Upper bound for tracking error constraint

[] (default) | scalar

Upper bound for tracking error constraint, specified as a scalar. For more information, see “Tracking Error Constraints” on page 4-15.

Data Types: double

TrackingPort — Tracking portfolio for tracking error constraint

[] (default) | vector

Tracking portfolio for tracking error constraint, specified as a vector.

Data Types: double

UpperBound — Upper-bound constraint

[] (default) | vector

Upper-bound constraint, specified as a vector. For more information, see “Simple' Bound Constraints” on page 4-10 and “Conditional' Bound Constraints” on page 4-11.

Data Types: double

UpperBudget — Upper-bound budget constraint

[] (default) | scalar

Upper-bound budget constraint, specified as a scalar. For more information, see “Budget Constraints” on page 4-11.

Data Types: double

ConditionalUpperBudget — Maximum aggregate weight of the portfolio that can be invested in assets that exceed ConditionalBudgetThreshold

[] (default) | scalar

Maximum aggregate weight of the portfolio that can be invested in assets that exceed ConditionalBudgetThreshold, specified as a scalar. For more information, see “Working with Conditional Budget Constraints Using Portfolio Object” on page 4-67.

Data Types: double

UpperGroup — Upper-bound group constraint

[] (default) | vector

Upper-bound group constraint, specified as a vector. For more information, see “Group Constraints” on page 4-12.

Data Types: double

UpperRatio — Maximum ratio of allocations between Groups A and B

[] (default) | vector

Maximum ratio of allocations between GroupA and GroupB, specified as a vector. For more information, see “Group Ratio Constraints” on page 4-13.

Data Types: double

BoundType — Type of bounds for each asset weight

'Simple' (default) | character vector with value 'Simple' or 'Conditional' | string with value "Simple" or "Conditional" | cell array of character vectors with values 'Simple' or 'Conditional' | string array with values "Simple" or "Conditional"

Type of bounds for each asset weight, specified as a scalar character vector or string, or a cell array of character vectors or a string array. For more information, see setBounds.

Data Types: char | cell | string

MinNumAssets — Minimum number of assets allocated in portfolio

[] (default) | numeric

Minimum number of assets allocated in portfolio, specified as a scalar numeric value. For more information, see `setMinMaxNumAssets` and “Cardinality Constraints” on page 4-16.

Data Types: `double`

MaxNumAssets — Maximum number of assets allocated in portfolio

`[]` (default) | `numeric`

Maximum number of assets allocated in portfolio, specified as a scalar numeric value. For more information, see `setMinMaxNumAssets` and “Cardinality Constraints” on page 4-16.

Data Types: `double`

SellTurnover — Turnover constraint on sales

`[]` (default) | `scalar`

Turnover constraint on sales, specified as a scalar. For more information, see “Average Turnover Constraints” on page 4-14 and “One-Way Turnover Constraints” on page 4-14.

Data Types: `double`

BuyTurnover — Turnover constraint on purchases

`[]` (default) | `scalar`

Turnover constraint on purchases, specified as a scalar. For more information, see “Average Turnover Constraints” on page 4-14 and “One-Way Turnover Constraints” on page 4-14.

Data Types: `double`

Turnover — Turnover constraint

`[]` (default) | `scalar`

Turnover constraint, specified as a scalar. For more information, see “Average Turnover Constraints” on page 4-14 and “One-Way Turnover Constraints” on page 4-14.

Data Types: `double`

Portfolio Object Modeling

AssetCovar — Covariance of asset returns

`[]` (default) | `square matrix`

Covariance of asset returns, specified as a square matrix.

- If `AssetCovar` is not a symmetric positive semidefinite matrix, use `nearcorr` to create a positive semidefinite matrix for a correlation matrix.
- In some situations, the traditional sample approximation of the covariance matrix performs poorly. An intuitive example is when the number of variables is larger than the number of observations; this results in a noninvertible covariance matrix. Furthermore, because covariance estimation is performed using observations from random data, the estimator contains a certain amount of noise. This noise usually results in ill-conditioned covariance estimates. Working with an ill-conditioned matrix magnifies the impact of estimation errors. You can use `covarianceShrinkage` and `covarianceDenoising` for covariance estimation and noise reduction.

Data Types: `double`

AssetMean — Mean of asset returns

`[]` (default) | `vector`

Mean of asset returns, specified as a vector.

Data Types: `double`

BuyCost — Proportional purchase costs

[] (default) | vector

Proportional purchase costs, specified as a vector. For more information, see “Net Portfolio Returns” on page 4-5.

Data Types: `double`

RiskFreeRate — Risk-free rate

[] (default) | scalar

Risk-free rate, specified as a scalar.

Data Types: `double`

SellCost — Proportional sales costs

[] (default) | vector

Proportional sales costs, specified as a vector. For more information, see “Net Portfolio Returns” on page 4-5.

Data Types: `double`

Object Functions

<code>setAssetList</code>	Set up list of identifiers for assets
<code>setInitPort</code>	Set up initial or current portfolio
<code>setDefaultConstraints</code>	Set up portfolio constraints with nonnegative weights that sum to 1
<code>getAssetMoments</code>	Obtain mean and covariance of asset returns from Portfolio object
<code>setAssetMoments</code>	Set moments (mean and covariance) of asset returns for Portfolio object
<code>estimateAssetMoments</code>	Estimate mean and covariance of asset returns from data
<code>setCosts</code>	Set up proportional transaction costs for portfolio
<code>addEquality</code>	Add linear equality constraints for portfolio weights to existing constraints
<code>addGroupRatio</code>	Add group ratio constraints for portfolio weights to existing group ratio constraints
<code>addGroups</code>	Add group constraints for portfolio weights to existing group constraints
<code>addInequality</code>	Add linear inequality constraints for portfolio weights to existing constraints
<code>getBounds</code>	Obtain bounds for portfolio weights from portfolio object
<code>getBudget</code>	Obtain budget constraint bounds from portfolio object
<code>getCosts</code>	Obtain buy and sell transaction costs from portfolio object
<code>getEquality</code>	Obtain equality constraint arrays from portfolio object
<code>addGroupRatio</code>	Obtain group ratio constraint arrays from portfolio object
<code>addGroups</code>	Obtain group constraint arrays from portfolio object
<code>getInequality</code>	Obtain inequality constraint arrays from portfolio object
<code>getOneWayTurnover</code>	Obtain one-way turnover constraints from portfolio object

setGroups	Set up group constraints for portfolio weights
setInequality	Set up linear inequality constraints for portfolio weights
setBounds	Set up bounds for portfolio weights for portfolio
setBudget	Set up budget constraints for portfolio
setConditionalBudget	Set up conditional budget constraints for portfolio
setCosts	Set up proportional transaction costs for portfolio
setEquality	Set up linear equality constraints for portfolio weights
setGroupRatio	Set up group ratio constraints for portfolio weights
setOneWayTurnover	Set up one-way portfolio turnover constraints
setTurnover	Set up maximum portfolio turnover constraint
setTrackingPort	Set up benchmark portfolio for tracking error constraint
setTrackingError	Set up maximum portfolio tracking error constraint
setMinMaxNumAssets	Set cardinality constraints on the number of assets invested in a portfolio
checkFeasibility	Check feasibility of input portfolios against portfolio object
estimateBounds	Estimate global lower and upper bounds for set of portfolios
estimateFrontier	Estimate specified number of optimal portfolios on the efficient frontier
estimateFrontierByReturn	Estimate optimal portfolios with targeted portfolio returns
estimateFrontierByRisk	Estimate optimal portfolios with targeted portfolio risks
estimateFrontierLimits	Estimate optimal portfolios at endpoints of efficient frontier
plotFrontier	Plot efficient frontier
estimateMaxSharpeRatio	Estimate efficient portfolio to maximize Sharpe ratio for Portfolio object
estimatePortMoments	Estimate moments of portfolio returns for Portfolio object
estimatePortReturn	Estimate mean of portfolio returns
estimatePortRisk	Estimate portfolio risk according to risk proxy associated with corresponding object
estimateCustomObjectivePortfolio	Estimate optimal portfolio for user-defined objective function for Portfolio object
setSolver	Choose main solver and specify associated solver options for portfolio optimization
setSolverMINLP	Choose mixed integer nonlinear programming (MINLP) solver for portfolio optimization

Examples

Create an Empty Portfolio Object

You can create a Portfolio object, `p`, with no input arguments and display it using `disp`.

```
p = Portfolio;
disp(p);
```

Portfolio with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: []
AssetMean: []
AssetCovar: []
TrackingError: []
```

```
TrackingPort: []
  Turnover: []
  BuyTurnover: []
  SellTurnover: []
  Name: []
  NumAssets: []
  AssetList: []
  InitPort: []
  AInequality: []
  bInequality: []
  AEquality: []
  bEquality: []
  LowerBound: []
  UpperBound: []
  LowerBudget: []
  UpperBudget: []
  GroupMatrix: []
  LowerGroup: []
  UpperGroup: []
  GroupA: []
  GroupB: []
  LowerRatio: []
  UpperRatio: []
  MinNumAssets: []
  MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: []
```

This approach provides a way to set up a portfolio optimization problem with the `Portfolio` function. You can then use the associated set functions to set and modify collections of properties in the `Portfolio` object.

Create a Portfolio Object Using a Single-Step Setup

You can use the `Portfolio` object directly to set up a “standard” portfolio optimization problem, given a mean and covariance of asset returns in the variables `m` and `C`.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio('assetmean', m, 'assetcovar', C, ...
  'lowerbudget', 1, 'upperbudget', 1, 'lowerbound', 0)

p =
  Portfolio with properties:

      BuyCost: []
      SellCost: []
RiskFreeRate: []
   AssetMean: [4×1 double]
   AssetCovar: [4×4 double]
```

```

TrackingError: []
TrackingPort: []
    Turnover: []
    BuyTurnover: []
    SellTurnover: []
    Name: []
    NumAssets: 4
    AssetList: []
    InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: [4×1 double]
    UpperBound: []
    LowerBudget: 1
    UpperBudget: 1
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
    GroupA: []
    GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
    BoundType: []

```

Note that the `LowerBound` property value undergoes scalar expansion since `AssetMean` and `AssetCovar` provide the dimensions of the problem.

Create a Portfolio Object Using a Sequence of Steps

Using a sequence of steps is an alternative way to accomplish the same task of setting up a “standard” portfolio optimization problem, given a mean and covariance of asset returns in the variables `m` and `C` (which also illustrates that argument names are not case sensitive).

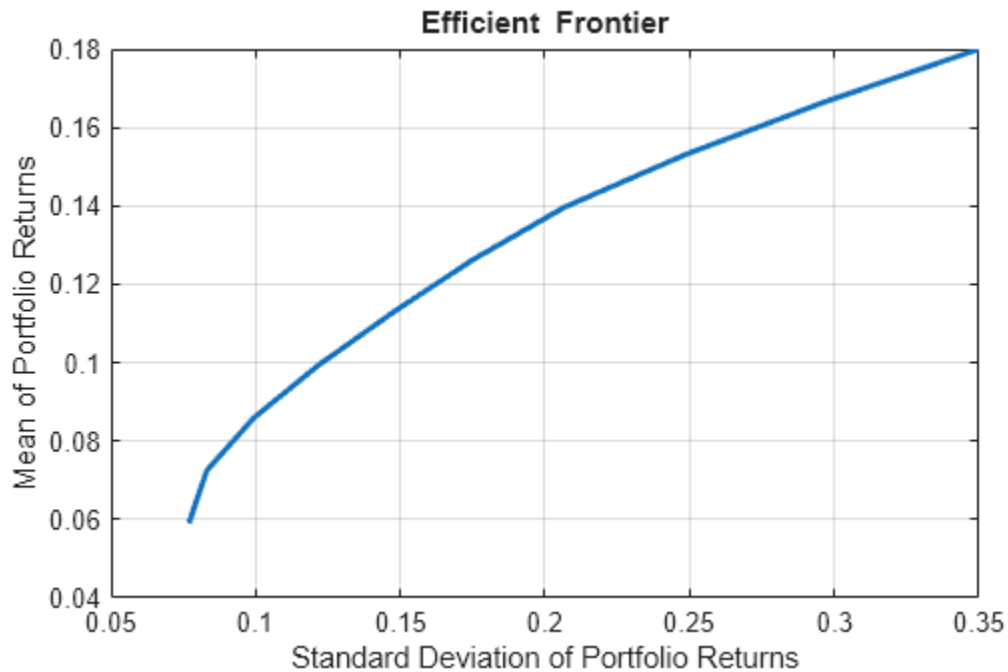
```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = Portfolio(p, 'assetmean', m, 'assetcovar', C);
p = Portfolio(p, 'lowerbudget', 1, 'upperbudget', 1);
p = Portfolio(p, 'lowerbound', 0);

plotFrontier(p);

```

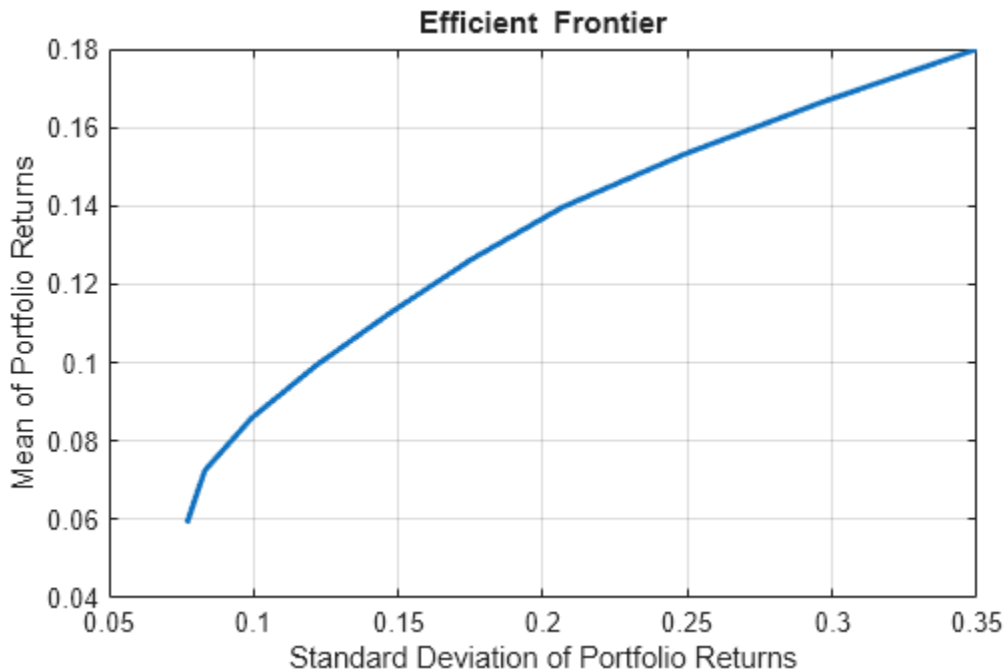


This way works because the calls to `Portfolio` are in this particular order. In this case, the call to initialize `AssetMean` and `AssetCovar` provides the dimensions for the problem. If you were to do this step last, you would have to explicitly dimension the `LowerBound` property as follows:

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio;
p = Portfolio(p, 'LowerBound', zeros(size(m)));
p = Portfolio(p, 'LowerBudget', 1, 'UpperBudget', 1);
p = Portfolio(p, 'AssetMean', m, 'AssetCovar', C);

plotFrontier(p);
```



If you did not specify the size of `LowerBound` but, instead, input a scalar argument, the `Portfolio` object assumes that you are defining a single-asset problem and produces an error at the call to set asset moments with four assets.

Create a Portfolio Object Using Shortcuts for Property Names

You can create a `Portfolio` object, `p` with `Portfolio` using shortcuts for property names.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

p = Portfolio('mean', m, 'covar', C, 'budget', 1, 'lb', 0)

p =
Portfolio with properties:

    BuyCost: []
    SellCost: []
RiskFreeRate: []
    AssetMean: [4x1 double]
    AssetCovar: [4x4 double]
TrackingError: []
TrackingPort: []
    Turnover: []
    BuyTurnover: []
    SellTurnover: []
        Name: []
```

```
NumAssets: 4
AssetList: []
InitPort: []
AInequality: []
bInequality: []
AEquality: []
bEquality: []
LowerBound: [4×1 double]
UpperBound: []
LowerBudget: 1
UpperBudget: 1
GroupMatrix: []
LowerGroup: []
UpperGroup: []
GroupA: []
GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: []
```

Direct Setting of Portfolio Object Properties

Although not recommended, you can set properties directly, however no error-checking is done on your inputs.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
```

```
p = Portfolio;
p.NumAssets = numel(m);
p.AssetMean = m;
p.AssetCovar = C;
p.LowerBudget = 1;
p.UpperBudget = 1;
p.LowerBound = zeros(size(m));
disp(p)
```

Portfolio with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: []
AssetMean: [4×1 double]
AssetCovar: [4×4 double]
TrackingError: []
TrackingPort: []
Turnover: []
```



```

    BuyTurnover: []
    SellTurnover: []
        Name: []
        NumAssets: 4
        AssetList: []
        InitPort: []
    AInequality: []
    bInequality: []
        AEquality: []
        bEquality: []
    LowerBound: [4×1 double]
    UpperBound: []
    LowerBudget: 1
    UpperBudget: 1
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
        GroupA: []
        GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
    ConditionalBudgetThreshold: []
    ConditionalUpperBudget: []
    BoundType: []

```

Create a Portfolio Object and Determine Efficient Portfolios

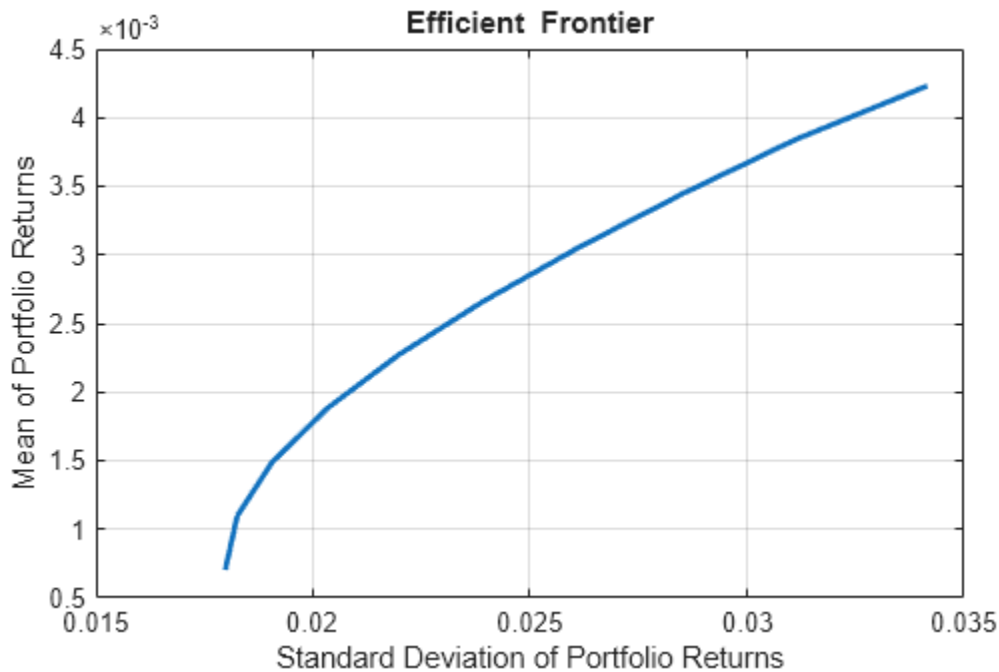
Create efficient portfolios:

```

load CAPMuniverse

p = Portfolio('AssetList',Assets(1:12));
p = estimateAssetMoments(p, Data(:,1:12),'missingdata',true);
p = setDefaultConstraints(p);
plotFrontier(p);

```



```

pwgt = estimateFrontier(p, 5);

pnames = cell(1,5);
for i = 1:5
    pnames{i} = sprintf('Port%d',i);
end

Blotter = dataset([pwgt],pnames,'obsnames',p.AssetList);

disp(Blotter);

```

	Port1	Port2	Port3	Port4	Port5
AAPL	0.017926	0.058247	0.097816	0.12955	0
AMZN	0	0	0	0	0
CSCO	0	0	0	0	0
DELL	0.0041906	0	0	0	0
EBAY	0	0	0	0	0
GOOG	0.16144	0.35678	0.55228	0.75116	1
HPQ	0.052566	0.032302	0.011186	0	0
IBM	0.46422	0.36045	0.25577	0.11928	0
INTC	0	0	0	0	0
MSFT	0.29966	0.19222	0.082949	0	0
ORCL	0	0	0	0	0
YHOO	0	0	0	0	0

Mixed-Integer Mean-Variance Portfolio Optimization Problem

This example shows how to solve a mean-variance portfolio optimization problem with constraints in the number of selected assets or conditional (semicontinuous) bounds. To solve this problem, you can use a `Portfolio` object along with different mixed integer nonlinear programming (MINLP) solvers.

Mean-Variance Portfolio

Load the returns data in `CAPMuniverse.mat`. Then, create a mean-variance `Portfolio` object with default constraints and a long-only portfolio whose weights sum to 1. For this example, you can define the feasible region of weights X as

$$X = \left\{ x \mid \sum_{i=1}^n x_i = 1, x_i \geq 0 \right\}.$$

```
% Load data
load CAPMuniverse.mat

% Create a mean-variance Portfolio object with default constraints.
p = Portfolio(AssetList=Assets(1:12));
p = estimateAssetMoments(p,Data(:,1:12));
p = setDefaultConstraints(p);
```

Include binary variables for this scenario by setting conditional (semicontinuous) bounds. Conditional bounds are those such that $x_i = 0$ or $x_i \geq \ell_i$. In this example, $\ell_i = 0.1$ for all assets.

```
% Set conditional bounds
condLB = 0.1;
condUB = 0.5;
p = setBounds(p,condLB,condUB,BoundType="conditional");
```

Use `estimateFrontier` to estimate a set of portfolios on the efficient frontier. The efficient frontier is a curve that shows the trade-off between the return and risk achieved by Pareto-optimal portfolios. For a given return level, the portfolio on the efficient frontier is the one that minimizes the risk while maintaining the desired return. Conversely, for a given risk level, the portfolio on the efficient frontier is the one that maximizes return while maintaining the desired risk level.

```
% Compute the efficient frontier.
pwgt = estimateFrontier(p)
```

```
pwgt = 12×10
```

0	0	0.1000	0.1253	0.1745	0.2236	0.2715	0.3327	0.4111	0.5
0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	
0.1350	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	
0.1000	0.1450	0.1406	0.1910	0.2344	0.2778	0.3200	0.3726	0.4415	0.5
0.1000	0.1609	0.1642	0.2121	0.2415	0.2709	0.3085	0.2947	0.1474	
0.2354	0.1875	0.1290	0	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	
0.4296	0.4066	0.3662	0.3717	0.2496	0.1277	0	0	0	
0	0.1000	0.1000	0.1000	0.1000	0.1000	0.1000	0	0	
0	0	0	0	0	0	0	0	0	
:									

```
% Compute risk and returns of the portfolios on the efficient frontier.
[rsk,ret] = estimatePortMoments(p,pwgt)
```

```
rsk = 10×1
```

```
0.0076
```

```

0.0080
0.0085
0.0094
0.0105
0.0117
0.0132
0.0147
0.0168
0.0193

```

```
ret = 10×1
```

```

0.0008
0.0012
0.0017
0.0021
0.0026
0.0030
0.0034
0.0039
0.0043
0.0048

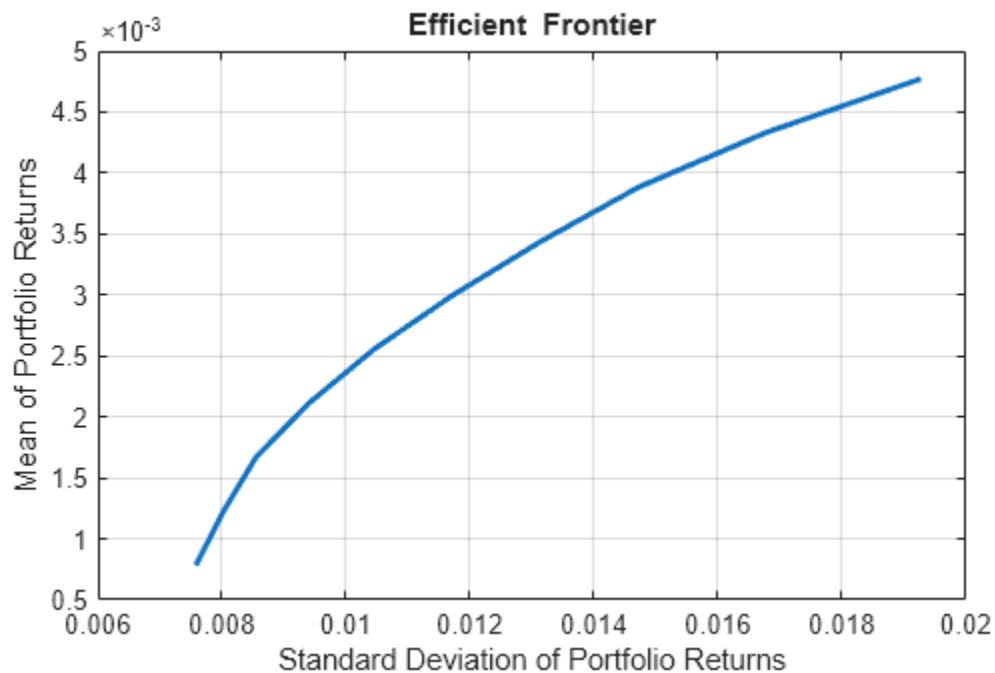
```

Plot the weights from the frontier estimation using `plotFrontier`. The resulting curve is piece-wise concave with vertical jumps (discontinuities) between the concave intervals.

```

% Plot the efficient frontier.
plotFrontier(p,pwgt)

```



Changing MINLP Solvers

In the previous section, you use the default solver for `estimateFrontier`. However, you can solve mixed-integer portfolio problems using any of the three algorithms supported by `setSolverMINLP`: `OuterApproximation`, `ExtendedCP`, and `TrustRegionCP`. Furthermore, the `OuterApproximation` algorithm accepts an additional name-value argument (`ExtendedFormulation`) for Portfolio problems, which reformulates problems with quadratic functions to work in an extended space that usually decreases the computation time. All algorithms, including the extended formulation variation of the `OuterApproximation` algorithm, return the same values within numerical accuracy. The available solvers are:

- `OuterApproximation` — The default algorithm, which is robust and usually faster than `ExtendedCP`
- `OuterApproximation` with `ExtendedFormulation` set to `true` — A robust algorithm that is usually faster than other algorithms, but only supported for Portfolio object problems
- `ExtendedCP` — The most robust solver, but usually the slowest
- `TrustRegionCP` — The fastest algorithm, but one that is less robust and may provide suboptimal solutions

For more information on solvers for mixed-integer portfolio problems, see “Choose MINLP Solvers for Portfolio Problems” on page 4-140.

To change the MINLP solvers, use `setSolverMINLP`.

```
% Select the extended formulation version of 'OuterApproximation' as the
% solver.
p_EOA = setSolverMINLP(p, 'OuterApproximation', ...
    ExtendedFormulation=true);
pwgt_EOA = estimateFrontier(p_EOA);
[rskEOA, retEOA] = estimatePortMoments(p_EOA, pwgt_EOA);

% Select 'TrustRegionCP' as the solver.
p_TR = setSolverMINLP(p, 'TrustRegionCP');
pwgt_TR = estimateFrontier(p_TR);
[rskTR, retTR] = estimatePortMoments(p_TR, pwgt_TR);

% Select 'ExtendedCP' as the solver using 'midway' cuts as 'CutGeneration'.
p_ECP = setSolverMINLP(p, 'ExtendedCP', 'CutGeneration', 'midway');
pwgt_ECP = estimateFrontier(p_ECP);
[rskECP, retECP] = estimatePortMoments(p_ECP, pwgt_ECP);
```

Compare the returns and risks obtained by the portfolios on the efficient frontier from the different solvers. These are same within a numerical accuracy, where the absolute difference is $\leq 10^{-4}$.

```
retTable = table(ret, retEOA, retTR, retECP, ...
    'VariableNames', {'OA', 'EOA', 'TR', 'ECP'})
```

retTable=10×4 table

OA	EOA	TR	ECP
0.00078336	0.00078336	0.00078334	0.00078348
0.0012267	0.0012267	0.0012267	0.0012268
0.00167	0.00167	0.00167	0.0016701
0.0021133	0.0021133	0.0021133	0.0021134

```

0.0025566    0.0025566    0.0025566    0.0025567
0.0029999    0.0029999    0.0029999    0.003
0.0034432    0.0034432    0.0034432    0.0034433
0.0038865    0.0038865    0.0038865    0.0038866
0.0043298    0.0043298    0.0043298    0.0043299
0.0047731    0.0047731    0.0047731    0.0047731

rskTable = table(rsk,rskEOA,rskTR,rskECP,...
    'VariableNames',{'OA','EOA','TR','ECP'})
rskTable=10x4 table
      OA      EOA      TR      ECP
      ---      ---      ---      ---
0.0075778    0.0075778    0.0075778    0.0075778
0.0080234    0.0080234    0.0080235    0.0080236
0.0085488    0.0085488    0.0085488    0.008549
0.0094024    0.0094024    0.009405    0.0094025
0.010456    0.010456    0.010456    0.010456
0.011727    0.011727    0.011727    0.011728
0.013155    0.013155    0.013155    0.013155
0.014729    0.014729    0.014729    0.014729
0.016764    0.016764    0.016764    0.016764
0.019273    0.019273    0.019273    0.019273

% Compare the risks from the different OuterApproximation formulations.
norm(rskTable.OA-rskTable.EOA,Inf) <= 1e-4

ans = logical
     1

```

More About

Mean-Variance Portfolio Optimization

A mean-variance optimization problem is completely specified with three elements.

The three elements for a mean-variance optimization problem are:

- A universe of assets with estimates for the prospective mean and covariance of asset total returns for a period of interest.
- A portfolio set that specifies the set of portfolio choices in terms of a collection of constraints.
- A model for portfolio return and risk, which, for mean-variance optimization, is either the gross or net mean of portfolio returns and the standard deviation of portfolio returns.

After you specify these three elements in an unambiguous way, you can solve and analyze portfolio optimization problems. The simplest mean-variance portfolio optimization problem has:

- A mean and covariance of asset total returns
- Nonnegative weights for all portfolios that sum to 1 (the summation constraint is known as a budget constraint)
- Built-in models for portfolio return and risk that use the mean and covariance of asset total returns

For more information on the theory and definition of mean-variance optimization supported by portfolio optimization tools in Financial Toolbox, see “Portfolio Optimization Theory” on page 4-4.

Portfolio Problem Sufficiency

A mean-variance portfolio optimization is completely specified with the `Portfolio` object if two conditions are met.

The following are the two conditions that must be met:

- The moments of asset returns must be specified such that the property `AssetMean` contains a valid finite mean vector of asset returns and the property `AssetCovar` contains a valid symmetric positive-semidefinite matrix for the covariance of asset returns.

The first condition is satisfied by setting the properties associated with the moments of asset returns.

- The set of feasible portfolios must be a nonempty compact set, where a compact set is closed and bounded.

The second condition is satisfied by an extensive collection of properties that define different types of constraints to form a set of feasible portfolios. Since such sets must be bounded, either explicit or implicit constraints can be imposed, and several functions, such as `estimateBounds`, provide ways to ensure that your problem is properly formulated.

Given mean and covariance of asset returns in the variables `AssetMean` and `AssetCovar`, this problem is completely specified by:

```
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar, ...
'LowerBound', 0, 'UpperBudget', 1, 'LowerBudget', 1)
```

or equivalently by:

```
p = Portfolio;
p = setAssetMoments(p, AssetMean, AssetCovar);
p = setDefaultConstraints(p);
```

Although the general sufficiency conditions for mean-variance portfolio optimization go beyond these two conditions, the `Portfolio` object implemented in Financial Toolbox implicitly handles all these additional conditions. For more information on the Markowitz model for mean-variance portfolio optimization, see “Portfolio Optimization” on page A-5.

Shortcuts for Property Names

The `Portfolio` object has shorter argument names that replace longer argument names associated with specific properties of the `Portfolio` object.

For example, rather than enter `'assetcovar'`, the `Portfolio` object accepts the case-insensitive name `'covar'` to set the `AssetCovar` property in a `Portfolio` object. Every shorter argument name corresponds with a single property in the `Portfolio` object. The one exception is the alternative argument name `'budget'`, which signifies both the `LowerBudget` and `UpperBudget` properties. When `'budget'` is used, then the `LowerBudget` and `UpperBudget` properties are set to the same value to form an equality budget constraint.

Shortcuts for Property Names

Shortcut Argument Name	Equivalent Argument / Property Name
ae	AEquality
ai	AInequality
covar	AssetCovar
assetnames or assets	AssetList
mean	AssetMean
be	bEquality
bi	bInequality
group	GroupMatrix
lb	LowerBound
n or num	NumAssets
rfr	RiskFreeRate
ub	UpperBound
budget	UpperBudget and LowerBudget

Version History**Introduced in R2011a****R2024b: Added Properties for ConditionalBudgetThreshold and ConditionalUpperBudget***Behavior changed in R2024b*

To set conditional budget constraints using the Portfolio object, use `ConditionalBudgetThreshold` and `ConditionalUpperBudget` properties. In addition, you can use the `setConditionalBudget` function. For more information, see “Working with Conditional Budget Constraints Using Portfolio Object” on page 4-67.

References

[1] For a complete list of references for the Portfolio object, see “Portfolio Optimization” on page A-5.

See Also

`plotFrontier` | `estimateFrontier` | `PortfolioCVaR` | `PortfolioMAD` | `nearcorr` | `covarianceShrinkage` | `covarianceDenoising`

Topics

“Creating the Portfolio Object” on page 4-25

“Working with Portfolio Constraints Using Defaults” on page 4-58

“Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object” on page 4-98

“Estimate Efficient Frontiers for Portfolio Object” on page 4-122

“Asset Allocation Case Study” on page 4-180

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

"Portfolio Optimization with Semicontinuous and Cardinality Constraints" on page 4-190
 "Black-Litterman Portfolio Optimization Using Financial Toolbox" on page 4-222
 "Portfolio Optimization Using Factor Models" on page 4-231
 "Bond Portfolio Optimization Using Portfolio Object" on page 10-30
 "Use Extended Formulation of OuterApproximation Solver Type" on page 15-1517
 "Single Period Goal-Based Wealth Management" on page 4-362
 "Dynamic Portfolio Allocation in Goal-Based Wealth Management for Multiple Time Periods" on page 4-367
 "Multiperiod Goal-Based Wealth Management Using Reinforcement Learning" on page 4-379
 "Adding Constraints to Satisfy UCITS Directive" on page 4-457
 "Portfolio Optimization Theory" on page 4-4
 "Portfolio Object Workflow" on page 4-18
 "Portfolio Object Properties and Functions" on page 4-20
 "Working with Portfolio Objects" on page 4-20
 "Setting and Getting Properties" on page 4-20
 "Displaying Portfolio Objects" on page 4-21
 "Saving and Loading Portfolio Objects" on page 4-21
 "Estimating Efficient Portfolios and Frontiers" on page 4-21
 "Arrays of Portfolio Objects" on page 4-22
 "Subclassing Portfolio Objects" on page 4-23
 "Conventions for Representation of Data" on page 4-23
 Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9
 "Role of Convexity in Portfolio Problems" on page 4-157
 "When to Use Portfolio Objects Over Optimization Toolbox" on page 4-132
 "Solver Guidelines for Portfolio Objects" on page 4-115
 "Choosing and Controlling the Solver for Mean-Variance Portfolio Optimization" on page 4-114
 "Choose MINLP Solvers for Portfolio Problems" on page 4-140

External Websites

Getting Started with Portfolio Optimization (4 min 12 sec)
 Portfolio Optimization Across Risk, Returns, and Climate (58 min 22 sec)
 MATLAB for Advanced Portfolio Construction and Stock Selection Models (30 min 28 sec)
 Advanced Software Development Techniques for Finance (46 min 04 sec)
 Managing and Fine-Tuning Portfolio Optimization Workflows with Experiment Manager (20 min 21 sec)

PortfolioCVaR

Creates PortfolioCVaR object for conditional value-at-risk portfolio optimization and analysis

Description

Use `PortfolioCVaR` to create a `PortfolioCVaR` object for conditional value-at-risk portfolio optimization.

The main workflow for CVaR portfolio optimization is to create an instance of a `PortfolioCVaR` object that completely specifies a portfolio optimization problem and to operate on the `PortfolioCVaR` object using supported functions to obtain and analyze efficient portfolios. For details on this workflow, see “PortfolioCVaR Object Workflow” on page 5-16.

You can use the `PortfolioCVaR` object in several ways. To set up a portfolio optimization problem in a `PortfolioCVaR` object, the simplest syntax is:

```
p = PortfolioCVaR;
```

This syntax creates a `PortfolioCVaR` object, `p`, such that all object properties are empty.

The `PortfolioCVaR` object also accepts collections of name-value pair arguments for properties and their values. The `PortfolioCVaR` function accepts inputs for properties with the general syntax:

```
p = PortfolioCVaR('property1',value1,'property2',value2, ... );
```

If a `PortfolioCVaR` object already exists, the syntax permits the first (and only the first argument) of the `PortfolioCVaR` object to be an existing object with subsequent name-value pair arguments for properties to be added or modified. For example, given an existing `PortfolioCVaR` object in `p`, the general syntax is:

```
p = PortfolioCVaR(p,'property1',value1,'property2',value2, ... );
```

Input argument names are not case sensitive, but must be completely specified. In addition, several properties can be specified with alternative argument names (see “Shortcuts for Property Names” on page 15-1316). The `PortfolioCVaR` object tries to detect problem dimensions from the inputs and, once set, subsequent inputs can undergo various scalar or matrix expansion operations that simplify the overall process to formulate a problem. In addition, a `PortfolioCVaR` object is a value object so that, given portfolio `p`, the following code creates two objects, `p` and `q`, that are distinct:

```
q = PortfolioCVaR(p, ...)
```

After creating a `PortfolioCVaR` object, you can use the associated object functions to set portfolio constraints, analyze the efficient frontier, and validate the portfolio model.

For more detailed information on the theoretical basis for conditional value-at-risk portfolio optimization, see “Portfolio Optimization Theory” on page 5-3.

Creation

Syntax

```
p = PortfolioCVaR
p = PortfolioCVaR(Name,Value)

p = PortfolioCVaR(p,Name,Value)
```

Description

`p = PortfolioCVaR` creates an empty `PortfolioCVaR` object for conditional value-at-risk portfolio optimization and analysis. You can then add elements to the `PortfolioCVaR` object using the supported "add" and "set" functions. For more information, see "Creating the PortfolioCVaR Object" on page 5-22.

`p = PortfolioCVaR(Name,Value)` creates a `PortfolioCVaR` object (`p`) and sets Properties on page 15-1297 using name-value pairs. For example, `p = PortfolioCVaR('AssetList',Assets(1:12))`. You can specify multiple name-value pairs.

`p = PortfolioCVaR(p,Name,Value)` creates a `PortfolioCVaR` object (`p`) using a previously created `PortfolioCVaR` object `p` and sets Properties on page 15-1297 using name-value pairs. You can specify multiple name-value pairs.

Input Arguments

p — Previously constructed PortfolioCVaR object

object

Previously constructed `PortfolioCVaR` object, specified using `PortfolioCVaR`.

Output Arguments

p — PortfolioCVaR optimization

`PortfolioCVaR` object

`PortfolioCVaR` optimization, returned as a `PortfolioCVaR` object.

Properties

Setting Up the PortfolioCVaR Object

AssetList — Names or symbols of assets in universe

[] (default) | cell array of character vectors | string array

Names or symbols of assets in the universe, specified as a cell array of character vectors or a string array.

Data Types: `cell` | `string`

InitPort — Initial portfolio

[] (default) | vector

Initial portfolio, specified as a vector.

Data Types: double

Name — Name for instance of PortfolioCvAR object

[] (default) | character vector | string

Name for instance of the PortfolioCvAR object, specified as a character vector or string.

Data Types: char | string

NumAssets — Number of assets in the universe

[] (default) | positive integer scalar

Number of assets in the universe, specified as a positive integer scalar.

Data Types: double

PortfolioCvAR Object Constraints

AEquality — Linear equality constraint matrix

[] (default) | matrix

Linear equality constraint matrix, specified as a matrix. For more information, see “Linear Equality Constraints” on page 5-9.

Data Types: double

AInequality — Linear inequality constraint matrix

[] (default) | matrix

Linear inequality constraint matrix, specified as a matrix. For more information, see “Linear Inequality Constraints” on page 5-8.

Data Types: double

bEquality — Linear equality constraint vector

[] (default) | vector

Linear equality constraint vector, specified as a vector. For more information, see “Linear Equality Constraints” on page 5-9.

Data Types: double

bInequality — Linear inequality constraint

[] (default) | vector

Linear inequality constraint vector, specified as a vector. For more information, see “Linear Inequality Constraints” on page 5-8.

Data Types: double

GroupA — Group A weights to be bounded by weights in group B

[] (default) | matrix

Group A weights to be bounded by weights in group B, specified as a matrix. For more information, see “Group Constraints” on page 5-11.

Data Types: double

GroupB — Group B weights

[] (default) | matrix

Group B weights, specified as a matrix. For more information, see “Group Constraints” on page 5-11.

Data Types: double

GroupMatrix — Group membership matrix

[] (default) | matrix

Group membership matrix, specified as a matrix. For more information, see “Group Ratio Constraints” on page 5-12.

Data Types: double

LowerBound — Lower-bound constraint

[] (default) | vector

Lower-bound constraint, specified as a vector. For more information, see “‘Simple’ Bound Constraints” on page 5-9 and “‘Conditional’ Bound Constraints” on page 5-10.

Data Types: double

LowerBudget — Lower-bound budget constraint

[] (default) | scalar

Lower-bound budget constraint, specified as a scalar. For more information, see “Budget Constraints” on page 5-10.

Data Types: double

ConditionalBudgetThreshold — Weights threshold above which the asset is accounted for in conditional budget

[] (default) | scalar | vector

Weights threshold above which the asset is accounted for in conditional budget, specified as a scalar or a NumAssets-by-1 vector. For more information, see “Conditional Budget Constraints” on page 5-11.

Data Types: double

LowerGroup — Lower-bound group constraint

[] (default) | vector

Lower-bound group constraint, specified as a vector. For more information, see “Group Constraints” on page 5-11.

Data Types: double

LowerRatio — Minimum ratio of allocations between Groups A and B

[] (default) | vector

Minimum ratio of allocations between GroupA and GroupB, specified as a vector. For more information, see “Group Ratio Constraints” on page 5-12.

Data Types: double

UpperBound — Upper-bound constraint

[] (default) | vector

Upper-bound constraint, specified as a vector. For more information, see “‘Simple’ Bound Constraints” on page 5-9 and “‘Conditional’ Bound Constraints” on page 5-10.

Data Types: double

UpperBudget — Upper-bound budget constraint

[] (default) | scalar

Upper-bound budget constraint, specified as a scalar. For more information, see “Budget Constraints” on page 5-10.

Data Types: double

ConditionalUpperBudget — Maximum aggregate weight of the portfolio that can be invested in assets that exceed ConditionalBudgetThreshold

[] (default) | scalar

Maximum aggregate weight of the portfolio that can be invested in assets that exceed ConditionalBudgetThreshold, specified as a scalar. For more information, see “Conditional Budget Constraints” on page 5-11.

Data Types: double

UpperGroup — Upper-bound group constraint

[] (default) | vector

Upper-bound group constraint, specified as a vector. For more information, see “Group Constraints” on page 5-11.

Data Types: double

UpperRatio — Maximum ratio of allocations between Groups A and B

[] (default) | vector

Maximum ratio of allocations between GroupA and GroupB, specified as a vector. For more information, see “Group Ratio Constraints” on page 5-12.

Data Types: double

BoundType — Type of bounds for each asset weight

'Simple' (default) | character vector with value 'Simple' or 'Conditional' | string with value "Simple" or "Conditional" | cell array of character vectors with values 'Simple' or 'Conditional' | string array with values "Simple" or "Conditional"

Type of bounds for each asset weight, specified as a scalar character vector or string, or a cell array of character vectors or a string array. For more information, see `setBounds`.

Data Types: char | cell | string

MinNumAssets — Minimum number of assets allocated in portfolio

[] (default) | numeric

Minimum number of assets allocated in portfolio, specified as a scalar numeric value. For more information, see `setMinMaxNumAssets` and “Cardinality Constraints” on page 5-14.

Data Types: double

MaxNumAssets — Maximum number of assets allocated in portfolio

[] (default) | numeric

Maximum number of assets allocated in portfolio, specified as a scalar numeric value. For more information, see `setMinMaxNumAssets` and “Cardinality Constraints” on page 5-14.

Data Types: double

Turnover — Turnover constraint

[] (default) | scalar

Turnover constraint, specified as a scalar. For more information, see “Average Turnover Constraints” on page 5-13 and “One-way Turnover Constraints” on page 5-13.

Data Types: double

SellTurnover — Turnover constraint on sales

[] (default) | scalar

Turnover constraint on sales, specified as a scalar. For more information, see “Average Turnover Constraints” on page 5-13 and “One-way Turnover Constraints” on page 5-13.

Data Types: double

BuyTurnover — Turnover constraint on purchases

[] (default) | scalar

Turnover constraint on purchases, specified as a scalar. For more information, see “Average Turnover Constraints” on page 5-13 and “One-way Turnover Constraints” on page 5-13.

Data Types: double

PortfolioCVaR Object Modeling**BuyCost — Proportional purchase costs**

[] (default) | vector

Proportional purchase costs, specified as a vector. For more information, see “Net Portfolio Returns” on page 5-4.

Data Types: double

RiskFreeRate — Risk-free rate

[] (default) | scalar

Risk-free rate, specified as a scalar.

Data Types: double

ProbabilityLevel — Value-at-risk probability level which is 1 – (loss probability)

[] (default) | scalar

Value-at-risk probability level which is 1 – (loss probability), specified as a scalar.

Data Types: double

NumScenarios — Number of scenarios

[] (default) | integer scalar

Number of scenarios, specified as an integer scalar.

Data Types: `double`

SellCost — Proportional sales costs

[] (default) | `vector`

Proportional sales costs, specified as a vector. For more information, see “Net Portfolio Returns” on page 5-4.

Data Types: `double`

Object Functions

<code>setAssetList</code>	Set up list of identifiers for assets
<code>setInitPort</code>	Set up initial or current portfolio
<code>setDefaultConstraints</code>	Set up portfolio constraints with nonnegative weights that sum to 1
<code>estimateAssetMoments</code>	Estimate mean and covariance of asset returns from data
<code>setCosts</code>	Set up proportional transaction costs for portfolio
<code>addEquality</code>	Add linear equality constraints for portfolio weights to existing constraints
<code>addGroupRatio</code>	Add group ratio constraints for portfolio weights to existing group ratio constraints
<code>addGroups</code>	Add group constraints for portfolio weights to existing group constraints
<code>addInequality</code>	Add linear inequality constraints for portfolio weights to existing constraints
<code>getBounds</code>	Obtain bounds for portfolio weights from portfolio object
<code>getBudget</code>	Obtain budget constraint bounds from portfolio object
<code>getCosts</code>	Obtain buy and sell transaction costs from portfolio object
<code>getEquality</code>	Obtain equality constraint arrays from portfolio object
<code>addGroupRatio</code>	Obtain group ratio constraint arrays from portfolio object
<code>addGroups</code>	Obtain group constraint arrays from portfolio object
<code>getInequality</code>	Obtain inequality constraint arrays from portfolio object
<code>getOneWayTurnover</code>	Obtain one-way turnover constraints from portfolio object
<code>setGroups</code>	Set up group constraints for portfolio weights
<code>setInequality</code>	Set up linear inequality constraints for portfolio weights
<code>setBounds</code>	Set up bounds for portfolio weights for portfolio
<code>setMinMaxNumAssets</code>	Set cardinality constraints on the number of assets invested in a portfolio
<code>setBudget</code>	Set up budget constraints for portfolio
<code>setConditionalBudget</code>	Set up conditional budget constraints for portfolio
<code>setCosts</code>	Set up proportional transaction costs for portfolio
<code>setDefaultConstraints</code>	Set up portfolio constraints with nonnegative weights that sum to 1
<code>setEquality</code>	Set up linear equality constraints for portfolio weights
<code>setGroupRatio</code>	Set up group ratio constraints for portfolio weights
<code>setInitPort</code>	Set up initial or current portfolio
<code>setOneWayTurnover</code>	Set up one-way portfolio turnover constraints
<code>setTurnover</code>	Set up maximum portfolio turnover constraint
<code>checkFeasibility</code>	Check feasibility of input portfolios against portfolio object
<code>estimateBounds</code>	Estimate global lower and upper bounds for set of portfolios

<code>estimateFrontier</code>	Estimate specified number of optimal portfolios on the efficient frontier
<code>estimateFrontierByReturn</code>	Estimate optimal portfolios with targeted portfolio returns
<code>estimateFrontierByRisk</code>	Estimate optimal portfolios with targeted portfolio risks
<code>estimateFrontierLimits</code>	Estimate optimal portfolios at endpoints of efficient frontier
<code>plotFrontier</code>	Plot efficient frontier
<code>estimatePortReturn</code>	Estimate mean of portfolio returns
<code>estimatePortRisk</code>	Estimate portfolio risk according to risk proxy associated with corresponding object
<code>setSolver</code>	Choose main solver and specify associated solver options for portfolio optimization
<code>setProbabilityLevel</code>	Set probability level for VaR and CVaR calculations
<code>setScenarios</code>	Set asset returns scenarios by direct matrix
<code>getScenarios</code>	Obtain scenarios from portfolio object
<code>simulateNormalScenariosByData</code>	Simulate multivariate normal asset return scenarios from data
<code>simulateNormalScenariosByMoments</code>	Simulate multivariate normal asset return scenarios from mean and covariance of asset returns
<code>estimateScenarioMoments</code>	Estimate mean and covariance of asset return scenarios
<code>estimatePortVaR</code>	Estimate value-at-risk for PortfolioCVaR object
<code>estimatePortStd</code>	Estimate standard deviation of portfolio returns

Examples

Create an Empty PortfolioCVaR Object

You can create a PortfolioCVaR object, `p`, with no input arguments and display it using `disp`.

```
p = PortfolioCVaR;
disp(p);
```

PortfolioCVaR with properties:

```

    BuyCost: []
    SellCost: []
    RiskFreeRate: []
    ProbabilityLevel: []
    Turnover: []
    BuyTurnover: []
    SellTurnover: []
    NumScenarios: []
    Name: []
    NumAssets: []
    AssetList: []
    InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: []
    UpperBound: []
    LowerBudget: []
    UpperBudget: []
    GroupMatrix: []
```

```

LowerGroup: []
UpperGroup: []
GroupA: []
GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: []

```

This approach provides a way to set up a portfolio optimization problem with the `PortfolioCVaR` function. You can then use the associated set functions to set and modify collections of properties in the `PortfolioCVaR` object.

Create a PortfolioCVaR Object Using a Single-Step Setup

You can use the `PortfolioCVaR` object directly to set up a “standard” portfolio optimization problem. Given scenarios of asset returns in the variable `AssetScenarios`, this problem is completely specified as follows:

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR('Scenarios', AssetScenarios, ...
'LowerBound', 0, 'LowerBudget', 1, 'UpperBudget', 1, ...
'ProbabilityLevel', 0.95)

p =
PortfolioCVaR with properties:

```

```

BuyCost: []
SellCost: []
RiskFreeRate: []
ProbabilityLevel: 0.9500
Turnover: []
BuyTurnover: []
SellTurnover: []
NumScenarios: 20000
Name: []
NumAssets: 4
AssetList: []
InitPort: []
AInequality: []
bInequality: []
AEquality: []
bEquality: []

```

```

        LowerBound: [4×1 double]
        UpperBound: []
        LowerBudget: 1
        UpperBudget: 1
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
        GroupA: []
        GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
        ConditionalBudgetThreshold: []
        ConditionalUpperBudget: []
        BoundType: []

```

Note that the `LowerBound` property value undergoes scalar expansion since `AssetScenarios` provides the dimensions of the problem.

Create a PortfolioCVaR Object Using a Sequence of Steps

Using a sequence of steps is an alternative way to accomplish the same task of setting up a “standard” CVaR portfolio optimization problem, given `AssetScenarios` variable is:

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

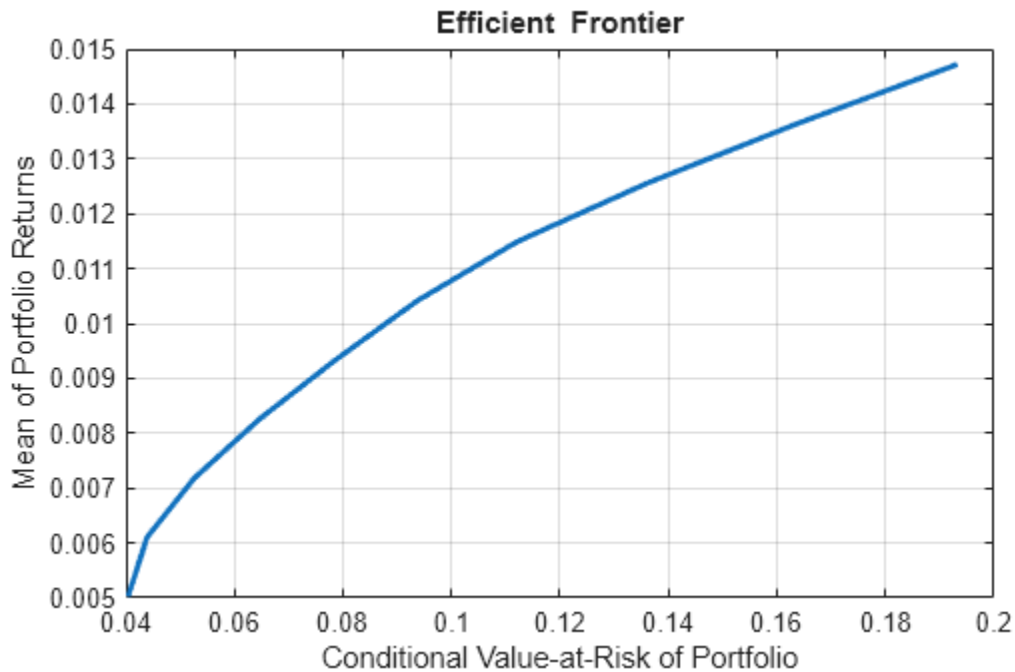
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = PortfolioCVaR(p, 'LowerBound', 0);
p = PortfolioCVaR(p, 'LowerBudget', 1, 'UpperBudget', 1);
p = setProbabilityLevel(p, 0.95);

plotFrontier(p);

```



This way works because the calls to `PortfolioCVaR` are in this particular order. In this case, the call to initialize `AssetScenarios` provides the dimensions for the problem. If you were to do this step last, you would have to explicitly dimension the `LowerBound` property as follows:

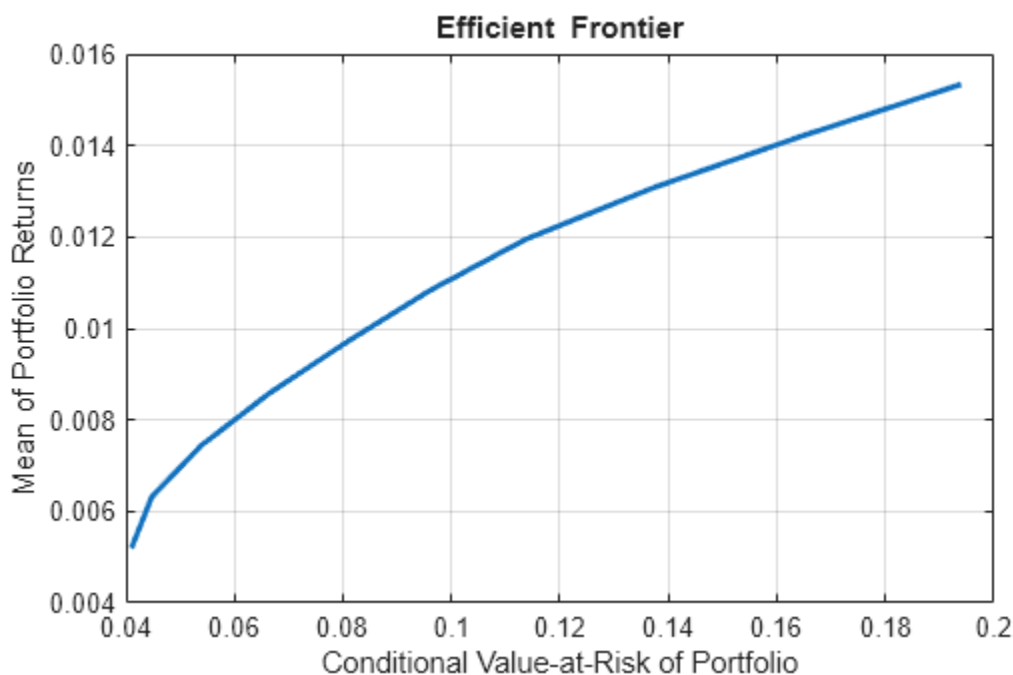
```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = PortfolioCVaR(p, 'LowerBound', zeros(size(m)));
p = PortfolioCVaR(p, 'LowerBudget', 1, 'UpperBudget', 1);
p = setProbabilityLevel(p, 0.95);
p = setScenarios(p, AssetScenarios);

plotFrontier(p);
```



If you did not specify the size of `LowerBound` but, instead, input a scalar argument, the `PortfolioCVaR` object assumes that you are defining a single-asset problem and produces an error at the call to set asset scenarios with four assets.

Create a PortfolioCVaR Object Using Shortcuts for Property Names

You can create a `PortfolioCVaR` object, `p` with the `PortfolioCVaR` object using shortcuts for property names.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
```

```
m = m/12;
C = C/12;
```

```
AssetScenarios = mvnrnd(m, C, 20000);
```

```
p = PortfolioCVaR('scenario', AssetScenarios, 'lb', 0, 'budget', 1, 'plevel', 0.95)
```

```
p =
PortfolioCVaR with properties:
    BuyCost: []
    SellCost: []
    RiskFreeRate: []
    ProbabilityLevel: 0.9500
    Turnover: []
```

```
BuyTurnover: []
SellTurnover: []
NumScenarios: 20000
    Name: []
    NumAssets: 4
    AssetList: []
    InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: [4×1 double]
    UpperBound: []
    LowerBudget: 1
    UpperBudget: 1
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
    GroupA: []
    GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
    BoundType: []
```

Direct Setting of PortfolioCVaR Object Properties

Although not recommended, you can set properties directly, however no error-checking is done on your inputs.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;

p = setScenarios(p, AssetScenarios);
p.ProbabilityLevel = 0.95;

p.LowerBudget = 1;
p.UpperBudget = 1;
p.LowerBound = zeros(size(m));
disp(p)

PortfolioCVaR with properties:
```

```

        BuyCost: []
        SellCost: []
        RiskFreeRate: []
        ProbabilityLevel: 0.9500
        Turnover: []
        BuyTurnover: []
        SellTurnover: []
        NumScenarios: 20000
        Name: []
        NumAssets: 4
        AssetList: []
        InitPort: []
        AInequality: []
        bInequality: []
        AEquality: []
        bEquality: []
        LowerBound: [4×1 double]
        UpperBound: []
        LowerBudget: 1
        UpperBudget: 1
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
        GroupA: []
        GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
        ConditionalBudgetThreshold: []
        ConditionalUpperBudget: []
        BoundType: []

```

Scenarios cannot be assigned directly to a PortfolioCVaR object. Scenarios must always be set through either the `PortfolioCVaR` function, the `setScenarios` function, or any of the scenario simulation functions.

Construct a PortfolioCVaR Object and Determine Efficient Portfolios

Create efficient portfolios:

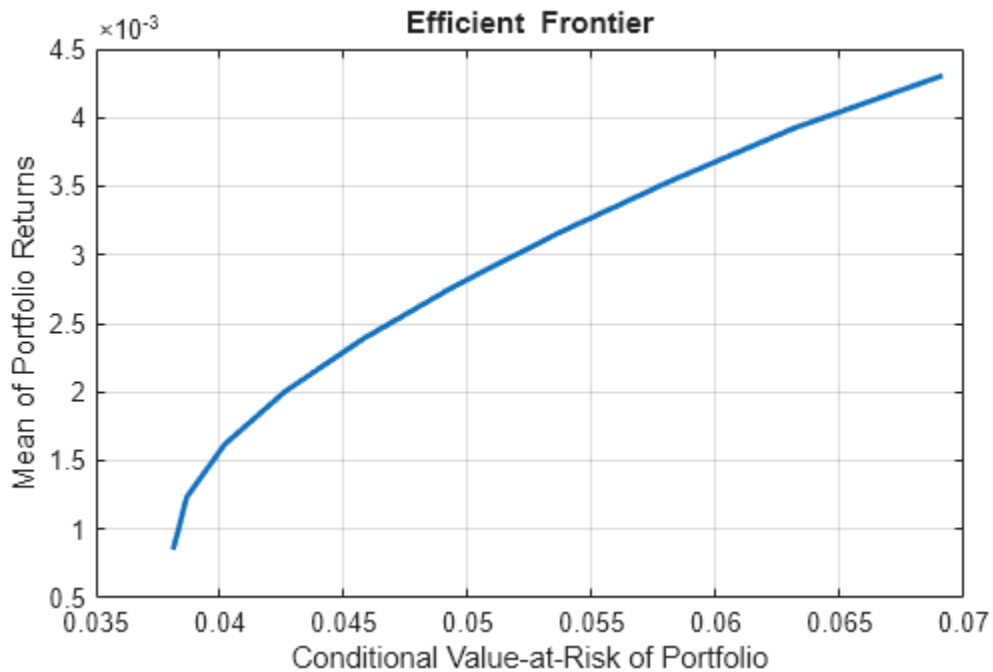
```

load CAPMuniverse

p = PortfolioCVaR('AssetList',Assets(1:12));
p = simulateNormalScenariosByData(p, Data(:,1:12), 20000 , 'missingdata', true);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);

plotFrontier(p);

```



```

pwgt = estimateFrontier(p, 5);

pnames = cell(1,5);
for i = 1:5
    pnames{i} = sprintf('Port%d',i);
end

Blotter = dataset([pwgt],pnames,'obsnames',p.AssetList);

disp(Blotter);

```

	Port1	Port2	Port3	Port4	Port5
AAPL	0.010223	0.073393	0.11939	0.13137	0
AMZN	0	0	0	0	0
CSCO	0	0	0	0	0
DELL	0.02301	0	0	0	0
EBAY	0	0	0	0	0
GOOG	0.20389	0.38068	0.56253	0.75919	1
HPQ	0.041396	0.009472	0	0	0
IBM	0.44369	0.36472	0.26247	0.10944	0
INTC	0	0	0	0	0
MSFT	0.27779	0.17174	0.055611	0	0
ORCL	0	0	0	0	0
YHOO	0	0	0	0	0

Mixed-Integer CVaR Portfolio Optimization Problem

This example shows how to solve a CVaR portfolio optimization problem with constraints in the number of selected assets or conditional (semicontinuous) bounds. To solve this problem, you can use

a PortfolioCVaR object along with different mixed integer nonlinear programming (MINLP) solvers.

CVaR Portfolio

Load the returns data in CAPMuniverse.mat. Then, create a PortfolioCVaR object with default constraints and a long-only portfolio whose weights sum to 1. For this example, you can define the feasible region of weights X as

$$X = \left\{ x \mid \sum_{i=1}^n x_i = 1, x_i \geq 0 \right\}.$$

```
% Load data
```

```
load CAPMuniverse.mat
```

```
% Create portfolio with default constraints
```

```
p = PortfolioCVaR(ProbabilityLevel=0.95);
```

```
p = simulateNormalScenariosByData(p,Data(:,1:12),1000,missingdata=true);
```

```
p = setDefaultConstraints(p);
```

Include binary variables for this scenario by setting conditional (semicontinuous) bounds. Conditional bounds are those such that $x_i = 0$ or $x_i \geq \ell_i$. In this example, $\ell_i = 0.1$ for all assets.

```
% Set conditional bounds
```

```
condLB = 0.1;
```

```
condUB = 0.5;
```

```
p = setBounds(p,condLB,condUB,BoundType="conditional");
```

Use estimateFrontier to estimate a set of portfolios on the efficient frontier. The efficient frontier is a curve that shows the trade-off between the return and risk achieved by Pareto-optimal portfolios. For a given return level, the portfolio on the efficient frontier is the one that minimizes the risk while maintaining the desired return. Conversely, for a given risk level, the portfolio on the efficient frontier is the one that maximizes return while maintaining the desired risk level.

```
% Compute efficient frontier
```

```
p = setSolverMINLP(p,'TrustRegionCP',DeltaLowerBound=condLB);
```

```
pwgt = estimateFrontier(p)
```

```
pwgt = 12x10
```

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0.1000	0.1090	0.1000	0.1000	0	0	0	0.0000	0	0
0	0	0	0.1000	0.1662	0.2634	0.3369	0.4051	0.4647	0.5
0.1328	0.2387	0.3331	0.3263	0.3622	0.3543	0.3701	0.3854	0.4287	0.5
0.1000	0	0	0	0	0	0	0	0	0
0.4132	0.3887	0.2755	0.2552	0.2288	0.1812	0.1110	0	0	0
0	0	0	0	0	0	0	0	0	0
0.2540	0.2636	0.2914	0.2185	0.2427	0.2011	0.1821	0.2095	0.1067	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
:	:	:	:	:	:	:	:	:	:

```
% Compute risk and returns of the portfolios on the efficient frontier
```

```
rsk = estimatePortRisk(p,pwgt)
```

```
rsk = 10×1
```

```
0.0368  
0.0377  
0.0398  
0.0423  
0.0453  
0.0485  
0.0520  
0.0558  
0.0598  
0.0647
```

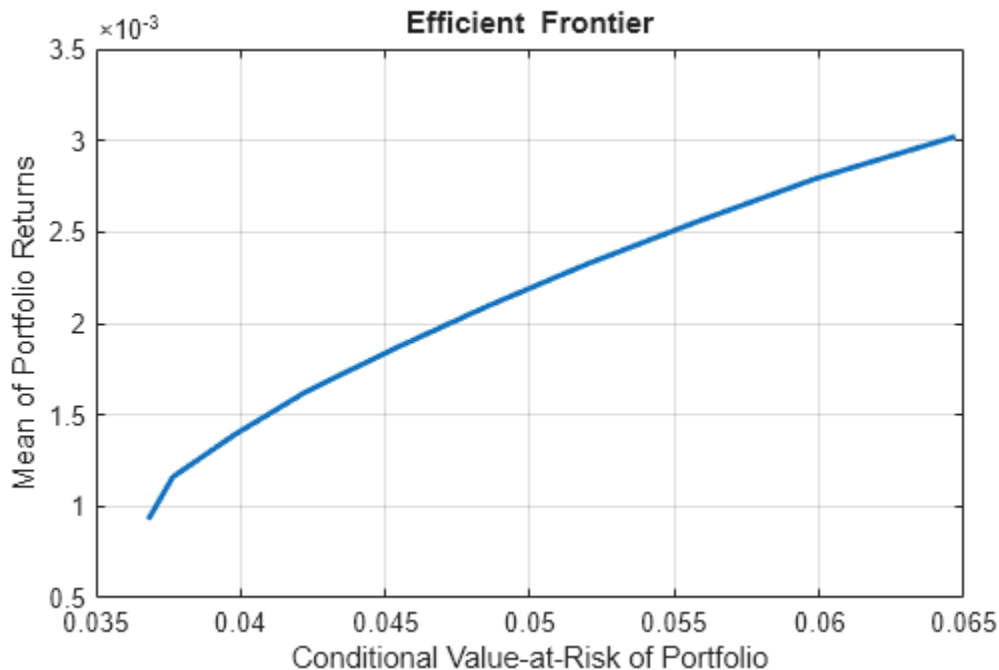
```
ret = estimatePortReturn(p,pwgt)
```

```
ret = 10×1
```

```
0.0009  
0.0012  
0.0014  
0.0016  
0.0019  
0.0021  
0.0023  
0.0026  
0.0028  
0.0030
```

Plot the weights from the frontier estimation using `plotFrontier`. The resulting curve is piece-wise concave with possible vertical jumps (discontinuities) between the concave intervals.

```
% Plot efficient frontier  
plotFrontier(p,pwgt)
```



Changing MINLP Solvers

In the previous section, you use the default solver for `estimateFrontier`. However, you can solve mixed-integer portfolio problems using any of the three algorithms supported by `setSolverMINLP`: `OuterApproximation`, `ExtendedCP`, and `TrustRegionCP`. Furthermore, the `OuterApproximation` algorithm accepts an additional name-value argument (`ExtendedFormulation`) for `Portfolio` problems, which reformulates problems with quadratic functions to work in an extended space that usually decreases the computation time. All algorithms, including the extended formulation variation of the `OuterApproximation` algorithm, return the same values within numerical accuracy. The available solvers are:

- `OuterApproximation` — The default algorithm, which is robust and usually faster than `ExtendedCP`
- `OuterApproximation` with `ExtendedFormulation` set to `true` — A robust algorithm that is usually faster than other algorithms, but only supported for `Portfolio` object problems
- `ExtendedCP` — The most robust solver, but usually the slowest
- `TrustRegionCP` — The fastest algorithm, but one that is less robust and may provide suboptimal solutions

For more information on solvers for mixed-integer portfolio problems, see “Choose MINLP Solvers for Portfolio Problems” on page 4-140.

To change the MINLP solvers, use `setSolverMINLP`.

```
% Select 'TrustRegionCP' as solver
p_OA = setSolverMINLP(p, 'OuterApproximation');
pwgt_OA = estimateFrontier(p_OA);
rsk_OA = estimatePortRisk(p, pwgt_OA);
ret_OA = estimatePortReturn(p, pwgt_OA);

% Select 'ExtendedCP' as solver using 'midway' cuts as 'CutGeneration'
```

```
p_ECP = setSolverMINLP(p, 'ExtendedCP', CutGeneration="midway");
pwgt_ECP = estimateFrontier(p_ECP);
rskECP = estimatePortRisk(p, pwgt_ECP);
retECP = estimatePortReturn(p, pwgt_ECP);
```

Compare the returns and risks obtained by the portfolios on the efficient frontier from the different solvers. They are all the same within a numerical accuracy that is an absolute difference $\leq 10^{-4}$.

```
retTable = table(retOA, ret, retECP, 'VariableNames', {'OA', 'TR', 'ECP'})
```

```
retTable=10×3 table
      OA      TR      ECP
-----
0.00092881  0.00092876  0.00092876
0.0011614   0.0011613   0.0011613
0.001394    0.0013939    0.0013939
0.0016265   0.0016265    0.0016265
0.0018591   0.0018591    0.0018591
0.0020917   0.0020917    0.0020917
0.0023243   0.0023243    0.0023243
0.0025569   0.0025569    0.0025569
0.0027894   0.0027894    0.0027894
0.003022    0.003022     0.003022
```

```
rskTable = table(rskOA, rsk, rskECP, 'VariableNames', {'OA', 'TR', 'ECP'})
```

```
rskTable=10×3 table
      OA      TR      ECP
-----
0.036808    0.036808    0.036808
0.037658    0.037658    0.037658
0.039808    0.039808    0.039808
0.042256    0.042256    0.042256
0.04527     0.045269    0.045269
0.048489    0.048488    0.048488
0.051963    0.051963    0.051963
0.055783    0.055783    0.055783
0.059828    0.059828    0.059828
0.064705    0.064705    0.064705
```

```
% Compare risks from the different OuterApproximation formulations
norm(rskTable.OA-rskTable.TR, Inf) <= 1e-4
```

```
ans = logical
      1
```

More About

Conditional Value-at-Risk Portfolio Optimization

A CVaR optimization problem is completely specified with four elements.

The four elements for a CVaR optimization problem are:

- A universe of assets with scenarios of asset total returns for a period of interest, where scenarios comprise a collection of samples from the underlying probability distribution for asset total returns. This collection must be sufficiently large for asymptotic convergence of sample statistics. Asset return moments and related statistics are derived exclusively from the scenarios.
- A portfolio set that specifies the set of portfolio choices in terms of a collection of constraints.
- A model for portfolio return and risk proxies, which, for CVaR optimization, is either the gross or net mean of portfolio returns and the conditional value-at-risk of portfolio returns.
- A probability level that specifies the probability that a loss is less than or equal to the value-at-risk. Typical values are 0.9 and 0.95, which indicate 10% and 5% loss probabilities.

After these four elements have been specified in an unambiguous way, it is possible to solve and analyze CVaR portfolio optimization problems.

The simplest CVaR portfolio optimization problem has:

- Scenarios of asset total returns
- A requirement that all portfolios have nonnegative weights that sum to 1 (the summation constraint is known as a budget constraint)
- Built-in models for portfolio return and risk proxies that use scenarios of asset total returns
- A probability level of 0.95

Given scenarios of asset returns in the variable `AssetScenarios`, this problem is completely specified by:

```
p = PortfolioCVaR('Scenarios', AssetScenarios, 'LowerBound', 0, 'Budget', 1, ...
    'ProbabilityLevel', 0.95);
```

or equivalently by:

```
p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);
```

To confirm that this is a valid portfolio optimization problem, the following function determines whether the set of PortfolioCVaR choices is bounded (a necessary condition for portfolio optimization).

```
[lb, ub, isbounded] = estimateBounds(p);
```

Given the problem specified in the `PortfolioCVaR` object `p`, the efficient frontier for this problem can be displayed with:

```
plotFrontier(p);
```

and efficient portfolios can be obtained with:

```
pwgt = estimateFrontier(p);
```

For more information on the theory and definition of conditional value-at-risk optimization supported by portfolio optimization tools in Financial Toolbox, see “Portfolio Optimization Theory” on page 5-3.

PortfolioCVaR Problem Sufficiency

A CVaR portfolio optimization problem is completely specified with the `PortfolioCVaR` object if three conditions are met.

The following are the three conditions that must be met:

- You must specify a collection of asset returns or prices known as scenarios such that all scenarios are finite asset returns or prices. These scenarios are meant to be samples from the underlying probability distribution of asset returns. This condition can be satisfied by the `setScenarios` function or with several canned scenario simulation functions.
- The set of feasible portfolios must be a nonempty compact set, where a compact set is closed and bounded. You can satisfy this condition using an extensive collection of properties that define different types of constraints to form a set of feasible portfolios. Since such sets must be bounded, either explicit or implicit constraints can be imposed and several tools, such as the `estimateBounds` function, provide ways to ensure that your problem is properly formulated.
- You must specify a probability level to locate the level of tail loss above which the conditional value-at-risk is to be minimized. This condition can be satisfied by the `setProbabilityLevel` function.

Although the general sufficient conditions for CVaR portfolio optimization go beyond the first three conditions, the `PortfolioCVaR` object handles all these additional conditions.

Shortcuts for Property Names

The `PortfolioCVaR` object has shorter argument names that replace longer argument names associated with specific properties of the `PortfolioCVaR` object.

For example, rather than enter `'ProbabilityLevel'`, the `PortfolioCVaR` object accepts the case-insensitive name `'plevel'` to set the `ProbabilityLevel` property in a `PortfolioCVaR` object. Every shorter argument name corresponds with a single property in the `PortfolioCVaR` object. The one exception is the alternative argument name `'budget'`, which signifies both the `LowerBudget` and `UpperBudget` properties. When `'budget'` is used, then the `LowerBudget` and `UpperBudget` properties are set to the same value to form an equality budget constraint.

Shortcuts for Property Names

Shortcut Argument Name	Equivalent Argument / Property Name
<code>ae</code>	<code>AEquality</code>
<code>ai</code>	<code>AInequality</code>
<code>assetnames</code> or <code>assets</code>	<code>AssetList</code>
<code>be</code>	<code>bEquality</code>
<code>bi</code>	<code>bInequality</code>
<code>budget</code>	<code>UpperBudget</code> and <code>LowerBudget</code>
<code>group</code>	<code>GroupMatrix</code>
<code>lb</code>	<code>LowerBound</code>
<code>n</code> or <code>num</code>	<code>NumAssets</code>
<code>level</code> , <code>problevel</code> , or <code>plevel</code>	<code>ProbabilityLevel</code>
<code>rfr</code>	<code>RiskFreeRate</code>
<code>scenario</code> or <code>assetscenarios</code>	<code>Scenarios</code>
<code>ub</code>	<code>UpperBound</code>

Version History

Introduced in R2012b

R2024b: Added Properties for ConditionalBudgetThreshold and ConditionalUpperBudget

Behavior changed in R2024b

To set conditional budget constraints using the PortfolioCVaR object, use `ConditionalBudgetThreshold` and `ConditionalUpperBudget` properties. In addition, you can use the `setConditionalBudget` function. For more information, see “Conditional Budget Constraints” on page 5-11.

References

[1] For a complete list of references for the PortfolioCVaR object, see “Portfolio Optimization” on page A-5.

See Also

`plotFrontier` | `estimateFrontier` | `setScenarios` | `Portfolio` | `PortfolioMAD` | `nearcorr`

Topics

“Creating the PortfolioCVaR Object” on page 5-22
 “Common Operations on the PortfolioCVaR Object” on page 5-29
 “Working with CVaR Portfolio Constraints Using Defaults” on page 5-51
 “Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36
 “Estimate Efficient Portfolios for Entire Frontier for PortfolioCVaR Object” on page 5-85
 “Estimate Efficient Frontiers for PortfolioCVaR Object” on page 5-105
 “Postprocessing Results to Set Up Tradable Portfolios” on page 5-115
 “Hedging Using CVaR Portfolio Optimization” on page 5-125
 “Bond Portfolio CVaR Optimization Using Diebold-Li Model” on page 5-146
 “Portfolio Optimization Theory” on page 5-3
 “PortfolioCVaR Object Workflow” on page 5-16
 “PortfolioCVaR Object Properties and Functions” on page 5-17
 “Working with PortfolioCVaR Objects” on page 5-17
 “Setting and Getting Properties” on page 5-18
 “Displaying PortfolioCVaR Objects” on page 5-18
 “Saving and Loading PortfolioCVaR Objects” on page 5-18
 “Estimating Efficient Portfolios and Frontiers” on page 5-18
 “Arrays of PortfolioCVaR Objects” on page 5-19
 “Subclassing PortfolioCVaR Objects” on page 5-20
 “Conventions for Representation of Data” on page 5-20
 Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8
 “Adding Constraints to Satisfy UCITS Directive” on page 4-457
 “Choosing and Controlling the Solver for PortfolioCVaR Optimizations” on page 5-98
 “Choose MINLP Solvers for Portfolio Problems” on page 4-140

External Websites

Getting Started with Portfolio Optimization (4 min 13 sec)
 CVaR Portfolio Optimization (4 min 56 sec)
 Portfolio Optimization Across Risk, Returns, and Climate (58 min 22 sec)

Managing and Fine-Tuning Portfolio Optimization Workflows with Experiment Manager (20 min 21 sec)

PortfolioMAD

Create PortfolioMAD object for mean-absolute deviation portfolio optimization and analysis

Description

The PortfolioMAD object implements mean-absolute deviation portfolio optimization, where MAD stands for “mean-absolute deviation.” PortfolioMAD objects support functions that are specific to MAD portfolio optimization.

The main workflow for MAD portfolio optimization is to create an instance of a PortfolioMAD object that completely specifies a portfolio optimization problem and to operate on the PortfolioMAD object to obtain and analyze efficient portfolios. For more information on the workflow when using PortfolioMAD objects, see “PortfolioMAD Object Workflow” on page 6-16.

You can use the PortfolioMAD object in several ways. To set up a portfolio optimization problem in a PortfolioMAD object, the simplest syntax is:

```
p = PortfolioMAD;
```

This syntax creates a PortfolioMAD object, *p*, such that all object properties are empty.

The PortfolioMAD object also accepts collections of name-value pair arguments for properties and their values. The PortfolioMAD object accepts inputs for properties with the general syntax:

```
p = PortfolioMAD('property1',value1,'property2',value2, ... );
```

If a PortfolioMAD object exists, the syntax permits the first (and only the first argument) of the PortfolioMAD object to be an existing object with subsequent name-value pair arguments for properties to be added or modified. For example, given an existing PortfolioMAD object in *p*, the general syntax is:

```
p = PortfolioMAD(p,'property1',value1,'property2',value2, ... );
```

Input argument names are not case-sensitive, but must be completely specified. In addition, several properties can be specified with alternative argument names (see “Shortcuts for Property Names” on page 15-1338). The PortfolioMAD object tries to detect problem dimensions from the inputs and, once set, subsequent inputs can undergo various scalar or matrix expansion operations that simplify the overall process to formulate a problem. In addition, a PortfolioMAD object is a value object so that, given portfolio *p*, the following code creates two objects, *p* and *q*, that are distinct:

```
q = PortfolioMAD(p, ...)
```

After creating a PortfolioMAD object, you can use the associated object functions to set portfolio constraints, analyze the efficient frontier, and validate the portfolio model.

For more detailed information on the theoretical basis for conditional value-at-risk portfolio optimization, see “Portfolio Optimization Theory” on page 6-3.

Creation

Syntax

```
p = PortfolioMAD
p = PortfolioMAD(Name,Value)

p = PortfolioMAD(p,Name,Value)
```

Description

`p = PortfolioMAD` creates an empty `PortfolioMAD` object for mean-absolute deviation portfolio optimization and analysis. You can then add elements to the `PortfolioMAD` object using the supported "add" and "set" functions. For more information, see "Creating the PortfolioMAD Object" on page 6-22.

`p = PortfolioMAD(Name,Value)` creates a `PortfolioMAD` object (`p`) and sets Properties on page 15-1320 using name-value pairs. For example, `p = PortfolioMAD('AssetList',Assets(1:12))`. You can specify multiple name-value pairs.

`p = PortfolioMAD(p,Name,Value)` creates a `PortfolioMAD` object (`p`) using a previously created `PortfolioMAD` object `p` and sets Properties on page 15-1320 using name-value pairs. You can specify multiple name-value pairs.

Input Arguments

p — Previously constructed PortfolioMAD object

object

Previously constructed `PortfolioMAD` object, specified using `PortfolioMAD`.

Output Arguments

p — PortfolioMAD optimization

`PortfolioMAD` object

`PortfolioMAD` optimization, returned as a `PortfolioMAD` object.

Properties

Setting Up the PortfolioMAD Object

AssetList — Names or symbols of assets in universe

[] (default) | cell array of character vectors | string array

Names or symbols of assets in the universe, specified as a cell array of character vectors or a string array.

Data Types: `cell` | `string`

InitPort — Initial portfolio

[] (default) | vector

Initial portfolio, specified as a vector.

Data Types: double

Name — Name for instance of PortfolioMAD object

[] (default) | character vector | string

Name for instance of the PortfolioMAD object, specified as a character vector.

Data Types: char | string

NumAssets — Number of assets in the universe

[] (default) | positive integer scalar

Number of assets in the universe, specified as a positive integer scalar.

Data Types: double

PortfolioMAD Object Constraints

AEquality — Linear equality constraint matrix

[] (default) | matrix

Linear equality constraint matrix, specified as a matrix. For more information, see “Linear Equality Constraints” on page 6-9.

Data Types: double

AInequality — Linear inequality constraint matrix

[] (default) | matrix

Linear inequality constraint matrix, specified as a matrix. For more information, see “Linear Inequality Constraints” on page 6-8.

Data Types: double

bEquality — Linear equality constraint vector

[] (default) | vector

Linear equality constraint vector, specified as a vector. For more information, see “Linear Equality Constraints” on page 6-9.

Data Types: double

bInequality — Linear inequality constraint

[] (default) | vector

Linear inequality constraint vector, specified as a vector. For more information, see “Linear Inequality Constraints” on page 6-8.

Data Types: double

GroupA — Group A weights to be bounded by weights in group B

[] (default) | matrix

Group A weights to be bounded by weights in group B, specified as a matrix. For more information, see “Group Constraints” on page 6-11.

Data Types: double

GroupB — Group B weights

[] (default) | matrix

Group B weights, specified as a matrix. For more information, see “Group Constraints” on page 6-11.

Data Types: double

GroupMatrix — Group membership matrix

[] (default) | matrix

Group membership matrix, specified as a matrix. For more information, see “Group Ratio Constraints” on page 6-12.

Data Types: double

LowerBound — Lower-bound constraint

[] (default) | vector

Lower-bound constraint, specified as a vector. For more information, see “‘Simple’ Bound Constraints” on page 6-9 and “‘Conditional’ Bound Constraints” on page 6-10.

Data Types: double

LowerBudget — Lower-bound budget constraint

[] (default) | scalar

Lower-bound budget constraint, specified as a scalar. For more information, see “Budget Constraints” on page 6-10.

Data Types: double

ConditionalBudgetThreshold — Weights threshold above which the asset is accounted for in conditional budget

[] (default) | scalar | vector

Weights threshold above which the asset is accounted for in conditional budget, specified as a scalar or a NumAssets-by-1 vector. For more information, see “Conditional Budget Constraints” on page 6-11.

Data Types: double

LowerGroup — Lower-bound group constraint

[] (default) | vector

Lower-bound group constraint, specified as a vector. For more information, see “Group Constraints” on page 6-11.

Data Types: double

LowerRatio — Minimum ratio of allocations between Groups A and B

[] (default) | vector

Minimum ratio of allocations between GroupA and GroupB, specified as a vector. For more information, see “Group Ratio Constraints” on page 6-12.

Data Types: double

UpperBound — Upper-bound constraint

[] (default) | vector

Upper-bound constraint, specified as a vector. For more information, see “‘Simple’ Bound Constraints” on page 6-9 and “‘Conditional’ Bound Constraints” on page 6-10.

Data Types: double

UpperBudget — Upper-bound budget constraint

[] (default) | scalar

Upper-bound budget constraint, specified as a scalar. For more information, see “Budget Constraints” on page 6-10.

Data Types: double

ConditionalUpperBudget — Maximum aggregate weight of the portfolio that can be invested in assets that exceed ConditionalBudgetThreshold

[] (default) | scalar

Maximum aggregate weight of the portfolio that can be invested in assets that exceed ConditionalBudgetThreshold, specified as a scalar. For more information, see “Conditional Budget Constraints” on page 6-11.

Data Types: double

UpperGroup — Upper-bound group constraint

[] (default) | vector

Upper-bound group constraint, specified as a vector. For more information, see “Group Constraints” on page 6-11.

Data Types: double

UpperRatio — Maximum ratio of allocations between Groups A and B

[] (default) | vector

Maximum ratio of allocations between GroupA and GroupB, specified as a vector. For more information, see “Group Ratio Constraints” on page 6-12.

Data Types: double

BoundType — Type of bounds for each asset weight

'Simple' (default) | character vector with value 'Simple' or 'Conditional' | string with value "Simple" or "Conditional" | cell array of character vectors with values 'Simple' or 'Conditional' | string array with values "Simple" or "Conditional"

Type of bounds for each asset weight, specified as a scalar character vector or string, or a cell array of character vectors or a string array. For more information, see setBounds.

Data Types: char | cell | string

MinNumAssets — Minimum number of assets allocated in portfolio

[] (default) | numeric

Minimum number of assets allocated in portfolio, specified as a scalar numeric value. For more information, see setMinMaxNumAssets and “Cardinality Constraints” on page 6-14.

Data Types: double

MaxNumAssets — Maximum number of assets allocated in portfolio

[] (default) | numeric

Maximum number of assets allocated in portfolio, specified as a scalar numeric value. For more information, see `setMinMaxNumAssets` and “Cardinality Constraints” on page 6-14.

Data Types: double

BuyTurnover — Turnover constraint on purchases

[] (default) | scalar

Turnover constraint on purchases, specified as a scalar. For more information, see “Average Turnover Constraints” on page 6-13 and “One-way Turnover Constraints” on page 6-13.

Data Types: double

SellTurnover — Turnover constraint on sales

[] (default) | scalar

Turnover constraint on sales, specified as a scalar. For more information, see “Average Turnover Constraints” on page 6-13 and “One-way Turnover Constraints” on page 6-13.

Data Types: double

Turnover — Turnover constraint

[] (default) | scalar

Turnover constraint, specified as a scalar. For more information, see “Average Turnover Constraints” on page 6-13 and “One-way Turnover Constraints” on page 6-13.

Data Types: double

PortfolioMAD Object Modeling**BuyCost — Proportional purchase costs**

[] (default) | vector

Proportional purchase costs, specified as a vector. For more information, see “Net Portfolio Returns” on page 6-4.

Data Types: double

RiskFreeRate — Risk-free rate

[] (default) | scalar

Risk-free rate, specified as a scalar.

Data Types: double

ProbabilityLevel — Value-at-risk probability level which is 1 – (loss probability)

[] (default) | scalar

Value-at-risk probability level which is 1 – (loss probability), specified as a scalar.

Data Types: double

NumScenarios — Number of scenarios

[] (default) | integer scalar

Number of scenarios, specified as an integer scalar.

Data Types: `double`

SellCost — Proportional sales costs

[] (default) | `vector`

Proportional sales costs, specified as a vector. For more information, see “Net Portfolio Returns” on page 6-4.

Data Types: `double`

Object Functions

<code>setAssetList</code>	Set up list of identifiers for assets
<code>setInitPort</code>	Set up initial or current portfolio
<code>setDefaultConstraints</code>	Set up portfolio constraints with nonnegative weights that sum to 1
<code>estimateAssetMoments</code>	Estimate mean and covariance of asset returns from data
<code>setCosts</code>	Set up proportional transaction costs for portfolio
<code>addEquality</code>	Add linear equality constraints for portfolio weights to existing constraints
<code>addGroupRatio</code>	Add group ratio constraints for portfolio weights to existing group ratio constraints
<code>addGroups</code>	Add group constraints for portfolio weights to existing group constraints
<code>addInequality</code>	Add linear inequality constraints for portfolio weights to existing constraints
<code>getBounds</code>	Obtain bounds for portfolio weights from portfolio object
<code>getBudget</code>	Obtain budget constraint bounds from portfolio object
<code>getCosts</code>	Obtain buy and sell transaction costs from portfolio object
<code>getEquality</code>	Obtain equality constraint arrays from portfolio object
<code>addGroupRatio</code>	Obtain group ratio constraint arrays from portfolio object
<code>addGroups</code>	Obtain group constraint arrays from portfolio object
<code>getInequality</code>	Obtain inequality constraint arrays from portfolio object
<code>getOneWayTurnover</code>	Obtain one-way turnover constraints from portfolio object
<code>setGroups</code>	Set up group constraints for portfolio weights
<code>setInequality</code>	Set up linear inequality constraints for portfolio weights
<code>setBounds</code>	Set up bounds for portfolio weights for portfolio
<code>setMinMaxNumAssets</code>	Set cardinality constraints on the number of assets invested in a portfolio
<code>setBudget</code>	Set up budget constraints for portfolio
<code>setConditionalBudget</code>	Set up conditional budget constraints for portfolio
<code>setCosts</code>	Set up proportional transaction costs for portfolio
<code>setDefaultConstraints</code>	Set up portfolio constraints with nonnegative weights that sum to 1
<code>setEquality</code>	Set up linear equality constraints for portfolio weights
<code>setGroupRatio</code>	Set up group ratio constraints for portfolio weights
<code>setInitPort</code>	Set up initial or current portfolio
<code>setOneWayTurnover</code>	Set up one-way portfolio turnover constraints
<code>setTurnover</code>	Set up maximum portfolio turnover constraint
<code>checkFeasibility</code>	Check feasibility of input portfolios against portfolio object
<code>estimateBounds</code>	Estimate global lower and upper bounds for set of portfolios

<code>estimateFrontier</code>	Estimate specified number of optimal portfolios on the efficient frontier
<code>estimateFrontierByReturn</code>	Estimate optimal portfolios with targeted portfolio returns
<code>estimateFrontierByRisk</code>	Estimate optimal portfolios with targeted portfolio risks
<code>estimateFrontierLimits</code>	Estimate optimal portfolios at endpoints of efficient frontier
<code>plotFrontier</code>	Plot efficient frontier
<code>estimatePortReturn</code>	Estimate mean of portfolio returns
<code>estimatePortRisk</code>	Estimate portfolio risk according to risk proxy associated with corresponding object
<code>setSolver</code>	Choose main solver and specify associated solver options for portfolio optimization
<code>setProbabilityLevel</code>	Set probability level for VaR and CVaR calculations
<code>setScenarios</code>	Set asset returns scenarios by direct matrix
<code>getScenarios</code>	Obtain scenarios from portfolio object
<code>simulateNormalScenariosByData</code>	Simulate multivariate normal asset return scenarios from data
<code>simulateNormalScenariosByMoments</code>	Simulate multivariate normal asset return scenarios from mean and covariance of asset returns
<code>estimateScenarioMoments</code>	Estimate mean and covariance of asset return scenarios
<code>estimatePortStd</code>	Estimate standard deviation of portfolio returns

Examples

Create an Empty PortfolioMAD Object

You can create a PortfolioMAD object, `p`, with no input arguments and display it using `disp`.

```
p = PortfolioMAD;  
disp(p);
```

PortfolioMAD with properties:

```
BuyCost: []  
SellCost: []  
RiskFreeRate: []  
Turnover: []  
BuyTurnover: []  
SellTurnover: []  
NumScenarios: []  
Name: []  
NumAssets: []  
AssetList: []  
InitPort: []  
AInequality: []  
bInequality: []  
AEquality: []  
bEquality: []  
LowerBound: []  
UpperBound: []  
LowerBudget: []  
UpperBudget: []  
GroupMatrix: []  
LowerGroup: []  
UpperGroup: []
```



```

        GroupA: []
        GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
        ConditionalBudgetThreshold: []
        ConditionalUpperBudget: []
        BoundType: []

```

This approach provides a way to set up a portfolio optimization problem with the `PortfolioMAD` function. You can then use the associated set functions to set and modify collections of properties in the `PortfolioMAD` object.

Create a PortfolioMAD Object Using a Single-Step Setup

You can use the `PortfolioMAD` object directly to set up a “standard” portfolio optimization problem. Given scenarios of asset returns in the variable `AssetScenarios`, this problem is completely specified as follows:

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD('Scenarios', AssetScenarios, ...
    'LowerBound', 0, 'LowerBudget', 1, 'UpperBudget', 1)

p =
    PortfolioMAD with properties:

        BuyCost: []
        SellCost: []
    RiskFreeRate: []
        Turnover: []
    BuyTurnover: []
    SellTurnover: []
    NumScenarios: 20000
        Name: []
        NumAssets: 4
        AssetList: []
        InitPort: []
    AInequality: []
    bInequality: []
        AEquality: []
        bEquality: []
    LowerBound: [4×1 double]
    UpperBound: []
    LowerBudget: 1
    UpperBudget: 1

```

```
GroupMatrix: []
LowerGroup: []
UpperGroup: []
GroupA: []
GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: []
```

Note that the `LowerBound` property value undergoes scalar expansion since `AssetScenarios` provides the dimensions of the problem.

Create a PortfolioMAD Object Using a Sequence of Steps

Using a sequence of steps is an alternative way to accomplish the same task of setting up a “standard” MAD portfolio optimization problem, given `AssetScenarios` variable is:

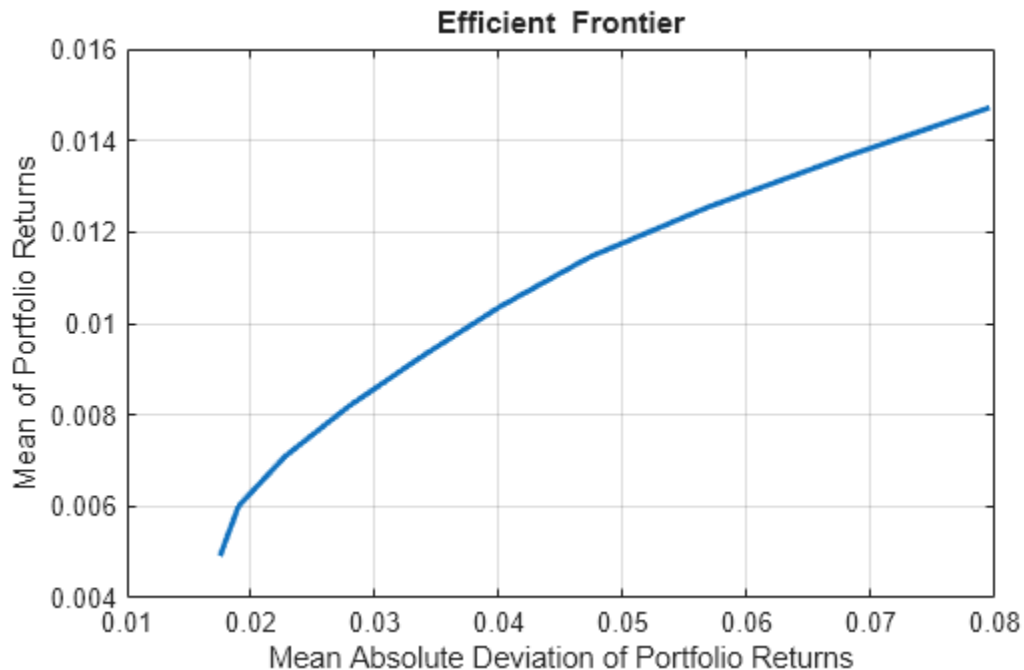
```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = PortfolioMAD(p, 'LowerBound', 0);
p = PortfolioMAD(p, 'LowerBudget', 1, 'UpperBudget', 1);

plotFrontier(p);
```



This way works because the calls to `PortfolioMAD` are in this particular order. In this case, the call to initialize `AssetScenarios` provides the dimensions for the problem. If you were to do this step last, you would have to explicitly dimension the `LowerBound` property as follows:

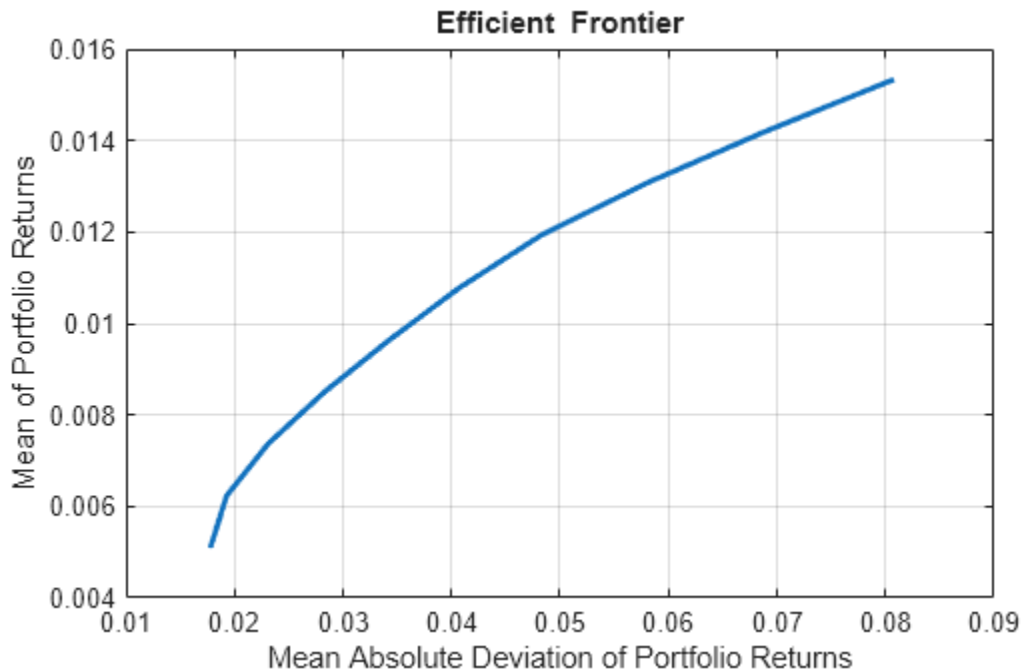
```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];

m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = PortfolioMAD(p, 'LowerBound', zeros(size(m)));
p = PortfolioMAD(p, 'LowerBudget', 1, 'UpperBudget', 1);
p = setScenarios(p, AssetScenarios);

plotFrontier(p);
```



If you did not specify the size of `LowerBound` but, instead, input a scalar argument, the `PortfolioMAD` object assumes that you are defining a single-asset problem and produces an error at the call to set asset scenarios with four assets.

Create a PortfolioMAD Object Using Shortcuts for Property Names

You can create a `PortfolioMAD` object, `p` with the `PortfolioMAD` object using shortcuts for property names.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
```

```
m = m/12;
C = C/12;
```

```
AssetScenarios = mvnrnd(m, C, 20000);
```

```
p = PortfolioMAD('scenario', AssetScenarios, 'lb', 0, 'budget', 1)
```

```
p =
PortfolioMAD with properties:
```

```
BuyCost: []
SellCost: []
RiskFreeRate: []
Turnover: []
BuyTurnover: []
```

```

SellTurnover: []
NumScenarios: 20000
    Name: []
    NumAssets: 4
    AssetList: []
    InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: [4×1 double]
    UpperBound: []
    LowerBudget: 1
    UpperBudget: 1
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
    GroupA: []
    GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: []

```

Direct Setting of PortfolioMAD Object Properties

Although not recommended, you can set properties directly, however no error-checking is done on your inputs.

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;

p = setScenarios(p, AssetScenarios);

p.LowerBudget = 1;
p.UpperBudget = 1;
p.LowerBound = zeros(size(m));
disp(p);

PortfolioMAD with properties:

    BuyCost: []

```

```
SellCost: []
RiskFreeRate: []
Turnover: []
BuyTurnover: []
SellTurnover: []
NumScenarios: 20000
    Name: []
    NumAssets: 4
    AssetList: []
    InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: [4×1 double]
    UpperBound: []
    LowerBudget: 1
    UpperBudget: 1
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
    GroupA: []
    GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
    ConditionalBudgetThreshold: []
    ConditionalUpperBudget: []
    BoundType: []
```

Scenarios cannot be assigned directly to a PortfolioMAD object. Scenarios must always be set through either the `PortfolioMAD` function, the `setScenarios` function, or any of the scenario simulation functions.

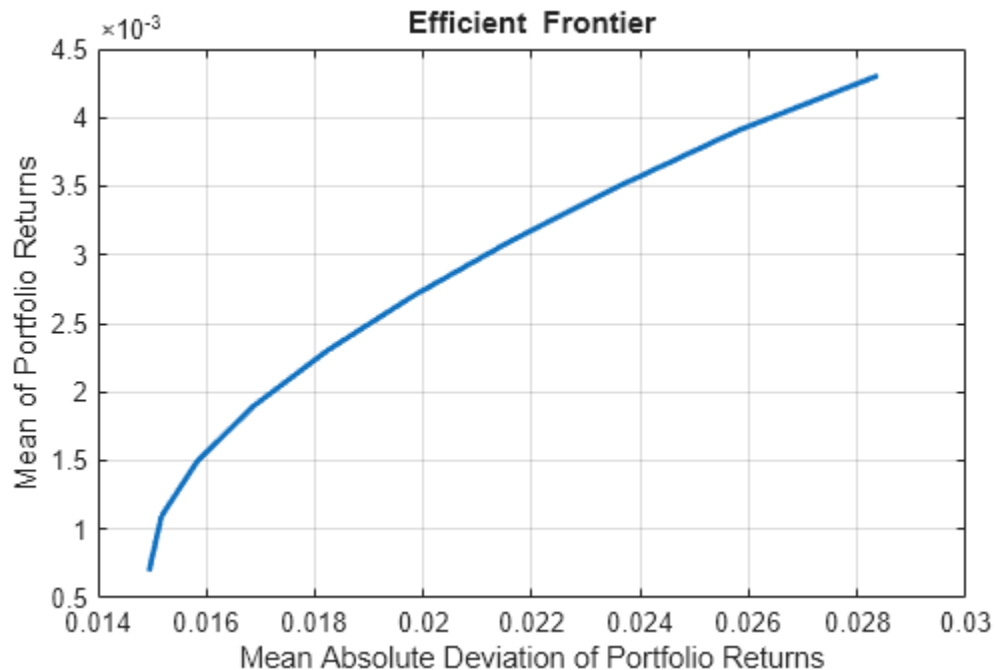
Create a PortfolioMAD Object and Determine Efficient Portfolios

Create efficient portfolios:

```
load CAPMuniverse

p = PortfolioMAD('AssetList',Assets(1:12));
p = simulateNormalScenariosByData(p, Data(:,1:12), 20000, 'missingdata',true);
p = setDefaultConstraints(p);

plotFrontier(p);
```



```

pwgt = estimateFrontier(p, 5);

pnames = cell(1,5);
for i = 1:5
    pnames{i} = sprintf('Port%d',i);
end

Blotter = dataset([pwgt],pnames,'obsnames',p.AssetList);

disp(Blotter);

```

	Port1	Port2	Port3	Port4	Port5
AAPL	0.029643	0.075874	0.11335	0.13405	0
AMZN	0	0	0	0	0
CSCO	0	0	0	0	0
DELL	0.0086367	0	0	0	0
EBAY	0	0	0	0	0
GOOG	0.16177	0.35217	0.54489	0.74913	1
HPQ	0.056891	0.023419	0	0	0
IBM	0.45916	0.37921	0.29376	0.11682	0
INTC	0	0	0	0	0
MSFT	0.2839	0.16933	0.048005	0	0
ORCL	0	0	0	0	0
YHOO	0	0	0	0	0

Mixed-Integer MAD Portfolio Optimization Problem

This example shows how to solve a MAD portfolio optimization problem with constraints in the number of selected assets or conditional (semicontinuous) bounds. To solve this problem, you can use a PortfolioMAD object along with different mixed integer nonlinear programming (MINLP) solvers.

MAD Portfolio

Load the returns data in `CAPMuniverse.mat`. Then, create a `PortfolioMAD` object with default constraints and a long-only portfolio whose weights sum to 1. For this example, you can define the feasible region of weights X as

$$X = \left\{ x \mid \sum_{i=1}^n x_i = 1, x_i \geq 0 \right\}.$$

```
% Load data
load CAPMuniverse.mat

% Create MAD portfolio with default constraints
p = PortfolioMAD;
p = simulateNormalScenariosByData(p,Data(:,1:12),1000,missingdata=true);
p = setDefaultConstraints(p);
```

Include binary variables for this scenario by setting conditional (semicontinuous) bounds. Conditional bounds are those such that $x_i = 0$ or $x_i \geq \ell_i$. In this example, $\ell_i = 0.1$ for all assets.

```
% Set conditional bounds
condLB = 0.1;
condUB = 0.5;
p = setBounds(p,condLB,condUB,BoundType="conditional");
```

Use `estimateFrontier` to estimate a set of portfolios on the efficient frontier. The efficient frontier is a curve that shows the trade-off between the return and risk achieved by Pareto-optimal portfolios. For a given return level, the portfolio on the efficient frontier is the one that minimizes the risk while maintaining the desired return. Conversely, for a given risk level, the portfolio on the efficient frontier is the one that maximizes return while maintaining the desired risk level.

```
% Compute efficient frontier
p = setSolverMINLP(p,'TrustRegionCP',DeltaLowerBound=condLB);
pwgt = estimateFrontier(p)
```

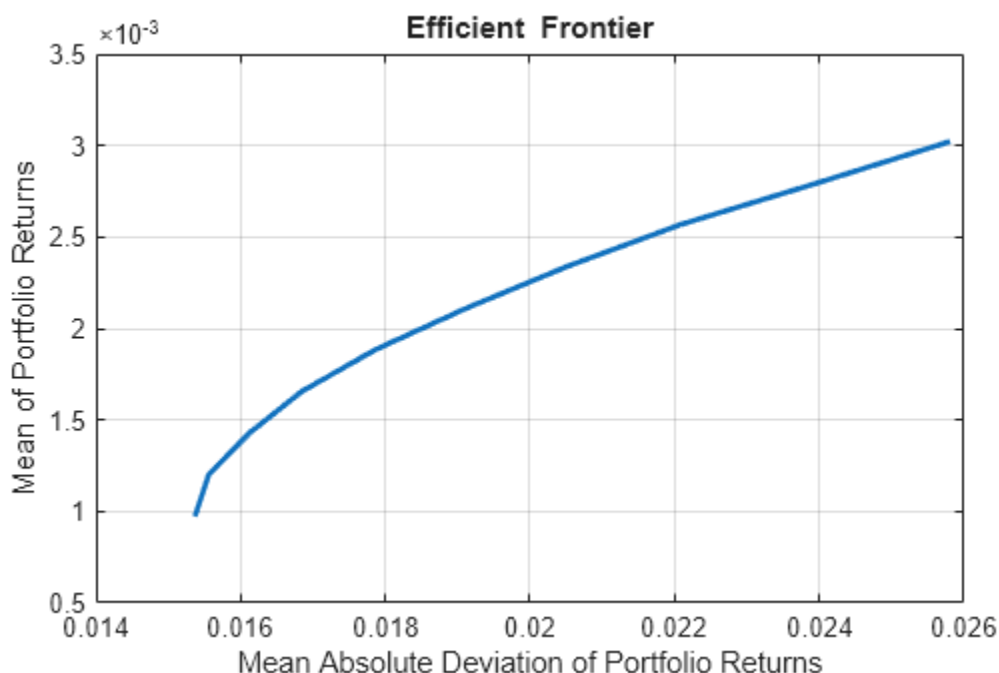
```
pwgt = 12x10
```

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0.1000	0.1283	0.1973	0.2316	0.3102	0.4375	0.5
0.1709	0.2574	0.3502	0.3371	0.3992	0.4272	0.4812	0.5000	0.4625	0.5
0	0.1000	0.1000	0.1000	0.1000	0	0	0	0	0
0.5000	0.3436	0.2380	0.1933	0.1000	0.1000	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0.3291	0.2990	0.3117	0.2696	0.2726	0.2755	0.2872	0.1898	0.1000	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
:	:	:	:	:	:	:	:	:	:

```
% Compute risk and returns of the portfolios on the efficient frontier
rsk = estimatePortRisk(p,pwgt);
ret = estimatePortReturn(p,pwgt);
```


Plot the weights from the frontier estimation with `plotFrontier`. The resulting curve is piece-wise concave with possible vertical jumps (discontinuities) between the concave intervals.

```
% Plot efficient frontier
plotFrontier(p,pwgt)
```



Changing MINLP Solvers

In the previous section, you use the default solver for `estimateFrontier`. However, you can solve mixed-integer portfolio problems using any of these three algorithms supported by `setSolverMINLP`: `OuterApproximation`, `ExtendedCP`, and `TrustRegionCP`. Furthermore, the `OuterApproximation` algorithm accepts an additional name-value argument (`ExtendedFormulation`) for `Portfolio` problems, which reformulates problems with quadratic functions to work in an extended space that usually decreases the computation time. All algorithms, including the extended formulation variation of the `OuterApproximation` algorithm, return the same values within numerical accuracy. The available solvers are:

- `OuterApproximation` — The default algorithm, which is robust and usually faster than `ExtendedCP`
- `OuterApproximation` with `ExtendedFormulation` set to `true` — A robust algorithm that is usually faster than other algorithms, but supported only for `Portfolio` object problems
- `ExtendedCP` — The most robust solver, but usually the slowest
- `TrustRegionCP` — The fastest algorithm, but one that is less robust and may provide suboptimal solutions

For more information on solvers for mixed-integer portfolio problems, see “Choose MINLP Solvers for Portfolio Problems” on page 4-140.

To change the MINLP solvers, use `setSolverMINLP`.

```
% Select 'TrustRegionCP' as solver
p_OA = setSolverMINLP(p,'OuterApproximation');
```

```

pwgt_OA = estimateFrontier(p_OA);
rskOA = estimatePortRisk(p,pwgt_OA);
retOA = estimatePortReturn(p,pwgt_OA);

% Select 'ExtendedCP' as solver using 'midway' cuts as 'CutGeneration'
p_ECP = setSolverMINLP(p,'ExtendedCP',CutGeneration="midway");
pwgt_ECP = estimateFrontier(p_ECP);
rskECP = estimatePortRisk(p,pwgt_ECP);
retECP = estimatePortReturn(p,pwgt_ECP);

```

Compare the returns and risks obtained by the portfolios on the efficient frontier from the different solvers. They are all the same within a numerical accuracy where the absolute difference $\leq 10^{-4}$.

```
retTable = table(retOA,ret,retECP,'VariableNames',{'OA','TR','ECP'})
```

```
retTable=10×3 table
      OA      TR      ECP
      ---      ---      ---
0.00097434  0.00097297  0.0009746
0.0012019  0.0012006  0.0012021
0.0014294  0.0014283  0.0014296
0.0016569  0.001656  0.0016571
0.0018844  0.0018837  0.0018846
0.0021119  0.0021113  0.0021121
0.0023395  0.002339  0.0023395
0.002567  0.0025667  0.002567
0.0027945  0.0027944  0.0027945
0.003022  0.003022  0.003022
```

```
rskTable = table(rskOA,rsk,rskECP,'VariableNames',{'OA','TR','ECP'})
```

```
rskTable=10×3 table
      OA      TR      ECP
      ---      ---      ---
0.015368  0.015368  0.015368
0.01556  0.015558  0.01556
0.016123  0.01612  0.016124
0.016849  0.016846  0.01685
0.017864  0.01786  0.017864
0.019127  0.019123  0.019127
0.020516  0.020513  0.020517
0.022074  0.022072  0.022074
0.023968  0.023965  0.023968
0.025807  0.025807  0.025807
```

```

% Compare risks from the different OuterApproximation formulations
norm(rskTable.OA-rskTable.TR,Inf) <= 1e-4

```

```
ans = logical
      1
```

More About

Mean-Absolute Deviation Portfolio Optimization

A MAD optimization problem is completely specified with three elements.

The three elements for a A MAD optimization problem are:

- A universe of assets with scenarios of asset total returns for a period of interest, where scenarios comprise a collection of samples from the underlying probability distribution for asset total returns. This collection must be sufficiently large for asymptotic convergence of sample statistics. Asset return moments and related statistics are derived exclusively from the scenarios.
- A portfolio set that specifies the set of portfolio choices in terms of a collection of constraints.
- A model for portfolio return and risk proxies, which, for MAD optimization, is either the gross or net mean of portfolio returns and the mean-absolute deviation of portfolio returns.

After these three elements have been specified unambiguously, it is possible to solve and analyze MAD portfolio optimization problems.

The simplest MAD portfolio optimization problem has:

- Scenarios of asset total returns
- A requirement that all portfolios have nonnegative weights that sum to 1 (the summation constraint is known as a budget constraint)
- Built-in models for portfolio return and risk proxies that use scenarios of asset total returns

Given scenarios of asset returns in the variable `AssetScenarios`, this problem is completely specified by:

```
p = PortfolioMAD('Scenarios', AssetScenarios, 'LowerBound', 0, 'Budget', 1);
```

or equivalently by:

```
p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
```

To confirm that this is a valid portfolio optimization problem, the following function determines whether the set of PortfolioMAD choices is bounded (a necessary condition for portfolio optimization).

```
[lb, ub, isbounded] = estimateBounds(p);
```

Given the problem specified in the `PortfolioMAD` object `p`, the efficient frontier for this problem can be displayed with:

```
plotFrontier(p);
```

and efficient portfolios can be obtained with:

```
pwgt = estimateFrontier(p);
```

For more detailed information on the theoretical basis for mean-absolute deviation optimization, see “Portfolio Optimization Theory” on page 6-3.

PortfolioMAD Problem Sufficiency

A MAD portfolio optimization problem is completely specified with the `PortfolioMAD` object if three conditions are met.

The following are the three conditions that must be met:

- You must specify a collection of asset returns or prices known as scenarios such that all scenarios are finite asset returns or prices. These scenarios are meant to be samples from the underlying probability distribution of asset returns. This condition can be satisfied by the `setScenarios` function or with several canned scenario simulation functions.
- The set of feasible portfolios must be a nonempty compact set, where a compact set is closed and bounded. You can satisfy this condition using an extensive collection of properties that define different types of constraints to form a set of feasible portfolios. Since such sets must be bounded, either explicit or implicit constraints can be imposed and several tools, such as the `estimateBounds` function, provide ways to ensure that your problem is properly formulated.

Although the general sufficient conditions for MAD portfolio optimization go beyond these conditions, the `PortfolioMAD` object handles all these additional conditions.

Shortcuts for Property Names

The `PortfolioMAD` object has shorter argument names that replace longer argument names associated with specific properties of the `PortfolioMAD` object.

For example, rather than enter `'AInequality'`, `PortfolioMAD` accepts the case-insensitive name `'ai'` to set the `AInequality` property in a `PortfolioMAD` object. Every shorter argument name corresponds with a single property in the `PortfolioMAD` function. The one exception is the alternative argument name `'budget'`, which signifies both the `LowerBudget` and `UpperBudget` properties. When `'budget'` is used, then the `LowerBudget` and `UpperBudget` properties are set to the same value to form an equality budget constraint.

Shortcuts for Property Names

Shortcut Argument Name	Equivalent Argument / Property Name
ae	AEquality
ai	AInequality
assetnames or assets	AssetList
be	bEquality
bi	bInequality
budget	UpperBudget and LowerBudget
group	GroupMatrix
lb	LowerBound
n or num	NumAssets
rfr	RiskFreeRate
scenario or assetscenarios	Scenarios
ub	UpperBound

Version History

Introduced in R2013b

R2024b: Added Properties for ConditionalBudgetThreshold and ConditionalUpperBudget

Behavior changed in R2024b

To set conditional budget constraints using the PortfolioMAD object, use ConditionalBudgetThreshold and ConditionalUpperBudget properties. In addition, you can use the setConditionalBudget function. For more information, see “Conditional Budget Constraints” on page 6-11.

References

[1] For a complete list of references for the PortfolioMAD object, see “Portfolio Optimization” on page A-5.

See Also

plotFrontier | estimateFrontier | setScenarios | PortfolioCVar | Portfolio | nearcorr

Topics

“Creating the PortfolioMAD Object” on page 6-22

“Common Operations on the PortfolioMAD Object” on page 6-29

“Working with MAD Portfolio Constraints Using Defaults” on page 6-50

“Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

“Validate the MAD Portfolio Problem” on page 6-80

“Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object” on page 6-84

“Estimate Efficient Frontiers for PortfolioMAD Object” on page 6-105

“Postprocessing Results to Set Up Tradable Portfolios” on page 6-113

“Portfolio Optimization Theory” on page 6-3

“PortfolioMAD Object Workflow” on page 6-16
“PortfolioMAD Object Properties and Functions” on page 6-17
“Working with PortfolioMAD Objects” on page 6-17
“Setting and Getting Properties” on page 6-18
“Displaying PortfolioMAD Objects” on page 6-18
“Saving and Loading PortfolioMAD Objects” on page 6-18
“Estimating Efficient Portfolios and Frontiers” on page 6-18
“Arrays of PortfolioMAD Objects” on page 6-19
“Subclassing PortfolioMAD Objects” on page 6-20
“Conventions for Representation of Data” on page 6-20
Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8
“Adding Constraints to Satisfy UCITS Directive” on page 4-457
“Choosing and Controlling the Solver for PortfolioMAD Optimizations” on page 6-99
“Choose MINLP Solvers for Portfolio Problems” on page 4-140

External Websites

Getting Started with Portfolio Optimization (4 min 13 sec)
Portfolio Optimization Across Risk, Returns, and Climate (58 min 22 sec)
Managing and Fine-Tuning Portfolio Optimization Workflows with Experiment Manager (20 min 21 sec)

portopt

Portfolios on constrained efficient frontier

Note portopt has been partially removed and will no longer accept `ConSet` or `varargin` arguments. Use `Portfolio` instead to solve portfolio problems that are more than a long-only fully-invested portfolio. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow”. For more information on migrating portopt code to `Portfolio`, see “portopt Migration to Portfolio Object”.

Syntax

```
[PortRisk,PortReturn,PortWts] = portopt(ExpReturn,ExpCovariance)
[PortRisk,PortReturn,PortWts] = portopt( ___,NumPorts,PortReturn)
portopt( ___,NumPorts,PortReturn)
```

Description

`[PortRisk,PortReturn,PortWts] = portopt(ExpReturn,ExpCovariance)` sets up the most basic portfolio problem with weights greater than or equal to 0 that must sum to 1. All that is necessary to solve this problem is the mean and covariance of asset returns. By default, portopt returns 10 equally-spaced points on the efficient frontier.

portopt solves the "standard" mean-variance portfolio optimization problem for a long-only fully-invested investor with no additional constraints. Specifically, every portfolios on the efficient frontier has non-negative weights that sum to 1.

Note An alternative for portfolio optimization is to use the `Portfolio` object for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

`[PortRisk,PortReturn,PortWts] = portopt( ___,NumPorts,PortReturn)` specifies options using one or more optional arguments in addition to the input arguments in the previous syntax.

`portopt( ___,NumPorts,PortReturn)` returns a plot of the efficient frontier if portopt is invoked with no output arguments.

Examples

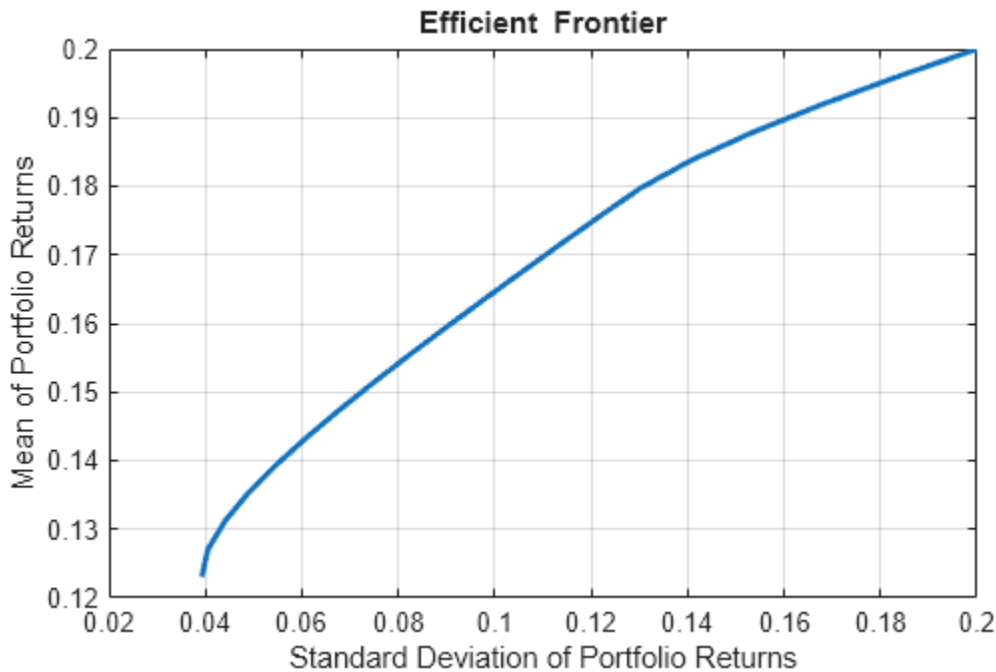
Plot the Risk-Return Efficient Frontier

Use portopt to connect 20 portfolios along the efficient frontier having evenly spaced returns. By default, choose among portfolios without short-selling and scale the value of the portfolio to 1.

```
ExpReturn = [0.1 0.2 0.15];

ExpCovariance = [0.005    -0.010    0.004
                 -0.010    0.040    -0.002
                 0.004    -0.002    0.023];

NumPorts = 20;
portopt(ExpReturn, ExpCovariance, NumPorts)
```



Input Arguments

ExpReturn — Expected (mean) return of each asset

vector

Expected (mean) return of each asset, specified as a 1-by-number of assets (NASSETS) vector.

Data Types: double

ExpCovariance — Covariance of the asset returns

matrix

Covariance of the asset returns, specified as a NASSETS-by-NASSETS matrix.

Data Types: double

NumPorts — Number of portfolios generated along the efficient frontier

10 (default) | scalar numeric

(Optional) Number of portfolios generated along the efficient frontier, specified as a scalar numeric. Returns are equally spaced between the maximum possible return and the minimum risk point. If NumPorts is empty (entered as []), portopt computes 10 equally spaced points. If you specify 1, portopt returns the minimum-risk portfolio.

Note If not over-ridden by `PortReturn`, these portfolios are spaced evenly from the minimum to the maximum return on the efficient frontier. If `NumPorts = 1`, then the minimum-risk portfolio is computed (positive integer).

Data Types: double

PortReturn — Target portfolio returns to be computed on the efficient frontier

[] (default) | vector

(Optional) Target portfolio returns to be computed on the efficient frontier, specified as a number of portfolios (NPORTS-by-1 vector). If not entered or empty, `NumPorts` equally spaced returns between the minimum and maximum possible values are used.

Note portopt requires that if you set `PortReturn`, `NumPorts` should be empty. If you specify `PortReturn` with a nonempty vector, `PortReturn` overrides `NumPorts`. If any returns in `PortReturn` fall outside the range of returns on the efficient frontier, portopt generates a warning and the efficient portfolios closest to the endpoints of the efficient frontier are computed.

Data Types: double

Output Arguments

PortRisk — Standard deviation of each portfolio

vector

Standard deviation of each portfolio, returned as a NPORTS-by-1 vector.

`PortWts` is an NPORTS-by-NASSETS matrix of weights allocated to each asset. Each row represents a portfolio. The total of all weights in a portfolio is 1.

PortReturn — expected return of each portfolio

vector

Expected return of each portfolio, returned as a NPORTS-by-1 vector.

PortWts — Weights allocated to each asset

matrix

Weights allocated to each asset, returned as a NPORTS-by-NASSETS matrix. Each row represents a portfolio. The total of all weights in a portfolio is 1.

More About

Efficient Frontier

The efficient frontier is a key concept in modern portfolio theory (MPT) that represents a set of optimal portfolios offering the highest expected return for a given level of risk, or the lowest risk for a given level of expected return.

The efficient frontier illustrates the tradeoff between risk (often measured as the standard deviation of portfolio returns) and expected return. Investors aim to construct portfolios that lie on this frontier

to maximize their returns for a given level of risk. Portfolios that lie on the efficient frontier are considered optimal because they provide the best possible expected return for their level of risk. Portfolios that lie below the frontier are suboptimal, as they do not provide sufficient return for the amount of risk taken. The efficient frontier emphasizes the importance of diversification. By combining different assets with varying returns and risks, investors can reduce the overall risk of the portfolio without sacrificing expected returns.

Version History

Introduced before R2006a

See Also

`ewstats` | `frontier` | `portstats` | `portcons` | `Portfolio`

Topics

“Portfolio Construction Examples” on page 3-5

“Plotting an Efficient Frontier Using `portopt`” on page 10-22

“Portfolio Selection and Risk Aversion” on page 3-7

“Bond Portfolio Optimization Using Portfolio Object” on page 10-30

“Active Returns and Tracking Error Efficient Frontier” on page 3-27

“`portopt` Migration to Portfolio Object” on page 3-11

“Analyzing Portfolios” on page 3-2

“Portfolio Optimization Functions” on page 3-3

portrand

Randomized portfolio risks, returns, and weights

Syntax

```
[PortRisk,PortReturn,PortWts] = portrand(Asset)
[PortRisk,PortReturn,PortWts] = portrand( ___,Return,Points,Method)
portrand(Asset,Return,Points,Method)
```

Description

`[PortRisk,PortReturn,PortWts] = portrand(Asset)` returns the risks, rates of return, and weights of random portfolio configurations. Portfolios are selected at random from a set of portfolios such that portfolio weights are nonnegative and sum to 1. The sample mean and covariance of asset returns are used to compute portfolio returns for each random portfolio.

Note An alternative for portfolio optimization is to use the `Portfolio` object for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

`[PortRisk,PortReturn,PortWts] = portrand( ___,Return,Points,Method)` returns the risks, rates of return, and weights of random portfolio configurations using optional arguments.

`portrand(Asset,Return,Points,Method)` plots the points representing each portfolio configuration. It does not return any data to the MATLAB workspace.

Input Arguments

Asset — Financial time series data

matrix

Financial time series data, specified as a matrix where each row is an observation and each column represents a single security.

Data Types: `double`

Return — Rate of return for corresponding security in Asset

Return computed by taking average value of each column of `Asset` (default) | row vector

(Optional) Rate of return for corresponding security in `Asset`, specified as a row vector, where each column represents the rate of return for the corresponding security.

Data Types: `double`

Points — Defines number of random points generated

1000 (default) | numeric

(Optional) Defines number of random points generated, specified as a numeric.

Data Types: double

Method — Method to generate random portfolios from set of portfolios

'uniform' (default) | character vector with value 'uniform' or 'geometric'

(Optional) Method to generate random portfolios from set of portfolios, specified as a character vector for one of the following:

- 'uniform' — Portfolio weights are generated that are uniformly distributed on the set of portfolio weights.
- 'geometric' — Portfolio weights are generated that are concentrated around the geometric center of the set of portfolio weights.

Note The 'uniform' and 'geometric' methods generate weights that are distributed symmetrically around the geometric center of the set of weights.

Data Types: double

Output Arguments

PortRisk — Portfolio risk

vector

Portfolio risk, returned as a POINTS-by-1 vector of standard deviations.

PortReturn — Portfolio return

vector

Portfolio return, returned as a POINTS-by-1 vector of expected rates of return.

PortWts — Portfolio asset weights

matrix

Portfolio asset weights, returned as a POINTS-by-(number of securities) matrix of asset weights. Each row of PortWts is a different portfolio configuration.

Version History

Introduced before R2006a

References

[1] Bodie, Kane, and Marcus. *Investments*. Chapter 7.

See Also

portror | portvar

Topics

“Portfolio Construction Examples” on page 3-5

“Portfolio Optimization Functions” on page 3-3

portror

Portfolio expected rate of return

Syntax

```
R = portror(Return,Weight)
```

Description

`R = portror(Return,Weight)` returns a 1-by-M vector for the expected rate of return.

Note An alternative for portfolio optimization is to use the `Portfolio` object for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

Examples

Portfolio Expected Rate of Return

This example shows a portfolio that is made up of two assets ABC and XYZ having expected rates of return of 10% and 14%, respectively. If 40% percent of the portfolio's funds are allocated to asset ABC and the remaining funds are allocated to asset XYZ, the portfolio's expected rate of return is:

```
r = portror([.1 .14],[.4 .6])  
  
r =  
0.1240
```

Input Arguments

Return — Rates of return

matrix

Rates of return, specified as a 1-by-N matrix. Each column of `Return` represents the rate of return for a single security.

Data Types: `double`

Weight — Weights

matrix

Weights, specified as a M-by-N matrix. Each row of `Weight` represents a different weighting combination of the assets in the portfolio.

Data Types: `double`

Output Arguments

R — Expected rate of return

vector

Expected rate of return, returned as a 1-by-M vector.

Version History

Introduced before R2006a

References

[1] Zvi Bodie, Alex Kane, Alan Marcus. *Investments*. McGraw-Hill Education; 10th edition (September 9, 2013).

See Also

portrand | portvar

Topics

“Portfolio Construction Examples” on page 3-5

“Portfolio Optimization Functions” on page 3-3

portsim

Monte Carlo simulation of correlated asset returns

Syntax

```
RetSeries = portsim(ExpReturn,ExpCovariance,NumObs)
RetSeries = portsim( ___,RetIntervals,NumSim,Method)
```

Description

`RetSeries = portsim(ExpReturn,ExpCovariance,NumObs)` simulates correlated returns of NASSETS assets over NUMOBS consecutive observation intervals. Asset returns are simulated as the proportional increments of constant drift, constant volatility stochastic processes, thereby approximating continuous-time geometric Brownian motion.

Note An alternative for portfolio optimization is to use the `Portfolio` object for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

`RetSeries = portsim( ___,RetIntervals,NumSim,Method)` specifies options using one or more optional arguments in addition to the input arguments in the previous syntax.

Examples

Distinction Between Simulation Methods

This example shows the distinction between the `Exact` and `Expected` methods of simulation.

Consider a portfolio of five assets with the following expected returns, standard deviations, and correlation matrix based on daily asset returns (where `ExpReturn` and `Sigmas` are divided by 100 to convert percentages to returns).

```
ExpReturn    = [0.0246  0.0189  0.0273  0.0141  0.0311]/100;
Sigmas       = [0.9509  1.4259  1.5227  1.1062  1.0877]/100;
Correlations = [1.0000  0.4403  0.4735  0.4334  0.6855
                0.4403  1.0000  0.7597  0.7809  0.4343
                0.4735  0.7597  1.0000  0.6978  0.4926
                0.4334  0.7809  0.6978  1.0000  0.4289
                0.6855  0.4343  0.4926  0.4289  1.0000];
```

Convert the correlations and standard deviations to a covariance matrix.

```
ExpCovariance = corr2cov(Sigmas, Correlations)
```

```
ExpCovariance = 5×5
10-3 ×
```



```

0.0904    0.0597    0.0686    0.0456    0.0709
0.0597    0.2033    0.1649    0.1232    0.0674
0.0686    0.1649    0.2319    0.1175    0.0816
0.0456    0.1232    0.1175    0.1224    0.0516
0.0709    0.0674    0.0816    0.0516    0.1183

```

Assume that there are 252 trading days in a calendar year, and simulate two sample paths (realizations) of daily returns over a two-year period. Since `ExpReturn` and `ExpCovariance` are expressed daily, set `RetIntervals = 1`.

```

StartPrice = 100;
NumObs     = 504; % two calendar years of daily returns
NumSim     = 2;
RetIntervals = 1; % one trading day
NumAssets  = 5;

```

To illustrate the distinction between methods, simulate two paths by each method, starting with the same random number state.

```

rng('default');
RetExact = porthsim(ExpReturn, ExpCovariance, NumObs, ...
RetIntervals, NumSim, 'Exact');
rng(0);
RetExpected = porthsim(ExpReturn, ExpCovariance, NumObs, ...
RetIntervals, NumSim, 'Expected');

```

Compare the mean and covariance of `RetExact` with the inputs (`ExpReturn` and `ExpCovariance`), you will observe that they are almost identical.

At this point, `RetExact` and `RetExpected` are both 504-by-5-by-2 arrays. Now assume an equally weighted portfolio formed from the five assets and create arrays of portfolio returns in which each column represents the portfolio return of the corresponding sample path of the simulated returns of the five assets. The portfolio arrays `PortRetExact` and `PortRetExpected` are 504-by-2 matrices.

```

Weights      = ones(NumAssets, 1)/NumAssets;
PortRetExact  = zeros(NumObs, NumSim);
PortRetExpected = zeros(NumObs, NumSim);

for i = 1:NumSim
    PortRetExact(:,i) = RetExact(:, :, i) * Weights;
    PortRetExpected(:,i) = RetExpected(:, :, i) * Weights;
end

```

Finally, convert the simulated portfolio returns to prices and plot the data. In particular, note that since the `Exact` method matches expected return and covariance, the terminal portfolio prices are virtually identical for each sample path. This is not true for the `Expected` simulation method. Although this example examines portfolios, the same methods apply to individual assets as well. Thus, `Exact` simulation is most appropriate when unique paths are required to reach the same terminal prices.

```

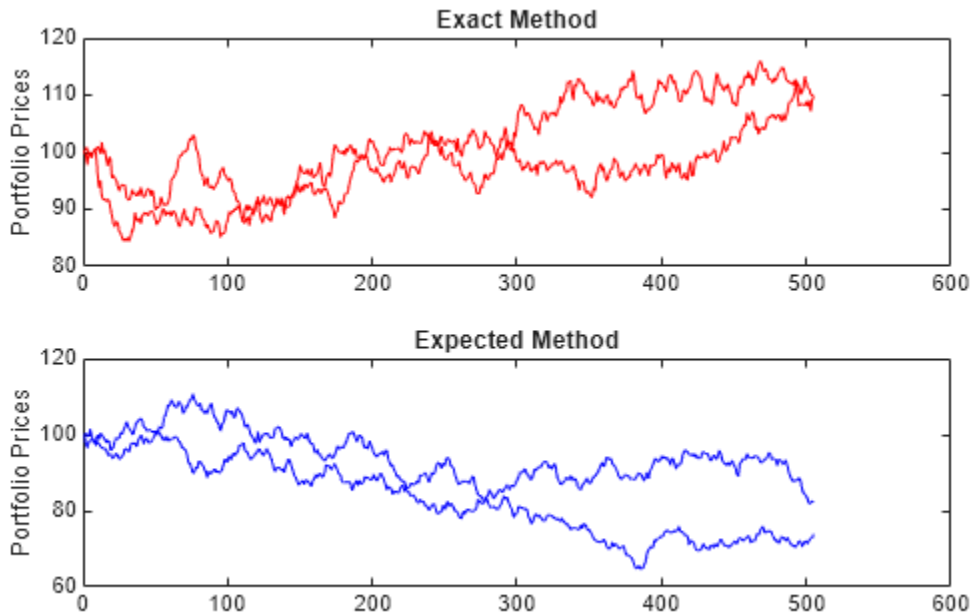
PortExact = ret2tick(PortRetExact, ...
repmat(StartPrice, 1, NumSim));
PortExpected = ret2tick(PortRetExpected, ...
repmat(StartPrice, 1, NumSim));
subplot(2,1,1), plot(PortExact, '-r')

```

```

ylabel('Portfolio Prices')
title('Exact Method')
subplot(2,1,2), plot(PortExpected, '-b')
ylabel('Portfolio Prices')
title('Expected Method')

```



Interaction Between ExpReturn, ExpCovariance, and RetIntervals

This example shows the interplay among ExpReturn, ExpCovariance, and RetIntervals. Recall that `portsim` simulates correlated asset returns over an interval of length dt , given by the equation

$$\frac{dS}{S} = \mu dt + \sigma dz = \mu dt + \sigma \varepsilon \sqrt{dt},$$

where S is the asset price, μ is the expected rate of return, σ is the volatility of the asset price, and ε represents a random drawing from a standardized normal distribution.

The time increment dt is determined by the optional input `RetIntervals`, either as an explicit input argument or as a unit time increment by default. Regardless, the periodicity of `ExpReturn`, `ExpCovariance`, and `RetIntervals` must be consistent. For example, if `ExpReturn` and `ExpCovariance` are annualized, then `RetIntervals` must be in years. This point is often misunderstood.

To illustrate the interplay among `ExpReturn`, `ExpCovariance`, and `RetIntervals`, consider a portfolio of five assets with the following expected returns, standard deviations, and correlation matrix based on daily asset returns.

```

ExpReturn    = [0.0246  0.0189  0.0273  0.0141  0.0311]/100;
Sigmas       = [0.9509  1.4259  1.5227  1.1062  1.0877]/100;

```

```
Correlations = [1.0000 0.4403 0.4735 0.4334 0.6855
               0.4403 1.0000 0.7597 0.7809 0.4343
               0.4735 0.7597 1.0000 0.6978 0.4926
               0.4334 0.7809 0.6978 1.0000 0.4289
               0.6855 0.4343 0.4926 0.4289 1.0000];
```

Convert the correlations and standard deviations to a covariance matrix of daily returns.

```
ExpCovariance = corr2cov(Sigmas, Correlations);
```

Assume 252 trading days per calendar year, and simulate a single sample path of daily returns over a four-year period. Since the ExpReturn and ExpCovariance inputs are expressed daily, set RetIntervals = 1.

```
StartPrice = 100;
NumObs = 1008; % four calendar years of daily returns
RetIntervals = 1; % one trading day
NumAssets = length(ExpReturn);
randn('state',0);
RetSeries1 = portsim(ExpReturn, ExpCovariance, NumObs, ...
RetIntervals, 1, 'Expected');
```

Now annualize the daily data, thereby changing the periodicity of the data, by multiplying ExpReturn and ExpCovariance by 252 and dividing RetIntervals by 252 (RetIntervals = 1/252 of a year). Resetting the random number generator to its initial state, you can reproduce the results.

```
rng('default');
RetSeries2 = portsim(ExpReturn*252, ExpCovariance*252, ...
NumObs, RetIntervals/252, 1, 'Expected');
```

Assume an equally weighted portfolio and compute portfolio returns associated with each simulated return series.

```
Weights = ones(NumAssets, 1)/NumAssets;
```

```
PortRet1 = RetSeries2 * Weights;
PortRet2 = RetSeries2 * Weights;
```

Comparison of the data reveals that PortRet1 and PortRet2 are identical.

Univariate Geometric Brownian Motion

This example shows how to simulate a univariate geometric Brownian motion process. It is based on an example found in Hull, *Options, Futures, and Other Derivatives*, 5th Edition (see example 12.2 on page 236). In addition to verifying Hull's example, it also graphically illustrates the lognormal property of terminal stock prices by a rather large Monte Carlo simulation.

Assume that you own a stock with an initial price of \$20, an annualized expected return of 20% and volatility of 40%. Simulate the daily price process for this stock over the course of one full calendar year (252 trading days).

```
StartPrice = 20;
ExpReturn = 0.2;
ExpCovariance = 0.4^2;
NumObs = 252;
NumSim = 10000;
RetIntervals = 1/252;
```

`RetIntervals` is expressed in years, consistent with the fact that `ExpReturn` and `ExpCovariance` are annualized. Also, `ExpCovariance` is entered as a variance rather than the more familiar standard deviation (volatility).

Set the random number generator state, and simulate 10,000 trials (realizations) of stock returns over a full calendar year of 252 trading days.

```
rng('default');
RetSeries = squeeze(portsim(ExpReturn, ExpCovariance, NumObs, ...
RetIntervals, NumSim, 'Expected'));
```

The `squeeze` function reformats the output array of simulated returns from a 252-by-1-by-10000 array to more convenient 252-by-10000 array. (Recall that `portsim` is fundamentally a multivariate simulation engine).

In accordance with Hull's equations 12.4 and 12.5 on page 236

$$E(S_T) = S_0 e^{\mu T}$$

$$\text{var}(S_T) = S_0^2 e^{2\mu T} (e^{\sigma^2 T} - 1)$$

convert the simulated return series to a price series and compute the sample mean and the variance of the terminal stock prices.

```
StockPrices = ret2tick(RetSeries, repmat(StartPrice, 1, NumSim));
```

```
SampMean = mean(StockPrices(end,:))
SampVar = var(StockPrices(end,:))
```

```
SampMean =
```

```
24.4489
```

```
SampVar =
```

```
101.4243
```

Compare these values with the values you obtain by using Hull's equations.

```
ExpValue = StartPrice*exp(ExpReturn)
ExpVar = ...
StartPrice*StartPrice*exp(2*ExpReturn)*(exp((ExpCovariance)) - 1)
```

```
ExpValue =
```

```
24.4281
```

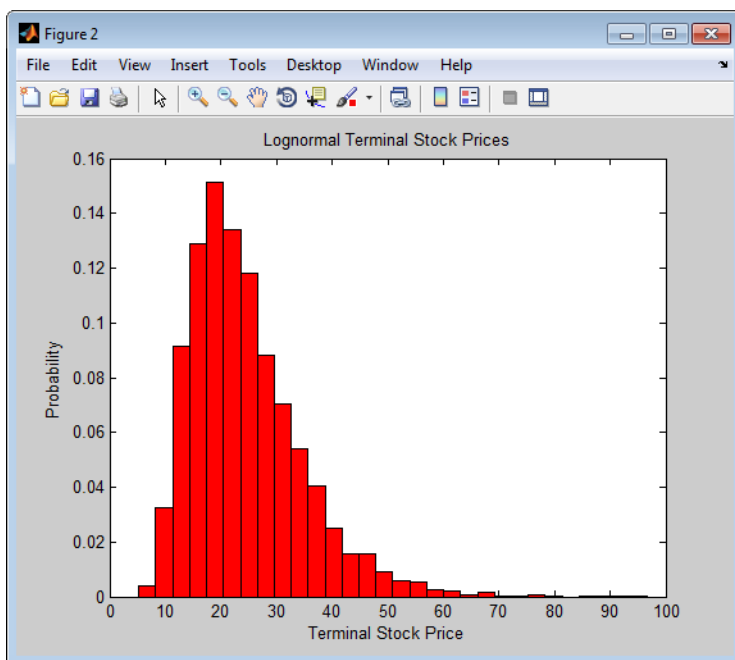
```
ExpVar =
```

```
103.5391
```

These results are very close to the results shown in Hull's example 12.2.

Display the sample density function of the terminal stock price after one calendar year. From the sample density function, the lognormal distribution of terminal stock prices is apparent.

```
[count, BinCenter] = hist(StockPrices(end,:), 30);
figure
bar(BinCenter, count/sum(count), 1, 'r')
xlabel('Terminal Stock Price')
ylabel('Probability')
title('Lognormal Terminal Stock Prices')
```



Input Arguments

ExpReturn — Expected (mean) return of each asset

vector

Expected (mean) return of each asset, specified as a 1-by-NASSETS vector.

Data Types: double

ExpCovariance — Asset return covariances

matrix

Asset return covariances, specified as an NASSETS-by-NASSETS matrix. ExpCovariance must be symmetric and positive semidefinite (no negative eigenvalues). The standard deviations of the returns are $\text{ExpSigma} = \sqrt{\text{diag}(\text{ExpCovariance})}$. If ExpCovariance is not a symmetric positive semidefinite matrix, use `nearcorr` to create a positive semidefinite matrix for a correlation matrix.

Data Types: double

NumObs — Number of consecutive observations in the return time series

positive scalar integer

number of consecutive observations in the return time series, specified as a positive scalar integer. If NumObs is entered as the empty matrix `[]`, the length of RetIntervals is used.

Data Types: double

RetIntervals — Interval times between observations

1 (default) | positive scalar | vector

(Optional) Interval times between observations, specified as a positive scalar or a number of observations NUMOBS-by-1 vector. If RetIntervals is not specified, all intervals are assumed to have length 1.

Data Types: double

NumSim — Number of simulated sample paths (realizations) of NUMOBS observations

1 (default) | positive scalar integer

(Optional) Number of simulated sample paths (realizations) of NUMOBS observations, specified as a positive scalar integer. The default value for NumSim is 1 (single realization of NUMOBS correlated asset returns).

Data Types: double

Method — Type of Monte Carlo simulation

'Exact' (default) | character vector

(Optional) Type of Monte Carlo simulation, specified as a character vector with one of the following values:

- 'Exact' (default) generates correlated asset returns in which the sample mean and covariance match the input mean (ExpReturn) and covariance (ExpCovariance) specifications.
- 'Expected' generates correlated asset returns in which the sample mean and covariance are statistically equal to the input mean and covariance specifications. (The expected values of the sample mean and covariance are equal to the input mean (ExpReturn) and covariance (ExpCovariance) specifications.)

For either Method, the sample mean and covariance returned are appropriately scaled by RetIntervals.

Data Types: char

Output Arguments

RetSeries — Three-dimensional array of correlated, normally distributed, proportional asset returns

array

Three-dimensional array of correlated, normally distributed, proportional asset returns, returned as a NUMOBS-by-NASSETS-by-NUMSIM three-dimensional array.

Asset returns over an interval of length dt are given by

$$\frac{dS}{S} = \mu dt + \sigma dz = \mu dt + \sigma \varepsilon \sqrt{dt},$$

where S is the asset price, μ is the expected rate of return, σ is the volatility of the asset price, and ε represents a random drawing from a standardized normal distribution.

Notes

- When Method is 'Exact', the sample mean and covariance of all realizations (scaled by RetIntervals) match the input mean and covariance. When the returns are then converted to asset prices, all terminal prices for a given asset are in close agreement. Although all realizations are drawn independently, they produce similar terminal asset prices. Set Method to 'Expected' to avoid this behavior.

- The returns from the portfolios in `PortWts` are given by `PortReturn = PortWts * RetSeries(:, :, 1)'`, where `PortWts` is a matrix in which each row contains the asset allocations of a portfolio. Each row of `PortReturn` corresponds to one of the portfolios identified in `PortWts`, and each column corresponds to one of the observations taken from the first realization (the first plane) in `RetSeries`. See `portopt` and `portstats` for portfolio specification and optimization.

Version History

Introduced before R2006a

References

[1] Hull, J. C. *Options, Futures, and Other Derivatives*. Prentice-Hall, 2003.

See Also

`ewstats` | `portopt` | `portstats` | `randn` | `ret2tick` | `squeeze` | `nearcorr`

Topics

“Portfolio Construction Examples” on page 3-5

“Portfolio Optimization Functions” on page 3-3

portstats

Portfolio expected return and risk

Syntax

```
[PortRisk,PortReturn] = portstats(ExpReturn,ExpCovariance)
[PortRisk,PortReturn] = portstats( ___,Wts)
```

Description

[PortRisk,PortReturn] = portstats(ExpReturn,ExpCovariance) computes the expected rate of return and risk for a portfolio of assets.

Note An alternative for portfolio optimization is to use the `Portfolio` object for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

[PortRisk,PortReturn] = portstats(___,Wts) specifies options using one or more optional arguments in addition to the input arguments in the previous syntax.

Examples

Computes the Expected Rate of Return and Risk for a Portfolio of Assets

This example shows how to calculate the expected rate of return and risk for a portfolio of assets.

```
ExpReturn = [0.1 0.2 0.15];
```

```
ExpCovariance = [0.0100    -0.0061    0.0042
                 -0.0061    0.0400   -0.0252
                 0.0042   -0.0252    0.0225 ];
```

```
PortWts=[0.4 0.2 0.4; 0.2 0.4 0.2];
```

```
[PortRisk, PortReturn] = portstats(ExpReturn, ExpCovariance,...
PortWts)
```

```
PortRisk = 2×1
```

```
    0.0560
    0.0550
```

```
PortReturn = 2×1
```

```
    0.1400
```


0.1300

Input Arguments

ExpReturn — Expected (mean) return of each asset

vector

Expected (mean) return of each asset, specified as a 1-by-NASSETS vector.

Data Types: double

ExpCovariance — Asset return covariances

matrix

Asset return covariances, specified as an NASSETS-by-NASSETS matrix.

Data Types: double

Wts — Weights allocated to each asset

1/NASSETS (equally weighted) (default) | matrix

(Optional) Weights allocated to each asset, specified as an NPORTS-by-NASSETS matrix. Each row represents a different weighting combination of the assets in the portfolio. If Wts is not entered, weights of 1/NASSETS are assigned to each security.

Data Types: double

Output Arguments

PortRisk — Standard deviation of each portfolio

vector

Standard deviation of each portfolio, returned as an NPORTS-by-1 vector.

PortReturn — Expected return of each portfolio

vector

Expected return of each portfolio, returned an NPORTS-by-1 vector.

Version History

Introduced before R2006a

See Also

ewstats | portalloc

Topics

“Portfolio Construction Examples” on page 3-5

“Portfolio Selection and Risk Aversion” on page 3-7

“Active Returns and Tracking Error Efficient Frontier” on page 3-27

“Analyzing Portfolios” on page 3-2

“Portfolio Optimization Functions” on page 3-3

portvar

Variance for portfolio of assets

Syntax

```
V = portvar(Asset)
V = portvar(Asset,Weight)
```

Description

`V = portvar(Asset)` assigns each security an equal weight when calculating the portfolio variance.

Note An alternative for portfolio optimization is to use the `Portfolio` object for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

`V = portvar(Asset,Weight)` returns the portfolio variance as an R-by-1 vector (assuming `Weight` is a matrix of size R-by-N) with each row representing a variance calculation for each row of `Weight`.

Examples

Compute Variance of Portfolio Assets

This example shows how to use `portvar` to compute the variance of portfolio assets. When you don't specify weights, `portvar` assigns each security an equal weight when calculating the portfolio variance.

```
load FundMarketCash
Returns = tick2ret(TestData);
Fund = Returns(:,1);
portvar(Fund)
```

```
ans =
5.3465e-04
```

Input Arguments

Asset — Asset returns

matrix

Asset Returns, specified as a M-by-N matrix of M asset returns for N securities.

Data Types: double

Weight — Portfolio weights

matrix

Portfolio weights, specified as a R-by-N matrix of R portfolio weights for N securities. Each row of **Weight** constitutes a portfolio of securities in **Asset**.

Data Types: double

Output Arguments**V — Variance of portfolio assets**

numeric

Variance of portfolio assets, returned as a numeric value.

Version History

Introduced before R2006a

References

[1] Bodie, Kane, and Marcus. *Investments*. McGraw Hill, Chapter 7, 2013.

See Also

portrand | portror

Topics

“Portfolio Construction Examples” on page 3-5

“Portfolio Optimization Functions” on page 3-3

portvrisk

Portfolio value at risk (VaR)

Syntax

```
ValueAtRisk = portvrisk(PortReturn,PortRisk)
ValueAtRisk = portvrisk( ___,RiskThreshold,PortValue)
```

Description

`ValueAtRisk = portvrisk(PortReturn,PortRisk)` returns the maximum potential loss in the value of a portfolio over one period of time (that is, monthly, quarterly, yearly, and so on) given the loss probability level. `portvrisk` calculates `ValueAtRisk` using a normal distribution.

Note An alternative for portfolio optimization is to use the `Portfolio` object for mean-variance portfolio optimization. This object supports gross or net portfolio returns as the return proxy, the variance of portfolio returns as the risk proxy, and a portfolio set that is any combination of the specified constraints to form a portfolio set. For information on the workflow when using `Portfolio` objects, see “Portfolio Object Workflow” on page 4-18.

`ValueAtRisk = portvrisk( ___,RiskThreshold,PortValue)` adds optional arguments for `RiskThreshold` and `PortValue`.

Examples

Compute the Maximum Potential Loss in the Value of a Portfolio Over One Period of Time

This example shows how to return the maximum potential loss in the value of a portfolio over one period of time, where `ValueAtRisk` is computed on a per-unit basis.

```
PortReturn = 0.29/100;
PortRisk = 3.08/100;
RiskThreshold = [0.01;0.05;0.10];
PortValue = 1;
ValueAtRisk = portvrisk(PortReturn,PortRisk,...
    RiskThreshold,PortValue)
```

```
ValueAtRisk = 3×1
```

```
0.0688
0.0478
0.0366
```

Compute the Maximum Potential Loss in the Value of a Portfolio Over One Period of Time Using Actual Values

This example shows how to return the maximum potential loss in the value of a portfolio over one period of time, where `ValueAtRisk` is computed with actual values.

```
PortReturn = [0.29/100;0.30/100];
PortRisk = [3.08/100;3.15/100];
RiskThreshold = 0.10;
PortValue = [1000000000;500000000];
ValueAtRisk = portvrisk(PortReturn,PortRisk,...
RiskThreshold,PortValue)
```

```
ValueAtRisk = 2×1
107 ×
```

```
3.6572
1.8684
```

Compute the Maximum Potential Loss in the Value of a Portfolio Over One Period of Time Using Dollar Units for PortReturn and PortRisk

This example shows how to return the maximum potential loss in the value of a portfolio over one period of time, where the portfolio return (`PortReturn`) and risk (`PortRisk`) are specified in dollar units. The returned `ValueAtRisk` is also in dollar units. Note that the `PortValue` input for `portvrisk` is not used in this example. `PortValue` is only used as a scaling factor to convert from percent to dollars when `PortReturn` and `PortRisk` are specified on a percentage basis.

```
PortReturn = [29000000;1500000];
PortRisk = [30800000;15750000];
RiskThreshold = 0.10;
ValueAtRisk = portvrisk(PortReturn,PortRisk,RiskThreshold)
```

```
ValueAtRisk = 2×1
107 ×
```

```
3.6572
1.8684
```

Input Arguments

PortReturn — Expected return of each portfolio over period

scalar numeric | vector

Expected return of each portfolio over the period, specified as a scalar numeric or an NPORTS-by-1 vector.

Data Types: double

PortRisk — Standard deviation of each portfolio over period

scalar numeric | vector

Standard deviation of each portfolio over period, specified as a scalar numeric or NPORTS-by-1 vector.

Data Types: double

RiskThreshold — Loss probability

0.05 (5%) (default) | scalar decimal | vector

(Optional) Loss probability, specified as a scalar decimal or an NPORTS-by-1 vector.

Data Types: double

PortValue — Total value of asset portfolio

1 (default) | scalar numeric | vector

(Optional) Total value of asset portfolio, specified as a scalar numeric or an NPORTS-by-1 vector.

Note The PortValue input is used as a scaling factor for ValueAtRisk. The ValueAtRisk output is computed using the PortReturn and PortRisk inputs first, and then scaled by PortValue. Therefore, if you specify PortValue in dollar units, then PortReturn and PortRisk must be given on a percentage basis. Conversely, when PortReturn and PortRisk are specified in dollar units, PortValue must be 1 (default value).

Data Types: double

Output Arguments

ValueAtRisk — Estimated maximum loss in portfolio

vector

Estimated maximum loss in the portfolio, returned as an NPORTS-by-1 vector. ValueAtRisk is predicted with a confidence probability of $1 - \text{RiskThreshold}$.

Note When PortValue is the default value of 1, ValueAtRisk is presented as a percent. A value of 0 for ValueAtRisk indicates no losses.

More About

Value at Risk

Value at Risk (VaR) is a statistical measure used to assess the potential loss in value of an investment or portfolio over a defined period for a given confidence interval.

Version History

Introduced before R2006a

See Also

portopt | Portfolio

Topics

"Portfolio Construction Examples" on page 3-5

"Portfolio Optimization Functions" on page 3-3

posvolidx

Positive volume index

Syntax

```
volume = posvolidx(Data)  
volume = posvolidx( ___,Name,Value)
```

Description

`volume = posvolidx(Data)` calculates the positive volume index from the series of closing stock prices and trade volume.

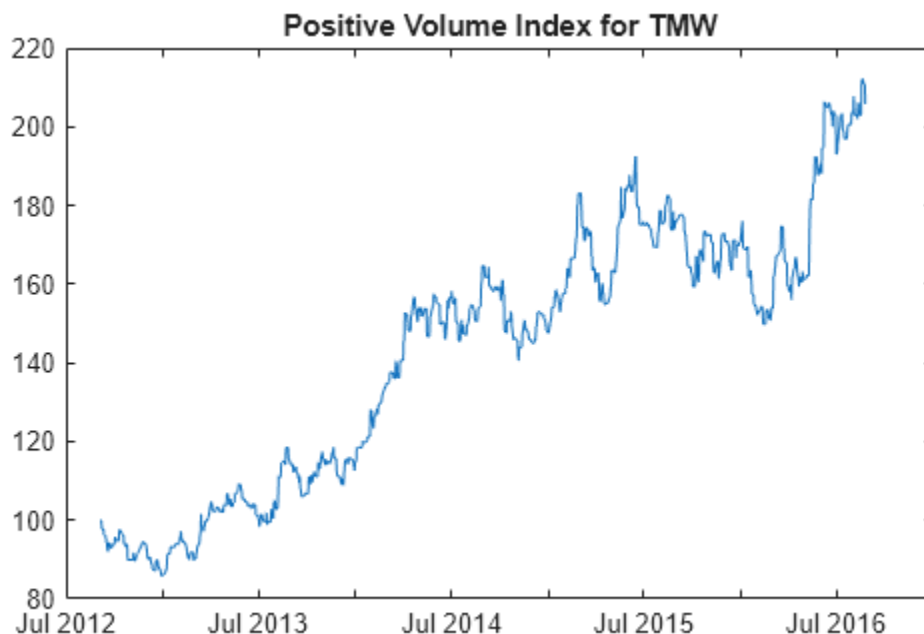
`volume = posvolidx( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Calculate the Positive Volume Index for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat  
volume = posvolidx(TMW);  
plot(volume.Time,volume.PositiveVolume)  
title('Positive Volume Index for TMW')
```



Input Arguments

Data — Data with closing prices and trade volume

matrix | table | timetable

Data with closing prices and trade volume, specified as a matrix, table, or timetable. For matrix input, Data is an M-by-2 with closing prices and trade volume stored in the first and second columns. Timetables and tables with M rows must contain variables named 'Close' and 'Volume' (case insensitive).

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as Name1=Value1,...,NameN=ValueN, where Name is the argument name and Value is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `volume = posvolidx(TMW,'InitialValue',500)`

InitialValue — Initial value for positive volume index

100 (default) | positive integer

Initial value for positive volume index, specified as the comma-separated pair consisting of 'InitialValue' and a scalar positive integer.

Data Types: double

Output Arguments

volume — Positive volume index

matrix | table | timetable

Positive volume index, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input Data.

More About

Positive Volume Index

Positive volume index is a technical analysis indicator that measures the strength of price movements based on trading volume.

The positive volume index chart is designed to provide insights into the strength of a trend by focusing on days when trading volume increases. The positive volume index is particularly useful for identifying bullish trends and confirming price movements, as it reflects the relationship between price changes and positive volume days.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the Data input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 236-238.

See Also

timetable | table | onbalvol | negvolidx

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

prbyzero

Price bonds in portfolio by set of zero curves

Syntax

```
BondPrices = prbyzero(Bonds,Settle,ZeroRates,ZeroDates)
BondPrices = prbyzero( ___,Compounding)
```

Description

`BondPrices = prbyzero(Bonds,Settle,ZeroRates,ZeroDates)` computes the bond prices in a portfolio using a set of zero curves.

`BondPrices = prbyzero( ___,Compounding)` adds an optional argument for Compounding.

Examples

Compute the Bond Prices in a Portfolio Using a Set of Zero Curves

This example uses the function `zbtprice` to compute a zero curve given a portfolio of coupon bonds and their prices. It then reverses the process, using the zero curve as input to the function `prbyzero` to compute the prices.

```
Bonds = [datenum('6/1/1998') 0.0475 100 2 0 0;
          datenum('7/1/2000') 0.06 100 2 0 0;
          datenum('7/1/2000') 0.09375 100 6 1 0;
          datenum('6/30/2001') 0.05125 100 1 3 1;
          datenum('4/15/2002') 0.07125 100 4 1 0;
          datenum('1/15/2000') 0.065 100 2 0 0;
          datenum('9/1/1999') 0.08 100 3 3 0;
          datenum('4/30/2001') 0.05875 100 2 0 0;
          datenum('11/15/1999') 0.07125 100 2 0 0;
          datenum('6/30/2000') 0.07 100 2 3 1;
          datenum('7/1/2001') 0.0525 100 2 3 0;
          datenum('4/30/2002') 0.07 100 2 0 0];
```

```
Prices = [ 99.375;
           99.875;
          105.75 ;
           96.875;
          103.625;
          101.125;
          103.125;
           99.375;
          101.0  ;
          101.25 ;
           96.375;
          102.75 ];
```

```
Settle = datenum('12/18/1997');
```

Set semiannual compounding for the zero curve, on an actual/365 basis.

```
OutputCompounding = 2;
```

Execute the function `zbtprice` which returns the zero curve at the maturity dates.

```
[ZeroRates, ZeroDates] = zbtprice(Bonds, Prices, Settle,...
OutputCompounding)
```

```
ZeroRates = 11×1
```

```
0.0616
0.0609
0.0658
0.0590
0.0647
0.0655
0.0606
0.0601
0.0642
0.0621
0.0627
⋮
```

```
ZeroDates = 11×1
```

```
729907
730364
730439
730500
730667
730668
730971
731032
731033
731321
731336
⋮
```

Execute the function `prbyzero`.

```
BondPrices = prbyzero(Bonds, Settle, ZeroRates, ZeroDates)
```

```
BondPrices = 12×1
```

```
99.3750
98.7980
106.8270
96.8750
103.6249
101.1250
103.1250
99.3637
101.0000
101.2500
96.3750
102.7384
```

:

In this example `zbtprice` and `prbyzero` do not exactly reverse each other. Many of the bonds have the end-of-month rule off (`EndMonthRule = 0`). The rule subtly affects the time factor computation. If you set the rule on (`EndMonthRule = 1`) everywhere in the `Bonds` matrix, then `prbyzero` returns the original prices, except when the two incompatible prices fall on the same maturity date.

Compute the Bond Prices in a Portfolio Using a Set of Zero Curves and `datetime` Inputs

This example uses the function `zbtprice` to compute a zero curve given a portfolio of coupon bonds and their prices. It then reverses the process, using the zero curve as input to the function `prbyzero` with `datetime` inputs to compute the prices.

```
Bonds = [datenum('6/1/1998') 0.0475 100 2 0 0;
          datenum('7/1/2000') 0.06 100 2 0 0;
          datenum('7/1/2000') 0.09375 100 6 1 0;
          datenum('6/30/2001') 0.05125 100 1 3 1;
          datenum('4/15/2002') 0.07125 100 4 1 0;
          datenum('1/15/2000') 0.065 100 2 0 0;
          datenum('9/1/1999') 0.08 100 3 3 0;
          datenum('4/30/2001') 0.05875 100 2 0 0;
          datenum('11/15/1999') 0.07125 100 2 0 0;
          datenum('6/30/2000') 0.07 100 2 3 1;
          datenum('7/1/2001') 0.0525 100 2 3 0;
          datenum('4/30/2002') 0.07 100 2 0 0];

Prices = [ 99.375;
          99.875;
          105.75 ;
          96.875;
          103.625;
          101.125;
          103.125;
          99.375;
          101.0 ;
          101.25 ;
          96.375;
          102.75 ];

Settle = datenum('12/18/1997');
OutputCompounding = 2;

[ZeroRates, ZeroDates] = zbtprice(Bonds, Prices, Settle, OutputCompounding);

dates = datetime(Bonds(:,1), 'ConvertFrom', 'datenum', 'Locale', 'en_US');
data = Bonds(:,2:end);
t=[table(dates) array2table(data)];
BondPrices = prbyzero(t, datetime(Settle, 'ConvertFrom', 'datenum', 'Locale', 'en_US'), ...
ZeroRates, datetime(ZeroDates, 'ConvertFrom', 'datenum', 'Locale', 'en_US'))

BondPrices = 12x1

    99.3750
    98.7980
```

```

106.8270
 96.8750
103.6249
101.1250
103.1250
 99.3637
101.0000
101.2500
 96.3750
102.7384
  :
```

Input Arguments

Bonds — Coupon bond information to compute prices

table | matrix

Coupon bond information to compute prices, specified as a 6-column table or a NumBonds-by-6 matrix of bond information where the table columns or matrix columns contains:

- **Maturity** (Required) Maturity date of the bond, specified as a datetime, string array, or date character vector.

To support existing code, prbyzero also accepts serial date numbers as inputs, but they are not recommended.

If the input Bonds is a table, the Maturity dates can be a datetime array, string array, or date character vectors.

- **CouponRate** (Required) Decimal number indicating the annual percentage rate used to determine the coupons payable on a bond.
- **Face** (Optional) Face or par value of the bond. Default = 100.
- **Period** (Optional) Coupons per year of the bond. Allowed values are 0, 1, 2 (default), 3, 4, 6, and 12.
- **Basis** (Optional) Day-count basis of the bond. A vector of integers.
 - 0 = actual/actual (default)
 - 1 = 30/360 (SIA)
 - 2 = actual/360
 - 3 = actual/365
 - 4 = 30/360 (BMA)
 - 5 = 30/360 (ISDA)
 - 6 = 30/360 (European)
 - 7 = actual/365 (Japanese)
 - 8 = actual/actual (ICMA)
 - 9 = actual/360 (ICMA)
 - 10 = actual/365 (ICMA)
 - 11 = 30/360E (ICMA)

- 12 = actual/365 (ISDA)
- 13 = BUS/252
- For more information, see “Basis” on page 2-16.
- **EndMonthRule** (Optional) End-of-month rule. This rule applies only when **Maturity** is an end-of-month date for a month having 30 or fewer days. 0 = ignore rule, meaning that a bond's coupon payment date is always the same numerical day of the month. 1 = set rule on (default), meaning that a bond's coupon payment date is always the last actual day of the month

:

Note

- If **Bonds** is a table, the table columns have the same meaning as when a matrix is used, but the **Maturity** dates can be a datetime array, string array, or date character vectors.
 - If **Bonds** is a matrix, it is a **NUMBONDS**-by-6 matrix of bonds where each row describes one bond. The first two columns are required; the remaining columns are optional but must be added in order. All rows in **Bonds** must have the same number of columns. The columns are **Maturity**, **CouponRate**, **Face**, **Period**, **Basis**, and **EndMonthRule**.
-

.

Data Types: double | char | string | datetime | table

Settle — Settlement date

datetime scalar | string scalar | date character vector

Settlement date, specified as a scalar datetime, string, or date character vector.

To support existing code, **prbyzero** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: datetime | string | char

ZeroRates — Observed zero rates

decimal fractions

Observed zero rates, specified as **NUMDATES**-by-**NUMCURVES** matrix of decimal fractions. Each column represents a rate curve. Each row represents an observation date.

Data Types: double

ZeroDates — Observed dates for ZeroRates

datetime array | string array | date character vector

Observed dates for **ZeroRates**, specified as a **NUMDATES**-by-1 column vector using a datetime array, string array, or date character vectors.

To support existing code, **prbyzero** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: datetime | string | char

Compounding — Compounding frequency of input ZeroRates when annualized

2 (default) | numeric values: 1, 2, 3, 4, 6, 12,

(Optional) Compounding frequency of input ZeroRates when annualized, specified using the allowed values:

- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Data Types: double

Output Arguments**BondPrices — Clean bond prices**

numeric

Clean bond prices, returned as a NUMBONDS-by-NUMCURVES matrix. Each column is derived from the corresponding zero curve in ZeroRates.

In addition, you can use the Financial Instruments Toolbox method `getZeroRates` for an `IRDataCurve` object with a `Dates` property to create a vector of dates and data acceptable for prbyzero. For more information, see “Converting an `IRDataCurve` or `IRFunctionCurve` Object” (Financial Instruments Toolbox).

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although prbyzero supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

tr2bonds | zbtprice | datetime

Topics

"Term Structure of Interest Rates" on page 2-29

"Fixed-Income Terminology" on page 2-15

prcroc

Price rate of change

Syntax

```
PriceChangeRate = prcroc(Data)  
PriceChangeRate = prcroc( ___,Name,Value)
```

Description

`PriceChangeRate = prcroc(Data)` calculates the price rate-of-change, `PriceChangeRate`, from the series of closing stock prices. By default, the price rate-of-change is calculated between the current closing price and the closing price 12 periods ago.

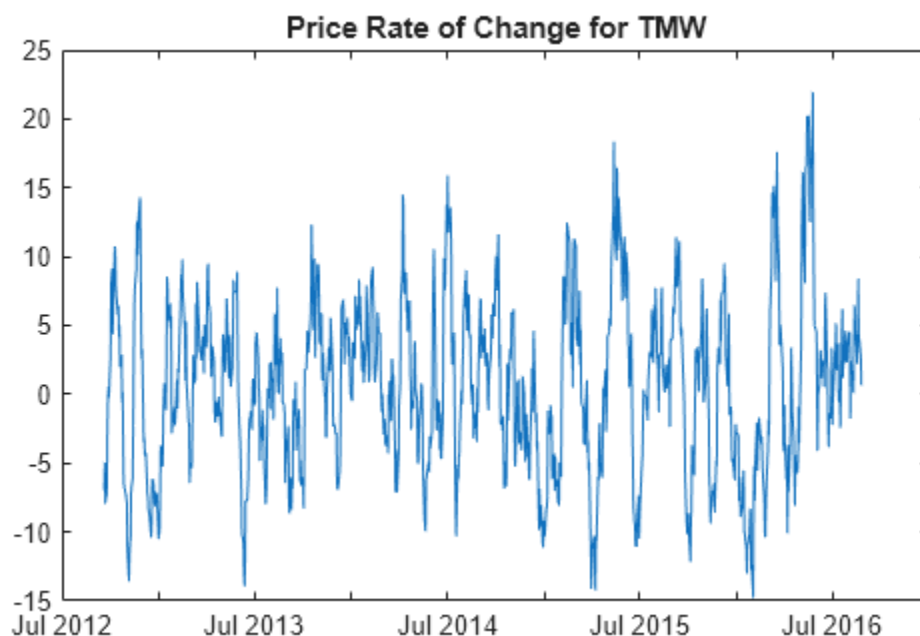
`PriceChangeRate = prcroc( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Calculate the Price Rate-of-Change for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat  
PriceChangeRate = prcroc(TMW);  
plot(PriceChangeRate.Time,PriceChangeRate.PriceRoc)  
title('Price Rate of Change for TMW')
```



Input Arguments

Data — Data for closing prices

matrix | table | timetable

Data for closing prices, specified as a matrix, table, or timetable. For matrix input, Data is an M-by-1 matrix of closing prices. Timetables and tables with M rows must contain a variable named 'Close' (case insensitive).

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as Name1=Value1,...,NameN=ValueN, where Name is the argument name and Value is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: PriceChangeRate = prcroc(TMW,'NumPeriods',18)

NumPeriods — Period difference

12 (default) | positive integer

Period difference, specified as the comma-separated pair consisting of 'NumPeriods' and a scalar positive integer.

Data Types: double

Output Arguments

PriceChangeRate — Closing price rate-of-change

matrix | table | timetable

Closing price rate-of-change, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input Data.

More About

Price Rate of Change

The price rate of change chart is a technical analysis tool used to measure the percentage change in the price of an asset over a specified period.

The price rate of change chart is designed to identify the momentum of price movements, helping traders and investors evaluate the strength of a trend and potential reversal points.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument*Behavior changed in R2023a*

fints object support for the Data input argument is removed.

R2022b: Support for negative price data*Behavior changed in R2022b*

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 243–245.

See Also

timetable | table | volroc

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

prdisc

Price of discounted security

Syntax

```
Price = prdisc(Settle,Maturity,FaceDiscount)
Price = prdisc( ____,Basis)
```

Description

`Price = prdisc(Settle,Maturity,FaceDiscount)` returns the price of a security whose yield is quoted as a bank discount rate (for example, U. S. Treasury bills).

`Price = prdisc( ____,Basis)` adds an optional argument for Basis.

Examples

Calculate the Price of a Security Whose Yield is Quoted as a Bank Discount Rate

This example shows how to return the price of a security whose yield is quoted as a bank discount rate (for example, U. S. Treasury bills).

```
Settle = '10/14/2000';
Maturity = '03/17/2001';
Face = 100;
Discount = 0.087;
Basis = 2;
```

```
Price = prdisc(Settle, Maturity, Face, Discount, Basis)
```

```
Price =
96.2783
```

Calculate the Price of a Security Whose Yield is Quoted as a Bank Discount Rate Using datetime Inputs

This example shows how to use `datetime` inputs to return the price of a security whose yield is quoted as a bank discount rate (for example, U. S. Treasury bills).

```
Settle = datetime(2000,10,14);
Maturity = datetime(2001,3,17);
Face = 100;
Discount = 0.087;
Basis = 2;
```

```
Price = prdisc(Settle,Maturity,Face, Discount, Basis)
```

```
Price =
96.2783
```

Input Arguments

Settle — Settlement date

datetime scalar | string scalar | date character vector

Settlement date, specified as a scalar datetime, string, or date character vector.

Settle must be earlier than Maturity.

To support existing code, prdisc also accepts serial date numbers as inputs, but they are not recommended.

Data Types: datetime | string | char

Maturity — Maturity date

datetime scalar | string scalar | date character vector

Maturity date, specified as a scalar datetime, string, or date character vector.

To support existing code, prdisc also accepts serial date numbers as inputs, but they are not recommended.

Data Types: datetime | string | char

Face — Redemption (par, face) value

numeric

Redemption (par, face) value, specified as a numeric value.

Data Types: double

Discount — Bank discount rate of the security

decimal fraction

Bank discount rate of the security, specified as a decimal fraction value.

Data Types: double

Basis — Day-count basis of instrument

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

(Optional) Day-count basis of the instrument, specified as a numeric value. Allowed values are:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)

- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

Output Arguments

Price — Price of discounted security

numeric

Price of discounted security, returned as a numeric value.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `prdisc` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

References

[1] Mayle. “*Standard Securities Calculation Methods.*” Volumes I-II, 3rd edition. Formula 2.

See Also

`acrudisc` | `bndprice` | `discrate` | `prmat` | `ylddisc` | `datetime`

Topics

“Pricing Functions” on page 2-21

“Fixed-Income Terminology” on page 2-15

priceandvol

Price and Volume chart

Note priceandvol is updated to accept data input as a matrix, timetable, or table.

The syntax for priceandvol has changed. Previously, when using table input, the first column of dates could be a datetime array, date character vectors, or serial date numbers, and you were required to have specific number of columns.

When using table input, the new syntax for priceandvol supports:

- No need for time information. If you want to pass in date information, use timetable input.
 - No requirement of specific number of columns. However, you must provide valid column names. `linebreak` must contain columns named 'open', 'high', 'low', 'close', and 'volume' (case insensitive).
-

Syntax

```
priceandvol(Data)
h = priceandvol(Data)
```

Description

priceandvol(Data) plots two charts from a series of opening, high, low, closing prices, and traded volume. Opening, high, low, and closing prices are on one axis and the volume series are on a second axis.

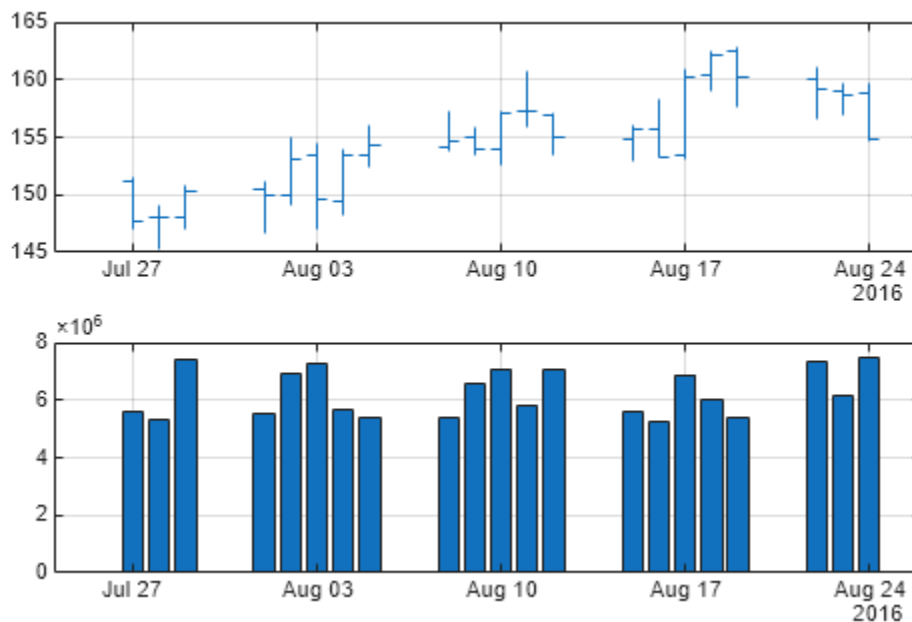
h = priceandvol(Data) adds a Graphic handle for the figure.

Examples

Generate a Price and Volume Chart for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock. This price and volume chart for the TMW stock contains the open, high, low, close, and volume for the most recent 21 days.

```
load SimulatedStock.mat
priceandvol(TMW(end-20:end,:));
```



Input Arguments

Data — Data for opening, high, low, closing prices, and volume traded

matrix | table | timetable

Data for opening, high, low, closing prices, and volume traded, specified as a matrix, table, or timetable. For matrix input, Data is an M-by-5 matrix of opening, high, low, closing prices and traded volume. Timetables and tables with M rows must contain variables named 'Open', 'High', 'Low', 'Close', and 'Volume' (case insensitive).

Data Types: double | table | timetable

Output Arguments

h — Graphic handle of the figure

handle object

Graphic handle of the figure, returned as a handle object.

More About

Price and Volume Chart

The price and volume chart is used in technical analysis to display the price movements of a financial asset alongside its trading volume over a specific period.

This type of chart provides valuable insights into market dynamics, helping traders and investors understand the relationship between price changes and trading activity.

Version History

Introduced in R2008a

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

R2018a: Support for matrix, time, or timetable input for data

Behavior changed in R2018a

priceandvol accepts Data input as a matrix, timetable, or table.

The syntax for priceandvol has changed. Previously, when using table input, the first column of dates could be a datetime array, date character vectors, or serial date numbers, and you were required to have specific number of columns.

When using table input, the new syntax for priceandvol supports:

- No need for time information. If you want to pass in date information, use timetable input.
- No requirement of specific number of columns. However, you must provide valid column names. linebreak must contain columns named 'open', 'high', 'low', 'close', and 'volume' (case insensitive).

See Also

timetable | table | movavg | linebreak | highlow | kagi | volarea | candle | pointfig

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

prmat

Price with interest at maturity

Syntax

```
[Price,AccruInterest] = prmat(Settle,Maturity,Issue,Face,CouponRateYield)
[Price,AccruInterest] = prmat( ____,Basis)
```

Description

[Price,AccruInterest] = prmat(Settle,Maturity,Issue,Face,CouponRateYield) returns the price and accrued interest of a security that pays interest at maturity. This function also applies to zero coupon bonds or pure discount securities by setting CouponRate = 0.

[Price,AccruInterest] = prmat(____,Basis) adds an optional argument for Basis.

Examples

Compute Price and Accrued Interest of a Security That Pays Interest at Maturity

This example shows how to compute the price and accrued interest of a security that pays interest at maturity.

```
Settle = '02/07/2002';
Maturity = '04/13/2002';
Issue = '10/11/2001';
Face = 100;
CouponRate = 0.0608;
Yield = 0.0608;
Basis = 1;
```

```
[Price, AccruInterest] = prmat(Settle, Maturity, Issue, Face,...
CouponRate, Yield, Basis)
```

```
Price =
99.9784
```

```
AccruInterest =
1.9591
```

Compute Price and Accrued Interest of a Security That Pays Interest at Maturity Using datetime Inputs

This example shows how to use datetime inputs compute the price and accrued interest of a security that pays interest at maturity.

```
Settle = datetime(2002,2,7);
Maturity = datetime(2002,4,13);
```

```

Issue = datetime(2001,10,11);
Face = 100;
CouponRate = 0.0608;
Yield = 0.0608;
Basis = 1;

[Price, AccruInterest] = prmat(Settle, Maturity, Issue, Face, CouponRate, Yield, Basis)

Price =
99.9784

AccruInterest =
1.9591

```

Input Arguments

Settle — Settlement date of security

datetime scalar | string scalar | date character vector

Settlement date of the security, specified as a scalar datetime, string, or date character vector. The **Settle** date must be before the **Maturity** date.

To support existing code, **prmat** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date of security

datetime scalar | string scalar | date character vector

Maturity date of the security, specified as a scalar datetime, string, or date character vector.

To support existing code, **prmat** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | datetime | string

Issue — Issue date

datetime scalar | string scalar | date character vector

Issue date of the security, specified as a scalar datetime, string, or date character vector.

To support existing code, **prmat** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Face — Redemption value

numeric

Redemption value (par value), specified as a numeric value.

Data Types: double

CouponRate — Coupon rate

decimal fraction

Coupon rate, specified as a decimal fraction value.

Data Types: `double`

Yield — Annual yield

decimal fraction

Annual yield, specified as a decimal fraction value.

Data Types: `double`

Basis — Day-count basis

0 (actual/actual) (default) | integers of the set [0 . . . 13] | vector of integers of the set [0 . . . 13]

(Optional) Day-count basis of the security, specified using the following values:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

Output Arguments

Price — Security price

numeric

Security price, returned as a numeric value.

AccruInterest — Accrued interest

numeric

Accrued interest for security, returned as a numeric value.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `prmat` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

References

[1] Mayle, J. *Standard Securities Calculation Methods*. Volumes I-II, 3rd edition. Formula 3.

See Also

`acrudisc` | `bndprice` | `bndyield` | `ylddisc` | `yldtbill` | `datetime`

Topics

"Yield Functions" on page 2-21

"Yield Conventions" on page 2-21

prtbill

Price of Treasury bill

Syntax

```
Price = prtbill(Settle,Maturity,Face,Discount)
```

Description

`Price = prtbill(Settle,Maturity,Face,Discount)` returns the price for a Treasury bill.

Examples

Calculate the Price for a Treasury Bill

This example shows how to return the price for a Treasury bill, where the settlement date of a Treasury bill is February 10, 2002, the maturity date is August 6, 2002, the discount rate is 3.77%, and the par value is \$1000.

```
Price = prtbill('2/10/2002', '8/6/2002', 1000, 0.0377)
```

```
Price =  
981.4642
```

Calculate the Price for a Treasury Bill Using datetime Inputs

This example shows how to use `datetime` inputs to return the price for a Treasury bill, where the settlement date of a Treasury bill is February 10, 2002, the maturity date is August 6, 2002, the discount rate is 3.77%, and the par value is \$1000.

```
Price = prtbill(datetime(2002,2,10), datetime(2002,8,6), 1000, 0.0377)
```

```
Price =  
981.4642
```

Input Arguments

Settle — Settlement date for Treasury bill

`datetime` scalar | `string` scalar | `date` character vector

Settlement date for the Treasury bill, specified as a scalar `datetime`, `string`, or `date` character vector. The `Settle` date must be before the `Maturity` date.

To support existing code, `prtbill` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date for Treasury bill

datetime scalar | string scalar | date character vector

Maturity date for the Treasury bill, specified as a scalar datetime, string, or date character vector.

To support existing code, prtbill also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Face — Redemption value of Treasury bill

numeric

Redemption value (par value) of the Treasury bill, specified as a numeric value.

Data Types: double

Discount — Discount rate of Treasury bill

decimal fraction

Discount rate of the Treasury bill, specified as a decimal fraction value.

Data Types: double

Output Arguments**Price — Treasury bill price**

numeric

Treasury bill price, returned as a numeric value.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although prtbill supports serial date numbers, datetime values are recommended instead. The datetime data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to datetime values, use the datetime function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

References

[1] Bodie, Kane, and Marcus. *Investments*. McGraw-Hill Education, 2013.

See Also

beytbill | yldtbill | datetime

Topics

“Computing Treasury Bill Price and Yield” on page 2-26

“Treasury Bills Defined” on page 2-25

pvfix

Present value with fixed periodic payments

Syntax

```
PresentVal = pvfix(Rate,NumPeriods,Payment)  
PresentVal = pvfix( ___,ExtraPayment,Due)
```

Description

`PresentVal = pvfix(Rate,NumPeriods,Payment)` computes the present value of a series of equal payments.

`PresentVal = pvfix( ___,ExtraPayment,Due)` adds optional arguments.

Examples

Calculate the Present Value of a Series of Equal Payments

This example shows how to return the present value of a series of equal payments, where \$200 is paid monthly into a savings account earning 6%. The payments are made at the end of the month for five years.

```
PresentVal = pvfix(0.06/12, 5*12, 200, 0, 0)
```

```
PresentVal =  
1.0345e+04
```

Input Arguments

Rate — Periodic interest rate

decimal

Periodic interest rate, specified as a decimal.

Data Types: double

NumPeriods — Number of periods

integer

Number of periods, specified as an integer.

Data Types: double

Payment — Periodic payment

numeric

Periodic payment., specified as a numeric.

Data Types: double

ExtraPayment — Payment received other than Payment in the last period

0 (default) | numeric

(Optional) Payment received other than Payment in the last period, specified as a numeric.

Data Types: double

Due — Indicator for when payments are due

0 (default) | logical with value of 1 or 0

(Optional) Indicator for when payments are due, specified as a logical with a value of 0 = end of period (default), or 1 = beginning of period.

Data Types: logical

Output Arguments

PresentVal — Present value

numeric

Present value, returned as a series of equal payments.

More About

Present Value

Present value is a fundamental financial concept that refers to the current worth of a sum of money or a stream of cash flows that will be received or paid in the future, discounted at a specific interest rate.

The idea behind present value is that a certain amount of money today is worth more than the same amount in the future due to its potential earning capacity. This concept is crucial in finance for valuing investments, making financial decisions, and assessing the worth of future cash flows.

Version History

Introduced before R2006a

See Also

fvfix | fvvar | payper | pvvar

Topics

“Analyzing and Computing Cash Flows” on page 2-11